

openMIS Developer Academy

A hands-on architectural handbook and course for engineers who want to understand — and extend — **openMIS**, the open-source platform for health insurance and social health protection. Written for people who already ship Django, but have never touched GraphQL or an enterprise-scale plugin architecture.

Welcome. This site is a **course disguised as reference documentation**. You can read it front-to-back like a book, or jump to the chapter you need and treat it as a manual. Either way, the goal is the same: to take a competent Django engineer and make them productive — and confident — inside the openMIS codebase.

” The one-sentence version

openMIS is a modular **Django + React** platform that many countries deploy to run their health-financing schemes; its whole design revolves around **plugins (modules)** so that no country ever has to fork the core.

What this handbook is

openMIS is not a small project. The backend alone is assembled from roughly **47 independently versioned modules**, each living in its own Git repository, wired together at startup into a single Django project and a single GraphQL schema. That design is powerful, but it is also unusual — you cannot learn it by reading one `models.py`. You have to understand the

seams: how modules discover each other, how the schema is stitched together, how a mutation actually runs, and why the database still carries the scars of a 15-year-old .NET application.

This handbook explains those seams from first principles. It assumes you know Python, Django, the ORM, Django REST Framework, Docker, and PostgreSQL cold. It assumes you know **nothing** about GraphQL or "enterprise architecture" as a discipline — so we teach both, carefully, where they appear.

 Did you know?

openIMIS is a recognized **Digital Public Good (DPG)**. It is developed as a global commons, historically funded by the Swiss Agency for Development and Cooperation (SDC) and GIZ, and governed by the openIMIS Initiative together with a community Technical Advisory Group. The code you will read is the same code running national schemes in countries across Africa and Asia.

Who this is for

You are

A backend engineer fluent in Django and the ORM who has just been dropped into the openIMIS codebase and needs a mental model fast.

You are

A full-stack developer who knows React and Redux but wants to understand how the frontend talks to a GraphQL backend it doesn't own end-to-end.

You are

A technical lead evaluating openIMIS for a national deployment, who needs to judge the architecture, not just the feature list.

You are

A contributor from the openIMIS community who wants a single, coherent narrative to onboard new teammates.

If you have **never written GraphQL**, that is expected and fine. If you have never seen a "module manifest" or an "assembly repo", that is exactly what we are here to explain.

How the course is structured

The material is layered. Each layer assumes the ones before it, but every chapter also restates its prerequisites so you can enter sideways.

Stage	You will learn	Chapters
Orient	What openIMIS is, the domain vocabulary, and how to stand up a local environment	Getting Started
Understand	The plugin architecture, the request lifecycle, the repository map	Architecture
Speak the API	GraphQL from first principles, as openIMIS uses it	GraphQL
Know the domain	The business modules — insuree, policy, claim, product, payment	Modules
Go deep	Database, security, Docker, configuration, integrations	Database, Security, Docker
Build	Extend openIMIS with your own module, end to end	Extending openIMIS
Practice	A graded, seven-level guided track	Developer Academy

Course map

1. Getting Started

What openIMIS is, the domain terminology every developer must know, and a working local dev environment in under an hour.

2. Architecture

The plugin system, the assembly repos, dynamic `INSTALLED_APPS`, schema stitching, and the full request lifecycle.

3. GraphQL

GraphQL taught from zero: queries, mutations, connections, and the asynchronous `OpenIMISMutation` pattern.

4. The Core Module

`HistoryModel`, the custom `User`, rights-based permissions, service signals, and the calculation-rule framework.

5. Business Modules

Insuree, policy, product, claim, contribution, payment — the health-financing domain, module by module.

6. Frontend

The React SPA, Redux (not Apollo), the `ModulesManager`, contributions,

and the Vite migration.

7. Integrations

FHIR R4, DHIS2 ETL, payment gateways, and OpenSearch analytics.

8. Extending openIMIS

Write, register, and ship your own module — backend and frontend — following an end-to-end walkthrough.

9. Developer Academy

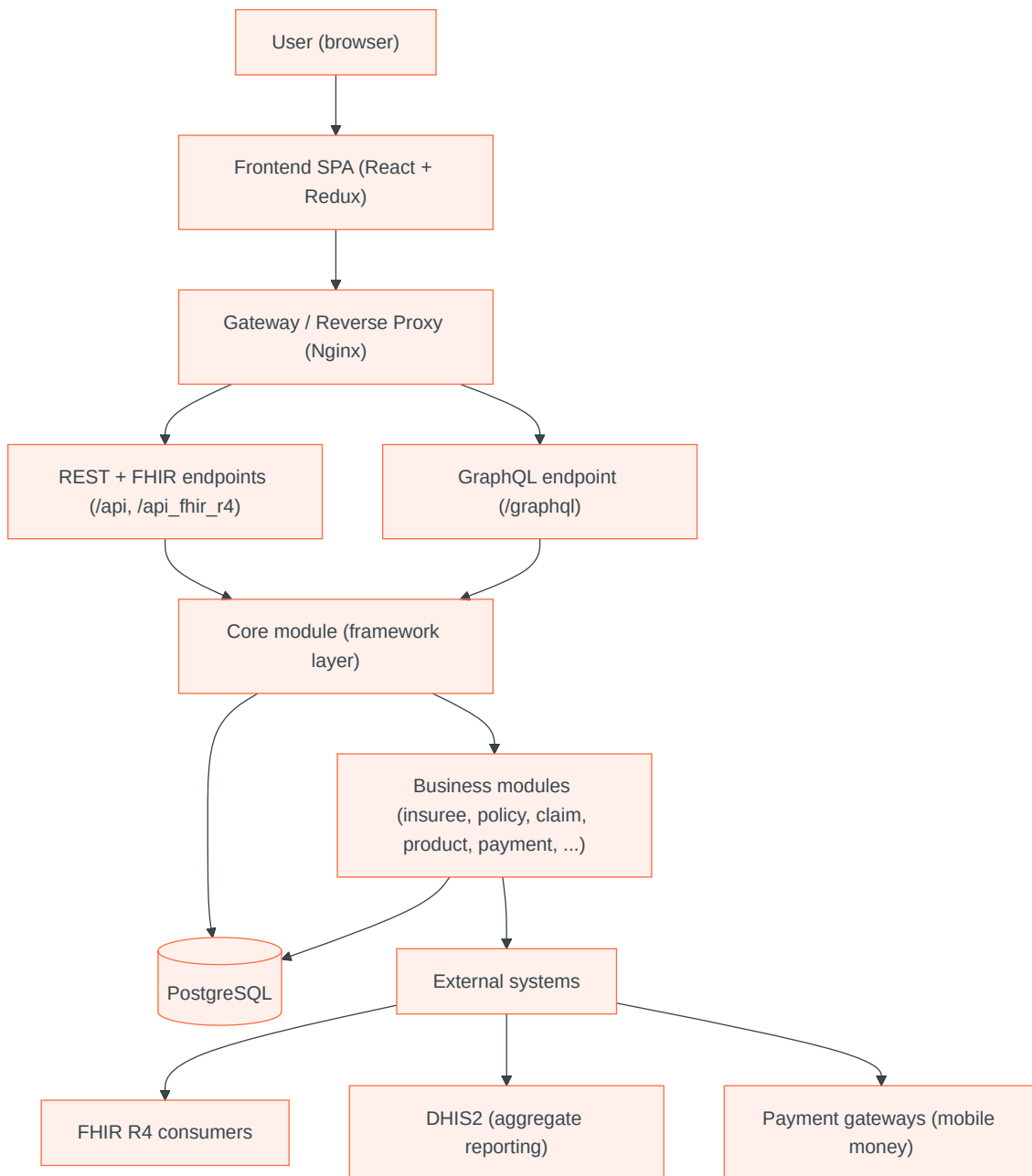
A seven-level guided track, from Foundations to Expert, with exercises and knowledge checks.

10. Reference

The full glossary, troubleshooting guide, and a repository appendix you will return to often.

The big picture

Before any detail, hold this shape in your head. Everything else in the handbook is a zoom-in on one of these layers.



Read that diagram top to bottom and you have the itinerary for this course. The browser talks to a **React SPA**. The SPA talks through a **gateway** to a **GraphQL** endpoint (and, for reports and interoperability, some **REST/FHIR** endpoints). Those endpoints are served by the **Core module**, which provides the framework that every **business module** builds on. The business modules read and write **PostgreSQL** and, at the edges, push and pull data to **external systems** like FHIR consumers, DHIS2, and payment providers.



A note on the two 'assemblies'

openIMIS has two top-level *assembly* repositories that glue everything together: `openimis-be_py` on the backend and `openimis-fe_js` on the frontend. Neither contains much business logic itself — each is a manifest plus wiring that assembles dozens of module repositories into one running application. Understanding these two repos is the single highest-leverage thing you can do early, and [Architecture](#) starts there.

How to use this site



Reading paths

Start at [Getting Started](#) and follow the navigation top to bottom. Do the **Hands-on lab** and **Knowledge check** at the end of each chapter. Then work through the graded [Developer Academy](#) levels.

Jump straight to the chapter you need — every substantial page lists its **Prerequisites** at the top with links, so you can back-fill just enough context. The [Reference glossary](#) and [Repository appendix](#) are your quick-lookup companions.

Point them at this home page, then at the [Architecture overview](#) and the [End-to-End Code Walkthrough](#). Those three pages give the fastest path to a correct mental model.

Read it as a course

Use it as a manual

Onboard a teammate

A few conventions used throughout:

- **Relative links** connect chapters — e.g. the [Core module](#) or the [Request Lifecycle](#). Follow them freely; you will not get lost.
- Code labeled *illustrative* or *simplified* is written to faithfully reflect openIMIS conventions, not copied line-for-line. When exact detail matters, we cite the real file — for example (see `openimis-be-core_py/core/apps.py`) — and tell you to read the source.
- Collapsible **Deep dive** blocks hold advanced tangents. Skip them on a first pass.
- We **never invent line numbers**. Files and classes, yes; line numbers, never — they drift with every release.



Common mistake

Do not try to understand openIMIS by opening a single repository and reading it like a normal Django project. There is no single repository. The application only exists once the assembly repo has read its **module manifest** and stitched the pieces together. If a class seems to come from nowhere, it almost certainly lives in another module's repo — the [Repository Map](#) will help you find it.

A word on accuracy and scope



This is a community handbook

This site is an independent, community-oriented educational resource. It is **not** official openIMIS documentation and is not endorsed by the openIMIS Initiative. openIMIS is a fast-moving, modular platform: module counts, exact file layouts, and configuration keys change between releases. Where a detail is version-dependent, we say so and point you to the authoritative repository under github.com/openimis to confirm. Trademarks and code belong to their respective owners. Always treat the source code and the [official openIMIS wiki](#) as the final authority.

Ready? Begin with [Getting Started](#) — it orients you to the platform, teaches the vocabulary, and gets a local environment running.

Further reading

- The openIMIS website and community hub: openimis.org
- All source repositories: github.com/openimis
- Official product and functional wiki: [openIMIS on Atlassian](#)
- Digital Public Goods Alliance registry: digitalpublicgoods.net
- GraphQL, from the source: graphql.org/learn

Getting Started

This is the on-ramp. Three short chapters take you from "I have heard of openIMIS" to "I have a working local environment and I know what the words mean." Do them in order the first time; return to any of them as reference later.



You are here

You have read the [course home page](#) and you have the big-picture layer diagram in your head. Good. Now we fill in the *why*, the *vocabulary*, and the *how do I run this*.

What this section covers

Platform Overview

What openIMIS is, the real-world problem it solves, the health-financing ecosystem it models, and how it evolved from a monolithic .NET/MSSQL system into a modular Django/React platform. Start here.

Terminology Primer

The domain and platform vocabulary a new developer must know: insuree vs. individual, policy vs. product, contribution vs. payment, rights, `MutationLog`, `json_ext`, and more.

Set Up a Dev Environment

Clone the assembly repos, understand the `openimis.json` manifest, boot the stack with Docker, and run the backend and frontend locally with editable module installs.

How much time to budget

Chapter	Reading	Hands-on
Platform Overview	~20 min	—
Terminology Primer	~15 min	keep it open as a reference
Set Up a Dev Environment	~15 min	30–60 min the first time

What comes next

Once you can load the app in a browser and you recognize the vocabulary, move on to the [Architecture](#) section, where we open up the plugin system and explain how ~47 independent repositories become one running application. If GraphQL is new to you, the [GraphQL](#) chapter teaches it from zero before you need it in anger.



A reminder on scope

This is a community handbook, not official documentation. Where details are version-dependent, we point you to the authoritative repositories under github.com/openimis. See the full disclaimer on the [home page](#).

Platform Overview

Part 1 of the course. Before any code, you need to know what openIMIS *is for*. This chapter builds the domain and architectural intuition that every later chapter assumes. It is long on purpose — read it once, carefully, and the rest of the handbook will click into place.

Learning objectives

By the end of this chapter you will be able to:

- Explain what openIMIS is and the real-world problem it exists to solve.
- Describe the **health-financing / social health protection** ecosystem it models — members, policies, contributions, providers, and claims — and how those pieces fit together.
- Explain what a **Digital Public Good** is and why openIMIS is governed as one.
- Articulate the **modular ("plugin") architecture philosophy** and, crucially, *why* it exists.
- Trace the platform's evolution from the legacy **.NET + Microsoft SQL Server IMIS** to the modular **Django + React openIMIS**, and recognize the legacy heritage still visible in the code.

Prerequisites

- The [course home page](#) and its high-level layer diagram.
- Working knowledge of Django and PostgreSQL (assumed throughout this handbook).
- No prior knowledge of health insurance, GraphQL, or enterprise architecture is required — we build all three from first principles.

What is openIMIS?

openIMIS stands for **open Insurance Management Information System**. It is an open-source software platform for administering **health insurance and social health protection schemes**: registering the people who are covered, recording the policies that cover them, collecting the contributions (premiums) that fund those policies, and processing the claims that health facilities submit when they deliver care.

Put plainly: if a government or an NGO in a low- or middle-income country runs a health-financing scheme — a national health insurance fund, a community-based mutual, a voucher programme — openIMIS is a ready-made system to run the *information* side of it. Who is covered? Until when? Have they paid? Is this clinic allowed to treat them? How much do we owe the clinic for last month's visits?

Did you know?

openIMIS is deployed in production in a number of countries across Africa and Asia — including Tanzania, Cameroon, Nepal, Chad, and the Democratic Republic of the Congo, among others. The same modular codebase you will study here is configured differently in each country to match its scheme rules, languages, and organizational structure. That "configure, don't fork" property is not an accident — it is the entire point of the architecture, as you will see below.

Why was openIMIS created?

Health financing in low- and middle-income countries (LMICs) has a chronic tooling problem. Schemes are often run on spreadsheets, paper registers, or bespoke software that a single vendor built and now holds hostage. Each new scheme reinvents the same wheel — enrolment, eligibility, claims — at great cost, with no interoperability and no shared learning.

openIMIS exists to break that cycle. It is a **shared, free, open-source "core banking system" for health protection schemes**: a common platform that many countries can adopt, adapt, and improve together, rather than each paying to rebuild the basics. Its development has been supported over the years by the **Swiss Agency for Development and Cooperation (SDC)** and **GIZ**, and it is steered by the **openIMIS Initiative** and a community **Technical Advisory Group**.

The problems it solves

Concretely, openIMIS addresses the operational headaches of running a health-financing scheme:

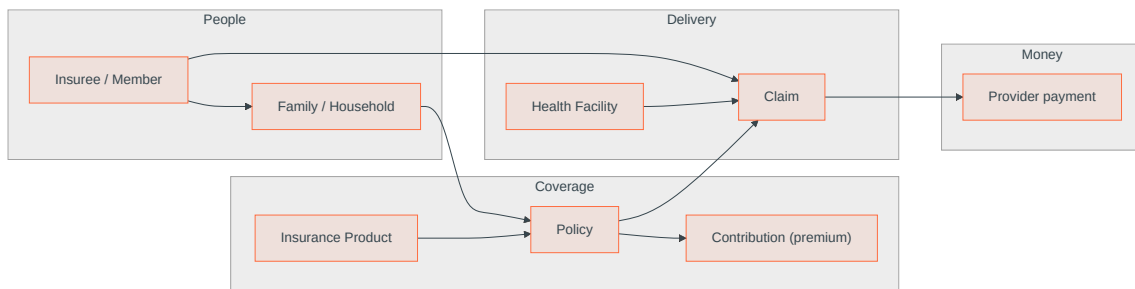
Problem	How openIMIS helps
Beneficiary registry — knowing who is enrolled and their family structure	Insuree / individual registries with household and group modelling
Coverage — which product covers whom, and when it starts and ends	Policies link insurees to insurance products with validity periods
Financing — collecting and tracking premiums	Contribution records against policies; payment reconciliation
Provider network — which facilities can serve which members	A location hierarchy plus a registry of health facilities
Claims — paying providers for care actually delivered	Claim capture, adjudication, valuation via pricing rules, and payment
Reporting & interoperability — telling the story to ministries and donors	FHIR R4 endpoints, DHIS2 aggregate exports, OpenSearch analytics
Local adaptation — every country's rules are different	A modular architecture and a runtime configuration model

” The mental model that helps most

Think of openIMIS as **insurance-scheme middleware**. It sits between *the people who are covered*, *the facilities that treat them*, and *the fund that pays for it all* — keeping the shared truth about coverage, eligibility, and money owed.

The health-financing ecosystem openIMIS models

You cannot understand the code until you understand the domain it encodes. Here is the ecosystem in the smallest number of moving parts. (The [Terminology Primer](#) defines each term precisely; this section is the story that connects them.)



Walk the ecosystem in the order a real scheme experiences it:

1. **Members and families.** People enrol, usually as a **family or household**, not as isolated individuals. One member is often the head; dependents are covered under the same policy. openIMIS models the person as an **insuree** (and, in newer social-protection work, as a more general **individual**).
2. **Products.** A scheme offers one or more **insurance products** — the "plan" definitions. A product says what is covered, the premium, the waiting periods, ceilings, and which medical services and items are included, often via a **price list**.
3. **Policies.** A **policy** is the contract that links a family to a product for a period of time. It has a start date, an expiry date, and a status. The policy is what makes a person *actually covered* right now.
4. **Contributions.** To keep a policy active, someone pays **contributions** (premiums). openIMIS records these against the policy. No contributions, no valid coverage.
5. **Providers and the location hierarchy.** Care is delivered by **health facilities** (clinics, hospitals), which sit inside a **location hierarchy** — typically region → district → municipality → village. The hierarchy governs both administration and which facilities serve which members.
6. **Claims.** When a covered member receives care, the facility submits a **claim** listing the services and items provided. openIMIS checks eligibility, adjudicates the claim against the product's rules, and **values** it — often using pluggable **calculation rules** (for example, capitation or fee-for-service).
7. **Payment.** Approved, valued claims translate into money the scheme owes providers, handled by the **payment** and **invoice** modules and, at the edges, external payment gateways.



One sentence that uses every term

A member in a family holds a policy on a health-insurance product, kept active by contributions; when they visit a health facility in their location, the facility files a claim, which openIMIS values and turns into a provider payment.

Every business module you will study — [insuree](#), [policy](#), [product](#), [claim](#), [contribution & payment](#), [calculation](#) — implements one node of this ecosystem.

openIMIS as a Digital Public Good

A **Digital Public Good (DPG)** is open-source software (or open data, content, or standards) that helps achieve the UN Sustainable Development Goals, is built to be freely adopted and adapted, and meets a published set of criteria around openness, privacy, and "do no harm." openIMIS is a recognized DPG.

Why does this matter to you as an engineer? Because it shapes the codebase and the way you are expected to work in it:

- **Openness is a hard requirement, not a nicety.** The whole platform — every module — is open source under permissive terms. There is no proprietary "enterprise edition" hiding the interesting parts.
- **Reusability is a design goal.** The architecture is deliberately built so that one deployment's improvements can flow back to everyone. This is why country-specific behavior lives in configuration and add-on modules, not in forks.
- **Global commons governance.** Decisions are made in the open by the openIMIS Initiative and its Technical Advisory Group. Contributions follow a public process (see the [Developer Workflow](#) chapter).

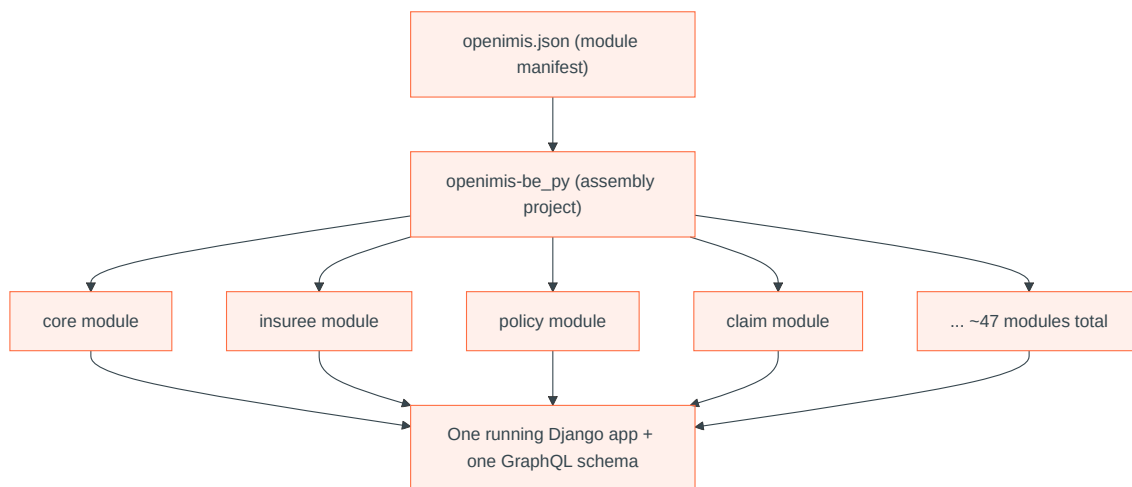
Did you know?

The DPG philosophy is the *social* explanation for the *technical* modularity you are about to study. "Don't fork, extend" is not just good engineering advice here — it is how a global public good stays a single, shared thing instead of fragmenting into dozens of incompatible national codebases.

The modular architecture philosophy — and why it exists

This is the most important idea in the entire handbook, so we introduce it here and expand it in [Architecture](#).

openIMIS is not one big Django project with dozens of apps in one repository. It is a small **assembly project** (the repo `openimis-be_py`) plus roughly **47 independent modules**, each in its own repository, each versioned on its own, wired together at startup.



Why build it this way? Because every country's scheme is different, and the alternative — one monolith with country-specific `if` statements, or a fork per country — is a maintenance catastrophe. Modularity buys openIMIS four things:

1. **Country-specific customization without forking core.** A deployment picks which modules to include (via `openimis.json`) and configures each one at runtime. Nepal and Tanzania run the *same core* with different modules and different configuration.
2. **Independent evolution.** The `claim` module can release on its own schedule without a lock-step release of the entire platform.
3. **A clean extension seam.** New behavior arrives as a **new module** that hooks into existing ones through well-defined seams (service signals on the backend, "contributions" on the frontend) — without editing the modules it extends.
4. **Shared improvements flow upstream.** Because customization is additive, a feature built for one country can be packaged as a module and offered to all.

Common mistake

Newcomers open `openimis-be_py`, see almost no business logic, and conclude the project is empty or that they are in the wrong place. They are not. The assembly repo is *supposed* to be thin — it is a manifest plus wiring. The business logic lives in the module repos named `openimis-be-<name>_py`. Learn to read the [Repository Map](#) early and this confusion disappears.

We will not fully unpack *how* the assembly happens here — dynamic `INSTALLED_APPS`, GraphQL schema stitching by Python multiple inheritance, service signals — that is the job of the [Architecture](#) and [Plugin System](#) chapters. For now, hold the *shape* and the *why*.



Deep dive: how this compares to architectures you may know



If you have worked with Django's own third-party app ecosystem (`INSTALLED_APPS` full of pip packages), openIMIS will feel familiar — it is that idea, industrialized. Each module *is* a Django app, but with extra conventions: an `AppConfig` carrying a `DEFAULT_CFG`, a `schema.py` exposing `Query / Mutation` fragments, integer permission codes, and service-signal registration. If you have worked with microservices, note the contrast: openIMIS modules are **plugins in one process**, not separate services over a network. You get modular *code* and independent *release cadence*, but a single deployable app and a single database — which sidesteps the distributed-systems tax while keeping most of the organizational benefits. The [Architecture Critique](#) chapter weighs these trade-offs honestly.

Countries and deployment models

openIMIS is designed to be deployed and operated by in-country teams, not consumed as a hosted SaaS. Common deployment shapes:

- **National scheme, self-hosted.** A ministry or national health insurance fund runs openIMIS on its own infrastructure, configured for its products, locations, and languages.
- **Programme or pilot.** An NGO or donor programme runs a scoped deployment for a specific population or region, often as a pilot that later scales.
- **Docker-based.** The reference distribution (`openimis-dist_dkr`) is `docker-compose` - based, which is also how most developers run it locally — see [Set Up a Dev Environment](#).

Because deployments differ so much, **configuration** is a first-class concern: which modules are loaded, and how each is tuned at runtime. The [Configuration](#) chapter covers the runtime model in depth; the short version is that each module ships defaults (`DEFAULT_CFG`) which are overlaid by per-deployment JSON stored in the database, so operators reconfigure without touching code.

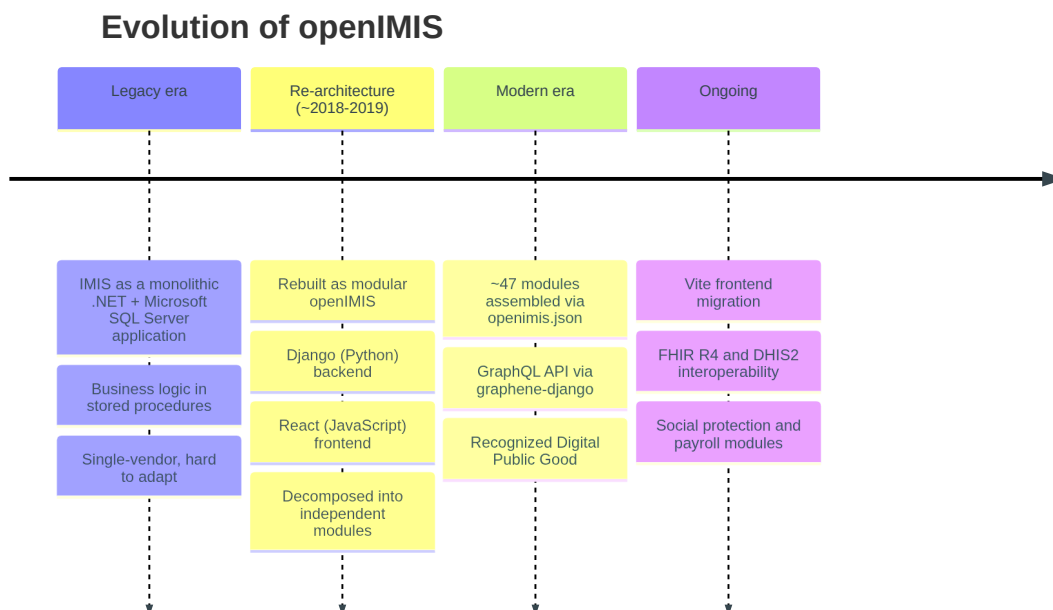


Localization is everywhere

Multi-language support (via `react-intl` on the frontend), configurable location hierarchies, and country-tunable product and calculation rules mean a single codebase adapts to very different national contexts. Keep this in mind whenever you are tempted to hard-code a label, a level of the location tree, or a business rule.

From legacy IMIS to modern openIMIS

To read the openIMIS database and API without confusion, you must know where it came from.



The legacy: IMIS. openIMIS began life as **IMIS**, a monolithic application built on **Microsoft .NET** with a **Microsoft SQL Server** database. Much of its business logic lived in **stored procedures**. It worked, but it was hard to adapt per country and tied to a single technology stack and vendor.

The re-architecture (~2018–2019). The platform was rebuilt as **modular openIMIS**: a **Django** backend and a **React** frontend, decomposed into the independently versioned modules described above. This modularization is the platform's central design idea — it is what lets many countries share one core.

The heritage you will still see. The rewrite kept the existing database shape so that legacy data could migrate, so the modern code carries visible fingerprints of its past:

Legacy fingerprint	What you will see	Why
<code>tbl</code> table prefixes	<code>tblInsuree</code> , <code>tblClaim</code> , <code>tblPolicy</code> , ...	Table names inherited from the IMIS schema
camelCase columns	<code>InsureeID</code> , <code>ClaimStatus</code> , ...	MSSQL naming preserved through migration
Dual keys	Legacy integer IDs and newer UUIDs	UUIDs added for the modern API without dropping legacy keys (<code>legacy_id</code>)
Temporal columns	<code>validity_from</code> / <code>validity_to</code> on many models	Historical versioning inherited and formalized in <code>HistoryModel</code>
MSSQL → PostgreSQL	PostgreSQL today, MSSQL heritage in names	The datastore was modernized; the schema names largely were not



Common mistake

Expecting idiomatic Django `snake_case` table and column names everywhere. Many core models map to legacy `tbl*` tables with camelCase columns via explicit `Meta.db_table` and `db_column`. When a migration or raw query looks "wrong," it is usually just the legacy schema showing through. The [Database](#) chapter maps this in detail.



Deep dive: why keep the legacy schema at all?



Rewrites that also redesign the database force a risky, all-at-once data migration and break every existing report and integration. By preserving the schema and layering a modern ORM, API, and UUID keys on top, the openIMIS team let real deployments migrate incrementally and kept legacy reporting alive during the transition. It is a pragmatic decision that trades some aesthetic cleanliness for a dramatically safer migration path — a recurring theme in enterprise systems, and a good lesson in why "just rewrite it cleanly" is rarely the whole story.

Putting it together

You now have the three foundations the rest of the course builds on:

1. **The domain** — the health-financing ecosystem of members, families, products, policies, contributions, facilities, claims, and payments.
2. **The philosophy** — an open, reusable Digital Public Good whose modular "configure, don't fork" architecture lets many countries share one core.
3. **The history** — a modular Django/React platform layered over a legacy .NET/MSSQL schema, whose fingerprints you will keep meeting in the code.

Next, lock in the vocabulary in the [Terminology Primer](#), then get the platform running locally in [Set Up a Dev Environment](#). After that, the [Architecture](#) section opens the hood on how the modules actually assemble into one application.

Hands-on lab

You do not need any code running for this lab — it is about building an accurate map before you touch the system.

Lab 1 — Map the ecosystem to the modules

Goal: connect the domain story to the real repositories.

1. Open github.com/openimis in a browser and skim the list of repositories. Notice the naming pattern `openimis-be-<name>_py` (backend) and `openimis-fe-<name>_js` (frontend).
2. For each node in the ecosystem diagram above — member, product, policy, contribution, health facility, claim, payment — find at least one repository whose name plausibly implements it. (Hint: `insuree`, `product/medical`, `policy`, `contribution`, `location`, `claim`, `payment/invoice`.)
3. Open the `openimis-be_py` repository and find the file `openimis.json`. Skim it. This is the module manifest — the list of every module the assembled backend includes. Count roughly how many modules are listed.
4. In your own words, write two or three sentences explaining to an imaginary teammate *why* the business logic is not in `openimis-be_py` itself.

Success looks like: you can name the repository behind each domain concept, and you can explain the assembly-repo-versus-module-repo distinction without looking it up.

Knowledge check

? Q1: In one sentence, what is openIMIS for? (click for answer) ✓

openIMIS is an open-source platform for administering health insurance and social health protection schemes — registering covered members, recording the policies and products that cover them, collecting contributions, and processing the claims that health facilities submit — primarily for low- and middle-income countries.

? Q2: A policy links which two things, and what keeps it active? (click for answer) ✓

A **policy** links a **family/insuree** to an insurance **product** for a period of time. It is kept active by **contributions** (premium payments) recorded against it.

? Q3: Why is openIMIS built as ~47 independent modules rather than one monolith? (click for answer) ✓

So that each country can customize its deployment by choosing and configuring modules **without forking the core**. Modularity enables country-specific customization, independent release cadence per module, a clean additive extension seam, and upstream sharing of improvements — which also aligns with openIMIS being a reusable Digital Public Good.

? Q4: Name two legacy fingerprints you will see in the openIMIS database and explain where they come from. (click for answer) ✓

Any two of: `tbl`-prefixed table names, camelCase column names, dual integer + UUID keys (with `legacy_id`), and `validity_from/validity_to` temporal columns. They come from the original **IMIS** application, a monolithic **.NET + Microsoft SQL Server** system whose schema was preserved when openIMIS was rebuilt on Django and PostgreSQL, to allow safe data migration.

? Q5: What does it mean that openIMIS is a Digital Public Good, and how does that connect to its architecture? (click for answer) ✓

A Digital Public Good is open-source software built to be freely adopted and adapted in service of the UN Sustainable Development Goals, meeting published openness and "do no harm" criteria. It connects to the architecture because the DPG goal of *reusability* is the social reason for the technical *modularity*: "configure and extend, don't fork" keeps the platform a single shared commons rather than fragmenting into incompatible national forks.

Further reading

- openimis.org — the initiative, community, and country stories.
- github.com/openimis — every module and assembly repository.
- [Official openIMIS wiki](#) — functional and product documentation, the authoritative source for scheme concepts.
- [Digital Public Goods Alliance](#) — what a DPG is and the registry openIMIS appears in.
- Next in this handbook: [Terminology Primer](#) and [Set Up a Dev Environment](#).

Terminology & Glossary Primer

Part 2 of the course. openIMIS mixes two vocabularies: the **health-financing domain** (insuree, policy, contribution, claim) and the **platform's own conventions** (module, assembly repo, `MutationLog`, `json_ext`). Confusing them — or, worse, mapping a domain word onto the wrong platform concept — is the single most common source of early bugs and miscommunication. This chapter is your primer.

Learning objectives

By the end of this chapter you will be able to:

- Use the core **domain** vocabulary — insuree, individual, family, product, policy, contribution, claim, health facility, location, officer, payer, price list, capitation — correctly and precisely.
- Distinguish pairs that newcomers routinely conflate: **insuree vs. individual**, **product vs. policy**, **contribution vs. payment**.
- Recognize the platform's own vocabulary — **module**, **assembly repo**, **rights/permissions**, `MutationLog`, `validity_from / validity_to`, `json_ext` — and know which module owns each concept.

Prerequisites

- [Platform Overview](#) — the ecosystem story that these terms label.
- No code required. Keep this page open as a reference while you read the rest of the handbook.

This is a primer, not the full glossary

This chapter teaches the *essential* vocabulary and the distinctions that trip people up. The complete, alphabetized glossary — every term, every acronym — lives at [Reference → Glossary](#). Think of this page as the "words you need before chapter 3" and the glossary as the "look anything up" companion.

Part A — The domain vocabulary

These are concepts from the world of health insurance. They exist whether or not openIMIS exists; openIMIS just models them.

People and beneficiaries

Insuree

A **person covered by (or being enrolled into) a scheme**. The classic openIMIS beneficiary, implemented by the `insuree module` (`openimis-be-insuree.py`). An insuree belongs to a **family** and is linked to policies. Backed historically by the `tblInsuree` table.

Family / Household

A **group of insurees enrolled together**, usually with a designated head and dependents. Policies are typically held at the family level, so covering the family covers its members. The unit most schemes actually enrol.

Individual

A **more general "person" concept** introduced with the social-protection work, implemented by the `individual module` (`openimis-be-individual.py`). Where **insuree** is specific to health-insurance beneficiaries, **individual** is a broader registry of people used by social-protection, payroll, and benefit-plan modules. See the distinction below — this pair confuses everyone at first.

Officer / Enrolment Officer

A **field agent responsible for enrolling members and collecting contributions** in a given area. Modeled in core (linked from the `User` model via `Officer`). Officers are tied to locations and often to the policies and contributions they process. Not the same as a system administrator — an officer is a domain actor.

Payer

An **entity that pays contributions on behalf of insurees** — for example a government subsidy programme, an employer, or a donor. Lets a scheme model third-party or subsidized premiums rather than assuming every member self-pays.

Coverage and financing

Product

The **definition of an insurance plan**: premium, covered medical services and items, waiting periods, ceilings, deductibles, and the associated **price lists**. A product is a *template* of coverage. Implemented alongside the [medical/product modules](#). **A product does not cover anyone by itself** — it must be attached to a policy.

Policy

The **contract that applies a product to a family for a period of time**, with a start date, an expiry date, and a status. The policy is what makes a person *actually covered right now*. Implemented by the [policy module](#) (`openimis-be-policy_py`). If **product** is the plan, **policy** is the subscription to that plan.

Contribution

A **premium payment recorded against a policy** to keep it active. Implemented by the [contribution module](#) (`openimis-be-contribution_py`). Contributions are the *money coming in* to fund coverage. See "Contribution vs. Payment" below — they are not the same thing.

Price List

A **schedule of prices for medical services and items** attached to products and health facilities. Determines what a given service or item is worth under a given product, feeding claim valuation.

Delivery and providers

Health Facility

A **provider of care** — clinic, health center, hospital, pharmacy — that serves insurees and submits claims. Registered with a type and a place in the location hierarchy. Implemented via the [location module](#) (`openimis-be-location_py`).

Location hierarchy

The **administrative geography** of a deployment — typically region → district → municipality/ward → village, though the exact levels are country-configurable. Governs administration, officer assignments, and which facilities serve which members. Also implemented by the [location module](#).

Claim

A **request for payment submitted by a health facility for care delivered** to a covered insuree. It lists the services and items provided, is checked for eligibility, adjudicated

against the product's rules, and **valued** (often via calculation rules). Implemented by the `claim module` (`openimis-be-claim_py`). Backed historically by `tblClaim`.

Capitation

A **provider-payment method that pays a facility a fixed amount per enrolled person per period**, regardless of services delivered — as opposed to fee-for-service, which pays per service. In openIMIS, capitation and other valuation schemes are implemented as pluggable **calculation rules** (`calcrule_*` modules) on top of the `calculation` framework.

Payment

The **actual movement of money** — the money going *out* to providers (settling valued claims via the invoice/payment modules) or, at the edges, integrating with external payment gateways such as mobile money. Distinct from a contribution.



Common mistake: three pairs you must not conflate

- **Insuree ≠ Individual.** *Insuree* is the classic health-insurance beneficiary (`insuree module`). *Individual* is the newer, more general person concept used by social-protection and payroll (`individual module`). Different modules, different tables, related but not interchangeable.
- **Product ≠ Policy.** A *product* is the plan definition; a *policy* is a specific family's time-bounded subscription to that plan. A product covers nobody until a policy attaches it to a family.
- **Contribution ≠ Payment.** A *contribution* is a **premium coming in** to fund a policy. A *payment* is money moving — most notably **going out** to providers for valued claims. They live in different modules for a reason.

Part B — The platform vocabulary

These are openIMIS-specific engineering concepts. They are *not* domain terms — they are how the platform is built. Learn these before the `Architecture` chapter.

Structure and packaging

Module

An **independently versioned plugin** that provides part of openIMIS. On the backend it is a Django app in a repo named `openimis-be-<name>_py`; the installed Python package is just `<name>` (e.g. `core`, `claim`, `policy`). On the frontend it is `openimis-fe-<name>_js`

exporting a config object. Modules are the unit of everything: code, release cadence, and configuration. Covered in depth in the [Plugin System](#) chapter.

Assembly repo

The **thin top-level project that stitches modules into one running application**.

Backend: `openimis-be_py` (a Django project). Frontend: `openimis-fe_js` (a React SPA). Each is mostly a manifest plus wiring; almost no business logic lives here. See the [Repository Map](#).

`openimis.json`

The **module manifest** — the list of modules (~47 on the backend) that the assembly repo should load, each with a pip/git source. A build step turns it into `modules-requirements.txt` for installation. There is a backend `openimis.json` (in `openimis-be_py`) and a frontend one (in `openimis-fe_js`).

Core module

The **framework layer every other module depends on** (`openimis-be-core_py`, package `core`). It provides the base models, the custom `User`, the permissions machinery, the GraphQL/Graphene helpers, the `OpenIMISMutation` base class, service signals, the calculation-rule framework, and the scheduler. Nearly everything in Part B below is defined here. Its own chapter is [Modules → Core](#).

Permissions and users

Rights / Permissions

openIMIS authorizes actions using **integer permission codes** ("rights"), defined per module in each module's `apps.py` (for example `gql_query_claims_perms = [111001]`). Users hold rights through their roles; resolvers and mutations check them with `user.has_perms([<int codes>])`. On the frontend, route guards check the same integer rights. Do not expect Django's string-based permission names here — the codes are numbers. Detailed in the [Security](#) chapter.

User (and InteractiveUser / TechnicalUser / Officer)

openIMIS ships a **custom User model** (in core) that links to several role objects: - **InteractiveUser** — a human who logs into the UI. - **TechnicalUser** — a service account for machine-to-machine access. - **Officer** — the domain actor (enrolment officer) described in Part A. A single `User` unifies these so authentication and rights work uniformly.

The framework primitives

`MutationLog`

The **audit-and-status record for every GraphQL mutation**. openIMIS mutations are **asynchronous**: the `OpenIMISMutation` base class creates a `MutationLog`, does the real work in a service, and immediately returns a `clientMutationId`; the client then **polls** the mutation's status (received → checked/processing → success/failed). This gives every write a durable audit trail and a uniform async contract. The pattern is taught in the [GraphQL](#) chapter.

`ValidityFrom` / `ValidityTo` (`validity_from` / `validity_to`)

Temporal versioning fields carried by core's `HistoryModel` / `VersionedModel`. Instead of hard-deleting or overwriting, openIMIS often "closes" a row by stamping `validity_to` and creating a new version — so you can reconstruct the state of the data at any past date. A row with an empty `validity_to` is the currently valid version. Inherited and formalized from the legacy IMIS design. Covered in [Database](#).

`json_ext`

A **JSONField present on many models for extensibility**. Because a module cannot always add columns to another module's table, `json_ext` provides a per-row "extension bag" where extra, deployment- or module-specific attributes can live without a schema migration. It is the pragmatic escape hatch that keeps the modular system flexible — and, used carelessly, a place where undocumented data hides. Use it deliberately.

`legacy_id`

The **old integer primary key from IMIS**, retained alongside the modern UUID on versioned models so legacy data and reports still line up after migration.

Did you know?

The asynchronous `MutationLog` pattern means the openIMIS frontend is *not* a simple "fire request, await response" client. After sending a mutation it "journalizes" and **polls** for the mutation's final status. This is why the React layer uses a custom GraphQL integration over Redux rather than Apollo — see the [Frontend Architecture](#) chapter.

Quick-reference table

A compressed cheat sheet. Domain terms on top, platform terms below.

Term	Category	One-line meaning	Owning module
Insuree	Domain	Health-insurance beneficiary	insuree
Individual	Domain	General person (social protection)	individual
Family	Domain	Household enrolled together	insuree
Officer	Domain	Enrolment/field agent	core
Payer	Domain	Third party paying premiums	contribution / payment
Product	Domain	Insurance plan definition	product / medical
Policy	Domain	Family's subscription to a product	policy
Contribution	Domain	Premium coming in	contribution
Price list	Domain	Prices for services/items	medical / product
Health facility	Domain	Care provider that submits claims	location
Location hierarchy	Domain	Administrative geography	location
Claim	Domain	Facility's request for payment	claim
Capitation	Domain	Per-enrollee provider payment	calculation / calcrule_*
Payment	Domain	Money going out (and gateways)	payment / invoice

Term	Category	One-line meaning	Owning module
Module	Platform	Independently versioned plugin	—
Assembly repo	Platform	Thin project stitching modules	<code>openimis-be_py</code> / <code>openimis-fe_js</code>
<code>openimis.json</code>	Platform	Module manifest	assembly repos
Rights / permissions	Platform	Integer authorization codes	<code>core</code> + each module
<code>MutationLog</code>	Platform	Async mutation audit + status	<code>core</code>
<code>validity_from</code> / <code>validity_to</code>	Platform	Temporal versioning	<code>core</code>
<code>json_ext</code>	Platform	Per-row extension JSON	<code>core</code> (used everywhere)

Hands-on lab

Lab 2 — Build your own term map

Goal: cement the domain/platform split and the confusable pairs.

1. On a sheet of paper (or a note), draw two columns: **Domain** and **Platform**. Place each term from the quick-reference table in the correct column *from memory*, then check yourself against the table.
2. For each of the three confusable pairs — insuree/individual, product/policy, contribution/payment — write one sentence that uses **both** terms correctly in the same sentence.
3. Open the `openimis-be-core_py` repository and locate `core/models.py`. Find the base model classes mentioned here (`HistoryModel`, `VersionedModel`) and confirm the presence of the `validity_from` / `validity_to` fields and a `json_ext` field. You do not need to understand the code yet — just verify the vocabulary maps to real fields. (See `openimis-be-core_py/core/models.py`.)

Success looks like: you never again write "the product covers this family" when you mean "the policy," and you can point at the real fields behind `validity_to` and `json_ext`.

Knowledge check

Q1: What is the difference between a product and a policy? (click for answer)

A **product** is the definition of an insurance plan (premium, covered services/items, ceilings, price lists) — a template. A **policy** is a specific family's time-bounded subscription to a product, with a start date, expiry date, and status. A product covers nobody until a policy attaches it to a family.

Q2: A contribution and a payment both involve money. How do they differ in openIMIS? (click for answer)

A **contribution** is a **premium coming in** to fund a policy (contribution module). A **payment** is money **moving**, most notably **going out** to providers to settle valued claims (payment/invoice modules), plus integration with external payment gateways. Different direction, different modules.

? Q3: What is a MutationLog and why does it exist? (click for answer)



`MutationLog` is core's audit-and-status record created for every GraphQL mutation. openIMIS mutations are **asynchronous**: the `OpenIMISMutation` base class creates a `MutationLog`, runs the work in a service, returns a `clientMutationId` immediately, and the client polls the status. It exists to give every write a durable **audit trail** and a uniform **async contract**.

? Q4: What is `json_ext` for, and what is the risk of using it? (click for answer)



`json_ext` is a `JSONField` on many models that acts as a per-row extension bag, letting a module or deployment attach extra attributes without a schema migration — essential flexibility in a modular system. The risk is that data placed there is **schema-less and easy to hide**: undocumented `json_ext` contents become invisible, un-queryable, and hard to maintain if used carelessly.

? Q5: The custom User model links to three role objects. Name them and their purpose. (click for answer)



InteractiveUser (a human logging into the UI), **TechnicalUser** (a service/machine account), and **Officer** (the enrolment/field-agent domain actor). The single `User` model unifies them so authentication and integer rights work uniformly across human, machine, and domain roles.

Further reading

- [Reference](#) → [Glossary](#) — the complete, alphabetized term list this primer distills from.
- [Modules](#) → [Core](#) — where most of the Part B primitives are defined.
- [Security](#) — the integer rights/permissions model in depth.
- [Database](#) — `validity_from`/`validity_to`, `legacy_id`, and the legacy schema heritage.
- Official concepts on the [openIMIS wiki](#) — the authoritative source for scheme terminology.

Set Up a Dev Environment

Part 3 of the course. Reading about openIMIS only gets you so far — you learn the plugin architecture fastest by watching it assemble itself on your own machine. This chapter is a **concise quickstart**: clone the assembly repos, understand the manifest, boot the stack with Docker, and run the backend and frontend with editable module installs. The deeper material — CI, testing, migrations discipline, releases — lives in the [Developer Workflow](#) chapter, which this page deliberately does **not** duplicate.

Learning objectives

By the end of this chapter you will be able to:

- Clone the two backend/frontend **assembly repos** and locate their `openimis.json` manifests.
- Boot a full local stack with the Docker distribution (`openimis-dist_dkr`).
- Run the **backend** locally with a module installed **editable** (`pip install -e`), and the **frontend** with **Vite**.
- Reach the running app at its default URLs and know where to turn when something breaks.

Prerequisites

- [Platform Overview](#) and [Terminology Primer](#) — you should know what an *assembly repo*, a *module*, and the *manifest* are before running any of this.
- Installed locally: **Git**, **Docker + Docker Compose**, **Python 3.x** with `venv`, and **Node.js + npm** (or Yarn). These are assumed background for this handbook.



Two ways to run openIMIS locally

- **Docker-first (recommended to start):** bring the whole stack up with `openimis-dist_dkr`. Fastest path to "it's running in my browser," great for exploring.
- **Native module dev:** run the backend and frontend directly on your host with one or more modules installed **editable**, so your code changes take effect immediately. This is how you actually develop a module.

Most developers use **both**: Docker for the database and the parts they are not editing, native for the module they are working on.

The pieces you will clone

Three repositories matter for local development. (See the [Repository Map](#) for the complete picture.)

Repo	Role	You will clone it to...
<code>openimis-be_py</code>	Backend assembly project (Django)	run/serve the backend, edit <code>openimis.json</code>
<code>openimis-fe_js</code>	Frontend assembly SPA (React + Vite)	run the UI
<code>openimis-dist_dkr</code>	Docker distribution (<code>docker-compose</code>)	boot the whole stack quickly
<code>openimis-be- <name>_py</code>	Any module you want to edit	<code>pip install -e</code> it into the backend

```
# Clone the assemblies and the Docker distribution side by side
git clone https://github.com/openimis/openimis-be_py.git
git clone https://github.com/openimis/openimis-fe_js.git
git clone https://github.com/openimis/openimis-dist_dkr.git

# Clone a module you intend to edit (example: core)
git clone https://github.com/openimis/openimis-be-core_py.git
```





Clone side by side

Keep the module repos as **siblings** of `openimis-be_py`, not nested inside it. The editable-install commands below use relative paths like `../openimis-be-core_py/`, which assumes a flat directory layout:

```
openimis/  
  openimis-be_py/  
  openimis-fe_js/  
  openimis-dist_dkr/  
  openimis-be-core_py/
```

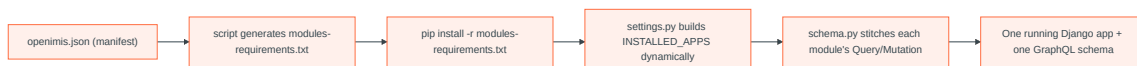
The manifest: `openimis.json`

Everything starts with the **module manifest**. In `openimis-be_py` the file `openimis.json` lists every backend module the assembled app should load, each with a source (a pip package and/or a Git reference).

```
{  
  "modules": [  
    { "name": "core",      "pip": "openimis-be-core" },  
    { "name": "insuree",   "pip": "openimis-be-insuree" },  
    { "name": "policy",    "pip": "openimis-be-policy" },  
    { "name": "claim",     "pip": "openimis-be-claim" }  
  ]  
}
```

Illustrative — the real manifest lists roughly **47** modules and richer source specs (see `openimis-be_py/openimis.json`).

The assembly turns this manifest into installable requirements and a dynamic app list:



You edit `openimis.json` when you want to **add, remove, or re-point a module** (for example, point `core` at your local checkout during development). *How* `settings.py` and `schema.py` consume this is the subject of the [Backend Deep Dive](#) and [Plugin System](#) chapters — here we just run it.

Did you know?

The frontend has its **own** `openimis.json` in `openimis-fe_js`. On the frontend, `openimis-config-vite.js` reads that manifest and generates `src/modules.js`, the file that imports every frontend module's config object. Same idea, two assemblies.

Path A — Docker quickstart

The fastest way to see openIMIS running is the Docker distribution. It brings up the database, backend, frontend, and a gateway together.

```
cd openimis-dist_dkr

# Provide configuration/secrets (copy the example env and edit as needed)
cp .env.example .env      # exact filename may vary – check the repo README

# Bring the stack up
docker compose up -d

# Watch the backend do its first-run migrations and config loads
docker compose logs -f backend
```

The compose stack typically includes these services (names may vary slightly by version — confirm in `openimis-dist_dkr`):

Service	What it is
<code>db</code>	PostgreSQL
<code>backend</code>	Django served via Gunicorn; runs migrations + loads module configs on start
<code>frontend</code>	React build served by Nginx
<code>gateway</code>	Nginx reverse proxy tying the frontend, <code>/api</code> , and <code>/graphql</code> together
<code>opensearch</code> (optional)	Backing store for <code>opensearch_reports</code> analytics



Common mistake: startup order matters

The **backend runs database migrations and loads each module's configuration on first start**, so it must come up *after* `db` is accepting connections. If the backend container appears to crash-loop on a fresh checkout, it is usually racing the database — give `db` time to become healthy (or rely on the compose health/ordering config) and retry. Watch `docker compose logs -f backend` to confirm migrations actually completed before you go hunting for other causes.

Once the stack is healthy, open the app (see [Default URLs](#) below). At this point you have a working openIMIS with no code of your own — perfect for exploring the UI and the GraphQL endpoint.

Path B — Native backend with an editable module

To actually develop, you want your changes in a module to take effect without rebuilding a container. That is what an **editable install** (`pip install -e`) gives you.

```
cd openimis-be_py

# 1. Create and activate a virtualenv
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

# 2. Install the assembled backend's requirements
pip install -r requirements.txt

# 3. Install the module(s) you are editing in EDITABLE mode.
#    This overrides the pip-installed version with your local checkout.
pip install -e ../openimis-be-core_py/

# 4. Point the backend at a database (Docker's db is convenient) via env/.env,
#    then run migrations and start the dev server.
python openimis/manage.py migrate
python openimis/manage.py runserver 0.0.0.0:8000
```

Illustrative command layout — confirm exact paths and settings-module conventions in `openimis-be_py` (its README and `openimis/settings.py`). The key idea: `pip install -e ../openimis-be-<name>_py/` makes Python import your working copy of that module, so edits to its `models.py`, `schema.py`, `services.py`, etc. are picked up on reload.



Editable install + the manifest

An editable install changes *where Python finds the code*; the module must still be **listed in `openimis.json`** so the assembly loads it into `INSTALLED_APPS` and stitches its schema. Editable-installing a module that the manifest does not include will not make it appear in the app.

Path C — Native frontend with Vite

The frontend is a React SPA that recently migrated to **Vite** (from Create React App + CRACO).

```
cd openimis-fe_js

# 1. Install dependencies
npm install

# 2. Generate src/modules.js from the frontend openimis.json manifest
#     (Vite config helper – see openimis-config-vite.js)
npm run build:modules      # exact script name may vary; check package.json

# 3. Start the Vite dev server (hot module reload)
npm run start              # or: npm run dev – check package.json scripts
```

Vite serves the SPA with hot-reload and proxies API/GraphQL calls to the backend (configured in the Vite config). Because the frontend uses a **custom Redux-based GraphQL layer — not Apollo** — and the async `MutationLog` polling described in the [Terminology Primer](#), expect the network tab to show a mutation request followed by **status-polling** queries. That is normal; it is explained fully in [Frontend Architecture](#).

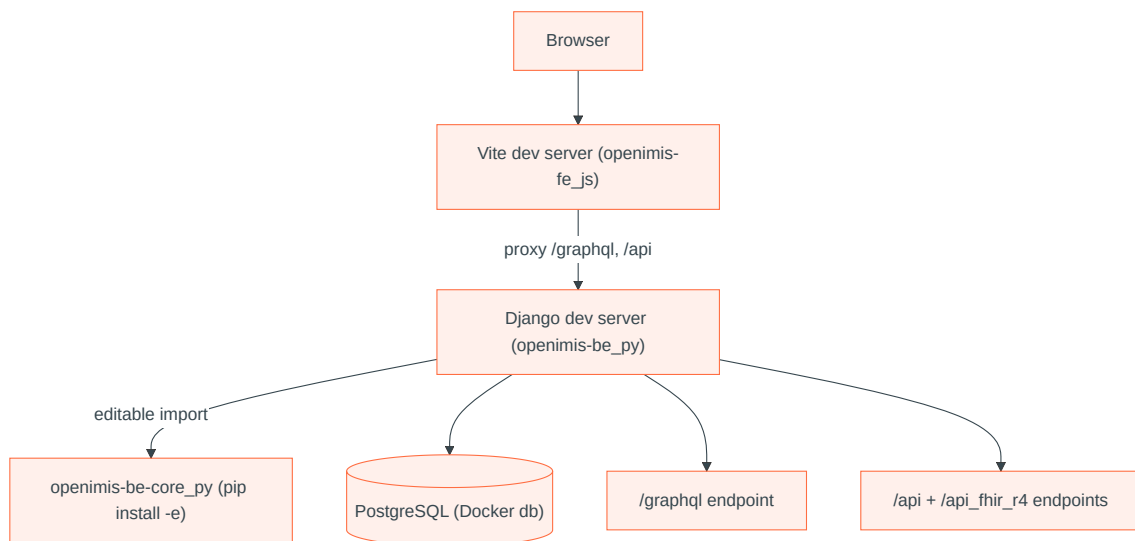


Common mistake

Forgetting to (re)generate `src/modules.js` after changing the frontend `openimis.json`. If a frontend module's menu, route, or component "isn't there," the manifest-to-`modules.js` generation step (via `openimis-config-vite.js`) very likely did not run. Regenerate, then restart the dev server.

The local dev topology

Here is how the pieces talk to each other when you mix Docker and native processes — the common "database and gateway in Docker, the module I'm editing native" setup:



The frontend dev server proxies GraphQL and REST calls to your native Django server; your Django server imports the module you are editing directly from its checkout; and both share the PostgreSQL instance (often the one from the Docker distribution).

Default URLs and ports

Exact ports depend on your compose file and dev-server config — always confirm against `openimis-dist_dkr` and the two assembly repos — but the typical local layout is:

Surface	Typical local URL
Frontend SPA (via gateway)	<code>http://localhost/front</code>
Frontend (native Vite dev server)	<code>http://localhost:3000</code>
Backend, native Django dev server	<code>http://localhost:8000</code>
GraphQL endpoint	<code>http://localhost:8000/graphql</code> (or <code>/api/graphql</code> behind the gateway)
GraphiQL explorer	the GraphQL URL in a browser (when enabled)
REST / FHIR	<code>.../api/</code> and <code>.../api_fhir_r4/</code>



Meet GraphQL early

When the backend is up, open the GraphQL endpoint in a browser to reach **GraphQL**, an interactive query explorer. Even before you know GraphQL, poke at it — run the introspection, expand `Query`, and see the fields each module contributes. It is the single best way to *feel* the stitched-together schema the [GraphQL chapter](#) explains.

Hands-on lab



Lab 3 — From zero to a running query

Goal: stand up openIMIS and run your first GraphQL query against it.

1. Clone `openimis-be_py`, `openimis-fe_js`, and `openimis-dist_dkr` as **siblings** in one parent directory.
2. In `openimis-dist_dkr`, create your `.env` and run `docker compose up -d`. Follow `docker compose logs -f backend` until you see migrations complete and the module configs load. Note in your own words *why* the backend must start after `db`.
3. Open the frontend in your browser (see [Default URLs](#)). Confirm the login screen renders.
4. Open the **GraphQL endpoint** in your browser to reach GraphQL. Run a tiny query — for example, introspect the schema or fetch the current user — and confirm you get a JSON response. (If auth blocks you, that is expected; note *which* status the response carries.)
5. **Editable-install challenge:** stop the Docker `backend`, run the backend natively per Path B with `pip install -e ../openimis-be-core_py/`, add a trivial harmless change to a docstring in `core`, reload, and confirm your change is picked up without reinstalling.

Success looks like: the app renders in your browser, GraphQL returns a response, and you have watched an editable module change take effect live.

Exercises

1. Open both `openimis.json` files (backend and frontend). List three modules that appear in the backend manifest and find their matching `openimis-be-<name>_py` repositories.
2. In the frontend, locate `openimis-config-vite.js` and describe, in two sentences, what it generates and why the app cannot start without it.
3. Identify, from `openimis-dist_dkr`, which service acts as the **gateway** and what three things it routes together.

Knowledge check

? Q1: What does `pip install -e ../openimis-be-core_py/` do, and why is it useful in development? (click for answer) ✓

It installs the `core` module in **editable** mode, so Python imports your local working copy instead of the pip-published version. Edits to that checkout take effect on the next reload without reinstalling — the standard way to develop a module against the assembled backend.

? Q2: Why must the Docker `backend` service start after `db`? (click for answer) ✓

Because on first start the backend **runs database migrations and loads each module's configuration into the database**. Without a ready `db`, those steps fail and the container may crash-loop. Startup order is `db` → `backend` → `frontend / gateway`.

? Q3: You editable-installed a module but it doesn't appear in the running app. What did you likely forget? (click for answer) ✓

To list it in `openimis.json`. The editable install only changes where Python finds the code; the module must be in the manifest so the assembly adds it to `INSTALLED_APPS` and stitches its GraphQL schema.

? Q4: The frontend uses which build tool and which state library — and notably which library does it NOT use for GraphQL? (click for answer) ✓

It builds with **Vite** and manages state with **Redux** (with `thunks`). It does **not** use **Apollo** — GraphQL goes through a custom Redux-based layer in core, including the async `MutationLog` polling ("journalize") helper.

? Q5: A frontend module's menu item isn't showing up after you edited the frontend manifest. What's the most likely cause? (click for answer) ✓

`src/modules.js` was not regenerated. After changing the frontend `openimis.json`, you must re-run the generation step (via `openimis-config-vite.js`) and restart the dev server so the new module config is imported.

Where to go next

- If something above broke, start at [Reference → Troubleshooting](#) — it catalogs the common local-setup failures (migration races, missing `.env` values, `manifest/modules.js` drift, port conflicts).
- For CI, automated testing, migration discipline, and the release/contribution process, go to the [Developer Workflow](#) chapter — this quickstart deliberately stays out of that territory.
- With the app running, open the hood: [Architecture Overview](#), then learn the API in [GraphQL](#).

Further reading

- `openimis-be_py`, `openimis-fe_js`, and `openimis-dist_dkr` **READMEs** — the authoritative, version-current setup steps.
- github.com/openimis — all module repositories.
- [Official openIMIS wiki](#) — deployment and installation guides.
- [Vite documentation](#) — the frontend build tool.

High-Level Architecture

This chapter is the map you will keep coming back to. It gives you the whole of openIMIS at one altitude — every major layer, why it exists, and how a request travels from a clinician's browser down to a PostgreSQL row and back out to an external FHIR client. Later chapters zoom into each layer; this one keeps them all in frame at once.

If you have built Django services before, most of the pieces here will feel familiar in isolation. What is *unfamiliar* — and what this chapter exists to teach — is the **assembly model**: openIMIS is not one application but a large set of independently versioned modules that are *assembled* into a running system by two thin "assembly" repositories. Internalize that idea and the rest of the platform stops looking sprawling and starts looking deliberate.

Learning objectives

By the end of this chapter you will be able to:

- Draw the full openIMIS layer stack from memory: **User** → **Frontend** → **Gateway** → **API layer** → **Core framework** → **Business modules** → **PostgreSQL** → **External systems**.
- Explain *why* each layer exists and what would break without it.
- Distinguish an **assembly repository** (`openimis-be_py` , `openimis-fe_js`) from a **module repository** (`openimis-be-claim_py` , `openimis-fe-insuree_js`).
- Articulate the "assembly + plugins" mental model and why openIMIS chose it over a monolith.
- Trace a GraphQL query and an asynchronous mutation through the stack.
- Know which later chapter to open when you need the next level of detail.

Prerequisites

- [Platform Overview](#) — what openIMIS is and the problem domain (health financing) it serves.
 - [Terminology & Glossary Primer](#) — insuree, policy, contribution, claim, product.
 - Working knowledge of Django, the Django ORM, PostgreSQL, and Docker. You do **not** need to know GraphQL yet — it is introduced where it appears and taught in full in [GraphQL](#).
-

1. A one-paragraph mental model

openIMIS is a **modular Django + React platform** for running health-insurance and social-health-protection schemes. The backend is a Django project that dynamically assembles ~47 Django apps ("modules") listed in a manifest. The frontend is a React single-page application that assembles a matching set of JavaScript modules. Between them sits a GraphQL API (plus REST/FHIR endpoints), fronted by an Nginx gateway. Underneath sits a single PostgreSQL database whose schema still carries the fingerprints of the platform's .NET/MSSQL ancestor. On top and to the side sit external systems: FHIR clients, DHIS2, payment gateways, and OpenSearch for analytics.

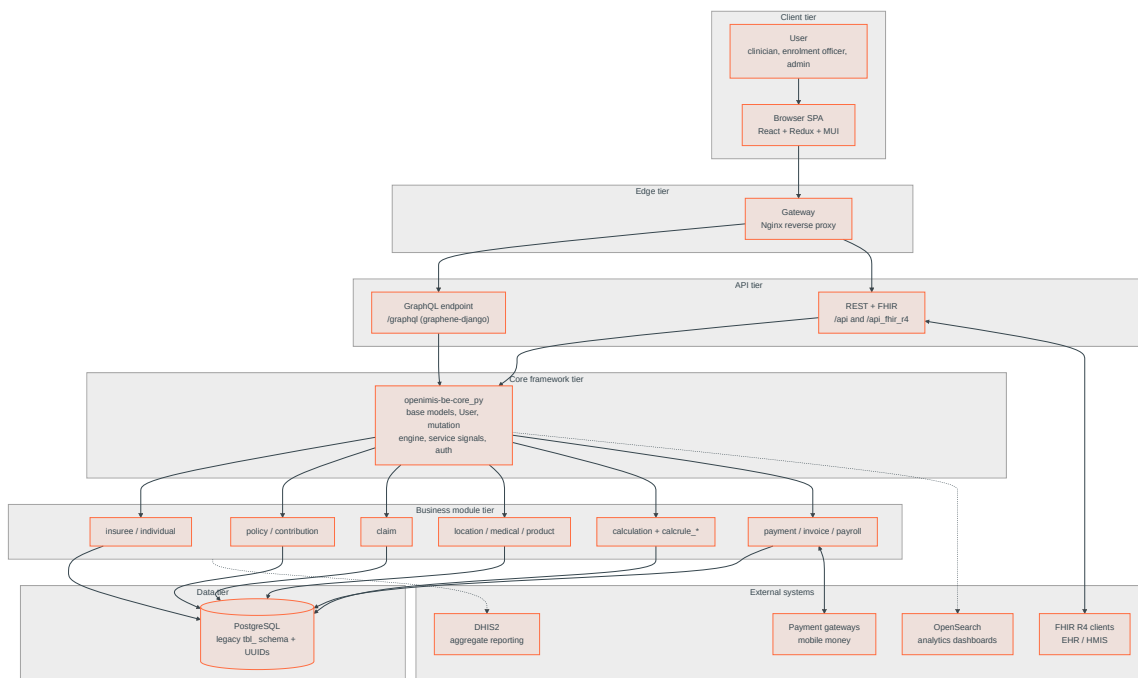
Everything else in this handbook is detail hanging off that sentence.

Did you know?

openIMIS did not start life modular. Its ancestor, **IMIS**, was a monolithic **Microsoft .NET + Microsoft SQL Server** application with heavy business logic in stored procedures. The 2018–2019 re-architecture into Django + React modules was a deliberate strategic bet: countries needed to customize the system without forking a monolith. The modular design *is* the product strategy — not an implementation detail.

2. The layered architecture, end to end

Here is the whole stack. Read it top to bottom: each layer only talks to the layer directly beneath it (with the deliberate exception of external systems, which attach at several points).



The rest of this section walks each layer, top to bottom, and answers one question for each: **what is it, and what would break without it?**

2.1 User

The people the system serves are not anonymous web visitors — they are **role-bearing operators**: enrolment officers registering families, clinicians submitting claims, scheme administrators configuring products, finance staff reconciling payments. Every one of them authenticates and carries a set of integer **rights** (openIMIS's permission codes). The whole security model — from route guards in React down to `user.has_perms([...])` checks in GraphQL resolvers — exists to map these humans to what they may see and do. See [Security](#).

2.2 Frontend (React + Redux)

The browser runs a **single-page application** built from React. It is assembled from many `openimis-fe-<name>.js` modules by the `openimis-fe.js` assembly repo. State lives in **Redux** (with thunks), *not* Apollo — openIMIS talks to GraphQL through its own Redux action layer. The frontend is responsible for rendering the UI, holding form state, dispatching GraphQL operations, and — crucially — **polling** for the result of asynchronous mutations. It is covered in full in [Frontend Architecture](#).

Why it exists: to give distributed, often low-connectivity users a responsive, role-aware interface without shipping business logic to the client. The client renders and orchestrates; it never decides what a claim is worth.

2.3 Gateway (Nginx)

A single **reverse proxy** presents one origin to the browser and fans requests out behind it: static frontend assets, `/graphql`, `/api`, `/api_fhir_r4`. In the reference `openimis-dist_dkr` deployment this is an Nginx container.

Why it exists: the browser must see one host. Without a gateway you would face cross-origin (CORS) headaches, cookie-scoping problems (the JWT lives in an **HttpOnly cookie**, which is far easier to manage on a single origin), and no single place to terminate TLS or apply rate limits. The gateway is the seam between "the internet" and "the application."

2.4 API layer (GraphQL + REST/FHIR)

This is the layer most likely to be new to you, so it gets extra attention.

- `/graphql` is the primary API. It is a single endpoint that accepts *queries* (reads) and *mutations* (writes) expressed in the GraphQL language, served by **graphene-django**. Unlike REST, where each resource has its own URL and the server dictates the response shape, GraphQL exposes one typed graph and lets the client ask for exactly the fields it needs. openIMIS builds one giant root `Query` and root `Mutation` by combining every module's schema (more on this in §4).
- `/api` and `/api_fhir_r4` are conventional REST endpoints (Django views / DRF) used for reports, tooling, and the **FHIR R4** integration surface that external health systems speak.

Why two styles? GraphQL is superb for the rich, deeply nested, client-driven screens of the SPA. FHIR/REST is a fixed, standardized contract that the outside world already knows how to speak. Different consumers, different tools. See [Request Lifecycle](#) and [GraphQL](#).

Did you know?

Authentication in openIMIS is done with `django-graphql-jwt`, but the token is not handed to JavaScript. It is set as an **HttpOnly cookie**, so the SPA code can never read it and cannot leak it via XSS. There is also OpenID Connect / OAuth2 support for plugging in external identity providers.

2.5 Core framework (`openimis-be-core_py`)

Every business module stands on **core**. Core is not a business feature; it is the *framework within the framework*. It provides:

- **Base models** — `HistoryModel`, `HistoryBusinessModel`, `VersionedModel` (with `validity_from` / `validity_to` temporal columns and `legacy_id`), `UUIDModel`, and a `json_ext` JSON field for country-specific extension without schema changes.

- **The custom User model** and its identity types (`InteractiveUser` , `TechnicalUser` , `Officer`), plus the integer **rights** permission system.
- **The GraphQL mutation engine** (`OpenIMISMutation`) — the asynchronous, audited mutation pattern that every module reuses.
- **Service signals** (`register_service_signal` / `bind_service_signal`) — the server-side extension seam that lets one module hook another's business logic without importing it.
- Graphene helpers (`ExtendedConnection` , `OrderedDjangoFilterConnectionField`), the configuration overlay (`ModuleConfiguration`), scheduling (`APScheduler`), audit, and utilities.

Why it exists: to make 40+ modules *feel like one product*. History, versioning, permissions, auditing, config, and the mutation lifecycle are solved **once** in core and inherited everywhere. Read the whole story in [Backend Deep Dive](#).

2.6 Business modules

The actual domain lives here, one concern per module: `insuree / individual` (beneficiaries), `location / medical / product` (reference data), `policy` (coverage), `contribution` (premiums), `claim` (service delivery), `calculation + calcrule_*` (pricing/valuation rules), `payment / invoice / payroll` (money out). Each is an independent Django app in its own repository, depending on core and on a handful of sibling modules. The dependency ordering is the subject of [Repository Map](#).

Why decomposed? Because a scheme in Nepal and a scheme in Cameroon share maybe 80% of their logic and diverge on the other 20%. Modules let a country swap or extend the 20% (a new calculation rule, an extra field via `json_ext` , a replacement component) without touching — or re-testing, or re-forking — the 80%.

2.7 PostgreSQL

One relational database holds it all. Its schema is unusually shaped for a Django app: many tables keep legacy **tbl prefixes** and **camelCase columns**, and rows carry both legacy integer keys *and* UUIDs. That is the visible scar tissue of the MSSQL-to-PostgreSQL migration from the IMIS era. See [Database](#).

Why one database? Insurance is inherently relational and transactional — a claim references a policy references an insuree references a product, and money must reconcile. A single ACID database keeps those invariants honest. openIMIS is a modular *codebase*, not a distributed *data* architecture.

2.8 External systems

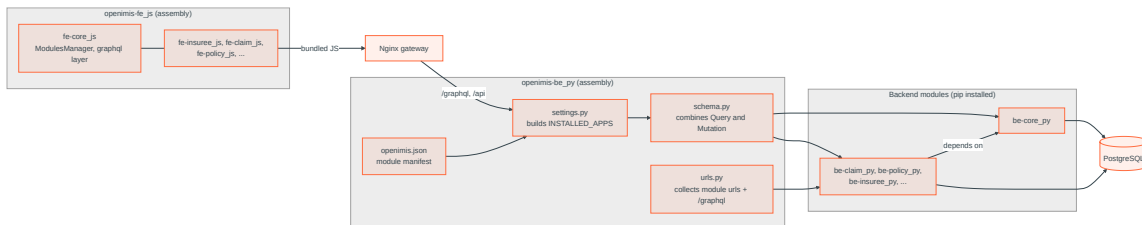
- **FHIR R4 clients** (EHRs, national HMIS) exchange data through `/api_fhir_r4/` .

- **DHIS2** receives aggregate indicators via the `dhis2_etl` module.
- **Payment gateways** (mobile money, etc.) connect through the payment/invoice modules.
- **OpenSearch** powers analytical dashboards via `opensearch_reports`.

Why external? openIMIS runs inside a national digital-health ecosystem. It is a good citizen: it speaks the standards (FHIR), feeds the national dashboards (DHIS2), and moves real money (gateways) rather than trying to be all of those things itself.

3. The component view

The layer diagram shows *tiers*. This component diagram shows the **concrete software artifacts** and who imports/serves whom — useful when you are staring at a `docker ps` list wondering which container does what.

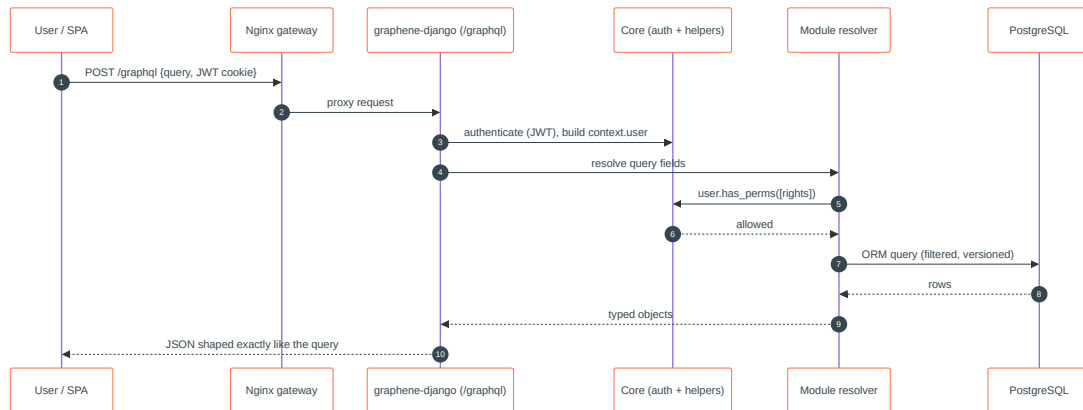


Notice the symmetry: **each side has one thin assembler and many fat modules**. The backend assembler (`openimis-be_py`) reads a manifest and wires modules into one Django project. The frontend assembler (`openimis-fe_js`) reads *its own* manifest and wires modules into one React bundle. The two assemblers never share code — they mirror each other in spirit.

4. Data flow: two journeys through the stack

Diagrams of tiers are static. To really understand openIMIS you must watch data *move*. Here are the two canonical journeys. (The full treatment, with resolver internals and JWT handling, is in [Request Lifecycle](#).)

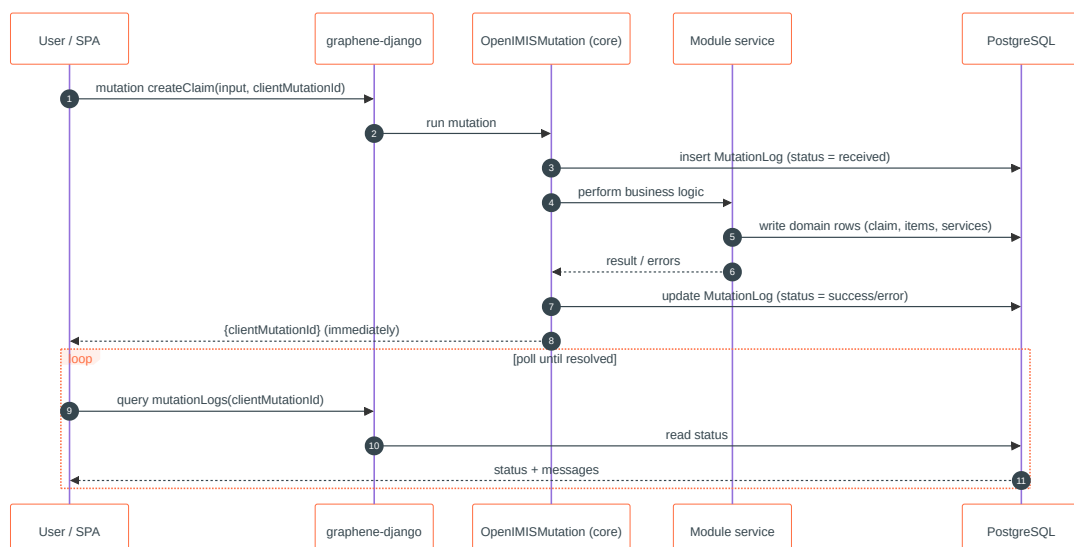
4.1 A read (GraphQL query)



The key openIMIS-specific beats: the JWT arrives in a cookie and is turned into `context.user` by core; permission checks use **integer rights**; and ORM queries usually respect **temporal validity** (`validity_to` IS NULL for "current" rows), because core's base models are versioned.

4.2 A write (asynchronous mutation)

This is the pattern that surprises Django engineers most. openIMIS mutations are **asynchronous and audited**: the mutation does not return the created object. It records a job, kicks off the work, and hands back a `clientMutationId`. The client then **polls** for the outcome.





Did you know?

Because **every** mutation is funneled through `OpenIMISMutation`, openIMIS gets a complete, uniform **audit trail** for free: who changed what, when, and whether it succeeded — one `MutationLog` row per attempt. In a system handling public health funds, that audit property is not a nice-to-have; it is a compliance requirement satisfied by architecture.



Common mistake

Treating a mutation response as the finished result. The mutation returns a `clientMutationId`, **not** your new claim. If your integration code assumes synchronous CRUD semantics, it will "succeed" while the actual work later fails silently in the `MutationLog`. Always poll the mutation status. This is the single most common integration bug for engineers new to openIMIS.

5. The assembly + plugins mental model

Now the central idea, stated plainly. Two kinds of repository exist, and keeping them straight is the difference between understanding openIMIS and being perpetually confused by it.

	Assembly repos	Module repos
Examples	<code>openimis-be_py</code> , <code>openimis-fe_js</code>	<code>openimis-be-claim_py</code> , <code>openimis-fe-insuree_js</code>
What it is	The deployable project that wires modules together	An installable plugin that provides one concern
Contains business logic?	Almost none	Yes — this is where the domain lives
Count	Two (one BE, one FE)	Dozens (~47 backend modules alone)
Key artifact	<code>openimis.json</code> manifest + <code>settings.py</code> / <code>schema.py</code> / <code>urls.py</code>	<code>apps.py</code> (<code>AppConfig</code>), <code>models.py</code> , <code>schema.py</code> , <code>services.py</code>
Versioned	As a deployment	Independently, per module
You edit it to...	Choose <i>which</i> modules and <i>which</i> versions run	Change <i>what a module does</i>

Think of it like a Linux distribution. The **kernel + a package manifest** is the assembly; the **packages** are the modules. `openimis-be_py` is analogous to a distro's package list plus its init system: it decides what is installed and brings it up in the right order. It intentionally holds no domain logic of its own.

How assembly actually happens (backend)

1. `openimis.json` lists every module and where to get it (pip/git). A script renders it to `modules-requirements.txt`; pip installs each module as a Python package (`core`, `claim`, `policy`, ...). For development you install modules editable: `pip install -e ../openimis-be-core_py/`.
2. `openimis/settings.py` reads the loaded module list and builds **INSTALLED_APPS** dynamically — you never hand-edit the app list.
3. `openimis/schema.py` imports each module's `schema.Query` and `schema.Mutation` and fuses them into one root `Query` / `Mutation` using **Python multiple inheritance**. One endpoint, every module's fields.

4. `openimis/urls.py` collects each module's `urls.py` patterns and mounts the `/graphql` endpoint.

How assembly actually happens (frontend)

The frontend mirrors this. `openimis.json` is the FE manifest; `openimis-config-vite.js` generates `src/modules.js`; each `openimis-fe-<name>.js` module exports a **config object** with keys like `reducers`, `queries`, `mutations`, `routes`, `menus`, and — the extension seam — **contributions**, named slots into which any module can inject a component. The **ModulesManager** from `fe-core` is the runtime registry (`getRef`, `getConfig`, `getContribs`).

The deep mechanics of both seams — backend service signals and frontend contributions — live in [Plugin / Module System](#).

Deep dive: why multiple inheritance for the schema?

Combining N module `Query` classes into one root could be done many ways: schema stitching, namespacing, a registry. openIMIS chose the bluntest tool Python offers — **multiple inheritance** — so that `class Query(ClaimQuery, PolicyQuery, InsureeQuery, ...)` yields a single class exposing the union of every module's fields and resolvers. It is simple and requires zero glue code per module. The cost is that field-name collisions across modules are real and must be avoided by convention (module-prefixed field names), and the Method Resolution Order matters if two modules define a same-named resolver. It is a pragmatic choice that trades a little namespace discipline for a lot of assembly simplicity — very much in keeping with openIMIS's overall philosophy.

Tip

When you get lost in openIMIS, ask yourself: *"Am I looking at an assembler or a module?"* If a file lives under `openimis/` in `openimis-be.py`, it is wiring. If it lives under a package name like `claim/`, it is domain. That single question resolves most "where does this belong?" confusion.

6. Why this architecture (the design rationale)

It is worth pausing on *why* openIMIS looks the way it does, because the reasons recur throughout the handbook:

- **Country customization without forking.** The founding requirement. Modules + `json_ext` + `ModuleConfiguration` + calculation rules let a deployment diverge in controlled seams instead of forking core. This one requirement explains 90% of the architecture.

- **Independent versioning.** Each module ships on its own cadence. A claim-logic fix does not force a policy-module release.
- **Auditability by construction.** The uniform async-mutation engine gives every write an audit record — essential for public funds.
- **Standards at the edges, freedom at the center.** FHIR/REST present a stable external contract; GraphQL gives internal screens flexibility; the core is free to evolve behind both.
- **Legacy respected, not worshiped.** The `tbl` -prefixed schema and dual integer/UUID keys are a bridge from IMIS, kept deliberately so existing deployments and data could migrate.

” Quote

A good way to hold openIMIS in your head: *"A thin assembler brings up a fleet of versioned plugins, all standing on a shared core, over one relational database, behind one gateway, speaking GraphQL inward and FHIR outward."*

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be_py</code>	<code>openimis.json</code>	The backend module manifest (~47 modules); the source of truth for what gets assembled.
<code>openimis-be_py</code>	<code>openimis/settings.py</code>	Builds <code>INSTALLED_APPS</code> dynamically from the loaded modules.
<code>openimis-be_py</code>	<code>openimis/schema.py</code>	Fuses every module's <code>Query / Mutation</code> into one GraphQL root via multiple inheritance.
<code>openimis-be_py</code>	<code>openimis/urls.py</code>	Collects module URL patterns and mounts <code>/graphql</code> .
<code>openimis-be_py</code>	<code>script/</code>	Container entrypoints: run migrations and load module configs at startup.
<code>openimis-be-core_py</code>	<code>core/apps.py</code> , <code>core/models.py</code>	The framework layer: <code>AppConfig</code> , base/versioned models, <code>User</code> , mutation engine, service signals.
<code>openimis-fe_js</code>	<code>openimis.json</code> , <code>openimis-config-vite.js</code>	Frontend manifest and the generator that emits <code>src/modules.js</code> .
<code>openimis-fe-core_js</code>	<code>ModulesManager</code>	Cross-module runtime registry and the custom GraphQL/Redux layer.
<code>openimis-dist_dkr</code>	<code>docker-compose.*</code> , <code>.env</code>	The reference deployment: <code>db</code> , <code>backend</code> , <code>frontend</code> , Nginx gateway, optional OpenSearch.
<code>openimis-be-api_fhir_r4_py</code>	<code>api_fhir_r4/</code>	The external FHIR R4 surface under <code>/api_fhir_r4/</code> .

Hands-on lab

Goal: confirm the assembly model with your own eyes, no code changes.

1. Stand up the reference stack from [Set Up a Dev Environment](#) (Docker Compose).
2. Run `docker ps` and match each running container to a **tier** in the layer diagram of §2. Which container is the gateway? Which runs Django? Where is PostgreSQL?
3. Open `openimis.json` in `openimis-be_py`. Count the modules. Pick three and note their pip/git source.
4. Hit the API: open `/graphql` in the browser (the GraphQL explorer) and run a trivial query. Watch the Network tab — confirm it is a single `POST` to one endpoint, and that a JWT **cookie** (not a header your JS can read) rides along.
5. Now trigger any create action in the UI and watch the Network tab: find the mutation returning a `clientMutationId`, then the follow-up **poll** queries reading its status. You have just observed §4.2 live.

Deliverable: a short note mapping every container to a tier, plus the `clientMutationId` you captured in step 5.

Exercises

1. In one sentence each, state what would break if you deleted (a) the gateway, (b) the core module, (c) the `openimis.json` manifest.
2. A colleague says "let's add the new claim-scoring feature directly in `openimis-be_py`." Explain, using the assembly/module distinction, why that is the wrong repository and where it belongs.
3. Sketch (on paper) the query data-flow diagram from memory, then check it against §4.1. Which beats did you forget? Those are the ones to re-read.

Knowledge check

? Q1: What is the difference between an assembly repo and a module repo? (click for answer) ✓

An **assembly repo** (`openimis-be_py`, `openimis-fe_js`) is the deployable project that *wires modules together* using a manifest (`openimis.json`) and thin glue (`settings.py`, `schema.py`, `urls.py` on the backend). It holds almost no business logic. A **module repo** (e.g. `openimis-be-claim_py`) is an installable plugin that provides one domain concern and is where the business logic lives. There are two assemblers and dozens of modules.

? Q2: Why does openIMIS use both GraphQL and REST/FHIR? (click for answer) ✓

They serve different consumers. **GraphQL** (`/graphql`) is the primary, client-driven API for the rich SPA screens — the client asks for exactly the fields it needs from one typed graph. **REST/FHIR** (`/api` , `/api_fhir_r4`) is a fixed, standardized contract for the outside world (EHRs, national HMIS), which already knows how to speak FHIR. Flexibility inward, standardized stability outward.

? Q3: Why does a mutation return a `clientMutationId` instead of the created object? (click for answer) ✓

Because openIMIS mutations are **asynchronous and audited**. The `OpenIMISMutation` engine writes a `MutationLog` , runs the work in a service, and hands back a `clientMutationId` immediately; the client then **polls** for status. This yields a uniform audit trail for every write — critical when handling public health funds — at the cost of clients needing to poll rather than assume synchronous CRUD.

? Q4: What is the role of the Nginx gateway, and why not let the browser hit Django directly? (click for answer) ✓

The gateway presents **one origin** to the browser and fans requests to static assets, `/graphql` , `/api` , and `/api_fhir_r4` . A single origin avoids CORS problems, keeps the **HttpOnly JWT cookie** correctly scoped, and gives one place to terminate TLS and apply rate limiting. Hitting Django directly would reintroduce all of those cross-origin and security concerns.

? Q5: In one line, what is the 'assembly + plugins' model? (click for answer) ✓

A thin assembler reads a module manifest and brings up a fleet of independently versioned plugin modules — all standing on a shared **core**, over **one PostgreSQL** database, behind **one gateway**, speaking GraphQL inward and FHIR outward. It is the Linux-distribution pattern applied to a health-financing platform.

Further reading

- [Backend Deep Dive](#) — how `openimis-be_py` builds `INSTALLED_APPS` and the core framework in detail.
- [Plugin / Module System](#) — service signals and frontend contributions, the two extension seams.
- [Request Lifecycle](#) — the full query and async-mutation journey with resolver internals.

- [Frontend Architecture](#) — the React/Redux assembly and `ModulesManager`.
- [Repository Map](#) — every repo, its purpose, and the dependency graph.
- The [openIMIS wiki](#) and the [official site](#) for governance and roadmap.
- [GraphQL](#) if the query/mutation language is new to you.

Backend Deep Dive

You already know Django. This chapter is not about Django — it is about the **specific, unusual way openMIS uses Django** to turn a manifest and a pile of independently versioned packages into one running system. We will open the `openimis-be_py` project, watch it boot, and follow the machinery that makes 40+ modules behave as a single coherent application: dynamic `INSTALLED_APPS`, the `AppConfig / DEFAULT_CFG` convention, the database-backed configuration overlay, two distinct signal systems, permissions as integers, and the container startup lifecycle.

Read [High-Level Architecture](#) first if you have not. This chapter zooms into the "Core framework" and "Business module" tiers of that map.

Learning objectives

By the end of this chapter you will be able to:

- Navigate the `openimis-be_py` project layout and explain what each top-level file does.
- Explain how `settings.py` builds `INSTALLED_APPS` **dynamically** from `openimis.json`, and why that matters.
- Describe the `AppConfig` + `DEFAULT_CFG` convention and how a module reads its runtime config.
- Explain the `ModuleConfiguration` DB overlay: how operators reconfigure a deployment without changing code.
- Distinguish **Django ORM signals** from **openMIS service signals** (`register_service_signal` / `bind_service_signal`) and know when each is used.
- Understand openMIS **permissions as integer rights** and `user.has_perms`.
- Trace the container **startup lifecycle** from `script/` entrypoint to a request-ready server, including migrations and config loading.
- Know the common management commands and where logging/caching/middleware are configured.

Prerequisites

- [High-Level Architecture](#) — the layer stack and the assembly/module distinction.
- [Repository Map](#) — which modules exist and how they depend on each other.

- Comfort with Django apps, `AppConfig`, migrations, signals, and settings.
- [Security](#) is referenced for the authentication details that this chapter deliberately keeps at overview level.

1. Project anatomy of `openimis-be_py`

`openimis-be_py` is the **assembly repository** — the deployable Django project. It contains almost no domain logic. Its job is to *choose modules, configure Django, assemble the GraphQL schema and URLs, and bring the system up*.

The important paths (names are real; treat the tree as the canonical layout):

Path	Role
<code>openimis.json</code>	The module manifest : ~47 modules, each with a pip/git source.
<code>modules-requirements.txt</code>	Generated from <code>openimis.json</code> ; what pip actually installs.
<code>openimis/settings.py</code>	Standard Django settings plus dynamic <code>INSTALLED_APPS</code> , Graphene, and <code>graphql_jwt</code> config.
<code>openimis/schema.py</code>	Assembles the root GraphQL <code>Query / Mutation</code> from every module.
<code>openimis/urls.py</code>	Collects each module's <code>urls.py</code> and mounts <code>/graphql</code> .
<code>openimisconf/load_openimis_conf.py</code>	Loads the module configuration (which modules, their options).
<code>script/</code>	Container entrypoints : migrate, load configs, start the server.
<code>manage.py</code>	The usual Django CLI (now driving an assembled project).

Mentally, the flow is: **manifest** → **installed packages** → **Django apps** → **schema + urls** → **running server**. Everything below expands one arrow of that chain.



i Did you know?

For local development you rarely rely on the generated requirements file. Instead you install modules **editable** so your source edits take effect live: `pip install -e ../openimis-be-core_py/`. The manifest still governs *which* modules load; editable installs just change *where the code comes from*.

2. Dynamic `INSTALLED_APPS`: the heart of the assembler

In an ordinary Django project you hand-write `INSTALLED_APPS`. openIMIS does not. It **computes** it at startup from the module configuration, so that adding a module to the manifest is enough to wire it in — no edit to `settings.py` required.

The logic lives in `openimis/settings.py` together with `openimisconf/load_openimis_conf.py`. Conceptually (illustrative, simplified):

```

# illustrative – openimis/settings.py (simplified)
from openimisconf import load_openimis_conf

# The loaded configuration knows which modules are enabled, in order.
OPENIMIS_CONF = load_openimis_conf()
openimis_modules = [m["name"] for m in OPENIMIS_CONF["modules"]] # e.g.
"core", "claim", ...

INSTALLED_APPS = [
    # Django + third-party that must come first
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    # ...
    "graphene_django",
    "rest_framework",
    # ... then every openIMIS module, in manifest order:
    *openimis_modules,
]
  
```

- 1 Because `core` is listed first in the manifest, it initializes before any module that depends on it. **Manifest order is dependency order**. This is why the manifest is not just a list — it is a topological load plan.

Two consequences worth internalizing:

- **The manifest is authoritative.** If a module misbehaves, confirm it is actually in `openimis.json` and installed — a package can be pip-installed yet not enabled, or enabled yet not installed.
- **Order matters.** `core` first; reference-data modules before the modules that consume them. The dependency intuition in [Repository Map](#) is the ordering you will see reflected here.



Common mistake

Adding `some_module` to `INSTALLED_APPS` by hand. That fights the assembler: on the next config load your edit is ignored or overwritten, and the module's position relative to `core` may be wrong. **Add modules to `openimis.json`, not to `settings.py`.**

3. The standard module layout and `AppConfig`

Every backend module follows the same shape. Learn it once and you can read any of the 47.

```
openimis-be-<name>_py/
  <name>/
    apps.py          # AppConfig subclass + DEFAULT_CFG (+ permission
wiring)
    models.py        # Django models (usually extending core base models)
    schema.py        # GraphQL Query / Mutation for this module
    gql_queries.py   # Graphene ObjectTypes / connections
    gql_mutations/   # one file per mutation (larger modules)
    services.py      # business logic – the real work lives here
    signals.py       # ORM signal handlers and service-signal bindings
    migrations/
    urls.py          # REST/other routes contributed by this module
    tests/
```

The linchpin is `apps.py`, which defines an `AppConfig` subclass. This is where a module declares its identity, its default configuration, and its permission codes.

```
# illustrative – core/apps.py (simplified, faithful to conventions)
from django.apps import AppConfig

DEFAULT_CFG = {
    "default_audit_user_id": 1,
    "gql_query_users_perms": [121701], # integer rights
    "gql_mutation_create_users_perms": [121702],
    # ... many more keys
}

class CoreConfig(AppConfig):
    name = "core"
    default_cfg = DEFAULT_CFG

    def ready(self):
        # 1. overlay DEFAULT_CFG with DB-stored ModuleConfiguration
        self._load_config()
        # 2. register permissions, connect signals, etc.
        self._configure_permissions()
        import core.signals # noqa: ensures signal handlers
register
```

- 1 `DEFAULT_CFG` is the module's **built-in default configuration** — feature flags, default IDs, and the integer permission codes each query/mutation requires. It is a plain dict, so it is trivial to read and override.
- 2 `ready()` runs once per process when Django populates its app registry. This is the module's constructor: load config, wire permissions, import signal handlers so their `@receiver / bind_service_signal` decorators execute.

Did you know?

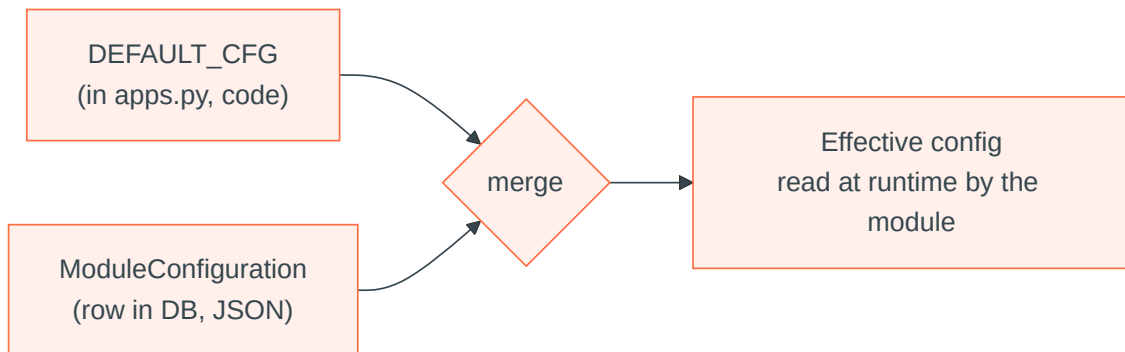
Importing `core.signals` inside `ready()` is not incidental — it is *how* signal handlers get registered. A handler that is never imported is never connected. If a signal "isn't firing," the first thing to check is whether its module actually imports the file that defines the handler during `ready()`.

4. Configuration: `DEFAULT_CFG` overlaid by `ModuleConfiguration`

Here is one of openIMIS's most important operational ideas, and it directly serves the "customize a country deployment without changing code" requirement from [High-Level Architecture](#).

Each module ships a `DEFAULT_CFG` dict in `apps.py`. At startup, core overlays that dict with a **per-module JSON document stored in the database** — the `ModuleConfiguration` model

(defined in core). The merged result is the module's **effective runtime configuration**, which the rest of the module reads (often via a module-level `<Name>Config` object populated in `ready()`).



Why this design?

- **Operators reconfigure without a deploy.** Change a feature flag, a default officer, a permission set — edit the DB config, restart, done. No code change, no image rebuild.
- **Sensible defaults ship in code.** A fresh deployment works with zero DB config because `DEFAULT_CFG` fills every key.
- **Config is auditable and per-environment.** Dev, staging, and prod can carry different `ModuleConfiguration` rows against the same code.

```
# illustrative – reading effective config inside a module
from core.models import ModuleConfiguration

def load_config(app_config):
    cfg = dict(app_config.default_cfg)           # start from defaults
    stored = ModuleConfiguration.get_or_default(app_config.name) # DB overlay
    cfg.update(stored)                           # DB wins over defaults
    return cfg
```

Common mistake

Editing `DEFAULT_CFG` in a module's `apps.py` to change a production setting. That is a **code change to a shared module** — it affects everyone and must go through release. For deployment-specific settings, write a `ModuleConfiguration` DB entry instead. Reserve `DEFAULT_CFG` for genuine defaults.

Deep dive: precedence and timing

The overlay happens in `AppConfig.ready()`, i.e. **once at process start**. That means changing a `ModuleConfiguration` row does **not** take effect until the workers restart — there is no live hot-reload of config by default. Precedence is: DB `ModuleConfiguration` **over** `DEFAULT_CFG`. A key absent from the DB falls through to the code default; a key present in the DB overrides it. When debugging "my config change did nothing," check (a) did the workers restart, and (b) is the key spelled exactly as in `DEFAULT_CFG`.

5. Two signal systems

openIMIS uses **two** distinct signal mechanisms, and conflating them is a classic source of confusion. One is stock Django; the other is an openIMIS invention that is the backend's primary extension seam.

5.1 Django ORM signals

The familiar ones: `pre_save`, `post_save`, `pre_delete`, `post_delete`, etc., connected with `@receiver`. openIMIS uses them for the usual reasons — keeping denormalized fields in sync, cache invalidation, audit side effects tied to model persistence.

```
# illustrative – a stock Django ORM signal
from django.db.models.signals import post_save
from django.dispatch import receiver
from claim.models import Claim

@receiver(post_save, sender=Claim)
def on_claim_saved(sender, instance, created, **kwargs):
    if created:
        ... # e.g. enqueue a notification
```

These fire on **ORM persistence events**. They know nothing about business operations — only that a row was saved or deleted.

5.2 openIMIS service signals

This is the openIMIS-specific one, and it is the mechanism that lets one module hook another module's **business logic** without importing it. It operates at the level of **service method calls**, not ORM events.

- `register_service_signal("module.service.method")` declares a named signal around a service method — a *before* and *after* hook point.

- `bind_service_signal("module.service.method", handler, ...)` attaches a handler to that hook from **any** module.

```
# illustrative – declaring a service signal (in the owning module)
from core.service_signals import register_service_signal

class ContributionService:
    @register_service_signal("contribution.service.create")    # (1)!
    def create(self, data):
        ... # create the contribution
        return contribution
```

```
# illustrative – binding to it from a DIFFERENT module (e.g. calcrule_*)
from core.service_signals import bind_service_signal

def after_contribution_created(sender, result, **kwargs):
    # run a calculation rule against the new contribution
    ...

bind_service_signal(
    "contribution.service.create",
    after_contribution_created,
    bind_type="after",          # before / after the service call
)
```

1. Wrapping the service method registers a signal that fires *around* the call. Subscribers registered via `bind_service_signal` run before and/or after the body, receiving the arguments and (for `after`) the result.

Why this exists — the crucial point: it inverts the dependency. The `contribution` module does not know that a calculation rule wants to run after a contribution is created. The `calcrule_*` module *binds* itself to the contribution service signal. Contribution has **zero import** of calculation. That is exactly what makes calculation rules, third-party payment logic, and country-specific hooks pluggable — the whole [calculation-rule framework](#) is built on this seam. See also [Plugin / Module System](#).

	Django ORM signals	openMIS service signals
Fires on	Model save/delete (persistence)	Service method calls (business operations)
API	<code>@receiver(post_save, ...)</code>	<code>register_service_signal / bind_service_signal</code>
Granularity	"a Claim row was saved"	"the contribution.create business op ran"
Primary use	Denormalization, cache, audit-on-save	Cross-module extension without imports
Defined by	Django	openMIS core

Deep dive: before/after semantics and error handling

A service signal exposes *before* and *after* bind points. A *before* handler can inspect (and by convention influence) the arguments; an *after* handler receives the result. Because handlers run in-process around the service call, a raising handler can affect the operation — so bindings should be defensive and fast. This is a synchronous, in-process extension mechanism, **not** a message bus: there is no delivery guarantee across process boundaries, and long-running work should be handed to the APScheduler-based background tasks rather than blocking the service call. Confirm the exact contract in `openimis-be-core.py` (`core/service_signals.py` and the calculation module's bindings) before relying on subtle ordering.

Common mistake

Reaching for a Django `post_save` signal when you actually want to react to a **business operation**. If the logic belongs to "after a contribution is created as a business act," bind to the **service signal**. If it truly belongs to "any time this row is written, from anywhere including migrations and admin," use the ORM signal. Picking wrong leads to handlers firing during data loads and migrations when you never intended them to.

6. Permissions: integer rights and `has_perms`

openMIS does not use Django's string permission codenames for business authorization. It uses **integer rights codes**, defined per module in `apps.py` (often inside `DEFAULT_CFG`), and

checks them with `user.has_perms([...])`.

```
# illustrative – permission codes declared in a module's apps.py
DEFAULT_CFG = {
    "gql_query_claims_perms": [111001],          # view claims
    "gql_mutation_create_claims_perms": [111002], # create claims
    "gql_mutation_update_claims_perms": [111003],
}
```

```
# illustrative – enforcing them in a resolver / mutation
def resolve_claims(self, info, **kwargs):
    user = info.context.user
    if not user.has_perms(ClaimConfig.gql_query_claims_perms): # [111001]
        raise PermissionError("Not allowed to view claims")
    return Claim.objects.filter(...)
```

The integers are not arbitrary: they are grouped by module and function, and they map to **roles** carried by the openIMIS `User`. A user is an `InteractiveUser` (a human logging into the UI), a `TechnicalUser` (a service account), or is tied to an `Officer / i_user`; roles aggregate the integer rights that `user.has_perms` checks. The same integer codes also gate routes in the React frontend, so the permission model is consistent end to end. The full identity and authentication story — JWT in an `HttpOnly` cookie, OpenID Connect / OAuth2 for external IdPs — is covered in [Security](#).

Did you know?

The integer rights scheme is inherited from the IMIS era, where permissions were numeric. Keeping the numbers made migrating existing role definitions straightforward. It is a small but telling example of the platform's rule: *respect the legacy where breaking it buys nothing*.

7. Middleware, logging, and caching

These are configured in `openimis/settings.py` like any Django project, with a few openIMIS-specific notes.

- **Middleware.** The stack includes standard Django middleware plus what the GraphQL/JWT layer needs. `graphql_jwt` participates in turning the JWT cookie into `context.user` for every `/graphql` request. Because the token lives in an **HttpOnly cookie**, the authentication middleware reads it server-side; the SPA never touches it. See [Request Lifecycle](#).
- **Logging.** Standard Django `LOGGING` dict-config. Because openIMIS is assembled from many modules, prefer per-module loggers (`logging.getLogger(__name__)`) so you can

raise or lower verbosity for a single module (e.g. turn `claim` to `DEBUG`) without drowning in the rest.

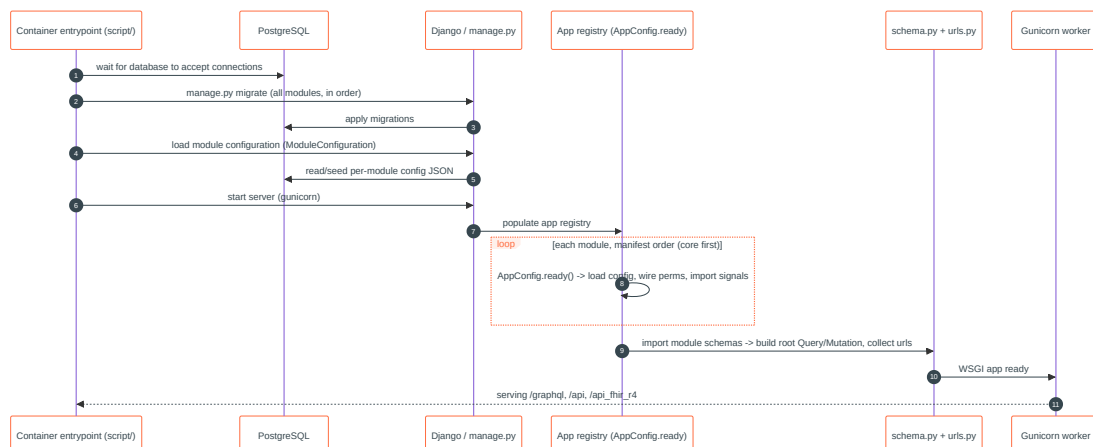
- **Caching.** Django's cache framework is available; modules use it for reference data and computed lookups. Remember the config-overlay timing from §4: cached config and effective config are both established around startup, so cache invalidation strategy matters when data changes at runtime.

Tip

When something behaves differently between two deployments running the *same* image, the difference is almost always in one of three places: the `openimis.json` module set, the `ModuleConfiguration` DB rows, or environment variables consumed by `settings.py`. Diff those three before suspecting the code.

8. The startup lifecycle

Now put it together as it actually happens when a container boots. The reference `openimis-dist_dkr` deployment starts `db` first, then `backend` (which runs migrations and loads module configs), then `frontend / gateway`. Inside the backend container, the `script/` entrypoints drive the sequence.



Step by step:

1. **Wait for the database.** The entrypoint blocks until PostgreSQL is reachable — the compose start order (`db` → `backend`) plus a wait loop.
2. **Migrate.** `manage.py migrate` applies every module's migrations. Because `INSTALLED_APPS` is built in manifest order with `core` first, dependent schemas are created after the tables they reference.

3. **Load configuration.** Module configs are read/seeded into `ModuleConfiguration` so the overlay of §4 has something to merge.
4. **Start the server.** Gunicorn boots the WSGI app. Django populates the app registry, calling each module's `AppConfig.ready()` — which loads effective config, wires permissions, and imports signal handlers.
5. **Assemble schema and URLs.** `openimis/schema.py` fuses every module's `Query / Mutation`; `openimis/urls.py` collects routes and mounts `/graphql`.
6. **Serve.** The worker is now request-ready.

Common mistake

Starting the backend before migrations have completed against a fresh database, then being surprised by "relation does not exist" errors. Respect the lifecycle: `db` healthy → migrate → load config → serve. In Compose this is encoded in service start order and wait loops; if you run pieces by hand, reproduce that order yourself.

9. Management commands and the `script/` entrypoints

- `manage.py` is the ordinary Django CLI, now operating over the assembled project. `migrate`, `makemigrations`, `shell`, `createsuperuser`, and each module's custom commands all work as usual.
- **Module management commands.** Several modules ship custom commands (data loads, one-off maintenance, index rebuilds for `opensearch_reports`, ETL runs for `dhis2_etl`). Discover them with `manage.py help` — they appear grouped by their module.
- **`script/` entrypoints** are the container-facing wrappers that sequence "wait for DB → migrate → load config → start". They exist so the container has a single, deterministic boot procedure. When you need to run a maintenance task in a deployed environment, prefer the provided commands/entrypoints over ad-hoc `manage.py` invocations, so config loading and ordering stay consistent.

Did you know?

Because config loading is itself a step in the entrypoint, a brand-new deployment can go from empty database to serving with **no manual admin steps** — the entrypoint seeds enough to boot. That zero-touch boot is what makes the reference Docker stack feel turnkey.

Repository references

Repository	Directory / File	Why it matters
openimis-be_py	openimis.json	Module manifest; the source of <code>INSTALLED_APPS</code> order.
openimis-be_py	openimis/settings.py	Builds <code>INSTALLED_APPS</code> dynamically; Graphene + <code>graphql_jwt</code> config; logging/caching/middleware.
openimis-be_py	openimisconf/load_openimis_conf.py	Loads the module configuration consumed by settings.
openimis-be_py	openimis/schema.py , openimis/urls.py	Assemble the root schema and collect module routes.
openimis-be_py	script/	Startup entrypoints: wait-for-db, migrate, load config, serve.
openimis-be-core_py	core/apps.py	<code>CoreConfig</code> , <code>DEFAULT_CFG</code> , permission wiring — the template every module follows.
openimis-be-core_py	core/models.py	<code>ModuleConfiguration</code> , <code>User</code> , <code>HistoryModel</code> / <code>VersionedModel</code> base classes.
openimis-be-core_py	core/service_signals.py	<code>register_service_signal</code> / <code>bind_service_signal</code> — the service-signal seam.
openimis-be-<name>_py	<name>/apps.py , <name>/services.py , <name>/signals.py	Per-module config, business logic, and signal bindings.
openimis-be-calculation_py	calculation/	Consumer of service signals; anchors the <code>calcrule_*</code> framework.

Hands-on lab

Goal: see dynamic assembly, config overlay, and signals with your own eyes.

1. From a running backend container, open a Django shell: `manage.py shell`.
2. Print the assembled app list: `from django.conf import settings;`
`print([a for a in settings.INSTALLED_APPS if not a.startswith("django")])`.
Confirm the modules match `openimis.json`, with `core` early in the list.
3. Inspect a module's effective config: import its `AppConfig` and print its loaded config object.
Compare it against the module's `DEFAULT_CFG` in `apps.py`. Any differences are coming from a `ModuleConfiguration` DB row.
4. Find one `register_service_signal` usage and one `bind_service_signal` usage across two different modules (start in `calculation / calcrule_*` binding onto `contribution` or `claim`). Draw the arrow: which module *owns* the signal, which module *hooks* it, and note that the owner does not import the hooker.
5. Find one integer permission code in a module's `apps.py` and trace it into a resolver's `user.has_perms([...])` call.

Deliverable: a one-page note answering, for one module of your choice: what does it install, what is its effective config vs default, which service signals does it own or bind, and which integer rights guard its main query.

Exercises

1. Explain why manifest order equals dependency order, and predict what fails if `claim` were listed before `core`.
2. You need a country-specific rule to run after every contribution is created. Which signal system do you use, and why not the other one? Sketch the `bind_service_signal` call.
3. Ops asks you to disable a feature flag in production without a code release. Where do you change it, and when does the change take effect?
4. Given the permission code `111002` for "create claims," describe the full path by which a logged-in user with the wrong role is stopped from creating a claim, from React route guard to resolver check.

Knowledge check

? Q1: How does openIMIS decide what goes into `INSTALLED_APPS`? (click for answer) ✓

`settings.py` does **not** hard-code it. It reads the loaded module configuration (from `openimis.json` via `openimisconf/load_openimis_conf.py`) and builds `INSTALLED_APPS` dynamically, appending each enabled module in manifest order (with `core` first so dependents initialize after it). Adding a module to the manifest is what enables it — you never hand-edit the app list.

? Q2: What is the relationship between `DEFAULT_CFG` and `ModuleConfiguration`? (click for answer) ✓

`DEFAULT_CFG` (a dict in the module's `apps.py`) holds built-in default configuration. At startup (in `AppConfig.ready()`), `core` overlays it with a per-module JSON document stored in the DB via the `ModuleConfiguration` model. DB values win over defaults; absent keys fall through to the default. This lets operators reconfigure a deployment without changing code — the change takes effect on the next worker restart.

? Q3: When do you use a service signal instead of a Django ORM signal? (click for answer) ✓

Use a **service signal** (`register_service_signal` / `bind_service_signal`) to react to a **business operation** — e.g. "after the `contribution.create` service runs" — especially to hook another module without importing it. Use a **Django ORM signal** (`post_save`, etc.) to react to raw persistence — "any time this row is saved, from anywhere." The service signal is the openIMIS cross-module extension seam; the ORM signal is generic and also fires during migrations, admin, and data loads.

? Q4: Why are permissions integers, and how are they checked? (click for answer) ✓

Permissions are **integer rights codes** (e.g. `111001`) declared per module in `apps.py`, inherited from the IMIS legacy where permissions were numeric. They are checked with `user.has_perms([codes])` in resolvers and mutations, and the same codes gate routes in the React frontend. Users carry rights via roles; the user may be an `InteractiveUser`, `TechnicalUser`, or tied to an `Officer`.

? Q5: What are the ordered steps of backend startup? (click for answer)



(1) Wait for PostgreSQL to be reachable; (2) run `manage.py migrate` across all modules in manifest order; (3) load/seed module configuration into `ModuleConfiguration`; (4) start Gunicorn, which populates the app registry and calls each `AppConfig.ready()` (load effective config, wire permissions, import signal handlers); (5) assemble the root schema and collect URLs; (6) serve `/graphql`, `/api`, `/api_fhir_r4`. The `script/` entrypoints sequence steps 1–4.

Further reading

- [High-Level Architecture](#) — where this backend sits in the whole stack.
- [Plugin / Module System](#) — service signals and the frontend contribution seam in depth.
- [Request Lifecycle](#) — how a request becomes `context.user` and flows through resolvers and the async mutation engine.
- [Repository Map](#) — the module dependency ordering reflected in the manifest.
- [Security](#) — JWT-in-cookie, OpenID Connect/OAuth2, and the identity model behind integer rights.
- [Calculation Rules](#) — the biggest real-world user of service signals.
- The [openIMIS GitHub org](#) to read the real `core/apps.py` and `settings.py`.

Plugin / Module Architecture

The modular plugin system is the single most important idea in openIMIS. Nearly every other design decision — the dynamic settings, the assembled GraphQL schema, the integer permission codes, the service signals — exists to make **independently versioned modules** compose into one running application. If you understand this chapter, the rest of the backend stops looking like magic.

This is PART 5 of the architecture track and it is deliberately long. Read it with a checkout of the assembly repo (`openimis-be_py`) and the core module (`openimis-be-core_py`) open beside you.

Learning objectives

By the end of this chapter you will be able to:

- Explain how openIMIS turns a JSON **manifest** into a set of installed Django apps at container start.
- Describe how a module **registers itself** through an `AppConfig` subclass, its `DEFAULT_CFG` , and the `ready()` hook.
- Trace how ~47 independent modules combine their GraphQL `Query` and `Mutation` classes into one root schema using Python **multiple inheritance**.
- Add integer **permission rights** in a module's `apps.py` and understand where they are enforced.
- Register module **URLs** and understand how they are collected centrally.
- Choose between **Django ORM signals** and openIMIS **service signals**, and wire a before/after hook across modules **without a hard import**.
- Explain the DB-backed `ModuleConfiguration` overlay over `DEFAULT_CFG` .
- Install a brand-new module end to end.

Prerequisites

- [Backend Deep Dive](#) — Django project layout, the core module's base models and helpers.
- [Architecture Overview](#) — the macro picture of backend, frontend, gateway.
- [GraphQL in openIMIS](#) — how Graphene builds a schema (this chapter assembles what that chapter defines).

- **Core module** — `AppConfig` conventions, `OpenIMISMutation`, service signals live here.
 - Comfort with Python packaging (`pip install -e`), Django `AppConfig`, and Django migrations.
-

1. Why a plugin architecture at all?

openIMIS began life as **IMIS**, a monolithic .NET + Microsoft SQL Server application. A single codebase served every country, so any national requirement — a different premium formula, an extra beneficiary attribute, a local payment gateway — meant patching the shared core or maintaining a private fork. Forks drift. Drifted forks cannot receive upstream security fixes. For a **Digital Public Good** deployed across many countries, that is an existential problem.

The 2018–2019 re-architecture answered it with a plugin model borrowed from the Django ecosystem and pushed much further:

- The deployable unit is an **assembly project** (`openimis-be_py`) that contains almost no business logic. It is a thin Django project whose job is to *discover, install, and wire together* modules.
- Each **module** is a separately versioned Git repository and pip package (`openimis-be-<name>_py`, importable as `<name>`). A country deployment picks a set of module versions the way you pin dependencies in `requirements.txt`.
- Modules extend each other through **named seams** — schema inheritance, permission codes, service signals, configuration overlays — rather than by importing each other's internals.

The payoff: a country can ship a `calcrule_*` module implementing its own premium math, or a payment-gateway module for its local mobile-money provider, **without touching core** and without forking. Upstream core keeps flowing in.

Did you know?

The assembly project is intentionally "dumb." Open `openimis-be_py` and you will find manifest files, a settings module that builds `INSTALLED_APPS` from a list, a `schema.py` that stitches modules together, and startup scripts — but the actual domain code (insurees, claims, policies) lives entirely in the module repositories. The project is the *frame*; modules are the *pictures*.

2. The boot sequence: from manifest to running apps

At the highest level, four artifacts drive module loading. Follow them in order.

Step	Artifact	Repo / file	Role
1	Module manifest	<code>openimis-be_py</code> → <code>openimis.json</code>	Declares which modules (and pip/git sources) make up this deployment.
2	Requirements file	generated <code>modules-requirements.txt</code>	A script converts the manifest into pip-installable lines.
3	Installed packages	Python environment	<code>pip install -r modules-requirements.txt</code> puts each <code><name></code> package on the path.
4	Dynamic app list	<code>openimis/settings.py</code> → <code>INSTALLED_APPS</code>	Settings reads the same module list and appends each module's app.

2.1 The manifest — `openimis.json`

`openimis.json` is the source of truth for *what this deployment is made of*. It lists every module and where to get it. Simplified and illustrative:

```
{
  "modules": [
    { "name": "core",      "pip": "openimis-be-core" },
    { "name": "location", "pip": "openimis-be-location" },
    { "name": "insuree",   "pip": "openimis-be-insuree" },
    { "name": "policy",    "pip": "openimis-be-policy" },
    { "name": "claim",     "pip": "openimis-be-claim" },
    { "name": "calculation", "pip": "openimis-be-calculation" }
  ]
}
```

(illustrative — see the real, ~47-entry `openimis-be_py/openimis.json` for the exact schema, which also carries version pins and git sources.)

Order matters: `core` is listed and loaded first because every other module depends on it. Reference-data modules (`location` , `medical` , `product`) come before the modules that use them (`insuree` , `policy` , `claim`).

2.2 Manifest → `modules-requirements.txt` → `pip`

A helper script reads `openimis.json` and emits a plain `modules-requirements.txt` — one pip requirement per module — which the container image installs. Conceptually:

```
1 python modules-requirements.py openimis.json > modules-requirements.txt
2 pip install -r modules-requirements.txt
```

- 1 The script walks the manifest's `modules` array and turns each entry into a pip line (a PyPI version or a `git+https://...@<ref> URL`).
- 2 After this, each module's package (`core`, `claim`, ...) is importable. In production these are pinned releases; nothing is installed that the manifest did not name.

For **local development** you swap the pinned installs for **editable installs** so your working copy is the running code:

```
pip install -e ../openimis-be-core_py/
pip install -e ../openimis-be-claim_py/
```

Now editing `../openimis-be-claim_py/claim/services.py` changes the running app on the next reload — no reinstall.

Common mistake

Adding a module to `openimis.json` is **not** enough on its own, and neither is `pip install`-ing a package that the manifest never mentions. The manifest is what `settings.py` reads to build `INSTALLED_APPS`. If the module is installed but not in the loaded module list, Django never registers its app — no models, no migrations, no schema, no URLs. Keep the manifest and the environment in sync.

2.3 The manifest → `INSTALLED_APPS`

This is the hinge of the whole system. Instead of a hand-written `INSTALLED_APPS`, `openimis/settings.py` loads the module list (via `openimisconf/load_openimis_conf.py`) and **appends each module's app config** to the standard Django + third-party apps. Simplified:

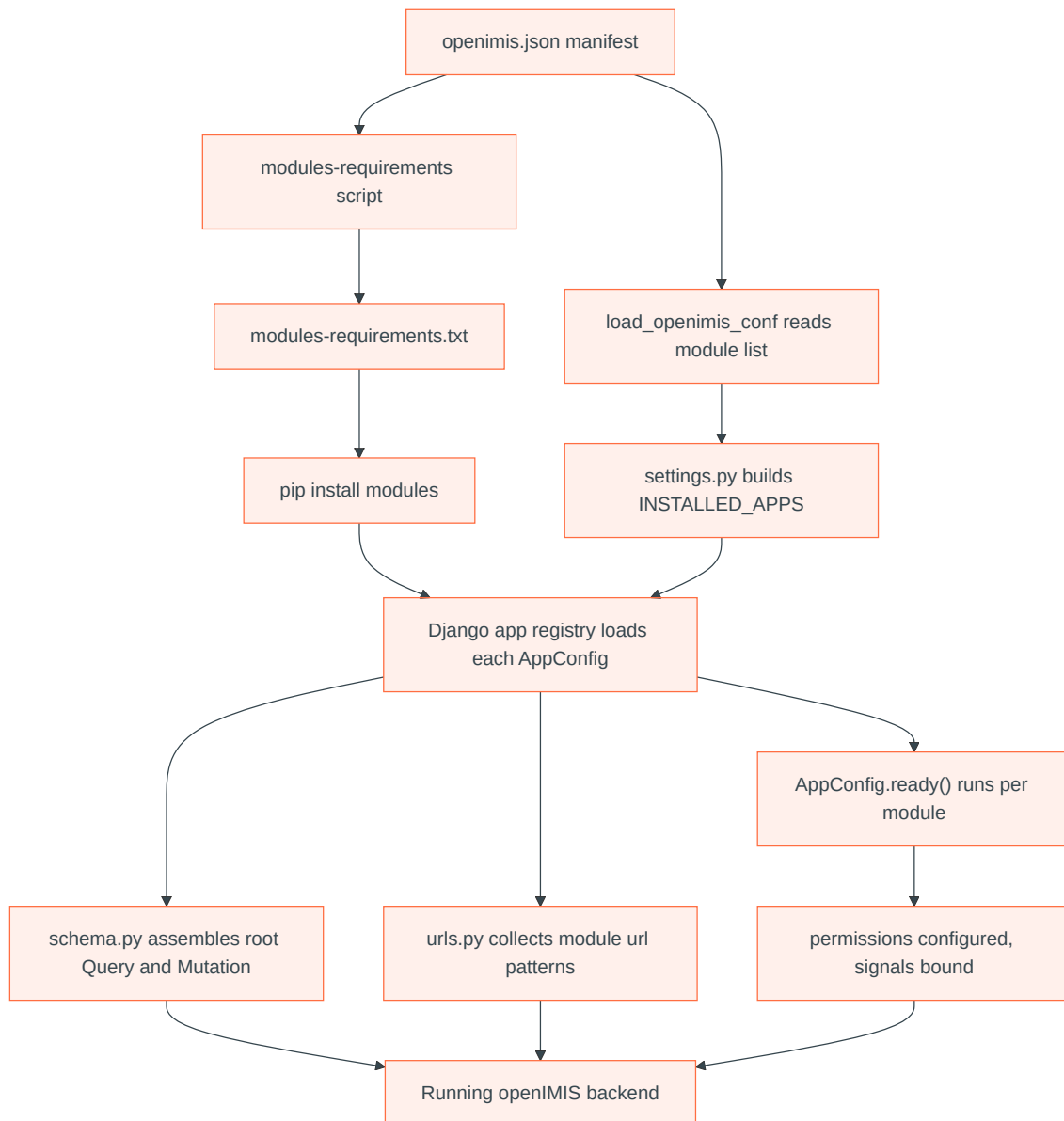
```
# openimis/settings.py (simplified / illustrative)
from openimisconf import load_openimis_conf

OPENIMIS_CONF = load_openimis_conf() ❶

INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    # ... standard Django ...
    "graphene_django", ❷
    "rest_framework",
    "rules",
]

❸
for module in OPENIMIS_CONF["modules"]:
    INSTALLED_APPS.append(module["name"])
```

- ❶ `load_openimis_conf()` reads the manifest / merged configuration so settings and the schema assembler agree on one ordered module list.
- ❷ Graphene, DRF, and `rules` are configured here too — Graphene points at the assembled root schema in `openimis/schema.py`.
- ❸ Because this is ordinary Python running at import time, the app list is truly **dynamic**: change the manifest, restart, and the set of installed apps changes. No core code edit required.



3. How a module registers itself: the `AppConfig`

Once a module's package is installed **and** its name is in `INSTALLED_APPS`, Django imports the module's `apps.py` and instantiates its `AppConfig` subclass. This is the module's front door. By openIMIS convention every module ships one, named `<Name>Config`, carrying three things:

1. A `DEFAULT_CFG` dictionary — the module's built-in configuration defaults.
2. Attributes that hold **permission rights** (integer codes), populated from config.
3. A `ready()` method — the lifecycle hook where the module wires itself in: applies its runtime configuration, publishes its permissions, and binds signals.

Here is a faithful-but-minimal `apps.py`:

```

# mymodule/apps.py (illustrative / simplified)
from django.apps import AppConfig

MODULE_NAME = "mymodule"

DEFAULT_CFG = {
    ❶ "gql_query_things_perms": [160001],
    "gql_mutation_create_thing_perms": [160002],
    "gql_mutation_update_thing_perms": [160003],
    ❷ "default_validity_days": 365,
}

class MyModuleConfig(AppConfig):
    name = MODULE_NAME
    default_auto_field = "django.db.models.BigAutoField"

    # Attributes filled from configuration at startup.
    gql_query_things_perms = []
    gql_mutation_create_thing_perms = []
    gql_mutation_update_thing_perms = []
    default_validity_days = 365

    def _configure_permissions(self, cfg):
        ❸ MyModuleConfig.gql_query_things_perms = cfg["gql_query_things_perms"]
        MyModuleConfig.gql_mutation_create_thing_perms = \
            cfg["gql_mutation_create_thing_perms"]
        MyModuleConfig.gql_mutation_update_thing_perms = \
            cfg["gql_mutation_update_thing_perms"]

    def ready(self):
        ❹ from core.models import ModuleConfiguration
        cfg = ModuleConfiguration.get_or_default(MODULE_NAME, DEFAULT_CFG)
        self._configure_permissions(cfg)
        self.default_validity_days = cfg["default_validity_days"]
        # Bind signals here (see section 7) so they register exactly once.
        from . import signals # noqa: F401

```

- ❶ Permissions are plain integers grouped by the operation they guard. openIMIS uses numeric **rights**, not Django's string `app_label.codename` permissions.
- ❷ `DEFAULT_CFG` also holds non-permission behaviour so a deployment can retune the module from the database without a code change.
- ❸ `name` must equal the importable package name — the same string that appears in the manifest and in `INSTALLED_APPS`.
- ❹ `_configure_permissions` copies the (possibly DB-overridden) values onto the config class so the rest of the module reads them as `MyModuleConfig.gql_query_things_perms`.
- ❺ `ready()` runs **once**, after the app registry is populated. It is the safe place to import models and bind signals — never do that at module top level, or you risk `AppRegistryNotReady`.

Did you know?

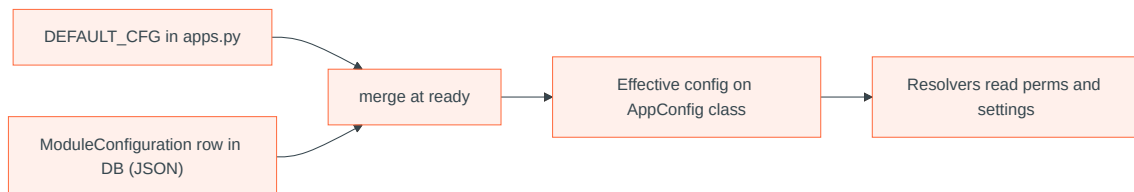
Because permission attributes live on the `AppConfig` class, resolvers read them as `MyModuleConfig.gql_query_things_perms`. That indirection is what lets an operator remap which right guards an operation from the database — the code never hard-codes the integer at the call site, it reads the class attribute that `ready()` populated.

The real core `AppConfig` (`class CoreConfig(AppConfig)` in `openimis-be-core_py/core/apps.py`) does exactly this at a larger scale, and it is the best reference to copy from.

4. Configuration: `DEFAULT_CFG` overlaid by `ModuleConfiguration`

openIMIS separates **code defaults** from **deployment configuration**:

- `DEFAULT_CFG` in a module's `apps.py` is the fallback baked into the code.
- `ModuleConfiguration` (a model in **core**) stores a per-module JSON blob in the database. At startup, `ready()` fetches it and **overlays** it on `DEFAULT_CFG` — DB values win, missing keys fall back to the default.



This is why a country can change which integer right guards "create claim," or change a default validity period, or toggle a feature — by editing a database row (loaded at container start from JSON config files under `openimisconf/module config`), **not** by patching Python. It is the operational half of the plugin promise: not just "add modules without forking," but "reconfigure modules without redeploying code."

See [Configuration](#) for the full config-loading story and where the JSON lives.

5. Assembling the GraphQL schema by multiple inheritance

Each module defines its own slice of the API in its `schema.py`: a `Query` class (read fields + resolvers) and often a `Mutation` class (write operations). None of them is the *whole* API. The

assembly project's `openimis/schema.py` combines them into **one root Query** and **one root Mutation** using Python multiple inheritance, then hands that root to Graphene.

Think of each module's `Query` as a **mixin** contributing a few fields. The root `Query` inherits from all of them, so it exposes the **union** of every module's fields. Simplified:

```
# openimis/schema.py (simplified / illustrative)
import graphene
from core import schema as core_schema
from location import schema as location_schema
from insuree import schema as insuree_schema
from claim import schema as claim_schema

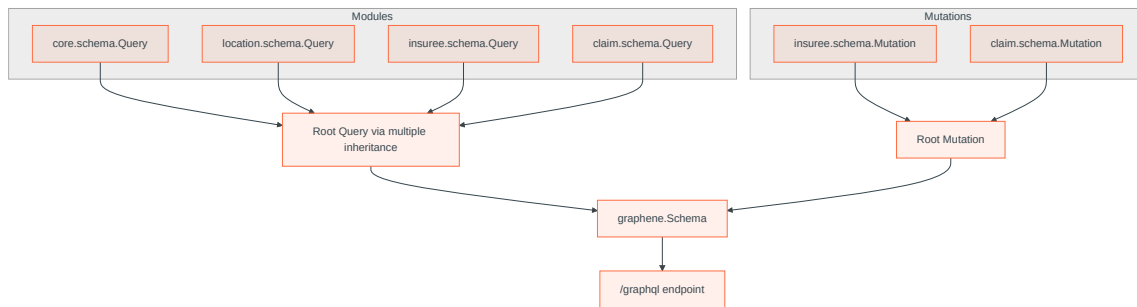
class Query(
    core_schema.Query,           ❶
    location_schema.Query,
    insuree_schema.Query,
    claim_schema.Query,
    graphene.ObjectType,        ❷
):
    pass

class Mutation(
    core_schema.Mutation,       ❸
    insuree_schema.Mutation,
    claim_schema.Mutation,
    graphene.ObjectType,
):
    pass

schema = graphene.Schema(query=Query, mutation=Mutation) ❹
```

- ❶ Each module's `Query` mixin brings its own fields (e.g. `claim_schema.Query` contributes a `claims` field and its `resolve_claims`).
- ❷ `graphene.ObjectType` must be **last** in the MRO so the mixins' fields are collected before Graphene's metaclass finalizes the type.
- ❸ The same pattern for mutations: the root `Mutation` inherits every module's write operations.
- ❹ Graphene walks the combined class, gathers all inherited fields, and builds a single executable schema exposed at `/graphql`.

In the real assembly this list is longer and is often built by iterating the loaded module list rather than by a hand-written class — but the mechanism is the same: **many module mixins** → **one root type**.



🔗 Deep dive: how multiple inheritance actually merges the fields

Graphene declares fields as **class attributes**. When Python builds the root `Query`, its namespace is the union of the base classes' namespaces resolved along the **Method Resolution Order (MRO)**. Graphene's metaclass then scans the resulting class for anything that is a `Field` (or a `resolve_*` method) and registers it on the GraphQL type.

Two consequences follow directly:

1. **`graphene.ObjectType` goes last.** It carries the metaclass machinery that finalizes the type. If it appeared first, MRO would let its (empty) field set shadow the mixins and you would get a schema with no fields. Put the domain mixins first, `ObjectType` last.
2. **Field-name collisions are silent — and dangerous.** If two modules both declare a field literally named `things`, the one earlier in the base list wins by MRO; the other is shadowed with no error. This is why openIMIS modules **namespace their fields** by domain (`claimsByStatus`, `insureeFamily`, ...) rather than using generic names. Two modules resolving different data under the same field name is one of the nastiest bugs in this architecture because nothing raises — the API just quietly returns the wrong resolver's data.

A resolver lives with its field on the mixin (`resolve_claims` sits on `claim.schema.Query`), so behaviour travels with the field into the root type. A resolver added in a module is automatically part of the combined schema the moment that module joins the base list.

To learn the resolver, argument, and connection conventions these mixins follow, read [GraphQL in openIMIS](#).

6. Permissions: integer rights defined per module

openIMIS does **not** use Django's string permission codenames for its business authorization. Instead each right is an **integer**, defined in the owning module's `apps.py` (as shown in section 3), grouped by the operation it guards:

```
# in some module's DEFAULT_CFG (illustrative)
"gql_query_claims_perms": [111001],
"gql_mutation_create_claims_perms": [111002],
"gql_mutation_update_claims_perms": [111003],
```

At startup `_configure_permissions()` copies these onto the `AppConfig` class. Resolvers and mutations then enforce them against the logged-in user's rights:

```
# in a resolver or mutation (illustrative)
from claim.apps import ClaimConfig

def resolve_claims(self, info, **kwargs):
    user = info.context.user
    if not user.has_perms(ClaimConfig.gql_query_claims_perms): ①
        raise PermissionError("Not authorized to read claims")
    return Claim.objects.filter(...)
```

- ① `has_perms` takes the **list of integer rights** the operation requires. The openIMIS `User` gathers rights from the roles attached to its `InteractiveUser`. If the user's rights don't include the required code(s), the operation is refused before any data is touched.

Why integers? They are stable across languages and UIs, compact to store on roles, and — because the *mapping* from operation to right lives in configuration — an operator can re-map which right guards which operation without a code change. The same integer rights guard the **frontend** routes and menus (the React side checks the user's rights too), so backend and frontend agree on one authorization vocabulary.

Enforcement, the `User / InteractiveUser` /role model, and JWT are covered in [Security](#).

Common mistake

Do not scatter literal integers like `111002` through your resolvers. Always reference them through the config class (`ClaimConfig.gql_mutation_create_claims_perms`). Hard-coding the integer defeats the configuration overlay and makes the operation impossible to re-map from the database.

7. URL registration

REST-style and legacy endpoints (reports, FHIR, callbacks) still need Django URL routing. Each module that exposes them ships a `urls.py` with a `urlpatterns` list, and the assembly project's `openimis/urls.py` **collects** them — typically by iterating the loaded modules and including each one's patterns under a prefix — alongside the single `/graphql` endpoint. Simplified:


```
# openimis/urls.py (simplified / illustrative)
from django.urls import path, include
from graphene_django.views import GraphQLView
from openimisconf import load_openimis_conf

urlpatterns = [
    path("graphql", GraphQLView.as_view(graphiql=True)), ❶
]

for module in load_openimis_conf()["modules"]: ❷
    try:
        urlpatterns.append(
            path(f"api/{module['name']}/", include(f"{module['name']}.urls"))
        )
    except ModuleNotFoundError:
        pass ❸
```

- ❶ There is **one** GraphQL endpoint for the whole application — the assembled schema from section 5 serves every module's fields through it.
- ❷ The same ordered module list drives URL collection, so a module's REST routes appear automatically once it is in the manifest.
- ❸ A module without a `urls.py` is simply skipped — many modules are GraphQL-only and ship no REST routes at all.

The FHIR module (`openimis-be-api_fhir_r4_py`) is the clearest example: its `urls.py` mounts the `/api_fhir_r4/` REST surface that maps openIMIS models to FHIR resources. See [FHIR R4 API](#).

8. Signals: the loose-coupling extension seam

Modules must react to each other's actions *without importing each other's internals* — otherwise you rebuild the monolith's tangle of dependencies. openIMIS gives you two signal mechanisms; pick by what you are reacting to.

Mechanism	Reacts to	Defined by	Use when
Django ORM signals	Model persistence events (<code>post_save</code> , <code>pre_delete</code> , ...)	Django	You care that a <i>row</i> changed.
openIMIS service signals	A <i>service method</i> call, with before/after hooks	core (<code>register_service_signal</code> / <code>bind_service_signal</code>)	You care that a <i>business operation</i> ran, and want its arguments/result.

8.1 Django ORM signals

Standard Django. A module connects a receiver to another module's model without importing that module's services:

```
# mymodule/signals.py (illustrative)
from django.db.models.signals import post_save
from django.dispatch import receiver
from claim.models import Claim

@receiver(post_save, sender=Claim)
def on_claim_saved(sender, instance, created, **kwargs):
    if created:
        # react to a new claim row — e.g. queue a notification
        ...
```

Good for "a row appeared/changed." But ORM signals only see the **database event** — not the business intent, the caller's arguments, or the service's return value. For that, use service signals.

8.2 openIMIS service signals — the extension seam

The **service signal** framework (in core) lets a module wrap **before** and **after** hooks around another module's *service method* — the unit of business logic — without a hard import. The service that wants to be extensible registers a signal; any module can bind to it.

```
# claim/services.py (illustrative – the extensible service)
from core.signals import register_service_signal

class ClaimService:
    @register_service_signal("claim_service.submit") ❶
    def submit(self, claim):
        # ... core submission logic ...
        return result
```

- ❶ `register_service_signal("claim_service.submit")` publishes a named seam. openIMIS fires a **before** signal (with the call arguments) and an **after** signal (with the result) around every call to `submit`.

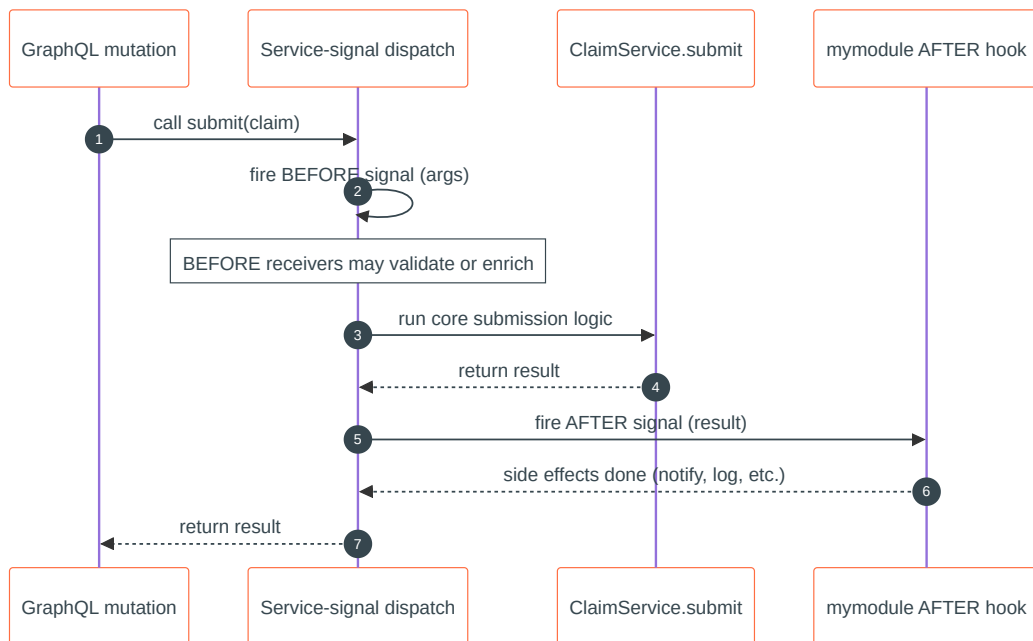
Now a *different* module hooks in — again, no import of `ClaimService` internals:

```
# mymodule/signals.py (illustrative – the extending module)
from core.service_signals import bind_service_signal
from core.signals import ServiceSignalBindType

def _after_claim_submit(sender, result=None, **kwargs): ❷
    # runs after every ClaimService.submit, sees the result
    notify_external_system(result)

def bind_signals():
    bind_service_signal(
        "claim_service.submit", ❸
        _after_claim_submit,
        bind_type=ServiceSignalBindType.AFTER, ❹
    )
```

- ❶ The receiver gets the service result (for AFTER) or the incoming arguments (for BEFORE) — the business context ORM signals cannot give you.
- ❷ Binding is **by name string**, so `mymodule` never imports `claim`'s service class. The two modules are coupled only through the signal name — the essence of the plugin seam.
- ❸ BEFORE hooks can validate or enrich inputs; AFTER hooks react to results. Call `bind_signals()` from your `AppConfig.ready()` (section 3) so it registers exactly once at startup.



i Did you know?

The **calculation-rule** framework is built entirely on this seam. That is how a country can inject its own premium math into contribution and claim valuation without core knowing the formula exists — see the deep dive below.

Deep dive: the calculation-rule plugin pattern

Pricing and valuation vary by country: how a premium is computed from a product, how a claim is valued, how capitation or third-party payment is calculated. Hard-coding any one formula in core would force every deployment to share it. So `openimis-be-calculation_py` defines a **rule registry**, and each concrete rule ships as its own `calcrule_*` module (`calcrule_contribution`, `calcrule_capitation_payment`, ...).

The pattern, at a glance:

1. **Registration by signal.** On startup each `calcrule_*` module registers its rule class with the calculation module through a signal — no direct import of the calculation internals, and no edit to core.
2. **Capability check.** When core needs a value, it asks the registry "which rules apply to *this* object in *this* context?" Each rule answers whether it is applicable (by product configuration, class of object, etc.).
3. **Dispatch.** The registry calls the winning rule's `calculate(...)`; the rule returns the computed value, which core stores against the contribution, claim, or payment.

```
# calcrule_myrule/calculation_rule.py (illustrative)
class MyContributionRule:
    @classmethod
    def check_calculation(cls, instance):
        # is this rule applicable to `instance`?
        return getattr(instance, "product_code", None) == "MY_PLAN"

    @classmethod
    def calculate(cls, instance, **kwargs):
        return instance.income * 0.05 # a country-specific premium formula
```

The lesson generalizes: **a rule is a plugin registered by signal, selected by a capability check, and dispatched by name.** Whenever you need country-specific behaviour to vary at a fixed point in a core workflow, reach for this pattern rather than a conditional in core. See [Calculation Rules](#).

9. Migrations per module

Each module owns its own `migrations/` directory and its own schema. openIMIS does not maintain one central migrations tree; instead every module migrates independently, exactly as any reusable Django app does. Consequences:

- A module's tables are created/updated by **its** migrations, run at container start (the startup scripts under `openimis-be_py/script/` run `migrate` before serving traffic).

- **Ordering** across modules is expressed with Django's `dependencies = [("core", "0001_initial"), ...]`. A module whose model has a foreign key into `location` declares a dependency on the relevant `location` migration so Django applies them in a valid order.
- Adding a module to the manifest means its migrations join the run automatically on the next start — no central registry to edit.

Because of the legacy MSSQL heritage, many migrations create tables with `tbl` prefixes and camelCase columns, and add UUID keys alongside legacy integer keys. That is expected; see [Backend Deep Dive](#) and [Database](#) for the temporal/versioned model conventions (`validity_from` / `validity_to`, `legacy_id`, `json_ext`).

⚠ Common mistake

Cross-module foreign keys without a declared migration `dependencies` entry lead to intermittent "relation does not exist" errors that only appear on a **fresh** database (where apply order matters) and never on your already-built dev DB. Always declare the dependency on the target module's migration.

10. Putting it together: install a new module end to end

Here is the full checklist to add a hypothetical `mymodule` to a deployment.

Scaffold the standard layout in a new repo `openimis-be-mymodule_py`:

```
openimis-be-mymodule_py/
  mymodule/
    __init__.py
    apps.py          # MyModuleConfig, DEFAULT_CFG, ready()
    models.py
    schema.py        # Query / Mutation mixins
    services.py
    signals.py       # ORM + service-signal bindings
    urls.py          # optional REST routes
    migrations/
    tests/
    setup.py
```

Add an entry to `openimis-be_py/openimis.json`:

```
{ "name": "mymodule", "pip": "openimis-be-mymodule" }
```

Place it **after** the modules it depends on (at least after `core`).

```
# production: regenerate requirements + install
python modules-requirements.py openimis.json > modules-requirements.txt
pip install -r modules-requirements.txt

# development: editable install of your working copy
pip install -e ../openimis-be-mymodule_py/
```

- `apps.py`: define `DEFAULT_CFG` (with integer perms), `_configure_permissions`, and a `ready()` that overlays `ModuleConfiguration` and binds signals.
- `schema.py`: define `Query` / `Mutation` mixins (they join the root schema automatically because the module is in the loaded list).
- `urls.py`: only if you expose REST endpoints.

```
python manage.py makemigrations mymodule
python manage.py migrate
python manage.py runserver
```

Restart picks up the new `INSTALLED_APPS` entry, runs the module's migrations, merges its schema fields, collects its URLs, and binds its signals.

1. Create the package
2. Register in the manifest
3. Install
4. Wire the module
5. Migrate & run

The reward: your module's fields appear under the single `/graphql` endpoint, its permissions are enforceable and re-mappable, its services are extensible by other modules via signals, and **you changed no core code to get there.**

For a narrated, real-code build of a full feature across models → service → schema → frontend, follow [End-to-End Code Walkthrough](#) and [Extending openIMIS](#).

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be_py</code>	<code>openimis.json</code>	The module manifest — the source of truth for which modules make up a deployment.
<code>openimis-be_py</code>	<code>modules-requirements.txt</code> (generated)	pip-installable module list derived from the manifest.
<code>openimis-be_py</code>	<code>openimis/settings.py</code>	Builds <code>INSTALLED_APPS</code> dynamically from the loaded module list.
<code>openimis-be_py</code>	<code>openimis/schema.py</code>	Assembles the root GraphQL <code>Query / Mutation</code> from module mixins via multiple inheritance.
<code>openimis-be_py</code>	<code>openimis/urls.py</code>	Collects each module's <code>urls.py</code> and mounts the single <code>/graphql</code> endpoint.
<code>openimis-be_py</code>	<code>openimisconf/load_openimis_conf.py</code>	Loads the ordered module list / merged config used by settings and schema.
<code>openimis-be_py</code>	<code>script/</code>	Container-start entrypoints that run migrations and load module configs.
<code>openimis-be-core_py</code>	<code>core/apps.py</code>	Reference <code>AppConfig (CoreConfig)</code> with <code>DEFAULT_CFG</code> and <code>_configure_permissions</code> .
<code>openimis-be-core_py</code>	<code>core/models.py</code>	<code>ModuleConfiguration</code> , base temporal/versioned models.
<code>openimis-be-core_py</code>	<code>core/signals.py</code> , <code>core/service_signals.py</code>	<code>register_service_signal / bind_service_signal</code> — the extension seam.
<code>openimis-be-calculation_py</code>	<code>calculation/</code>	Rule registry underpinning the <code>calcrule_*</code> plugin pattern.

Repository	Directory / File	Why it matters
<code>openimis-be-claim_py</code>	<code>claim/apps.py</code> , <code>claim/schema.py</code> , <code>claim/services.py</code>	A full-featured module to copy conventions from.

Hands-on lab

Goal: trace one module from manifest to running fields, then extend it with a signal — all against a running dev backend.

1. **Find the module list.** Open `openimis-be_py/openimis.json` and list the modules in load order. Confirm `core` is first and note where `claim` sits relative to `insuree`, `policy`, `location`.
2. **Follow it into settings.** In `openimis/settings.py`, find the loop that appends module names to `INSTALLED_APPS`. Confirm the same list drives it.
3. **Find a field's origin.** In `openimis/schema.py`, identify which module's `Query` mixin contributes a claim-related field. Open that module's `schema.py` and read the matching `resolve_*`.
4. **Find its permission.** In that module's `apps.py`, find the integer right(s) the resolver checks via `has_perms`. Note the config key name.
5. **Add a service-signal hook.** In a scratch module (or a fork of an existing one), bind an `AFTER` hook to a core service signal (e.g. a claim submit service). Log the result. Restart, exercise the operation via `/graphql`, and confirm your hook fired — **without importing** the target service class.

Exercises

1. Your new module's GraphQL field does not appear at `/graphql`. List three distinct causes rooted in this chapter and how you'd check each.
2. Two modules accidentally define a `Query` field named `summaries`. Explain what the running schema returns and why no error is raised. Propose a naming rule to prevent it.
3. An operator wants "create claim" to require a different integer right than the default. Describe the change with **no** code deployment.
4. You need to run code after every insuree is created — but you also need the *service's* computed result, not just the row. Which signal mechanism do you choose, and where do you bind it?

Knowledge check

? **Q1: What single file decides which modules make up a deployment, and what two things consume it? (click for answer)** ✓

`openimis-be_py/openimis.json`, the module manifest. It is consumed (via `load_openimis_conf`) by `settings.py` to build `INSTALLED_APPS`, and it is turned into `modules-requirements.txt` for `pip install`. The same ordered list also drives schema assembly and URL collection.

? **Q2: Why must `graphene.ObjectType` appear last in the root Query's base classes? (click for answer)** ✓

Graphene collects fields along the MRO and its metaclass (carried by `ObjectType`) finalizes the type. If `ObjectType` came first, its empty field set would win by MRO and shadow the module mixins, producing a schema with no fields. Domain mixins first, `ObjectType` last.

? **Q3: How does one module hook into another module's business operation without importing it? (click for answer)** ✓

Through **service signals**. The owning service is decorated with `register_service_signal("name")`; the extending module calls `bind_service_signal("name", receiver, bind_type=BEFORE|AFTER)` from its `AppConfig.ready()`. Coupling is by the signal-name string only — no import of the service class.

? **Q4: What is the difference between `DEFAULT_CFG` and `ModuleConfiguration`? (click for answer)** ✓

`DEFAULT_CFG` (in a module's `apps.py`) is the code-level default. At startup `ready()` overlays the module's `ModuleConfiguration` DB row (JSON) on top — DB values win, missing keys fall back to the default. This lets operators reconfigure a deployment (including which integer right guards an operation) without changing code.

? Q5: Why are permission rights integers referenced through the config class rather than literals in resolvers? (click for answer) ✓

Integers are language- and UI-neutral, compact to store on roles, and shared with the frontend's route guards. Reading them through the `AppConfig` class attribute (populated by `_configure_permissions`) lets the operation-to-right mapping be re-configured from the database. Hard-coded literals would break that overlay.

Further reading

- openIMIS backend assembly repo: `openimis-be_py` (manifest, settings, schema, urls, startup scripts).
- Core module: `openimis-be-core_py` (`apps.py`, `models.py`, `signals.py`, `service_signals.py`).
- [GraphQL in openIMIS](#) — resolver and schema conventions the mixins follow.
- [Security](#) — how integer rights and JWT enforce the permissions defined here.
- [Calculation Rules](#) and `openimis-be-calculation_py` — the flagship signal-driven plugin pattern.
- [Configuration](#) — the `ModuleConfiguration` overlay and where JSON config lives.
- Django docs: application configuration (`AppConfig.ready()`), signals, and migration dependencies.
- Graphene-Django docs: schema, `ObjectType`, and field resolution.

Request Lifecycle

This chapter follows a single request all the way through openIMIS — from the browser, across the network, into Django, through GraphQL, down to the database, and back — and then does it again for a **mutation**, because writes in openIMIS work very differently from reads. When you finish, you should be able to put your finger on *exactly* where authentication happens, where authorization happens, where business logic lives, and why a mutation returns *before* its work is necessarily finished.

This is PART 8 of the architecture track. It assumes you already know how the pieces are assembled; here we make them *move*.

Learning objectives

By the end of this chapter you will be able to:

- Trace an HTTP request through the Nginx gateway, WSGI, Django middleware, authentication, the GraphQL view, the assembled schema, a resolver, the permission check, the service layer, the ORM, and back.
- Explain **where** JWT authentication happens and **why** it lives in an HttpOnly cookie.
- Distinguish the roles of **resolver**, **permission check**, **service**, and **ORM** — and why openIMIS keeps them separate.
- Describe the **asynchronous mutation pattern** (`OpenIMISMutation`): a `MutationLog` row, a returned `clientMutationId`, and client **polling**.
- Read both a **query** and a **mutation** sequence diagram fluently.

Prerequisites

- [Architecture Overview](#) — the gateway/backend/db topology.
 - [Backend Deep Dive](#) — Django project, services, base models.
 - [Plugin / Module System](#) — how the schema and resolvers you'll meet here got assembled.
 - [GraphQL in openIMIS](#) — queries, mutations, resolvers.
 - [Security](#) — JWT, the `User` model, integer rights.
-

1. The cast of characters

Before we trace anything, meet the layers a request passes through. Each has one job; the discipline of keeping them separate is what makes the codebase tractable.

Layer	What it is	Its one job	Where it lives
Gateway	Nginx reverse proxy	Terminate TLS, route <code>/</code> , <code>/api</code> , <code>/graphql</code> ; serve the built React app	<code>openimis-dist_dkr</code>
WSGI server	Gunicorn	Turn HTTP into Python <code>environ</code> ; run worker processes	backend container
Middleware	Django middleware stack	Cross-cutting concerns: sessions, CSRF, auth, JWT	<code>openimis/settings.py</code>
Authentication	<code>django-graphql-jwt</code>	Identify <i>who</i> the caller is from the JWT cookie	core + jwt middleware
GraphQL view	<code>GraphQLView</code> (graphene-django)	Parse the query, run it against the schema	<code>openimis/urls.py</code>
Schema / resolver	Assembled root <code>Query / Mutation</code>	Map a field to code that fetches/changes data	<code>openimis/schema.py</code> + module <code>schema.py</code>
Authorization	<code>user.has_permissions(...)</code>	Decide <i>whether</i> this user may do this	resolver / mutation
Service	Plain Python service class	Business logic, validation, orchestration	module <code>services.py</code>
ORM	Django ORM	Talk to PostgreSQL; temporal/versioned models	module <code>models.py</code>
Database	PostgreSQL	Persist state	<code>db</code> container

Did you know?

There is exactly **one** GraphQL endpoint — `/graphql` — for the entire application, no matter how many modules are installed. Every module's fields were merged into one root schema at boot (see [Plugin / Module System](#)). The client always talks to the same URL; the *schema* is what varies by deployment.

2. Tracing a query, step by step

Follow a read: the frontend asks for a page of claims. We walk each hop and, for each, say **what happens** and **why it is there**.

2.1 Browser → Gateway

The React SPA (Redux-based, **not** Apollo) issues an HTTP `POST` to `/graphql` with a JSON body containing the GraphQL query string and variables. Because the JWT sits in an **HttpOnly cookie**, the browser attaches it automatically — the JavaScript never reads or forwards a token by hand.

The request hits **Nginx**, the single public entry point. Nginx:

- terminates TLS,
- serves the static React bundle for `/`,
- **reverse-proxies** `/graphql` and `/api/*` to the Django backend (Gunicorn).

Why a gateway? It gives frontend, `/api`, and `/graphql` one origin (so the cookie is same-origin and CSRF/CORS stay simple), centralizes TLS, and lets you scale or relocate the backend without the browser knowing.

2.2 Gateway → WSGI (Gunicorn) → Django

Nginx forwards the proxied request to **Gunicorn**, which translates HTTP into a WSGI `environ` and hands it to a Django worker. Django wraps it in an `HttpRequest` and sends it **down the middleware stack**.

2.3 Middleware & authentication

Django runs each middleware in order. The security-relevant ones:

- Session and common middleware (housekeeping).
- **CSRF** protection.

- The **JWT authentication** middleware from `django-graphql-jwt`, plus openIMIS additions. It reads the JWT from the **HttpOnly cookie**, verifies its signature and expiry, and resolves it to the openIMIS **User**, which it attaches to `request.user`. If there is no valid token, `request.user` is anonymous — the request still proceeds; individual resolvers decide what an anonymous caller may see.

Why a cookie, not a header? An **HttpOnly** cookie is unreadable from JavaScript, which blunts token theft via XSS. The browser attaches it automatically, so the SPA never handles the raw token. (openIMIS also supports OpenID Connect / OAuth2 for external identity providers.) See [Security](#).



Common mistake

Authentication (**who are you**) is *not* authorization (**may you do this**). The JWT middleware only establishes identity and populates `request.user`. It performs **no** permission checks. Every resolver and mutation must still call `has_perms`. Assuming "the user got past auth, so they're allowed" is how data leaks.

2.4 The GraphQL view

The middleware stack delivers the request to `GraphQLView` (mounted at `/graphql` in `openimis/urls.py`). The view:

1. parses the query string and variables from the JSON body,
2. validates them against the **assembled schema** (`openimis/schema.py`),
3. builds an **execution context** whose `info.context` is the Django request — crucially carrying `info.context.user`,
4. executes the query, calling the resolver for each requested field.

If the query names a field no module contributed, validation fails here with a GraphQL error **before** any resolver runs.

2.5 Resolver → permission check

Graphene invokes the field's resolver — say `resolve_claims` on the `claim` module's `Query` mixin. **The first thing a well-behaved resolver does is authorize:**

```
# claim/schema.py (illustrative / simplified)
from claim.apps import ClaimConfig
from claim.models import Claim

class Query(graphene.ObjectType):
    claims = OrderedDjangoFilterConnectionField(ClaimGQLType)

    def resolve_claims(self, info, **kwargs):
        user = info.context.user
        if not user.has_perms(ClaimConfig.gql_query_claims_perms):
            raise PermissionError("Not authorized to read claims")
        return Claim.objects.filter(*filter_validity())
```

- ❶ `info.context` is the Django request, so `info.context.user` is exactly the `User` the JWT middleware attached in step 2.3.
- ❷ **Authorization** happens here, in the resolver — against the module's integer rights read from its `AppConfig` (see [Plugin / Module System](#)). No rights, no data — the resolver raises before touching the DB.
- ❸ For a simple read the resolver may build the queryset directly. `filter_validity()` restricts to currently-valid rows (the temporal `validity_from/validity_to` convention) — you rarely want expired history in a normal read.

2.6 Service layer, ORM, database

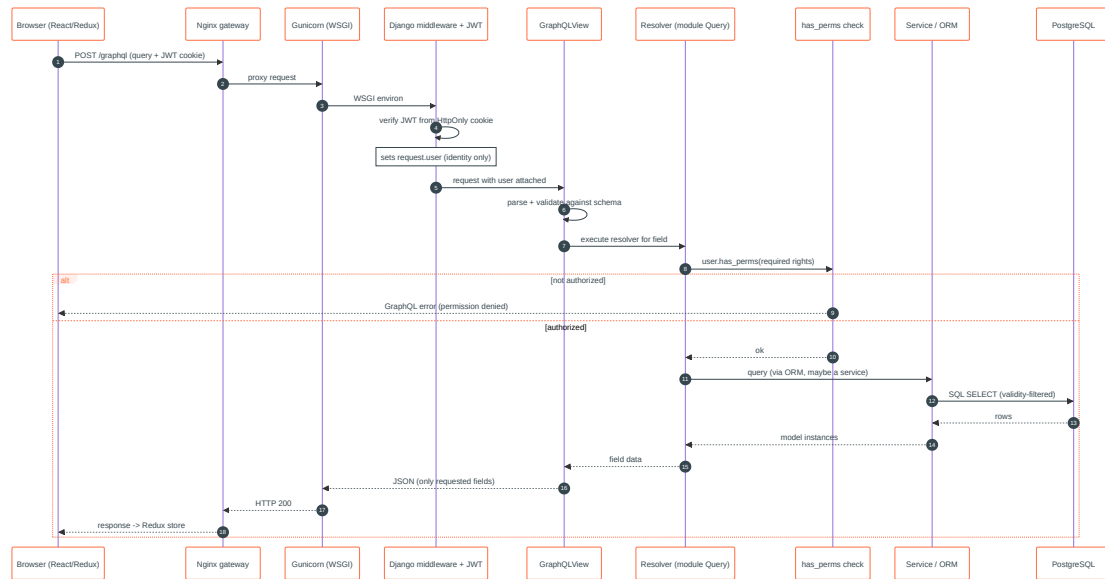
For a **read**, a resolver often queries the ORM directly (as above). For reads with real business logic — or any **write** — the resolver delegates to a **service** class in the module's `services.py`. The service is where validation, cross-model orchestration, and service-signal firing live; keeping that out of the resolver means the same logic is reachable from REST, tests, and other services, not just GraphQL.

The ORM turns the queryset into SQL against **PostgreSQL**. openIMIS models carry the legacy-derived temporal fields and `json_ext`, so a "row" is often a *version* of an entity valid over a time window (see [Backend Deep Dive](#) and [Database](#)).

2.7 Back up the stack

The rows become model instances, Graphene shapes them into the exact JSON the query asked for (Graphene returns **only** the fields requested — one of GraphQL's core wins over REST over-fetching), the view serializes the result, Gunicorn returns HTTP, and Nginx relays it to the browser, which reduces it into the Redux store.

2.8 The full query sequence



3. Tracing a mutation: the asynchronous pattern

Writes do not follow the query path. openIMIS mutations are built on the `OpenIMISMutation` base class (in core), which imposes an **asynchronous, audited** pattern. Understanding *why* is the point of this section.

3.1 Why writes are different

A create/update/delete in openIMIS is rarely a single INSERT. It can:

- validate against several modules' rules,
- fire **service signals** so other modules react (see [Plugin / Module System](#)),
- recompute values through **calculation rules**,
- and must be **audited** — every business change is attributable to a user.

To keep the API responsive and every change traceable, openIMIS models a mutation as a **job**: it records the request, kicks off the work, and hands the client a **ticket** to check on it. The client does not block waiting for all downstream effects.

3.2 The `OpenIMISMutation` flow

When a mutation resolver runs, `OpenIMISMutation`:

1. **Authorizes** — checks the caller's integer rights with `has_perms`, exactly like a query resolver, before doing anything.
2. **Creates a `MutationLog`** row (in core) capturing who, what, when, the input, and an initial status (e.g. *received/pending*). This is the audit record and the job's status holder.
3. **Runs the work** — delegates to the module's **service**, which validates, writes through the ORM, and fires service signals. As it progresses, the `MutationLog` status is updated (e.g. *success* or *failed*, with any error messages).
4. **Returns immediately** with a `clientMutationId` — the client-supplied correlation id echoed back — *without* necessarily waiting for every downstream effect to settle.
5. **The client polls.** Using the `clientMutationId`, the frontend queries the `MutationLog` status (the FE's "journalize"/polling helper does this) until it reads *success* or *failed*, then updates the UI.

```
# claim/gql_mutations.py (illustrative / simplified)
class CreateClaimMutation(OpenIMISMutation):
    _mutation_class = "CreateClaimMutation"

    class Input(OpenIMISMutation.Input):
        code = graphene.String(required=True)
        # ... claim fields ...

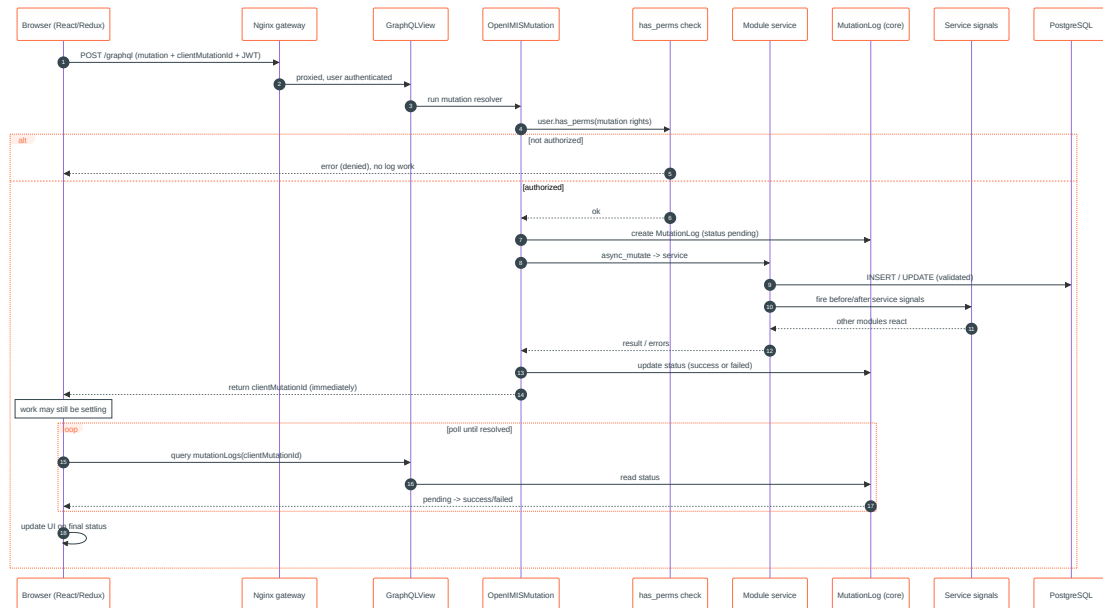
    @classmethod
    def async_mutate(cls, user, **data):
        if not user.has_perms(ClaimConfig.gql_mutation_create_claims_perms):
            raise PermissionDenied("Not authorized")
        errors = ClaimService(user).create(data)
        return errors
```

- 1 Subclasses implement `async_mutate`; the base class wraps it with the `MutationLog` bookkeeping — create the log, run this, record the outcome.
- 2 Authorization again lives right here, against the module's integer mutation rights.
- 3 Real work is delegated to the **service** (validation, ORM writes, signals) — not written inline in the mutation.
- 4 Returning an empty error list means success; returned errors are written to the `MutationLog` for the client to read when it polls. Either way the GraphQL response carries the `clientMutationId` so the client can correlate.

Did you know?

The `clientMutationId` is **supplied by the client** and echoed back. That is what makes polling possible: the frontend generated the id, so it knows which `MutationLog` to watch. It also makes writes **idempotent to correlate** — the client can always tie a later status to the exact request it sent.

3.3 The full mutation sequence



3.4 Query vs mutation at a glance

Aspect	Query (read)	Mutation (write)
Base machinery	plain resolver	<code>OpenIMISMutation</code>
Returns	the requested data	a <code>clientMutationId</code> (a ticket)
Completion	synchronous — data is in the response	asynchronous — client polls <code>MutationLog</code>
Audit	none required	every mutation writes a <code>MutationLog</code>
Business logic	often direct ORM; service if complex	always a service; fires signals
Authorization	<code>has_perms</code> in resolver	<code>has_perms</code> in <code>async_mutate</code>



Common mistake

Do not treat a mutation's HTTP 200 as "the write finished and every side effect ran." The response means "the request was accepted and a `MutationLog` exists." The **authoritative outcome** is the `MutationLog` status the client polls. UIs that skip polling show stale or wrong state when downstream signal-driven work (e.g. calculation rules) is still settling.

4. Where each concern lives — and why the separation pays off

Concern	Whose job	Consequence of putting it elsewhere
Routing / TLS	Nginx gateway	Backend would need to serve statics and manage certs.
Identity (authn)	JWT middleware	Every resolver would re-parse tokens; inconsistent.
Authorization (authz)	resolver / mutation via <code>has_perms</code>	Middleware can't know a field's required rights; leaks.
Query shape	GraphQL schema + resolver	REST-style over/under-fetching returns.
Business logic + validation	service	Logic trapped in resolvers can't be reused/tested; no signal seam.
Persistence + temporality	ORM / models	Ad-hoc SQL loses <code>validity_*</code> , <code>json_ext</code> , versioning.
Audit + async status	<code>OpenIMISMutation</code> + <code>MutationLog</code>	No traceability; UI can't track long-running writes.

The single most important boundary to internalize: **resolvers authorize and delegate; services do the work**. Keep them apart and the same business operation is reachable from GraphQL, REST, tests, and other modules' signals — mix them and you get logic you can only trigger through one API.

Repository references

Repository	Directory / File	Why it matters
openimis-dist_dkr	docker-compose*.yaml , Nginx config	The gateway that terminates TLS and routes /graphql and /api .
openimis-be_py	openimis/urls.py	Mounts GraphQLView at /graphql .
openimis-be_py	openimis/settings.py	Middleware stack, Graphene + django-graphql-jwt config.
openimis-be_py	openimis/schema.py	The assembled root schema the view executes against.
openimis-be-core_py	core/schema.py (OpenIMISMutation)	The asynchronous, audited mutation base class.
openimis-be-core_py	core/models.py (MutationLog)	The audit + status record the client polls.
openimis-be-core_py	User / auth code	The User / InteractiveUser and has_perms that back authorization.
openimis-be-claim_py	claim/schema.py , claim/gql_mutations.py , claim/services.py	A real query resolver, a mutation, and a service to trace end to end.
openimis-fe_js	core graphql / graphqlWithVariables + polling helper	The Redux layer that sends queries and polls MutationLog .

Hands-on lab

Goal: watch both lifecycles happen against a running dev stack.

1. **Find the endpoint.** In openimis-be_py/openimis/urls.py , locate the GraphQLView mount. Confirm there is only one /graphql .

2. **Watch a query.** With the backend running, open GraphQL at `/graphql`. Run a small `claims` query. In the backend logs, observe the single request; note that only the fields you asked for come back.
3. **Break authorization on purpose.** Log in as a user *without* the claims-read right (or temporarily require a right the user lacks) and re-run the query. Confirm you get a permission error from the **resolver**, not the middleware.
4. **Watch a mutation.** Run a create mutation and include a `clientMutationId`. Observe that the response returns almost immediately with that id.
5. **Poll the log.** Query the `MutationLog` (or `mutationLogs`) by your `clientMutationId` and watch the status move to *success/failed*. This is exactly what the frontend's polling helper does for you.

Exercises

1. A teammate says "we authenticate in middleware, so resolvers don't need to check anything." Refute this in two sentences, naming what middleware does and does not do.
2. Explain why the JWT lives in an **HttpOnly cookie** rather than being read by JavaScript and sent in an `Authorization` header. Name one threat it mitigates.
3. A mutation returns HTTP 200 but the UI shows stale data. Give the most likely cause rooted in the async pattern, and the fix.
4. Draw (in words) where a **service signal** fires in the mutation sequence and name one thing another module might do in an `AFTER` hook.
5. Why does openIMIS return **only** the fields a query requested, and how does that differ from a typical REST endpoint?

Knowledge check

 **Q1: Which layer populates `request.user`, and does it perform authorization? (click for answer)** 

The **JWT authentication middleware** (`django-graphql-jwt` plus openIMIS additions) reads the token from the HttpOnly cookie, verifies it, and attaches the openIMIS `User` to `request.user`. It performs **authentication only** — no authorization. Permission checks happen later, in each resolver/mutation via `has_perms`.

? **Q2: In a query, where exactly does the permission check happen and against what? (click for answer)** ✓

Inside the **resolver** (e.g. `resolve_claims`), which calls `user.has_perms(SomeConfig.gql_query_*_perms)` — the module's **integer rights** read from its `AppConfig`. It runs before any DB access; failure raises a GraphQL error and no rows are read.

? **Q3: What does a mutation return immediately, and how does the client learn the final outcome? (click for answer)** ✓

It returns the `clientMutationId` (the id the client supplied), not the finished result. The client then **polls** the `MutationLog` — filtered by that id — until the status reads *success* or *failed*, and updates the UI on the final status.

? **Q4: Why is a MutationLog created for every mutation? (click for answer)** ✓

Two reasons: **audit** (every business change is attributable to a user, with inputs and outcome recorded) and **status** (it holds the async job's state so the client can poll it). `OpenIMISMutation` creates it before the work runs and updates it as the service completes.

? **Q5: Why keep business logic in a service instead of the resolver/mutation? (click for answer)** ✓

So the same operation is reusable and testable from GraphQL, REST, other services, and signal receivers — and so it can fire **service signals** that let other modules extend it. Logic embedded in a resolver is reachable only through that one GraphQL field and can't participate in the extension seam.

Further reading

- [GraphQL in openIMIS](#) — schema, resolvers, connections, the argument conventions used above.
- [Security](#) — JWT, HttpOnly cookies, OIDC/OAuth2, the `User / InteractiveUser` model, and integer rights.
- [Plugin / Module System](#) — how the resolvers and mutations you traced were assembled and how service signals fire.
- [Backend Deep Dive](#) — services, base temporal models, `MutationLog`.

- `openimis-be-core_py/core/schema.py` — read the real `OpenIMISMutation` to see the `MutationLog` bookkeeping first-hand.
- `django-graphql-jwt` docs — cookie-based JWT authentication for Graphene.
- `graphene-django` docs — `GraphQLView`, execution context, and resolvers.

Repository Map

openIMIS is not one repository — it is a whole **GitHub organization** of them. The first time you browse github.com/openimis it looks like dozens of similarly named projects with no obvious center of gravity. This chapter is the map that fixes that. It groups every major repository by category, tells you what each one is *for*, what it depends on, how mature it is, and where you would customize it — and it draws the dependency graph so you can see the whole org as one ordered structure rather than a flat list.

Keep this page open in a tab while you work. It is the at-a-glance reference. A still-fuller table (with more peripheral and calculation-rule repos) lives in the [Repository Appendix](#); the two are complementary — this page is the working map, the appendix is the exhaustive index.

Learning objectives

By the end of this chapter you will be able to:

- Recognize the **naming conventions** (`openimis-be-<name>_py` , `openimis-fe-<name>_js`) and read a repo's role from its name alone.
- Distinguish **assembly**, **core/infra**, and the various **business** and **integration** repository categories.
- State the purpose, dependencies, and customization points of every major module repository.
- Read the **dependency graph** from `core` at the base up to `api_fhir_r4` at the top, and use it to reason about load order and change impact.
- Choose the right repository for a given change without guessing.

Prerequisites

- [High-Level Architecture](#) — the assembly/module distinction and the layer stack.
 - [Backend Deep Dive](#) — how the manifest turns these repos into one running Django project (manifest order = dependency order).
-

1. Naming conventions: read the repo from its name

openIMIS repository names are rigidly systematic. Learn the pattern and most navigation questions answer themselves.

Pattern	Meaning	Example
<code>openimis-be_py</code>	Backend assembly (the deployable Django project)	<code>openimis-be_py</code>
<code>openimis-fe_js</code>	Frontend assembly (the deployable React SPA)	<code>openimis-fe_js</code>
<code>openimis-be-<name>_py</code>	Backend module ; Python package is <code><name></code>	<code>openimis-be-claim_py</code> → package <code>claim</code>
<code>openimis-fe-<name>_js</code>	Frontend module ; the matching UI	<code>openimis-fe-claim_js</code>
<code>openimis-dist_dkr</code>	Distribution : Docker Compose deployment	<code>openimis-dist_dkr</code>
<code>calcrule_<name></code> (inside/with calculation)	A pluggable calculation rule	<code>calcrule_contribution</code>

Two things fall straight out of this:

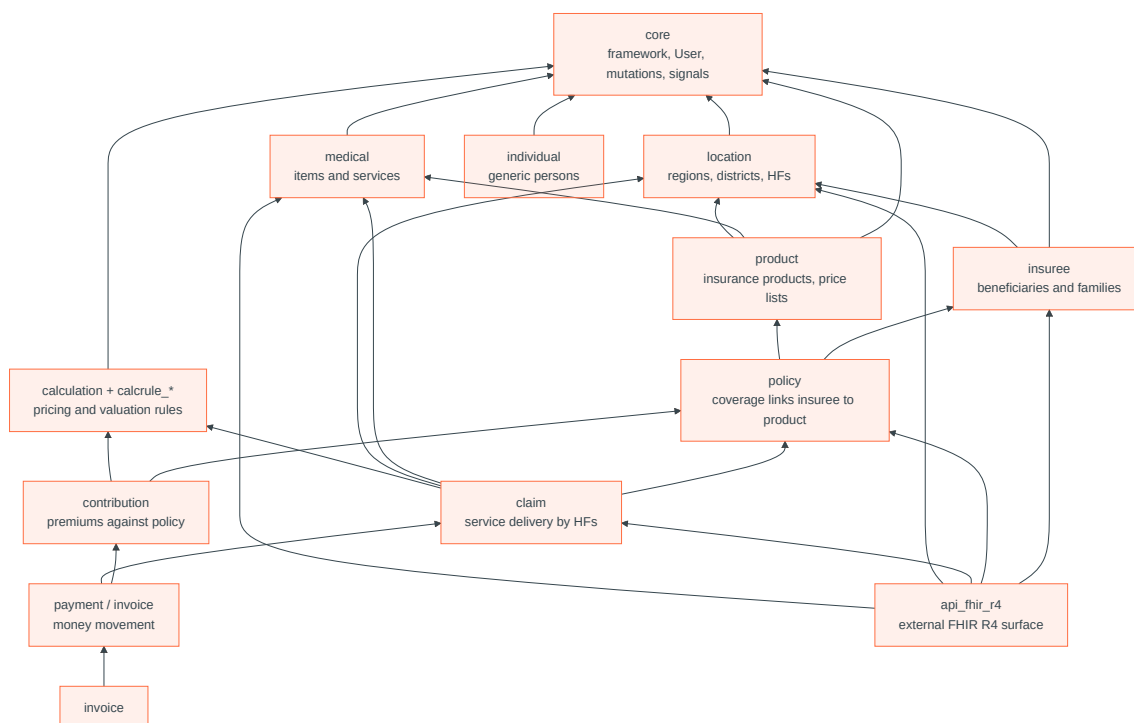
- **be ↔ fe pairing.** Most business concerns have both a backend module and a frontend module of the same `<name>` (`claim`, `insuree`, `policy`, ...). The backend owns data, GraphQL, and services; the frontend owns the screens.
- **Package name ≠ repo name.** The repo is `openimis-be-core_py` but you `import core`. When reading tracebacks or `INSTALLED_APPS`, you see the *package* name (`core`, `claim`); when cloning, you use the *repo* name.

Did you know?

The `_py` / `_js` suffix and `-be-` / `-fe-` infix are not decoration — tooling and the manifests rely on the pattern. When a new module is created it is expected to follow the convention exactly, which is why the whole org reads so uniformly.

2. The dependency graph

Before the tables, the shape. This is the mental model that makes every table below meaningful: a layered stack with `core` at the base, reference data above it, then beneficiaries, coverage, financial flows, and finally the FHIR surface on top. Arrows mean "depends on / builds upon."



Read it bottom-up and it tells a story: you cannot have a **policy** without an **insuree** and a **product**; you cannot have a **contribution** without a **policy**; you cannot value a **claim** without **medical** items/services and a **calculation** rule; and **FHIR** re-exposes many of these to the outside world. That story is exactly the domain of health financing, encoded as a dependency order — and it is the same order the module manifest loads them in (see [Backend Deep Dive](#)).

Did you know?

The dependency graph is also a **change-impact map**. A change in `core` ripples to everything; a change in `api_fhir_r4` ripples to nothing (nothing depends on it). When estimating the blast radius of a change, find the repo in this graph and count what sits *above* it.

3. Maturity legend

The tables use a simple maturity marker so you know how much to trust and how carefully to tread:

Marker	Meaning
Foundational	Stable, heavily depended upon; change with great care.
Mature	Core domain, well established, broadly deployed.
Growing	Actively evolving; APIs may still shift.
Emerging	Newer or specialized; expect gaps and change.

Treat these as directional guidance, not a formal support statement — confirm the current state in each repo's README and release history.

4. Assembly repositories

The two thin projects that turn everything else into a running system. They hold almost no business logic; they *wire*. See the assembly model in [High-Level Architecture](#).

Repository	Purpose	Key responsibilities	Depends on	Maturity	Typical custom points
<code>openimis-be_py</code>	Backend assembly / deployable Django project	Reads <code>openimis.json</code> ; builds <code>INSTALLED_APPS</code> ; assembles root GraphQL schema and URLs; runs migrations and config load at startup	Every backend module (via manifest)	Foundational	Edit <code>openimis</code> to choose module environment settings startup script
<code>openimis-fe_js</code>	Frontend assembly / deployable React SPA	Reads FE <code>openimis.json</code> ; generates <code>src/modules.js</code> ; bundles all FE modules; hosts routing and the Redux store shell	Every frontend module (via manifest); <code>openimis-fe-core_js</code>	Foundational	Edit FE <code>openimis</code> theme/build via config
<code>openimis-dist_dkr</code>	Distribution / deployment	Docker Compose for <code>db</code> , <code>backend</code> , <code>frontend</code> , Nginx gateway, optional OpenSearch; startup ordering; <code>.env</code> secrets	Both assemblies	Mature	<code>.env</code> ; override gateway config;

5. Core / infrastructure repositories

The framework layer. Everything else depends on `core`; the rest here provide cross-cutting platform services (tasks, workflow, reporting infrastructure).

Repository	Purpose	Key responsibilities	Depends on	Maturity
openimis-be-core_py	The framework within the framework	Base/versioned models (HistoryModel , VersionedModel), json_ext ; custom User , InteractiveUser , TechnicalUser , Officer ; integer rights ; OpenIMISMutation async engine + MutationLog ; service signals ; Graphene helpers; ModuleConfiguration ; scheduler; audit	(base)	Foundational
openimis-be-tasks_management_py	Human/automated task workflow	Task queues, approvals, assignment of work items	core	Growing
openimis-be-workflow_py	Workflow orchestration	Multi-step process definitions used by other modules	core	Growing
openimis-be-report_py	Reporting infrastructure	Report definitions and rendering (often PDF), template plumbing	core (+ domain modules for data)	Mature
openimis-be-opensearch	Analytics via OpenSearch	Index domain data; power dashboards/search analytics	core , domain modules, OpenSearch	Emerging

Repository	Purpose	Key responsibilities	Depends on	Maturity
h_reports _py				

6. Reference-data repositories (location / medical / product)

Foundational domain data that the transactional modules point at. These change slowly and everything downstream assumes them.

Repository	Purpose	Key responsibilities	Depends on	Maturity	Type
openimis-be-location_py	Geographic + facility registry	Regions, districts, municipalities; health facilities ; user–location scoping	core	Foundational	Location data
openimis-be-medical_py	Medical catalog	Medical items (drugs/consumables) and services (procedures)	core	Mature	Medical data
openimis-be-product_py	Insurance products	Product definitions, benefit packages, price lists , ceilings/limits	core, medical, location	Mature	Product data

7. Beneficiary repositories (insuree / individual)

Who is covered. `insuree` is the classic insurance beneficiary model; `individual` is a more generic person model introduced to support social-protection use cases.

Repository	Purpose	Key responsibilities	Depends on	Maturity	Type
openimis-be-insuree_profile	Insurance beneficiaries	Insurees, families/households , photos, enrolment data; insuree numbering	core, location	Mature	Entity
openimis-be-individual_registry	Generic person registry	Individual, groups; the beneficiary backbone for social protection	core	Growing	Component

8. Coverage & transactional repositories (policy / contribution / claim)

The heart of the insurance workflow: who is covered (policy), what they paid (contribution), and what care was delivered and valued (claim).

Repository	Purpose	Key responsibilities	Depends on	Maturity
openimis-be-policy_py	Coverage	Links insuree → product for a validity period; policy status/lifecycle; eligibility	core , insuree , product	Mature
openimis-be-contribution_py	Premiums	Records premiums/contributions against policies; payment allocation to coverage	core , policy , calculation	Mature
openimis-be-claim_py	Service delivery	Claims by health facilities against policies; claim items/services; adjudication/review; valuation	core , policy , medical , location , calculation	Mature

9. Financial repositories (payment / invoice / payroll)

Money movement, on both the premium-in and benefit/payment-out sides.

Repository	Purpose	Key responsibilities	Depends on	Maturity	Typical customer point
<code>openimis-be-payment_plugin</code>	Payments	Payment records tied to contributions/claims; reconciliation; gateway integration hooks	<code>core</code> , <code>contribution</code> , <code>claim</code>	Mature	Payment gateway adapter (mostly monitoring rules)
<code>openimis-be-invoice_plugin</code>	Invoicing / billing	Invoices, bills, line items; a more generalized financial-document layer	<code>core</code> , <code>payment</code>	Growing	Invoice template billing
<code>openimis-be-payroll_plugin</code>	Benefit payouts	Payroll/benefit payment runs, notably for social protection	<code>core</code> , <code>payment</code> , <code>individual</code>	Emerging	Payment cycle adapter
<code>openimis-be-contract_plugin</code>	Contracts	Formal contracts (e.g. group/employer coverage agreements)	<code>core</code> , <code>policy</code> , <code>insuree</code>	Growing	Contract term enrolment

10. Calculation-rule repositories

The pluggable pricing/valuation engine. `calculation` is the framework; each `calcrule_*` is a rule that **binds via service signals** (see [Backend Deep Dive](#)) to value contributions, claims, capitation, third-party payments, and more — without the valued module importing the rule.

Repository	Purpose	Key responsibilities	Depends on	Maturity	Type
openimis-be-calculation.py	Calculation framework	Rule registration/discovery; the signal-based hook that runs rules at the right moment	core	Mature	Rule extension mechanism
calcrule_contribution_*	Contribution valuation rules	Compute contribution/premium amounts	calculation, contribution, product	Growing	Rate processor for
calcrule_validations	Validation rules	Cross-field/business validations as calculation rules	calculation	Growing	Validation
calcrule_capitation_n_*	Capitation payment rules	Capitation-based provider payment	calculation, claim, product	Emerging	Capitation for
calcrule_third_party_payment_t	Third-party payment rules	Value provider/third-party payments	calculation, claim, payment	Emerging	Payment for

Did you know?

The `calcrule_*` repos are the clearest proof that openIMIS's plugin model works: a country can drop in a new pricing rule as a **separate repository** that binds to a service signal, and neither `contribution`, `claim`, nor `core` needs a single line changed to accommodate it.

11. Integration & reporting repositories

The bridges to the wider digital-health ecosystem. `api_fhir_r4` sits at the very top of the dependency graph, re-exposing many modules through the FHIR standard.

Repository	Purpose	Key responsibilities	Depends on	Maturity
<code>openimis-be-api_fhir_r4_py</code>	FHIR R4 REST surface	Map openIMIS models to FHIR resources (Insuree → Patient, Policy → Coverage, Claim → Claim, HF → Location/Organization, staff → Practitioner); endpoints under <code>/api_fhir_r4/</code>	<code>core</code> , <code>insuree</code> , <code>policy</code> , <code>claim</code> , <code>location</code> , <code>medical</code>	Mature
<code>openimis-be-dhis2_etl_py</code>	DHIS2 integration	Extract/transform/push aggregate indicators to DHIS2	<code>core</code> , domain modules	Growing
<code>openimis-be-tools_py</code>	Data tooling	Bulk import/export, registers, data exchange utilities	<code>core</code> , domain modules	Mature
<code>openimis-be-opensearch_h_reports_py</code>	Analytics search	Feed OpenSearch for dashboards/analytics	<code>core</code> , domain modules, OpenSearch	Emerging

See [Integrations](#) and [FHIR R4 API](#) for how these are used in practice.

12. Social-protection repositories

A newer family extending openIMIS beyond classic health insurance toward social-protection / cash-transfer programs, built on the generic `individual` person model rather than `insuree`.

Repository	Purpose	Key responsibilities	Depends on	Maturity	Typical custom points
<code>openimis-be-social_protection.py</code>	Social protection programs	Benefit plans, beneficiary enrolment, program cycles for cash/in-kind transfers	<code>core</code> , <code>individual</code> , <code>payment / payroll</code>	Growing	Program definition, eligibility, benefit r
<code>openimis-be-individual.py</code>	Person backbone (shared)	Generic <code>Individual</code> / group registry underpinning social protection	<code>core</code>	Growing	Custom attribute, import pipelines
<code>openimis-be-payroll.py</code>	Benefit payouts (shared)	Payment runs for social-protection benefits	<code>core</code> , <code>payment</code> , <code>individual</code> , <code>l</code>	Emerging	Payout cycles/a

Did you know?

The split between `insuree` (insurance beneficiary) and `individual` (generic person) reflects openIMIS's expansion in scope. Health-insurance schemes lean on `insuree / policy / claim`; social-protection programs lean on `individual / social_protection / payroll`. Both families share the same `core`, which is precisely what lets one platform serve both.

13. Frontend repositories

The `fe` mirror of the backend. Each business concern generally has a matching `openimis-fe-
<name>_js` providing its screens; `openimis-fe-core_js` provides the shared runtime — `ModulesManager`, the Redux/GraphQL layer, and the base UI.

Repository	Purpose	Key responsibilities	Depends on	Maturity	Typical custom points
<code>openimis-fe-core_js</code>	Frontend framework	<code>ModulesManager</code> registry; custom <code>graphql / graphqlWithVariables</code> Redux layer; async-mutation polling ("journalize"); base MUI components, <code>i18n</code> , route guards	(base)	Foundational	Theme components authentication flow
<code>openimis-fe-insuree_js</code>	Insuree UI	Insuree/family screens, pickers, search	<code>fe-core</code> , <code>fe-location</code>	Mature	Forms injection controls
<code>openimis-fe-policy_js</code>	Policy UI	Enrolment, renewal, coverage screens	<code>fe-core</code> , <code>fe-insuree</code> , <code>fe-product</code>	Mature	Enrolment forms
<code>openimis-fe-claim_js</code>	Claim UI	Claim entry, review, adjudication screens	<code>fe-core</code> , <code>fe-medical</code> , <code>fe-location</code>	Mature	Claim review
<code>openimis-fe-location_js</code>	Location UI	Region/district/HF management pickers	<code>fe-core</code>	Mature	Hierarchical
<code>openimis-fe-medical_js /</code>	Reference-data UI	Item/service catalogs; product/price-list editors	<code>fe-core</code>	Mature	Catalog editor

Repository	Purpose	Key responsibilities	Depends on	Maturity	Typical customer points
openimis-fe-product_js					
openimis-fe- <name>_js (others)	Per-module UI	Screens for contribution, payment, invoice, individual, social_protection, tools, etc.	fe-core + peers	Varies	Screens, contributions, slow

The frontend extension seam (config objects, `contributions`, `ModulesManager`) is covered in [Frontend Architecture](#) and [Plugin / Module System](#).

14. How to choose the right repository for a change

A quick decision aid — the questions to ask, in order:

1. **Is it wiring or domain?** Choosing/pinning modules, gateway config, boot sequence → an **assembly** or **distribution** repo. Behavior of a concern → the matching **module** repo.
2. **Which concern?** Follow the domain: beneficiary → `insuree / individual`; coverage → `policy`; premium → `contribution`; care delivered → `claim`; money out → `payment / invoice / payroll`; pricing → `calculation / calculule_*`; external exchange → `api_fhir_r4 / dhis2_etl / tools`.
3. **Backend or frontend?** Data/API/business rule → `be`. Screen/interaction → `fe`. Many changes touch **both** the `be` and `fe` of the same `<name>`.
4. **Would it belong in core?** Only if it is genuinely cross-cutting (a new base model behavior, a new mutation-engine capability). Resist putting domain logic in `core` — it raises the change's blast radius to *everything*.



Common mistake

Adding a country-specific feature to `core` or to an assembly repo "because it was convenient." That maximizes blast radius and breaks the customization model. Prefer a new/extended **module** — or a `calcrule_*` rule, or a `json_ext` field, or a `ModuleConfiguration` entry — so the change stays in a controlled seam. Revisit the rationale in [High-Level Architecture](#).

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be_py</code>	<code>openimis.json</code>	The definitive list of which backend repos are assembled, and in what order.
<code>openimis-fe_js</code>	<code>openimis.json</code>	The frontend counterpart module list.
<code>openimis-be-core_py</code>	<code>core/</code>	The base every other repo depends on; read it first.
<code>openimis-be-calculation_py</code>	<code>calculation/</code>	Anchor for the <code>calcrule_*</code> family; shows the plugin model in action.
<code>openimis-be-api_fhir_r4_py</code>	<code>api_fhir_r4/</code>	The top-of-graph integration surface; depends on many modules, is depended on by none.
<code>openimis-dist_dkr</code>	<code>docker-compose.*</code>	Ties the whole org into a runnable stack.
(reference)	Repository Appendix	The fuller, exhaustive repo index complementary to this map.

Hands-on lab

Goal: turn the map into muscle memory using the real org.

1. Browse github.com/openimis. For ten repos you have never seen, predict category and purpose **from the name alone**, then open each README to check. Score yourself.
2. Open `openimis.json` in `openimis-be_py`. Confirm `core` appears early and that reference-data modules (`location`, `medical`, `product`) precede the modules that consume them (`policy`, `contribution`, `claim`). Note where `api_fhir_r4` sits — near the end.
3. Pick one business concern (say `claim`) and find **both** its `be` and `fe` repos. List the concerns each side owns.
4. Using the dependency graph in §2, pick any module and write down its full "blast radius": everything that sits *above* it and would be affected by a breaking change.

Deliverable: your name-prediction score from step 1 and the blast-radius list from step 4.

Exercises

1. Explain, from names alone, the difference between `openimis-be-claim_py`, `openimis-fe-claim_js`, and the package you would `import` for the first.
2. A partner wants a country-specific premium formula. Which repository (or new repo) should it live in, and which existing repo must it **not** modify? Why?
3. Order these by dependency, base first: `claim`, `core`, `policy`, `product`, `insuree`, `api_fhir_r4`. Justify each step.
4. `payroll` and `social_protection` are newer than `claim` and `policy`. What does that tell you about openIMIS's evolving scope, and which person model does each family build on?

Knowledge check

? Q1: What do the naming conventions tell you? (click for answer) ✓

`openimis-be-<name>_py` is a **backend module** whose Python package is `<name>`; `openimis-fe-<name>_js` is the matching **frontend module**; `openimis-be_py` / `openimis-fe_js` are the **assembly** projects; and `openimis-dist_dkr` is the **Docker distribution**. From a name you can infer tier (assembly vs module), side (backend vs frontend), and concern — before opening the repo.

? Q2: Why is `core` at the base of the dependency graph and `api_fhir_r4` at the top? (click for answer) ✓

`core` provides the shared framework (base models, User, mutation engine, service signals) that **every** module builds on, so it must load first and a change in it affects everything. `api_fhir_r4` sits on top because it *re-exposes* other modules (insuree, policy, claim, location, medical) as FHIR resources; it depends on many modules but **nothing depends on it**, so it loads last and a change in it has minimal blast radius.

? Q3: What is the difference between the insuree and individual families? (click for answer) ✓

`insuree` is the classic **insurance beneficiary** model (insurees, families, enrolment) underpinning `policy / contribution / claim`. `individual` is a **generic person** model introduced to support **social protection** (cash/in-kind transfer programs) via `social_protection` and `payroll`. Both families share the same `core`, which is what lets one platform serve health insurance and social protection at once.

? Q4: Where should a new country-specific pricing rule live, and what must it not touch? (click for answer) ✓

In a new/extended `calcrule_*` repository that **binds via service signals** to the relevant service (e.g. contribution or claim valuation). It must **not** modify `contribution`, `claim`, or `core` — the whole point of the calculation framework is that pricing rules plug in through the signal seam without those modules importing or knowing about the rule.

? Q5: You need to change how a claim is adjudicated in the UI and how it is valued on the server. Which repositories? (click for answer) ✓

Two, one per side of the same concern: `openimis-fe-claim_js` for the adjudication **screens/interaction**, and `openimis-be-claim_py` (plus, likely, a `calcrule_*` rule for the **valuation** logic) for the server-side business rule. Many real changes are exactly this kind of coordinated `be + fe` pair.

Further reading

- [High-Level Architecture](#) — why the org is split this way.
- [Backend Deep Dive](#) — how the manifest turns these repos into one Django project (manifest order = dependency order).

- [Plugin / Module System](#) — the service-signal and contribution seams that make new modules/rules pluggable.
- [Frontend Architecture](#) — the `fe` side of the map.
- [Repository Appendix](#) — the exhaustive, complementary repo index.
- The [openIMIS GitHub organization](#) — the ground truth; every repo's README and releases.

Frontend Architecture

Part 11 — Frontend Architecture

The openIMIS frontend is a **React single-page application** (SPA) that is assembled from the same modular philosophy as the backend: ~dozens of independently versioned repositories are stitched together at build time into one running app. If you have already read the [Plugin / Module System](#) chapter, the shape will feel familiar — but the frontend has its own manifest, its own registry (`ModulesManager`), its own extension seam (`contributions`), and a deliberately **un-Apollo** GraphQL layer built on Redux. This chapter opens all of that up.

Learning objectives

By the end of this chapter you will be able to:

- Explain how `openimis-fe_js` assembles a single SPA from many `openimis-fe-<name>_js` modules via the `openimis.json` manifest and `openimis-config-vite.js`.
- Describe the **Vite** build (and why the project migrated off Create React App / CRACO).
- Read a module's **config object** and know what each key (`translations` , `reducers` , `refs` , `queries` , `mutations` , `menus` , `routes` , `contributions`) does.
- Trace a GraphQL query and an asynchronous mutation through the **custom Redux GraphQL layer** — `graphql` / `graphqlWithVariables` action creators plus `journalize` /polling — and articulate why openIMIS does **not** use Apollo Client.
- Use the `ModulesManager` registry and `publishedComponents` (`getRef` , `getConfig` , `getContribs`) as the frontend extension seam.
- Understand authentication (JWT cookie, integer **rights** in Redux, route guards), theming (MUI), and internationalization (`react-intl`).

Prerequisites

- [Platform Overview](#) — the monolith-to-modular story.
- [Set Up a Dev Environment](#) — you should be able to boot the frontend locally.
- [Plugin / Module System](#) — the backend module pattern this chapter mirrors.
- [The GraphQL Layer](#) — GraphQL from first principles; this chapter assumes you know what a query and a mutation *are*.

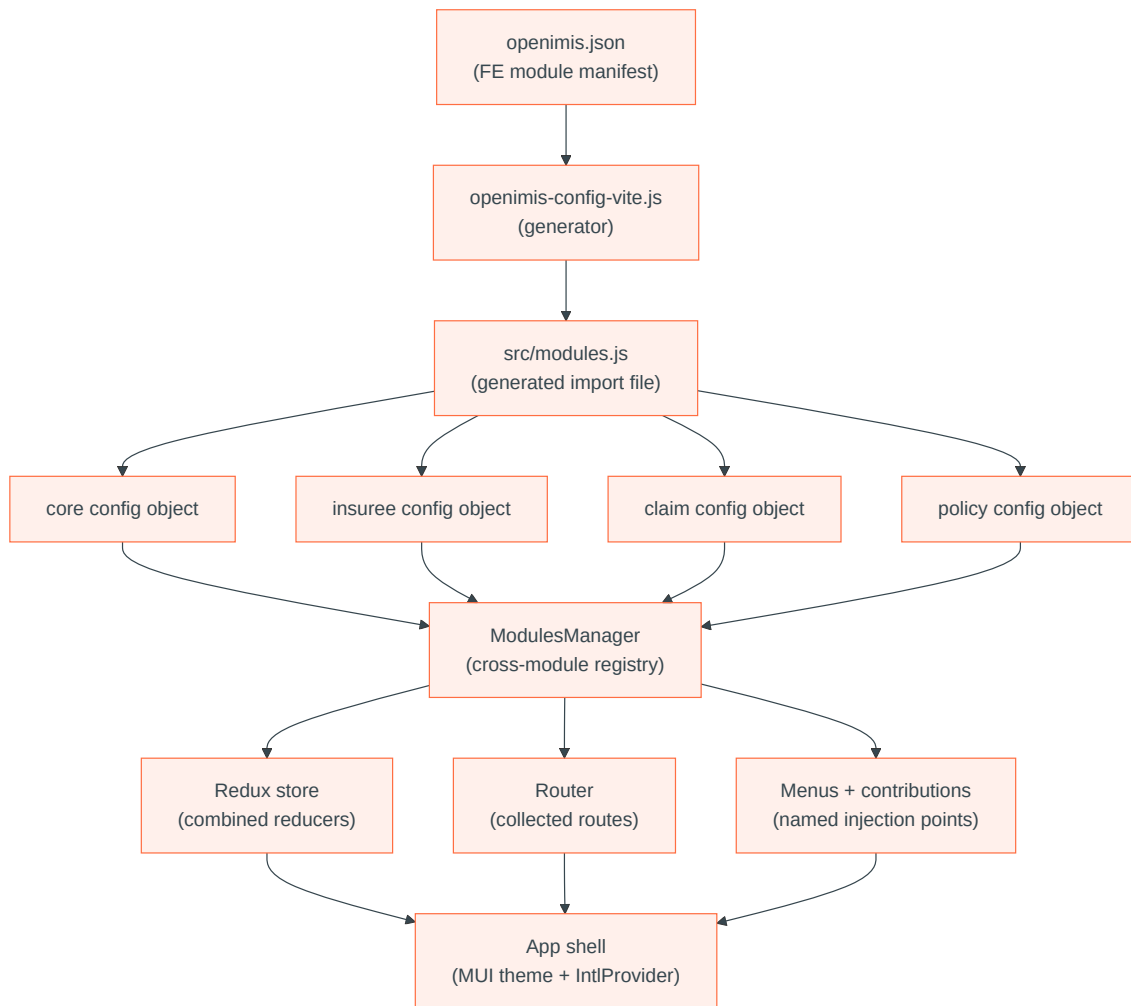
- **Core module** — the frontend `ModulesManager` and the Redux GraphQL helpers live in the frontend **core** module.
-

1. The mental model: an SPA made of plugins

A conventional React app has one `package.json`, one source tree, one `App.jsx`. openIMIS inverts this. The deployable frontend is the **assembly repo** `openimis-fe_js`; it contains almost no business logic. Instead it contains:

- a **manifest** (`openimis.json`) listing which frontend modules to include and at which version,
- a **generator** (`openimis-config-vite.js`) that reads the manifest and writes a `src/modules.js` file importing every module's config object,
- the **shell**: the root component, the Redux store wiring, the router, the theme provider, and the `<IntlProvider>`.

Each business capability — insurances, policies, claims, locations, products — lives in its own repository named `openimis-fe-<name>_js` and ships a single **config object** as its public contract. The shell never imports a claim component directly; it imports the *claim module's config* and lets the registry do the rest.



i Did you know?

The frontend manifest and the backend manifest share the same filename — `openimis.json` — and the same intent: *declare which modules, at which versions, compose this deployment*. The backend one lives in `openimis-be_py`; the frontend one lives in `openimis-fe_js`. Learning one teaches you the other.

2. The module manifest and the generated import file

2.1 `openimis.json`

The frontend `openimis.json` is a small JSON document that lists the modules to load. Conceptually it looks like this (illustrative):

```
{
  "modules": [
    { "name": "@openimis/fe-core",    "npm": "@openimis/fe-core@^1.9.0" },
    { "name": "@openimis/fe-insuree", "npm": "@openimis/fe-insuree@^1.9.0" },
    { "name": "@openimis/fe-policy",  "npm": "@openimis/fe-policy@^1.9.0" },
    { "name": "@openimis/fe-claim",   "npm": "@openimis/fe-claim@^1.9.0" }
  ]
}
```

Each entry names an npm package (published from an `openimis-fe-<name>_js` repository). During local development you replace a published package with a **local, linked** checkout so you can edit a module and see it live (covered in [Developer Workflow](#)).

2.2 `openimis-config-vite.js` → `src/modules.js`

You cannot `import` from a list in a JSON file at runtime — JavaScript bundlers need static `import` statements to tree-shake and resolve dependencies. So openIMIS runs a **code generator** at (pre)build time. `openimis-config-vite.js` reads `openimis.json` and emits a JavaScript module — commonly `src/modules.js` — that imports each module's default config and exports them as an ordered array. Illustrative output:

```
// src/modules.js — GENERATED, do not edit by hand
import coreModule    from "@openimis/fe-core";
import insureeModule from "@openimis/fe-insuree";
import policyModule  from "@openimis/fe-policy";
import claimModule   from "@openimis/fe-claim";

const modules = [
  coreModule(CFG),
  insureeModule(CFG),
  policyModule(CFG),
  claimModule(CFG),
];

export default modules;
```

The shell imports this generated array once and hands it to the `ModulesManager`.

Common mistake

Do not hand-edit `src/modules.js`. It is regenerated from `openimis.json` on every build; your edits will vanish. To add or remove a module, edit `openimis.json` and re-run the generator (it runs automatically as part of the Vite build). If a module you added does not appear, 90% of the time the manifest entry is missing or the package is not installed/linked — not a bug in your component.

3. The build: Vite (migrated from CRA/CRACO)

Historically the frontend was a **Create React App** (CRA) project, customized with **CRACO** (Create React App Configuration Override) because CRA does not let you touch its webpack config directly — and openIMIS *needs* to, in order to run the module-generation step and alias local module checkouts. That stack was slow to start and heavy to maintain.

The project migrated to **Vite**. The practical differences you will notice:

Concern	Old (CRA + CRACO)	New (Vite)
Dev server start	slow (webpack bundles up front)	near-instant (native ESM, on-demand)
Hot reload	full webpack HMR	fast HMR
Config file	<code>craco.config.js</code>	<code>vite.config.js</code>
Module generation	<code>openimis-config.js</code>	<code>openimis-config-vite.js</code>
Env vars	<code>REACT_APP_*</code> (<code>process.env</code>)	<code>VITE_*</code> (<code>import.meta.env</code>)
Entry HTML	<code>public/index.html</code>	root <code>index.html</code> with <code><script type=module></code>

Migration footgun

When porting an old module or an old how-to, watch for `process.env.REACT_APP_*`. Under Vite those become `import.meta.env.VITE_*`. A module that reads a CRA-style env var will silently get `undefined` and behave as if the value were unset.



Deep dive: why a generator instead of dynamic import?



You *could* imagine loading modules with dynamic `import()` at runtime driven by JSON. openIMIS deliberately does not, for three reasons. First, **static analysis**: a bundler can only tree-shake, code-split, and type-check imports it can see statically. Second, **single bundle determinism**: the set of modules is fixed for a given deployment build, so resolving it at build time yields a reproducible artifact served by Nginx. Third, **shared singletons**: React, MUI, Redux, and the `ModulesManager` must be *one* instance across all modules; a static build with peer dependencies guarantees that, whereas independently loaded runtime bundles risk duplicate React copies (the classic "invalid hook call" disaster).

4. The module config object

Every frontend module's public API is a single object (often produced by a factory function so it can read global config). Its keys are the extension points the shell knows how to consume. Illustrative, faithful to openIMIS conventions:

```
// openimis-fe-claim_js/src/index.js (illustrative / simplified)
import messages_en from "../translations/en.json";
import reducer from "../reducer";
import ClaimsPage from "../pages/ClaimsPage";
import ClaimMainMenu from "../menus/ClaimMainMenu";
import { CLAIM_SEARCHER_ROUTE } from "../constants";

const DEFAULT_CONFIG = {
  "translations": [{ key: "en", messages: messages_en }],
  "reducers": [{ key: "claim", reducer }],
  "refs": [
    { key: "claim.ClaimStatusPicker", ref: ClaimStatusPicker },
    { key: "claim.route.claims", ref: CLAIM_SEARCHER_ROUTE },
  ],
  "queries": [{ key: "claim.ClaimsQuery", query: CLAIMS_QUERY }],
  "mutations": [{ key: "claim.submit", mutation: SUBMIT_CLAIM }],
  "menus": [{ key: "insuree.MainMenu", menu: ClaimMainMenu }],
  "core.Router": [
    { path: CLAIM_SEARCHER_ROUTE, component: ClaimsPage },
  ],
  "insuree.MainMenu": [
    { text: "Claims", icon: <ReceiptIcon />, route: "/" + CLAIM_SEARCHER_ROUTE },
  ],
];

export function ClaimModule(cfg) {
  return { ...DEFAULT_CONFIG, ...cfg };
}
```

The keys fall into two families.

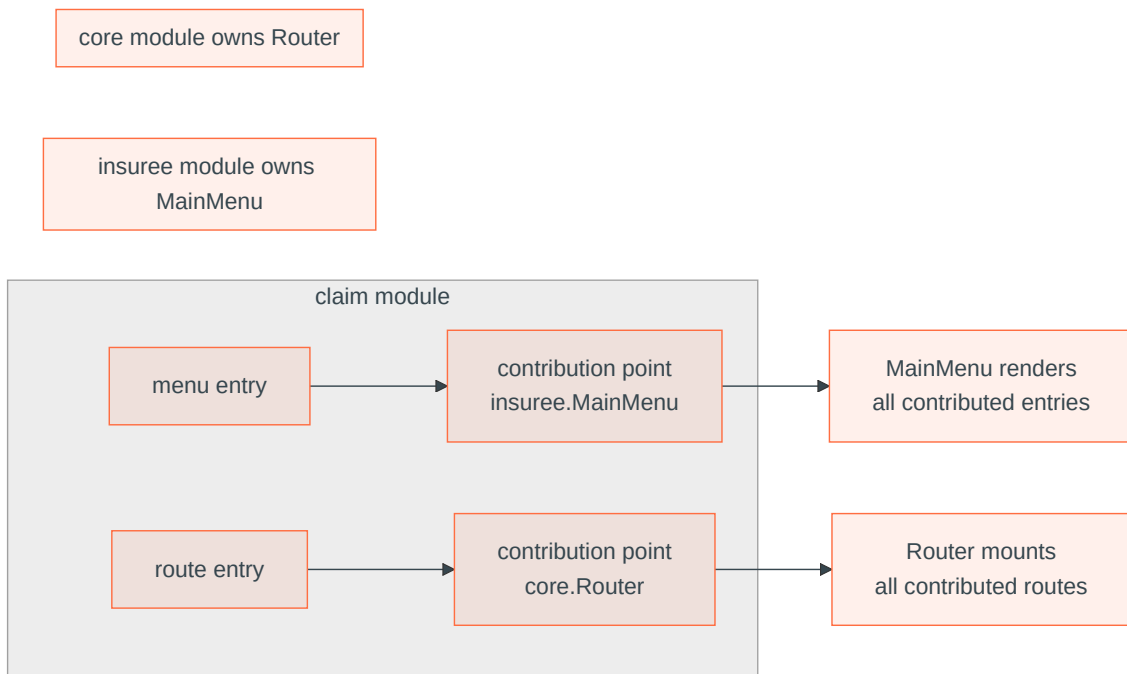
Registry keys — consumed by `ModulesManager` so *any* module can look them up by name:

Key	Purpose	Looked up with
<code>translations</code>	<code>react-intl</code> message catalogs per locale	(merged into <code>IntlProvider</code>)
<code>reducers</code>	Redux slice reducers, keyed by name	(merged into the store)
<code>refs</code>	Named, reusable components/values (the FE "published components")	<code>modulesManager.getRef(key)</code>
<code>queries / mutations</code>	Reusable GraphQL documents	<code>modulesManager.getRef(key)</code>

Contribution keys — *named injection points*. Any string key like `"insuree.MainMenu"`, `"core.Router"`, or `"core.AppBar"` is a **contribution point**. A module *contributes* entries to a point owned by another module (or itself), and the owner renders whatever it collects:

```
// The insuree module owns "insuree.MainMenu"; the claim module contributes to it.  
const contribs = modulesManager.getContribs("insuree.MainMenu");
```

This is the frontend analogue of the backend's service signals: a module extends another **without importing it**, purely by agreeing on a string key.



Did you know?

Because contribution points are just strings, you can add a **new** injection point in your own module simply by calling `modulesManager.getContribs("myModule.SomePanel")` in a component and documenting the key. Other teams can then extend your UI without you shipping a new release. That is the whole extensibility bet of openIMIS, on the frontend.

5. The `ModulesManager` registry and `publishedComponents`

`ModulesManager` (provided by the frontend **core** module, `openimis-fe-core_js`) is constructed once from the array of config objects and made available throughout the tree (via context / props). It is the single source of truth for "what did the loaded modules publish?"

Its three most-used methods:

Method	Answers	Typical use
<code>getRef(key)</code>	"Give me the component/value published under this name."	Reuse another module's <code>InsureePicker</code> , or override it.
<code>getConfig(module, key, default)</code>	"What is the effective config value?"	Read module config injected at build/runtime.
<code>getContribs(key)</code>	"Give me every entry contributed to this point."	Render menus, routes, dashboard tiles.

`publishedComponents` is the pattern behind `refs`: a module *publishes* a component under a stable string name so other modules can reference it by name rather than by import path. This decouples consumers from file locations and — crucially — lets a country-specific module **override** a core component by publishing its own component under the same key later in the module load order.

```
// A claim form reusing the insuree module's published picker (illustrative)
function ClaimHeader({ modulesManager, ...props }) {
  const InsureePicker = modulesManager.getRef("insuree.InsureePicker");
  const readOnly = modulesManager.getConfig("fe-claim", "claimForm.readOnly",
false);
  return <InsureePicker readOnly={readOnly} {...props} />;
}
```

Common mistake

Importing another module directly — `import InsureePicker from "@openimis/fe-insuree/..."` — defeats the entire architecture. It creates a hard compile-time dependency, breaks override-by-key, and can pull a second copy of shared libraries into the bundle. **Always** go through `modulesManager.getRef("insuree.InsureePicker")`. If `getRef` returns `undefined`, the owning module is not in `openimis.json` — that is a manifest problem, not a reason to reach for a direct import.

6. State management: Redux, one store, many module reducers

openIMIS uses **Redux** (with thunk middleware), not React Context alone and not Apollo's cache. Each module contributes a reducer under a namespaced key; the shell combines them

with `combineReducers` into a single store.

```
// Assembled by the shell (illustrative)
import { combineReducers } from "redux";
// modules.js exposes each module's { key, reducer } entries
const rootReducer = combineReducers({
  core: coreReducer,
  insuree: insureeReducer,
  policy: policyReducer,
  claim: claimReducer, // <- from the claim module's config.reducers
});
```

So `state.claim` is owned entirely by the claim module, `state.insuree` by the insuree module, and so on. Modules read cross-module state by namespace but should mutate only their own slice.

7. The custom GraphQL layer (NOT Apollo)

This is the single most important thing to understand about the openIMIS frontend, and the one that surprises people coming from a typical React + GraphQL stack.

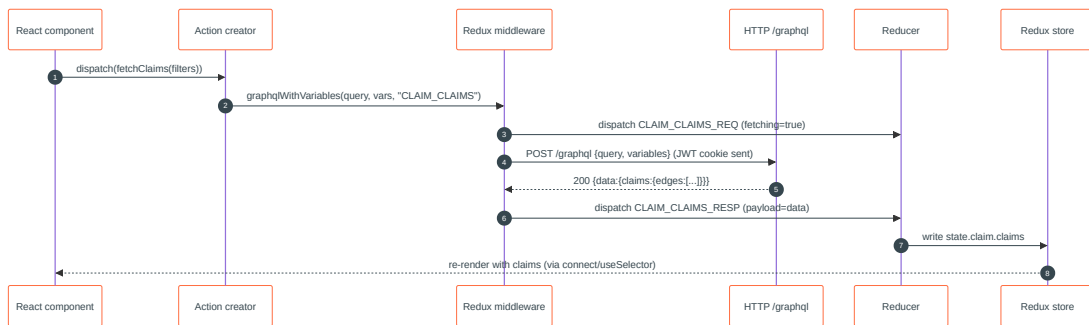
There is no Apollo Client. GraphQL requests are dispatched as **Redux actions** through helpers provided by the frontend core module: `graphql` and `graphqlWithVariables`. They send a `POST` to `/graphql`, and their lifecycle (request → success → error) is expressed as Redux actions that module reducers handle. The GraphQL *response* becomes ordinary Redux *state*.

7.1 A query

```
// In a module's action creators (illustrative / simplified)
import { graphqlWithVariables } from "@openimis/fe-core";

export function fetchClaims(modulesManager, filters) {
  const payload = formatPageQuery("claims", filters, CLAIM_PROJECTION);
  return graphqlWithVariables(
    payload, // the GraphQL query string / doc
    { /* variables */ },
    "CLAIM_CLAIMS", // action-type prefix
    { /* extra meta */ },
  );
}
```

`graphqlWithVariables` dispatches three actions in sequence — request, success (with `response.data`), error — using the prefix you pass (`CLAIM_CLAIMS_REQ`, `CLAIM_CLAIMS_RESP`, `CLAIM_CLAIMS_ERR`, by convention). The claim reducer listens for those types and writes the results into `state.claim`.



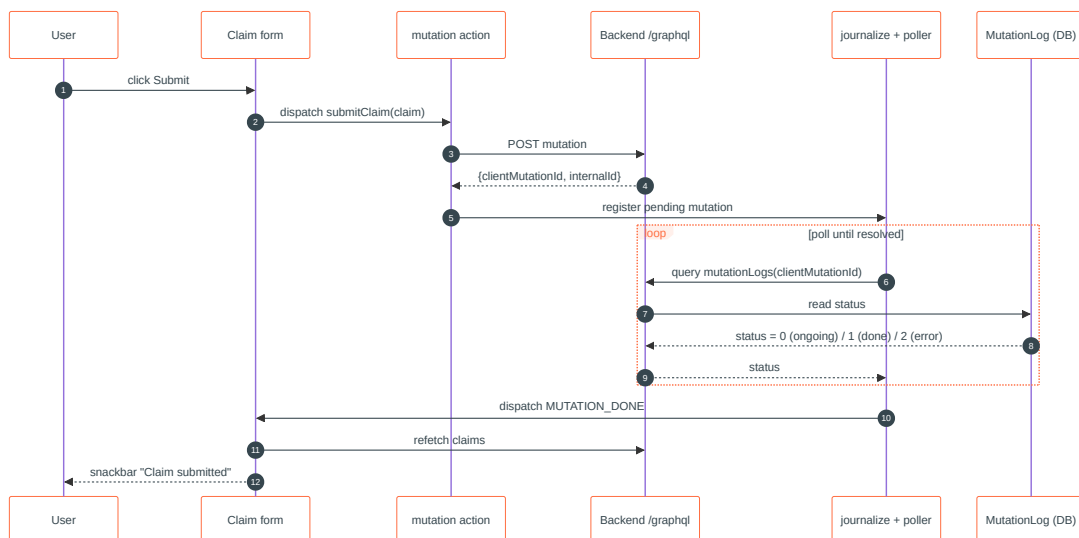
Compared with Apollo, the mental model is: **GraphQL is just an async data source; Redux is the cache.** You already know Redux; you do not have to learn Apollo's normalized cache, `InMemoryCache` policies, or `useQuery` semantics.

7.2 An asynchronous mutation — `journalize` and polling

Backend mutations in openIMIS are **asynchronous** by design (see the `OpenIMISMutation` pattern in [Core module](#)). A mutation does not return the finished business result inline. Instead it:

1. creates a `MutationLog` row (status = *received/ongoing*),
2. returns a `clientMutationId`,
3. does the real work in a backend **service**, updating the `MutationLog` status when done.

The frontend therefore fires the mutation and then **polls** the mutation's status until it resolves. The core helper that manages this is commonly called **journalize**: it tracks in-flight mutations in Redux (a "journal" of pending mutations), polls the backend for their `MutationLog` status, and dispatches a completion action when a mutation succeeds or fails. Modules react to that completion by refetching affected data and showing a toast/snackbar.



Did you know?

Every mutation being logged in `MutationLog` is not just a frontend inconvenience — it is an **audit** feature. Because openIMIS manages public health-insurance money, *who changed what, when* is a first-class requirement. The polling UX is the price of a fully audited, asynchronous mutation pipeline. See the backend [Request Lifecycle](#) chapter for the server side.

Common mistake

Do not treat an openIMIS mutation like a normal GraphQL mutation whose payload contains the created object. The immediate response only confirms the mutation was **accepted**; the actual result and any validation errors surface later via the `MutationLog` status that `journalize` polls. If your UI updates optimistically and never waits for the completion action, it will happily show success for a mutation that the backend later rejected.

Deep dive: why not Apollo?

openIMIS predates Apollo's dominance and, more importantly, its data-flow does not fit Apollo's synchronous request/response cache model. Apollo assumes a mutation returns its result so the cache can update. openIMIS mutations return a *ticket* (`clientMutationId`) and complete out-of-band. Modeling that in Apollo would mean fighting the cache; modeling it in Redux is natural — an in-flight mutation is just a piece of state, and a poller is just a thunk. It also keeps *one* state paradigm (Redux) across the whole app rather than two (Redux for UI + Apollo for server data).

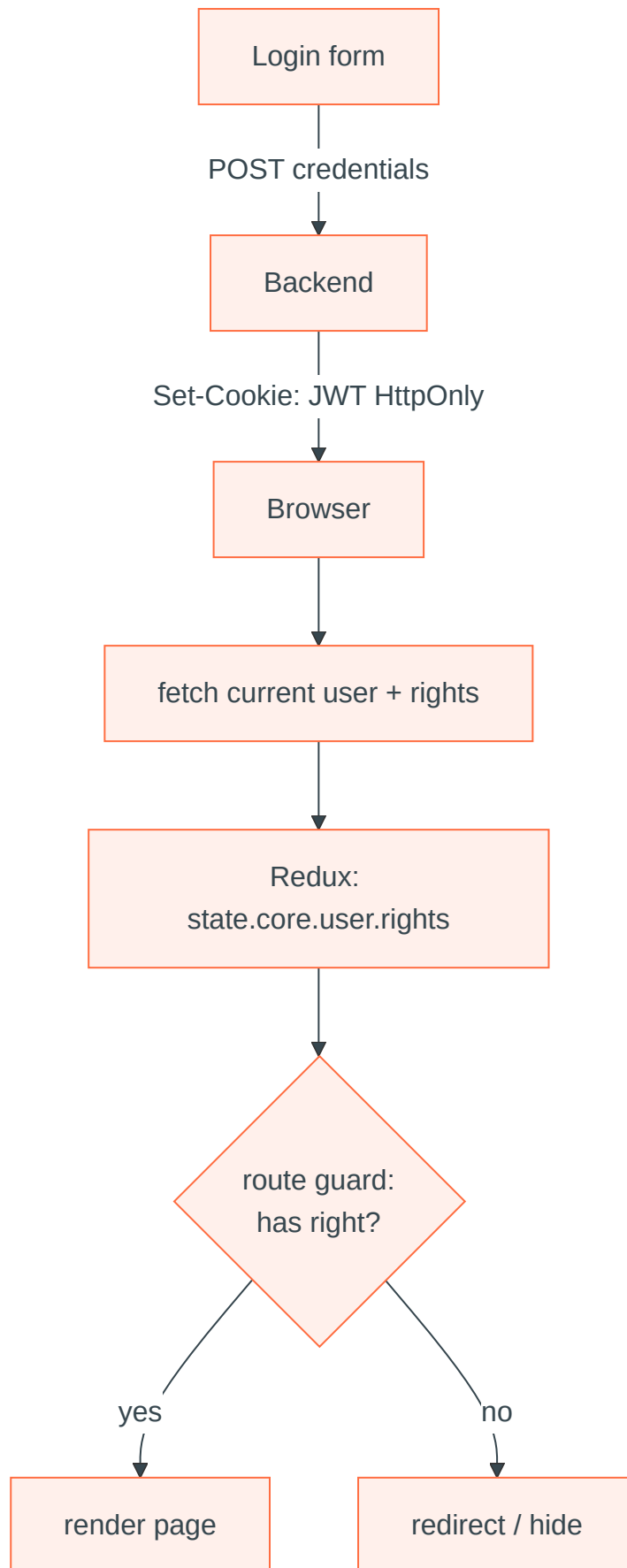
8. Routing

Routes are **contributed**, not centralized. The core module owns the router and a route contribution point (commonly `"core.Router"`); every module contributes `{ path, component }` entries. At startup the router collects `modulesManager.getContribs("core.Router")` and mounts them. This means adding a page is a *data* change (one contribution) rather than editing a central route table.

Route access is **guarded** by rights (next section): a guarded route checks the logged-in user's integer rights before rendering, redirecting to a "forbidden" or login view otherwise.

9. Authentication and authorization

- **Login** posts credentials to the backend, which issues a **JWT stored in an `HttpOnly` cookie** (`django-graphql-jwt` on the backend). Because the cookie is `HttpOnly`, JavaScript cannot read the token — every `POST /graphql` simply carries the cookie automatically. This is a security feature: an XSS bug cannot exfiltrate the token.
- After login the app fetches the current user (including their **rights** — the same integer permission codes used on the backend, e.g. `111001`) and stores them in Redux (`state.core / user`).
- **Route guards** and **conditional UI** check those rights: `modulesManager` / helper utilities expose the user's rights so a component can hide a "Submit Claim" button, or a whole route, when the right is absent.





Rights are integers, and they must match the backend

Frontend right checks use the **same integer codes** the backend defines in each module's `apps.py` (e.g. `gql_query_claims_perms = [111001]`). If you gate a button on the wrong integer, the UI and the API will disagree — the button shows but the mutation is rejected, or vice-versa. Treat the backend `apps.py` permission codes as the single source of truth.

10. Component structure, forms, theming, and i18n

10.1 Components and forms

A typical business module ships:

- **Pages** (route targets) — e.g. a searcher page and an edit page.
- **Searchers** — filterable, paginated tables backed by a GraphQL query and Redux state; openIMIS core provides reusable searcher scaffolding.
- **Forms** — controlled components that build up a business object in local/Redux state and dispatch a mutation on save. Core provides form primitives, pickers, and input components that modules reuse via `getRef`.

10.2 Theming (MUI)

The UI is built on **Material-UI (MUI)**. The shell wraps the app in a `<ThemeProvider>` with an openIMIS theme (palette, typography, spacing). Modules should consume theme tokens (`theme.palette.primary`, `theme.spacing(2)`) rather than hard-coding colors, so a deployment can re-theme centrally.

10.3 Internationalization (`react-intl`)

openIMIS is deployed across many countries, so **every user-facing string is translated**. The stack is `react-intl`:

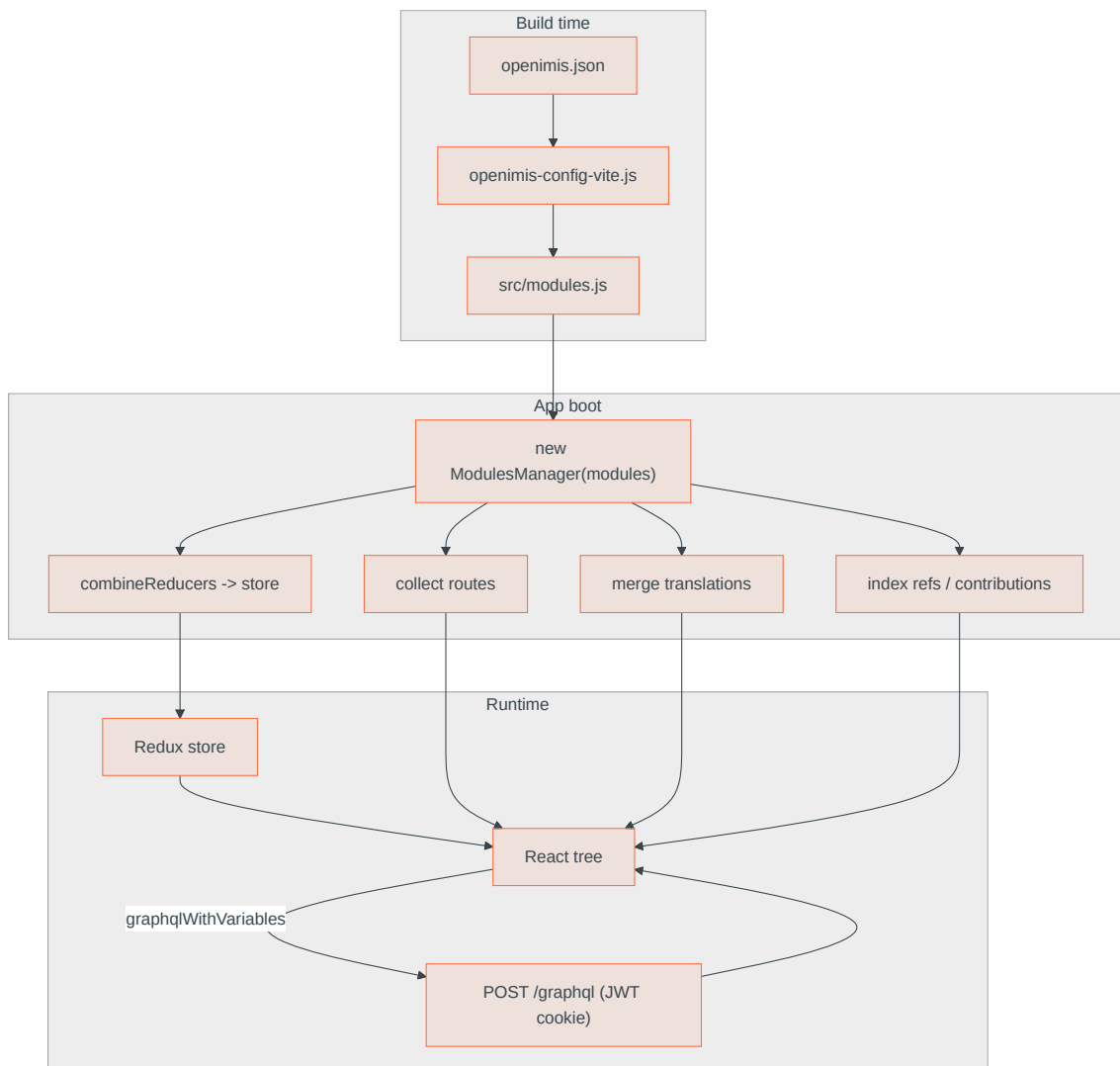
- Each module ships message catalogs (`translations/en.json`, `fr.json`, ...) and declares them under the `translations` config key.
- The shell merges all catalogs and provides them via `<IntlProvider locale=... messages=...>`.
- Components render text with `<FormattedMessage id="claim.submit" />` or the `formatMessage` API, using **namespaced** ids (`"<module>.<key>"`) to avoid collisions.



Common mistake

Hard-coding a visible English string (`<Button>Submit</Button>`) breaks localization for every non-English deployment and will fail review. Always add the string to your module's catalog and render it with a namespaced `react-intl` id.

11. Putting it together: the assembly, end to end



Repository references

Repository	Directory / File	Why it matters
<code>openimis-fe_js</code>	<code>openimis.json</code>	The frontend module manifest — which modules, which versions, compose this build.
<code>openimis-fe_js</code>	<code>openimis-config-vite.js</code>	Generator that reads the manifest and writes the module import file (<code>src/modules.js</code>).
<code>openimis-fe_js</code>	<code>vite.config.js</code>	Vite build config (replaced <code>craco.config.js</code> after the CRA → Vite migration).
<code>openimis-fe_js</code>	<code>src/index.js</code> , <code>src/App.js</code>	The shell: store wiring, router, <code>ThemeProvider</code> , <code>IntlProvider</code> .
<code>openimis-fe-core_js</code>	<code>src/helpers/api</code> (graphql helpers)	<code>graphql</code> / <code>graphqlWithVariables</code> Redux action creators — the custom GraphQL layer.
<code>openimis-fe-core_js</code>	<code>src/ModulesManager.js</code>	The cross-module registry: <code>getRef</code> , <code>getConfig</code> , <code>getContribs</code> .
<code>openimis-fe-core_js</code>	<code>src/reducers</code> , <code>journalize helpers</code>	The mutation journal + polling of <code>MutationLog</code> status.
<code>openimis-fe-core_js</code>	<code>src/components</code> (pickers, searchers, forms)	Published components other modules reuse via <code>getRef</code> .
<code>openimis-fe-<name>_js</code>	<code>src/index.js</code>	Each module's config object — its entire public API.
<code>openimis-fe-<name>_js</code>	<code>src/translations/*.json</code>	<code>react-intl</code> message catalogs.

Hands-on lab



Lab 11.1 — Trace the assembly and add a menu entry

1. Clone `openimis-fe_js` and open `openimis.json`. List the modules currently included and note their versions.
2. Run the dev server and find the **generated** module import file (`src/modules.js`). Confirm it imports exactly the modules from the manifest — then edit it and reload to prove your edit is overwritten.
3. In any business module, locate the config object in `src/index.js`. Identify one entry under a contribution key (e.g. `"insuree.MainMenu"`).
4. Add a new menu entry that links to an existing route. Reload and confirm it renders. You have now used a contribution point without touching the owning module.



Lab 11.2 — Follow a query into Redux

1. Open your browser's **Redux DevTools**. Navigate to a searcher page (e.g. claims).
2. Watch the dispatched actions: you should see a `_REQ` then a `_RESP` action with the GraphQL `data` payload.
3. Inspect the store slice the module wrote to (e.g. `state.claim`). Confirm the rendered table reads from exactly that slice.
4. Now submit an editable form. Watch for the mutation being registered in the journal, the polling queries, and the final completion action. Correlate it with a new row in the backend `MutationLog` (via GraphQL at `/graphql`).

Exercises

1. Explain, in two sentences, why `src/modules.js` is generated rather than written by hand.
2. Your new claims dashboard should let *other* modules add tiles. Describe the exact mechanism you would use.
3. A teammate imports `InsureePicker` directly from the insuree package. Give two concrete failure modes this can cause.
4. Why can an XSS bug not steal the openIMIS JWT?

Knowledge check

? Q1: openIMIS does not use Apollo Client. What does it use for GraphQL, and why does the mutation flow fit it? (click for answer) ✓

It uses a **custom Redux layer** — the `graphql / graphqlWithVariables` action creators that `POST` to `/graphql` and express request/success/error as Redux actions, with responses stored as Redux state. It fits because openIMIS mutations are **asynchronous**: they return a `clientMutationId` and complete out-of-band via `MutationLog`, which maps cleanly onto Redux (a pending mutation is state; a poller is a thunk) but fights Apollo's synchronous, result-returning cache model.

? Q2: What does `modulesManager.getContribs("insuree.MainMenu")` return, and who put things there? (click for answer) ✓

It returns **every entry any loaded module contributed** to the named injection point `"insuree.MainMenu"`. The insuree module *owns* the point and renders whatever it collects; other modules (e.g. claim) contribute menu entries to it via their config object — without importing the insuree module.

? Q3: You added `@openimis/fe-report` to the build but `getRef("report.SomeComponent")` returns undefined. Where do you look first? (click for answer) ✓

The **manifest**. Confirm the module is listed in `openimis.json` and that the package is installed/linked so the generator included it in `src/modules.js`. A missing `getRef` almost always means the owning module was never loaded, not that the component is broken.

? Q4: Why is the JWT kept in an `HttpOnly` cookie instead of `localStorage`? (click for answer) ✓

An `HttpOnly` cookie is not readable by JavaScript, so an XSS vulnerability cannot exfiltrate the token, yet the browser still attaches it automatically to every `POST /graphql`. Storing a token in `localStorage` would expose it to any injected script.



Q5: After the CRA → Vite migration, a ported module reads



`process.env.REACT_APP_API_URL` and gets undefined. Fix? (click for answer)

Vite exposes env vars as `import.meta.env.VITE_*`. Rename the variable to `VITE_API_URL` and read it via `import.meta.env.VITE_API_URL`. CRA-style `REACT_APP_*` / `process.env` are not populated under Vite.

Further reading

- [openimis-fe_js](#) — the assembly repo (manifest, generator, shell).
- [openimis-fe-core_js](#) — `ModulesManager`, the Redux GraphQL helpers, published components.
- [The GraphQL Layer](#) — GraphQL from first principles, backend and frontend.
- [Plugin / Module System](#) — the backend counterpart to this chapter.
- [Extension Guide](#) — build a new frontend module from scratch.
- [React](#), [Redux](#), [MUI](#), [react-intl](#), [Vite](#) official docs.

GraphQL from First Principles

Part 6 of the openIMIS Developer Academy. You already write Django models, serializers, and DRF viewsets. This chapter teaches you GraphQL from zero — what it is, why openIMIS bet its entire API surface on it, and how the `graphene-django` stack turns your ORM models into a single typed endpoint. By the end you will read, write, and reason about openIMIS queries and mutations with the same fluency you have with a DRF `ViewSet`.

Learning objectives

By the end of this chapter you will be able to:

- Explain what GraphQL is and articulate, point by point, how it differs from a REST/DRF API.
- Justify **why** openIMIS chose GraphQL over REST for its primary API.
- Trace how `graphene-django` builds a schema from Django models using `ObjectType`, Relay connections, and `OrderedDjangoFilterConnectionField`.
- Write resolvers and understand where permission checks and validation belong.
- Read and write openIMIS **queries** and, critically, the asynchronous `OpenIMISMutation / MutationLog mutation` pattern with `clientMutationId` polling.
- Diagnose and fix the **N+1 query problem** in resolvers.
- Follow a request end to end: `Client → /graphql → schema → resolver → service → ORM → DB`.

Prerequisites

This chapter assumes you have read:

- [Architecture Overview](#) — the modular backend.
- [Request Lifecycle](#) — how a request moves through the Django project.
- [Plugin / Module System](#) — how modules are assembled, because the GraphQL schema is assembled the same way.
- [Core module](#) — where `OpenIMISMutation`, `ExtendedConnection`, and the `User` model live.

You should be comfortable with Django models, the ORM, and DRF. You do **not** need any prior GraphQL knowledge — we build it up from nothing.

1. What is GraphQL?

GraphQL is a **query language for your API** plus a **runtime** that executes those queries against your data. It was created at Facebook and is now an open standard governed by the GraphQL Foundation.

Strip away the hype and there are exactly four ideas. If you internalize these four, everything else is detail.

1. **One endpoint, not many.** A REST API exposes dozens of URLs (`/api/insurees/`, `/api/insurees/42/`, `/api/policies/?insuree=42`). A GraphQL API exposes **a single URL** — in openIMIS, `/graphql` — and you describe what you want in the request body.
2. **The client specifies the fields.** In REST the *server* decides the shape of every response. In GraphQL the *client* writes a query that names exactly the fields it wants, and the response mirrors that shape — no more, no less.
3. **One round trip for a graph of data.** Because you can ask for related objects in the same query (an insuree *and* their policies *and* each policy's product), you replace a waterfall of REST calls with a single request.
4. **A typed schema is the contract.** Every field has a type. The schema is introspectable — tools can read it, validate queries against it, and generate documentation and autocompletion. There is no "read the wiki to find out what fields exist"; the schema *is* the documentation.

Did you know?

The "QL" in GraphQL is real: a GraphQL query is a string sent to the server, parsed, validated against the schema, and executed — much like SQL is a string sent to a database. The difference is that GraphQL resolves each field by calling a **resolver function** you (or the framework) wrote, so the "database" behind a GraphQL field can be a Django ORM query, a REST call to another service, or a computed value.

A first taste

Here is a REST interaction you already understand, followed by its GraphQL equivalent. Suppose you want an insuree's chosen name plus the products of each of their policies.

```
# 1. Fetch the insuree
GET /api/insurees/9a3f.../
# -> { "id": "9a3f...", "chfId": "100001", "otherNames": "Amina",
#      "lastName": "Diallo", "gender": "F", ...30 more fields you didn't want
#    }

# 2. Fetch their policies (separate endpoint, separate round trip)
GET /api/policies/?insuree=9a3f...
# -> [ { "id": "...", "product": 12, ... }, ... ]

# 3. For each policy, fetch the product to get its name (N more round trips!)
GET /api/products/12/

query {
  insurees(chfId: "100001") {
    edges {
      node {
        otherNames
        lastName
        policies {
          edges {
            node {
              product { code name }
            }
          }
        }
      }
    }
  }
}
```

One request. Exactly the fields named. The response is JSON shaped like the query (we show the full response in [§6](#)).

REST / DRF (multiple calls)

GraphQL (one call)

2. GraphQL vs REST — a side-by-side

You will get the most out of GraphQL by mapping each concept onto something you already know from DRF.

Concern	REST / DRF	GraphQL (graphene-django)
Endpoints	Many URLs, one per resource/action	One URL: <code>/graphql</code>
Who chooses response fields	Server (the serializer)	Client (the query)
Over-fetching	Common — you get all serializer fields	Eliminated — you name fields
Under-fetching	Common — extra calls for related data	Eliminated — traverse the graph in one query
Round trips for related data	Often N+1 network calls	Usually one
Type contract	OpenAPI/Swagger (bolted on, optional)	Built-in, introspectable schema
Versioning	<code>/api/v1</code> , <code>/api/v2</code>	Evolve the schema; deprecate fields with <code>@deprecated</code>
Reads	<code>GET</code> verbs	Query operations
Writes	<code>POST</code> / <code>PUT</code> / <code>PATCH</code> / <code>DELETE</code>	Mutation operations
List filtering	Query params + <code>django-filter</code>	Filter arguments on connection fields
Pagination	<code>?limit=&offset=</code> or page numbers	Relay cursor connections (<code>first</code> / <code>after</code>)
Error model	HTTP status codes (404, 400, 403)	HTTP 200 + an <code>errors</code> array in the body
Discovery	Read the docs	Introspect the schema (GraphQL autocompletes)



GraphQL usually returns HTTP 200 even on 'errors'

This trips up every REST engineer once. A GraphQL response is `200 OK` even when a field failed to resolve; the failure appears as an `errors` array alongside (possibly partial) `data`. Do not write client code that keys off the HTTP status alone. openIMIS layers its **own** business-error convention on top of this via `MutationLog` (see [§8](#)).

The two problems GraphQL solves by design

- **Over-fetching:** a REST endpoint returns a fixed, fat payload. A mobile enrolment app on a 2G connection in a rural district does not want 40 insuree fields to render a name badge. GraphQL lets it ask for two.
- **Under-fetching (the N+1 network problem):** a dashboard needs an insuree, their policies, and each product. REST forces a request waterfall. GraphQL collapses it into one typed request.

3. Why openIMIS chose GraphQL

openIMIS is not a single product with one UI. It is a **platform** deployed by many countries, each with different modules enabled, different rules, and often **country-specific frontends and mobile apps**. That context makes GraphQL a natural fit for three concrete reasons.

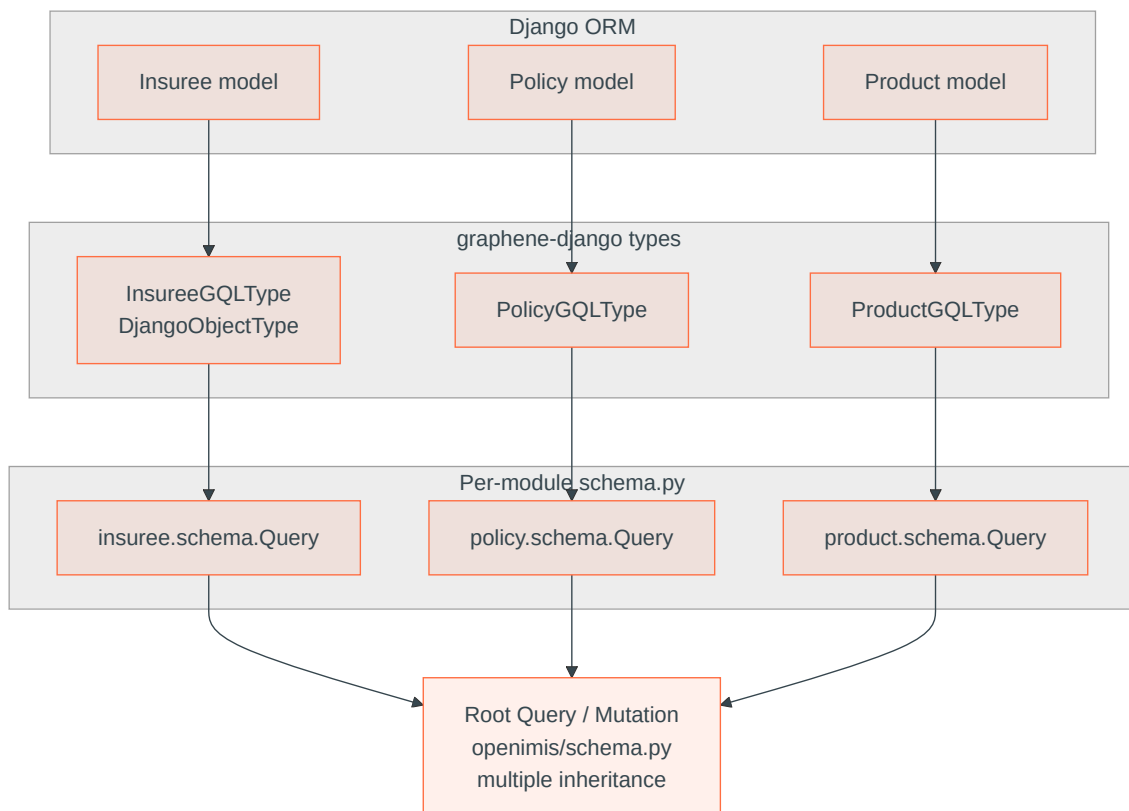
- **Heterogeneous, evolving clients.** Tanzania's UI, Nepal's UI, a claims mobile app, and an FHIR bridge all read the same data differently. A client-specified query language means each client fetches its own shape without the backend shipping a new endpoint per screen.
- **Over/under-fetching would be brutal here.** openIMIS models form a deep graph — insuree → family → policies → product → items/services → price lists. REST would force either enormous fixed payloads or chatty request waterfalls. Field selection and graph traversal solve both.
- **A modular schema that grows with the platform.** openIMIS has ~47 backend modules, and deployments enable different subsets. Because the root schema is **assembled** from each installed module's `Query` and `Mutation` (via Python multiple inheritance in `openimis/schema.py`), enabling a module *automatically extends the API*. There is no central API layer to edit. A single typed schema that different deployments compose differently is exactly what a plugin platform needs.

Did you know?

The openIMIS **frontend does not use Apollo Client**. It talks to `/graphql` through a thin custom layer in `openimis-fe-core_js` — Redux action creators called `graphql / graphqlWithVariables`, plus a "journalize"/polling helper that watches the asynchronous `MutationLog`. GraphQL the protocol does not oblige you to any particular client library; openIMIS chose Redux thunks over Apollo.

4. The schema architecture in openIMIS

openIMIS uses **graphene-django**, the Django integration for the Graphene GraphQL library. Here is the layered picture, from your Django model up to the single root schema.



4.1 ObjectTypes from Django models

A GraphQL **ObjectType** is the analogue of a DRF **serializer**: it declares which fields of a model are exposed and under what GraphQL types. With `graphene-django` you subclass `DjangoObjectType` and point it at a model.

```
# illustrative – faithful to openIMIS conventions (see openimis-be-insuree_py/
# insuree/gql_queries.py and openimis-be-core_py/core/schema.py)
import graphene
from graphene_django import DjangoObjectType
from core import ExtendedConnection
from core.schema import OrderedDjangoFilterConnectionField
from .models import Insuree

class InsureeGQLType(DjangoObjectType):
    class Meta:
        model = Insuree
        interfaces = (graphene.relay.Node,) ①
        connection_class = ExtendedConnection ②
        filter_fields = { ③
            "chf_id": ["exact", "istartswith", "icontains"],
            "last_name": ["exact", "icontains"],
            "gender__code": ["exact"],
        }
```

- ① Implementing the Relay `Node` interface gives every object a globally unique, opaque `id` and enables cursor-based pagination (connections). This is the Relay convention Graphene builds on.
- ② `connection_class = ExtendedConnection` swaps the default Relay connection for openIMIS's core subclass, which adds `totalCount` and `edgeCount` — you almost always want these for a paginated UI.
- ③ `filter_fields` is `django-filter` syntax. It declares which lookups become GraphQL filter arguments — the GraphQL equivalent of a DRF `FilterSet`.

Did you know?

Because `DjangoObjectType` reads your model's fields, adding a field to a model and exposing it in GraphQL is often a one-line change — much less ceremony than a matching REST serializer + viewset + URL route. This is a big part of why openIMIS modules can evolve their API quickly.

4.2 Relay-style connections and `ExtendedConnection`

When you make a type a Relay `Node` and expose it as a list, Graphene wraps the list in a **connection**. Instead of a bare JSON array you get:

```
Connection
├── edges: [ { node: <the object>, cursor: <opaque string> }, ... ]
└── pageInfo: { hasNextPage, hasPreviousPage, startCursor, endCursor }
```

This looks verbose, but it is what makes stable cursor pagination possible (covered in [§10](#)). openIMIS's core defines `ExtendedConnection` to add two fields every real UI needs:

```
# illustrative (see openimis-be-core_py/core/schema.py)
class ExtendedConnection(graphene.relay.Connection):
    class Meta:
        abstract = True

    total_count = graphene.Int() # total rows matching the filter
    edge_count = graphene.Int() # rows in THIS page

    def resolve_total_count(self, info, **kwargs):
        return self.length

    def resolve_edge_count(self, info, **kwargs):
        return len(self.edges)
```

`totalCount` lets the frontend render "Showing 25 of 3,410"; `edgeCount` is the size of the current page. Vanilla Relay connections give you neither.

4.3 OrderedDjangoFilterConnectionField

The field you actually put on a `Query` to expose a filterable, sortable, paginated list is openIMIS's `OrderedDjangoFilterConnectionField`. Think of it as `DjangoFilterConnectionField` (from `graphene-django`, which wires `filter_fields` into GraphQL arguments) **plus** an `orderBy` argument so clients can sort — the GraphQL analogue of DRF's `OrderingFilter`.

```
# illustrative (see openimis-be-insuree_py/insuree/schema.py)
class Query(graphene.ObjectType):
    insurees = OrderedDjangoFilterConnectionField(
        InsureeGQLType,
        orderBy=graphene.List(of_type=graphene.String), # e.g. ["lastName", "-chfId"]
        show_history=graphene.Boolean(),
    )

    def resolve_insurees(self, info, **kwargs):
        # permission check + queryset shaping happen here – see §9
        ...
```

Every openIMIS list query you meet — `insurees`, `policies`, `claims`, `locations` — is one of these fields. Learn this one field and you can read the query side of every module.

5. Resolvers

A **resolver** is the function that produces the value for a field. This is the single most important concept in GraphQL, so slow down here.

- Every field in the schema has a resolver.

- If you don't write one, Graphene supplies a **default resolver** that just reads the attribute off the parent object (`getattr(parent, field_name)`). For a `DjangoObjectType`, that means most scalar fields resolve for free by reading the model instance.
- You write an explicit resolver when a field needs logic: permissions, filtering the queryset, computing a value, or calling a service.

The resolver signature is `resolve_<field>(parent, info, **args)`:

```
# illustrative
def resolve_insurees(self, info, **kwargs):
    user = info.context.user           ❶
    Query._check_permissions(user)      ❷
    qs = Insuree.objects.filter(        ❸
        validity_to__isnull=True)
    return gql_optimizer.query(qs, info)  ❹
```

- ❶ `info.context` is the Django `HttpRequest`. So `info.context.user` is the authenticated Django user — the same object you'd read in a DRF view. This is the bridge between GraphQL and Django's auth. (`info` also carries the parsed query, useful for query optimization.)
- ❷ Permission enforcement lives in the resolver. See §9.
- ❸ `validity_to__isnull=True` is the openIMIS idiom for "the currently valid row" — most core models are **temporally versioned** (`validity_from` / `validity_to`), so you almost always filter to live records.
- ❹ `gql_optimizer.query(qs, info)` inspects the incoming query and applies `select_related` / `prefetch_related` automatically — the N+1 mitigation from §11.

Did you know?

The `self` (parent) argument of a root `Query` resolver is usually `None` — the root query has no parent object. For a *nested* field like `Insuree.policies`, the parent is the `Insuree` instance, which is how the resolver knows *whose* policies to return.

6. Your first query

Let's run a real query against openIMIS and see the response shape. Open the GraphQL IDE (typically served at `/graphql` in a dev build) and paste:


```
query InsureeWithPolicies {
  insurees(chfId_Iexact: "100001", first: 1) {
    totalCount
    edges {
      node {
        uuid
        chfId
        otherNames
        lastName
        gender { code }
        policies {
          edges {
            node {
              enrollDate
              expiryDate
              product { code name }
            }
          }
        }
      }
    }
  }
}
```

The response is JSON shaped exactly like the query:

```

{
  "data": {
    "insurees": {
      "totalCount": 1,
      "edges": [
        {
          "node": {
            "uuid": "9a3f8b12-4c7d-4e2a-bb90-2f0c1e6d7a55",
            "chfId": "100001",
            "otherNames": "Amina",
            "lastName": "Diallo",
            "gender": { "code": "F" },
            "policies": {
              "edges": [
                {
                  "node": {
                    "enrollDate": "2026-01-15",
                    "expiryDate": "2027-01-14",
                    "product": { "code": "BASIC", "name": "Basic Health Cover"

```

Notice three things a REST engineer should savour:

- You asked for `gender { code }` — a *nested* selection — and got exactly that, not the whole gender object.
- `policies` and `product` came back **in the same request**. No waterfall.
- `totalCount` came from `ExtendedConnection`.

Variables — the parameterized query

Never string-concatenate values into a query. GraphQL has **variables**, the equivalent of parameterized SQL:

```

query InsureeByChf($chf: String!, $limit: Int = 10) {
  insurees(chfId_Iexact: $chf, first: $limit) {
    edges { node { uuid lastName } }
  }
}

```

```

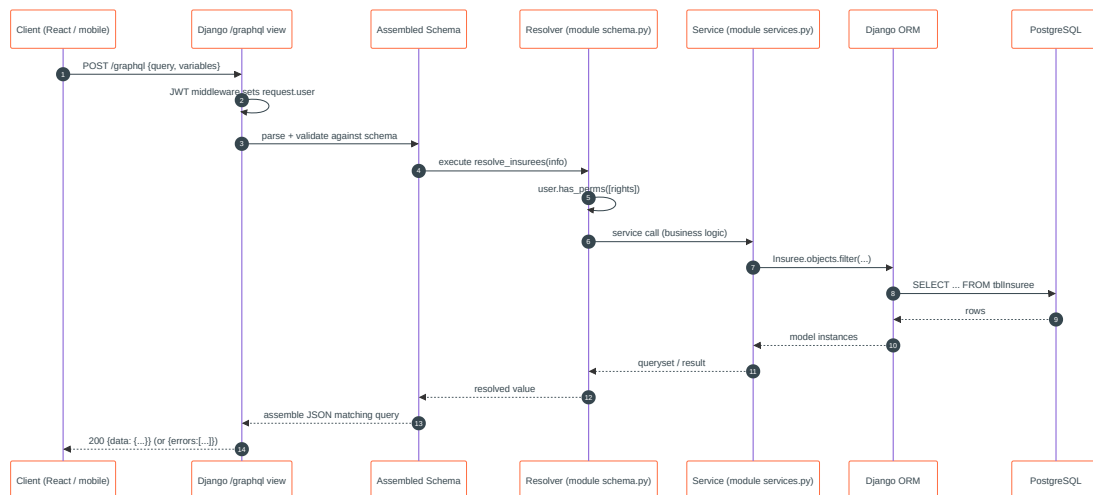
{ "chf": "100001", "limit": 5 }

```

The `String!` means "required string"; `Int = 10` gives a default. This is exactly what the frontend's `graphqlWithVariables` action creator sends.

7. The request lifecycle

Here is how a query travels through openIMIS, tying together everything from [Request Lifecycle](#) with the GraphQL layer.



The key insight: **GraphQL sits on top of your existing Django stack**. The `/graphql` view is just another Django view; `info.context` is the request; the resolver calls the same **services** and **ORM** a DRF view would. GraphQL changes the *shape of the contract*, not the layers beneath it. Business logic belongs in `services.py`, never in resolvers — the resolver is a thin authorization-and-shaping layer.

8. Mutations — the `OpenIMISMutation` pattern

Reads are queries; **writes are mutations**. A vanilla GraphQL mutation looks like a query that changes data and returns a result. openIMIS, however, wraps *all* writes in a distinctive **asynchronous, audited** pattern built on the core base class `OpenIMISMutation` and a `MutationLog` model. You must understand this pattern to do anything real in openIMIS, so we cover it thoroughly.

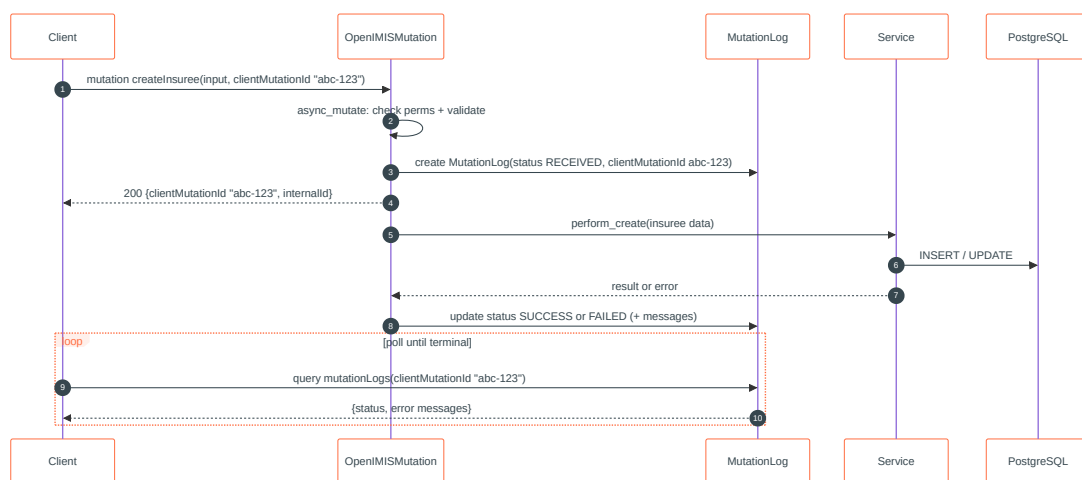
8.1 Why asynchronous?

Many openIMIS operations are heavy: enrolling a family, valuing a batch of claims through calculation rules, generating payments. Making the client block on an HTTP request for these is

fragile. So openIMIS mutations follow a **fire-and-poll** model:

1. The client sends the mutation. The server **validates**, creates a **MutationLog** row (status = *RECEIVED*), kicks off the work, and **immediately returns a clientMutationId** — an id the *client* generated to identify this operation.
2. The work proceeds (synchronously or via the scheduler), updating the **MutationLog** status to *SUCCESS* or *FAILED*, recording any error messages.
3. The client **polls** the **mutationLogs** query using its **clientMutationId** until the status is terminal, then reads the outcome.

This gives every write a durable **audit trail** (who, what, when, success or failure) and decouples slow work from the request.



8.2 What a mutation looks like

```
# illustrative – faithful to the OpenIMISMutation pattern
# (see openimis-be-core_py/core/schema.py and any module's gql_mutations/)
from core.schema import OpenIMISMutation
from .services import InsureeService
from .apps import InsureeConfig

class CreateInsureeMutation(OpenIMISMutation):
    _mutation_module = "insuree"
    _mutation_class = "CreateInsureeMutation"

    class Input(OpenIMISMutation.Input): ❶
        chf_id = graphene.String(required=True)
        last_name = graphene.String(required=True)
        other_names = graphene.String(required=True)
        gender_id = graphene.String(required=False)
        # clientMutationId + clientMutationLabel are inherited from the base
        Input

    @classmethod
    def async_mutate(cls, user, **data): ❷
        if not
user.has_perms(InsureeConfig.gql_mutation_create_insurees_perms):
            raise PermissionDenied(_("unauthorized"))
        try:
            service = InsureeService(user)
            service.create_or_update(data) ❸
            return None ❹
        except Exception as exc:
            return [{
                "message": _("insuree.mutation.failed_to_create_insuree"),
                "detail": str(exc),
            }]
    }
```

- ❶ Every mutation defines an inner `Input` subclassing `OpenIMISMutation.Input`, which already carries `clientMutationId` and `clientMutationLabel`. Inputs are the GraphQL equivalent of a DRF serializer's writable fields.
- ❷ You override `async_mutate`, not `mutate`. The base class handles creating the `MutationLog`, wrapping in a transaction, catching exceptions, and setting final status. Your job is: check permissions, call the service.
- ❸ **All business logic lives in the service**, not the mutation. The mutation is the authorization + input-marshalling boundary.
- ❹ Return `None` on success. Return a **list of error dicts** on failure — the base class writes them into the `MutationLog` for the client to poll.

8.3 Sending a mutation and polling

Step 1 — send the mutation (the client invents `clientMutationId`, here a UUID `abc-123`):

```
mutation CreateInsuree($input: CreateInsureeMutationInput!) {
  createInsuree(input: $input) {
    clientMutationId
    internalId
  }
}
```

```
{
  "input": {
    "clientMutationId": "abc-123",
    "clientMutationLabel": "Create insuree Amina Diallo",
    "chfId": "100042",
    "lastName": "Diallo",
    "otherNames": "Amina"
  }
}
```

Immediate response — note it does **not** contain the created insuree, only the handle:

```
{ "data": { "createInsuree": { "clientMutationId": "abc-123", "internalId":
"551" } } }
```

Step 2 — poll `mutationLogs` until status is terminal:

```
query PollMutation($id: String!) {
  mutationLogs(clientMutationId: $id) {
    edges {
      node {
        status      # 0 RECEIVED, 1 SUCCESS, 2 ERROR (values per core)
        error
        clientMutationId
      }
    }
  }
}
```

```
{
  "data": {
    "mutationLogs": {
      "edges": [
        { "node": { "status": 1, "error": "", "clientMutationId": "abc-123" }
      ]
    }
  }
}
```

Status `1` (SUCCESS) means the insuree now exists; the client re-queries `insurees` to display it. A non-empty `error` with a failure status tells the UI what went wrong. The frontend's "journalize" helper automates exactly this poll loop.



Common mistake: expecting the created object back synchronously

Coming from DRF, you expect `POST /insurees/` to return the created insuree in the same response. openIMIS mutations **do not** — they return a `clientMutationId`, and you must **poll** `mutationLogs` for success, then re-query the entity. Writing frontend code that reads `response.data.createInsuree.insuree.uuid` will fail: there is no such field. Always: send → poll `mutationLogs` → re-query.



Deep dive: transactions, signals, and where validation really happens



`OpenIMISMutation.mutate` (the method you do *not* override) wraps `async_mutate` so that: (a) the whole operation runs inside a database transaction — a failure rolls back cleanly; (b) `register_service_signal` hooks in *other* modules can fire **before/after** the service call, letting a country-specific module inject validation or side effects without touching the insuree module (the same extension seam described in the [Plugin / Module System](#)); and (c) the `MutationLog` is updated to its terminal status even if an exception is raised. Input-shape validation (required fields, types) is enforced by GraphQL **before** your resolver runs — a missing `chfId` is rejected by the schema. *Business* validation (is this CHF id unique? is the gender code valid?) belongs in the **service**, which raises exceptions that become `MutationLog` errors.

9. Permissions & authorization

openIMIS authorization is **integer rights** checked in resolvers and mutations. This is covered in depth in the [Authentication & Authorization](#) chapter; here is what you need for GraphQL specifically.

- Each module declares permission codes as **integers** in its `apps.py` `AppConfig` (e.g. `gql_query_claims_perms = [111001]`, `gql_mutation_create_insurees_perms = [101002]`).
- A role carries a set of rights; a user's rights are the union of their roles'.
- You enforce with Django's standard `user.has_perms([...])`, passing the integer codes:

```
# illustrative – the canonical guard at the top of a resolver/mutation
from django.core.exceptions import PermissionDenied
from django.utils.translation import gettext as _

def resolve_claims(self, info, **kwargs):
    user = info.context.user
    if user.is_anonymous or not
user.has_perms(ClaimConfig.gql_query_claims_perms):
    raise PermissionDenied(_("unauthorized"))
...
```





Common mistake: forgetting the permission check in a resolver

GraphQL has **no per-URL middleware** to lean on the way REST does. There is one endpoint. If a resolver or mutation omits its `has_perms` check, that data or operation is exposed to **any authenticated user**. Every resolver that returns protected data and every mutation must check rights explicitly — treat it as non-negotiable boilerplate, and grep the module for the config constant to confirm it is wired.

10. Pagination — Relay cursor connections

REST pagination is usually offset-based (`?limit=25&offset=50`). GraphQL/Relay uses **cursor-based** pagination, which is more stable when rows are being inserted concurrently. The connection arguments are:

Argument	Meaning	REST analogue
<code>first: N</code>	take the first N after the cursor	<code>limit</code> (forward)
<code>after: "<cursor>"</code>	start after this opaque cursor	<code>offset</code> (roughly)
<code>last: N</code>	take the last N before the cursor	<code>limit</code> (backward)
<code>before: "<cursor>"</code>	end before this cursor	—

A cursor is an **opaque** base64 string — do not parse it; just pass back the `endCursor` you were given. The pattern is: fetch a page, read `pageInfo.endCursor` and `pageInfo.hasNextPage`, then request the next page with `after: <endCursor>`.

```
query FirstPage {  
  insurees(first: 25, orderBy: ["lastName"]) {  
    totalCount          # from ExtendedConnection  
    pageInfo { hasNextPage endCursor }  
    edges { cursor node { uuid lastName } }  
  }  
}
```

Next page — reuse the `endCursor`:


```

query NextPage($cursor: String!) {
  insurees(first: 25, after: $cursor, orderBy: ["lastName"]) {
    pageInfo { hasNextPage endCursor }
    edges { node { uuid lastName } }
  }
}

```

i Did you know?

Because openIMIS filter fields, `orderBy`, and Relay pagination all live on the **same** `OrderedDjangoFilterConnectionField`, a single query can filter (`lastName_Icontains: "dia"`), sort (`orderBy: ["-chfId"]`), and paginate (`first: 25, after: ...`) at once — the whole DRF "filter + order + paginate" trio, expressed as arguments on one field.

11. Performance and the N+1 problem

This is the one place GraphQL can hurt you, and every openIMIS developer must understand it.

The problem

Recall the query that fetches insurees and, for each, their policies' products. The naive execution is:

1. One query to fetch the page of insurees. (1)
2. For **each** insuree, a query to fetch its policies. (N)
3. For **each** policy, a query to fetch its product. (N × M)

That is the **N+1 problem** (really $1 + N + N \times M$). It is the GraphQL cousin of the classic Django template `for obj in qs: obj.related.field` trap — except in GraphQL the *client* controls how deep the graph goes, so a well-meaning query can quietly detonate your database.



The mitigations

- `select_related` — for forward `ForeignKey` / `OneToOne` (a `JOIN`). Use for `policy.product`, `insuree.gender`.

- `prefetch_related` — for reverse FKs and `ManyToMany` (a second query, batched). Use for `insuree.policies`.
- `DataLoader` / `graphene-django-optimizer` — batches and caches per-request so nested resolvers don't each hit the DB. openIMIS resolvers commonly return `gql_optimizer.query(queryset, info)`, which reads the incoming GraphQL query and applies the right `select_related` / `prefetch_related` **automatically** based on the fields the client actually requested.

```
# illustrative – two ways to defuse N+1 in a resolver
def resolve_insurees(self, info, **kwargs):
    qs = Insuree.objects.filter(Validity__isnull=True)
    # Manual, explicit:
    qs = qs.select_related("gender").prefetch_related("policies__product")
    # ...or let the optimizer read the query and do it for you:
    return gql_optimizer.query(qs, info)
```



Common mistake: shipping a resolver with no query optimization

A resolver that returns a bare `Model.objects.filter(...)` will *work* in tests with three rows and *melt* in production when a client requests a deep nested selection over thousands of rows. Before you consider a list resolver done, ask: "what happens when the client selects a nested connection?" If the answer isn't `select_related` / `prefetch_related` / `gql_optimizer`, it's not done. Watch `django.db.connection.queries` (or Django Debug Toolbar) during a nested query to see the count.



Deep dive: what a DataLoader actually does



A `DataLoader` sits between resolvers and the database for the life of one request. When ten sibling resolvers each ask "give me product #X", the loader doesn't fire ten `SELECT`s — it **collects the keys within a tick of the event loop**, issues one `SELECT ... WHERE id IN (...)`, and hands each caller its row from an in-memory cache. It solves batching (one query instead of N) and caching (the same key isn't fetched twice). `graphene-django-optimizer` gives you much of this benefit declaratively for ORM-backed types; hand-written `DataLoader`s are for cases the optimizer can't infer.

Repository references

Repository	Directory / File	Why it matters
openimis-be_py	openimis/schema.py	Assembles the root <code>Query / Mutation</code> from every module via multiple inheritance — the whole GraphQL surface.
openimis-be_py	openimis/urls.py	Mounts the single <code>/graphql</code> endpoint (and REST/FHIR routes).
openimis-be_py	openimis/settings.py	Graphene + <code>graphql_jwt</code> configuration; builds <code>INSTALLED_APPS</code> dynamically.
openimis-be-core_py	core/schema.py	Defines <code>OpenIMISMutation</code> , <code>ExtendedConnection</code> , <code>OrderedDjangoFilterConnectionField</code> , <code>MutationLog</code> .
openimis-be-core_py	core/apps.py	<code>CoreConfig</code> with <code>DEFAULT_CFG</code> and integer permission codes.
openimis-be-insuree_py	insuree/gql_queries.py, insuree/schema.py	A canonical <code>DjangoObjectType + Query + OrderedDjangoFilterConnectionField</code> .
openimis-be-insuree_py	insuree/gql_mutations/	Real <code>OpenIMISMutation</code> subclasses to model your own on.
<any module>	services.py	Where business logic belongs — resolvers/mutations call these.
openimis-fe-core_js	src/actions.js (graphql helpers)	The <code>graphql / graphqlWithVariables</code> Redux action creators + the polling "journalize" helper.

Hands-on lab

Lab: read, write, and diagnose

Spin up a dev backend (see [Set Up a Dev Environment](#)) and open GraphQL at `/graphql`.

1. **Explore the schema.** In GraphQL, open the Docs panel and find the `insurees` query. Note its filter arguments and that it returns an `InsureeGQLTypeConnection` with `totalCount`.
2. **Field selection.** Write a query returning only `chfId` and `lastName` for the first 5 insurees. Then add `otherNames`. Observe the response grow to match the query — you changed the payload with no backend change.
3. **Traverse the graph.** Extend the query to include each insuree's `policies { edges { node { product { code } } } }`. One request, nested data.
4. **Paginate.** Add `first: 5` and select `pageInfo { hasNextPage endCursor }`. Copy the `endCursor`, run a second query with `after:` , and confirm you get the next 5.
5. **Mutate + poll.** Send `createInsuree` with a `clientMutationId` you choose. Capture the returned id, then run the `mutationLogs` poll query until `status` is terminal. Finally re-query `insurees` to see your row.
6. **Catch an N+1.** Enable query logging (or Django Debug Toolbar) and run the deep nested query from step 3 against a seeded dataset. Count the SQL queries. Now imagine the resolver used `gql_optimizer.query(qs, info)` — predict how the count changes.

Exercises

1. Rewrite the REST waterfall from [§1](#) as a single GraphQL query and count the round trips saved.
2. Given the mutation in [§8.2](#), add a new optional input field `phone` and describe where its **validation** should live (hint: not the mutation).
3. A resolver returns `Claim.objects.filter(Validity__isnull=True)` with no permission check and no optimization. List the **two** distinct defects and the fix for each.

Knowledge check

 **Q1: In GraphQL, who decides which fields appear in a response, and how does that differ from DRF? (click for answer)** 

The **client** decides, by naming fields in the query. In DRF the **server** decides via the serializer. This is what eliminates over-fetching: the response is shaped exactly like the request.

? **Q2: Why does openIMIS return a `clientMutationId` instead of the created object? (click for answer)** ✓

Because mutations are **asynchronous and audited**. `OpenIMISMutation` creates a `MutationLog`, does the work in a service, and returns the client-supplied `clientMutationId` immediately. The client **polls** `mutationLogs` for the terminal status, then re-queries the entity. This gives every write a durable audit trail and decouples slow work from the request.

? **Q3: What does `ExtendedConnection` add over a vanilla Relay connection, and why do UIs need it? (click for answer)** ✓

`totalCount` (total rows matching the filter) and `edgeCount` (rows on the current page). UIs need `totalCount` to render "Showing 25 of 3,410" and to build page controls; vanilla Relay connections expose neither.

? **Q4: You add a nested connection to a query and your DB melts. What happened and name two fixes. (click for answer)** ✓

The **N+1 problem**: each parent triggers a separate query for its children (1 + N, and worse when nested). Fixes: `select_related` (forward FK/O2O joins), `prefetch_related` (reverse FK / M2M batching), and/or `gql_optimizer.query(qs, info)` / `DataLoaders` to batch per request.

? **Q5: Why is forgetting a `has_perms` check more dangerous in GraphQL than in a REST API? (click for answer)** ✓

REST has many URLs and can lean on per-route middleware/guards. GraphQL has **one endpoint**, so there is no per-URL choke point — authorization must be enforced **inside each resolver and mutation** with `user.has_perms([codes])`. An omitted check exposes that field/operation to any authenticated user.

Further reading

- [Authentication & Authorization](#) — rights, roles, and how `info.context.user` gets populated by JWT.
- [Request Lifecycle](#) — the Django layers beneath `/graphql`.
- [Plugin / Module System](#) — how the schema is assembled from modules and how service signals extend mutations.
- [Core module](#) — `OpenIMISMutation`, `ExtendedConnection`, `MutationLog` reference.

- Official **GraphQL** learning site: <https://graphql.org/learn/>.
- **Graphene-Django** documentation: <https://docs.graphene-python.org/projects/django/>.
- **Relay** connection specification: <https://relay.dev/graphql/connections.htm>.
- **graphene-django-optimizer** (N+1 mitigation): the project README on GitHub.
- **openIMIS** GitHub organization: <https://github.com/openimis>.

Business Modules — The Domain Model at a Glance

openIMIS is not one program; it is a **fleet of independently versioned Django apps** (modules) assembled at deploy time into a single backend. The [plugin/module system](#) explains *how* they are wired together. This section of the Academy explains *what each module means* — the business domain of social health protection, expressed as models, services and GraphQL.

Learning objectives

By the end of this chapter you will be able to:

- Name the core business modules and state the one responsibility of each.
- Read the **module dependency graph** and predict which module a given concept lives in.
- Trace the canonical data-flow of a scheme: **reference data** → **beneficiary** → **coverage** → **contribution** → **claim** → **payment**.
- Distinguish the **classic health-insurance stack** (`insuree` + `policy`) from the **newer generic beneficiary registry** (`individual` + `social_protection`).

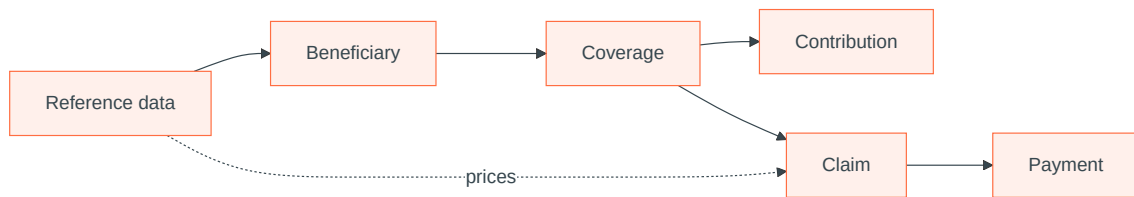
Prerequisites

- [Plugin / Module System](#) — how modules are discovered, installed and combined.
- [GraphQL in openIMIS](#) — the query/mutation layer every module exposes.
- [Database & Legacy Heritage](#) — the `tbl` -prefixed legacy schema these modules sit on.

The domain in one sentence

A **scheme operator** enrolls **beneficiaries**, sells them **coverage** under a priced **product**, collects **contributions**, receives **claims** from health facilities for services rendered, values those claims with pricing rules, and **pays** the providers.

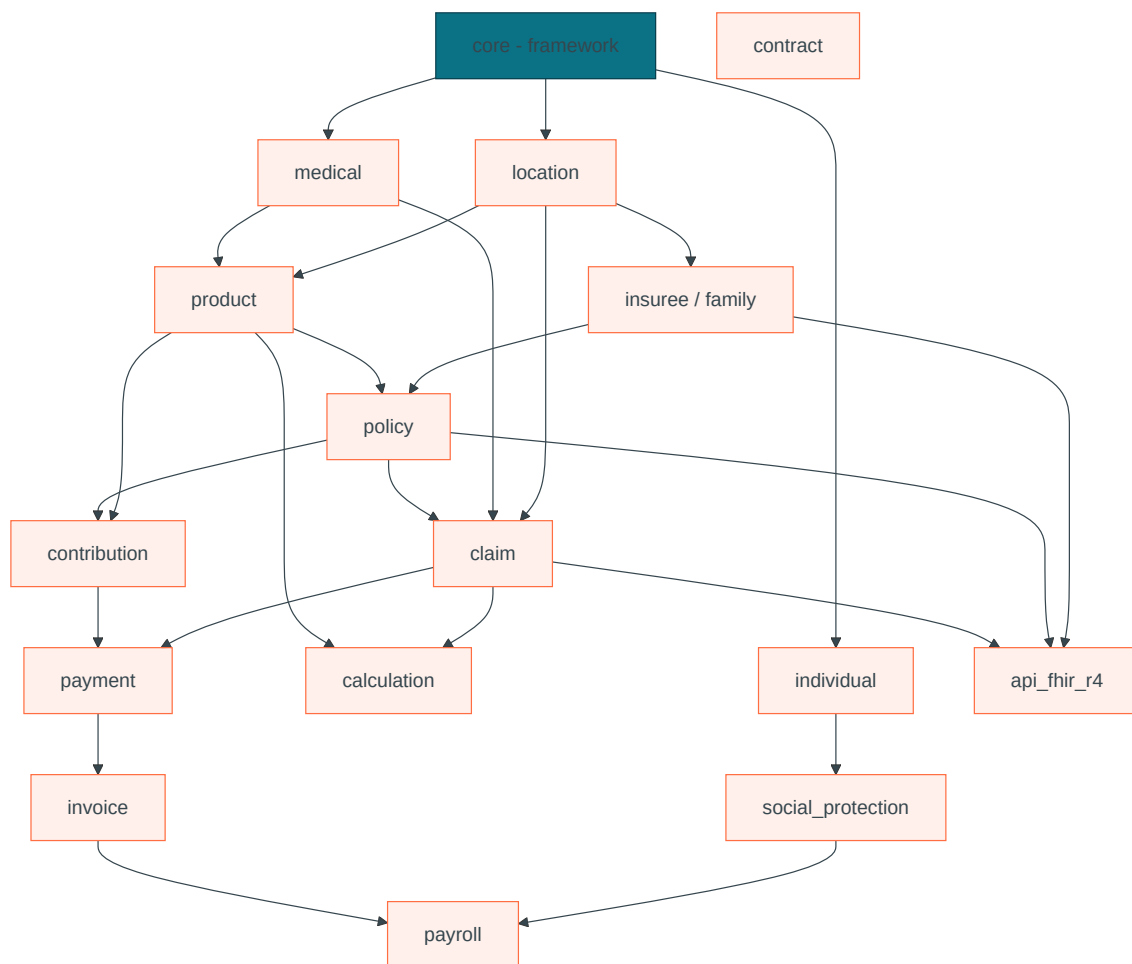
Every business module is a noun (or verb) in that sentence. Learn the sentence and you have learned the map.



Stage	Question it answers	Primary modules
Reference data	Where, who provides care, what is covered, at what price?	<code>location</code> , <code>medical</code> , <code>product</code>
Beneficiary	Who is the person / household?	<code>insuree</code> , <code>family</code> . <code>individual</code> , <code>group</code>
Coverage	Is this person insured, for which product, until when?	<code>policy</code> , <code>social_protection</code>
Contribution	What premium was paid against that coverage?	<code>contribution</code> , <code>contract</code>
Claim	What service was delivered, and what is it worth?	<code>claim</code> + <code>calculation</code>
Payment	Who gets paid, how much, and how?	<code>payment</code> , <code>invoice</code> , <code>payroll</code>

Module dependency graph

`core` is the framework everyone stands on. Reference-data modules sit above it; beneficiary, coverage, contribution and claim modules layer on top; integrations (`api_fhir_r4`) sit at the very top exposing everything.



i Did you know?

The arrows are **hard Python import + FK dependencies**, but modules almost never call each other's *services* by direct import. They subscribe to **service signals** instead. That is why you can swap the beneficiary layer (`insuree` → `individual`) without rewriting `claim`.

The two beneficiary worlds

openIMIS grew up as **health insurance** software. Its original beneficiary model is the household-centric `insuree / family` pair, tightly bound to `policy` and `contribution`. Later, donors needed openIMIS to run **cash-transfer and social protection** programmes that are *not* health insurance at all. Rather than bend the insurance models, the community added a **generic beneficiary registry** — `individual / group` — plus `social_protection` (benefit plans, beneficiaries, projects, payrolls).

Concern	Classic health-insurance stack	Generic social-protection stack
Person	<code>Insuree</code>	<code>Individual</code>
Household	<code>Family</code>	<code>Group</code> + <code>GroupIndividual</code>
Enrolment product	<code>Product</code> (insurance product)	<code>BenefitPlan</code> / programme
Coverage record	<code>Policy</code> + <code>InsureePolicy</code>	<code>Beneficiary</code> / <code>GroupBeneficiary</code>
Legacy DB heritage	<code>tblInsuree</code> , <code>tblFamilies</code> (MSSQL era)	new PostgreSQL tables, no <code>tbl</code> prefix
Extensibility	fixed columns + <code>json_ext</code>	schema-light, <code>json_ext</code> -first

Read [Insuree](#) and [Individual & Social Protection](#) back-to-back — the contrast is the single most important architectural story in the platform.

Module directory

Module page	Repository	One-line purpose
Core	<code>openimis-be-core_py</code>	The framework: base models, User, GraphQL helpers, mutations, signals, scheduler, calculation hook.
Insuree	<code>openimis-be-insuree_py</code>	Classic beneficiaries: <code>Insuree</code> , <code>Family</code> , photos, genders, enrolment.
Individual & Social Protection	<code>openimis-be-individual_py</code> , <code>openimis-be-social_protection_py</code>	Generic person/group registry and benefit-plan programmes beyond health insurance.
Policy	<code>openimis-be-policy_py</code>	Coverage: links a family/insuree to a product for a period; states and renewals.
<code>location</code>	<code>openimis-be-location_py</code>	Regions, districts, municipalities, villages, health facilities.
<code>medical / product</code>	<code>openimis-be-medical_py</code> , <code>openimis-be-product_py</code>	Medical items & services; insurance products and price lists.
<code>contribution</code>	<code>openimis-be-contribution_py</code>	Premiums (contributions/premiums) recorded against policies.
<code>claim</code>	<code>openimis-be-claim_py</code>	Service delivery by facilities, valued via calculation rules.
<code>calculation</code>	<code>openimis-be-calculation_py</code>	Pluggable pricing/valuation rules (<code>calcrule_*</code>).
<code>payment / invoice</code>	<code>openimis-be-payment_py</code> , <code>openimis-be-invoice_py</code>	Provider payment and billing artefacts.
<code>api_fhir_r4</code>	<code>openimis-be-api_fhir_r4_py</code>	FHIR R4 REST façade over the models above.



How to read a module page

Every module chapter follows the same skeleton — Purpose, Key models, Services, GraphQL, Dependencies, Signals, Business workflow, Database tables, Configuration, Extension points, Common mistakes, Exercises, Knowledge check. Once you have read [Core](#), the others will feel familiar because they all inherit the same base classes and mutation pattern.

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be_py</code>	<code>openimis.json</code>	The module manifest — the authoritative list of which business modules ship in a deployment.
<code>openimis-be_py</code>	<code>openimis/settings.py</code>	Builds <code>INSTALLED_APPS</code> from the manifest, so module load order mirrors the dependency graph.
<code>openimis-be_py</code>	<code>openimis/schema.py</code>	Merges every module's <code>schema.Query</code> / <code>schema.Mutation</code> into one root schema.
<code>openimis-be-core_py</code>	<code>core/models.py</code>	The base classes (<code>HistoryModel</code> , <code>VersionedModel</code> , <code>UUIDModel</code>) every business model below extends.

Further reading

- Official wiki: <https://openimis.atlassian.net/wiki/spaces/OP/overview>
- Module manifest: https://github.com/openimis/openimis-be_py/blob/develop/openimis.json
- GitHub org (all module repos): <https://github.com/openimis>

Knowledge check

? Q1: In the canonical flow, which module sits *between* coverage and payment on the claim side? (click for answer) ✓

`claim` — a health facility raises a claim against an active `policy`; the claim is valued by `calculation` rules and then settled by `payment / invoice`.

? Q2: Why does swapping `insuree` for `individual` not force a rewrite of `claim`? (click for answer) ✓

Because modules communicate through `service signals` and configuration rather than hard cross-service imports, so the claim workflow depends on an *abstraction*, not on the concrete beneficiary class.

? Q3: Which two modules make up the newer generic beneficiary registry, and what problem did it solve? (click for answer) ✓

`individual` and `social_protection`. They let openIMIS run non-health-insurance programmes (cash transfers, social protection) without distorting the health-insurance-specific `insuree / policy` models.

Core — The Framework Beneath Every Module

`openimis-be-core_py` is **not a business domain**. It ships almost no health-insurance concepts of its own. Instead it is the *framework* that every other module builds on: the base model classes, the custom `User`, the GraphQL plumbing, the asynchronous mutation pattern, the signal-based extension seam, the calculation hook, the scheduler and the audit machinery.

If you only read one module chapter, read this one. Every table below is inherited, extended or invoked by `insuree`, `policy`, `individual` and the rest.

Learning objectives

- Explain the four base model families — `UUIDModel`, `VersionedModel`, `HistoryModel`, `HistoryBusinessModel` — and when each is used.
- Read a temporal (`validity_from` / `validity_to`) row and reason about *why* openIMIS keeps history this way.
- Describe the custom `User` model and its three "faces": `InteractiveUser`, `TechnicalUser`, `Officer`.
- Use the Graphene helpers (`ExtendedConnection`, `OrderedDjangoFilterConnectionField`) that every module's schema relies on.
- Trace an `OpenIMISMutation` from client call to `MutationLog` status.
- Wire a **service signal** to extend another module without importing it.

Prerequisites

- [Plugin / Module System](#) — how `core` is loaded first and exposes shared code to every other app.
- [GraphQL in openIMIS](#) — Graphene basics.
- [Database & Legacy Heritage](#) — the legacy schema the base models straddle.

Purpose

`core` exists to give ~47 independently developed modules a **shared vocabulary and shared plumbing** so they interoperate and stay upgradeable:

- **Base models** that encode openIMIS's two hard requirements: *nothing is ever hard-deleted* (temporal history) and *every record straddles a legacy integer key and a new UUID*.
 - **A custom authentication/authorization model** unifying human logins, service accounts and field officers under integer **rights**.
 - **GraphQL helpers** so every module's schema looks and paginates the same way.
 - **OpenIMISMutation** — one asynchronous, always-audited mutation pattern.
 - **Signals** — the seam that lets modules extend each other without imports.
 - **Cross-cutting services**: the calculation-rule hook, the APScheduler-based scheduler, audit, and a large `utils` toolbox.
-

Key models

openIMIS models come in a small number of base flavours. Understanding the inheritance is 80% of understanding any module.

Model / base class	Key fields	Legacy table
UUIDModel (abstract)	id = UUIDField primary key	— (used by newer, non-legacy tables)
VersionedModel (abstract)	validity_from, validity_to (datetime), legacy_id (int, self-FK to prior version), audit_user_id	straddles legacy tbl* tables
HistoryModel (abstract)	id = UUID, json_ext (JSONField), date_created, date_updated, user_created, user_updated, is_deleted, version (int)	new-style tables (no tbl prefix)
HistoryBusiness Model (abstract)	HistoryModel + date_valid_from, date_valid_to	new-style business entities (e.g. BenefitPlan)
User	username, t_user (FK), i_user (FK), officer (FK), uuid	tblUsers
InteractiveUser	login_name, last_name, other_names, email, password, role_ids, private_key	tblUsers (interactive)
TechnicalUser	username, email, password (Django-style service account)	core_TechnicalUser
Officer	code, last_name, other_names, phone, location (FK)	tblOfficer
Role / RoleRight / UserRole	name, integer right_id (permission codes), validity fields	tblRole, tblRoleRight, tblUserRole
MutationLog	id (UUID), client_mutation_id, status, error, request, user (FK), autogenerated_code	core_Mutation_Log
ModuleConfiguration	module, version, config (JSON), layer, is_disabled_until	core_ModuleConfiguration

Did you know? Two keys per row

Legacy IMIS used integer primary keys in Microsoft SQL Server. When openIMIS added offline mobile clients and cross-database sync, it needed globally unique keys — so most legacy tables carry **both** an integer `id` and a `uuid` column. `VersionedModel` also keeps `legacy_id`, a pointer to the previous version of the same logical record.

Temporal history: `VersionedModel`

The single most surprising thing for a Django engineer new to openIMIS: **updates do not overwrite rows, and deletes do not remove them.** A `VersionedModel` row is valid over a time window `[validity_from, validity_to)`. "Deleting" sets `validity_to = now()`. "Editing" closes the current row and inserts a new one whose `legacy_id` points back at the old one.

```
# Illustrative – simplified from core/models.py conventions
class VersionedModel(models.Model):
    validity_from = models.DateTimeField(db_column="ValidityFrom",
default=py_datetime.now)
    validity_to = models.DateTimeField(db_column="ValidityTo", blank=True,
null=True)
    legacy_id = models.IntegerField(db_column="LegacyID", blank=True,
null=True)

    def delete_history(self):
        # "delete" == close the validity window, never DROP the row
        self.validity_to = py_datetime.now()
        self.save()

    class Meta:
        abstract = True
```

The practical consequence: **almost every query in openIMIS filters `validity_to__isnull=True`** to get "current" rows. Managers and the GraphQL filter fields do this for you; forget it in raw ORM and you will see ghosts of deleted records.

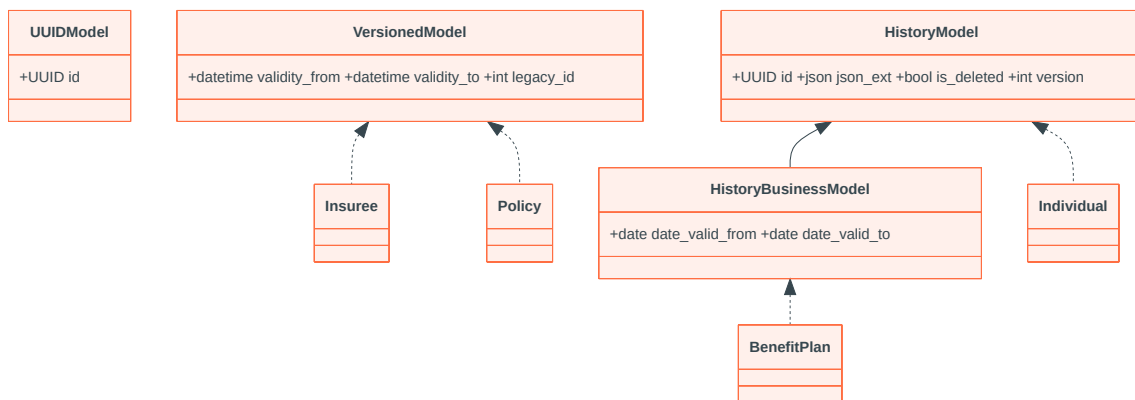
Common mistake: querying without the validity filter

```
Insuree.objects.filter(chf_id="12345")           # returns every historical version!
Insuree.objects.filter(chf_id="12345", validity_to__isnull=True) # correct: current row
```

Prefer the module's provided manager/queryset which applies the filter, or the GraphQL layer which applies it automatically.

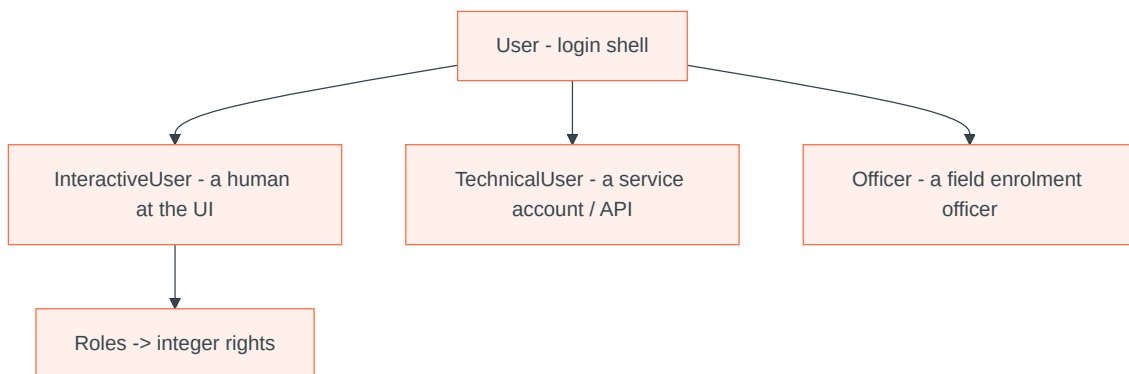
HistoryModel — the modern approach

Newer modules (`individual` , `social_protection`) use `HistoryModel` instead. It keeps history in a **separate history table** (via `simple_history` -style tracking) rather than the legacy in-table window, uses a **UUID primary key**, an `is_deleted` soft-delete flag, an integer `version` counter for optimistic locking, and a first-class `json_ext` JSONField for country-specific extension data.



The User model and its three faces

openIMIS authentication is unusual: the `User` row is a thin join that points at **one of three** identity types.



- **InteractiveUser** (`i_user`): a person logging into the React frontend. Carries `role_ids` ; roles expand to integer **rights** checked with `user.has_perms([121702])` .
- **TechnicalUser** (`t_user`): a Django-style service account for API/bot access.
- **Officer** : a field enrolment officer, referenced by policies and claims.

Did you know? Permissions are integers

openIMIS permission codes are **integers**, defined per module in `apps.py` (e.g. `gql_mutation_create_users_perms = ["121702"]`). Resolvers and mutations call `user.has_perms(...)`. There is no Django `Permission` string like `app.add_model`. See [Configuration](#) below.

Services

`core` ships framework services, not domain services, plus base classes other modules subclass.

Service / helper	Location	Responsibility
<code>create_or_update_interactive_user</code> , <code>user services</code>	<code>core/services.py</code>	Provision and update users, roles, rights.
<code>BaseService</code> / CRUD service base	<code>core/services.py</code>	Common create/update/delete-with-validation pattern reused by newer modules.
<code>register_service_signal</code> / <code>bind_service_signal</code>	<code>core/service_signals.py</code>	The signal-based extension seam (see Signals).
<code>ExtendedConnection</code> , <code>OrderedDjangoFilterConnectionField</code> , <code>PrefixFilterSet</code>	<code>core/schema.py</code>	Graphene helpers reused by every module schema.
<code>OpenIMISMutation</code>	<code>core/schema.py</code>	Base class for all mutations.
Scheduler bootstrap	<code>core/scheduler.py</code>	Registers APScheduler jobs modules contribute.
Calculation-rule registry hook	via signals + <code>openimis-be-calculation.py</code>	Lets pricing rules plug into valuation.

GraphQL

Every module's `schema.py` imports these helpers, so learning them once pays off everywhere. If you are new to GraphQL, read [GraphQL in openIMIS](#) first — here we cover only the openIMIS-specific pieces.

ExtendedConnection and the filtered connection field

Relay "connections" are GraphQL's cursor-paginated lists. openIMIS extends the standard connection to also report counts, and provides a filter+order field.

```
# Illustrative – the pattern every module repeats in schema.py
class InsureeGQLType(DjangoObjectType):
    class Meta:
        model = Insuree
        interfaces = (graphene.relay.Node,)
        connection_class = ExtendedConnection      # adds totalCount,
        edgeCount
        filter_fields = {"chf_id": ["exact", "icontains"], "last_name":
        ["icontains"]}

    class Query(graphene.ObjectType):
        insurees = OrderedDjangoFilterConnectionField( # filter + orderBy +
        validity handling
            InsureeGQLType, orderBy=graphene.List(of_type=graphene.String)
        )
```

- `ExtendedConnection` adds `totalCount` and `edgeCount` to results.
- `OrderedDjangoFilterConnectionField` merges Django-filter filtering with an `orderBy` argument and applies the `validity_to__isnull=True` "current rows" filter for versioned models.

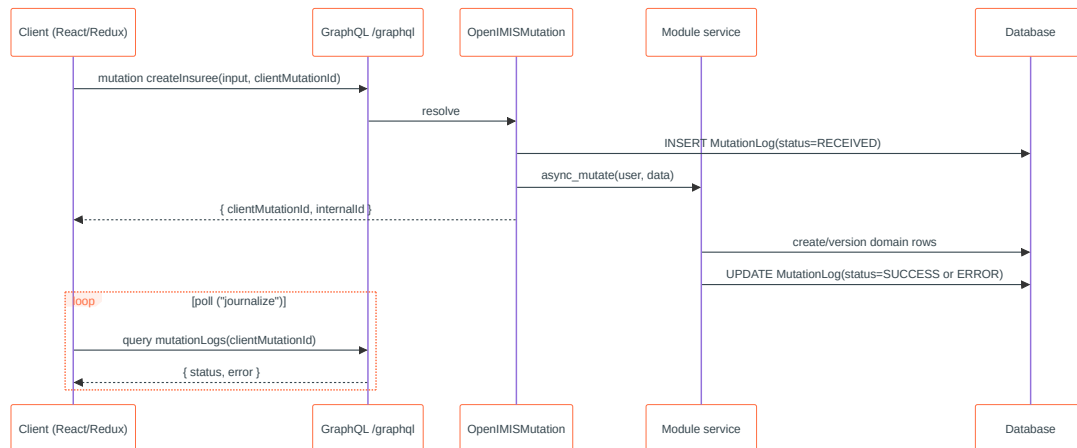
```
query {
  insurees(chfId_Icontains: "123", first: 10, orderBy: ["lastName"]) {
    totalCount
    edges { node { uuid chfId lastName } }
  }
}
```

OpenIMISMutation — the asynchronous, audited mutation pattern

This is the platform's signature pattern and it surprises everyone. A mutation does **not** synchronously return the created object. Instead it:

1. Creates a `MutationLog` row (status `RECEIVED`).
2. Runs the real work in a **service**, often in the background if `async_mutations` is on.
3. Returns a `clientMutationId` immediately.

4. The client **polls** the `MutationLog` by that id until status becomes `SUCCESS` (2) or `ERROR` (1).



```
# Illustrative – the shape of a module mutation
class CreateInsureeMutation(OpenIMISMutation):
    _mutation_module = "insuree"
    _mutation_class = "CreateInsureeMutation"

    class Input(OpenIMISMutation.Input):
        chf_id = graphene.String(required=True)
        last_name = graphene.String(required=True)

    @classmethod
    def async_mutate(cls, user, **data):
        if not
user.has_perms(InsureeConfig.gql_mutation_create_insurees_perms):
            raise PermissionDenied("unauthorized")
        # ... call the service; return None on success or a list of error
dicts
        return None
```

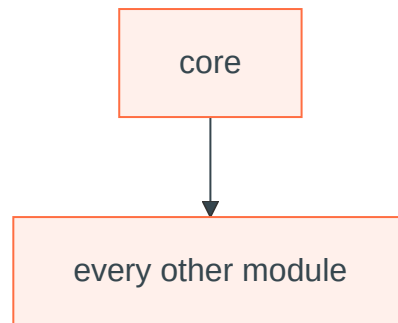
i Did you know? Why async?

Field officers work **offline** on mobile devices and later sync large batches. The `async+ MutationLog` pattern gives every write a durable, auditable, retrievable record — essential when connectivity is intermittent. On the frontend, Redux's "journalize" helper does the polling for you.

Dependencies

`core` depends on **no other openIMIS module** — it is the root of the graph. It depends on Django, `graphene-django`, `django-graphql-jwt`, `APScheduler` and `pandas / cryptography`

utilities. **Everything else depends on `core`**.



Signals

`core` provides **two** signalling mechanisms. Django's ORM signals (`post_save` , etc.) are used for local reactions. The openIMIS-specific **service signals** are the real extension seam.

Service signals — extend a service without importing it

A module *registers* a signal around a service method; other modules *bind* before/after handlers. The caller never imports the subscriber.

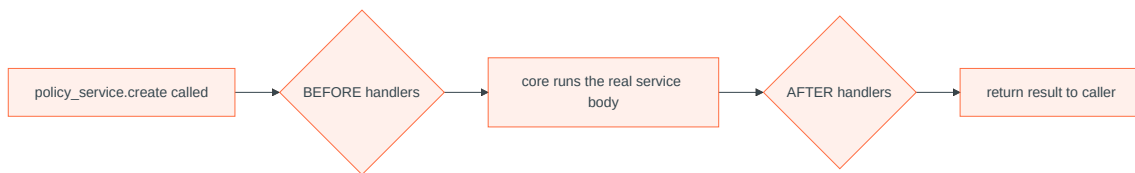
```
# In the OWNER module (illustrative)
from core.service_signals import register_service_signal

@register_service_signal("policy_service.create")
def create(self, policy_data):
    ...

# In a DIFFERENT module that wants to react
from core.signals import bind_service_signal
from core.service_signals import ServiceSignalBindType

def notify_after_policy_create(sender, **kwargs):
    result = kwargs.get("result")
    # send SMS, push to FHIR, etc.

bind_service_signal(
    "policy_service.create",
    notify_after_policy_create,
    bind_type=ServiceSignalBindType.AFTER,
)
```

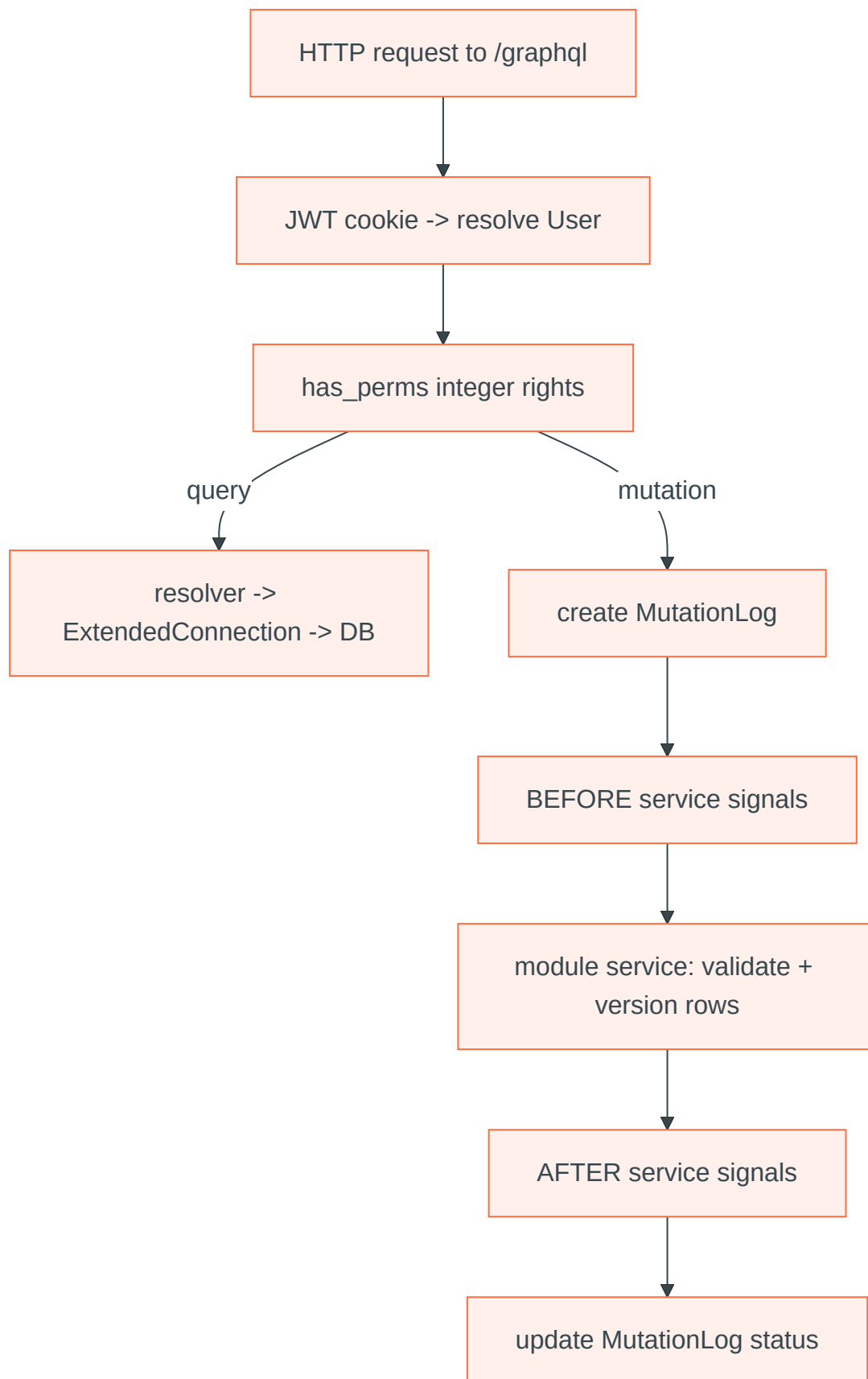


i Did you know?

The **calculation-rule framework** and the FHIR/DHIS2 integrations both plug in almost entirely through service signals. That is how `calculation` can price a claim without `claim` importing any specific pricing rule.

Business workflow

`core` has no domain workflow, but it owns the **request/mutation lifecycle** that every domain workflow runs inside.



Database tables

Table	Backed by	Notes
<code>tblUsers</code>	<code>User</code> / <code>InteractiveUser</code>	Legacy login table; interactive users.
<code>core_TechnicalUser</code>	<code>TechnicalUser</code>	New-style service accounts.
<code>tblOfficer</code>	<code>Officer</code>	Field enrolment officers.
<code>tblRole</code> , <code>tblRoleRight</code> , <code>tblUserRole</code>	<code>Role</code> , <code>RoleRight</code> , <code>UserRole</code>	Roles carry integer <code>right_id</code> permission codes.
<code>core_Mutation_Log</code>	<code>MutationLog</code>	One row per mutation; the audit + polling backbone.
<code>core_ModuleConfiguratio</code> <code>n</code>	<code>ModuleConfiguratio</code> <code>n</code>	Per-module JSON config overlay (see below).

See [Database & Legacy Heritage](#) for the `tbl` naming story.

Configuration

Each module declares a `DEFAULT_CFG` dict in its `AppConfig` (`core/apps.py` → `class CoreConfig`). At startup, core reads a matching row from `ModuleConfiguration` in the DB and **overlays** it, so operators reconfigure a deployment without touching code.

Notable `CoreConfig.DEFAULT_CFG` keys (see `openimis-be-core_py/core/apps.py`):

Key	Default	Meaning
<code>currency</code>	<code>"\$"</code>	Display currency symbol.
<code>async_mutations</code>	<code>True (prod)</code>	Whether mutations run in the background with polling.
<code>age_of_majority</code>	<code>"18"</code>	Adult threshold used across modules.
<code>username_code_length</code>	<code>"12"</code>	Generated username length.
<code>auto_provisioning_user_group</code>	<code>"user"</code>	Group assigned to auto-provisioned (OIDC) users.
<code>calendar_module / datetime_module</code>	<code>AD calendar</code>	Pluggable calendar (openIMIS supports e.g. Nepali calendar).
<code>gql_query_users_perms</code>	<code>["121701"]</code>	Integer permission codes for the users query.
<code>gql_mutation_create_users_perms</code>	<code>["121702"]</code>	Permission to create users.
<code>password_reset_template</code>	<code>"password_reset.txt"</code>	Email template.

Did you know? Pluggable calendars

`calendar_module / datetime_module` let a country swap the Gregorian (AD) calendar for another — Nepal runs openIMIS on the **Bikram Sambat** calendar. That is a `core` feature, configured, not forked.

Extension points

Seam	How you use it
Service signals	<code>register_service_signal</code> / <code>bind_service_signal</code> to hook before/after any registered service.
DEFAULT_CFG + ModuleConfiguration	Override behaviour per deployment without code changes.
Base model subclassing	Extend <code>HistoryModel</code> / <code>VersionedModel</code> in your own module to inherit history, <code>json_ext</code> , audit for free.
json_ext	Store country-specific fields without a migration.
Scheduler	Register APScheduler jobs from any module's <code>apps.ready()</code> .
Calculation rules	Register a <code>calcrule_*</code> module to price contributions/claims.
Graphene inheritance	Add your module's <code>Query</code> / <code>Mutation</code> ; <code>openimis/schema.py</code> merges them via multiple inheritance.

Common mistakes



Forgetting the validity filter

Querying a `VersionedModel` without `validity_to__isnull=True` returns historical versions. Use the provided managers or the GraphQL layer.



Expecting a mutation to return the created object synchronously

`OpenIMISMutation` returns a `clientMutationId`; you must poll `MutationLog`. Blocking on an immediate object breaks the offline-sync model.



Using Django string permissions

openIMIS rights are **integers** checked with `user.has_perms([code])`. Do not reach for `@permission_required("app.add_x")`.



Hard-deleting rows

Never `.delete()` a versioned record in a way that drops the row — close its validity window (`delete_history`) so history and offline sync stay intact.

Hands-on lab

1. Clone `openimis-be-core_py` and open `core/models.py`. Locate `VersionedModel` and `HistoryModel`; list every field each contributes.
2. In `core/schema.py`, find `ExtendedConnection` and confirm the extra fields it adds to a Relay connection.
3. In `core/apps.py`, print the `DEFAULT_CFG` dict. Pick three integer permission codes and trace where they are enforced in another module's mutations.
4. Grep the codebase for `register_service_signal()` and `bind_service_signal()` to see the extension seam in the wild.

Exercises

- **E1.** Write an illustrative `HistoryModel` subclass `Vehicle` with a `json_ext` field, and explain what happens in the history table on update vs. delete.
- **E2.** Given a mutation `createFoo`, sketch the sequence of `MutationLog` status transitions a client would observe.
- **E3.** You must send an SMS whenever any policy is created, but you may not edit the `policy` module. Which core mechanism do you use, and write the `bind_service_signal` call.

Knowledge check

? Q1: What does 'deleting' a VersionedModel row actually do? (click for answer) ✓

It sets `validity_to = now()`, closing the validity window. The row remains in the table so history and offline sync are preserved.

? Q2: Name the three identity types a `User` can point to. (click for answer) ✓

`InteractiveUser` (human UI login), `TechnicalUser` (service account) and `Officer` (field enrolment officer).

? Q3: Why does `OpenIMISMutation` return a `clientMutationId` instead of the object? (click for answer) ✓

Because mutations are asynchronous and audited via `MutationLog`, supporting intermittent connectivity and offline batch sync; the client polls the log for `SUCCESS / ERROR`.

? Q4: How can module B react to module A's service call without importing A? (click for answer) ✓

A registers the method with `register_service_signal`; B binds a BEFORE/AFTER handler with `bind_service_signal`. No direct import is needed.

? Q5: Where do a module's default settings live, and how are they overridden per deployment? (click for answer) ✓

In the module `AppConfig`'s `DEFAULT_CFG`, overlaid at startup by a `ModuleConfiguration` JSON row in the database.

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be-core_py</code>	<code>core/models.py</code>	<code>VersionedModel</code> , <code>HistoryModel</code> , <code>User</code> , <code>MutationLog</code> , <code>ModuleConfiguration</code> .
<code>openimis-be-core_py</code>	<code>core/schema.py</code>	<code>ExtendedConnection</code> , <code>OrderedDjangoFilterConnectionField</code> , <code>OpenIMISMutation</code> .
<code>openimis-be-core_py</code>	<code>core/service_signals.py</code>	<code>register_service_signal</code> / <code>bind_service_signal</code> seam.
<code>openimis-be-core_py</code>	<code>core/apps.py</code>	<code>CoreConfig.DEFAULT_CFG</code> and permission codes.
<code>openimis-be-core_py</code>	<code>core/scheduler.py</code>	APScheduler bootstrap for background jobs.
<code>openimis-be_py</code>	<code>openimis/schema.py</code>	Merges each module's <code>Query</code> / <code>Mutation</code> into one schema.

Further reading

- Core repo: https://github.com/openimis/openimis-be-core_py
- Graphene-Django docs: <https://docs.graphene-python.org/projects/django/en/latest/>
- `django-graphql-jwt` : <https://django-graphql-jwt.domake.io/>
- openIMIS wiki — architecture: <https://openimis.atlassian.net/wiki/spaces/OP/overview>

Insuree — Classic Beneficiaries and Households

`openimis-be-insuree_py` is where the platform's oldest and most central business concept lives: **the insured person and their household**. It is a direct descendant of legacy IMIS, and it wears that heritage openly — its tables are still `tblInsuree` and `tblFamilies`, with camelCase columns from the Microsoft SQL Server era.

Learning objectives

- Model a household with `Family`, its `head_insuree`, and member `Insuree` rows.
- Explain the `Insuree ↔ Family ↔ InsureePolicy` relationships.
- Read the legacy `tblInsuree` / `tblFamilies` schema and know why the columns look the way they do.
- Enrol an insuree and understand where coverage (`policy`) takes over.
- Locate the reference tables (`Gender`, `Relation`, `FamilyType`).

Prerequisites

- [Core](#) — `VersionedModel`, temporal history, `OpenIMISMutation`.
 - [Modules overview](#) — where beneficiaries sit in the domain flow.
 - [Database & Legacy Heritage](#) — the `tbl` prefix story.
-

Purpose

The insuree module answers "**who is this person, and which household do they belong to?**" It owns:

- The **person** (`Insuree`) with identity number (`chf_id`), demographics and status.
- The **household** (`Family`), which is the unit that actually holds a `policy` — coverage attaches to the family, members inherit it.
- **Enrolment** — creating/updating families and members, photos, and the join to policies (`InsureePolicy`).

- The small **reference tables** (gender, relation, family type) that the rest of the domain reads.

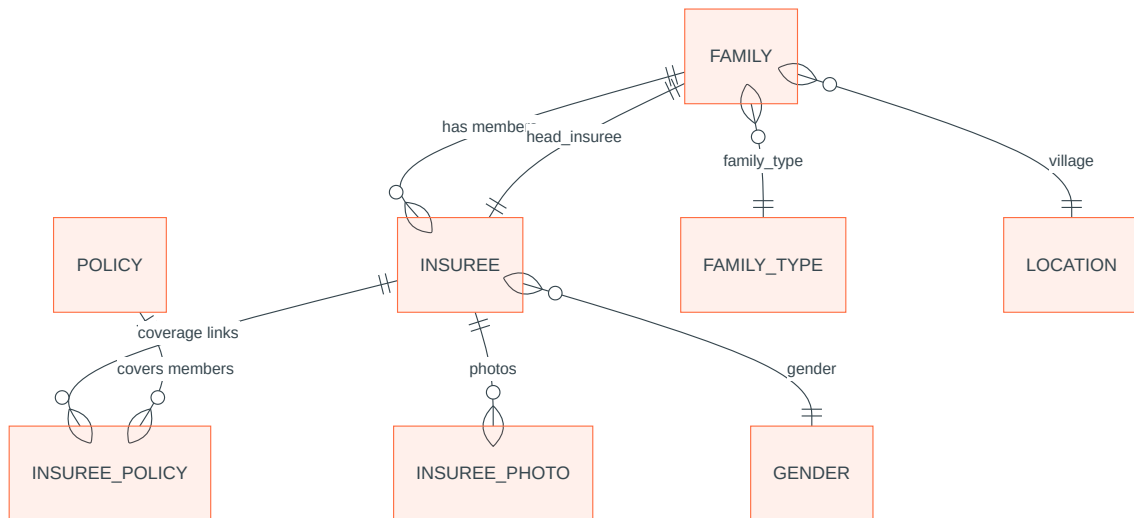
i Did you know? The household is the unit of coverage

In classic openIMIS a **policy is bought by a Family, not an individual**. Each member then gets an `InsureePolicy` row linking them to that policy for a validity window. This household-centric design is exactly what the newer `individual` registry generalises away from.

Key models

Model	Key fields	Legacy table
Insuree	uuid, chf_id (insurance number), last_name, other_names, dob, gender (FK), status, family (FK), head (bool), card_issued, json_ext	tblInsuree
Family	uuid, head_insuree (FK → Insuree), location (FK), family_type (FK), address, poverty, confirmation_no, json_ext	tblFamilies
InsureePolicy	insuree (FK), policy (FK), enrollment_date, start_date, effective_date, expiry_date	tblInsureePolicy
InsureePhoto	uuid, insuree (FK), photo / filename / folder, date	tblPhotos
Gender	code (PK, e.g. M / F), gender, sort_order	tblGender
Relation	id, relation, sort_order	tblRelations
FamilyType	code (PK), type, sort_order	tblFamilyTypes

All the "real" entities (`Insuree`, `Family`, `InsureePolicy`, `InsureePhoto`) extend `VersionedModel` — so they carry `validity_from` / `validity_to` / `legacy_id` and follow the "never overwrite, never hard-delete" rule. `Insuree` and `Family` also mix in an `ExtendableModel` for `json_ext`.



⚠ The head insuree is a chicken-and-egg

`Family.head_insuree` points at an `Insuree`, but every `Insuree.family` points back at a `Family`. Creating a household means creating the family and its head together in one service call — do not try to create the head insuree before its family exists in isolation.

Services

Domain logic lives in `insuree/services.py`, not in the GraphQL resolvers.

Service	Responsibility
<code>FamilyService</code> (create/update/delete)	Create a household with its head member; version rows on edit.
<code>InsureeService</code> (create/update/delete)	Add/edit a member; validate <code>chf_id</code> ; attach photo.
<code>validate_insuree_number</code>	Apply the configurable insurance-number rules (length, modulo checksum).
<code>InsureePolicyService</code> / <code>create_insuree_policy</code>	Materialise <code>InsureePolicy</code> rows when a family's policy activates.
Reset / family transfer helpers	Move an insuree between families, reset head, etc.

GraphQL

The module's `schema.py` exposes queries and `OpenIMISMutation` subclasses using the [core helpers](#).

Queries

Query	Returns
<code>insurees</code>	Filtered, paginated <code>Insuree</code> connection.
<code>families</code>	Filtered, paginated <code>Family</code> connection.
<code>insureePolicies</code> / <code>policiesByInsuree</code>	Coverage links for a person.
<code>insureeGenders</code> , <code>insureeRelations</code> , <code>familyTypes</code>	Reference lists.

Mutations

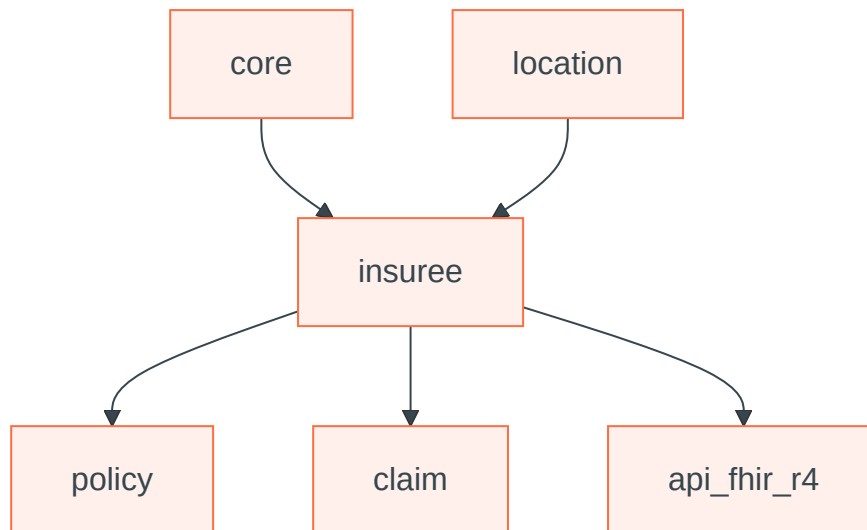
Mutation	Effect
<code>createInsuree</code> / <code>updateInsuree</code> / <code>deleteInsuree</code>	Manage members.
<code>createFamily</code> / <code>updateFamily</code> / <code>deleteFamily</code>	Manage households.
<code>removeInsurees</code> / <code>setInsureeFamilyHead</code>	Membership changes.

```
query {  
  families(location_Uuid: "...", first: 5) {  
    totalCount  
    edges { node {  
      uuid poverty  
      headInsuree { chfId lastName otherNames }  
      insurees { edges { node { chfId dob } } }  
    } }  
  }  
}
```



Permissions are integer codes from `InsureeConfig` — e.g. `gql_query_insurees_perms = ["101101"]` — checked with `user.has_perms(...)`.

Dependencies

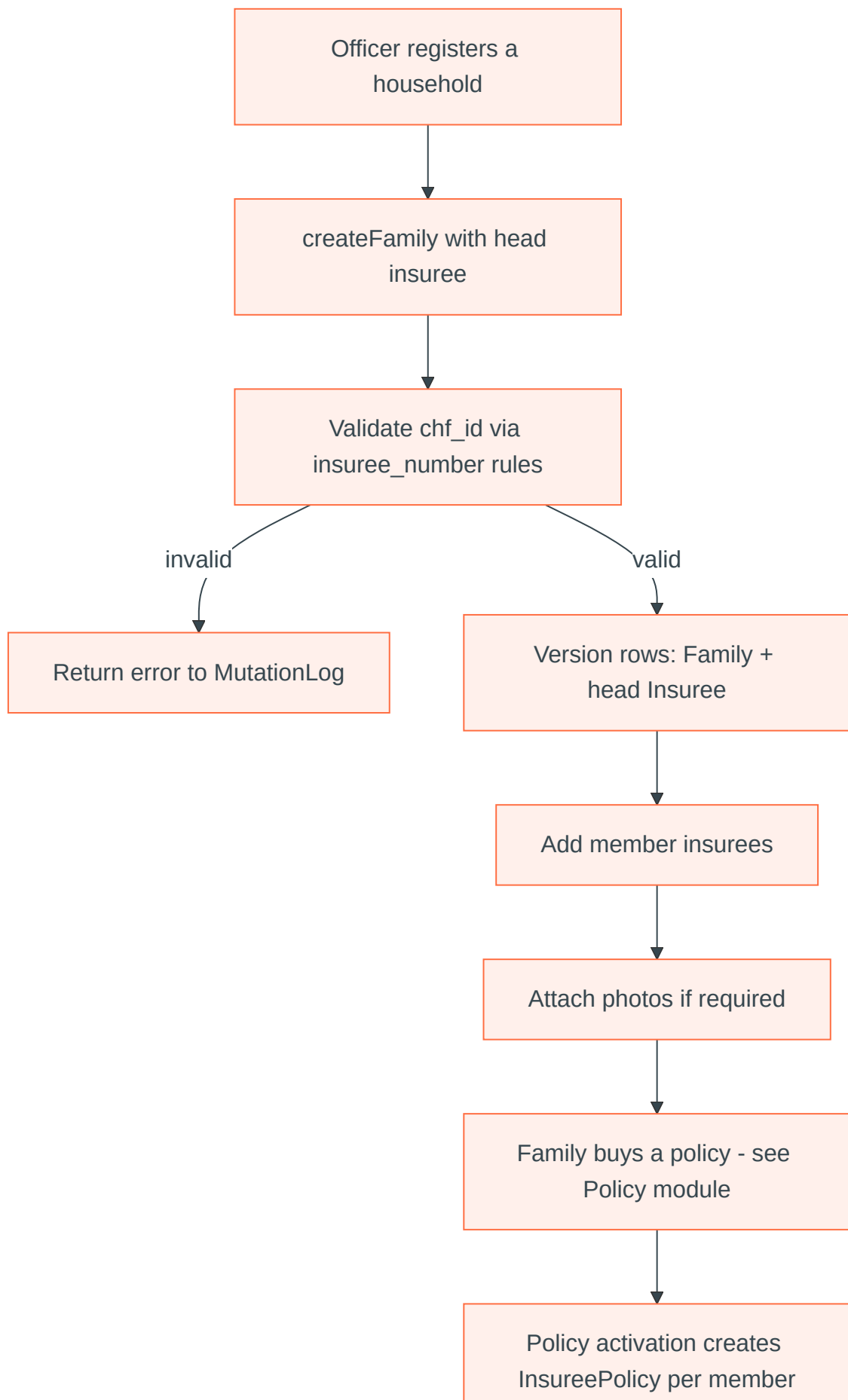


- **Needs:** `core` (base models, mutations, user), `location` (a family lives in a village/district).
 - **Needed by:** `policy` (a policy covers a family), `claim` (a claim is for an insuree), `contribution`, and `api_fhir_r4` (`Insuree` → `FHIR Patient`).
-

Signals

- Standard Django `post_save` signals maintain derived data and photo files.
 - Service methods are wrapped with `core service signals` so other modules react to enrolment — e.g. `api_fhir_r4` mapping an `Insuree` to a `FHIR Patient`, or a validation module checking the insurance number.
 - `Insuree/family` creation feeds the policy activation path that produces `InsureePolicy` rows.
-

Business workflow — enrolment



The insuree module stops at the household. The moment coverage is involved, control passes to `policy`, which creates the `InsureePolicy` links that make each member "covered".

Database tables

Table	Model	Heritage
<code>tblInsuree</code>	<code>Insuree</code>	Direct from legacy IMIS; camelCase columns, integer <code>InsureeID</code> + <code>uuid</code> .
<code>tblFamilies</code>	<code>Family</code>	Legacy household table.
<code>tblInsureePolicy</code>	<code>InsureePolicy</code>	Legacy per-member coverage link.
<code>tblPhotos</code>	<code>InsureePhoto</code>	Photo metadata; files on disk under the configured photos path.
<code>tblGender</code> , <code>tblRelations</code> , <code>tblFamilyTypes</code>	reference models	Small lookup tables seeded at install.

Did you know? Reading the legacy schema

Columns like `InsureeID`, `CHFID`, `LastName`, `DOB`, `ValidityFrom` are the original MSSQL names carried into PostgreSQL via `db_column=`. Django model field names are snake_case (`chf_id`, `last_name`), but the physical columns keep their 2010-era casing. See [Database heritage](#).

Configuration

Notable `InsureeConfig.DEFAULT_CFG` keys (see `openimis-be-insuree_py/insuree/apps.py`):

Key	Default	Meaning
<code>gql_query_insurees_perms</code>	<code>["101101"]</code>	Permission to list insurees.
<code>insuree_number_validator</code>	<code>None</code>	Optional custom callable for <code>chf_id</code> validation.
<code>insuree_number_max_length</code> <code>/ _min_length</code>	<code>None</code>	Length constraints on the insurance number.
<code>insuree_number_modulo_root</code>	<code>None</code>	Checksum modulo base.
<code>insuree_photos_root_path</code>	<code>"./images/insurees"</code>	Where photo files are stored (env-overridable).
<code>is_insuree_photo_required</code>	<code>False</code>	Whether a photo is mandatory to enrol.
<code>renewal_photo_age_adult</code> / <code>_child</code>	<code>60 / 12</code> (months)	When a fresh photo is required at renewal.
<code>excluded_insuree_chfids</code>	<code>['999999999']</code>	Placeholder numbers to ignore.
<code>insuree_as_worker</code>	<code>False</code>	Treat insuree as a worker (for social-protection blends).

Extension points

Seam	Use
<code>insuree_number_validator</code>	Plug a country-specific insurance-number rule (length + checksum).
<code>json_extension</code> <code>Insuree / Family</code>	Add country fields without a migration.
Service signals	React to enrolment (FHIR export, custom validation, notifications).
<code>ModuleConfiguration</code>	Reconfigure photo rules, permissions, etc. per deployment.

Common mistakes

Treating the insuree as the coverage unit

Coverage attaches to the **Family**; members get `InsureePolicy` links. Looking for a policy directly on an `Insuree` misses the model.

Ignoring validity when counting members

`family.insurees` includes historical versions unless you filter `validity_to__isnull=True`. Prefer the module's managers or GraphQL.

Hardcoding gender codes

`Gender` is a reference table (`M`, `F`, plus deployment-specific values). Do not assume a fixed set.

Hands-on lab

1. In `openimis-be-insuree_py/insuree/models.py`, list every field on `Insuree` and mark which come from `VersionedModel` vs. the model itself.
2. Run the `families` GraphQL query above against a dev instance and inspect `totalCount` vs. `edges` length.
3. Set `is_insuree_photo_required = True` in a local `ModuleConfiguration` and observe the enrolment mutation rejecting a photo-less insuree.

Exercises

- **E1.** Draw the ER diagram for a 4-member family and its policy links.
- **E2.** Write a `chf_id` validator (modulo-11 checksum) and describe where to register it via `insuree_number_validator`.
- **E3.** Explain what happens to `tblInsuree` rows when an insuree is edited twice, in terms of `validity_from/validity_to/legacy_id`.

Knowledge check

? Q1: Which entity actually holds a policy — the insuree or the family? (click for answer) ✓

The `Family`. Members receive `InsureePolicy` links when the family's policy activates.

? Q2: What are the legacy table names for insuree and family? (click for answer) ✓

`tblInsuree` and `tblFamilies` — carried over from legacy IMIS on MSSQL.

? Q3: How do you validate a country-specific insurance number without editing the module? (click for answer) ✓

Register a callable via the `insuree_number_validator` config key (with optional length/module settings).

? Q4: Why do the physical columns look like `CHFID` and `ValidityFrom` ? (click for answer) ✓

They are the original MSSQL/IMIS column names preserved via Django `db_column=` ; only the Python field names were modernised to snake_case.

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be-insuree_py</code>	<code>insuree/models.py</code>	<code>Insuree</code> , <code>Family</code> , <code>InsureePolicy</code> , <code>InsureePhoto</code> , reference models.
<code>openimis-be-insuree_py</code>	<code>insuree/services.py</code>	Enrolment services and number validation.
<code>openimis-be-insuree_py</code>	<code>insuree/schema.py</code> , <code>insuree/gql_mutations/</code>	Queries and <code>OpenIMISMutation</code> subclasses.
<code>openimis-be-insuree_py</code>	<code>insuree/apps.py</code>	<code>InsureeConfig.DEFAULT_CFG</code> and permission codes.

Further reading

- Insuree repo: https://github.com/openimis/openimis-be-insuree_py
- Compare with the generic registry: [Individual & Social Protection](#)
- Coverage that consumes families: [Policy](#)

Individual & Social Protection — The Generic Beneficiary Registry

openIMIS began life as **health insurance** software, and its original beneficiary model (`insuree` / `family`) is welded to that world: households buy policies, members inherit coverage, everything is `tbl` -prefixed legacy schema. But donors increasingly asked openIMIS to run programmes that are **not health insurance at all** — unconditional cash transfers, disability grants, food assistance, public-works wages. Those programmes have *people* and *households* too, but no "policy", no "product", no "claim".

Rather than distort the insurance models, the community built a **second, generic beneficiary registry**: `openimis-be-individual_py` (the registry) and `openimis-be-social_protection_py` (the programmes on top of it).

Learning objectives

- Explain **why** a generic registry exists alongside the classic insuree module.
- Model people and households with `Individual` , `Group` , `GroupIndividual` .
- Describe **benefit plans / programmes** and how `Beneficiary` links a person to one, with a lifecycle status.
- Map the classic concepts onto the generic ones and know when to reach for each.
- Recognise the modern `HistoryModel` + `json_ext` design and how it differs from the legacy `VersionedModel` .

Prerequisites

- **Core** — `HistoryModel` , `HistoryBusinessModel` , `json_ext` .
- **Insuree** — the classic model this one generalises.
- **Modules overview** — the two beneficiary worlds.

Purpose

Two modules, two jobs:

- **individual** — a **schema-light, domain-neutral registry** of persons (`Individual`) and households (`Group`), plus bulk **data import** machinery. It knows nothing about health, insurance, or cash; it just stores people.
- **social_protection** — the **programme layer**: `BenefitPlan` (a programme), `Beneficiary / GroupBeneficiary` (enrolment of an individual/group into a plan, with a lifecycle), `Project`, and links out to [payment/payroll](#).

i Did you know? Why not just extend insuree?

The `insuree` model hard-codes health-insurance assumptions: coverage attaches to a `Family`, a policy references a `Product`, members are validated by insurance-number rules. A cash-transfer programme has none of those. Forcing it into `insuree` would mean nullable insurance columns everywhere and confusing semantics. A **separate generic registry** keeps both domains clean and lets openIMIS be a *social protection* platform, not just a health insurer.

Key models

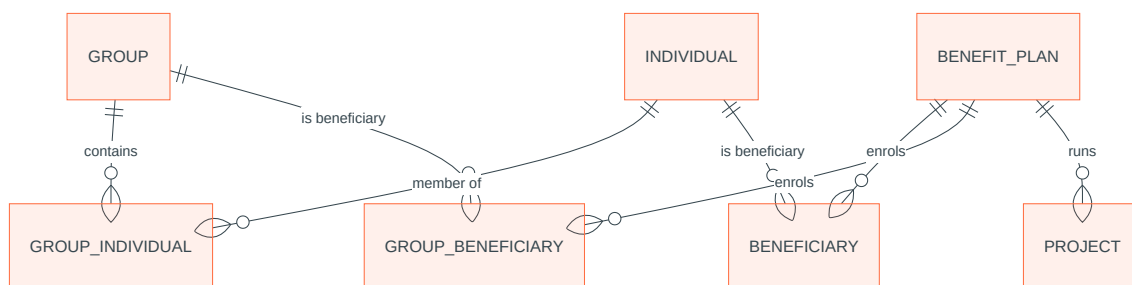
These are **new-style** tables: UUID primary keys, `json_ext` for extensibility, `HistoryModel / HistoryBusinessModel` base classes, and **no `tbl` prefix** — a deliberate break from the legacy schema.

individual module

Model	Key fields	Base c
Individual	first_name, last_name, dob, location (FK), json_ext	Histor del
Group	code, location (FK), json_ext	Histor del
GroupIndividual	group (FK), individual (FK), role (HEAD/SPOUSE/...), recipient_type (PRIMARY/SECONDARY), json_ext	Histor del
IndividualDataSource	individual (FK), upload (FK), json_ext, validations	Histor del
IndividualDataSourceUpload	status (PENDING/IN_PROGRESS/SUCCESS/PARTIAL_SUCCESS/FAIL/...), source_name	Histor del

social_protection module

Model	Key fields	Base class
BenefitPlan	code, name, type (INDIVIDUAL/GROUP), max_beneficiaries, ceiling_per_beneficiary, institution, json_ext	HistoryBusinessModel
Beneficiary	individual (FK), benefit_plan (FK), status (POTENTIAL/ACTIVE/GRADUATED/SUSPENDED), json_ext	HistoryBusinessModel
GroupBeneficiary	group (FK), benefit_plan (FK), status, json_ext	HistoryBusinessModel
Project	benefit_plan (FK), target beneficiaries, working days	HistoryBusinessModel
BenefitPlanDataUploadRecords	tracks import workflow for a plan	HistoryModel



i Did you know? A benefit plan has a type

BenefitPlan.type is INDIVIDUAL or GROUP. An INDIVIDUAL -type plan accepts Beneficiary rows (person-level); a GROUP -type plan accepts GroupBeneficiary rows (household-level). The model validates you cannot mix them.

The mapping: classic vs. generic

Concern	Classic (<code>insuree/policy</code>)	Generic (individual/social_protection)
Person	<code>Insuree</code> (<code>tblInsuree</code>)	<code>Individual</code> (UUID, <code>json_ext</code>)
Household	<code>Family</code> (<code>tblFamilies</code>)	<code>Group</code> + <code>GroupIndividual</code>
Membership role	<code>head</code> <code>bool</code> + <code>Relation</code>	<code>GroupIndividual.role</code>
Programme/product	<code>Product</code> (insurance)	<code>BenefitPlan</code> / programme
Enrolment record	<code>Policy</code> + <code>InsureePolicy</code>	<code>Beneficiary</code> / <code>GroupBeneficiary</code>
Enrolment lifecycle	policy states (idle/active/...)	<code>status</code> (POTENTIAL/ACTIVE/GRADUATED/SUSPENDED)
Base class	<code>VersionedModel</code> (in-table validity)	<code>HistoryModel</code> (separate history, UUID, <code>json_ext</code>)
DB heritage	legacy <code>tbl*</code> , <code>camelCase</code>	modern tables, <code>snake_case</code>
Domain	health insurance only	any social protection programme



Which do I use?

Building health-insurance features (policies, contributions, claims)? Stay in `insuree/policy`.
Building cash transfers, registries, or programme-based benefits? Use `individual` + `social_protection`. Some deployments run **both** and bridge them (e.g. `insuree_as_worker`).

Services

Both modules follow the newer `BaseService` **CRUD-with-validation** pattern from `core` rather than the older free-function services.

Service	Module	Responsibility
<code>IndividualService</code> , <code>GroupService</code> , <code>GroupIndividualService</code>	<code>individual</code>	Create/update/delete registry records with <code>json_ext</code> validation.
Import services (<code>IndividualImportService</code>)	<code>individual</code>	Bulk CSV/Excel upload → <code>IndividualDataSourceUpload</code> , with row-level validation and workflow.
<code>BenefitPlanService</code>	<code>social_protection</code>	Manage programmes.
<code>BeneficiaryService</code> / <code>GroupBeneficiaryService</code>	<code>social_protection</code>	Enrol beneficiaries; enforce plan-type match; drive status.
Payroll/payment integration	<code>social_protection</code>	Hand ACTIVE beneficiaries to <code>payroll / payment</code> .

GraphQL

Standard `core` helpers: `ExtendedConnection` , `OrderedDjangoFilterConnectionField` , `OpenIMISMutation` .

Queries

Query	Returns
<code>individual</code> , <code>group</code> , <code>groupIndividual</code>	Registry connections.
<code>benefitPlan</code>	Programmes.
<code>beneficiary</code> , <code>groupBeneficiary</code>	Enrolments (filterable by <code>status</code> , <code>benefitPlan</code>).
<code>individualDataSourceUpload</code>	Import batch status.

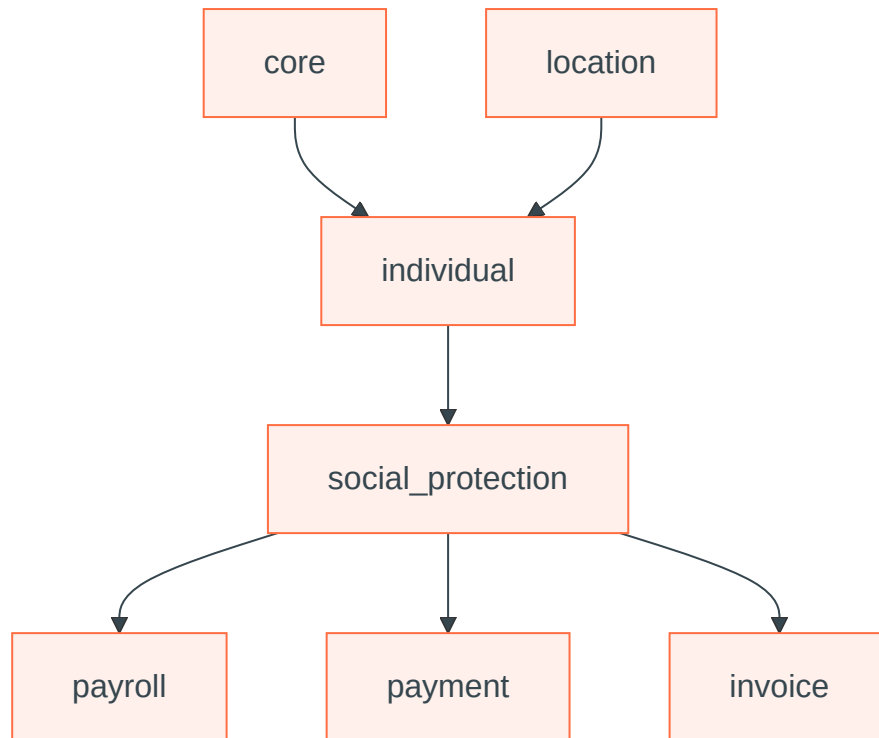
Mutations

Mutation	Effect
<code>createIndividual</code> / <code>updateIndividual</code> / <code>deleteIndividual</code>	Registry CRUD.
<code>createGroup</code> / <code>addIndividualToGroup</code>	Household composition.
<code>createBenefitPlan</code> / <code>updateBenefitPlan</code>	Programme CRUD.
<code>createBeneficiary</code> / <code>updateBeneficiaryStatus</code>	Enrolment and lifecycle transitions.

```
query {  
  beneficiary(benefitPlan_Id: "...", status: ACTIVE, first: 20) {  
    totalCount  
    edges { node {  
      status  
      individual { firstName lastName }  
      jsonExt  
    } }  
  }  
}
```



Dependencies

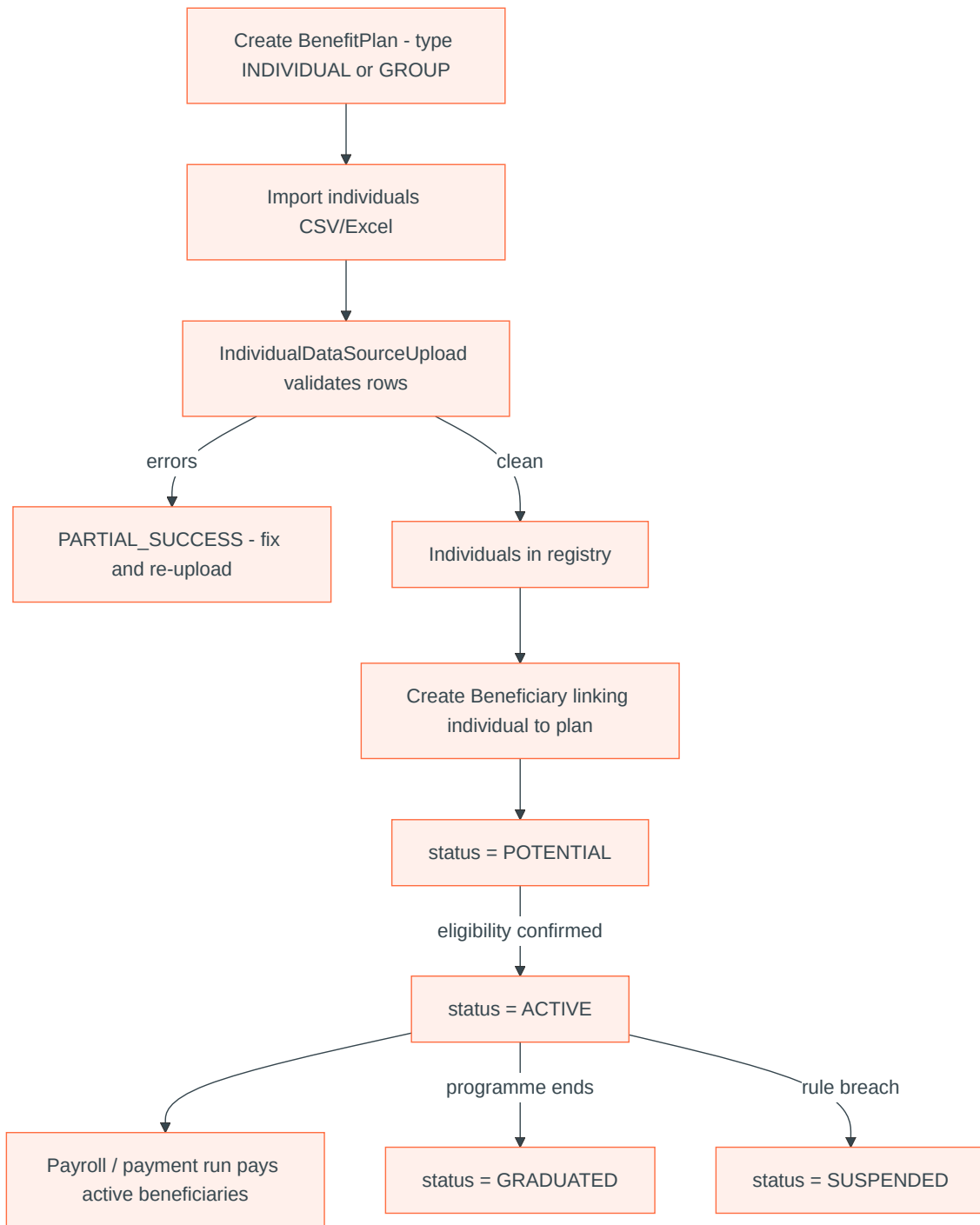


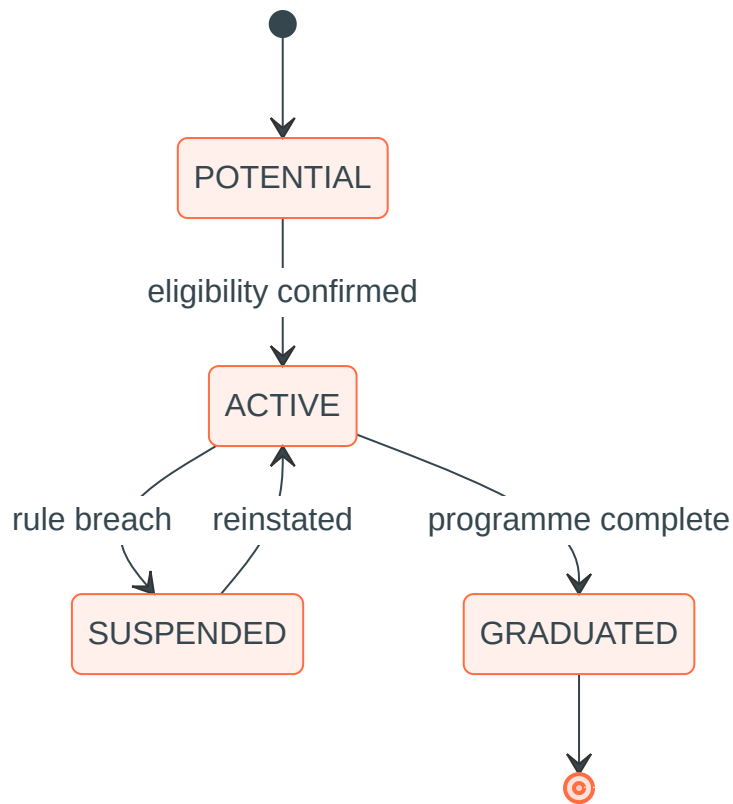
- **individual** needs: `core`, `location`.
 - **social_protection** needs: `individual`, `core`; integrates with `payment`, `invoice`, `payroll`, `tasks_management` and `opensearch_reports`.
 - It deliberately does **not** depend on `insuree` — the two registries are independent.
-

Signals

- Newer modules lean heavily on **core service signals**: import completion, beneficiary status changes and enrolment fire signals that `payroll`, `tasks_management` and reporting subscribe to.
 - Django `post_save` signals maintain history rows (`HistoryModel`) and update `json_ext`-derived aggregates.
 - Bulk imports emit progress events consumed by `tasks_management/workflow`.
-

Business workflow — enrol beneficiaries into a programme





Database tables

Unlike [insuree](#), these tables use **snake_case**, no **tbl** prefix, UUID PKs and a `json_ext` column — the modern openIMIS convention.

Table	Model
<code>individual_individual</code>	<code>Individual</code>
<code>individual_group</code>	<code>Group</code>
<code>individual_groupindividual</code>	<code>GroupIndividual</code>
<code>individual_individualdatasource</code> / <code>_upload</code>	import machinery
<code>social_protection_benefitplan</code>	<code>BenefitPlan</code>
<code>social_protection_beneficiary</code>	<code>Beneficiary</code>
<code>social_protection_groupbeneficiary</code>	<code>GroupBeneficiary</code>

(Exact table names come from the app label + model name; confirm in each repo's migrations.)
 See [Database & Legacy Heritage](#) for how these modern tables sit next to the legacy `tbl*` ones.

Configuration

`IndividualConfig` and `SocialProtectionConfig` follow the same `DEFAULT_CFG` + `ModuleConfiguration` pattern as [core](#). Notable areas (*confirm keys in each module's `apps.py`*):

Area	Example keys	Meaning
Permissions	<code>gql_query_individual_perms</code> , <code>gql_mutation_create_beneficiary_perms</code>	Integer permission codes.
Import	max upload size, allowed columns, validation calculation	Bulk import behaviour.
Workflow	which <code>workflow / tasks_management</code> flow to trigger on import	Ties into task management.
Python-based validation	maker-checker toggles	Whether beneficiary changes require approval.

Extension points

Seam	Use
<code>json_ext</code> everywhere	Add programme-specific fields with no migration — central to this design.
Import validation calculations	Plug a <code>calcrule_*</code> -style validator for uploaded rows.
Service signals	React to enrolment/status changes (payroll, reporting, tasks).
<code>tasks_management / workflow</code>	Insert maker-checker approvals into beneficiary lifecycle.
<code>BenefitPlan.type</code>	Choose individual- vs. group-level programmes.

Common mistakes

Mixing plan type and beneficiary type

An `INDIVIDUAL` plan takes `Beneficiary` (person); a `GROUP` plan takes `GroupBeneficiary`. The service rejects the mismatch — check `BenefitPlan.type` first.

Treating Individual like Insuree

`Individual` has no `chf_id`, no policy, no insurance-number validation. Do not port insuree assumptions across; the whole point is that they are decoupled.

Skipping the import status

A bulk upload can land in `PARTIAL_SUCCESS`. Always read `IndividualDataSourceUpload.status` before assuming everyone was registered.

Hands-on lab

1. In `openimis-be-social_protection_py/social_protection/models.py`, confirm the `BeneficiaryStatus` choices and the plan-type validation.
2. Create an `INDIVIDUAL` `BenefitPlan`, import two individuals, and enrol one as a `Beneficiary`; watch the status default to `POTENTIAL`.
3. Compare `individual/models.py` against `insuree/models.py` and list three concrete differences (PK type, base class, `tbl` prefix).

Exercises

- **E1.** Draw the mapping table from memory: for each classic concept, name its generic counterpart.
- **E2.** Explain to a stakeholder, in three sentences, why openIMIS did not just add nullable columns to `Insuree`.
- **E3.** Sketch the beneficiary state machine and name the event that triggers each transition.

Knowledge check

? Q1: Why does the generic registry exist alongside insuree? (click for answer) ✓

To support **social protection programmes beyond health insurance** (cash transfers, grants) without polluting the insurance-specific `insuree / policy` models with irrelevant nullable fields.

? Q2: What links an Individual to a BenefitPlan, and what tracks its lifecycle? (click for answer) ✓

A `Beneficiary` row (or `GroupBeneficiary` for households); its `status` field (POTENTIAL → ACTIVE → GRADUATED/SUSPENDED) tracks the lifecycle.

? Q3: Name two schema-level differences between Individual and Insuree. (click for answer) ✓

`Individual` uses a UUID primary key and `HistoryModel` base with a modern non-`tbl` table and `json_ext`; `Insuree` uses `VersionedModel` on the legacy `tblInsuree` table with in-table validity windows.

? Q4: What determines whether a plan enrolls people or households? (click for answer) ✓

`BenefitPlan.type` — `INDIVIDUAL` enrolls `Beneficiary` (persons), `GROUP` enrolls `GroupBeneficiary` (groups).

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be-individual_py</code>	<code>individual/models.py</code>	<code>Individual</code> , <code>Group</code> , <code>GroupIndividual</code> , import models.
<code>openimis-be-individual_py</code>	<code>individual/services.py</code>	CRUD + bulk import services.
<code>openimis-be-social_protection_py</code>	<code>social_protection/models.py</code>	<code>BenefitPlan</code> , <code>Beneficiary</code> , <code>GroupBeneficiary</code> , <code>Project</code> .
<code>openimis-be-social_protection_py</code>	<code>social_protection/services.py</code>	Enrolment and status services.

Further reading

- Individual repo: https://github.com/openimis/openimis-be-individual_py
- Social protection repo: https://github.com/openimis/openimis-be-social_protection_py
- Contrast with the classic stack: [Insuree](#), [Policy](#)

Policy — Coverage That Links People to Products

`openimis-be-policy_py` owns the concept of **coverage**: the contract that says "*this family, under this product, is covered for this period*". A `Policy` is the hinge between the [beneficiary](#) world and the [product/contribution/claim](#) world. Understanding its **states** and **stages** is essential — almost every downstream rule (can this claim be paid? is this contribution valid?) asks "what state is the policy in on the service date?"

Learning objectives

- Explain what a `Policy` connects: `Family` → `Product`, over a coverage period.
- Read the numeric **status** flags (idle / active / suspended / expired / ready) and the **stage** codes (new / renewal).
- Trace the **activation** path and how `InsureePolicy` rows make members covered.
- Follow the **renewal** workflow driven by `PolicyRenewal`.
- Relate policy to [contribution](#) and `product`.

Prerequisites

- [Core](#) — `VersionedModel`, temporal history, `OpenIMISMutation`.
 - [Insuree](#) — `Family` and `InsureePolicy`.
 - [Modules overview](#) — where coverage sits in the flow.
-

Purpose

The policy module answers "**is this household covered, for what, and until when?**" It owns:

- The `Policy` record: a `Family` enrolled in a `Product` for `[start_date, expiry_date]` at a computed `value`.
- The **coverage lifecycle**: idle → active → suspended/expired, across the new-vs-renewal stage.
- `InsureePolicy` materialisation — creating the per-member coverage links (owned physically by [insuree](#), driven by policy activation).

- The **renewal** workflow (`PolicyRenewal`): prompting, notifying and creating the next period's policy.
- **Eligibility** queries — is a given service/item covered for this insuree right now?

i Did you know? Coverage is per household, valued per product

A `Policy.family` FK plus a `Policy.product` FK is the whole idea. The product (`product module`) defines the price, benefit ceilings and covered services; the policy binds a specific family to it for a period and stores the computed `value` .

Key models

Model	Key fields	Legacy table
<code>Policy</code>	<code>uuid</code> , <code>family</code> (FK), <code>product</code> (FK), <code>officer</code> (FK), <code>stage</code> , <code>status</code> , <code>value</code> , <code>enroll_date</code> , <code>start_date</code> , <code>effective_date</code> , <code>expiry_date</code> , <code>contribution_plan</code> (FK)	<code>tblPolicy</code>
<code>PolicyRenewal</code>	<code>uuid</code> , <code>policy</code> (FK), <code>insuree</code> (FK), <code>new_officer</code> , <code>new_product</code> , <code>renewal_prompt_date</code> , <code>renewal_date</code> , <code>phone_number</code> , <code>sms_status</code> , <code>response_status</code>	<code>tblPolicyRenewals</code>
<code>InsureePolicy</code> (in <i>insuree</i>)	<code>insuree</code> (FK), <code>policy</code> (FK), <code>enrollment_date</code> , <code>start_date</code> , <code>effective_date</code> , <code>expiry_date</code>	<code>tblInsureePolicy</code>

`Policy` and `PolicyRenewal` extend `VersionedModel` : they carry `validity_from` / `validity_to` / `legacy_id` and follow the "never hard-delete" rule.

Status flags (bit values)

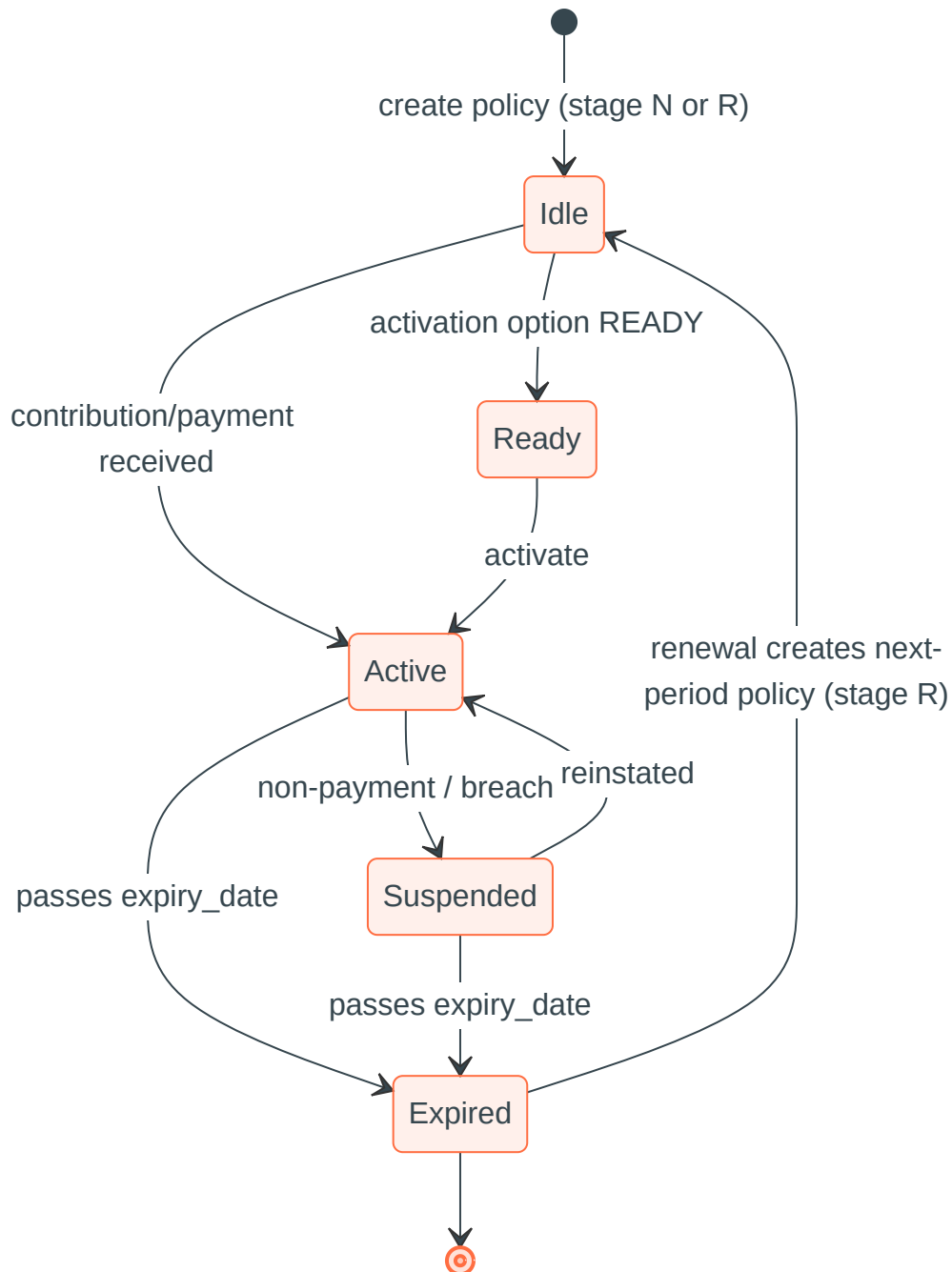
`Policy.status` is a small integer using **bit values** (see `openimis-be-policy_py/policy/models.py`):

Constant	Value	Meaning
STATUS_IDLE	1	Created but not yet started (awaiting activation).
STATUS_ACTIVE	2	In force — members are covered.
STATUS_SUSPENDED	4	Temporarily not in force (e.g. non-payment).
STATUS_EXPIRED	8	Past <code>expiry_date</code> .
STATUS_READY	16	Ready to be activated (used by the "ready" activation option).

Stage codes

Constant	Value	Meaning
STAGE_NEW	"N"	First-time enrolment for this family/product.
STAGE_RENEWED	"R"	A renewal of a previous policy.

Policy state machine



i Did you know? Activation is configurable

When a policy becomes **ACTIVE** depends on `PolicyConfig.activation_option`: 1 = **CONTRIBUTION** (active once a contribution is recorded), 2 = **PAYMENT** (active once payment is received), 3 = **READY** (active on an explicit ready step). Different countries run different rules.

Services

Logic lives in `policy/services.py`.

Service	Responsibility
<code>PolicyService</code> (create/update)	Create/version a policy; compute <code>value</code> from the product; set stage/status.
Activation logic	Transition <code>IDLE / READY</code> → <code>ACTIVE</code> per <code>activation_option</code> ; create <code>InsureePolicy</code> rows.
<code>PolicyRenewalService</code> / renewal batch	Detect policies nearing expiry, create <code>PolicyRenewal</code> prompts, send SMS, create the renewed policy.
<code>ByInsureeService</code> / eligibility	Answer "is X covered for service Y today?" — reads product benefit ceilings and <code>InsureePolicy</code> windows.
<code>EligibilityService</code>	Compute remaining benefit ceilings for claims.

GraphQL

Standard [core helpers](#).

Queries

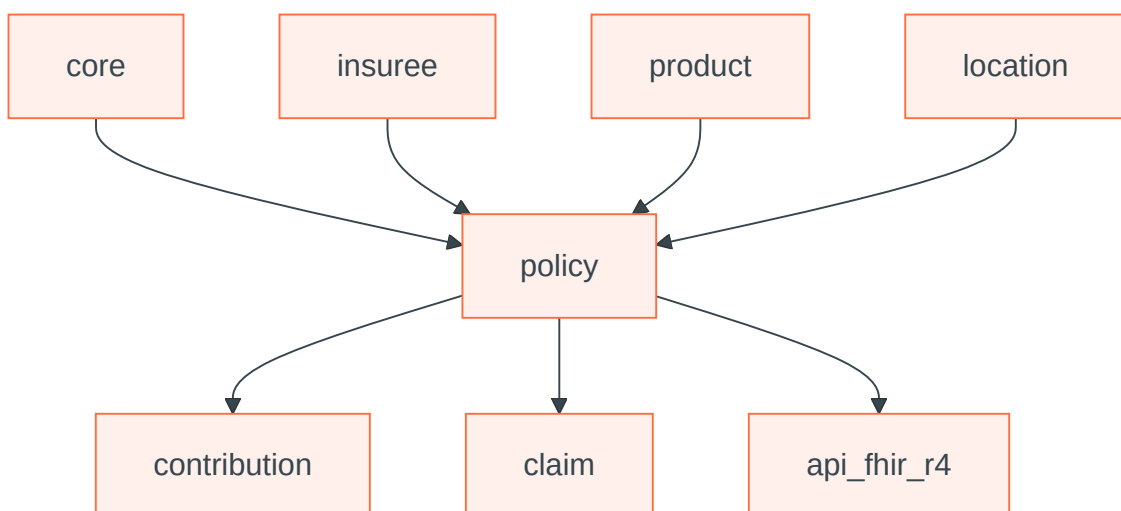
Query	Returns
<code>policies</code>	Filtered, paginated <code>Policy</code> connection.
<code>policiesByInsuree</code> / <code>policiesByFamily</code>	Coverage for a person / household.
<code>policyEligibility</code> / <code>policyEligibilityByInsuree</code>	Eligibility + remaining ceilings.
<code>policyOfficers</code>	Enrolment officers.

Mutations

Mutation	Effect	Permission (default)
<code>createPolicy</code>	New coverage	<code>["101202"]</code>
<code>updatePolicy</code>	Edit (versions the row)	<code>["101203"]</code>
<code>renewPolicy</code>	Create renewal	<code>["101205"]</code>
<code>suspendPolicy</code>	Suspend	<code>["101203"]</code>
<code>deletePolicy</code>	Close validity	<code>["101204"]</code>

```
query {  
  policiesByFamily(familyUuid: "...") {  
    totalCount  
    items { policyUuid productCode status stage expiryDate balance }  
  }  
}
```

Dependencies



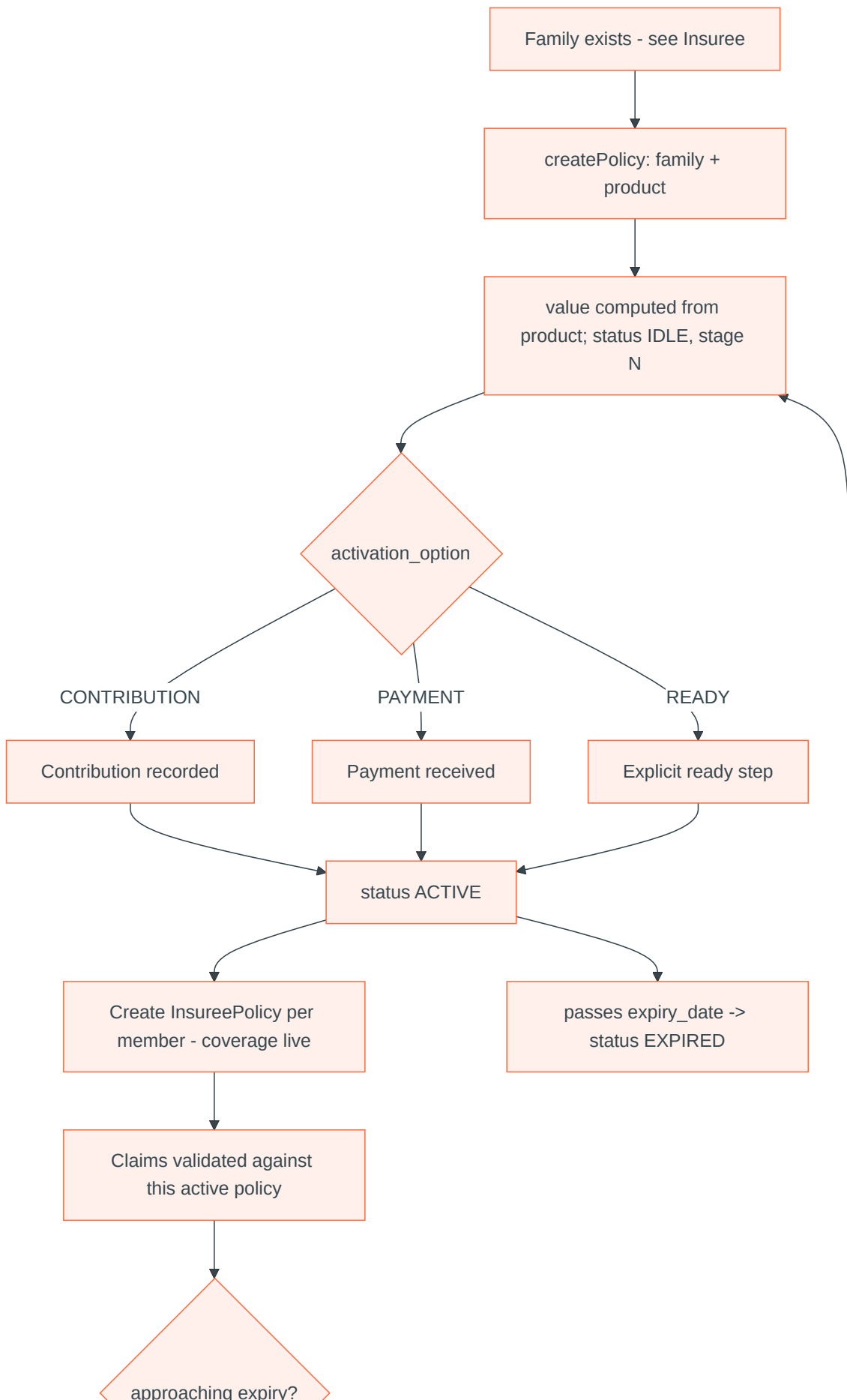
- **Needs:** `core`, `insuree` (Family, InsureePolicy), `product` (the priced product), `location` (officer/family location).

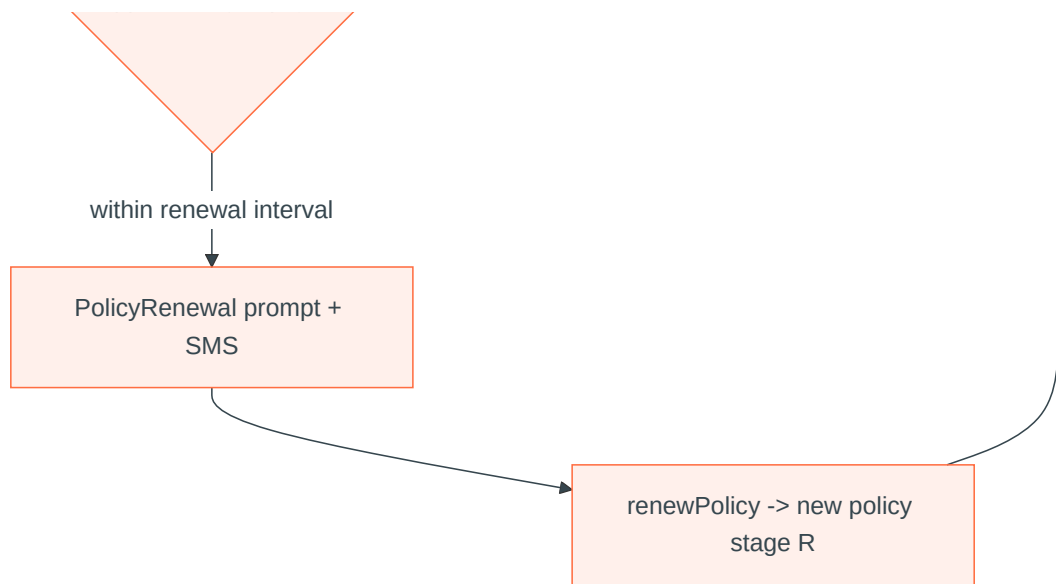
- **Needed by:** `contribution` (premiums recorded against a policy), `claim` (claims are validated against active coverage), `api_fhir_r4` (`Policy` → `FHIR Coverage`).
-

Signals

- `post_save` and `core service signals` fire on policy create/activate — this is how `contribution` links premiums, how `InsureePolicy` rows get created for each member, and how `api_fhir_r4` exports a `FHIR Coverage`.
 - The renewal batch job (registered with the `scheduler`) scans for policies within `policy_renewal_interval` days of expiry and emits renewal prompts / SMS.
-

Business workflow — enrolment, activation, renewal





Database tables

Table	Model	Notes
<code>tblPolicy</code>	<code>Policy</code>	Legacy coverage table; integer <code>PolicyID</code> + <code>uuid</code> ; <code>PolicyStatus</code> , <code>PolicyStage</code> columns.
<code>tblPolicyRenewals</code>	<code>PolicyRenewal</code>	Renewal prompts and outcomes.
<code>tblInsureePolicy</code>	<code>InsureePolicy</code>	Per-member coverage links (physically in the insuree module).

See [Database & Legacy Heritage](#) for the `tbl` convention.

Configuration

Notable `PolicyConfig.DEFAULT_CFG` keys (see `openimis-be-policy_py/policy/apps.py`):

Key	Default	Meaning
<code>gql_query_policies_perms</code>	<code>["101201"]</code>	List policies.
<code>gql_mutation_create_policies_perms</code>	<code>["101202"]</code>	Create policy.
<code>gql_mutation_renew_policies_perms</code>	<code>["101205"]</code>	Renew policy.
<code>gql_mutation_edit_policies_perms</code>	<code>["101203"]</code>	Edit/suspend policy.
<code>policy_renewal_interval</code>	<code>14</code>	Days before expiry to prompt renewal.
<code>policy_location_via</code>	<code>"family"</code>	Where a policy's location comes from (<code>family</code> or <code>product</code>).
<code>activation_option</code>	<code>1</code>	<code>1=CONTRIBUTION</code> , <code>2=PAYMENT</code> , <code>3=READY</code> .
<code>default_eligibility_disabled</code>	<code>False</code>	Turn off the default eligibility service.
<code>contribution_receipt_length</code>	<code>5</code>	Receipt number length.

Extension points

Seam	Use
<code>activation_option</code>	Choose the country's activation rule without code.
<code>policy_location_via</code>	Derive policy location from family or product.
Service signals	React to policy create/activate (FHIR, notifications, contribution linking).
Scheduler job	Customise renewal detection/notification.
<code>json_ext</code> (via product/contribution plan)	Attach deployment-specific coverage data.

Common mistakes

Comparing status with == on a single value

`status` uses **bit values** (1/2/4/8/16). Depending on the query you may need bitwise checks; do not assume it is a simple 1..5 enum.

Forgetting InsureePolicy

A policy being `ACTIVE` is not enough — members are only covered when their `InsureePolicy` rows exist and are within validity. Activation creates them; if you bypass the service, coverage silently fails.

Ignoring the stage on renewal

A renewed policy is `STAGE_RENEWED` ("R"), not a fresh "N". Reporting and renewal logic branch on stage.



Querying policies without the validity filter

Like all `VersionedModel` rows, filter `validity_to__isnull=True` (or use the module `managers/GraphQL`) to get current policies.

Hands-on lab

1. In `policy/models.py`, list the `STATUS_*` and `STAGE_*` constants and their integer/char values.
2. Create a policy for a test family, set `activation_option`, record a contribution, and verify the status moves to `ACTIVE` and `InsureePolicy` rows appear.
3. Set `policy_renewal_interval` low and run the renewal job; inspect the `tblPolicyRenewals` rows it creates.

Exercises

- **E1.** Draw the policy state machine and annotate each transition with the event that triggers it.
- **E2.** Given `activation_option = 2`, describe precisely when a new policy becomes `ACTIVE`.
- **E3.** Explain how a claim's eligibility check uses `InsureePolicy` windows and product ceilings.

Knowledge check



Q1: What two things does a Policy connect? (click for answer)



A `Family` (the covered household) and a `Product` (the priced insurance product), over a coverage period.



Q2: What are the numeric status flags? (click for answer)



`IDLE=1`, `ACTIVE=2`, `SUSPENDED=4`, `EXPIRED=8`, `READY=16` — bit values.

? Q3: What makes individual members actually covered? (click for answer)

`InsureePolicy` rows (per member), created when the policy activates; the policy being `ACTIVE` alone is not sufficient.

? Q4: What is the difference between stage N and stage R? (click for answer)

`N` (`STAGE_NEW`) is first-time enrolment; `R` (`STAGE_RENEWED`) is a renewal of a previous policy period.

? Q5: How is the moment of activation decided? (click for answer)

By `PolicyConfig.activation_option`: on contribution (1), on payment (2), or on an explicit ready step (3).

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be-policy_py</code>	<code>policy/models.py</code>	<code>Policy</code> , <code>PolicyRenewal</code> , <code>STATUS_*</code> / <code>STAGE_*</code> constants.
<code>openimis-be-policy_py</code>	<code>policy/services.py</code>	Activation, eligibility, renewal logic.
<code>openimis-be-policy_py</code>	<code>policy/apps.py</code>	<code>PolicyConfig.DEFAULT_CFG</code> , <code>activation_option</code> , permissions.
<code>openimis-be-insuree_py</code>	<code>insuree/models.py</code>	<code>InsureePolicy</code> per-member coverage link.

Further reading

- Policy repo: https://github.com/openimis/openimis-be-policy_py
- Upstream household: [Insuree](#)
- Downstream valuation: [Modules overview](#) → [claim & calculation](#)

Claim

The **claim** module is the operational heart of openIMIS. Everything else in the platform — beneficiaries, policies, products, price lists, contributions — exists so that this module can answer one question correctly: *when a health facility delivers care to an insured person, how much should the scheme pay, and to whom?*

A claim is the record of care delivered. Its life is a pipeline: a facility **submits** it, the system **checks** it against the rules of the patient's product, a human medical officer **reviews** it, the scheme **adjudicates and values** it against price lists and calculation rules, and finally the valued amount flows into **provider payment**. This chapter walks that pipeline end to end.

Learning objectives

By the end of this chapter you will be able to:

- Describe the four core entities — `Claim`, `ClaimItem`, `ClaimService`, `ClaimAdmin` — and how they relate.
- Read and reason about the **claim status bitfield** (`entered` → `checked` → `processed` → `valuated`, or `rejected`).
- Explain the difference between **automatic checking**, **manual review**, and **valuation**, and where each happens.
- Trace how valuation consults the **product price lists** and the **calculation** engine to turn "quantity provided" into "amount paid".
- Locate the models, services, GraphQL mutations and signals that implement the claim workflow in `openimis-be-claim_py`.



Prerequisites

- **The Core module** — `HistoryModel`, the async `OpenIMISMutation` pattern, rights-based permissions, and service signals.
- **Location & Health Facility** — a claim is always filed *by* a health facility *for* an insuree in a catchment.
- **Insuree and Policy** — a claim is only payable if the insuree holds an active policy on the service date.
- **Medical & Product** — items, services, price lists and the insurance product that defines coverage and ceilings.
- **GraphQL** — claims are created and adjudicated through GraphQL mutations.

Purpose

The claim module records **service delivery** and drives it through **adjudication** to a payable amount. Concretely it owns:

- The **claim header** (`Claim`): who, where, when, which diagnoses, the totals.
- The **claim lines**: `ClaimItem` (drugs, consumables) and `ClaimService` (consultations, procedures, hospital days) — each a billed line with a *provided* quantity and *asked* price.
- The **adjudication state machine**: automatic rule checks, manual medical review, and valuation.
- **Review and feedback** subsystems that let the scheme sample claims for quality control and collect patient feedback.

It does *not* own money movement to providers — that is **Payment & Invoice** — nor the pricing formulas themselves, which live in **Calculation** and the product price lists. Claim is the orchestrator that pulls those together.



Did you know?

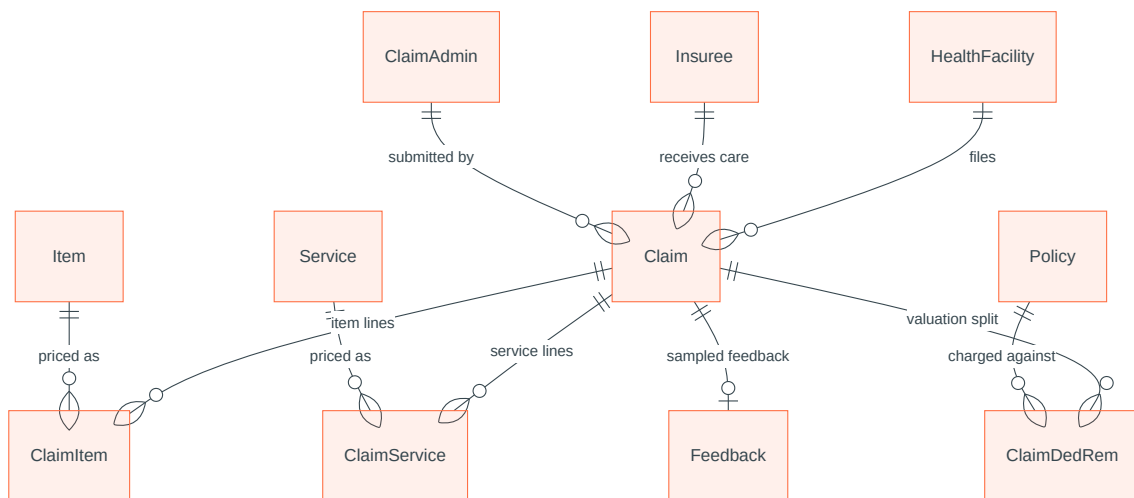
In the original .NET IMIS, claim adjudication lived largely in **SQL Server stored procedures**. The modern Django `claim` module reimplements that logic in Python services — which is exactly why you will still see legacy table names like `tblClaim`, `tblClaimItems` and `tblClaimServices`, and integer status codes rather than Django enums. The scars are load-bearing: they keep the schema compatible with historical data.

Key models

All names below are from `openimis-be-claim_py/claim/models.py` . Legacy table names are preserved for MSSQL-heritage compatibility.

Model	Table	Purpose	Notable fields
Claim	tblClaim	The claim header	code, insuree, health_facility, date_from, date_to, date_claimed, icd / icd_1..4 (diagnoses), claimed, approved, valuated, reinsured, status, review_status, feedback_status, adjuster, explanation
ClaimItem	tblClaimItems	A billed item line (drug/consumable)	claim, item, qty_provided, qty_approved, price_asked, price_adjusted, price_approved, price_valuated, status, rejection_reason, limitation_value
ClaimService	tblClaimServices	A billed service line (act/procedure)	claim, service, qty_provided, qty_approved, price_asked, price_approved, price_valuated, status, rejection_reason
ClaimAdmin	tblClaimAdmin	The claim administrator (facility staff who submits)	code, last_name, other_names, health_facility, phone, has_login

Model	Table	Purpose	Notable fields
Feedback	tblFeedback	Patient feedback on a sampled claim	claim, care_rendered, payment_asked, drug_prescribed, drug_received, assessment, officer
ClaimDedRem	tblClaimDedRem	Per-claim deductible/remuneration breakdown from valuation	claim, policy, insuree, ded_g, ded_op, ded_ip, rem_g, rem_op, rem_ip, rem_consult, rem_drug, rem_hospitalization
ClaimServiceItem / ClaimServiceService	tblClaimServicesItems / tblClaimServicesService	Sub-lines when a <i>package</i> service expands into contained items/services	link + quantities
ClaimAttachment	claim_ClaimAttachment	Scanned supporting documents	claim, type, document, mime, filename
ClaimMutation	claim_ClaimMutation	Join row linking a claim to a MutationLog	claim, mutation



i Did you know?

`Claim` extends core's `VersionedModel`, so every edit is **temporal**: a change closes the old row (`validity_to` set) and opens a new one (`validity_from` set). You never lose the prior state of a claim — critical when an auditor asks *"what did this claim look like before the reviewer touched it?"*

The status bitfield — read this carefully

Claim status is **not** a simple enum; the values are **powers of two** because the legacy system treated them as bit flags:

```
# openimis-be-claim_py/claim/models.py (illustrative excerpt)
class Claim(core_models.VersionedModel):
    STATUS_REJECTED = 1
    STATUS_ENTERED = 2
    STATUS_CHECKED = 4
    STATUS_PROCESSED = 8
    STATUS_VALUATED = 16
```

The *review* and *feedback* subsystems have their own parallel state constants (also powers of two), because a claim can be, say, "selected for review" and "feedback delivered" independently of its main adjudication status:

```
# Review lifecycle
REVIEW_IDLE = 1
REVIEW_NOT_SELECTED = 2
REVIEW_SELECTED = 4
REVIEW_DELIVERED = 8
REVIEW_BYPASSED = 16

# Feedback lifecycle
FEEDBACK_IDLE = 1
FEEDBACK_NOT_SELECTED = 2
FEEDBACK_SELECTED = 4
FEEDBACK_DELIVERED = 8
FEEDBACK_BYPASSED = 16
```

Individual **line** items (`ClaimItem` / `ClaimService`) carry a much simpler `status` :
`STATUS_PASSED = 1` or `STATUS_REJECTED = 2` , plus a `rejection_reason` integer.
Adjudication is really *per line*: the header status summarises where the whole claim is in the pipeline; the line statuses record which specific drugs and services survived.

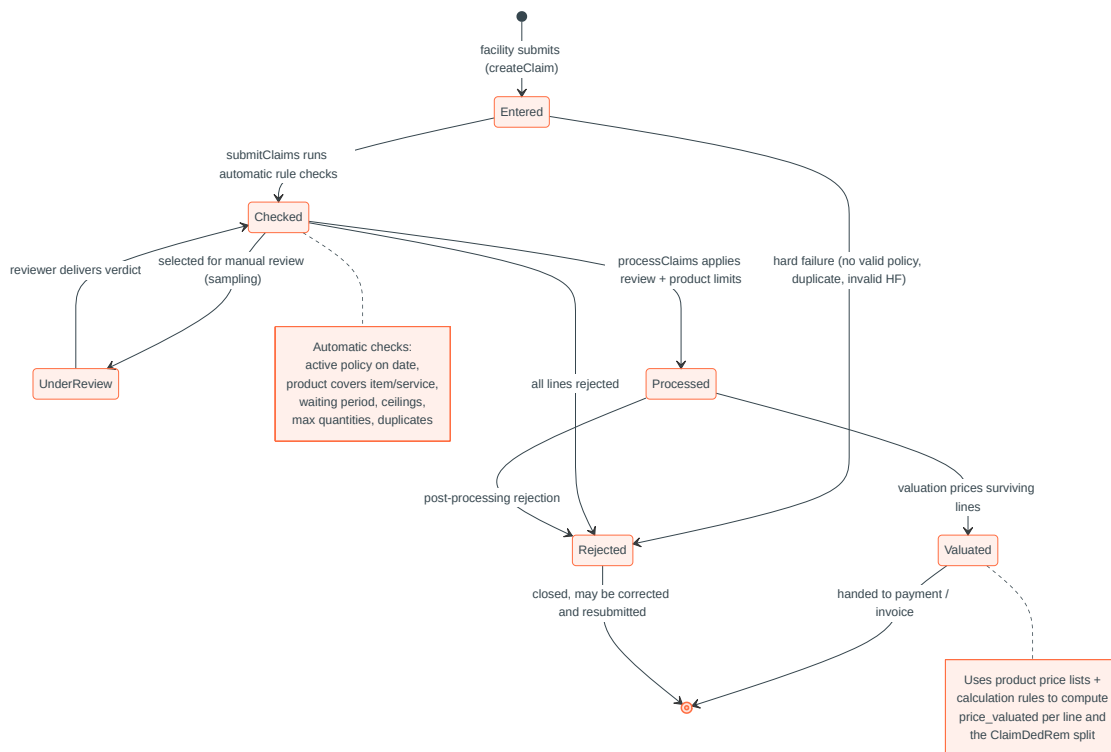


Common mistake

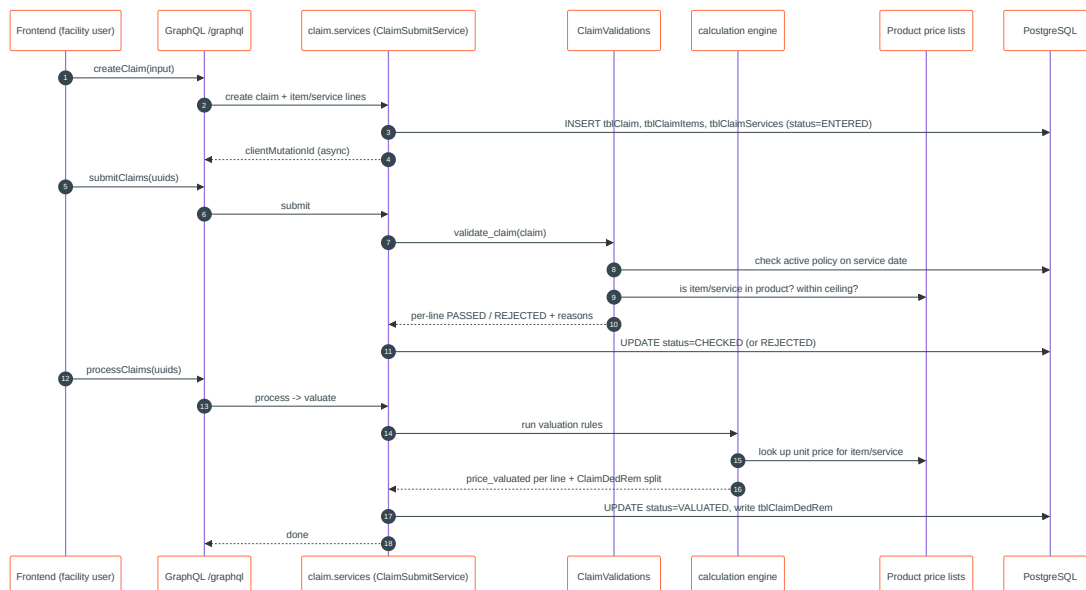
Do not assume the header being `STATUS_CHECKED` means every line passed. A checked claim can still have rejected lines. Always inspect `claimitem_set` / `claimservice_set` statuses and `rejection_reason` before reporting an amount to a facility. The header amount (`approved`) is the sum of *surviving* lines, not of *submitted* lines.

Business workflow

The whole point of the module is this pipeline. A claim moves left to right; at any checkpoint it may be knocked out to `rejected` .



Now the same journey as a **sequence** across the moving parts — the module seam view the guide asks request-flow chapters to include:



Stage by stage

Stage	Trigger (mutation / service)	What happens	Status after
Enter	<code>createClaim</code> → <code>ClaimSubmitService</code> (or <code>ClaimService.create</code>)	Facility captures the claim: patient, diagnoses, item and service lines with <i>provided</i> quantities and <i>asked</i> prices. Nothing is validated yet.	ENTERED
Check	<code>submitClaims</code> → <code>validate_claim</code> in <code>claim.validations</code>	The engine runs automatic checks against the patient's product: is a policy active on the service date? Is each item/service covered? Are waiting periods respected, ceilings and max quantities honoured, no duplicates? Failing lines get a <code>rejection_reason</code> .	CHECKED or REJECTED
Review (optional)	<code>selectClaimsForFeedback</code> / review mutations	A sample of checked claims is flagged for a medical officer to inspect. The reviewer can override line verdicts and adjust approved quantities.	<code>review_status</code> transitions; header stays CHECKED
Process	<code>processClaims</code>	Applies the review outcome and product-level limits, finalises <code>qty_approved</code> / <code>price_approved</code> per line, and prepares the claim for pricing.	PROCESSED
Valuate	valuation service	Prices each surviving line against the product price list and the calculation rules, writes <code>price_valuated</code> and the deductible/remuneration split (<code>ClaimDedRem</code>).	VALUATED
Pay	handed to payment/invoice	The valued amount becomes the basis for provider remuneration	—

Stage	Trigger (mutation / service)	What happens	Status after
(fee-for-service, capitation, or third-party payment).			

Two adjudication styles

openIMIS supports both **fee-for-service** (pay per valued line) and **capitation** (pay a facility a periodic per-head amount regardless of individual claim value). Both flow through the same claim pipeline, but the *valuation* step calls a different [calculation rule](#). The claim module does not hard-code either formula — it asks the engine.

Services

Business logic lives in `openimis-be-claim_py/claim/services.py` (and `claim/validations.py`), **not** in resolvers. Mutations are thin; they call a service, which does the work inside a transaction. Representative surface:

Service / function	Responsibility
<code>ClaimSubmitService</code> / <code>ClaimSubmit</code>	Ingest a claim with all its lines in one shot (used by the FHIR and bulk paths).
<code>update_or_create</code> (in <code>ClaimService</code>)	Create or edit a claim header and its <code>ClaimItem</code> / <code>ClaimService</code> lines.
<code>submit_claim</code> / <code>set_claims_status</code>	Move claims <code>ENTERED</code> → <code>CHECKED</code> , invoking validations.
<code>validate_claim(claim, check_max)</code> (in <code>claim/validations.py</code>)	The automatic checker: policy validity, coverage, waiting period, ceilings, max quantities, duplicates. Returns per-line errors.
<code>process_dedrem</code> / valuation helpers	Compute deductibles, remuneration and <code>price_valuated</code> , write <code>ClaimDedRem</code> .
<code>set_claim_submitted</code> / <code>set_claims_processed</code>	Batch state transitions used by the <code>processClaims</code> mutation.
<code>ClaimReportService</code>	Assemble data for printed claim / adjudication reports.

```
# Illustrative – the shape of a validation, not the exact code.
# See openimis-be-claim_py/claim/validations.py for the real implementation.
def validate_claim(claim, check_max=True):
    errors = []
    for line in claim.services.all():
        policy = active_policy_for(claim.insuree, claim.date_from,
line.service.product)
        if policy is None:
            reject(line, reason=REJECTION_REASON_NO_POLICY)
            continue
        if within_waiting_period(policy, line.service):
            reject(line, reason=REJECTION_REASON_WAITING_PERIOD)
        if check_max and exceeds_ceiling(policy, line):
            reject(line, reason=REJECTION_REASON_MAX_LIMIT)
    return errors
```

Deep dive: where 'rejection reasons' come from

Each rejection is an integer code, not free text, so the UI can translate it and reports can aggregate it. Codes cover situations like *no active policy*, *item/service not in product*, *waiting period not elapsed*, *quantity above the product maximum*, *duplicate claim*, and *care type mismatch* (out-patient item billed on an in-patient claim). When you extend the checker, add a **new integer code** rather than overloading an existing one — downstream analytics depend on stable meanings.

GraphQL

Claim is exposed almost entirely over [GraphQL](#). Queries live in `claim/gql_queries.py`; mutations in `claim/gql_mutations.py`, all subclassing core's asynchronous `OpenIMISMutation`.

```
query {  
  claims(first: 20, status: 4, healthFacility_Uuid: "...") {  
    totalCount  
    edges {  
      node {  
        uuid  
        code  
        status  
        claimed  
        approved  
        valuated  
        insuree { chfId lastName }  
        healthFacility { code name }  
        services { edges { node { service { code } qtyProvided status } } }  
        items { edges { node { item { code } qtyProvided status } } }  
      }  
    }  
  }  
}  
  
mutation {  
  createClaim(input: {  
    clientMutationId: "c-001"  
    code: "CLM-2026-0001"  
    insureeUuid: "..."  
    healthFacilityUuid: "..."  
    dateFrom: "2026-07-01"  
    dateClaimed: "2026-07-02"  
    icdId: 42  
    services: [{ serviceUuid: "...", qtyProvided: 1, priceAsked: "1200.00" }]  
    items: [{ itemUuid: "...", qtyProvided: 3, priceAsked: "150.00" }]  
  }) { clientMutationId internalId }  
}
```

```
# Move a batch through the pipeline. Each returns immediately;
# the client polls the MutationLog for the real outcome.
mutation { submitClaims(input:{ clientMutationId:"s-1", uuids:["..."] }) {
  clientMutationId } }
mutation { processClaims(input:{ clientMutationId:"p-1", uuids:["..."] }) {
  clientMutationId } }
```

Query a claim

Create a claim

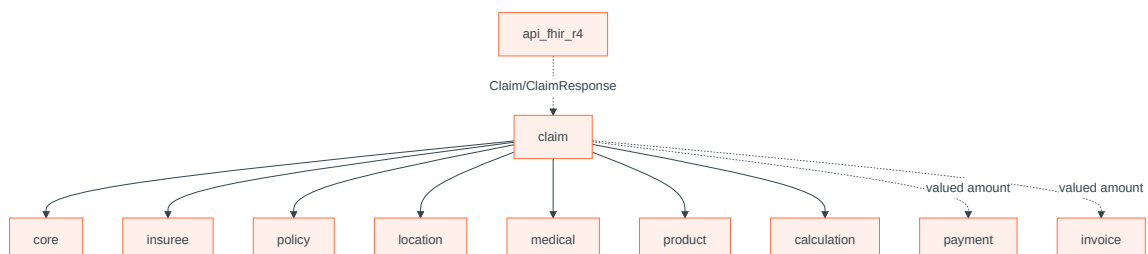
Adjudicate

⚠ GraphQL mutations here are asynchronous

Like all openIMIS mutations, `submitClaims / processClaims` return a `clientMutationId` **immediately** and do the heavy adjudication in the background, writing status to a `MutationLog`. The frontend "journalizes" (polls) that log. If you call these from a script, you must poll — a `200` response does **not** mean the claim was accepted. See the async pattern in [Core](#) and [GraphQL](#).

Permissions are integer **rights** checked in the resolver/mutation via `user.has_perms([...])`. The claim permission codes are defined in `claim/apps.py` (family 1110xx, e.g. query, add, submit, process, feedback, review). A facility user typically holds *add* and *submit*; a scheme medical officer holds *review* and *process*.

Dependencies



Depends on	Why
core	VersionedModel , OpenIMISMutation , rights, service signals.
insuree	The patient (Claim.insuree) and their family/policy links.
policy	Coverage check — the claim must map to an active policy on the service date.
location	HealthFacility that files the claim; catchment/scoping decides who can see it.
medical	Item and Service catalogues each line references.
product	Price lists, ceilings, deductibles, waiting periods that govern checking and valuation.
calculation	Pluggable valuation and provider-payment formulas.

Modules that depend on claim: [payment/invoice](#) (consume valued claims) and [api_fhir_r4](#) (exposes Claim and ClaimResponse).

Signals

Claim participates in core's **service-signal** seam (`register_service_signal` / `bind_service_signal`) so other modules can react to adjudication without importing the claim code. Typical hook points:

- **After a claim is submitted / checked** — a fraud-detection or notification module binds *after* `claim.submit` to flag anomalies or SMS the insuree.
- **After valuation** — the [payment/invoice](#) modules bind *after* the valuation service to generate a provider invoice line.
- **After processing** — capitation and third-party-payment [calculation](#) rules subscribe to aggregate figures.

```
# Illustrative – subscribing to claim valuation from another module's
apps.ready()
from core.service_signals import bind_service_signal

def on_claim_valuated(sender, result=None, **kwargs):
    claim = result
    if claim.status == claim.STATUS_VALUATED:
        create_provider_invoice_line(claim)

bind_service_signal("claim_service.valuate", on_claim_valuated,
bind_type=AFTER)
```

Standard Django ORM signals (`post_save` on `Claim`) are also used, e.g. to keep denormalised header totals in sync with line changes.

Did you know?

Because provider payment binds to a *signal* rather than being called directly from claim code, you can deploy a scheme **without** the payment module at all (some deployments reconcile payment out-of-band). The claim pipeline still runs to `VALUATED` ; nothing downstream fires. That decoupling is the plugin architecture paying off.

Database tables

Table	Model	Notes
<code>tblClaim</code>	<code>Claim</code>	Header. Integer <code>Status</code> , plus <code>ReviewStatus</code> , <code>FeedbackStatus</code> . Versioned rows.
<code>tblClaimItems</code>	<code>ClaimItem</code>	One row per billed item line.
<code>tblClaimServices</code>	<code>ClaimService</code>	One row per billed service line.
<code>tblClaimAdmin</code>	<code>ClaimAdmin</code>	Facility staff who submit claims.
<code>tblFeedback</code>	<code>Feedback</code>	Sampled patient feedback.
<code>tblClaimDedRem</code>	<code>ClaimDedRem</code>	Per-claim deductible/remuneration split written at valuation.
<code>tblClaimServicesItems</code> / <code>tblClaimServicesService</code>	package sub-lines	Populated when a package service expands.
<code>claim_ClaimAttachment</code>	<code>ClaimAttachment</code>	Uploaded documents (modern table, no <code>tbl</code> prefix).
<code>claim_ClaimMutation</code>	<code>ClaimMutation</code>	<code>Claim</code> ↔ <code>MutationLog</code> join for audit.



Common mistake

The legacy tables use **camelCase columns** (`ClaimID` , `HFID` , `InsureeID`) and integer primary keys *alongside* UUIDs. When you write a raw query or a report, filter on the UUID for API stability but remember the integer key is what the historical foreign keys use. Mixing them up silently returns the wrong claim.

Configuration

Claim reads its runtime config from `claim/apps.py` (`ClaimConfig.DEFAULT_CFG`), overlaid at startup by the DB-stored `ModuleConfiguration` (see [Configuration](#)). The keys you will actually touch:

Config key (illustrative)	Controls
<code>gql_query_claims_perms</code> , <code>gql_mutation_create_claims_perms</code> , <code>..._submit..._perms</code> , <code>..._process..._perms</code>	Integer rights required for each operation.
<code>claim_max_number_of_items</code> / <code>..._services</code>	Cap on how many lines a single claim may carry.
<code>default_validations_disabled</code>	Turn off specific automatic checks in deployments that adjudicate manually.
review / feedback sampling settings	How claims are selected for manual review and patient feedback.
<code>allowed_domains_attachments</code> , attachment size/type limits	Guardrails for <code>ClaimAttachment</code> .

```
// A ModuleConfiguration override for the claim module (illustrative)
{
  "module": "claim",
  "config": {
    "claim_max_number_of_items": 50,
    "gql_mutation_process_claims_perms": ["111005"],
    "default_validations_disabled": false
  }
}
```

Exact key names vary by release — confirm against `openimis-be-claim_py/claim/apps.py` for your version.

Extension points

The claim module is designed to be extended without forking. The main seams:

1. **Bind to claim service signals** (`bind_service_signal`) to run *before* or *after* `submit/process/valuate` — the clean way to add notifications, fraud checks, or downstream integrations.
2. **Register a calculation rule** in `calculation` to change how lines are **valued** (a country-specific fee schedule, capitation, or third-party-payment formula) — no change to claim code.
3. **Add rejection reason codes** and validation clauses in the checker for scheme-specific business rules.
4. **Extend via `json_ext`** — the core JSON field on the header lets you attach custom attributes (e.g. an external claim reference) without a migration.
5. **Expose new fields over GraphQL** by extending the `ClaimGraphQLType` in your own module and stitching it into the schema (see [GraphQL](#)).

Deep dive: valuation, price lists and the calculation engine

Valuation answers "what is a passed line *worth*?" in two layers. First, the **product price list** (from `medical/product`) gives a base unit price for the item or service *as sold under this product* — a facility's `services_pricelist` / `items_pricelist` can override the catalogue price. Second, the `calculation` engine applies the scheme's formula: it may cap against remaining ceilings, subtract deductibles, split the amount into remuneration buckets (consultation / drug / hospitalisation), and — under capitation — ignore the line value entirely in favour of a per-capita amount. The output is `price_valuated` on each line and the `ClaimDedRem` breakdown on the header. Because that formula is a registered rule, two countries can run identical claim code with completely different payment economics.

Common mistakes

Pitfalls when working with claims

- **Treating status as an enum.** It is a bitfield of powers of two. Compare against the constants; do not assume ordering beyond the pipeline.
- **Reading `claimed` as the payout.** `claimed` is what the facility *asked*. The payable figure is `valuated` (post-adjudication). `approved` sits in between (surviving quantities at approved price, pre-valuation).
- **Skipping the poll.** `submitClaims` returning success is not adjudication success — poll the `MutationLog`.
- **Editing a `VALUATED` claim in place.** Valuation results feed provider payment; correcting a paid claim usually means a **reversal + resubmission**, not a silent edit.
- **Ignoring line-level rejections.** A "checked" claim can still contain rejected lines. Always inspect line `status` and `rejection_reason`.

Hands-on lab

Lab: drive one claim through the pipeline

Using a local stack ([setup](#)) and the GraphQL playground at `/graphql`:

1. **Seed** an insuree with an active policy on a product that covers at least one item and one service (see [Policy](#) and [Product](#)).
2. **Create** a claim with one covered service line and one covered item line via `createClaim`. Query it back and confirm `status = 2` (ENTERED).
3. **Submit** it with `submitClaims`. Poll the `MutationLog`. Re-query and confirm `status = 4` (CHECKED) and inspect each line's `status`.
4. Deliberately add a line for an item **not** in the product, resubmit, and observe the `rejection_reason` on that line while others pass.
5. **Process** it with `processClaims`; confirm the claim reaches `status = 16` (VALUATED) and that `price_valuated` is populated. Inspect the `ClaimDedRem` row.
6. Compare `claimed`, `approved` and `valuated` on the header and explain the differences to yourself in one sentence each.

Knowledge check

? Q1: Why are claim status values 1, 2, 4, 8, 16 rather than 1, 2, 3, 4, 5? (click for answer) ✓

They are **bit flags** inherited from the legacy .NET/MSSQL IMIS, where a claim's state (and its parallel review/feedback states) were stored as powers of two. Keeping them preserves compatibility with historical data and lets independent lifecycles (adjudication vs. review vs. feedback) be tracked separately.

? Q2: A claim header shows STATUS_CHECKED. Can it still contain rejected lines? (click for answer) ✓

Yes. Adjudication is **per line**. The header status records how far the whole claim has advanced in the pipeline; individual `ClaimItem` / `ClaimService` rows carry their own `STATUS_PASSED` / `STATUS_REJECTED` and a `rejection_reason`. Always inspect the lines before quoting an amount.

? Q3: Where does the actual payable amount get computed, and what is written? (click for answer) ✓

In the **valuation** step, which consults the **product price list** for the base unit price and the **calculation** engine for the scheme's formula. It writes `price_valuated` on each surviving line and a `ClaimDedRem` breakdown (deductibles and remuneration buckets) on the header.

? Q4: How does the payment module get invoked when a claim is valued, without claim importing payment? (click for answer) ✓

Through core's **service-signal** seam. Payment/invoice code binds *after* the claim valuation service with `bind_service_signal`. Claim never references payment directly, so a deployment can run adjudication with or without the payment module installed.

? Q5: You call `submitClaims` over GraphQL from a script and get a 200. Is the claim checked? (click for answer) ✓

Not necessarily. openIMIS mutations are **asynchronous** — they return a `clientMutationId` and adjudicate in the background, writing the result to a `MutationLog`. You must **poll** that log for the real outcome before concluding the claim was checked or rejected.

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be-claim_py</code>	<code>claim/models.py</code>	<code>Claim</code> , <code>ClaimItem</code> , <code>ClaimService</code> , <code>ClaimAdmin</code> , <code>Feedback</code> , <code>ClaimDedRem</code> ; status constants.
<code>openimis-be-claim_py</code>	<code>claim/validations.py</code>	The automatic checker: coverage, waiting periods, ceilings, duplicates.
<code>openimis-be-claim_py</code>	<code>claim/services.py</code>	Submit / process / valuation orchestration.
<code>openimis-be-claim_py</code>	<code>claim/gql_queries.py</code> , <code>claim/gql_mutations.py</code>	GraphQL surface: <code>claims</code> , <code>createClaim</code> , <code>submitClaims</code> , <code>processClaims</code> .
<code>openimis-be-claim_py</code>	<code>claim/apps.py</code>	<code>ClaimConfig.DEFAULT_CFG</code> and integer permission codes.
<code>openimis-be-product_py</code>	<code>product/models.py</code>	Ceilings, deductibles, waiting periods used during checking/valuation.
<code>openimis-be-calculation_py</code>	<code>calculation/services.py</code>	The valuation/provider-payment rule engine claim delegates to.

Further reading

- Source: [openimis-be-claim_py](#)
- [Medical & Product](#) — price lists and coverage.
- [Calculation Rules](#) — how valuation is computed.
- [Contribution & Payment](#) — money-out to providers.
- [Integrations](#) — `Claim` / `ClaimResponse` over FHIR R4.
- Official functional docs: [openIMIS wiki](#).

Location & Health Facility

Almost everything in openIMIS is anchored to a **place**. An insuree lives in a village; a health facility sits in a district; a scheme officer is allowed to see only the regions assigned to them. The **location** module (`openimis-be-location_py`) provides that geographic backbone: a self-referential **administrative hierarchy**, the **health facilities** attached to it, and the **user–location access scoping** that decides who can see which data.

Learning objectives

By the end of this chapter you will be able to:

- Describe the `Location` hierarchy (Region → District → Municipality/Ward → Village) and how it is stored as a single self-referential table.
- Explain what a `HealthFacility` is and how it links to its price lists and catchment.
- Understand **user–location scoping**: how `UserDistrict` and `OfficerVillage` restrict which locations a user can act on.
- Locate the models, managers and query filters that enforce scoping in `openimis-be-location_py`.

Prerequisites

- **The Core module** — `VersionedModel`, the custom `User / InteractiveUser`, and integer rights.
- **Getting Started terminology** — the words *region*, *district*, *catchment*.

Purpose

The location module answers three questions the rest of the platform keeps asking:

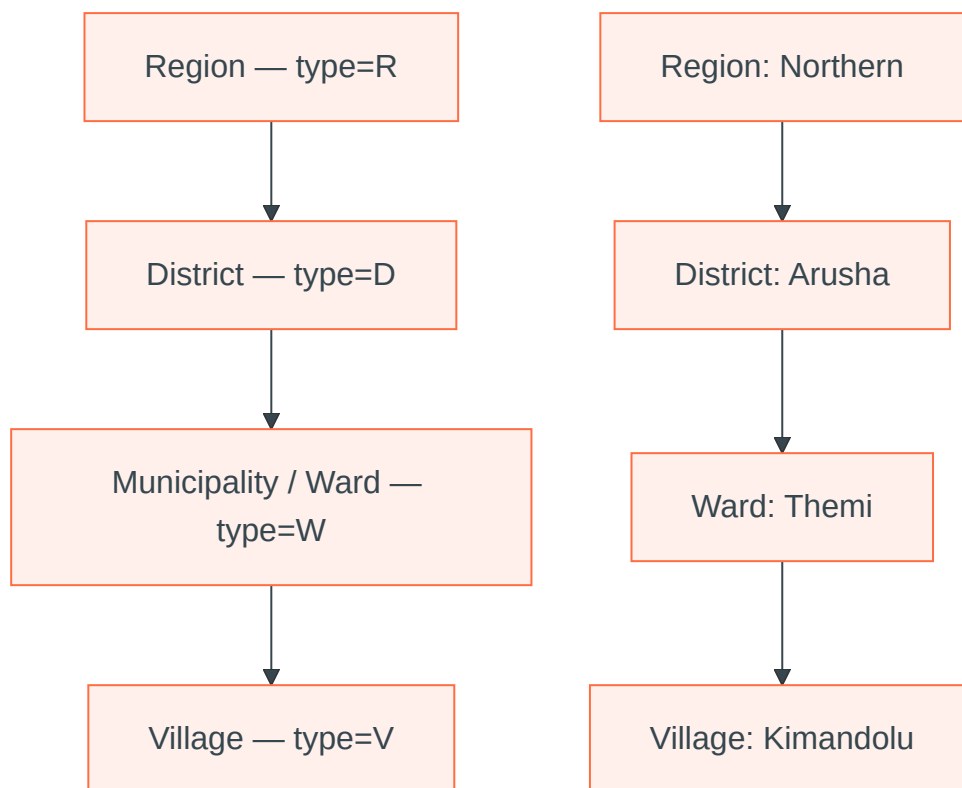
1. **Where is this?** — a normalised administrative tree every entity can point at (an insuree's village, a facility's district).
2. **Which facilities serve here?** — `HealthFacility` records, their care type, legal form, level, and the price lists they bill under.

3. **Who is allowed to see this?** — data-access scoping so a district officer cannot read another district's insurees or claims.

It is **foundational reference data**: loaded early, changed rarely, depended on by [insuree](#), [policy](#), [claim](#) and reporting.

The location hierarchy

A single table, `tblLocations`, holds every administrative unit at every level. A `type` character says what level a row is, and a `parent` self-foreign-key threads them into a tree.



Type code	Level	Typical role
R	Region	Top-level administrative division.
D	District	The scoping unit most user permissions are granted at.
W	Municipality / Ward	Sub-district grouping.
V	Village	Where an insuree/family actually lives.

Did you know?

The four levels are a **convention**, not a hard schema constraint — the hierarchy is just parent pointers, and deployments relabel the levels to match their country's administrative geography. What is fixed is the *shape* (a strict tree) and the type codes `R/D/W/V`. Always confirm a deployment's labelling against its seed data rather than assuming "level 3 = ward".

Why one self-referential table?

Storing every level in `tblLocations` (rather than four separate tables) means:

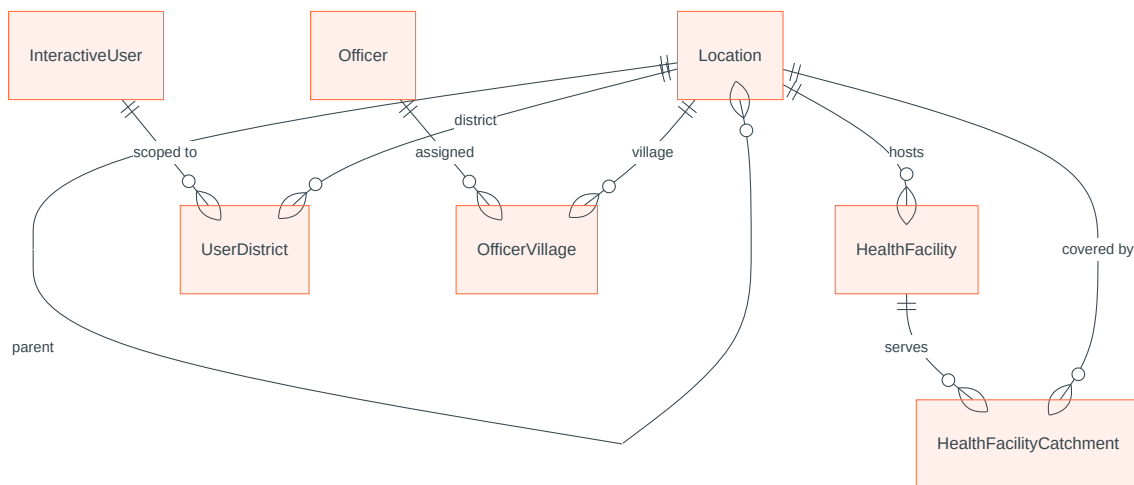
- **Uniform queries.** "Give me everything under this node" is one recursive walk, regardless of level.
- **Reparenting is cheap.** Moving a village to a new ward is a single `parent` update.
- **Recursive scoping.** The access filter can climb or descend the tree with a Common Table Expression (CTE) instead of joining four tables.

The trade-off is that the table mixes levels, so every query that means "all districts" must filter `type='D'`.

Key models

From `openimis-be-location_py/location/models.py`:

Model	Table	Purpose	Notable fields
Location	tblLocations	One administrative unit at any level	code, name, type (R/D/W/V), parent (Self FK), male_population, female_population, other_population, families, uuid
HealthFacility	tblHF	A clinic/hospital that delivers care and files claims	code, name, acc_code, legal_form, level, sub_level, location (FK), care_type, services_pricelist, items_pricelist, status, offline, contract_start_date, contract_end_date
UserDistrict	tblUsersDistricts	Grants an InteractiveUser access to a district	user, location (a type='D' row), audit_user_id
OfficerVillage	tblOfficerVillages	Assigns an enrolment officer to villages	officer, location (a type='V' row), audit_user_id
HealthFacilityCatchment	tblHFCatchment	Which locations a facility serves (population catchment %)	health_facility, location, catchment
HealthFacilityLegalForm	tblLegalForms	Reference list (government, private, NGO...)	code, legal_form
HealthFacilitySubLevel	tblHFSublevel	Reference list of facility sub-levels	code, health_facility_sub_level



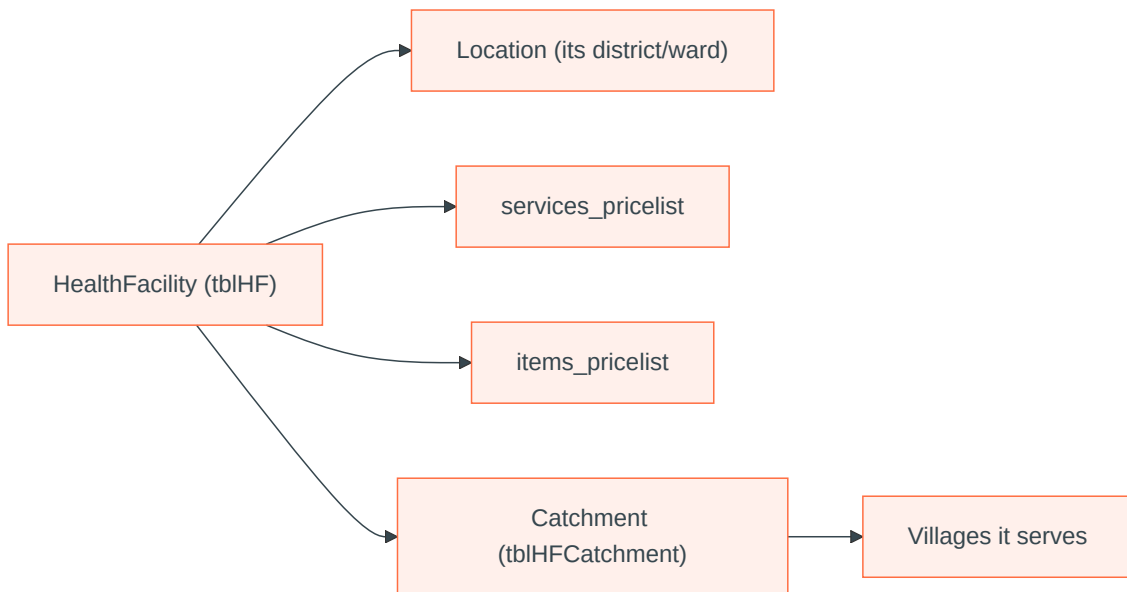
Did you know?

A `HealthFacility` carries **its own** `services_pricelist` and `items_pricelist`. That is how two hospitals in the same district can be paid different unit prices for the identical procedure — the facility's price list overrides the catalogue price during claim **valuation**. See [Medical & Product](#).

Health facilities & catchment

A `HealthFacility` is where the platform's money-out story begins: it delivers services, files **claims**, and gets paid. Key attributes:

- **care_type** — out-patient (`0`), in-patient (`I`), or both (`B`). Claims and coverage checks respect this: an out-patient-only facility cannot bill an in-patient line.
- **level / sub_level** — dispensary vs. health-centre vs. hospital, driving which products and prices apply.
- **status** — `AC` active, `IN` inactive, `DE` delisted, `ID` idle. Only active facilities can file claims.
- **Catchment** — `HealthFacilityCatchment` records the percentage of a location's population the facility is responsible for. Capitation **payment** uses catchment to distribute per-head amounts.



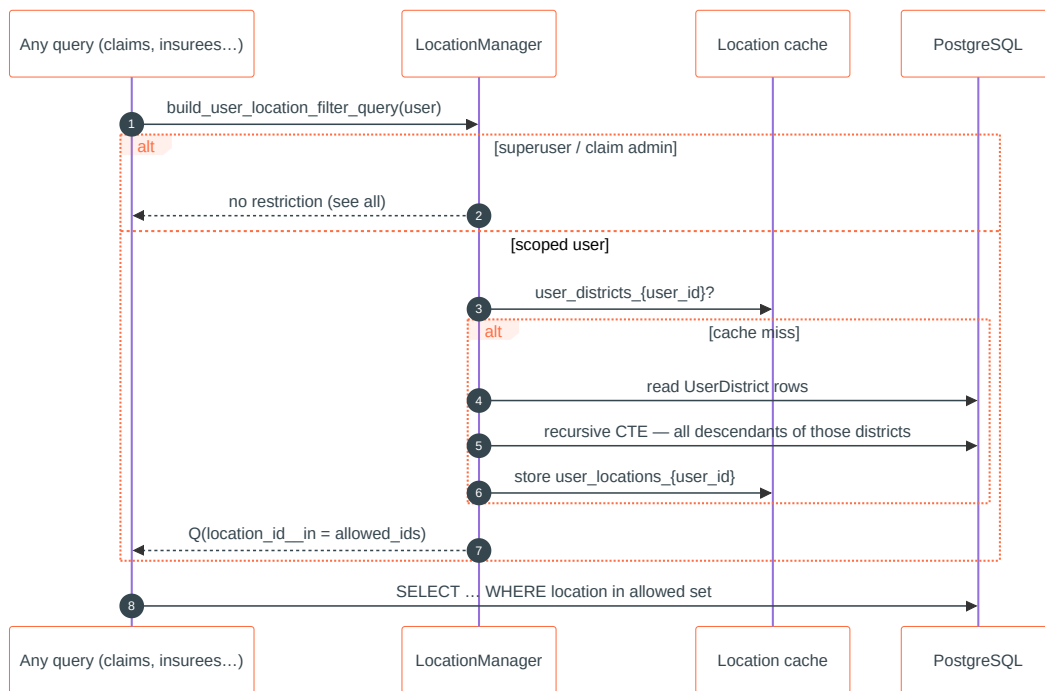
User–location access scoping

This is the module's least obvious but most important job. openIMIS is a **multi-tenant-by-geography** system: a district health officer must see only their district's insurees, policies and claims. Scoping is enforced at the **query** layer, not just the UI.

Two grant tables express access:

- **UserDistrict** — links an `InteractiveUser` to one or more districts (`type='D'` locations). This is the primary scoping grant for scheme staff.
- **OfficerVillage** — links an enrolment `Officer` to villages, used for field enrolment workflows.

A user's **effective set of locations** is the districts they are granted **plus everything beneath them** in the tree (the wards and villages under those districts). That descent is computed recursively.



Mechanics you will meet in the source (`location/models.py` , `LocationManager`):

- **`LocationManager.get_allowed_ids(user)`** — resolves the permitted location ids from `UserDistrict` (with officer/village fallbacks). **Superusers and claim admins bypass** all restriction.
- **`allowed(...)`** — uses a recursive **CTE** to pull child locations, and, in non-strict mode, conditionally includes parents.
- **`build_user_location_filter_query(user, strict=...)`** — returns a Django `Q` object callers `.filter(...)` their querysets with, so scoping is applied consistently everywhere.
- **Caching & invalidation** — results are cached under keys like `user_locations_{user_id}` and `user_districts_{user_id}`; `post_save` / `post_delete` signals call `free_cache_for_user()` to invalidate when grants or the tree change.

```

# Illustrative – how a downstream module scopes its queryset to the caller.
# See openimis-be-location_py/location/models.py (LocationManager).
def scoped_claims(user):
    loc_filter = Location.objects.build_user_location_filter_query(
        user._u, prefix="health_facility_location"
    )
    return Claim.objects.filter(loc_filter)
  
```



Common mistake

Do **not** enforce location scoping only in the frontend. The React UI hides out-of-scope data, but the authoritative filter must be applied on the backend queryset via

`build_user_location_filter_query`. A GraphQL query hand-written by a user with a valid token will otherwise return data outside their district. Scoping is a **backend** concern.



Cache staleness after re-assigning a user

Because allowed-location sets are cached per user, moving a user to a new district or reparenting a location must invalidate the cache. The module wires `post_save / post_delete` signals to `free_cache_for_user()` for exactly this reason — if you bulk-load location changes with `bulk_create` (which **skips** signals), clear the cache yourself.



Deep dive: strict vs. non-strict mode



Some operations should see a location **and its ancestors** (e.g. a report that rolls a village figure up to its region), while others must be confined strictly to descendants (e.g. "insurees I may edit"). The `strict` flag on the filter builder switches between "descendants only" and "descendants plus parents". Choosing the wrong mode is a subtle security bug: non-strict where you meant strict can widen a user's visible set upward. When in doubt for a write path, use **strict**.

Services

Location's logic is comparatively light — most of it lives in the manager and validation helpers rather than a large `services.py`:

Service / helper	Responsibility
LocationManager (custom manager on Location)	Cached hierarchy access; <code>get_allowed_ids</code> , <code>allowed</code> , <code>build_user_location_filter_query</code> .
<code>cache_location_graph()</code>	Builds and caches the full parent/child graph for fast traversal.
<code>free_cache_for_user()</code> / cache-invalidation helpers	Drop stale per-user allowed-location sets.
HealthFacility services / validations	Create/update facilities, validate <code>code</code> uniqueness, contract dates and status transitions.
LocationService (create/update)	CRUD for locations with reparenting and versioning.

GraphQL

Location exposes its tree and facilities over [GraphQL](#) (`location/schema.py`, `location/gql_queries.py`). Queries are commonly used to populate cascading pickers (region → district → ward → village) in the UI.

```

query {
  locations(type: "D", parent_Uuid: "...region-uuid...") {
    edges { node { uuid code name type parent { code } } }
  }
}

query {
  healthFacilities(first: 20) {
    totalCount
    edges {
      node {
        uuid code name careType status
        location { code name }
        servicesPricelist { name }
        itemsPricelist { name }
      }
    }
  }
}

```

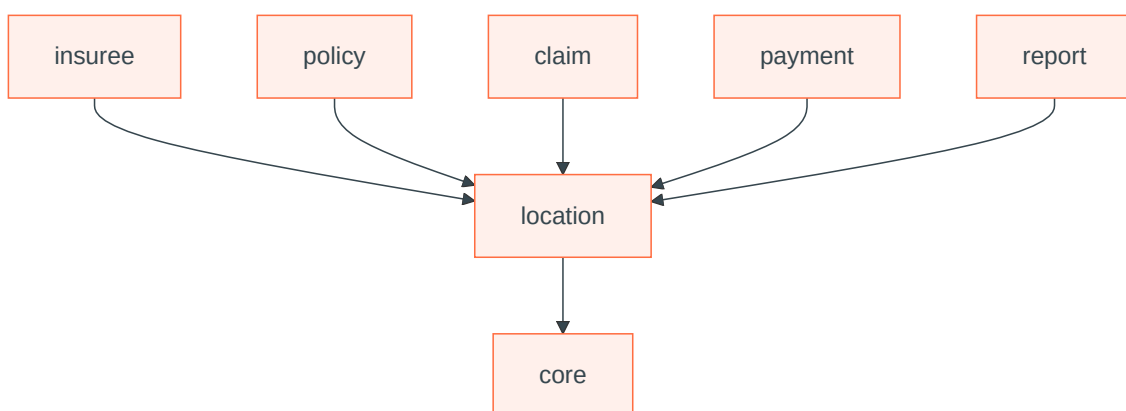
Walk the tree

Facilities in scope

Because the resolvers apply `build_user_location_filter_query`, these queries **automatically** return only what the authenticated user is scoped to see — the scoping is not something the client opts into.

Mutations (`createLocation`, `updateLocation`, `moveLocation`, `createHealthFacility`, ...) follow the async `OpenIMISMutation` pattern and are gated by integer rights from `location/apps.py`.

Dependencies



Relationship	Why
location → core	<code>VersionedModel</code> , <code>User / InteractiveUser</code> , caching utilities.
insuree → location	Families/insurees live in a village.
policy → location	Enrolment happens within a scoped area.
claim → location	A claim is filed by a <code>HealthFacility</code> in a location; scoping decides visibility.
payment → location	Capitation distributes by catchment.

Signals

- `post_save` / `post_delete` on `Location`, `UserDistrict`, `OfficerVillage` → invalidate the per-user allowed-location cache (`free_cache_for_user`) and rebuild the location graph cache. This keeps scoping correct after grants or the tree change.
- Location participates in core's **service-signal** seam so other modules can react to facility creation/updates (e.g. provisioning a price list).

Database tables

Table	Model	Notes
<code>tblLocations</code>	<code>Location</code>	Self-referential tree; filter <code>type</code> for a level. Versioned.
<code>tblHF</code>	<code>HealthFacility</code>	Facilities, their price-list FKs, care type, status.
<code>tblUsersDistricts</code>	<code>UserDistrict</code>	Primary user → district scoping grant.
<code>tblOfficerVillages</code>	<code>OfficerVillage</code>	Enrolment officer → village assignment.
<code>tblHFCatchment</code>	<code>HealthFacilityCatchment</code>	Facility population coverage.
<code>tblLegalForms</code>	<code>HealthFacilityLegalForm</code>	Reference list.
<code>tblHFSublevel</code>	<code>HealthFacilitySubLevel</code>	Reference list.



Common mistake

`tblLocations` mixes all four levels. A query for "all districts" that forgets `type='D'` will happily return regions, wards and villages too. Every level-specific query **must** filter on `type`.

Configuration

Config comes from `location/apps.py` (`LocationConfig.DEFAULT_CFG`), overlaid by the DB `ModuleConfiguration`:

Config key (illustrative)	Controls
<code>location_types</code>	The ordered list of level codes (<code>["R", "D", "W", "V"]</code>) and their labels.
<code>gql_query_locations_perms</code> , <code>gql_mutation*_locations_perms</code>	Rights for reading/editing the tree.
<code>gql_query_health_facilities_perms</code> , <code>..._mutation..._perms</code>	Rights for facility CRUD.
catchment / scoping strictness defaults	Default behaviour of the access filter.

Confirm exact keys and level labelling against your deployment's `location/apps.py` and seed data — labels are frequently localised.

Extension points

1. **Reuse the scoping filter.** Any module with a location-bearing model should call `build_user_location_filter_query` rather than reinventing access control.
2. **Add facility attributes via `json_ext`** without a migration.
3. **Bind to facility service signals** to provision related data (price lists, contracts) when a facility is created.
4. **Relabel/extend levels** through configuration for a country's geography.

Hands-on lab

Lab: prove scoping works

1. Create a small tree: one Region → two Districts → a Ward and Village under each.
2. Create a `HealthFacility` in District A.
3. Create an `InteractiveUser` and grant them **District A only** via `UserDistrict`.
4. As that user, run the `healthFacilities` and `claims` GraphQL queries. Confirm you see District A's facility but **not** District B's.
5. Add a `UserDistrict` row for District B, re-run the query, and confirm the new district appears (proving cache invalidation fired on the grant).

Knowledge check

Q1: Why are all four administrative levels stored in one table? (click for answer)

`tblLocations` is a **self-referential tree**: each row has a `type` (R/D/W/V) and a `parent` pointer. One table gives uniform recursive queries ("everything under this node"), cheap reparenting, and lets access scoping walk the tree with a single recursive CTE instead of joining four tables.

Q2: How does openIMIS stop a district officer from reading another district's claims? (click for answer)

Through **user-location scoping**. `UserDistrict` grants the user specific districts; `LocationManager.build_user_location_filter_query` turns those grants (plus all descendant locations) into a Django `Q` filter that downstream queriesets apply on the **backend**. Superusers and claim admins bypass it.

Q3: You bulk-load 5,000 new villages with `bulk_create` and users report stale visible areas. Why? (click for answer)

`bulk_create` **skips Django signals**, so the `post_save` handler that calls `free_cache_for_user()` / rebuilds the location graph never fires. The cached allowed-location sets are stale. Invalidate the location caches manually after a bulk load.

? Q4: Where can two facilities in the same district be paid different prices for the same service? (click for answer)



On the `HealthFacility` itself, via `services_pricelist / items_pricelist`. During claim valuation the facility's price list overrides the catalogue price, so identical services can carry facility-specific unit prices.

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be-location_py</code>	<code>location/models.py</code>	<code>Location</code> , <code>HealthFacility</code> , <code>UserDistrict</code> , <code>OfficerVillage</code> , <code>HealthFacilityCatchment</code> ; the <code>LocationManager</code> scoping logic.
<code>openimis-be-location_py</code>	<code>location/schema.py</code> , <code>location/graphql_queries.py</code>	GraphQL for the tree and facilities.
<code>openimis-be-location_py</code>	<code>location/apps.py</code>	Level configuration and permission codes.
<code>openimis-be-core_py</code>	<code>core/models.py</code>	<code>VersionedModel</code> , <code>User / InteractiveUser</code> , caching used by scoping.

Further reading

- Source: [openimis-be-location_py](#)
- [Claim](#) — facilities file claims; scoping decides visibility.
- [Medical & Product](#) — facility price lists.
- [Contribution & Payment](#) — catchment-based capitation.
- Official docs: [openIMIS wiki](#).

Medical & Product

This chapter covers the three modules that together answer *"what can be paid for, at what price, and under what insurance rules?"*:

- `openimis-be-medical_py` — the **catalogue**: every drug/consumable (`Item`) and every act/procedure (`Service`) the scheme knows about, plus the ICD `Diagnosis` list.
- `openimis-be-medical_pricelist_py` — **price lists**: named subsets of the catalogue with per-line prices, attached to health facilities and products.
- `openimis-be-product_py` — the **insurance product**: the benefit package that ties covered items/services to money — ceilings, deductibles, waiting periods, and contribution rates.

If **claim** is where money flows out and **contribution** is where money flows in, **product is the contract that governs both**.

Learning objectives

By the end of this chapter you will be able to:

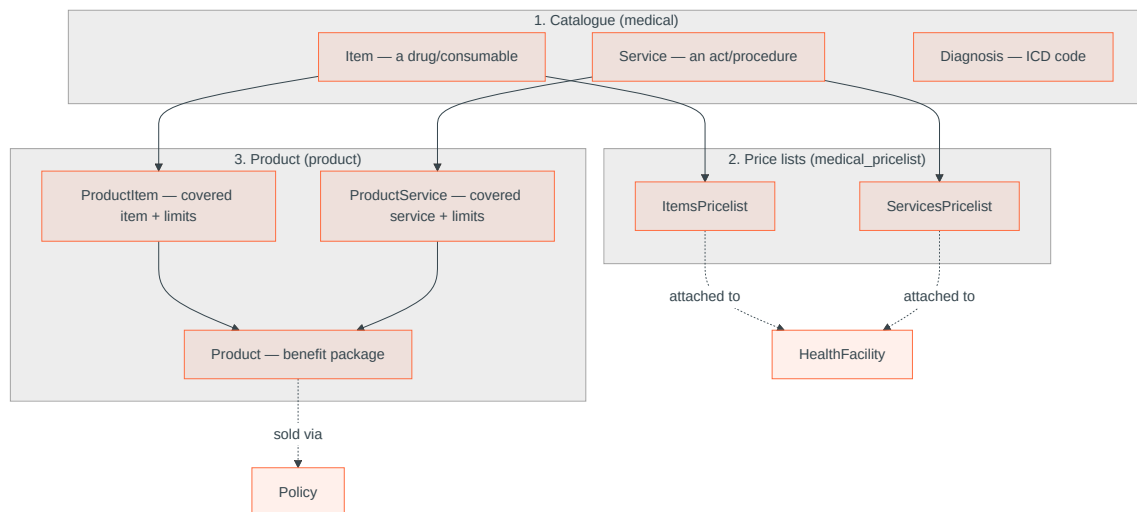
- Distinguish the **catalogue** (`Item` / `Service`) from a **price list** from a **product**.
- Explain how a `Product` defines coverage: which items/services, at what ceilings and deductibles, after what waiting periods, for what premium.
- Trace how a facility's price list plus the product's limits feed claim **checking** and valuation.
- Locate the models and tables in `openimis-be-medical_py` and `openimis-be-product_py`.

Prerequisites

- **The Core module** — `VersionedModel`, `json_ext`.
- **Location & Health Facility** — facilities carry price-list references.
- **Policy** — a policy links an insuree to a product.

Purpose & the three layers

Keep these three layers straight; conflating them is the classic beginner error.



Layer	Question it answers	Owns
Catalogue	<i>What things exist and what do they cost by default?</i>	<code>Item</code> , <code>Service</code> , <code>Diagnosis</code> with a base <code>price</code> .
Price list	<i>What does this facility charge for a subset of the catalogue?</i>	<code>ItemsPricelist</code> / <code>ServicesPricelist</code> and their detail rows.
Product	<i>What does this insurance package cover, up to what limits, for what premium?</i>	<code>Product</code> , <code>ProductItem</code> , <code>ProductService</code> .

Did you know?

The same physical drug can have **three** prices in play at once: its catalogue `price` (the default), a **facility** price-list price (what a given hospital charges), and an implicit **product** limit (the most the scheme will reimburse). Claim valuation reconciles all three. Understanding which wins is most of understanding openIMIS pricing.

Medical: the catalogue

From `openimis-be-medical_py/medical/models.py`:

Model	Table	Purpose	Notable fields
Item	tblItems	A drug or consumable	code, name, type, package, price, quantity, care_type, frequency, patient_category, maximum_amount
Service	tblServices	A medical act/procedure	code, name, type, category, packagetype, level, price, care_type, frequency, patient_category, maximum_amount
Diagnosis	tblICDCodes	An ICD diagnosis code	code, name
ServiceItem	tblProductContainedPackage	Items contained in a package service	link + quantity
ServiceService	tblServiceContainedPackage	Sub-services contained in a package service	link + quantity

Key attributes worth internalising:

- **care_type** — out-patient (0), in-patient (I) or both (B). Must match the [health facility](#) and claim care type.
- **patient_category** — a bitmask of who may receive it (adult male/female, child, etc.), used during claim checking.
- **frequency** — how often it may be billed within a period; the checker enforces it.
- **package / packagetype** — a service can be a **package** that expands into contained items/services (`ServiceItem` / `ServiceService`), which is why a claim can spawn `ClaimServiceItem` sub-lines (see [Claim](#)).

Did you know?

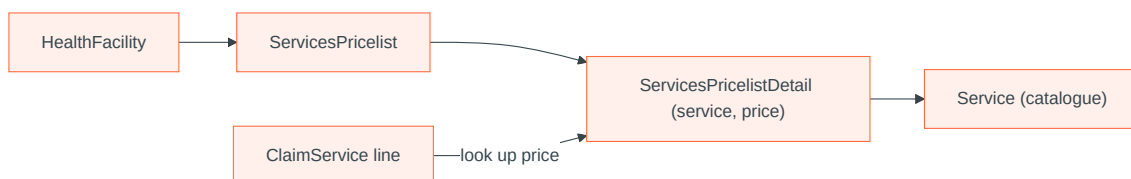
ICD diagnoses live in `tblICDCodes` and are referenced by claims (`Claim.icd`, `icd_1..4`). openIMIS ships a standard ICD-10 seed, but schemes routinely trim it to the diagnoses their reporting cares about.

Price lists

`openimis-be-medical_pricelist_py` defines **named price lists**: an `ItemsPricelist` and a `ServicesPricelist`, each with **detail rows** giving a per-line price for a subset of the catalogue.

Model	Table (illustrative)	Purpose
<code>ServicesPricelist</code>	<code>tblPLServices</code>	A named list of services with prices, valid for a location/date.
<code>ItemsPricelist</code>	<code>tblPLItems</code>	A named list of items with prices.
<code>ServicesPricelistDetail</code>	<code>tblPLServicesDetail</code>	One service + its price within a list.
<code>ItemsPricelistDetail</code>	<code>tblPLItemsDetail</code>	One item + its price within a list.

A `HealthFacility` references one `services_pricelist` and one `items_pricelist`. That is the mechanism by which **the same service costs different amounts at different facilities**: valuation looks up the price in the facility's list, falling back to the catalogue price.





Common mistake

Confusing a **price list** with a **product**. A price list says *how much a service costs at a facility*. A product says *whether the scheme covers it and up to what ceiling*. A service can be on a facility's price list yet **not** covered by a patient's product — the claim line is rejected as "not in product" even though it has a price.

Product: where everything ties to money

The `Product` (`openimis-be-product_py/product/models.py` , table `tblProduct`) is the **insurance benefit package**. A `policy` sells a product to an insuree/family; every coverage decision and every payout limit on a claim ultimately reads from the product.

The product model, grouped by what it controls

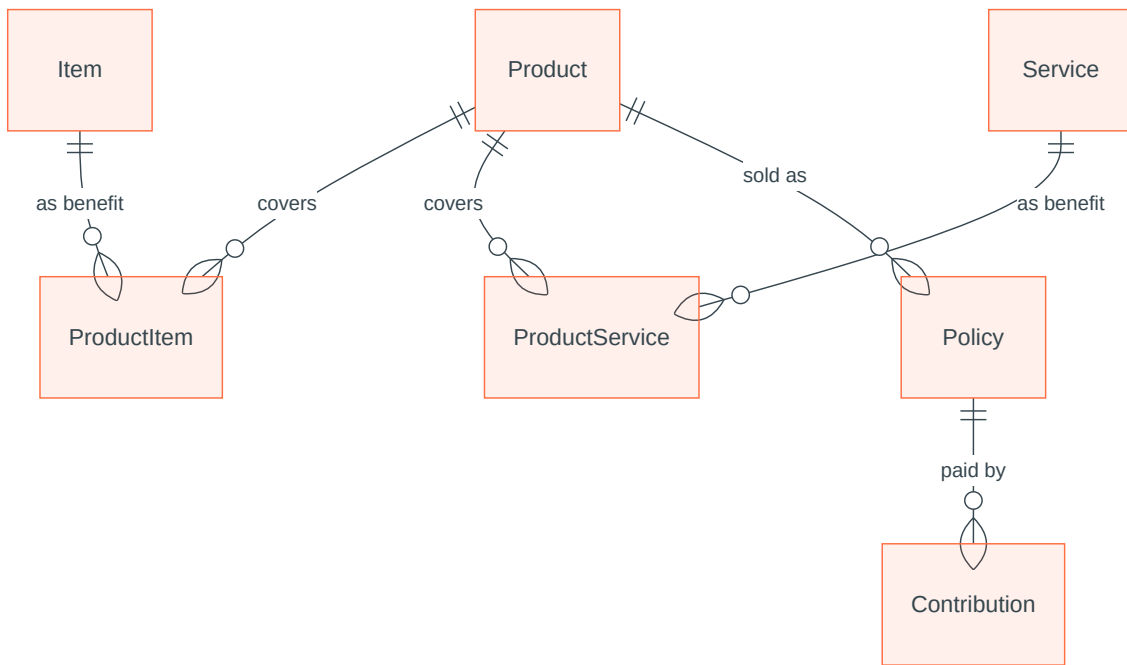
Group	Representative fields	Meaning
Identity & validity	<code>code</code> , <code>name</code> , <code>location</code> , <code>date_from</code> , <code>date_to</code> , <code>max_members</code>	Which scheme/area, valid window, family size cap.
Contributions (money in)	<code>premium_adult</code> , <code>premium_child</code> , <code>lump_sum</code> , <code>share_contribution</code> , <code>registration_fee</code> , <code>general_assembly_fee</code> , <code>threshold</code>	What members pay to enrol; <code>lump_sum</code> covers up to <code>threshold</code> members, extras add <code>premium_*</code> .
Grace & waiting	<code>grace_period_enrolment</code> , <code>grace_period_payment</code> , <code>grace_period_renewal</code>	How late payment/renewal may be; per-benefit waiting periods live on <code>ProductItem</code> / <code>ProductService</code> .
Ceilings (money out, caps)	<code>max_ceiling_policy</code> , <code>max_ceiling_policy_ip</code> , <code>max_ceiling_policy_op</code> , <code>max_insuree</code> , <code>max_op_insuree</code> , <code>max_ip_insuree</code> , <code>max_treatment</code> , <code>ceiling_type</code> , <code>ceiling_interpretation</code>	The most the scheme will pay — per policy, per insuree, per treatment, split by in/out-patient.
Deductibles	<code>ded_insuree</code> , <code>ded_op_insuree</code> , <code>ded_ip_insuree</code> , <code>ded_treatment</code> , <code>ded_policy</code>	Amounts the patient bears before the scheme pays.
Utilisation limits	<code>max_no_consultation</code> , <code>max_no_surgery</code> , <code>max_no_delivery</code> , <code>max_no_hospitalization</code> , <code>max_no_visits</code> , <code>max_amount_consultation</code> , <code>max_amount_surgery</code> , <code>max_amount_delivery</code> , <code>max_amount_hospitalization</code> , <code>max_amount_antenatal</code>	Count/amount caps per category.
Discounts & installments	<code>renewal_discount_perc</code> , <code>renewal_discount_period</code>	Renewal incentives and payment scheduling.

Group	Representative fields	Meaning
	enrolment_discount_perc , max_installments , recurrence	

Covered items/services

Model	Table	Purpose	Notable fields
ProductItem	tblProducts	Declares an Item covered by the product + its per-benefit limits	product , item , limitation_type , limit_no_adult , limit_no_child , waiting_period_adult , waiting_period_child , ceiling_exclusion_*
ProductService	tblProductsServices	Same, for a Service	product , service , limitation_type , limit_no_adult , limit_no_child , waiting_period_adult , waiting_period_child

The `limitation_type` (co-insurance percentage vs. fixed amount), the per-line `limit_no_*`, and the `waiting_period_*` are exactly the values the [claim checker](#) enforces line by line.

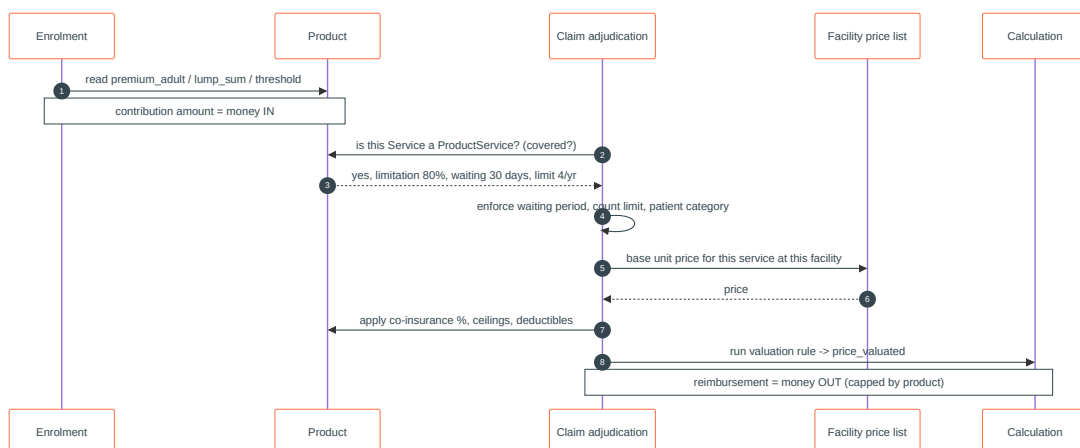


Did you know?

Product is the single richest model in openIMIS — dozens of ceiling and limit columns — because it encodes an entire insurance actuarial policy as data. That richness is deliberate: a country can define a completely new benefit package by inserting rows, **no code change**. It is the data-driven counterpart to the pluggable [calculation engine](#).

How Product ties everything to money

Follow a single covered service through the system and every layer shows up:



- **Money in** — enrolment reads `premium_adult`, `premium_child`, `lump_sum`, `threshold`, fees to compute the **contribution** that activates the policy.
- **Coverage gate** — `ProductItem` / `ProductService` decide whether a claim line is reimbursable at all.
- **Money out, capped** — `limitation_type`, `limit_no_*`, ceilings and deductibles bound the payout; the **calculation engine** computes the final `price_valuated`.

Services

Service (module)	Responsibility
<code>ItemService</code> / <code>ServiceService</code> (medical)	CRUD for catalogue entries with validity/versioning.
Pricelist services (medical_pricelist)	Build/clone price lists, add detail rows, attach to facilities.
<code>ProductService</code> / product services	Create/update products and their <code>ProductItem</code> / <code>ProductService</code> coverage rows; validate ceilings and premiums.
Product duplication helper	Clone an existing product (a new plan year) preserving coverage rows.

Products are cloned, not hand-built, each year

A scheme rarely authors a product from scratch. It **duplicates** last year's product, tweaks premiums and ceilings, and sets new validity dates. The duplication service copies all `ProductItem` / `ProductService` rows so coverage carries forward.

GraphQL

The catalogue, price lists and products are all exposed over **GraphQL** for the admin UI (`schema.py` / `gql_queries.py` in each module).

```

query {
  medicalItems(first: 10, name_Contains: "amox") {
    edges { node { uuid code name type price careType } }
  }
  medicalServices(first: 10, category: "S") {
    edges { node { uuid code name price careType } }
  }
}

query {
  products(first: 5, location_Uuid: "...") {
    edges {
      node {
        uuid code name premiumAdult lumpSum threshold
        maxCeilingPolicy gracePeriodEnrolment
        services { edges { node { service { code } limitationType limitNoAdult
waitingPeriodAdult } } }
        items { edges { node { item { code } limitationType limitNoAdult
} } }
      }
    }
  }
}

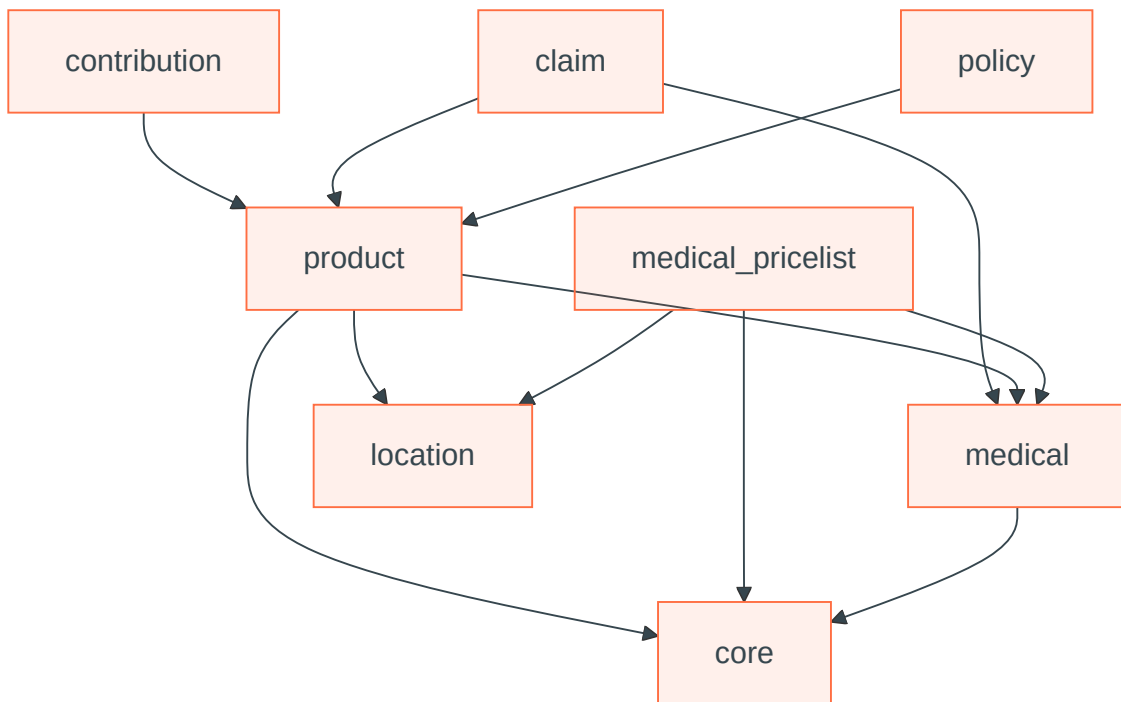
```

Catalogue

Product & coverage

Mutations (`createProduct` , `updateProduct` , `createItem` , ...) use the async `OpenIMISMutation` pattern and integer rights from each module's `apps.py` .

Dependencies



Relationship	Why
medical → core	Base models, versioning.
medical_pricelist → medical, location	Prices catalogue entries; lists attach to facilities.
product → medical, location	Coverage rows reference catalogue; product scoped to a location.
policy → product	A policy sells a product.
claim → product, medical	Coverage + limits + catalogue for adjudication.
contribution → product	Premium amounts to activate a policy.

Signals

- Product and catalogue changes emit core **service signals** other modules bind to (e.g. re-caching coverage, invalidating derived data).
- `post_save` on `Product` / `ProductItem` / `ProductService` can trigger recomputation of denormalised coverage used by the claim checker.
- Price-list changes can invalidate cached facility pricing.

Database tables

Table	Model	Notes
<code>tblItems</code>	<code>Item</code>	Catalogue drugs/consumables.
<code>tblServices</code>	<code>Service</code>	Catalogue acts/procedures.
<code>tblICDCodes</code>	<code>Diagnosis</code>	ICD diagnosis list.
<code>tblProductContainedPackage</code> / <code>tblServiceContainedPackage</code>	<code>ServiceItem</code> / <code>ServiceService</code>	Package expansion.
<code>tblPLServices</code> / <code>tblPLItems (+ ...Detail)</code>	price lists	Named lists + per-line prices (verify names in <code>medical_pricelist</code>).
<code>tblProduct</code>	<code>Product</code>	The benefit package.
<code>tblProductItems</code>	<code>ProductItem</code>	Covered items + limits/waiting periods.
<code>tblProductServices</code>	<code>ProductService</code>	Covered services + limits/waiting periods.



Common mistake

Deleting a catalogue `Item / Service` that is referenced by historical claims or products breaks referential integrity — and openIMIS's temporal model expects you to **close validity** (`validity_to`) rather than hard-delete. "Retire" catalogue entries; never `DELETE` them.

Configuration

From each module's `apps.py` (`MedicalConfig`, `ProductConfig`, `pricelist config`), overlaid by the DB `ModuleConfiguration`:

Config key (illustrative)	Controls
<code>gql_query_medical_items_perms / ..._services_perms</code>	Rights to read the catalogue.
<code>gql_query_products_perms , gql_mutation_*_products_perms</code>	Rights to read/edit products.
<code>default care_type / patient_category</code> handling	Validation defaults.
product duplication defaults	Behaviour when cloning a product year.

Extension points

1. **Add benefit dimensions via `json_ext`** on `Product` for scheme-specific rules without a migration.
2. **Register a `calculation rule`** to change how product limits translate into a payout (co-insurance, capitation).
3. **Extend the catalogue** with new item/service types and expose them over GraphQL from your own module.
4. **Custom price-list sources** — bind to price-list signals to sync prices from an external ERP.

Hands-on lab



Lab: build a minimal covered benefit

1. Create one `Item` (a drug) and one `Service` (a consultation) in the catalogue with base prices.
2. Create a `ServicesPricelist` and `ItemsPricelist`, add detail rows for your new entries, and attach both to a [health facility](#).
3. Create a `Product` with a modest `premium_adult`, a `max_ceiling_policy`, and a `ProductService / ProductItem` covering your two entries — set an 80% `limitation_type` and a 30-day `waiting_period_adult` on the service.
4. Sell the product via a [policy](#) and pay a [contribution](#) to activate it.
5. File a [claim](#) for the service **before** 30 days and confirm it is rejected for waiting period; then after 30 days and confirm it values at 80% of the facility price, capped by the ceiling.

Knowledge check



Q1: What is the difference between a price list and a product? (click for answer)



A **price list** says how much an item/service **costs** at a facility. A **product** says whether the scheme **covers** it and up to what ceiling/limit/waiting period. A line can be priced yet uncovered (rejected as "not in product"), or covered but priced differently at different facilities.



Q2: How can the same service cost different amounts at two hospitals? (click for answer)



Each `HealthFacility` references its own `services_pricelist`. Valuation reads the price from the facility's list (falling back to the catalogue `price`), so facility-specific prices apply to the identical `Service`.



Q3: Which product fields drive claim rejection during checking? (click for answer)



Coverage existence (`ProductService / ProductItem` present at all), plus `limitation_type`, `limit_no_adult / limit_no_child`, `waiting_period_*`, and the product ceilings/deductibles. The claim checker enforces these per line and sets a `rejection_reason`.

? Q4: Why does a scheme duplicate a product each plan year instead of editing it? ✓
(click for answer)

Products are **versioned** and referenced by historical policies and claims. Editing in place would rewrite the terms of already-sold coverage. Duplicating clones all coverage rows, lets the scheme adjust premiums/ceilings, and sets a new validity window while the old product stays intact for existing policies.

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be-medical_py</code>	<code>medical/model s.py</code>	<code>Item</code> , <code>Service</code> , <code>Diagnosis</code> , package models.
<code>openimis-be-medical-pricelist_py</code>	<code>pricelist/mod els.py</code>	<code>ServicesPricelist</code> / <code>ItemsPricelist</code> + detail rows.
<code>openimis-be-product_py</code>	<code>product/model s.py</code>	<code>Product</code> , <code>ProductItem</code> , <code>ProductService</code> — ceilings, deductibles, waiting periods, premiums.
<code>openimis-be-product_py</code>	<code>product/apps. py</code>	Product permissions and defaults.

Further reading

- Source: [openimis-be-medical_py](#), [openimis-be-product_py](#)
- [Claim](#) — coverage and valuation in action.
- [Policy](#) — selling a product.
- [Contribution & Payment](#) — premiums that activate policies.
- [Calculation Rules](#) — turning limits into payouts.
- Official docs: [openIMIS wiki](#).

Contribution & Payment

Money moves in two directions through a health-financing scheme, and openIMIS keeps the two rigorously separate:

- **Money in** — members (or the state on their behalf) pay **contributions** (premiums) to enrol; recorded by `openimis-be-contribution_py` as `Premium` rows against a policy.
- **Money out** — the scheme reimburses providers for valued `claims`; tracked by `openimis-be-payment_py` and the **invoice** module.

This chapter covers both flows, the models that record them, and the crucial distinction between them.

Learning objectives

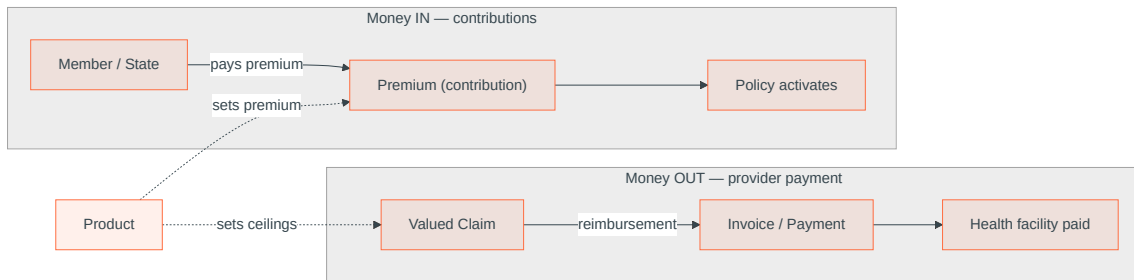
By the end of this chapter you will be able to:

- Explain the difference between a **contribution/premium** (money in) and a **payment** (which openIMIS uses primarily for money movement records).
- Trace **contribution** → **policy activation**: how paying a premium turns a policy active.
- Trace **valued claim** → **provider payment/invoice**: how money-out is recorded.
- Locate `Premium`, `Payment`, `PaymentDetail` and the invoice models.

Prerequisites

- **The Core module** — `VersionedModel`, async mutations, service signals.
- **Policy** — a contribution is paid **against a policy**.
- **Medical & Product** — the product defines the premium.
- **Claim** — valued claims are what gets paid out.

Purpose & the two flows



Common mistake

The word "payment" is overloaded, and mixing up the two flows is the single most common conceptual error here. A member paying their **premium** is a **contribution** (`openimis-be-contribution_py`, `Premium / tblPremium`). The scheme paying a **provider** is the money-out flow (`openimis-be-payment_py` + `invoice`). Keep "money in" and "money out" in separate mental columns.

Contribution — money in

Purpose

The contribution module records **premiums paid against policies**. A premium is the act of a member (or a subsidising party) putting money into the scheme to buy or renew coverage. Recording a valid premium is what **activates** a [policy](#).

Key model

From `openimis-be-contribution_py/contribution/models.py`:

Model	Table	Purpose	Notable fields
<code>Premium</code>	<code>tblPremium</code>	One contribution payment against a policy	<code>policy (FK)</code> , <code>amount</code> , <code>pay_date</code> , <code>pay_type</code> , <code>receipt</code> , <code>is_photo_fee</code>
<code>PremiumMutation</code>	<code>contribution_PremiumMutation</code>	Links a premium to its <code>MutationLog</code>	<code>premium</code> , <code>mutation</code>

Payment types (`Premium.PayTypeChoices`) are single characters:

Constant	Value	Meaning
<code>BANK_TRANSFER</code>	<code>B</code>	Bank transfer
<code>CASH</code>	<code>C</code>	Cash at a counter
<code>MOBILE</code>	<code>M</code>	Mobile money
<code>FUNDING</code>	<code>F</code>	Subsidised / third-party funding

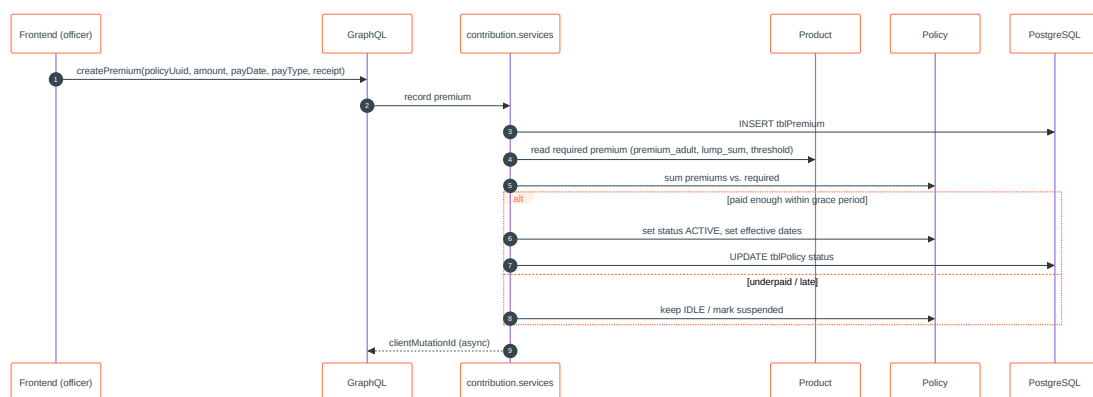
`Premium` extends core's `VersionedModel` , so premiums are temporal and audited like everything else.

i Did you know?

`is_photo_fee` is a legacy but still-used flag: some schemes charge a small fee to capture the insuree's **photo** for their membership card, recorded as a premium-like line. It is a good example of domain-specific fields the .NET heritage left behind.

Contribution → policy activation

Paying enough premium is not just bookkeeping — it changes the policy's state.



The activation logic reads the **required** contribution from the **product** (`premium_adult` , `premium_child` , `lump_sum` , `threshold` , fees) and compares it against the sum of premiums recorded, honouring the product's `grace_period_payment` . This coupling is why a premium is always tied to a policy, never floating on its own.



Contribution valuation is a calculation rule

How much premium a policy requires can itself be a pluggable **calculation rule** (e.g. income-percentage contributions rather than flat premiums). The contribution module records the money; the calculation engine can decide the amount.

Payment — money movement records

`openimis-be-payment_py` models **payment records and their reconciliation** — originally built around collecting contributions through payment channels (e.g. a mobile-money gateway sending a payment notification that must be matched to a policy), and reused for reconciliation of money movements.

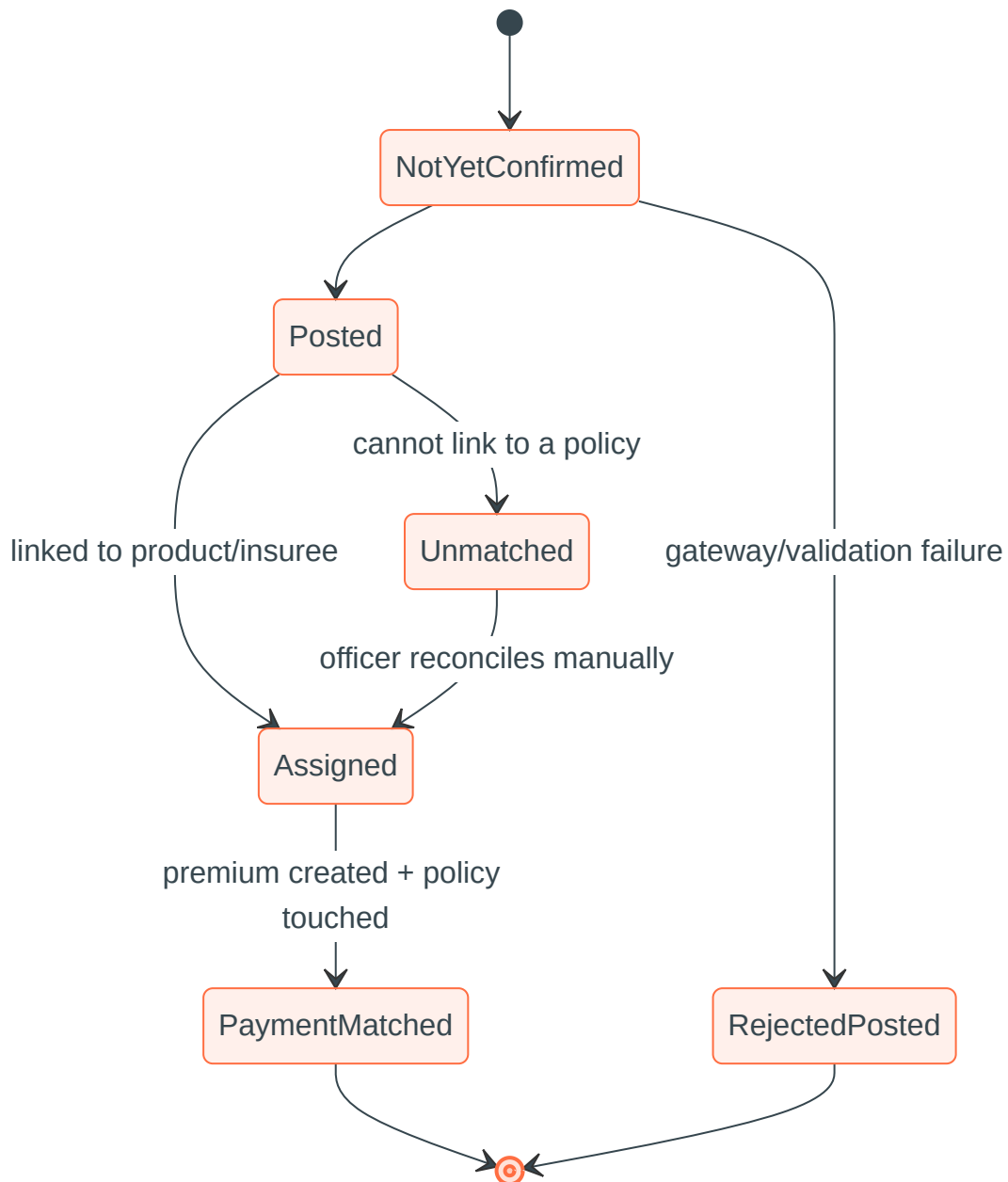
Key models

From `openimis-be-payment_py/payment/models.py`:

Model	Table	Purpose	Notable fields
<code>Payment</code>	<code>tblPayment</code>	A payment record to reconcile	<code>expected_amount</code> , <code>received_amount</code> , <code>officer_code</code> , <code>phone_number</code> , <code>request_date</code> , <code>received_date</code> , <code>status</code> , <code>transaction_no</code> , <code>receipt_no</code> , <code>payment_date</code> , <code>type_of_payment</code> , <code>transfer_fee</code> , <code>reconciliation_date</code>
<code>PaymentDetail</code>	<code>tblPaymentDetails</code>	A line linking a payment to a policy/premium	<code>payment</code> (FK), <code>product_code</code> , <code>insurance_number</code> , <code>policy_stage</code> , <code>amount</code> , <code>premium</code> (FK), <code>expected_amount</code>
<code>PaymentMutation</code>	<code>payment_PaymentMutation</code>	Links a payment to its <code>MutationLog</code>	<code>payment</code> , <code>mutation</code>

Payment **status** is an integer with negatives for rejections:

Constant	Value
STATUS_REJECTEDPOSTED_3 / _2 / _1	-3 / -2 / -1
STATUS_NOTYETCONFIRMED	1
STATUS_POSTED	2
STATUS_ASSIGNED	3
STATUS_UNMATCHED	4
STATUS_PAYMENTMATCHED	5



The `PaymentDetail.premium` foreign key is the seam where **payment** meets **contribution**: once a gateway payment is matched, it creates the `Premium` row that activates the policy.

Did you know?

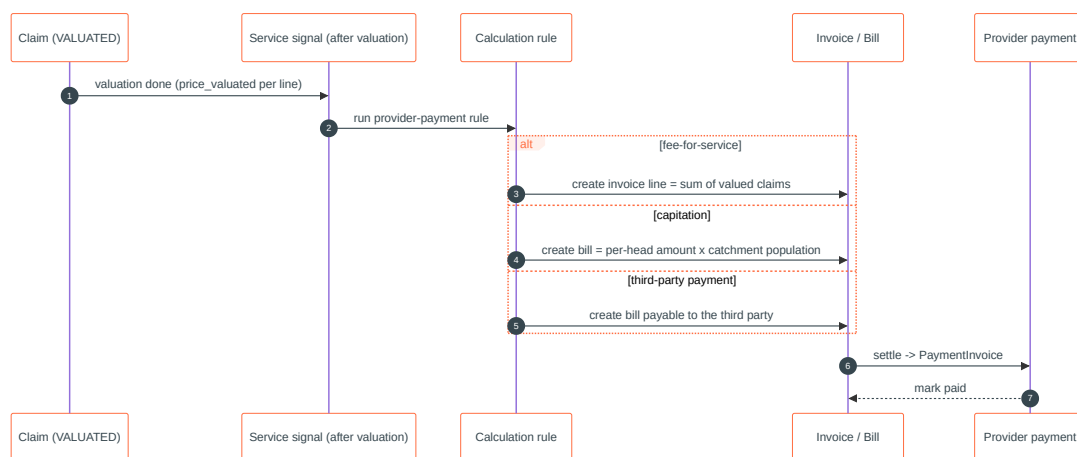
The `phone_number`, `transaction_no` and `date_last_sms` fields on `Payment` exist because openIMIS was an early adopter of **mobile-money enrolment**: a member pays by phone, the gateway posts a `Payment`, and openIMIS reconciles it to a policy — closing the loop by SMS. See [Integrations](#).

Invoice — modern money-out

The **invoice** module (`openimis-be-invoice_py` , package `invoice`) is the newer, general-purpose billing/receipting layer. Where legacy `payment` grew out of **collecting** contributions, invoice generalises **both** directions with a consistent `Invoice / InvoiceLineItem / PaymentInvoice / Bill` model, and is what modern provider-payment flows (fee-for-service, capitation, third-party payment) write to.

Model (invoice)	Purpose
<code>Invoice / InvoiceLineItem</code>	A bill and its lines (e.g. a facility's reimbursable claims for a period).
<code>Bill / BillItem</code>	The counterpart for amounts the scheme owes or is owed .
<code>PaymentInvoice / DetailPaymentInvoice</code>	Records a payment settling an invoice/bill.

Valued claim → provider payment



The claim module never calls invoice directly — it emits a **service signal** after valuation, and the invoice/calculation modules bind to it (see [Claim → Signals](#) and [Calculation](#)). That decoupling lets a deployment choose its provider-payment mechanism, or reconcile out-of-band entirely.

Services

Service (module)	Responsibility
<code>contribution.services</code> — create/update <code>Premium</code>	Record a premium and trigger policy activation logic.
Policy activation helper (policy + contribution)	Compare paid vs. required premium against product + grace period; set policy status.
<code>payment.services</code> — payment matching	Reconcile an incoming <code>Payment</code> to a policy/product; create <code>PaymentDetail</code> and the resulting <code>Premium</code> .
<code>invoice.services</code> — invoice/bill generation & settlement	Build invoices/bills from valued claims or capitation runs; record <code>PaymentInvoice</code> .

GraphQL

All three modules expose async `OpenIMISMutation`-based mutations and connection queries (see [GraphQL](#)).

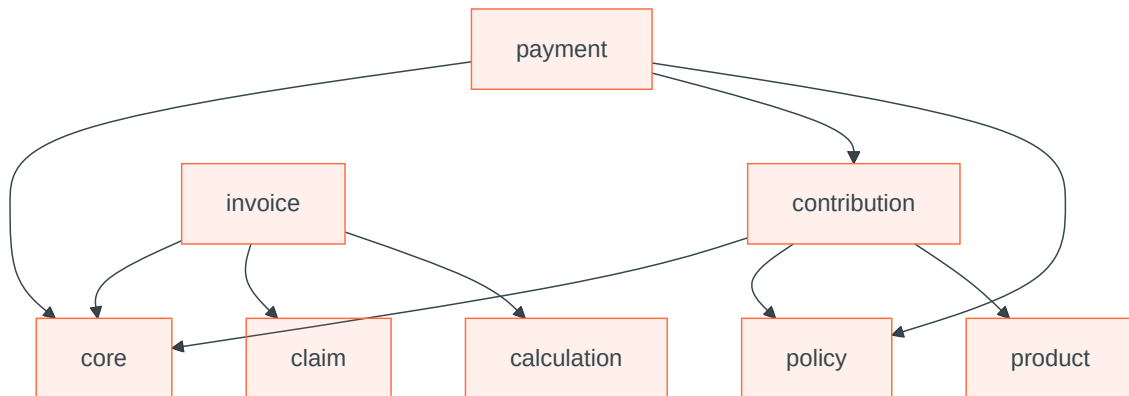
```
mutation {  
  createPremium(input: {  
    clientMutationId: "prm-1"  
    policyUuid: "..."  
    amount: "12000.00"  
    payDate: "2026-07-22"  
    payType: "M" # mobile money  
    receipt: "RCPT-0001"  
  }) { clientMutationId internalId }  
}  
  
query {  
  payments(status: 4) { # UNMATCHED  
    edges { node { uuid receivedAmount phoneNumber transactionNo status } }  
  }  
}  
  
query {  
  invoice(first: 10) {  
    edges { node { id code status amountTotal dateInvoice } }  
  }  
}
```

Record a premium

Query payments to reconcile

Invoices

Dependencies



Relationship	Why
contribution → policy , product	A premium is paid against a policy; the product sets the required amount.
payment → contribution	A matched payment creates a <code>Premium</code> .
invoice → claim , calculation	Provider payment is generated from valued claims via a rule.

Signals

- **Premium post_save** → recompute policy status (activate/renew) — the contribution → activation trigger.
- **Payment matched** → emits a signal that creates the `Premium` and touches the policy.
- **Claim valuation (service signal)** → invoice/calculation modules bind *after* it to generate provider-payment invoices/bills. This is the money-out seam and is why claim does not import invoice.

Database tables

Table	Model	Notes
<code>tblPremium</code>	<code>Premium</code>	Contribution against a policy. Money in.
<code>contribution_PremiumMutation</code>	<code>PremiumMutation</code>	Audit join.
<code>tblPayment</code>	<code>Payment</code>	Payment record / reconciliation. Integer status incl. negatives.
<code>tblPaymentDetails</code>	<code>PaymentDetail</code>	Links payment to policy/premium.
<code>payment_PaymentMutation</code>	<code>PaymentMutation</code>	Audit join.
invoice tables (<code>invoice_*</code>)	<code>Invoice</code> , <code>Bill</code> , <code>PaymentInvoice</code> , ...	Modern billing/receipting. Money out (and general).

Configuration

From each module's `apps.py`, overlaid by DB `ModuleConfiguration`:

Config key (illustrative)	Controls
<code>gql_mutation_create_premiums_permissions</code>	Rights to record contributions.
<code>receipt_length</code> / receipt format	Premium/payment receipt formatting.
payment gateway settings	Matching thresholds, SMS templates, gateway keys.
invoice numbering & tax config	Invoice/bill code sequences and any tax handling.

Extension points

1. **Contribution valuation rule** — plug an income-based premium formula via [calculation](#) instead of flat premiums.
 2. **Payment gateway adapter** — bind to payment signals to accept notifications from a new mobile-money or bank provider (see [Integrations](#)).
 3. **Provider-payment rule** — register a capitation or third-party-payment rule that binds to claim valuation and writes invoices/bills.
 4. `json_ext` on premiums/invoices for scheme-specific attributes.
-

Hands-on lab



Lab: money in, then money out

1. With a [policy](#) on a product whose `premium_adult` is, say, 12000, record a `createPremium` of the full amount with `payType: "C"`.
 2. Confirm the policy transitions to **active** with correct effective dates.
 3. Record a **partial** premium on a second policy and confirm it does **not** activate (underpaid).
 4. File and value a [claim](#) at that facility.
 5. Inspect whether an invoice/bill line was generated for the provider, and explain in one sentence why claim did not need to import the invoice module.
-

Knowledge check



Q1: A member pays their yearly premium by mobile money. Which module and model records it? (click for answer)



The **contribution** module: a `Premium` row (`tblPremium`) against their policy, with `pay_type = "M"` . If the money arrived via a gateway, a `Payment / PaymentDetail` may be reconciled first and then create the `Premium` .

? Q2: What actually activates a policy? (click for answer) ✓

Recording **sufficient premium** against it within the product's `grace_period_payment`. The contribution/policy activation logic compares the sum of premiums to the required amount (from the product) and flips the policy to active with effective dates.

? Q3: Why does 'payment' have negative status codes? (click for answer) ✓

`Payment` models reconciliation of incoming money, so it needs to represent **rejections** (`STATUS_REJECTEDPOSTED_1/_2/_3` = -1/-2/-3) distinctly from the positive progression `NotYetConfirmed(1) → Posted(2) → Assigned(3) → Unmatched(4) → PaymentMatched(5)`.

? Q4: How does a valued claim become a provider payment without claim importing the invoice module? (click for answer) ✓

Claim emits a **service signal** after valuation. The calculation/invoice modules bind to it and generate the appropriate invoice or bill (fee-for-service, capitation, or third-party payment). The coupling is via the signal seam, not a direct import.

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be-contribution_py</code>	<code>contribution/mode ls.py</code>	<code>Premium</code> ; pay-type choices.
<code>openimis-be-payment_py</code>	<code>payment/models.p y</code>	<code>Payment</code> , <code>PaymentDetail</code> ; status constants.
<code>openimis-be-invoice_py</code>	<code>invoice/models.p y</code>	<code>Invoice</code> , <code>Bill</code> , <code>PaymentInvoice</code> — modern billing.
<code>openimis-be-product_py</code>	<code>product/models.p y</code>	Premium amounts and grace periods that govern activation.

Further reading

- Source: [openimis-be-contribution_py](#), [openimis-be-payment_py](#)
- [Policy](#) — what a contribution activates.
- [Claim](#) — the money-out source.
- [Calculation Rules](#) — contribution and provider-payment formulas.
- [Integrations](#) — mobile-money gateways and SMS.
- Official docs: [openIMIS wiki](#).

Calculation Rules

Every health-financing scheme prices things differently. One country pays providers **fee-for-service**; another pays **capitation** per registered head; a third splits payments with a **third party**; contributions may be a flat premium in one scheme and a **percentage of income** in another. Hard-coding any of these formulas into the **claim** or **contribution** modules would force every country to fork the core. openIMIS instead ships a **pluggable calculation engine** (`openimis-be-calculation_py`) and a family of `calcrule_*` modules that register formulas at startup.

This chapter explains why that engine exists, how a rule registers itself via signals, and how the platform invokes the right rule at the right moment.

Learning objectives

By the end of this chapter you will be able to:

- Explain **why** openIMIS externalises pricing/valuation into pluggable rules.
- Describe the `ABSCalculationRule` interface a rule implements.
- Trace how a `calcrule_*` module **registers** via the calculation module's signals at `ready()`, and how `run_calculation_rules` **invokes** it.
- Name the common rule families (contribution valuation, capitation, third-party payment, commission, fees) and where each fires.

Prerequisites

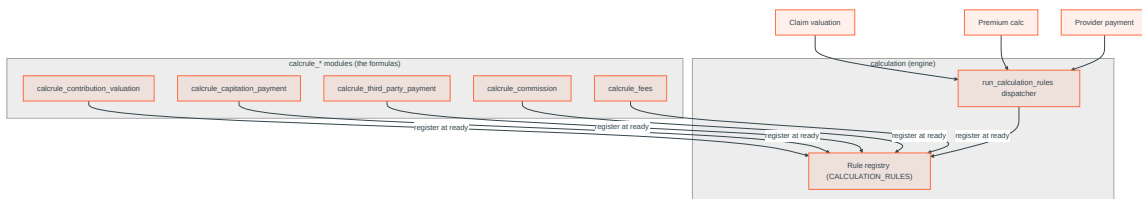
- **The Core module** — the **service-signal** seam (`register_service_signal / bind_service_signal`) and Django's `AppConfig.ready()`.
- **Claim** — valuation is a prime consumer of rules.
- **Contribution & Payment** — premiums and provider payment are computed by rules.

Purpose — why a pluggable engine?

Consider the alternatives for "how much does the scheme pay for this claim?":

- **Hard-code it** in the claim module → every country forks; upgrades become merge nightmares.
- **Make it configuration data** (like the [product](#) ceilings) → works for *parameters* but not for genuinely different **formulas** (a capitation head-count sum is not the same *shape* of computation as a fee-for-service line sum).
- **Make it code, but pluggable** → a country ships a small `calcrule_*` module implementing its formula; core discovers and calls it. Nobody forks.

openIMIS chose the third option. The calculation module is a thin **registry + dispatcher**; the actual arithmetic lives in independently versioned rule modules.



Did you know?

The calculation module deliberately uses a **no-database approach** for the rules themselves — a rule is a **class in code**, not a row you edit. Its *parameters* may be stored (e.g. a capitation rate on a product's `json_ext`), but the *formula* is versioned Python. That keeps computations auditable and reproducible: you can point at the exact commit that priced a historical claim.

The rule interface

Every rule subclasses `ABSCalculationRule` (the abstract base provided by the core/calculation framework). It is a **strategy object**: the engine holds a collection of them and asks each whether it applies to a given object, then asks the applicable one to compute.

Method (illustrative)	Responsibility
<code>ready()</code> (in the module's <code>apps.py</code>)	Instantiate the rule and register it with the engine's signals.
<code>active_for_object(instance, context, ...)</code>	Boolean — does this rule apply to <i>this</i> object in <i>this</i> context?
<code>check_calculation(instance)</code>	Is this rule the correct/configured one for the instance's linked class?
<code>calculate(instance, *args, **kwargs)</code>	The actual formula — returns the computed value(s).
<code>get_parameters(sender, class_name, instance)</code>	Expose the tunable parameters the rule reads (rendered in the UI).
<code>get_linked_class(sender, class_name)</code>	Declare which openIMIS models this rule attaches to.
<code>get_rule_name</code> / <code>get_rule_details</code>	Identity/metadata used by the dispatcher and UI.
<code>uuid</code> (property)	Stable identifier for the rule.

```
# Illustrative – the shape of a calcrule, not exact source.
# See openimis-be-calculation_py + any calcrule_* module.
from calculation.models import ABSCalculationRule

class CapitationPaymentRule(ABSCalculationRule):
    uuid = "0e1b6d4c-..."
    calculation_rule_name = "Capitation payment"

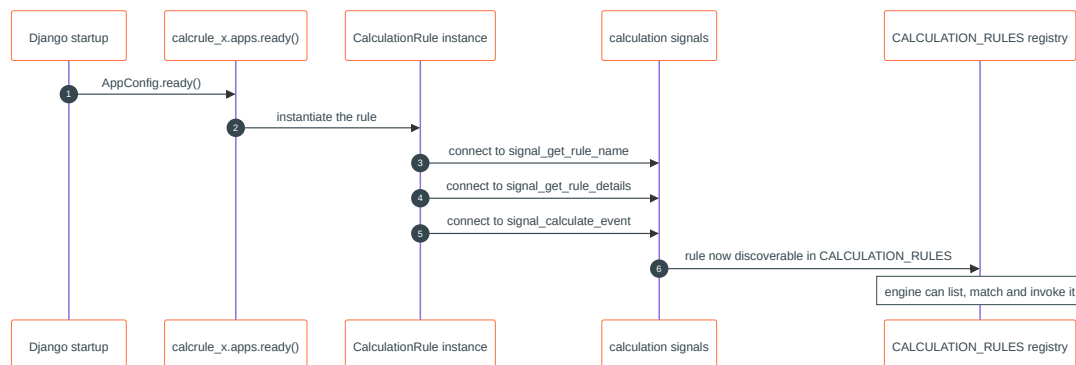
    @classmethod
    def active_for_object(cls, instance, context, type, sub_type):
        return context == "PaymentPlan" and sub_type == "capitation"

    @classmethod
    def check_calculation(cls, instance):
        return cls.uuid == str(instance.calculation)

    @classmethod
    def calculate(cls, instance, *args, **kwargs):
        population = catchment_population_for(instance.health_facility)
        rate = instance.json_ext["capitation_rate"]
        return population * rate
```

Registration — how a rule joins the engine

Registration happens **once, at startup**, using Django's `AppConfig.ready()` and the calculation module's **signals**. This is the same "publish yourself into a registry so core can find you without importing you" pattern used across openIMIS's plugin seams.



The calculation module defines signals such as `signal_get_rule_name`, `signal_get_rule_details`, and `signal_calculate_event`. A `calcrule_*` module's `ready()` connects its rule to these; the rule thereby lands in the engine's `CALCULATION_RULES` collection (imported from `calculation.apps`). No module imports another's class directly — they meet on the signal bus.

Common mistake

If your rule never fires, the usual cause is that its `apps.py` is not registering it in `ready()`, or the module isn't in the assembly's `openimis.json` manifest (so its `AppConfig` never loads). A rule that isn't connected to the signals at startup is invisible to the dispatcher — there is no lazy discovery.

Invocation — how a rule is called

When the claim, contribution, or payment code needs a number, it calls the engine's dispatcher rather than any specific rule:

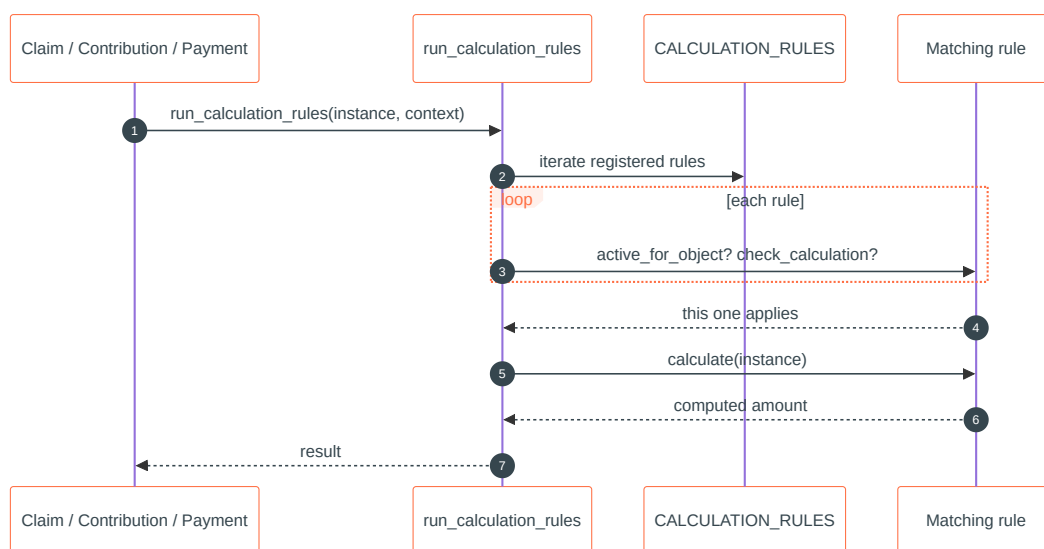
```
# Illustrative – consumer side.
# See openimis-be-calculation_py/calculation/services.py
from calculation.services import run_calculation_rules

result = run_calculation_rules(instance, context="Claim", user=user)
```

Inside, the dispatcher walks `CALCULATION_RULES`, asks each rule `active_for_object(...)` / `check_calculation(...)`, and calls `calculate(...)` on the one that matches. Supporting

helpers you will meet in `calculation/services.py` :

Function	Role
<code>run_calculation_rules(instance, context, user, **kwargs)</code>	Entry point — dispatch to applicable rules.
<code>get_rule_name(class_name)</code> / <code>get_rule_details(class_name)</code>	Identity/metadata lookups.
<code>get_calculation_object(uuid)</code>	Fetch a specific rule by UUID.
<code>get_parameters(class_name, instance)</code>	Gather a rule's tunable parameters.
<code>get_linked_class(class_name_list)</code>	Resolve which models a rule attaches to.



The rule families

The `calcrule_*` modules are separate repos, each a formula family. Common ones (names as they appear in the ecosystem — confirm exact repo names against the assembly manifest for your release):

Rule family (module)	Fires during	Computes
<code>calcrule_contribution_valuation</code>	premium recording / policy enrolment	The required contribution for a policy (flat, income-percentage, etc.).
<code>calcrule_capitation_payment</code>	provider payment run	A per-head amount per facility from <code>catchment population</code> , independent of individual claims.
<code>calcrule_third_party_payment</code>	provider payment	The share payable to a third party (e.g. a fund settling on behalf of members).
<code>calcrule_commission</code>	enrolment / renewal	Commission owed to enrolment officers/agents.
<code>calcrule_fees</code>	various	Administrative or registration fees.

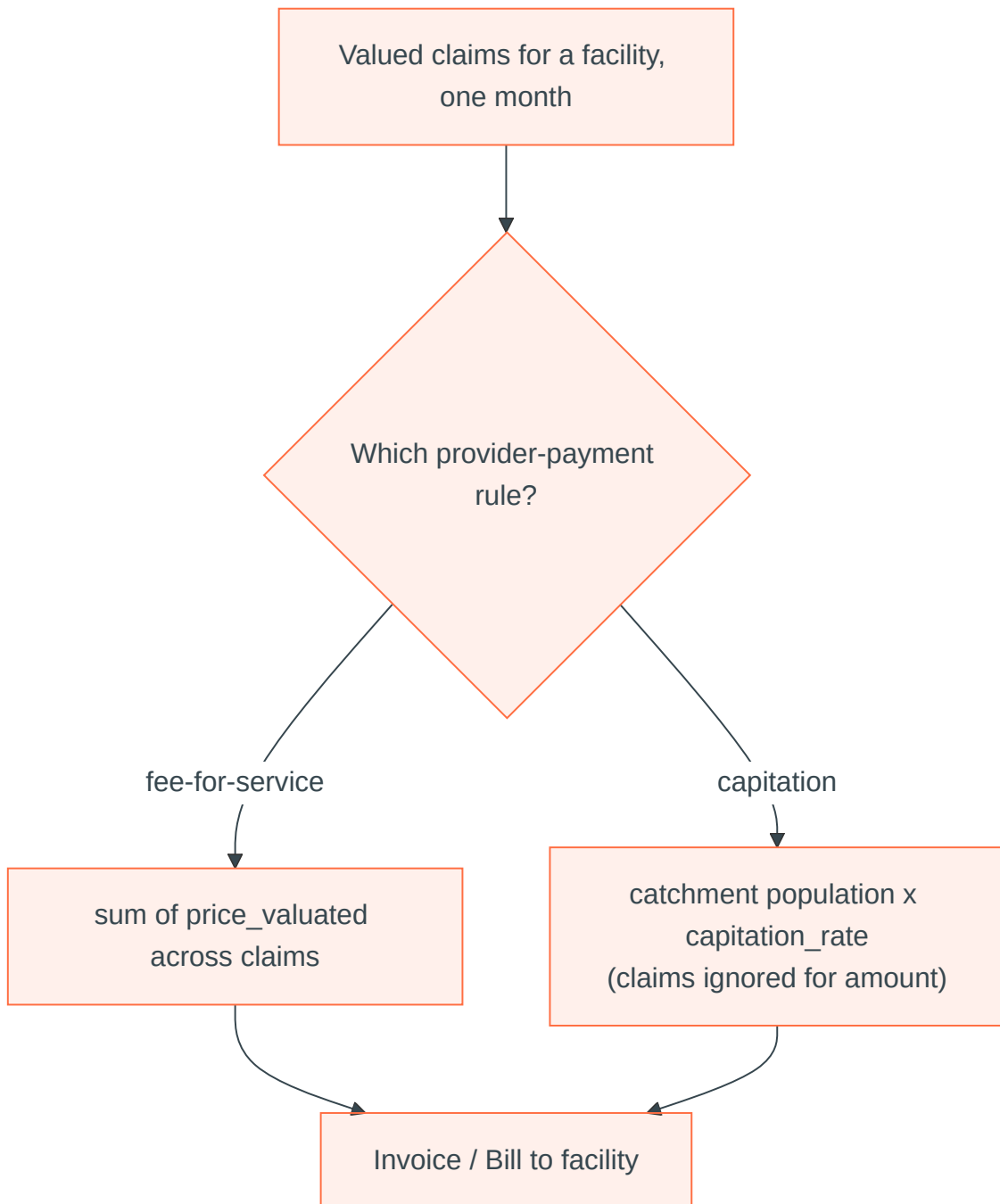
Because each is a plugin, a deployment includes **only the rules it needs**. A scheme that pays purely fee-for-service simply omits the capitation module.

Did you know?

The same engine now underpins newer domains too — `social_protection` and `payroll` use calculation rules to size benefit payments and payroll runs. The abstraction outgrew its original claim/contribution home, which is the best possible evidence that the pluggable design was the right call.

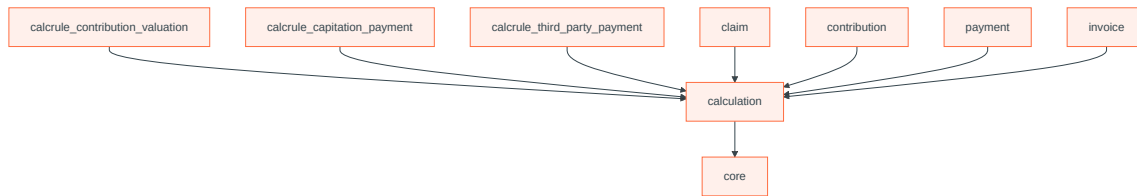
Worked example: capitation vs. fee-for-service

The clearest way to feel the value of the engine is to see the **same claim pipeline** yield **different money** depending on which rule is registered.



Same **claims**, same **products**, same **facilities** — the **only** thing that changes is which `calcrule_*` module is installed and configured. No core code differs between the two countries. That is the entire thesis of the calculation engine.

Dependencies



Relationship	Why
calculation → core	<code>AppConfig.ready()</code> , service signals, base abstractions.
<code>calcrule_*</code> → calculation	Rules register with the engine's signals.
claim / contribution / invoice → calculation	Consumers call <code>run_calculation_rules</code> .

Signals

The engine **is** a signal hub. The key signals (defined by the calculation module) that rules connect to and consumers drive:

Signal (illustrative name)	Purpose
<code>signal_get_rule_name</code>	A rule announces its human-readable identity.
<code>signal_get_rule_details</code>	The engine queries rules matching a linked class.
<code>signal_calculate_event</code>	Triggers the actual computation on the matching rule.

Rules connect to these in their `apps.ready()` ; consumers fan out through `run_calculation_rules` , which sends the calculate event.

Configuration

A rule reads its **parameters** from configuration/data even though its **logic** is code:

Config source	Example
Product / payment-plan <code>json_ext</code>	<code>capitation_rate</code> , income-percentage bands.
Module <code>apps.py</code> <code>DEFAULT_CFG</code> (per <code>calcrule_*</code>)	Which contexts/sub-types the rule activates for; permission codes.
DB <code>ModuleConfiguration</code> overlay	Operator overrides per deployment (see Configuration).



Common mistake

Do not put scheme-specific numbers **inside** the rule's Python. Read them from configuration or `json_ext` so the same rule module serves every deployment. Hard-coding a rate in `calculate()` defeats the purpose and forces a fork the moment two schemes disagree on the number.

Extension points

1. **Write a new `calcrule_*` module.** Subclass `ABSCalculationRule` , implement `active_for_object` / `check_calculation` / `calculate` , register in `apps.ready()` , add it to the assembly `openimis.json` .
2. **Parameterise via `json_ext`** on the linked object so operators tune it without code.
3. **Expose parameters over GraphQL** through `get_parameters` so the UI can edit them.
4. **Bind a rule to a claim/contribution service signal** if it must fire automatically at a lifecycle moment (see [Claim](#) and [Payment](#)).



Deep dive: how the engine decides *which* rule wins



Multiple rules can be registered at once. The dispatcher narrows by **context** and **sub-type** (`active_for_object`) and then by **configuration binding** (`check_calculation` compares the rule's `uuid` to the one selected on the object, e.g. a payment plan's `calculation` field). That two-stage match — *could this rule apply?* then *is it the configured one?* — lets a deployment install several capitation variants and pick one per facility or product without ambiguity.

Hands-on lab

Lab: sketch a calc rule

You will not deploy a full module, but you will design one on paper and verify the wiring points in the source.

1. In `openimis-be-calculation.py`, find `calculation/services.py` and locate `run_calculation_rules` and `CALCULATION_RULES`. Trace how a rule is chosen.
2. Find the signal names the module defines and confirm they match `signal_get_rule_name` / `signal_get_rule_details` / `signal_calculate_event`.
3. Pick a real `calcrule_*` repo and open its `apps.py`. Identify the exact lines in `ready()` that connect the rule to those signals.
4. Write (on paper) an `active_for_object` and `calculate` for a **flat 5% officer commission on each renewal**, and state where its 5% parameter should live (answer: `json_ext / config`, not code).

Knowledge check

Q1: Why not just store payment formulas as configuration data like product ceilings? (click for answer)

Configuration handles **parameters** but not different **shapes** of computation. Fee-for-service (sum valued lines), capitation (population × rate), and third-party splits are structurally different algorithms. A pluggable **code** rule expresses the algorithm; configuration/ `json_ext` supplies its numbers.

Q2: When and how does a calc rule become discoverable to the engine? (click for answer)

At **startup**, in the module's `AppConfig.ready()`, which connects the rule to the calculation module's signals (`signal_get_rule_name`, `signal_get_rule_details`, `signal_calculate_event`). It then lives in `CALCULATION_RULES`. There is no lazy discovery — an unregistered rule is invisible.

? Q3: A consumer needs a number. Which function does it call, and how is the right rule selected? (click for answer) ✓

It calls `run_calculation_rules(instance, context, user)`. The dispatcher iterates registered rules, filters by `active_for_object` (context/sub-type) and `check_calculation` (is this the configured rule, by `uuid` ?), then calls `calculate` on the winner.

? Q4: Two countries run identical claim code but pay providers completely differently. How? (click for answer) ✓

They install different `calcrule_*` modules (e.g. `calcrule_capitation_payment` vs. a fee-for-service rule) and configure the payment plan/product to select it. The claim pipeline is unchanged; only the registered valuation/payment rule differs.

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be-calculation_py</code>	<code>calculation/services.py</code>	<code>run_calculation_rules</code> , dispatcher helpers, <code>CALCULATION_RULES</code> .
<code>openimis-be-calculation_py</code>	<code>calculation/apps.py</code>	Signal definitions and the registry.
<code>openimis-be-calculation_py</code>	<code>calculation/models.py</code>	<code>ABSCalculationRule</code> base interface.
<code>calcrule_*</code> (e.g. <code>calcrule_capitation_payment</code>)	<code>*/apps.py</code> , <code>*/calculation_rule.py</code>	A concrete rule and its <code>ready()</code> registration.

Further reading

- Source: [openimis-be-calculation_py](#)
- [Claim](#) — valuation as a rule consumer.
- [Contribution & Payment](#) — premium and provider-payment rules.

- [Core module](#) — the service-signal seam these rules build on.
- Official docs: [openIMIS wiki](#).

FHIR R4 API Module

Module repository: `openimis-be-api_fhir_r4_py` · Python package: `api_fhir_r4`

The FHIR module is openIMIS's **standards-based interoperability layer**. While the rest of the platform speaks [GraphQL](#) to its own React frontend, `api_fhir_r4` exposes a parallel **REST API** that speaks **HL7 FHIR R4** — the global lingua franca for exchanging health data. This is how openIMIS talks to hospital information systems, national health data exchanges, mobile apps, and other clinical software that has never heard of an "Insuree" but knows exactly what a FHIR `Patient` is.

Learning objectives

By the end of this chapter you will be able to:

- Explain **why** openIMIS ships a FHIR REST API *in addition to* GraphQL.
- Map the core openIMIS domain models to their FHIR R4 resource equivalents.
- Trace a FHIR request from HTTP through the module's serializers to the ORM.
- Authenticate against and call the `/api_fhir_r4/` endpoints.
- Extend the mapping for a new resource or a country-specific profile.

Prerequisites

- [Platform Overview](#) and [Terminology](#)
- [Insuree](#), [Policy](#), [Claim](#), [Location & Health Facility](#) modules
- [Security: Authentication & Authorization](#)
- The broader [Integrations](#) chapter — this page is the **module deep dive**; that page is the **landscape view**.

Purpose

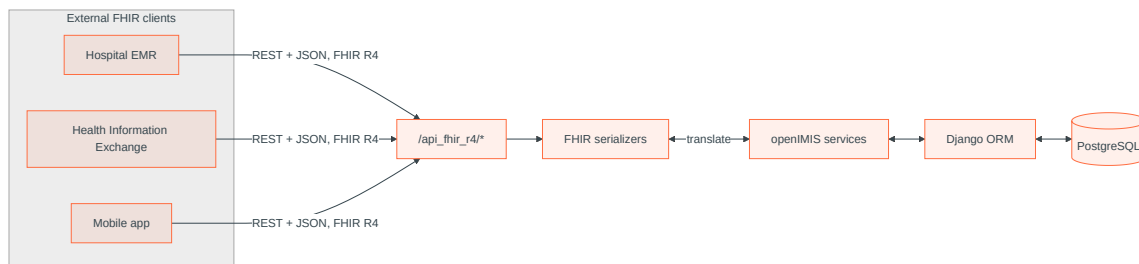
Health systems are ecosystems of software that must exchange data: an EMR at a hospital, a national beneficiary registry, a claims clearinghouse, a mobile enrolment app. Forcing every one of them to learn openIMIS's internal GraphQL schema — with its legacy table names and integer permission codes — would be a non-starter. Instead, openIMIS implements **FHIR R4**

(Fast Healthcare Interoperability Resources), the HL7 standard that the rest of the digital-health world already speaks.

Did you know?

FHIR is *resource-oriented REST*, the opposite design philosophy to openIMIS's own GraphQL API. openIMIS deliberately runs **both**: GraphQL for its own tightly-coupled frontend (fast iteration, exact field selection) and FHIR REST for **loosely-coupled external partners** (stable, standardized, self-describing). Choosing the right API style per consumer is an architecture lesson in itself — see the [Architecture Critique](#).

The module's job is therefore a **translation layer**: it maps openIMIS domain objects ↔ FHIR resources in both directions, exposes them over conventional REST endpoints, and enforces the same permission model as the rest of the platform.



Key resource mappings

FHIR calls its data objects **resources**. The heart of this module is the map between openIMIS models and FHIR resources. (See the fuller table with endpoints in the [Integrations chapter](#).)

openIMIS model	FHIR R4 resource	Notes
Insuree (insuree)	Patient	The beneficiary. CHF/insurance ID becomes a Patient identifier.
Family	Group / Patient links	Household grouping.
Policy (policy)	Coverage	Active insurance coverage for a period.
Product (medical/product)	InsurancePlan	The benefit package behind a Coverage.
Claim (claim)	Claim / ClaimResponse	Submission in, adjudication out.
ClaimItem / ClaimService	Claim item entries	Line items reference medical codes.
HealthFacility (location)	Location / Organization	Where care is delivered.
Location hierarchy	Location (nested)	Region → District → Ward → Village.
ClaimAdmin / medical staff	Practitioner / PractitionerRole	The clinician.
Item / Service (medical)	Medication / ActivityDefinition	The catalogue of billable things.

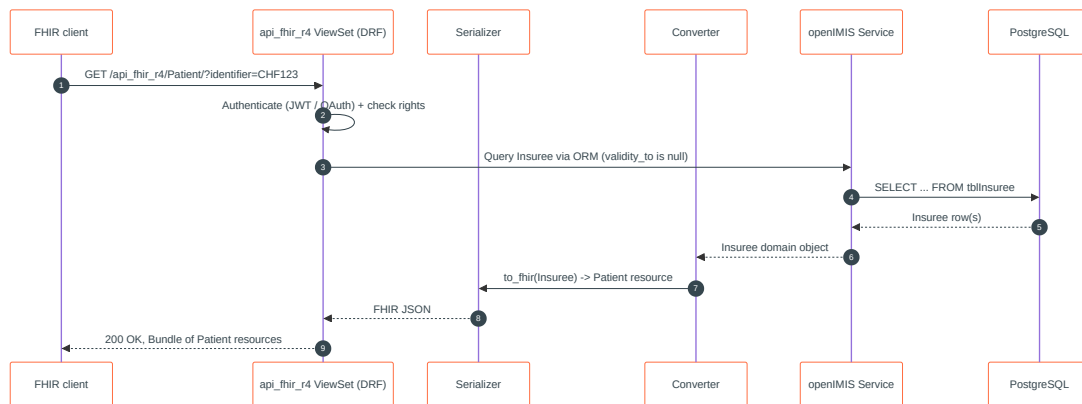


Read the source for the exact profile

FHIR is *profiled* per implementation — openIMIS constrains and extends base FHIR to fit its domain. The authoritative definition of each mapping lives in `api_fhir_r4/converters/` (one converter class per resource, e.g. `PatientConverter`, `ClaimConverter`, `CoverageConverter`) and the serializers in `api_fhir_r4/serializers/`. When in doubt, read the converter.

Architecture: how a FHIR request is served

Unlike the GraphQL side, this module is built on **Django REST Framework** (which you already know). Each resource has a DRF `ViewSet`, a `Serializer`, and a `Converter` that does the openIMIS ↔ FHIR object translation.



The reverse path (a `POST` of a FHIR `Claim`) runs the converter `from_fhir(...)` direction, then hands the resulting domain object to the `claim service` — reusing the *same* business logic and validation that the GraphQL mutation uses. **There is one business core; FHIR is just another door into it.**

⚡ Common mistake

Treating the FHIR module as a separate application with its own rules. It is a **thin translation shell over the existing services**. If you add validation or side effects in a FHIR serializer that the GraphQL path doesn't have, the two APIs will silently diverge. Put business rules in the service layer (see [Best Practices](#)); keep converters purely about shape.

Endpoints & authentication

All endpoints live under the `/api_fhir_r4/` prefix (registered via the module's `urls.py`, collected into `openimis-be_py/openimis/urls.py` — see the [Plugin System](#)).

```
# Base URL pattern
https://<host>/api_fhir_r4/<Resource>/

# Examples (illustrative)
GET /api_fhir_r4/Patient/ # search insurees as Patients
GET /api_fhir_r4/Patient/<uuid>/ # one insuree
GET /api_fhir_r4/Coverage/?patient=<uuid> # policies for an insuree
POST /api_fhir_r4/Claim/ # submit a claim as a FHIR Claim
GET /api_fhir_r4/CommunicationRequest/ # claim feedback requests
```

Authentication reuses the platform's mechanisms (see [Security](#)): a **JWT** bearer token or an **OAuth2 / OpenID Connect** access token from a configured external identity provider. The same **integer rights** that gate the GraphQL resolvers gate the FHIR viewsets — a client that cannot read claims in GraphQL cannot read them as FHIR either.

Did you know?

The OAuth2 support matters specifically *because* of FHIR. External systems can't log in through the React UI to get a cookie, so machine-to-machine FHIR clients obtain a bearer token from an IdP and present it on every REST call.

Configuration

Like every module, `api_fhir_r4` carries a `DEFAULT_CFG` in its `apps.py` (`ApiFhirConfig`) that is overlaid at startup by the DB-backed `ModuleConfiguration`. Typical settings include the identifier systems/URIs used for FHIR `identifier` slices (e.g. the URL that namespaces a CHF number), default issuer/base URLs, and which permission codes guard each resource.

Config concern	Where	Why it matters
Identifier system URIs	<code>DEFAULT_CFG</code> in <code>apps.py</code>	FHIR identifiers must be namespaced by a system URI; countries differ.
Per-resource permissions	<code>DEFAULT_CFG</code> / <code>ModuleConfiguration</code>	Reuse platform rights; no separate ACL.
Base / issuer URLs	<code>ModuleConfiguration</code>	Deployment-specific, set without code change.

Extension points

- **New resource / new mapping** — add a `Converter` + `Serializer` + `ViewSet` under `api_fhir_r4/` and register its route in the module `urls.py`.
- **Country profile** — extend an existing converter to emit/accept extra FHIR extensions or constrained value sets, driven by `ModuleConfiguration` so you don't fork.
- **Reuse the service layer** — always route writes through the owning module's service (`claim.services`, `insuree.services`, ...) so business rules stay in one place.

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be-api_fhir_r4_py</code>	<code>api_fhir_r4/converters /</code>	The openIMIS ↔ FHIR translation, one class per resource.
<code>openimis-be-api_fhir_r4_py</code>	<code>api_fhir_r4/serializers/</code>	DRF serializers producing/consuming FHIR JSON.
<code>openimis-be-api_fhir_r4_py</code>	<code>api_fhir_r4/views.py / urls.py</code>	The REST viewsets and <code>/api_fhir_r4/</code> routes.
<code>openimis-be-api_fhir_r4_py</code>	<code>api_fhir_r4/apps.py</code> (<code>ApiFhirConfig</code>)	<code>DEFAULT_CFG</code> : identifier systems, permissions.
<code>openimis-be_py</code>	<code>openimis/urls.py</code>	Where the module's routes are mounted into the project.
Owning modules	<code>claim</code> , <code>insuree</code> , <code>policy</code> , <code>location</code> <code>services</code>	The business logic FHIR delegates to.

Hands-on lab



Lab: Read and post FHIR

1. Bring up the stack ([Setup](#)) and obtain a JWT or OAuth token for a user with insuree/claim rights.
2. GET `/api_fhir_r4/Patient/` and confirm you receive a FHIR **Bundle**. Find the `identifier` that carries the insurance/CHF number.
3. GET `/api_fhir_r4/Coverage/?patient=<uuid>` for that patient and correlate the result with the [Policy](#) you can see in the UI.
4. Read `api_fhir_r4/converters/patient_converter.py` (name may vary) and match three FHIR `Patient` fields back to `Insuree` model fields.
5. **Stretch:** POST a minimal FHIR `Claim` and observe that it flows through the [same claim submission workflow](#) as a GraphQL mutation.

Exercises

1. Explain to a teammate why the same permission denies both the GraphQL and the FHIR path to a claim.
2. A partner needs an extra national identifier on every Patient. Describe the change (converter + config) without forking core.
3. Compare the FHIR `Claim` → `ClaimResponse` cycle to openIMIS's internal [claim status bitfield](#). Where do they line up, where not?

Knowledge check



Q1: Why does openIMIS expose FHIR REST when it already has GraphQL? (click for answer) ✓

GraphQL serves openIMIS's own tightly-coupled frontend, where exact field selection and rapid schema evolution matter. FHIR REST serves **external, loosely-coupled** systems that already speak the HL7 standard and should not need to learn openIMIS internals. Different consumers, different API styles.

? Q2: Which openIMIS model maps to a FHIR `Coverage` ? (click for answer) ✓

`Policy` (from the [policy module](#)) — an insuree's active coverage for a period. The `Product` behind it maps to `InsurancePlan`.

? Q3: Where should business validation for a FHIR claim POST live? (click for answer) ✓

In the **claim service layer**, not the FHIR serializer. FHIR converters should only translate shape; business rules belong in the shared service so GraphQL and FHIR never diverge.

? Q4: How do external machine clients authenticate to the FHIR API? (click for answer) ✓

They obtain an **OAuth2 / OpenID Connect** bearer token (or a JWT) and send it on each REST call — they cannot use the browser cookie flow. The same integer rights then authorize the request.

Further reading

- [Integrations](#) — the full external-systems landscape (FHIR, DHIS2, payments, SMS/email, webhooks).
- [Security](#) — JWT, OAuth2/OIDC, and rights enforcement.
- [HL7 FHIR R4 specification](#) — the standard itself.
- `openimis-be-api_fhir_r4_py` on GitHub — the authoritative converters and serializers.

Part 10 — The Database

openIMIS did not start life as a Django application. It started as **IMIS**, a monolithic Microsoft .NET application backed by **Microsoft SQL Server**, complete with stored procedures and a schema shaped by fifteen years of production use in national health-insurance schemes. When the platform was re-architected into modular Django/React openIMIS around 2018–2019, the team made a deliberate, consequential decision: **keep the data, port the schema**. They did not throw away the model and start clean.

That decision is the single most important thing to understand about the openIMIS database. Almost every "why is it like *this*?" question you will ask has the same answer: **because a running MSSQL IMIS database had to be migratable into it without losing a single insuree, policy, or claim**. This chapter teaches you to read the schema the way its authors think about it, so that the legacy quirks stop looking like mistakes and start looking like constraints honored.

Learning objectives

By the end of this chapter you will be able to:

- Explain the MSSQL → PostgreSQL heritage and read the `tbl`-prefixed, camelCase schema without confusion.
- Map a physical table to its Django model using `Meta.db_table` and column-level `db_column`.
- Describe the **dual-key** strategy (legacy integer PK alongside a UUID) and justify *why* both exist.
- Explain **temporal versioning / soft delete** via `validity_from` / `validity_to` / `legacy_id`, and how `HistoryModel` and `VersionedModel` implement it.
- Write ORM queries that return only *currently valid* rows, and recognize the classic bug of forgetting to.
- Use `json_ext` for country customization without schema migrations.
- Reason about indexes, migrations (per-module Django + legacy SQL), transactions, and performance.

Prerequisites

- [Backend Deep Dive](#) — how modules assemble into one Django project.

- [Core module](#) — where the base model classes live.
- [Platform Overview](#) — the .NET/MSSQL → Django/React history.
- Working knowledge of the Django ORM, model `Meta` options, and PostgreSQL. We do **not** re-explain those; we explain what openIMIS does *with* them.

1. Schema heritage: reading a ported database

The first time an experienced Django engineer opens the openIMIS schema, three things look wrong. None of them are.

What you see	What Django "normally" does	Why openIMIS does it differently
Table named <code>tblInsuree</code>	<code>insuree_insuree</code> (<code><app>_<model></code>)	The physical table name is inherited from IMIS. Django is told about it via <code>Meta.db_table</code> .
Column <code>LastName</code> , <code>OtherNames</code> , <code>CHFID</code>	<code>last_name</code> , <code>other_names</code> , <code>snake_case</code>	Columns keep the original MSSQL camelCase names; the model maps them with <code>db_column</code> .
Integer PK <code>InsureeID</code> and a <code>UUID</code> column	one PK, usually <code>id</code>	Dual keys. The legacy integer stayed as the physical PK; a UUID was added for the new API surface. See §3 .

The Django model is essentially a **translation layer** over the legacy physical schema. The Python attribute names are modern and snake_case; the `db_column` / `db_table` metadata pins each attribute back to the exact legacy identifier so the ORM emits SQL the old database recognizes.

Illustrative — a model mapped onto a legacy table

This is a *simplified* sketch of the pattern you will find in `openimis-be-insuree.py` (`insuree/models.py`). It is illustrative, not a verbatim copy — read the real source for the authoritative field list.

```
# illustrative
class Insuree(core_models.VersionedModel):
    id = models.AutoField(db_column="InsureeID", primary_key=True)
    uuid = models.CharField(db_column="InsureeUUID", max_length=36,
                           default=uuid.uuid4, unique=True)
    chf_id = models.CharField(db_column="CHFID", max_length=12, null=True)
    last_name = models.CharField(db_column="LastName", max_length=100)
    other_names = models.CharField(db_column="OtherNames", max_length=100)
    family = models.ForeignKey("insuree.Family", db_column="FamilyID",
                              on_delete=models.deletion.DO_NOTHING,
                              related_name="members", null=True)
    json_ext = models.JSONField(db_column="JsonExt", blank=True, null=True)

    class Meta:
        managed = True
        db_table = "tblInsuree"
```

Notice the shape of the decisions:

- `db_table = "tblInsuree"` — physical name, not `insuree_insuree`.
- Every field carries an explicit `db_column` in legacy casing.
- The primary key is a legacy **integer** `AutoField` on `InsureeID`.
- There is *also* a `uuid` column.
- `on_delete=DO_NOTHING` shows up a lot — because openIMIS **soft-deletes** (closes validity windows) rather than issuing SQL `DELETE`. More on that in §4.

Did you know?

Because column names are pinned with `db_column`, you can rename a *Python attribute* for readability in a new module version and the underlying database is untouched — no migration, no risk to a live IMIS dataset. The legacy name is the contract; the Python name is a convenience.

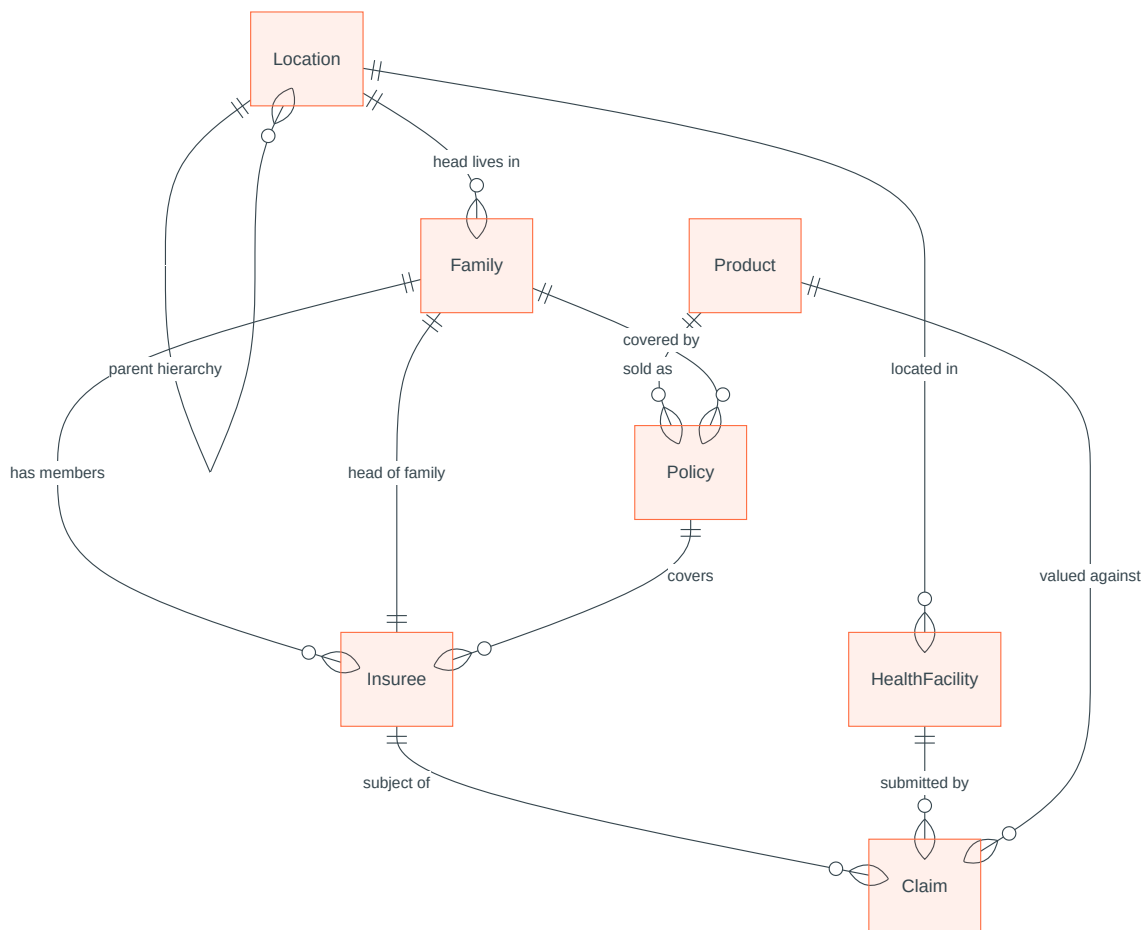
Naming conventions cheat-sheet

Convention	Example	Notes
Legacy tables keep <code>tbl</code> prefix	<code>tblInsuree</code> , <code>tblFamilies</code> , <code>tblPolicy</code> , <code>tblClaim</code>	Set via <code>Meta.db_table</code> .
Legacy columns are PascalCase/camelCase	<code>InsureeID</code> , <code>CHFID</code> , <code>ValidityFrom</code>	Set via <code>db_column</code> .
New modules (post-migration) often use plain Django names	<code>individual_ind</code> <code>ividual</code> , <code>tasks_manageme</code> <code>nt_task</code>	Modules with no IMIS ancestor (e.g. <code>individual</code> , <code>social_protection</code> , <code>tasks_management</code>) frequently use ordinary <code><app>_<model></code> tables and <code>snake_case</code> — they never had to migrate legacy data.
Foreign key columns end in <code>ID</code>	<code>FamilyID</code> , <code>ProdID</code> , <code>HFID</code>	Mapped with <code>db_column</code> ; the Python field is a normal <code>ForeignKey</code> .

This split matters: **not every table is legacy**. The `tbl` -prefixed, camelCase world is the IMIS core (insuree, family, policy, product, claim, location, medical...). The newer modules are "Django-native." When you can't guess a table name, open the module's `models.py` and read its `Meta` .

2. Relationships: the domain graph

Before the keys and versioning, fix the domain shape in your head. The core insurance model is a small, stable graph:



Read it in plain domain language:

- **Location** is a self-referencing hierarchy: region → district → municipality → village. Health facilities and families hang off a location.
- **Family** is the household unit; **Insuree** is a person, always belonging to a family (one member is the head).
- **Product** is an insurance product (benefit package + price list + rules). A **Policy** is a concrete instance of a product sold to a family for a period.
- A **Claim** is a service-delivery event: a health facility claims for care given to an insuree, valued against the product's price lists and [calculation rules](#).

Every one of those boxes is a *versioned* table. Which brings us to keys.

i Did you know?

Location modeling one physical table with a self-join is why a single misconfigured **parent** pointer can make an entire district "disappear" from reports — the row is valid, but it's orphaned in the hierarchy. Location integrity is a common source of support tickets.

3. The dual-key UUID strategy

Almost every core table has **two** identities:

1. A **legacy integer primary key** — e.g. `InsureeID`, the physical PK, unchanged from IMIS.
2. A **UUID column** — e.g. `InsureeUUID`, added during modularization.

Your instinct is to ask: pick one. Why carry both?

- **Existing data.** Millions of rows across a national deployment already have integer PKs, and thousands of foreign keys point at them. Rewriting every FK to a UUID during migration is a massive, error-prone, downtime-heavy operation. Keeping the integer PK makes migration a *copy*, not a *rewrite*.
- **Legacy SQL & stored procedures.** IMIS shipped stored procedures and reports that join on integer keys. Some of that SQL survives (see §6). It expects integers.
- **Index size & join cost.** A 4-byte integer key indexes tighter and joins cheaper than a 36-char UUID string. On the hot claim/policy joins this is not nothing.
- **Stable public identifier.** The GraphQL API exposes UUIDs, never raw integer PKs. A UUID is safe to put in URLs and API payloads: it is not guessable or enumerable, so you don't leak "how many insurees exist" or invite `id + 1` probing.
- **Distributed creation.** UUIDs can be generated client-side or offline (field enrollment on a laptop with no connectivity) and stay unique when the data syncs back. Integer autonumber can't do that.
- **Decoupling API from storage.** The API contract is the UUID; the storage PK is an implementation detail free to stay legacy.

Why keep the integer PK

Why add the UUID

So the rule of thumb is:

Integer inside, UUID outside

The **integer PK** is the **internal, physical identity** used for joins and legacy SQL. The **UUID** is the **external, logical identity** used by GraphQL, the frontend, and integrations. Resolvers look objects up by UUID; the ORM joins them by integer.



Common mistake

Do not expose or accept the integer PK in a new API. If a mutation takes an `id`, it should be the **UUID**, resolved to the row server-side. Leaking integer PKs re-introduces enumeration attacks that the UUID column was specifically added to prevent. See [Security](#).

4. Temporal versioning and soft delete

This is the concept that most surprises engineers new to openIMIS, and the one that causes the most bugs if misunderstood. **openIMIS core tables are not updated in place, and rows are not deleted.** Instead the database keeps a *temporal history*: every version of a record is a physical row, and time-validity columns say which version was "current" when.

The three columns

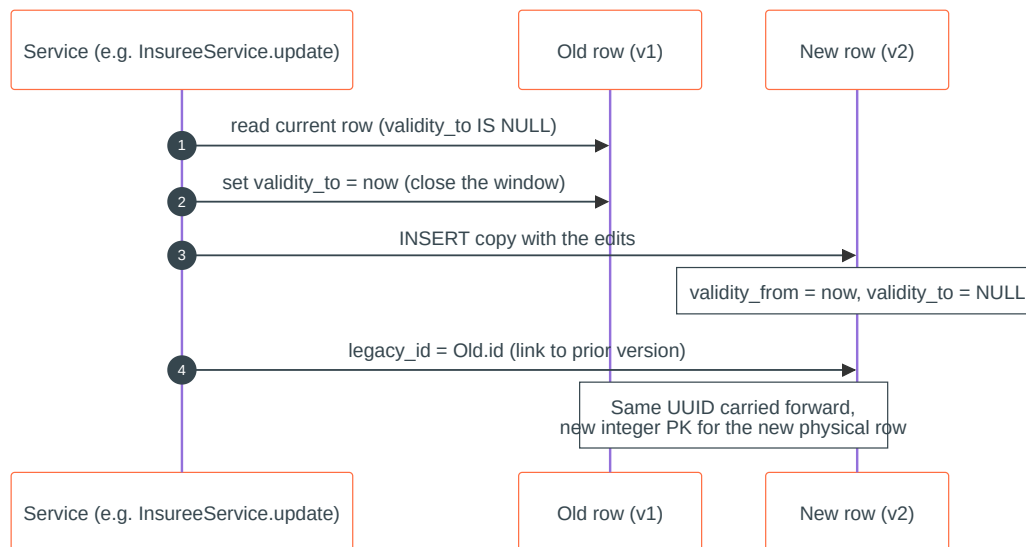
Column	Type	Meaning
<code>validity_from</code> (<code>ValidityFrom</code>)	datetime	When this version became effective.
<code>validity_to</code> (<code>ValidityTo</code>)	datetime, nullable	When this version was superseded. NULL ⇒ this is the currently valid row.
<code>legacy_id</code> (<code>LegacyID</code>)	int, nullable	Points to the integer PK of the previous version of the same logical record — the version chain.

The invariant you must internalize:

A row is "live" if and only if `validity_to IS NULL`. Everything else is history.

What an "update" actually does

When you change a versioned record, openIMIS does **not** run `UPDATE tblInsuree SET ...`. It performs a copy-on-write:



Step by step:

1. Load the current version (the one with `validity_to IS NULL`).
2. **Close** it: set its `validity_to = now`. It is now history — still in the table, no longer "live."
3. **Insert** a brand-new row with the edited values, `validity_from = now`, `validity_to = NULL`. It is the new live version.
4. Link them: the new row's `legacy_id` references the old row's integer PK, forming a backward chain.
5. The **UUID is carried forward** — the logical entity keeps its public identity across versions; only the physical integer PK differs per row.

A **delete** is the same minus the insert: just set `validity_to = now`. The row stays in the table forever (auditable), but no longer satisfies `validity_to IS NULL`, so it vanishes from normal queries. That is **soft delete**.

i Did you know?

This gives openIMIS *bitemporal-ish* auditability for free: you can reconstruct exactly what a policy or claim looked like on any past date by filtering `validity_from <= D AND (validity_to IS NULL OR validity_to > D)`. Financial and clinical systems in this domain are frequently required to prove historical state — versioning is a compliance feature, not just a convenience.

How the base classes implement it

The plumbing lives in `openimis-be-core.py` (`core/models.py`). You inherit it; you rarely write it.

Base class	Gives you	Typical use
<code>UUIDModel</code>	a UUID column + helpers	anything needing a public id
<code>VersionedModel</code>	<code>validity_from</code> , <code>validity_to</code> , <code>legacy_id</code> + <code>save_history()</code> / delete-as-close semantics	the classic legacy insurance tables (insuree, family, policy, claim...)
<code>HistoryModel</code>	versioning built on django-simple-history -style change tracking + <code>json_ext</code> + UUID, with <code>is_deleted</code> / validity handling	newer modules (individual, <code>social_protection</code> , <code>tasks_management</code> , <code>payroll</code> ...)
<code>HistoryBusinessModel</code>	<code>HistoryModel</code> plus business-date fields (<code>date_valid_from</code> / <code>date_valid_to</code>) distinct from <i>technical</i> validity	records where the <i>business</i> effective period differs from the <i>row</i> lifetime (e.g. a contract effective next month, created today)

Two subtle but important distinctions:

- **VersionedModel vs HistoryModel.** `VersionedModel` is the *legacy* mechanism — the `validity_from/to` + `legacy_id` chain described above, matching how IMIS worked. `HistoryModel` is the *modern* mechanism used by post-migration modules; it achieves the same "never lose the past" goal but with a cleaner, Django-native implementation. When you read core, expect to see both. Use whichever base class the module you're extending already uses.
- **Technical validity vs business validity.** `validity_from/to` describe the *row's* lifetime (when this version existed in the database). `HistoryBusinessModel`'s date fields describe the *real-world* period the record is about. A policy can be *created* today (technical) but *effective* next month (business). Don't conflate them.

Deep dive: querying only currently-valid rows



Because history rows live in the same table, **every normal query must exclude them**. The canonical filter is `validity_to__isnull=True`.

```
# illustrative – the everyday "give me live insurees" query
live = Insuree.objects.filter(validity_to__isnull=True)

# a single logical entity, current version, by its stable UUID:
insuree = Insuree.objects.get(uuid=some_uuid, validity_to__isnull=True)

# point-in-time: what did it look like on 2025-01-01?
from django.db.models import Q
as_of = datetime(2025, 1, 1)
snapshot = Insuree.objects.filter(
    uuid=some_uuid,
    validity_from__lte=as_of,
).filter(Q(validity_to__isnull=True) | Q(validity_to__gt=as_of))
```

In practice you rarely type `validity_to__isnull=True` by hand, because openIMIS provides help:

- Many models expose a **custom manager / queryset** whose default `.filter_validity()` (or an `active / current` manager) already applies the live filter. Check the model's manager before assuming you must filter manually.
- The Graphene filter connections in core (`OrderedDjangoFilterConnectionField` and friends) apply validity filtering as part of resolving list queries, so GraphQL clients get live rows by default.

The safe habit: **assume a query returns history unless something guarantees otherwise**, and know exactly which mechanism is doing the filtering for the query you're writing.

Common mistake — forgetting the validity filter

The number-one openIMIS data bug:

```
# WRONG: returns every historical version too – duplicates, stale data,
# inflated counts, wrong totals in reports.
Insuree.objects.filter(family=fam)

# RIGHT: only the live version of each insuree.
Insuree.objects.filter(family=fam, validity_to__isnull=True)
```

Symptoms of the missing filter: an insuree who "appears three times," a claim total that's double the real figure, a policy that "won't delete" (you closed one version but queried all of them), or a `get()` that raises `MultipleObjectsReturned`. If any of those appear, **your first suspicion should be a missing `validity_to__isnull=True`**. When you write raw SQL or a legacy report, you must add `WHERE ValidityTo IS NULL` yourself — there is no ORM to do it for you.

5. `json_ext` : extensibility without migrations

openIMIS is deployed in many countries, each with its own regulatory data needs: one country needs a national ID field on insurees, another needs a caste/ethnicity field for equity reporting, a third needs a tribal-authority code. If every country's fields became real columns, the schema would fork per country and core migrations would become impossible.

The solution is a single JSON column, `json_ext` (`JsonExt`), present on most core and modern models:

```
{
  "national_id": "NP-4471-2201",
  "enrollment_channel": "mobile-agent",
  "consent": { "given": true, "date": "2025-06-14" }
}
```

- It is a PostgreSQL `jsonb`-backed `JSONField`. You can add per-deployment fields **without a migration**.
- Country modules and [calculation rules](#) read and write it.
- It can be queried with Django's JSON lookups (`json_ext__national_id="..."`), and — because it's `jsonb` — indexed with a GIN index when a field is queried hot.

Config over forking

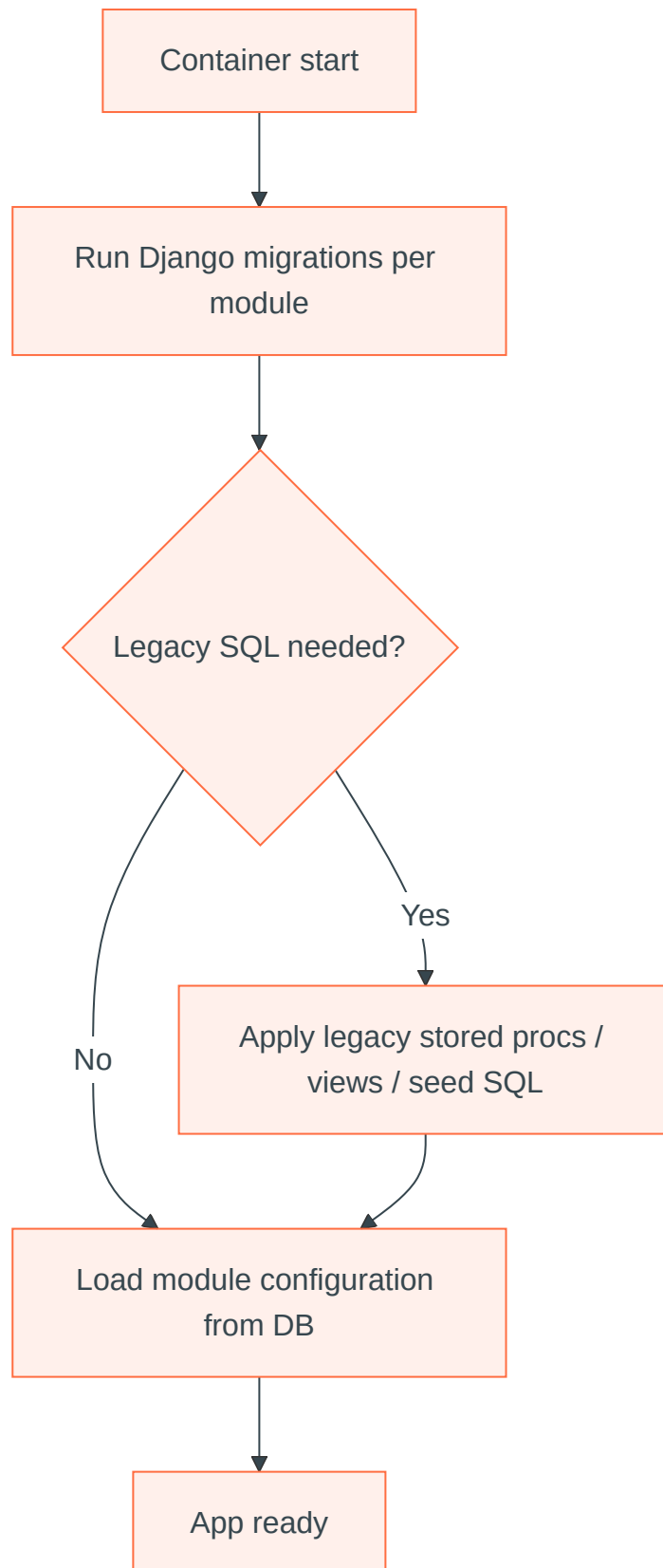
`json_ext` is the data-layer expression of openIMIS's central philosophy, the same one behind [module configuration](#): **customize by configuration and extension, never by forking core**. A new required field for one country belongs in `json_ext`, not in a patched `tblInsuree`.

Common mistake

Don't put **relational, heavily-queried, integrity-critical** data in `json_ext` just to skip a migration. JSON has no foreign keys, no `NOT NULL`, weaker indexing, and no referential integrity. If a field is core to the domain, joined on constantly, or must be consistent, it deserves a real column and a proper migration. `json_ext` is for the *long tail* of per-deployment extras.

6. Migrations: Django plus legacy SQL

openIMIS has **two** migration mechanisms, and you need both in your mental model.



1. **Per-module Django migrations.** Every module owns its `migrations/` package. Because the deployable project (`openimis-be_py`) assembles modules dynamically from

`openimis.json`, `manage.py migrate` walks *all installed modules* and applies each one's migrations in dependency order — `core` first, then `location`, `medical`, `product`, `insuree`, `policy`, `claim`, and so on. The [Docker startup](#) runs this automatically before the app serves traffic.

2. **Legacy SQL.** Some behavior — stored procedures, views, functions inherited from IMIS, or the initial schema for legacy tables — is applied as **raw SQL**, either through `migrations.RunSQL` operations inside Django migrations or through SQL scripts run at container init. This is why you'll occasionally see stored procedures in the running database that no Python code seems to create.



Migrations with versioned/legacy tables are delicate

When you write a migration that touches a `tbl_`-prefixed table, remember:

- The **table name is `db_table`**, not `<app>_<model>` — Django's autodetector handles this if the model's `Meta` is correct, but hand-written `RunSQL` must use the legacy name.
- You almost never `DELETE`; you close validity windows. A data migration that "removes" rows should set `validity_to`, not issue `DELETE`.
- Never rewrite integer PKs of live legacy tables in a migration. FKs and legacy SQL depend on them.

7. Indexes, performance, and transactions

Indexes

Index target	Why
<code>uuid</code> columns (unique)	Every API lookup is by UUID — this must be indexed and unique.
<code>validity_to</code>	Nearly every query filters <code>validity_to IS NULL</code> ; a partial or plain index here helps enormously on large tables.
Foreign key columns (<code>FamilyID</code> , <code>ProdID</code> , <code>HFID</code> , <code>PolicyID</code>)	The insurance domain is join-heavy; FK indexes keep claim/policy joins fast.
Natural business keys (<code>CHFID</code> , product code, HF code)	Frequently searched by operators; often unique-per-live-row.
GIN on hot <code>json_ext</code> fields	When a country queries a <code>json_ext</code> field at scale, index just that path.

Did you know?

A **partial index** `CREATE INDEX ... ON tblInsuree (FamilyID) WHERE ValidityTo IS NULL`; indexes only live rows. On a table where 80% of rows are closed history, this can be dramatically smaller and faster than a full index — a natural fit for temporal tables.

Performance considerations

- **History bloat.** Versioned tables grow forever. A frequently-edited entity accumulates many closed rows. Plan for it: partial indexes on live rows, periodic archival of ancient history to cold storage, and vacuum/autovacuum tuning so PostgreSQL reclaims space from closed-then-superseded churn.
- **N+1 across versions.** Because a naive query can return multiple versions per entity, an unfiltered list can be far bigger than the logical row count — pathological in reports. Filter validity *first*, then paginate.
- **Read the manager, not just the model.** Whether a query is fast often depends on whether it went through a validity-aware manager. A raw `.objects.all()` on a big versioned table is almost always a mistake.

Transactions

- **Copy-on-write must be atomic.** Closing the old version and inserting the new one is *one* logical change; if the process dies between them you get either two live rows or zero. Versioned updates run inside `transaction.atomic()` (in core services and the `OpenIMISMutation` machinery) so the close+insert commit or roll back together.
 - **Mutations are audited transactions.** The core [asynchronous mutation pattern](#) writes a `MutationLog`, performs work in a service inside a transaction, and records success/failure. A rolled-back mutation leaves a failed `MutationLog`, not a half-written entity.
 - **Isolation.** Standard PostgreSQL `READ COMMITTED` applies; the temporal design means most "conflicts" are avoided structurally (you insert a new version rather than fighting over an in-place update), but you still wrap multi-row invariants in a transaction.
-

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be-core_py</code>	<code>core/models.py</code>	Home of <code>UUIDModel</code> , <code>VersionedModel</code> , <code>HistoryModel</code> , <code>HistoryBusinessModel</code> , and the validity/ <code>json_ext</code> plumbing.
<code>openimis-be-core_py</code>	<code>core/apps.py</code> (<code>CoreConfig</code>)	<code>ModuleConfiguration</code> storage; the <code>DEFAULT_CFG</code> overlay pattern.
<code>openimis-be-insuree_py</code>	<code>insuree/models.py</code>	Canonical example of <code>tblInsuree</code> / <code>tblFamilies</code> mapped via <code>Meta.db_table</code> + <code>db_column</code> , dual keys, versioning.
<code>openimis-be-policy_py</code>	<code>policy/models.py</code>	<code>tblPolicy</code> versioned table linking families to products.
<code>openimis-be-claim_py</code>	<code>claim/models.py</code>	<code>tblClaim</code> — the join of insuree, health facility, and product; heavy versioning + valuation.
<code>openimis-be-location_py</code>	<code>location/models.py</code>	Self-referencing <code>Location</code> hierarchy and <code>HealthFacility</code> .
<code>openimis-be_py</code>	<code>openimis/settings.py</code> , <code>openimis.json</code>	How installed modules — and therefore which migrations run — are assembled.
<code>openimis-be_py</code>	<code>script/entrypoints</code>	Where migrations and legacy SQL/config loading run at container start.

Hands-on lab

Lab — see versioning with your own eyes

Prerequisite: a running dev stack ([Set Up a Dev Environment / Docker](#)) with a demo dataset loaded.

1. **Open a Django shell** against the running backend:

```
docker compose exec backend python manage.py shell
```

2. **Find a live insuree and inspect the keys:**

```
from insuree.models import Insuree
i = Insuree.objects.filter(visibility_to__isnull=True).first()
print(i.id, i.uuid, i.visibility_from, i.visibility_to) # note: visibility_to is None
```

3. **Count live vs. total** for that person's UUID:

```
Insuree.objects.filter(uuid=i.uuid).count() # all versions
Insuree.objects.filter(uuid=i.uuid, visibility_to__isnull=True).count() # should be 1
```

4. **Trigger an update** through the UI or a GraphQL mutation (edit the insuree's other names), then re-run step 3. You should now see the total climb while the live count stays **1**. Inspect the newly-closed row: its `visibility_to` is set, and the new row's `legacy_id` points back to it.
5. **Try the classic bug:** run `Insuree.objects.filter(uuid=i.uuid)` and observe the duplicate. Convince yourself why a report that forgot `visibility_to__isnull=True` would double-count.
6. **Peek at the physical schema:**

```
docker compose exec db psql -U <user> -d <db> -c "\d \"tblInsuree\""
```

Confirm the legacy table name, camelCase columns, integer PK, UUID column, and visibility columns are exactly as described.

Exercises

1. Write a queryset that returns, for one family, only the **currently-valid head of family**. Prove it survives an update to the head.

2. Given a UUID, reconstruct what the insuree looked like on `2024-01-01` using only `validity_from` / `validity_to`.
3. Design where a new country-specific field ("household electrification status") should live — real column or `json_ext`? Justify using the criteria in §5.
4. Explain, to a teammate who knows Django but not openIMIS, why `on_delete=DO_NOTHING` is common here and why that is *not* dangerous given soft delete.

Knowledge check

? **Q1: Why does a core openIMIS table carry both an integer PK and a UUID? (click for answer)** ✓

The **integer PK is legacy and internal** — it preserves IMIS data and lets existing foreign keys and legacy SQL keep working, and it's cheaper to index/join. The **UUID is new and external** — it's the stable, non-enumerable public identifier exposed by GraphQL and used across distributed/offline enrollment. Integer inside, UUID outside.

? **Q2: What single WHERE condition distinguishes a live row from history, and why is it so bug-prone? (click for answer)** ✓

`validity_to IS NULL` (`validity_to__isnull=True`). It's bug-prone because history rows live in the *same table*; omit the filter and every query silently includes superseded versions, causing duplicates, inflated totals, and `MultipleObjectsReturned`.

? **Q3: Describe what physically happens in the database when you 'update' a versioned insuree. (click for answer)** ✓

Copy-on-write inside a transaction: the current row's `validity_to` is set to now (closing it), a new row is inserted with the edits, `validity_from = now`, `validity_to = NULL`, and `legacy_id` pointing at the old row's integer PK. The UUID carries forward; the integer PK is new. Nothing is UPDATED in place and nothing is DELETED.

? **Q4: When should country-specific data go in `json_ext` versus a real column? (click for answer)** ✓

`json_ext` for the long tail of per-deployment extras that aren't heavily joined or integrity-critical — it avoids per-country schema forks and needs no migration. A real column when the field is core, joined on frequently, must be `NOT NULL` /foreign-keyed, or needs strong indexing/referential integrity.

? Q5: Why is `Meta.db_table = \"tblInsuree\"` present, and what does it tell you about the table's origin? (click for answer) ✓

It pins the Django model to the legacy physical table name inherited from MSSQL IMIS instead of Django's default `insuree_insuree`. Its presence (plus camelCase `db_columns` and a `tbl` prefix) marks the table as **migrated legacy data**, versus newer modules that use Django-native table names because they had no IMIS ancestor.

Further reading

- [Backend Deep Dive](#) — how modules and their migrations assemble.
- [Core module](#) — the base model classes in depth.
- [Configuration System](#) — the "config over forking" philosophy `json_ext` expresses at the data layer.
- [Docker Architecture](#) — where migrations and legacy SQL run at container start.
- [Troubleshooting](#) — symptoms of the missing-validity-filter bug and how to diagnose them.
- openIMIS repositories: github.com/openimis — read `core/models.py` and `insuree/models.py` for the authoritative field definitions.
- PostgreSQL docs on `jsonb`, GIN indexes, and partial indexes — the mechanisms behind `json_ext` and live-row indexing.

Authentication & Authorization

Part 9 of the openIMIS Developer Academy. This chapter explains **who a user is** in openIMIS, **how they prove it** (JWT, sessions, OpenID Connect), and **what they are allowed to do** (integer rights, roles, and `has_perms` checks in resolvers). openIMIS handles sensitive health and financial data for vulnerable populations, so authentication and authorization are not an afterthought — they are woven through the `core` module and every mutation.

Learning objectives

By the end of this chapter you will be able to:

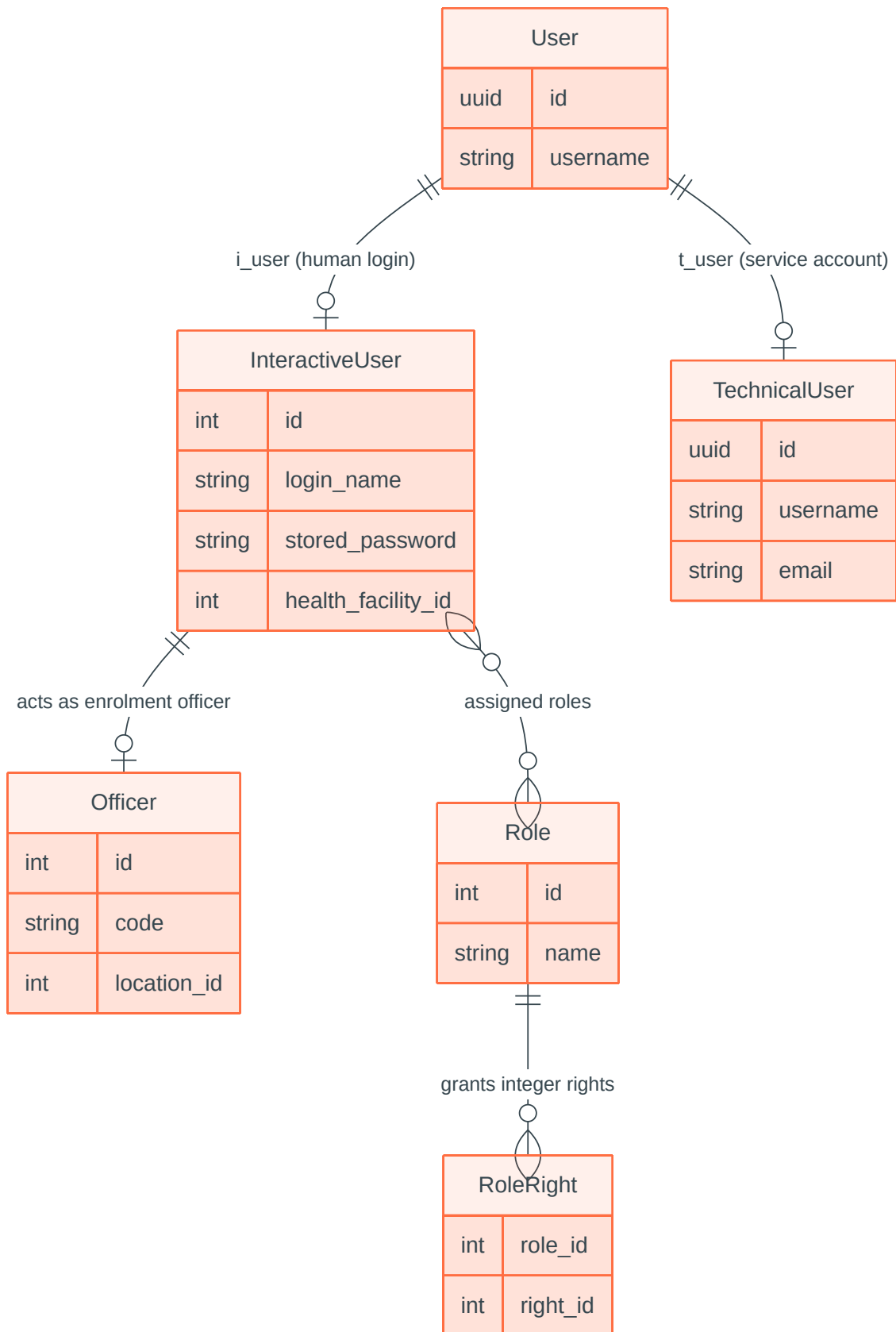
- Explain the openIMIS `User` model and how it links `InteractiveUser`, `TechnicalUser`, and `Officer / i_user`.
- Describe **roles** and the **integer rights** permission system, and map permission ranges to modules.
- Explain how **JWT authentication** works with `django-graphql-jwt`, including the `HttpOnly-cookie` storage and the obtain/refresh/verify/revoke lifecycle.
- Contrast **session vs token** authentication and know which openIMIS uses where.
- Describe the **OpenID Connect / OAuth2** option for external identity providers.
- Enforce authorization correctly with `has_perms` in resolvers/mutations and understand how the frontend mirrors it with **route guards**.
- Reason about the platform's **security hardening** posture.

Prerequisites

- [Architecture Overview](#) — the modular backend.
 - [Request Lifecycle](#) — where auth middleware sits in the request path.
 - [GraphQL from First Principles](#) — resolvers, mutations, and where `info.context.user` comes from. Authorization is enforced *inside* resolvers, so read that chapter first.
 - [Core module](#) — the `User` model and permission config live here.
-

1. The openIMIS User model

Most Django apps have one `User` model. openIMIS has more moving parts because it inherited a data model from the legacy .NET/MSSQL **IMIS** system and must serve several distinct kinds of "user" at once. The `core User` is a thin Django-auth-compatible model that **links** to one of several profile types.



Type	Who it represents	Typical use
User (core)	The Django-side login identity	The object <code>request.user</code> / <code>info.context.user</code> points at; glue for Django auth.
InteractiveUser	A human logging into the UI	Enrolment officers, claim administrators, scheme managers. Carries roles → rights.
TechnicalUser	A service account	Machine-to-machine access (integrations, scripts, FHIR clients).
Officer	An enrolment/claim officer profile	Links a human to a location and business role in insuree/policy/claim workflows.

The core `User` holds a nullable link to an `InteractiveUser` (`i_user`) and to a `TechnicalUser` (`t_user`); exactly one is populated depending on whether a human or a machine is authenticating. When you read `info.context.user` in a resolver, you get the core `User`, and you reach the human profile via its `i_user`.

Did you know?

The split exists partly for **legacy-database** reasons: `InteractiveUser` and `Officer` map onto original IMIS tables (note the `tbl` prefixes and camelCase columns you'll see in the DB), while the core `User` is the modern Django-native layer added during the 2018–2019 re-architecture. openIMIS keeps both so it can talk to legacy data *and* modern Django auth.

Deep dive: why not just use Django's `auth.User` ?

Django's default `User` assumes one flavour of account and stores password hashes in its own format. openIMIS must (a) authenticate humans whose credentials may live in the legacy IMIS schema, (b) authenticate service accounts, and (c) later federate against **external identity providers** via OIDC. A thin custom `User` that *links* to specialized profile models — rather than one fat table — lets each concern evolve independently. This is a common enterprise pattern: the authentication identity and the domain-specific profile are separate objects.

2. Roles and integer rights

Authorization in openIMIS is **not** string permission codenames like Django's default `"app.add_model"`. It is **integer rights**, inherited from the IMIS data model.

- A **Right** is an integer, e.g. `101101`. Each represents one capability ("search insurees", "create claim", "approve policy").
- A **Role** is a named bundle of rights (e.g. *Enrolment Officer*, *Claim Administrator*, *Scheme Administrator*).
- An `InteractiveUser` is assigned one or more roles; their effective rights are the **union** of all their roles' rights.
- Each backend **module declares its own permission codes** as integers in its `apps.py` `AppConfig`, exposed through `DEFAULT_CFG` so operators can retune them per deployment (see the [Plugin / Module System](#)).

```
# illustrative – permission codes declared in a module's apps.py
# (see openimis-be-insuree_py/insuree/apps.py, claim/apps.py, etc.)
class InsureeConfig(AppConfig):
    gql_query_insurees_perms = [101101]
    gql_mutation_create_insurees_perms = [101102]
    gql_mutation_update_insurees_perms = [101103]
    gql_mutation_delete_insurees_perms = [101104]
```

Permission ranges by module

The integer codes are **loosely grouped by module** using number ranges, which makes them readable once you know the convention. The exact numbers are defined in each module's `apps.py` and can be retuned per deployment — treat the table as an orienting map, not a spec, and confirm against the source.

Example range	Module	Kind of capability
101xxx	insuree / individual	Search, create, update, delete beneficiaries and families
102xxx	policy	View, create, renew, approve policies (coverage)
103xxx	contribution	Record and manage premium contributions
111xxx	claim	Submit, review, feedback, approve, value claims
121xxx	location	Manage regions, districts, health facilities
131xxx	medical / product	Manage medical items/services, insurance products, price lists
151xxx	payment / invoice	Payments, invoices, provider payment
152xxx	core / admin	Users, roles, rights, module configuration

Did you know?

Because rights are just integers checked with `user.has_perms([codes])`, and because each module owns its codes in `apps.py`, a country deployment can enable a module and its permissions become assignable to roles **without any change to core**. Authorization scales with the plugin system.

3. How JWT authentication works

openMIS authenticates GraphQL clients with **JSON Web Tokens (JWT)** via the `django-graphql-jwt` library. If you have only done session auth, here is the model.

A **JWT** is a signed, self-contained token with three dot-separated parts — `header.payload.signature`. The payload carries claims (who the user is, when the token expires); the signature (HMAC with the server secret, or RSA) lets the server verify the token **without a database lookup**. It is *stateless* auth: the token itself is the credential.

Storage: the HttpOnly cookie

openIMIS stores the JWT in an **HttpOnly cookie**, not in `localStorage`. This is a deliberate security choice:

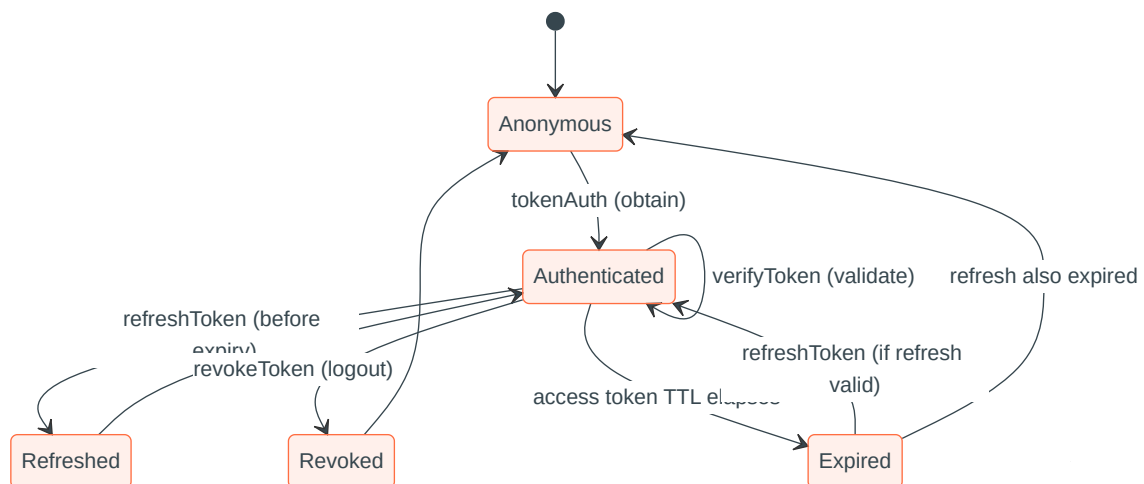
- **HttpOnly** means JavaScript **cannot read** the cookie — this is the single most important defence against token theft via XSS.
- The browser attaches the cookie automatically to every `/graphql` request, so the frontend never has to manually manage the token.
- Combined with `Secure` (HTTPS-only) and `SameSite` attributes, the cookie is hardened against interception and CSRF.

⚡ Common mistake: putting the JWT in localStorage

A very common (insecure) tutorial pattern is to store the token in `localStorage` and attach it as an `Authorization: Bearer` header from JS. That exposes the token to **any** XSS on the page. openIMIS uses an **HttpOnly cookie** precisely so JavaScript cannot touch the token. Do not "simplify" this by moving the token into JS-readable storage.

The token lifecycle

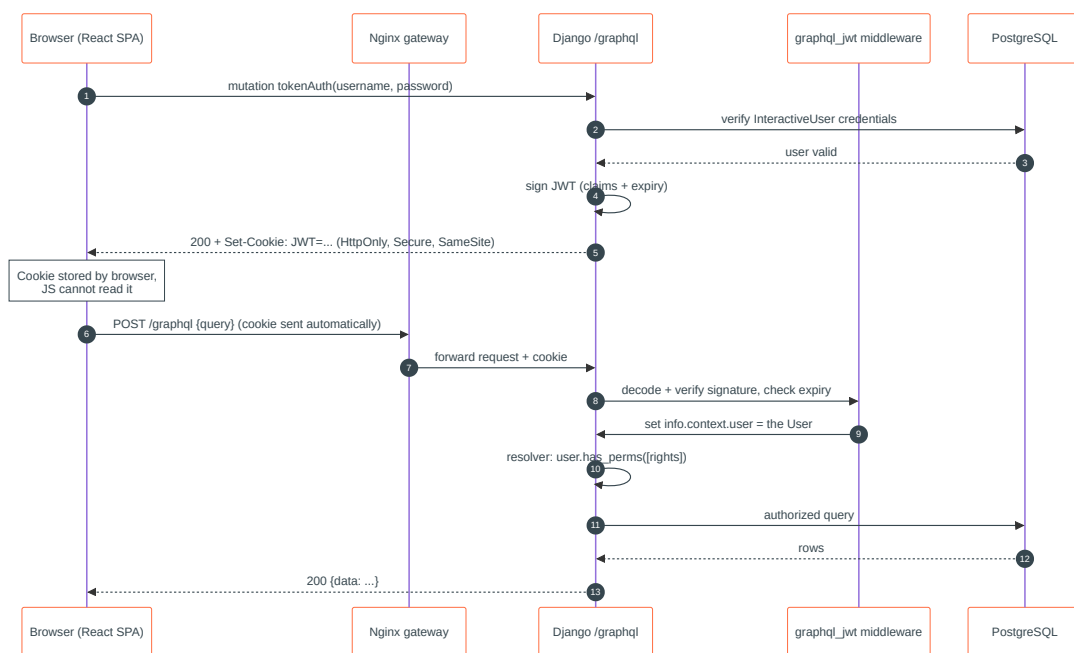
`django-graphql-jwt` exposes the lifecycle as **GraphQL mutations**, not REST endpoints. There are four operations:



Operation	Mutation	Purpose
Obtain	<code>tokenAuth(username, password)</code>	Verify credentials, issue an access token (short-lived) and set the cookie. This is <i>login</i> .
Verify	<code>verifyToken(token)</code>	Confirm a token is still valid and un-tampered; returns its claims.
Refresh	<code>refreshToken</code>	Exchange a still-valid (or long-lived refresh) token for a fresh access token, so the user isn't logged out mid-session.
Revoke	<code>revokeToken</code>	Invalidate a refresh token — <i>logout</i> . Clears the cookie server-side.

Access tokens are intentionally **short-lived** (minutes) to limit the blast radius of a leaked token; the longer-lived **refresh** token lets the client silently renew. The frontend refreshes proactively before expiry.

Login → authenticated call, end to end



The crucial handoff: the `graphql_jwt` middleware decodes the cookie's token on every request and populates `info.context.user`. From the resolver's point of view, `info.context.user` is just an authenticated Django user — the same object you'd get from

session auth. Authentication (proving identity) and authorization (checking rights) are cleanly separated: the middleware does the former, your resolver does the latter.

4. OpenID Connect / OAuth2 for external identity providers

Not every deployment wants openIMIS to be the password authority. A ministry of health may already run a central **identity provider** (Keycloak, Azure AD, an OIDC-compliant IdP). openIMIS supports **OpenID Connect (OIDC) / OAuth2** so that authentication can be delegated.

- **OAuth2** is an *authorization* framework: it issues access tokens without sharing the user's password with openIMIS.
- **OpenID Connect** is a thin *authentication* layer on top of OAuth2: it adds an **ID token** (a JWT) that tells openIMIS *who* the user is.

In this mode the user authenticates against the external IdP, which returns signed tokens; openIMIS validates them and maps the external identity onto an openIMIS `User` (and its roles/rights). This gives central deployments **single sign-on** and centralized account lifecycle (disable a user in the IdP, they lose openIMIS access) while keeping openIMIS's *authorization* model — the integer rights — fully in charge of what the user can do.

Did you know?

OIDC changes only **authentication** (who you are). **Authorization** — the integer rights checked by `has_perms` — stays entirely inside openIMIS. Federating login does not federate permissions; a user logged in via Keycloak still needs openIMIS roles to do anything.

5. Session vs token authentication

You'll meet both in openIMIS; know when each applies.

Aspect	Session auth	Token (JWT) auth
State	Server stores the session; cookie holds an opaque id	Stateless — the token itself carries the claims
Lookup per request	DB/cache hit to load the session	Verify signature; no DB hit
Used in openIMIS for	Django admin and some server-rendered/legacy flows	The primary SPA ↔ <code>/graphql</code> API
Storage	Session cookie	JWT in an HttpOnly cookie
Scales horizontally	Needs shared session store	Naturally (no shared state)
Revocation	Delete the session	Revoke the refresh token / short TTL

The React SPA and API integrations use **JWT**; Django's built-in admin uses **sessions**. Both ultimately resolve to the same core `User` and the same rights checks.

6. Enforcing authorization

Authentication tells you *who*; authorization decides *what*. In openIMIS the enforcement point is the **resolver / mutation**, using integer rights.

Backend: `has_perms` in resolvers and mutations

```
# illustrative – the canonical guard (see any module's schema.py /  
gql_mutations/)  
from django.core.exceptions import PermissionDenied  
from django.utils.translation import gettext as _  
  
def resolve_insurees(self, info, **kwargs):  
    user = info.context.user  
    if user.is_anonymous or not  
user.has_perms(InsureeConfig.gql_query_insurees_perms):  
        raise PermissionDenied(_("unauthorized"))  
    return Insuree.objects.filter(validity_to__isnull=True)  
  
class CreateInsureeMutation(OpenIMISMutation):  
    @classmethod  
    def async_mutate(cls, user, **data):  
        if not  
user.has_perms(InsureeConfig.gql_mutation_create_insurees_perms):  
            raise PermissionDenied(_("unauthorized"))  
        InsureeService(user).create_or_update(data)
```



Common mistake: trusting the frontend to hide a button

Hiding a UI button when the user lacks a right is **UX, not security**. A determined client can send any GraphQL query to the single `/graphql` endpoint. **Every** resolver returning protected data and **every** mutation must call `has_perms` server-side. The frontend guard and the backend check are two independent layers — you need *both*, and the backend one is the real fence. Never remove a backend check because "the UI already prevents it".

Frontend: route guards using rights

The React frontend mirrors the backend rights to **guard routes and menu items** so users don't see actions they can't perform. `openimis-fe-core_js` exposes the logged-in user's rights, and modules register routes with a required right.


```
// illustrative – faithful to openimis-fe route/menu registration
// (see each openimis-fe-<name>_js module config and core's ModulesManager)
const RIGHT_INSUREE_SEARCH = 101101;

export function rightGuard(rights) {
  return rights.includes(RIGHT_INSUREE_SEARCH);
}

// A menu entry only renders if the user holds the right:
menus: [
  { text: "Insurees", icon: <PeopleIcon />, route: "/insuree",
    filter: (rights) => rights.includes(RIGHT_INSUREE_SEARCH) },
]
```

The frontend guard is for **experience** (don't show dead-end screens); the backend `has_perms` is for **security**. They use the *same* integer rights, which is what keeps them consistent.

7. Security architecture and hardening

Because openIMIS handles health and financial PII for vulnerable populations, the deployment posture matters as much as the code. Key points from the [Docker & Deployment](#) topology:

- **TLS everywhere.** The Nginx gateway terminates HTTPS; the JWT cookie is set `Secure`, so it is never sent over plaintext.
- **HttpOnly + SameSite cookies.** Mitigates XSS token theft and CSRF (see §3).
- **Secrets outside the image.** DB passwords, JWT signing keys, and IdP secrets come from `.env` / environment, not baked into containers.
- **Short access-token TTL + refresh.** Limits the window a stolen token is useful; revocation invalidates refresh tokens on logout.
- **Least privilege via rights.** Roles should grant the *minimum* rights a job needs; the integer-rights model makes this auditable.
- **Every mutation is audited.** The `OpenIMISMutation` / `MutationLog` pattern (see [GraphQL](#)) records who did what and whether it succeeded — an authorization *and* compliance asset.
- **Temporal, non-destructive data.** Versioned models (`validity_from` / `validity_to`) mean records are *closed*, not hard-deleted, preserving an audit history that is valuable for security investigations.



Common mistake: leaking data through unfiltered nested resolvers

You can guard the top-level `resolve_claims` and still leak data if a **nested** field (say, `claim.insuree`) returns objects the user shouldn't see. Authorization must consider the *whole graph* a query can traverse, not just the entry field. When exposing sensitive relations, ensure the nested resolver (or the queryset it builds on) also respects the user's rights and data scope (e.g. their health facility / location).



Deep dive: signing keys, key rotation, and token theft



A JWT's security rests on its **signing key**. If the HMAC secret (or RSA private key) leaks, an attacker can mint valid tokens for any user, and no per-request DB lookup will catch them — that is the trade-off for stateless auth. Mitigations openIMIS deployments should apply: keep the key in environment/secret storage (never in the repo or image), rotate it periodically (invalidating outstanding tokens), keep access-token TTL short so a stolen token expires quickly, and use `revokeToken` on logout to kill the refresh token. For high-assurance deployments, front openIMIS with an OIDC provider so token issuance and revocation are centralized.

Repository references

Repository	Directory / File	Why it matters
openimis-be-core_py	core/models.py	Defines <code>User</code> , <code>InteractiveUser</code> , <code>TechnicalUser</code> , <code>Officer</code> , <code>Role</code> , and rights.
openimis-be-core_py	core/apps.py	<code>CoreConfig</code> with <code>DEFAULT_CFG</code> , <code>_configure_permissions()</code> , and core rights codes.
openimis-be-core_py	core/schema.py	Where <code>info.context.user</code> is used; <code>OpenIMISMutation</code> audit via <code>MutationLog</code> .
openimis-be_py	openimis/settings.py	<code>django-graphql-jwt</code> config, cookie flags, <code>OIDC/OAuth2</code> settings, <code>AUTHENTICATION_BACKENDS</code> .
openimis-be_py	openimis/urls.py	The <code>/graphql</code> endpoint that JWT middleware protects.
<any module>	apps.py	Per-module integer permission codes (e.g. <code>gql_mutation_create_*_perms</code>).
openimis-fe-core_js	user/rights state + <code>ModulesManager</code>	Exposes the logged-in user's rights for route/menu guards.
openimis-dist_dkr	<code>docker-compose.yml</code> , <code>.env</code> , gateway config	TLS termination, secrets, cookie/ <code>Secure</code> posture.

Hands-on lab



Lab: authenticate, inspect, and probe authorization

Use a running dev stack and GraphQL at `/graphql`.

1. **Obtain a token.** Run the `tokenAuth` mutation with a seeded user's credentials. Inspect the response headers / browser cookies and confirm a JWT cookie was set with **HttpOnly** and (in a TLS deployment) **Secure**.
2. **Decode the payload.** Paste the token into a JWT decoder (or use `verifyToken`) and read the claims and expiry. Note there is no password in it — only identity claims and metadata.
3. **Make an authorized call.** Query `insurees`. It works because the cookie is sent automatically and you hold the right.
4. **Probe authorization.** Log in as a user *without* the insuree-search right (or temporarily remove it from their role) and re-run the query. Confirm you get an authorization error — proving the check is server-side, not just UI.
5. **Refresh and revoke.** Run `refreshToken` and observe a new token; then `revokeToken` and confirm subsequent calls are rejected (logout).

Exercises

1. Draw the object graph linking a logged-in human to their effective rights, starting at the core `User` and ending at `RoleRight`.
2. A colleague proposes storing the JWT in `localStorage` "so the mobile web view can read it." Give two security reasons to refuse and state what openIMIS does instead.
3. Given a new module with a `resolve_reports` resolver, write the exact `has_perms` guard, and explain why hiding the menu item in the frontend is not a substitute.

Knowledge check



Q1: What is the difference between the core `User`, `InteractiveUser`, and `TechnicalUser` ? (click for answer)



The core `User` is the Django-side login identity (`info.context.user`). It links to an `InteractiveUser` for a **human** UI login (carrying roles → rights) or a `TechnicalUser` for a **service account** (machine-to-machine). Exactly one profile is populated per user.

? Q2: How are permissions represented in openIMIS, and where are the codes defined? (click for answer) ✓

As **integer rights**. Roles bundle rights; a user's effective rights are the union of their roles'. Each module declares its own integer codes in its `apps.py AppConfig` (e.g.

```
gql_mutation_create_insurees_perms = [101102] ), and they are checked with
user.has_perms([codes]) .
```

? Q3: Why does openIMIS store the JWT in an HttpOnly cookie instead of localStorage? (click for answer) ✓

Because **HttpOnly** cookies cannot be read by JavaScript, which defeats token theft via XSS. The browser also attaches the cookie automatically, and with `Secure + SameSite` it resists interception and CSRF. `localStorage` is readable by any script on the page and is unsafe for credentials.

? Q4: What are the four JWT lifecycle operations and what does each do? (click for answer) ✓

Obtain (`tokenAuth`) — verify credentials, issue a token (login). **Verify** (`verifyToken`) — confirm a token is still valid/un-tampered. **Refresh** (`refreshToken`) — exchange a valid token for a fresh one before expiry. **Revoke** (`revokeToken`) — invalidate the refresh token (logout).

? Q5: If OIDC handles login, what still governs what a user can do, and why? (click for answer) ✓

openIMIS's **integer rights** still govern authorization. OIDC/OAuth2 changes only **authentication** (who you are); it does not grant capabilities. A user federated via an external IdP still needs openIMIS roles/rights, enforced by `has_perms` in resolvers and mutations.

Further reading

- [GraphQL from First Principles](#) — resolvers, the `OpenIMISMutation` audit trail, and where `info.context.user` is checked.
- [Request Lifecycle](#) — where auth middleware runs in the Django request path.
- [Core module](#) — the `User`, `Role`, and rights models.
- [Docker & Deployment](#) — TLS, gateway, and secret handling.
- [django-graphql-jwt](#) documentation: <https://django-graphql-jwt.domake.io/>.

- **JWT** primer and debugger: <https://jwt.io/>.
- **OpenID Connect** specification: <https://openid.net/developers/how-connect-works/>.
- **OWASP** cheat sheets on JWT and session management:
<https://cheatsheetseries.owasp.org/>.
- **openIMIS** GitHub organization: <https://github.com/openimis>.

Part 12 — Docker Architecture

openIMIS is not one program. It is a **Django backend assembled from ~47 modules**, a **React single-page application assembled from its own set of frontend modules**, a **PostgreSQL database** carrying a schema descended from legacy IMIS, a **reverse proxy** stitching them into one origin, and optionally an **OpenSearch** cluster for analytics. Getting all of that to stand up, in the right order, with the right configuration, on any engineer's laptop *and* on a national production server — that is the job of the distribution repository, `openimis-dist_dkr`, and its `docker-compose` stack.

This chapter is the map of that stack: which containers exist, how they talk, what order they must start in, and why. By the end you will bring the whole thing up yourself.

Learning objectives

- Enumerate the containers in the openIMIS Docker stack and state each one's job.
- Explain the **startup order** — why `db` must be healthy before `backend`, and why `backend` must migrate and load module config before it serves.
- Trace a browser request through the **gateway** to either the frontend or the backend (`/api`, `/graphql`).
- Understand how the **backend image is built** from `openimis-be_py` and how it installs modules from `openimis.json`.
- Read and reason about the `docker-compose` **services, volumes, networks**, and the `.env` file.
- Bring the stack up and verify it end-to-end.

Prerequisites

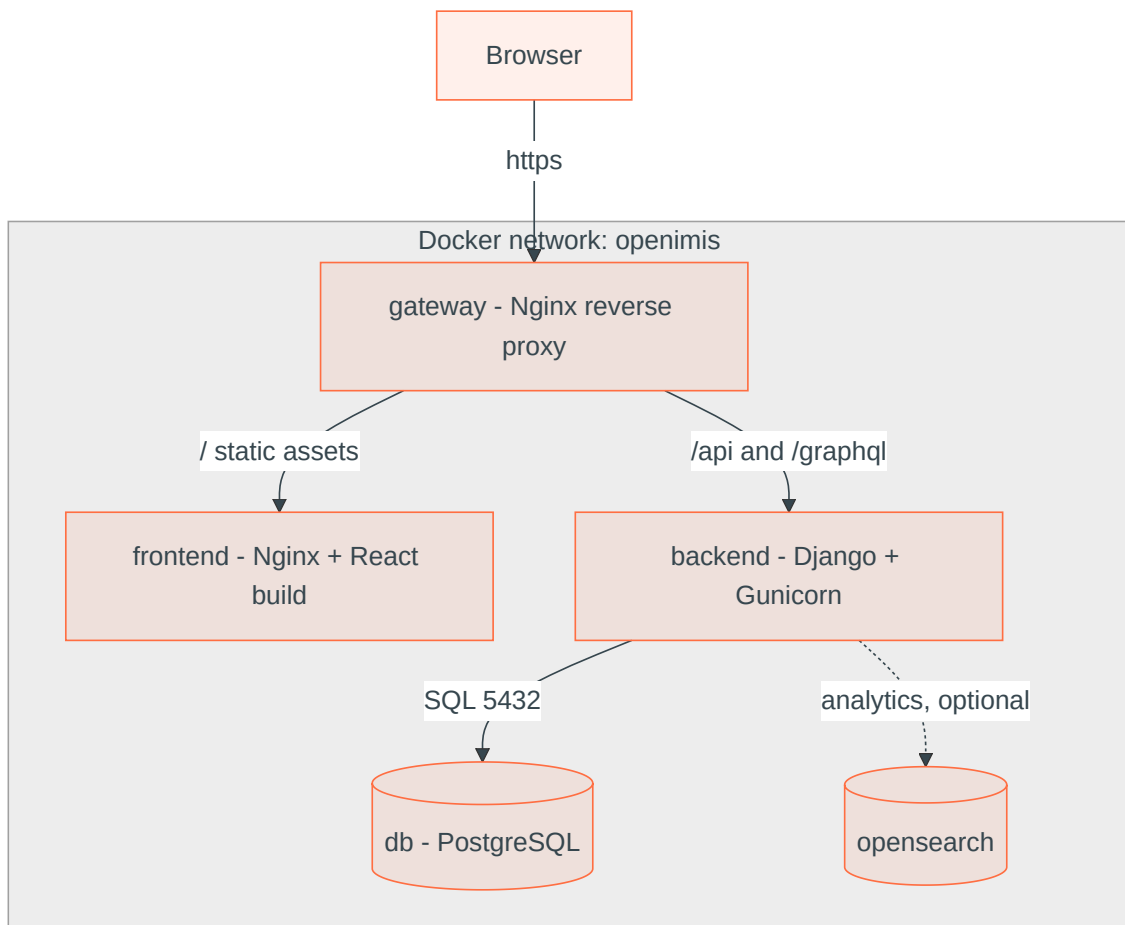
- [Backend Deep Dive](#) — how modules assemble into one Django project.
 - [The Database](#) — why migrations at startup matter and what they touch.
 - [Configuration System](#) — the layered config model the containers read.
 - Comfort with Docker, `docker compose`, images vs. containers, volumes, and Linux networking basics. We build on those, not re-teach them.
-

1. The services

The stack is a small set of cooperating containers, each a single responsibility. The exact service names and image tags evolve, so treat this as the shape — confirm specifics in the `docker-compose.yml` of `openimis-dist_dkr`.

Service	Image / base	Responsibility	Listens on
<code>db</code>	PostgreSQL	The single source of truth. Holds the legacy-derived schema, all module tables, and the DB-backed <code>ModuleConfiguration</code> .	5432 (internal)
<code>backend</code>	Built from <code>openimis-be_py</code>	Django app served by Gunicorn . Runs migrations + loads module config at start, then serves <code>/api</code> and <code>/graphql</code> .	8000 (internal)
<code>frontend</code>	Built from <code>openimis-fe_js</code> , served by Ngix	Static React SPA bundle (built with Vite). Pure static files — no application logic at runtime.	80 (internal)
<code>gateway</code>	Ngix reverse proxy	The only publicly exposed service. Routes <code>/</code> to <code>frontend</code> , <code>/api</code> + <code>/graphql</code> to <code>backend</code> , terminates the single origin.	80/443 (published)
<code>opensearch</code> (optional)	OpenSearch (+ Dashboards)	Backing store for <code>opensearch_reports</code> analytics; only present when analytics is enabled.	9200 (internal)

The mental model:



Read the crucial facts off that diagram:

- **Only the gateway is exposed.** `db`, `backend`, and `frontend` have no published ports; they're reachable only on the internal Docker network by service name. This is a security posture, not an accident — the database is never on the public internet, and the backend is never hit except through the proxy.
- **The frontend is dumb.** It's a bundle of static files. All "smarts" (auth, data, business logic) live in `backend`. The React app talks to `/graphql` on the *same origin*, so there is no CORS problem and the JWT cookie flows cleanly.
- **The backend is the only writer to the database.** Everything funnels through Django.

i Did you know?

Serving the SPA and the API from **one origin** (via the gateway) is what lets openIMIS store the auth JWT in an **HttpOnly cookie** instead of `localStorage`. Same origin $\Rightarrow$ the cookie is sent automatically with every `/graphql` request, and JavaScript can't read it — a meaningful XSS mitigation. Split origins would have forced a weaker token-in-JS design. See [Security](#).

2. Volumes and networks

Volumes

Volume	Mounted by	Purpose
<code>database</code> (named volume)	<code>db</code>	Persists PostgreSQL data across container restarts. Deleting this destroys your data.
<code>photos</code> / <code>media</code> (bind or named)	<code>backend</code>	Insuree photos and uploaded files — must survive container rebuilds.
gateway config	<code>gateway</code>	Nginx <code>.conf</code> routing rules and (in prod) TLS certificates.
<code>opensearch-data</code> (optional)	<code>opensearch</code> <code>h</code>	Analytics index persistence.

The rule: **anything stateful lives in a volume, not the container filesystem**. Containers are cattle — rebuilt freely. The database volume is the one you protect with your life (and backups — see [Deployment & Operations](#)).

Networks

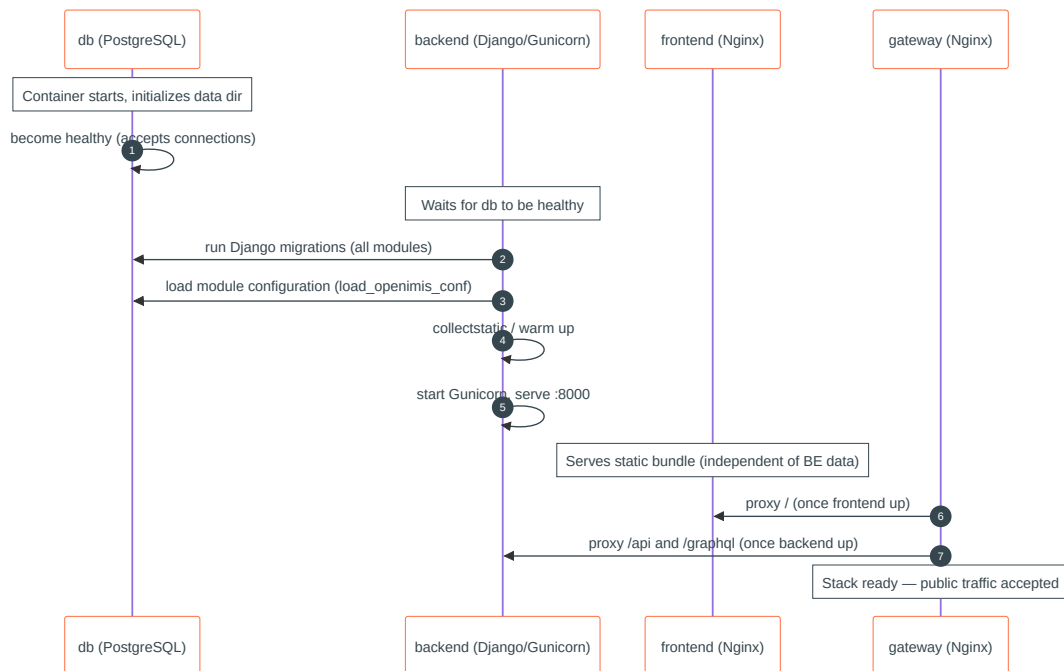
By default compose creates one bridge network for the project. Containers resolve each other by **service name** as DNS host: the backend connects to the database at host `db`, the gateway proxies to `backend:8000` and `frontend:80`. That name-based service discovery is why the `.env` database host is literally `db`, not an IP.

Two networks in hardened setups

Production configurations sometimes split into a *frontend* network (gateway ↔ frontend/backend) and a *backend* network (backend ↔ db), so the database is not even on the same network as the public-facing proxy. If you see two networks in the compose file, that's the reason.

3. Startup order — the part people get wrong

You cannot start these containers in any order. There is a strict dependency chain, and if you violate it the stack fails in confusing ways (backend crash-looping because the DB isn't ready; frontend served but every API call 502s).



Walk the chain:

1. **db first, and it must be *healthy*, not merely *started*.** PostgreSQL takes seconds to initialize its data directory on first boot. "Container running" $\neq$ "accepting connections." Compose expresses this with `depends_on` + a **healthcheck** (e.g. `pg_isready`) so the backend waits for *ready*, not just *created*.
2. **backend second, and it does real work before serving.** On start, the backend's entrypoint script (from `openimis-be_py`, under `script/`) runs:
3. `manage.py migrate` — applies every installed module's migrations in dependency order (`core` first). See [Database → Migrations](#).
4. **module config load** (`openimisconf/load_openimis_conf.py`) — seeds/updates the DB-backed `ModuleConfiguration` overlay so runtime config is present.
5. `collectstatic` / warmup, then **Gunicorn** starts and binds `:8000`. Only *after* all that does the backend answer requests. This is why the first boot is slow and why you must not front it with the gateway prematurely.
6. **frontend** can come up in parallel — it's static and doesn't need the database — but there's no point routing traffic to it before the backend is ready, because the app immediately calls `/graphql`.

7. **gateway last (logically)**. It ties the origin together. Until backend and frontend are up, its upstreams don't exist. In practice compose starts it early and Nginx tolerates upstreams appearing, but the stack is only *usable* once the backend has finished migrating and Gunicorn is live.

 Common mistake — assuming 'container up' means 'ready'

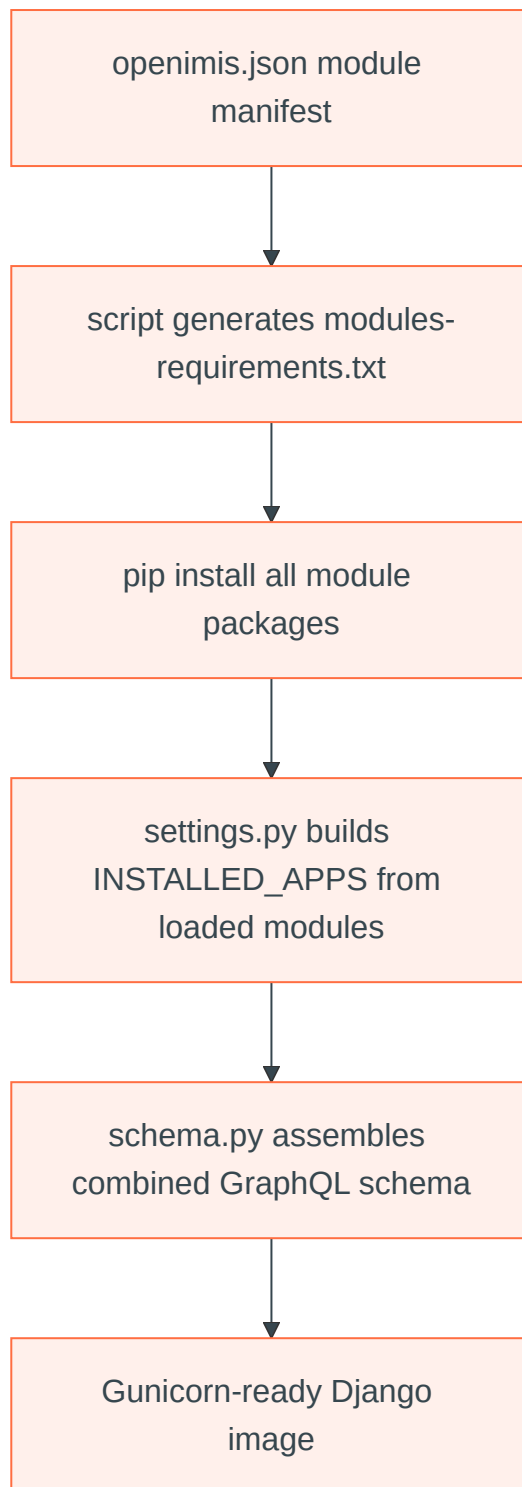
A backend that crash-loops on first boot is almost always racing the database: it tried to migrate before PostgreSQL accepted connections. The fix is a **healthcheck-gated `depends_on`** (wait for `db` healthy), not a `sleep`. Likewise, a frontend that loads but shows "network error" everywhere means the **backend isn't finished migrating** yet — wait for it, don't restart the frontend.

 Did you know?

Because migrations run automatically at backend start, **deploying a new version is often just: pull the new backend image and recreate the backend container**. It migrates itself forward. That convenience is also a hazard in production — see the zero-downtime discussion in [Deployment & Operations](#).

4. How the backend image is built

This is the piece that connects Docker to the [module architecture](#). The backend image is not a fixed application — it's **assembled at build time from a manifest**.



1. **openimis.json is the source of truth.** It lists ~47 modules, each with a pip/git source and version. A build script turns it into `modules-requirements.txt`.
2. **pip install** pulls every module into the image. For development you swap this for **editable installs** (`pip install -e ../openimis-be-core_py/`) so a bind-mounted checkout's changes are live without rebuilding.

3. At **runtime**, `openimis/settings.py` reads the loaded module list and builds `INSTALLED_APPS` dynamically; `openimis/schema.py` imports each module's `schema.Query` / `schema.Mutation` and combines them by multiple inheritance into one root schema; `openimis/urls.py` collects each module's URL patterns plus `/graphql`.

The consequence for Docker: **which modules a deployment runs is a property of the image (the manifest baked in), the migrations that run, and the config loaded** — not something you toggle at runtime. Add a module $\Rightarrow$ change `openimis.json` $\Rightarrow$ rebuild the backend image $\Rightarrow$ new migrations run on next start. This is the through-line from [Configuration](#): environment tunes behavior at runtime, but the *set of installed modules* is a build-time decision.

Dev vs. prod image

In development, the compose file typically **bind-mounts** your local backend and module checkouts into the container and uses editable installs, so you edit code on the host and Django's autoreloader (or a restart) picks it up. In production, the image is **self-contained** — code baked in, no bind mounts — so it's reproducible and immutable.

5. How the gateway routes

The gateway is a plain Nginx reverse proxy, and its config is short and worth understanding literally, because it's the whole reason the SPA and API feel like "one website."

```
# illustrative gateway routing – see openimis-dist_dkr for the real conf
location /api/ {
    proxy_pass http://backend:8000;
}
location /graphql {
    proxy_pass http://backend:8000;
}
location /site_static/ { # Django static/admin assets
    proxy_pass http://backend:8000;
}
location / { # everything else = the React SPA
    proxy_pass http://frontend:80;
}
```

Incoming path	Proxied to	Why
<code>/</code> and all app routes	<code>frontend:80</code>	The React SPA bundle. Client-side routing handles sub-paths.
<code>/graphql</code>	<code>backend:8000</code> <code>0</code>	The single GraphQL endpoint the whole app uses.
<code>/api/...</code>	<code>backend:8000</code> <code>0</code>	REST/legacy endpoints: reports, FHIR (<code>/api/api_fhir_r4/</code>), auth helpers.
<code>static/admin</code>	<code>backend:8000</code> <code>0</code>	Django-served static (admin, etc.).

Because everything is served from the **same origin** through this one proxy, the browser sees a single site. The JWT HttpOnly cookie set by the backend is scoped to that origin and rides along on every `/graphql` call automatically. In production this same layer terminates **TLS** ([Deployment & Operations](#)).



Common mistake — bypassing the gateway in dev

If you expose the backend on `localhost:8000` and the frontend on `localhost:3000` and hit them directly, you've created **two origins**. Now the auth cookie won't flow, you'll fight CORS, and "it works in prod but not locally" bugs appear. Prefer running the *gateway* locally so your dev topology matches production. When you must run split, know that you're deviating from the real routing.

6. Environment variables and `.env`

`docker-compose` reads a `.env` file for all deployment-specific values — this is layer one of the [configuration system](#). It never contains code, only values, and it is **never committed** with real secrets.

Variable (illustrative names)	Consumed by	Purpose
DB_HOST	backend	Database host — the service name <code>db</code> .
DB_NAME , DB_USER , DB_PASSWORD , DB_PORT	backend, db	Database connection + PostgreSQL init credentials.
DJANGO_SECRET_KEY	backend	Django cryptographic secret. Must be strong and secret in prod.
DEBUG	backend	<code>True</code> only in dev. Must be <code>False</code> in production.
ALLOWED_HOSTS	backend	Hostnames Django will serve.
SITE_ROOT / SITE_URL	backend, gateway	Base path/URL the app is served under.
REMOTE_USER_AUTHENTICATI ON / OIDC vars	backend	External identity provider (OpenID Connect) settings.
OPENSEARCH_*	backend, opensearch	Analytics connection, when enabled.

 Common mistake — a leaked or default `DJANGO_SECRET_KEY` / `DEBUG=True` in prod

Two of the most damaging misconfigurations: a **weak or shared `DJANGO_SECRET_KEY`** undermines session/token integrity, and **`DEBUG=True`** in production leaks stack traces, settings, and SQL to anyone who triggers an error. Both are `.env` values — get them right per environment. Deeper treatment in [Security](#) and [Deployment & Operations](#).

Repository references

Repository	Directory / File	Why it matters
<code>openimis-dist_dkr</code>	<code>docker-compose.yml</code>	The authoritative list of services, volumes, networks, and <code>depends_on</code> ordering.
<code>openimis-dist_dkr</code>	<code>.env</code> / <code>.env.example</code>	The environment variables every service reads. Start from the example.
<code>openimis-dist_dkr</code>	gateway Nginx conf	The real <code>/</code> , <code>/api</code> , <code>/graphql</code> routing and (in prod) TLS.
<code>openimis-be_py</code>	<code>Dockerfile</code> , <code>script/</code>	How the backend image is built and what the entrypoint runs (migrate + config load) at start.
<code>openimis-be_py</code>	<code>openimis.json</code>	The module manifest that determines which packages the image installs.
<code>openimis-be_py</code>	<code>openimisconf/load_o</code> <code>penimis_conf.py</code>	The module-config load step in the startup sequence.
<code>openimis-fe_js</code>	<code>Dockerfile</code> / Nginx conf	How the React bundle is built (Vite) and served statically.

Hands-on lab

Lab — bring the whole stack up

1. Clone the distribution repo:

```
git clone https://github.com/openimis/openimis-dist_dkr.git
cd openimis-dist_dkr
```

2. Create your environment file from the template and set at minimum a database password and a Django secret:

```
cp .env.example .env
# edit .env: set DB_PASSWORD, DJANGO_SECRET_KEY; keep DEBUG for dev only
```

3. Start the stack:

```
docker compose up -d
```

4. Watch the startup order happen. Follow the backend logs and observe migrations and config loading run *before* Gunicorn binds:

```
docker compose logs -f backend
```

You'll see `db` become healthy, then `Applying ... migrations`, then the module-config load, then Gunicorn workers boot. This is [§3](#) in the flesh.

5. Check every service:

```
docker compose ps
```

Confirm `db` is healthy and `backend`, `frontend`, `gateway` are up. Note that only the gateway publishes a host port.

6. Open the app at the gateway's URL (e.g. `http://localhost`). Log in with the demo credentials from the docs.

7. Prove the routing yourself:

```
curl -i http://localhost/graphql      # hits backend
curl -i http://localhost/             # hits frontend SPA
```

8. Inspect the internal network — confirm the backend reaches the DB by service name:

```
docker compose exec backend python manage.py dbshell -c "select 1;"
```

9. Tear down without losing data, then bring it back and confirm persistence:

```
docker compose down          # keeps named volumes
docker compose up -d         # data still there
```



(Only `docker compose down -v` deletes the database volume — never do that in anger.)

Exercises

1. From the real `docker-compose.yml`, list every published (host-mapped) port. Is anything besides the gateway exposed? Should it be?
2. Explain what breaks, and how it manifests, if you remove the healthcheck on `db`.
3. You add a new backend module. Enumerate every step from `openimis.json` to that module's tables existing in the running database.
4. Write the gateway `location` block that would route a new `/reports/` REST endpoint to the backend, and say why it must not fall through to the SPA.

Knowledge check

? Q1: Why is only the gateway published to the host, and what does that buy you? ✓
(click for answer)

Defense in depth: the database, backend, and frontend are reachable only on the internal Docker network by service name, so the DB is never on the public internet and the backend is only ever reached through the proxy. It also enables the single-origin model, which is what makes the HttpOnly JWT cookie work.

? Q2: What three things does the backend do at container start *before* it serves traffic, and why in that order? (click for answer) ✓

It waits for `db` to be healthy, runs all module Django migrations (core first), and loads the DB-backed module configuration — then starts Gunicorn. Migrations must precede serving so the schema exists; config load must precede serving so runtime behavior is defined. Serving before these would 500 on every request.

? Q3: Why must `db` be 'healthy' rather than merely 'started' before the backend begins? (click for answer) ✓

PostgreSQL needs time to initialize and accept connections; a running container isn't yet a ready database. If the backend migrates against a not-yet-listening DB it crash-loops. A healthcheck-gated `depends_on` (e.g. `pg_isready`) makes the backend wait for *ready*.

? Q4: How does the set of installed modules get 'into' the backend image, and is it a runtime toggle? (click for answer) ✓

`openimis.json` (the manifest) is turned into `modules-requirements.txt` and pip-installed at image build time; `settings.py` builds `INSTALLED_APPS` from the loaded list at runtime. It is **not** a runtime toggle — changing the module set means editing the manifest and rebuilding the image, after which new migrations run on next start.

? Q5: A teammate runs backend on :8000 and frontend on :3000 directly and can't stay logged in. Why? (click for answer) ✓

They created two origins, bypassing the gateway. The `HttpOnly` auth cookie is scoped to the backend's origin and won't be sent to the frontend's origin, and CORS gets in the way. Running the gateway locally (single origin) matches production and fixes it.

Further reading

- [Deployment & Operations](#) — taking this stack to production: TLS, scaling, backups, upgrades.
- [Configuration System](#) — how `.env`, settings, the manifest, and DB config layer together.
- [The Database](#) — what the startup migrations actually do.
- [Security](#) — the single-origin/`HttpOnly`-cookie rationale in full.
- openIMIS distribution repo: github.com/openimis/openimis-dist_dkr.
- Docker Compose docs on `depends_on` conditions and healthchecks.

Deployment & Operations

The [previous chapter](#) got the stack running on your laptop. This one is about running it in a country. openIMIS deployments are not toy apps — they are the operational backbone of **national social health protection** schemes, holding the beneficiary registry, the policies, and the claims that pay real health facilities for real care. Downtime and data loss have human consequences. Production operations therefore prioritize, in order: **don't lose data, stay available, stay secure, then scale**.

This chapter covers the deployment models, the reverse proxy and TLS, how to scale a stateless Django app against a shared database, backups and production migrations, secrets, observability, upgrades, and how each country tailors its deployment **without forking core**.

Learning objectives

- Choose between single-node `docker-compose` and orchestrated (Kubernetes) deployment, and know the trade-offs.
- Configure the reverse proxy for TLS termination and a single secure origin.
- Scale the backend horizontally by treating it as a **stateless app over a shared database** (Gunicorn workers + multiple replicas).
- Design a **backup + restore** strategy and run migrations safely in production.
- Manage **secrets** correctly across environments.
- Stand up **observability**: logs, health checks, and OpenSearch-based monitoring/analytics.
- Reason about **zero-downtime upgrades** given that the backend self-migrates.
- Apply **country-specific customization** through configuration, not code forks.

Prerequisites

- [Docker Architecture](#) — the stack, its startup order, and the gateway.
 - [Configuration System](#) — the layered `.env` → settings → manifest → DB-config model you'll tune per environment.
 - [Security](#) — auth, TLS, and hardening this chapter assumes.
 - [The Database](#) — migrations and the temporal schema you'll back up and upgrade.
-

1. Deployment models

There are two broad shapes, and most openIMIS deployments start with the first.

The entire stack from [Part 12](#) on one adequately-sized VM, orchestrated by `docker compose`, fronted by the gateway with TLS.

Good when: a single country instance, moderate and predictable load, a small ops team, and simplicity/auditability matter more than elastic scale. This is the **most common production topology** for openIMIS and it is entirely legitimate — a well-provisioned single node with good backups serves a national scheme reliably.

Trade-offs: the node is a single point of failure; scaling is *vertical* (bigger VM) plus more Unicorn workers; upgrades briefly touch the whole box. You mitigate with backups, a standby VM, and a rehearsed restore.

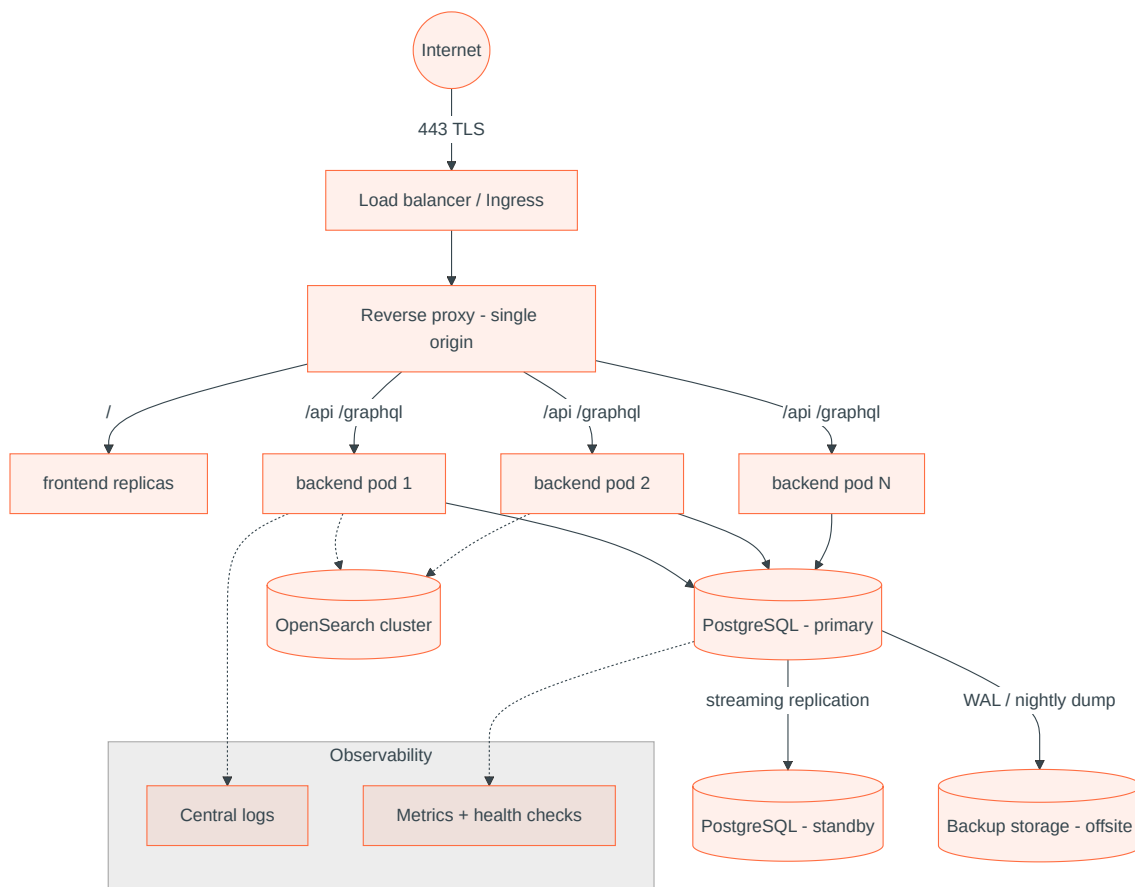
The same containers as Kubernetes workloads: the backend as a horizontally-scalable **Deployment** (many stateless pods behind a Service), the frontend as another, an **Ingress** as the gateway/TLS layer, PostgreSQL as either a managed cloud database or a carefully-run StatefulSet, and OpenSearch as its own operator-managed cluster.

Good when: high availability is required, load is variable, you already run Kubernetes, or you need rolling upgrades and self-healing.

Trade-offs: substantially more operational complexity. **Stateful components need care** — running PostgreSQL in Kubernetes is a specialist task; most teams use a *managed* database instead and keep only the stateless tiers in the cluster. Don't adopt Kubernetes for its own sake; adopt it when its HA/scaling guarantees are genuinely needed.

Single-node docker-compose

Orchestrated / Kubernetes



The invariant that makes everything on the right of that diagram possible: **the backend is stateless**. All state lives in PostgreSQL (and uploaded media / OpenSearch). That is what lets you run one backend or ten identically.

2. Reverse proxy and TLS

In production the gateway does two extra jobs beyond the routing you saw in [Part 12](#): it **terminates TLS** and it is the **only** thing on the public internet.

- **TLS termination at the edge.** The gateway (Nginx/Ingress) holds the certificate and serves HTTPS; traffic to `backend / frontend` stays on the private network. Certificates come from Let's Encrypt (ACME auto-renewal) or the country's own CA.
- **Redirect HTTP → HTTPS** and set security headers (HSTS, sane cache/security headers) at the proxy.
- **Single origin preserved.** TLS doesn't change routing: `/` → frontend, `/api + /graphql` → backend, all under one HTTPS origin. That single-origin design is what keeps the **HttpOnly JWT cookie** working and the app CORS-free (see [Security](#)).
- **Cookie flags in prod.** With HTTPS the auth cookie is served `Secure + HttpOnly` (+ appropriate `SameSite`). Ensure `DEBUG=False` and correct `ALLOWED_HOSTS / SITE_URL` so

Django emits secure cookies and absolute URLs correctly.

⚡ Common mistake — terminating TLS but leaving Django thinking it's on HTTP

Behind a TLS-terminating proxy, Django must be told the original request was HTTPS (via the forwarded proto header and `SECURE_PROXY_SSL_HEADER`), or it will build `http://` URLs and may refuse to set `Secure` cookies. Configure the proxy to pass `X-Forwarded-Proto` and Django to trust it. Symptom: redirect loops or "insecure cookie" behavior behind a perfectly good certificate.

3. Scaling the backend

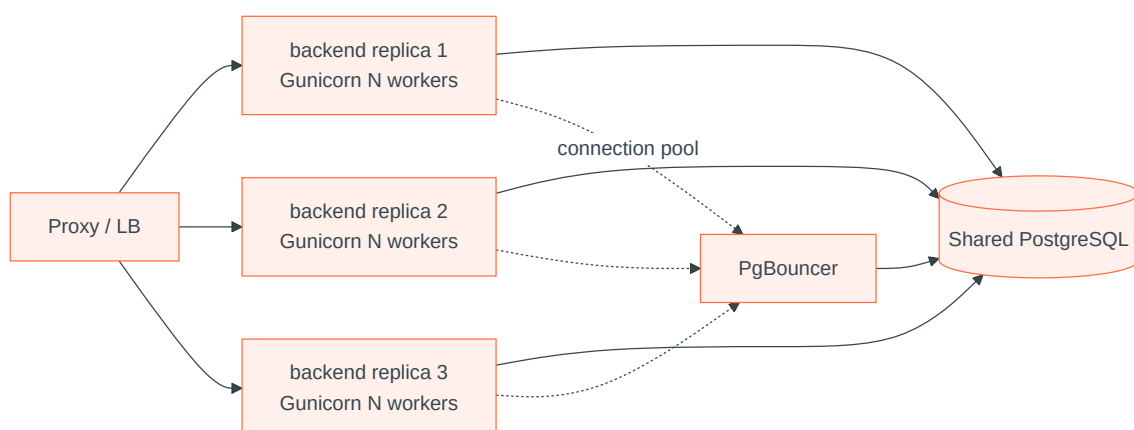
Because the backend is a **stateless Django app over a shared database**, scaling has two independent dials.

Dial 1 — Gunicorn workers (scale up, within one container)

Gunicorn runs multiple worker processes per container. More workers = more concurrent requests handled by one backend instance. The classic starting point is roughly $2 \times \text{CPU cores} + 1$ workers, tuned by profiling. Because openIMIS is I/O-bound (lots of database round-trips), you can often push worker count a bit higher, but watch **database connections**: every worker holds connections, and PostgreSQL has a `max_connections` ceiling.

Dial 2 — Replicas (scale out, more containers)

Run several identical backend containers/pods behind the proxy. Since no request depends on which instance served the last one (state is in the DB and the JWT cookie, not in-process), the load balancer can spread requests freely. This is *horizontal* scaling and it's the main lever under real load.





The database is the real bottleneck

Stateless app tiers scale trivially; the **shared PostgreSQL** does not. Before adding backend replicas, make sure the database can take the extra connections and query load. Put a **connection pooler (PgBouncer)** in front of PostgreSQL so $N \text{ replicas} \times M \text{ workers}$ don't exhaust `max_connections`. Offload heavy read/analytics traffic to **OpenSearch** (§6) and read replicas rather than hammering the primary.



Two things that are NOT the stateless web tier

- **The scheduler (APScheduler).** Background jobs must run in exactly one place, or you'll double-execute. If you scale the backend, run the scheduler as a *single* dedicated instance/replica, not once per web replica.
- **Migrations.** Applied once per deploy, not per replica (§4).

4. Database backups and migrations in production

Backups — the non-negotiable

The database volume holds everything irreplaceable. Protect it in layers:

Layer	Mechanism	Recovery capability
Logical dumps	<code>pg_dump</code> on a schedule (nightly), stored offsite	Restore to a point-in-day; portable across versions
Physical / continuous	WAL archiving + base backups (e.g. <code>pgBackRest</code> , <code>wal-g</code>)	Point-in-time recovery to any moment
Standby replica	Streaming replication to a second server	Fast failover; not a substitute for backups
Verification	Periodic test restores into a scratch environment	Proves the backups actually work



An untested backup is not a backup

The most common — and most catastrophic — operations failure is discovering during an incident that the backups were corrupt, incomplete, or never actually ran. **Rehearse the restore** on a schedule. Also remember openIMIS **soft-deletes**: because rows are versioned rather than physically deleted ([Database](#)), the database grows steadily — size your storage and dump windows accordingly.

Migrations in production

The backend **self-migrates at startup** ([Part 12 §3](#)) — hugely convenient, and a real hazard if unmanaged:

- **Back up immediately before migrating.** A migration is a schema change on live national data. Take a fresh dump first, every time.
- **Run migrations once, not per replica.** With multiple backend replicas, don't let all of them race to migrate. Run migrations as a **dedicated one-shot step** (a migration job/init container, or a single "primary" backend) *before* rolling out the new app replicas. Concurrent migrations corrupt state.
- **Test migrations on a restored copy first.** Restore last night's dump into staging, run the new migrations there, confirm they apply cleanly and in reasonable time on production-sized data. Long-running migrations on huge versioned tables can lock things — know the runtime before production.
- **Forward-compatible when possible.** For zero-downtime, prefer additive migrations (new nullable columns, new tables) that the *old* code tolerates, so old and new app versions can briefly coexist during a rollout ([§7](#)).

5. Secrets management

`.env` is fine for a laptop. Production secrets need more discipline.

Secret	Where it must not live	Where it should live
DJANGO_SECRET_KEY	git, image layers, shared chat	Secret store / orchestrator secret, injected at runtime
DB_PASSWORD	committed <code>.env</code> , Dockerfile	Secret store; rotated
OIDC client secrets, API keys	source, logs	Secret store; scoped per environment
TLS private keys	app repo	Proxy/Ingress secret; restricted file perms

Principles:

- **Never commit real secrets.** Commit `.env.example` with placeholders; keep the real `.env` out of git and off images.
- **Inject at runtime**, don't bake into images. Docker/Compose secrets, Kubernetes Secrets, or an external manager (Vault, cloud secret manager) delivered as env vars or mounted files.
- **Distinct secrets per environment.** Dev, staging, and production must not share a `DJANGO_SECRET_KEY` or database password.
- **Rotate**, and have a documented rotation procedure — especially after any suspected exposure or staff change.

Cross-reference: the security implications of these values are detailed in [Security](#), and their layering in [Configuration](#).

6. Observability

You cannot operate what you cannot see. Three pillars.

Logging

- Containers log to **stdout/stderr**; ship those to a central store (ELK/OpenSearch, Loki, or a cloud logging service) so logs survive container churn and are searchable across replicas.

- Log at appropriate levels; **never log secrets, tokens, or full request bodies** containing beneficiary data. With `DEBUG=False`, Django won't leak tracebacks to users — route them to your log sink instead.

Health checks

- The `db healthcheck` gates startup ([Part 12](#)).
- Give the **backend** a health/readiness endpoint the load balancer or Kubernetes probes hit, so unhealthy replicas are pulled from rotation automatically. Distinguish *liveness* (process alive) from *readiness* (can serve — DB reachable, migrations applied).
- Alert on: backend 5xx rate, DB connection saturation, disk space (remember the ever-growing versioned tables), certificate expiry, and replication lag.

Analytics and monitoring with OpenSearch

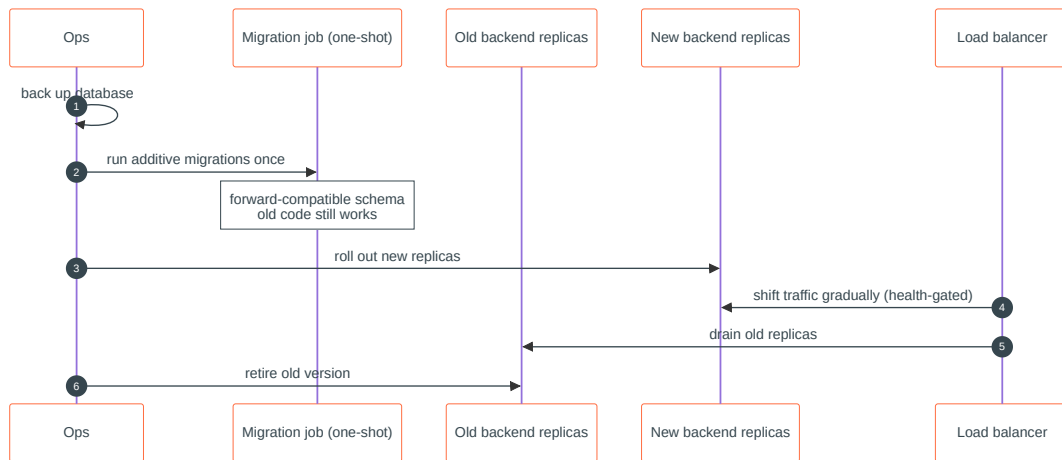
The optional **OpenSearch** service backs the `opensearch_reports` module. Beyond analytics dashboards for programme data, the same cluster commonly doubles as the **log/metrics sink**, giving operators one place to search operational logs and programme analytics. Running it well means a proper multi-node cluster with its own persistence and resourcing — treat it as infrastructure, not an afterthought.

Did you know?

Offloading reporting queries to OpenSearch isn't only a features decision — it's a **scaling** decision. Heavy analytical scans over the versioned PostgreSQL tables compete with transactional traffic (enrollment, claims). Serving reports from OpenSearch keeps the primary database responsive for the operations that must never slow down.

7. Zero-downtime upgrades

Because the backend self-migrates, naïve upgrades cause a blip: recreate the backend, it migrates, brief unavailability. To approach zero downtime:



Key ideas:

1. **Decouple migration from app rollout.** Run migrations as a single controlled step *before* bringing up new app replicas — never let replicas race to migrate (§4).
2. **Make migrations forward-compatible.** Additive changes (nullable columns, new tables) let old and new backend versions run simultaneously during the rollout window. Reserve destructive changes for a later, separate step once all replicas are new. The **soft-delete/versioning** model helps here: you close old versions rather than dropping data, so schema evolution rarely requires destructive DELETES.
3. **Roll replicas gradually behind health checks** so traffic only moves to instances that are ready.
4. **Keep a rollback path:** the pre-upgrade backup, the previous image tag, and a plan. If a migration can't be cleanly reversed, that's exactly why the backup exists.

⚠ Single-node reality check

True zero-downtime needs multiple replicas and forward-compatible migrations. On a **single-node** deployment you generally accept a short maintenance window: announce it, back up, upgrade, verify. That's a perfectly responsible posture for many national instances — honesty about the window beats a fragile pretense of zero downtime.

8. Country-specific deployment customization

openIMIS runs in many countries, each with different products, languages, regulatory fields, and integrations. The governing rule — the whole reason the platform is modular — is:

” The prime directive

Customize by configuration and extension, never by forking core. A forked core cannot receive upstream security fixes and features; a configured deployment can.

Where each kind of difference lives:

Country difference	Handled by	Not by
Which modules run	<code>openimis.json</code> manifest (image build)	patching another country's image
Runtime behavior, permissions, feature toggles	DB-backed <code>ModuleConfiguration</code> overlay	editing module <code>DEFAULT_CFG</code> in source
Environment values (URLs, DB, secrets)	<code>.env</code> per deployment	hard-coding in settings
Extra data fields	<code>json_ext</code> columns (<code>Database</code>)	new columns forked into <code>tblInsuree</code>
Language / labels	i18n (backend + react-intl) + config	forking the frontend
New country logic	a new module + service signals	editing core in place

All of these are the deployment-time expressions of the [Configuration System](#). The operational payoff is concrete: when upstream ships a security patch, a *configured* deployment pulls the new image, runs migrations, and keeps its `.env` + DB config + `json_ext` + custom modules intact. A *forked* deployment has to merge, and eventually stops merging, and eventually stops getting fixes.

Repository references

Repository	Directory / File	Why it matters
<code>openimis-dist_dkr</code>	<code>docker-compose.yml</code> , gateway conf, <code>.env.example</code>	The production stack, TLS/proxy config, and environment template you adapt per country.
<code>openimis-be_py</code>	<code>Dockerfile</code> , <code>script/</code>	Startup migration/config behavior you must control in prod deploys.
<code>openimis-be_py</code>	<code>openimis/settings.py</code>	Where <code>DEBUG</code> , <code>ALLOWED_HOSTS</code> , <code>secure-cookie/proxy-header</code> , and DB settings are read.
<code>openimis-be_py</code>	<code>openimis.json</code>	Per-country module set (build-time).
<code>openimis-be-core_py</code>	<code>core/apps.py</code> (<code>ModuleConfiguration</code>)	The runtime config overlay operators tune without code changes.
<code>openimis-be-opensearch_reports_py</code>	module root	Analytics/monitoring backed by OpenSearch.

Hands-on lab

Lab — production-shape a deployment

Do this against a staging VM or a second local project, never a live instance.

1. **Harden the environment file:** set `DEBUG=False`, a strong unique `DJANGO_SECRET_KEY`, correct `ALLOWED_HOSTS` / `SITE_URL`, and a real `DB_PASSWORD`. Confirm the app still boots and that error pages no longer show tracebacks.
2. **Add TLS at the gateway:** put a certificate (self-signed for the lab) on the proxy, redirect `HTTP → HTTPS`, and configure `X-Forwarded-Proto` + Django's `SECURE_PROXY_SSL_HEADER`. Verify the auth cookie is now `Secure`.
3. **Take and restore a backup:**

```
docker compose exec db pg_dump -U <user> <db> > backup.sql # dump
# simulate loss into a scratch DB, then:
docker compose exec -T db psql -U <user> <scratchdb> < backup.sql
```

Prove the restore contains your data. This is the single most important operational muscle to build.

4. **Scale the backend** to multiple replicas (`docker compose up -d --scale backend=3` or the compose `deploy.replicas` equivalent) and confirm the proxy balances across them and login still works on any replica (statelessness proven).
5. **Separate migrations from rollout:** disable auto-migrate-on-start for a run, apply migrations as a one-shot (`docker compose run --rm backend python manage.py migrate`), *then* start the app replicas. Note the difference from the default self-migrating behavior.
6. **Wire up observability:** ship container logs to a central sink and add a backend health probe the proxy consults; kill a replica and watch it get pulled from rotation.

Exercises

1. Draw the failure modes of the single-node model and the mitigation for each (SPOF, disk fills from versioned growth, bad migration, cert expiry).
2. You must add three national-ID-style fields for a new country. Show that this needs **no** core changes — specify exactly where each lives.
3. Your backend is at 6 replicas × 8 Gunicorn workers and PostgreSQL is refusing connections. Diagnose and propose a fix without adding database CPU.
4. Write the upgrade runbook for a schema change that adds a new nullable column, targeting zero downtime across 3 replicas.

Knowledge check

? Q1: What property of the backend makes horizontal scaling possible, and what must you watch as you add replicas? (click for answer) ✓

The backend is **stateless** — all state is in PostgreSQL (plus media/OpenSearch) and the JWT cookie — so any replica can serve any request. As you add replicas, watch database **connection exhaustion** (use PgBouncer) and don't run the single-instance scheduler or migrations per replica.

? Q2: Why must migrations be decoupled from app rollout when running multiple replicas? (click for answer) ✓

If every replica self-migrates at start, they race and can corrupt schema state. Migrations must run **once** as a controlled one-shot step before new app replicas roll out — ideally additive/forward-compatible so old and new code coexist during the rollout.

? Q3: What must you configure so Django sets Secure cookies correctly behind a TLS-terminating proxy? (click for answer) ✓

The proxy must forward `X-Forwarded-Proto`, and Django must trust it via `SECURE_PROXY_SSL_HEADER`, with `DEBUG=False` and correct `ALLOWED_HOSTS` / `SITE_URL`. Otherwise Django thinks it's on HTTP, builds `http://` URLs, and may refuse Secure cookies — causing redirect loops or insecure-cookie behavior.

? Q4: Why is offloading reports to OpenSearch a scaling decision, not just a feature? (click for answer) ✓

Heavy analytical scans over the versioned PostgreSQL tables compete with transactional enrollment/claims traffic. Serving reports from OpenSearch keeps the primary database responsive for operations that must not slow down, effectively separating the analytical and transactional workloads.

? Q5: A country needs extra beneficiary fields and a new language. Why is forking core the wrong answer? (click for answer) ✓

A fork can't cleanly receive upstream security fixes and features. Extra fields belong in `json_ext`, language in `i18n/config`, behavior in the DB-backed `ModuleConfiguration`, and new logic in a new module — all of which survive image upgrades. Configuration over forking keeps the deployment maintainable and patchable.

Further reading

- [Docker Architecture](#) — the stack this chapter operationalizes.
- [Configuration System](#) — the layered config you tune per environment and country.
- [Security](#) — TLS, secrets, auth, and hardening in depth.
- [The Database](#) — backups, migrations, and the versioned growth you must plan for.
- openIMIS distribution repo and wiki: github.com/openimis.
- PostgreSQL docs: continuous archiving / PITR, streaming replication; PgBouncer documentation.

Part 13 — The Configuration System

There is a question that decides whether an openIMIS deployment stays maintainable for a decade or rots into an unpatchable fork: **when a country needs the software to behave differently, where does that difference go?** The answer openIMIS gives — the answer that runs through every part of this handbook — is a **layered configuration system**. You change behavior by turning knobs, not by editing core code. This chapter is the definitive tour of those layers, their precedence, and how to use them.

If you internalize one sentence from Part 13, make it this: **environment configures the deployment, the manifest configures the modules present, and the database configures their runtime behavior — code stays untouched.**

Learning objectives

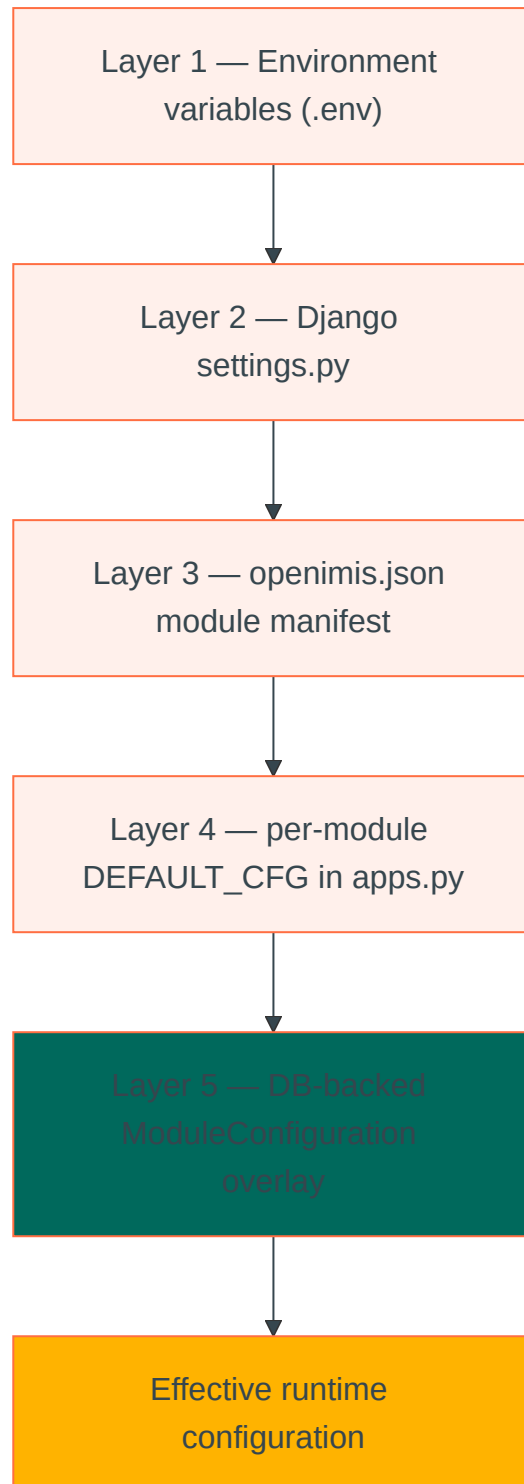
- Describe the five configuration layers and the **precedence** between them.
- Distinguish **build-time** configuration (which modules exist) from **runtime** configuration (how they behave).
- Read a module's `DEFAULT_CFG` in `apps.py` and understand how the DB overlay changes it *without a code change*.
- Manage **secrets**, **feature flags**, and **localization** through the right layer.
- Apply the **country-customization strategy**: configuration over forking.
- Override a module's configuration via `ModuleConfiguration`, with a worked example.

Prerequisites

- [Backend Deep Dive](#) — how modules assemble; where `apps.py` lives.
 - [Docker Architecture](#) — the `.env` file and the startup config-load step.
 - [The Database](#) — `ModuleConfiguration` is a DB-stored record; `json_ext` is the data-layer sibling of this idea.
 - [Deployment & Operations](#) — how these layers differ per environment and country.
-

1. The five layers

openIMIS configuration is a **stack of overlays**. Each layer sits on top of the previous one and can override it. Reading from the bottom (most static, set earliest) to the top (most dynamic, applied last):



#	Layer	Lives in	Scope	Changed by	When it takes effect
1	Environment variables	<code>.env</code> (Docker), OS env	Deployment-wide	Ops editing <code>.env</code> / secrets	Container start
2	Django settings	<code>openimis/settings.py</code>	Whole Django project	Developers (reads env vars)	Process start
3	Module manifest	<code>openimis.json</code>	Which modules exist at all	Developers / build (image rebuild)	Image build
4	<code>DEFAULT_CONFIG</code>	each module's <code>apps.py</code>	One module's defaults	Module developers	App load
5	<code>ModuleConfiguration</code>	database (core)	One module, per deployment, at runtime	Operators, no code change	App load, from DB — the winning layer

The single most important distinction cutting across this table:

Build-time vs. runtime

Layers 1–4 are largely **build/boot-time** and code-adjacent. Layer 5 is **runtime and data-driven** — it lives in the database, so an operator can change a running deployment's behavior with a record edit and a restart, *without touching source or rebuilding an image*. That is the layer that makes country customization possible.

2. Layer by layer

Layer 1 — Environment variables (`.env`)

The deployment's raw values: database connection, secrets, host names, feature switches that must exist before Django even imports. Supplied by `docker-compose` from `.env` ([Docker Architecture §6](#)). Never contains code; never committed with real secret values.

Layer 2 — Django `settings.py`

`openimis/settings.py` is the ordinary Django settings module — with one openIMIS twist: it **reads Layer 1** (`os.environ`) for anything deployment-specific, and it **builds `INSTALLED_APPS` dynamically** from the loaded module list rather than hard-coding it. So settings is where env vars become Django configuration and where the module set becomes real apps.

```
# illustrative – settings.py reading env and assembling apps
DEBUG = os.environ.get("DEBUG", "False") == "True"
SECRET_KEY = os.environ.get("DJANGO_SECRET_KEY")
DATABASES = {"default": {
    "ENGINE": "django.db.backends.postgresql",
    "NAME": os.environ.get("DB_NAME"),
    "HOST": os.environ.get("DB_HOST", "db"),
    # ...
}}
INSTALLED_APPS = [...core Django...] + OPENIMIS_MODULES # built from the
manifest
```

Layer 3 — `openimis.json` (the module manifest)

The manifest lists ~47 modules, each with a pip/git source. It answers **"which modules exist in this deployment at all?"** — a *build-time* question. Add or remove a module here and rebuild the image ([Docker §4](#)). You cannot enable a module at runtime that the manifest never installed; the manifest is the outer boundary of what's even possible.

Layer 4 — `DEFAULT_CFG` in `apps.py`

Each module ships sensible defaults in its `AppConfig` subclass. This is the module developer's statement of "here's how I behave out of the box."

```
# illustrative — a module's apps.py
from django.apps import AppConfig

class ClaimConfig(AppConfig):
    name = "claim"

    # default runtime configuration for this module
    DEFAULT_CFG = {
        "gql_query_claims_perms": [111001],
        "gql_mutation_create_claims_perms": [111002],
        "default_validations_disabled": False,
        "max_claim_length": 20,
    }

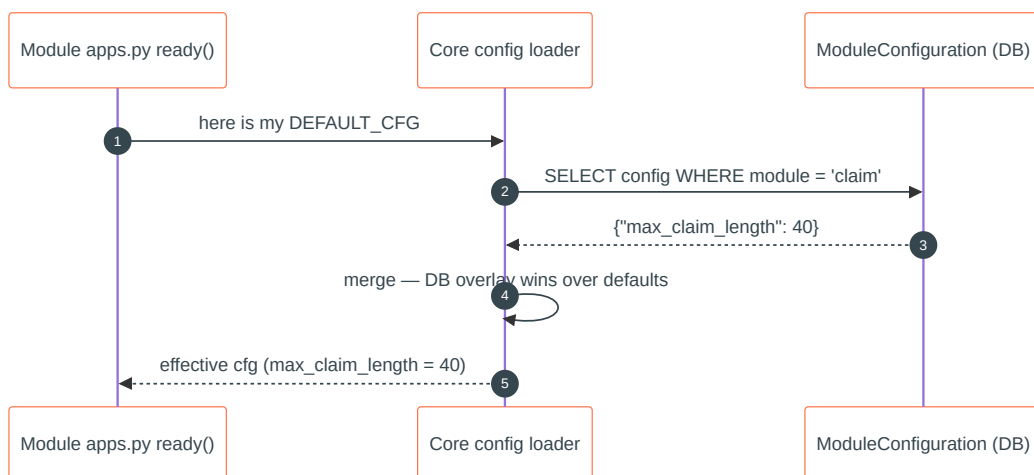
    def ready(self):
        # core reads DEFAULT_CFG, overlays the DB config, exposes the result
        self._load_config(cfg)
```

Two openIMIS conventions to note here (both grounded in [Core](#)):

- **Permission codes are integers** (e.g. `[111001]`), defined per module in `apps.py`, and enforced later via `user.has_perms([...])`.
- The `ready()` hook is where the module hands its `DEFAULT_CFG` to core so the **next layer** can overlay it.

Layer 5 — ModuleConfiguration (the DB overlay)

This is the keystone. Core (`openimis-be-core.py`) defines a `ModuleConfiguration` model — a database table keyed by module name, storing a JSON blob of configuration. At startup, for each module, core **merges the DB `ModuleConfiguration` on top of that module's `DEFAULT_CFG`**. Whatever the DB says **wins**.



Because this record lives in the database, an operator changes a deployment's behavior by editing a row (via the admin, a data migration, or the config-load step) and restarting — **no**

source edit, no image rebuild. The [Docker startup sequence](#) even loads/seeds these records (`openimisconf/load_openimis_conf.py`) before the app serves, so the overlay is present from the first request.

Did you know?

`ModuleConfiguration` is the runtime, *behavioral* sibling of the database's `json_ext`. One lets you extend **behavior** per deployment without code changes; the other lets you extend **data** per deployment without migrations. Same philosophy, two layers of the stack: **customize without forking**.

3. Precedence, precisely

When two layers speak to the same thing, the higher (later-applied) layer wins. Concretely for a module's runtime config:

`ModuleConfiguration` (DB) overrides `DEFAULT_CFG` (code).

And across the whole stack, the guiding order is: environment provides raw values → settings turns them into Django config → the manifest bounds which modules exist → each module supplies defaults → the DB overlay has the final say on runtime behavior.

Common mistake — editing `DEFAULT_CFG` to change one deployment

A tempting-but-wrong fix: a country needs `max_claim_length = 40`, so you edit the module's `apps.py` `DEFAULT_CFG` and redeploy. Now you've **forked the module**. Every future upstream update conflicts with your edit, and every *other* deployment that shares the image inherits your country's value. The correct fix is a `ModuleConfiguration` row for that deployment (see [§6](#)). Reserve `DEFAULT_CFG` edits for genuinely universal default changes that belong upstream.

Common mistake — trying to enable a module via DB config

`ModuleConfiguration` tunes modules that the **manifest already installed** (Layer 3). You cannot switch on a module that `openimis.json` never included — that's a build-time change requiring an image rebuild. Know which layer owns your change *before* you try to make it.

4. Secrets

Secrets belong in **Layer 1**, delivered as environment variables (or mounted secret files) — never in `settings.py` source, never in `openimis.json`, never committed.

Secret	Layer	Notes
<code>DJANGO_SECRET_KEY</code>	1 (env)	Strong, unique per environment.
<code>DB_PASSWORD</code>	1 (env)	Injected at runtime; rotated.
OIDC client secret / API keys	1 (env)	Scoped per environment.

`settings.py` (Layer 2) *reads* these from the environment but does not *contain* them. Full production handling — secret stores, rotation, per-environment separation — is in [Deployment & Operations §5](#) and [Security](#).



Common mistake

Putting a secret in `ModuleConfiguration` (the DB) or in `openimis.json` because "it's configuration too." Secrets are configuration, but they belong in the **environment layer** with restricted access — not in a database row visible to admins or in a manifest that lives in git.

5. Feature flags, localization, and country strategy

Feature flags via module config

A feature toggle is just a key in a module's config, overridable per deployment via Layer 5. `default_validations_disabled`, an optional workflow step, an integration on/off switch — these live in `DEFAULT_CFG` with a safe default and are flipped per country through `ModuleConfiguration`. No branching code paths compiled per deployment; one image, many behaviors.

Localization / i18n

openIMIS is multilingual by design:

- **Backend:** Django i18n plus per-module translation handling; user-facing strings are translatable, not hard-coded.

- **Frontend:** each `openimis-fe-<name>_js` module exports `translations`, and the UI uses `react-intl` (see [Frontend Architecture](#)). Language is a configuration/user concern, not a fork.
- **Data-level labels** (product names, location names) live in the data, and extra locale-specific fields can ride in `json_ext`.

Country customization strategy

Putting it together, here is the decision table every implementer should keep on the wall:

The country needs to change...	Do it in	Layer	Rebuild image?
Database, hostnames, secrets	<code>.env</code>	1	No (restart)
Which modules run at all	<code>openimis.json</code>	3	Yes
A module's runtime behavior / feature flag / permissions mapping	<code>ModuleConfiguration</code>	5	No (restart)
Extra beneficiary data fields	<code>json_ext</code>	data	No
Language / labels	i18n + config	2/5	No
Genuinely new business logic	a new module + service signals	3	Yes (add module)

” Configuration over forking

Everything above avoids editing core. The prize is upgradeability: when upstream ships a security fix, a **configured** deployment pulls the new image, migrates, and keeps its `.env` + DB config + `json_ext` + custom modules. A **forked** deployment merges — until it can't. This is the same prime directive as [Deployment §8](#); configuration is how you obey it.

6. Worked example — overriding a module's config

Scenario. A national scheme's claims can carry a longer claim code than the default. The `claim` module ships `DEFAULT_CFG = {"max_claim_length": 20, ...}` in `claim/apps.py`.

This country needs `40`. We must **not** touch source.

Step 1 — Identify the key

Read the module's `apps.py` to find the real key name and default (here, illustratively, `max_claim_length`). Never guess — the authoritative keys are in the module's `DEFAULT_CFG`.

Step 2 — Create/adjust the `ModuleConfiguration` row

The overlay is a DB record for module `claim`. You can set it several ways; all land in the same table.

```
# illustrative
from core.models import ModuleConfiguration
cfg, _ = ModuleConfiguration.objects.get_or_create(module="claim")
data = cfg.config or {}
data["max_claim_length"] = 40
cfg.config = data
cfg.layer = "be"          # backend-layer config
cfg.is_disabled = False
cfg.save()
```

Provide the module config JSON that `openimisconf/load_openimis_conf.py` loads at container start ([Docker §3](#)), so the row is seeded on deploy:

```
{
  "claim": { "max_claim_length": 40 }
}
```

Ship the override as a Django **data migration** in a country-specific module, so it's versioned and reproducible across environments — the most auditable option for production.

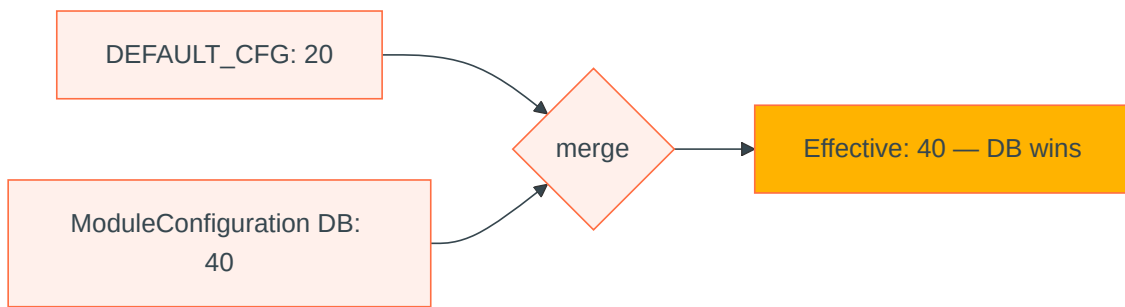
Django admin / shell

Config-load at startup

Data migration

Step 3 — Restart and verify

On restart, core reads `DEFAULT_CFG` (`max_claim_length = 20`), overlays the `ModuleConfiguration` (`= 40`), and the DB value wins. The effective config the module reads at runtime is `40`.



The result: this deployment behaves differently, the source is byte-for-byte upstream, and the next upgrade is a clean image pull. That is the entire point of the configuration system in one change.

⚠ Only override keys that exist

Overriding a key the module doesn't define does nothing useful — the module reads its own known keys. Confirm the key in `DEFAULT_CFG` first, and prefer overriding the minimum (merge onto existing config rather than replacing the whole blob, or you may wipe other overrides).

Repository references

Repository	Directory / File	Why it matters
openimis-be_py	openimis/settings.py	Layer 2 — reads env, builds <code>INSTALLED_APPS</code> from the manifest.
openimis-be_py	openimis.json	Layer 3 — the module manifest (which modules exist).
openimis-be_py	openimisconf/load_openimis_conf.py	Loads/seeds Layer 5 config at startup.
openimis-be_py	.env / .env.example	Layer 1 — environment values and secrets.
openimis-be-core_py	core/models.py (ModuleConfiguration)	Layer 5 — the DB overlay model.
openimis-be-core_py	core/apps.py	The config-merge machinery (<code>DEFAULT_CFG</code> overlaid by DB config).
any module	<name>/apps.py (<code>DEFAULT_CFG</code>)	Layer 4 — a module's default config and integer permission codes.
openimis-fe-*_js	module config translations	i18n/localization on the frontend.

Hands-on lab

Lab — change behavior without changing code

Against a running dev stack ([Docker Architecture](#)):

1. **Find a real config key.** Open a module's `apps.py` and read its `DEFAULT_CFG`. Pick a benign key to change (a length limit, a boolean flag).

2. **Confirm the default at runtime:**

```
docker compose exec backend python manage.py shell
```

```
from django.apps import apps
cfg = apps.get_app_config("claim") # or your chosen module
print(cfg.DEFAULT_CFG)             # inspect the shipped defaults
```

3. **Create a `ModuleConfiguration` override** (Step 2 above) for that module and key.
4. **Restart the backend** and verify the effective value changed to your override, while `apps.py` on disk is unchanged (`git status` clean).
5. **Prove precedence:** temporarily set a *different* value in `DEFAULT_CFG` locally and confirm the **DB value still wins** at runtime. Revert the source edit.
6. **Reflect:** you changed one deployment's behavior with a database row and a restart — no rebuild, no fork. Map this to the country-customization table in §5.

Exercises

1. For each change, name the correct layer and whether it needs an image rebuild: (a) new DB password, (b) enable a not-yet-installed analytics module, (c) raise a claim length limit for one country, (d) add a French translation, (e) add a national-ID field.
2. Explain, with the precedence rule, why editing `DEFAULT_CFG` to fix one deployment is a latent fork.
3. Where must a new OIDC client secret live, and why is `ModuleConfiguration` the wrong place?
4. Design the config for a feature flag that enables an optional claim validation only in Country A, shipped as a reproducible data migration.

Knowledge check

? **Q1: Name the five configuration layers from most static to most dynamic. (click for answer)** ✓

1) Environment variables (`.env`), 2) Django `settings.py` , 3) `openimis.json` module manifest, 4) per-module `DEFAULT_CFG` in `apps.py` , 5) DB-backed `ModuleConfiguration` overlay. Each overlays the previous; the DB layer has the final say on runtime behavior.

? **Q2: When `DEFAULT_CFG` and `ModuleConfiguration` set the same key, which wins and why does that matter? (click for answer)** ✓

`ModuleConfiguration` (the DB overlay) wins. It matters because it lets an operator change a single deployment's runtime behavior with a database record and a restart — no source edit, no image rebuild — which is what keeps deployments upgradeable instead of forked.

? **Q3: Why can't you enable a brand-new module via `ModuleConfiguration` ? (click for answer)** ✓

`ModuleConfiguration` only tunes modules that already exist — modules the `openimis.json` manifest (Layer 3) installed at build time. Adding a module is a build-time change requiring a manifest edit and image rebuild; the DB overlay can't install code.

? **Q4: In which layer do secrets belong, and where must they never go? (click for answer)** ✓

In the environment layer (Layer 1) as env vars or mounted secret files, injected at runtime. They must never be committed in `settings.py` , `openimis.json` , or stored in a `ModuleConfiguration` DB row.

? **Q5: A country needs a longer claim length. Outline the correct change and why it beats editing `DEFAULT_CFG` . (click for answer)** ✓

Create/update a `ModuleConfiguration` row for the `claim` module setting the `length` key to the new value (via admin/shell, the startup config-load, or a data migration), then restart. The source stays identical to upstream, so the next upgrade is a clean image pull — whereas editing `DEFAULT_CFG` forks the module and leaks the value to every deployment sharing the image.

Further reading

- [Backend Deep Dive](#) — how `apps.py`, settings, and the manifest fit the module architecture.
- [The Database](#) — `ModuleConfiguration` storage and its data-layer sibling `json_ext`.
- [Docker Architecture](#) — the `.env` layer and the startup config-load step.
- [Deployment & Operations](#) — per-environment/country configuration and secrets in production.
- [Security](#) — secret handling and the risks of misconfigured settings.
- openIMIS repositories: github.com/openimis — read `core/apps.py`, `core/models.py`, and any module's `apps.py` for the authoritative config machinery.

Integrations

openIMIS is rarely the only system in a country's health-financing landscape. It must exchange beneficiary and claim data with clinical systems using international standards, push aggregate figures to national monitoring dashboards, accept enrolment payments from mobile-money gateways, notify members by SMS, and sometimes sync identities with a national ID or ERP system. This part of the handbook covers those **outward-facing seams**.

The star of the show is **FHIR R4** (`openimis-be-api_fhir_r4_py`), openIMIS's standards-based REST API for clinical interoperability. We cover it thoroughly here; a separate deep-dive module page is not required — cross-link to this chapter.

Learning objectives

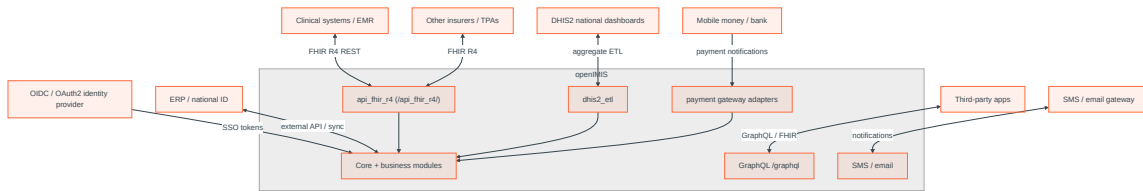
By the end of this chapter you will be able to:

- Map the main openIMIS models to their **FHIR R4** resource equivalents.
- Call the `/api_fhir_r4/` REST endpoints and understand how they differ from the [GraphQL](#) API.
- Describe how **DHIS2** aggregate export works.
- Explain the integration patterns for **payment gateways**, **SMS/email**, **ERP/identity**, external APIs, and **webhooks/eventing**.

Prerequisites

- [The Core module](#) — service accounts (`TechnicalUser`), rights, and the async mutation pattern.
- [Insuree](#), [Policy](#), [Claim](#), [Location](#) — the domain models FHIR resources map onto.
- Basic REST familiarity (you have this) and a passing awareness of HL7 FHIR (we teach the openIMIS-specific mapping).

The integration landscape



Two API surfaces coexist, by design:

Surface	Consumer	Shape	When to use
GraphQL <code>/graphql</code>	The openIMIS React frontend and bespoke integrators	Async mutations, connection queries, JWT-in- cookie	Rich internal operations, the admin UI.
FHIR R4 REST <code>/api_fhir_r4</code> <code>/</code>	External clinical systems, other insurers, HL7 tooling	Standard RESTful FHIR resources, JSON	Standards-based interoperability with the wider health IT world.

Did you know?

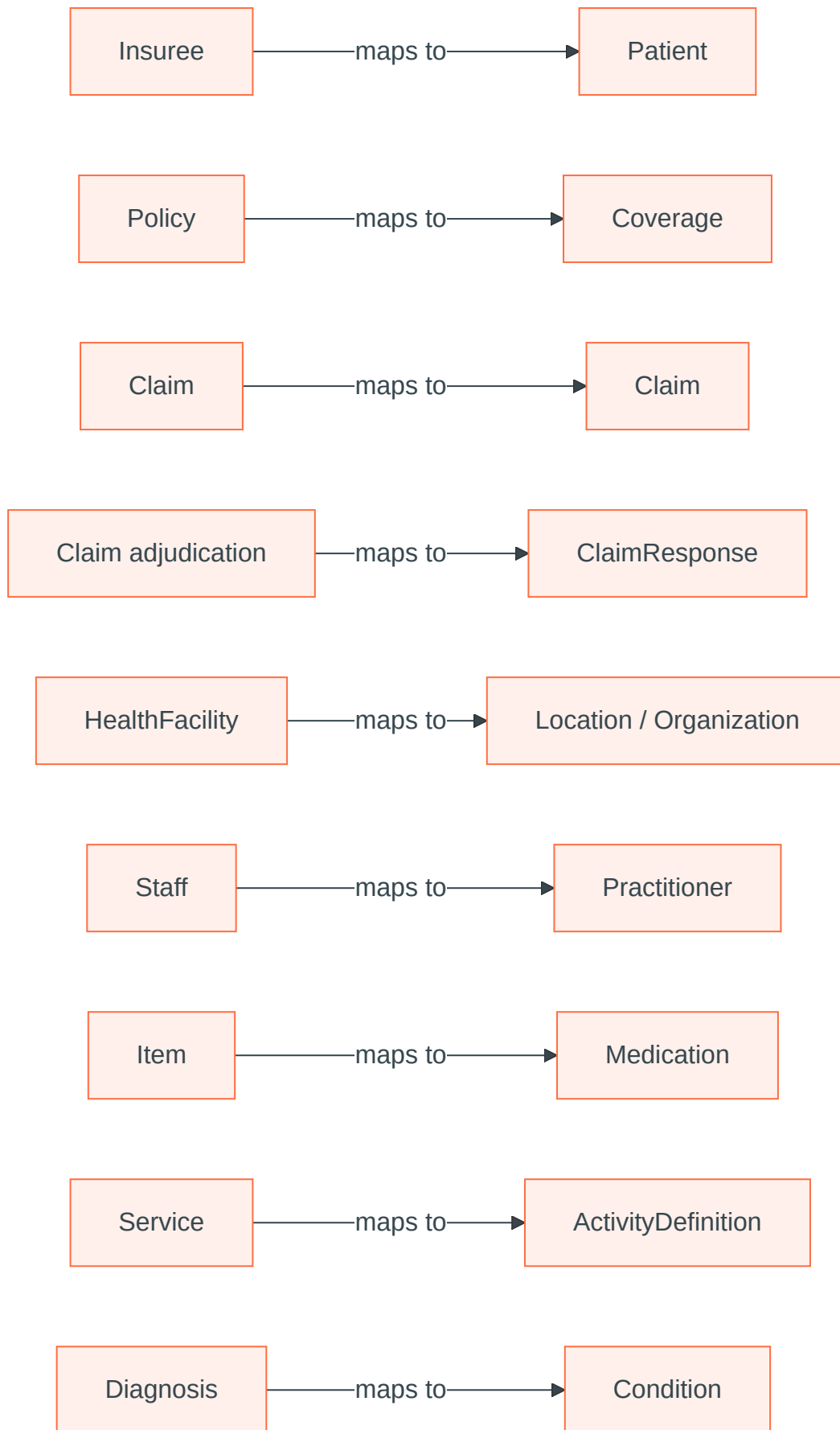
openIMIS keeps GraphQL for its own frontend but exposes **FHIR** to the outside world precisely because external clinical systems already speak FHIR. Rather than force partners to learn openIMIS's GraphQL schema, it meets them at an international standard. This is the classic "internal API vs. public integration API" split.

FHIR R4 — clinical interoperability

HL7 FHIR (Fast Healthcare Interoperability Resources) is the dominant modern standard for exchanging healthcare data. It models everything as **resources** (Patient, Coverage, Claim, Organization...) with a uniform REST interface. openIMIS does not *store* FHIR; `openimis-be-api_fhir_r4_py` is a **mapping layer** that translates openIMIS models to and from FHIR resources on the fly, under the base path `/api_fhir_r4/`.

Resource mapping table

openIMIS model	FHIR R4 resource	Notes
Insuree	Patient	The beneficiary. CHF/insurance number → Patient identifier.
Policy	Coverage	Active coverage of a patient under a product.
Policy agreement / contract	Contract	The legal coverage agreement.
HealthFacility	Location / Organization	Physical location and the org that runs it.
Medical staff / claim admin	Practitioner / PractitionerRole	Who delivers/administers care.
Medical Item	Medication	A drug/consumable.
Medical Service	ActivityDefinition	A billable act/procedure.
Claim	Claim	Service delivery submitted for reimbursement.
Claim adjudication result	ClaimResponse	The adjudicated outcome (approved/rejected, valued).
Diagnosis (ICD)	Condition	Diagnoses on a claim.
Healthcare provision	HealthcareService	Services a facility offers.
Claim feedback / communication	CommunicationRequest	Feedback and messaging.
Eligibility check	CoverageEligibilityRequest	"Is this patient covered for this?"
Contribution / invoice	Invoice (+ subscriptions)	Billing artifacts.



Calling the FHIR API

```
# Retrieve insurees as FHIR Patients
curl -H "Authorization: Bearer <JWT>" \
      https://your-openimis/api_fhir_r4/Patient/

# A single coverage
curl -H "Authorization: Bearer <JWT>" \
      https://your-openimis/api_fhir_r4/Coverage/<uuid>/

# Submit a claim as a FHIR Claim bundle (POST)
curl -X POST -H "Content-Type: application/fhir+json" \
      -H "Authorization: Bearer <JWT>" \
      --data @claim.fhir.json \
      https://your-openimis/api_fhir_r4/Claim/
```

Key behaviours (from `openimis-be-api_fhir_r4_py`):

- **Standard REST verbs** — `GET` collections and instances (e.g. `GET /api_fhir_r4/Patient/`), `POST` to create where supported.
- **Pagination** — FHIR Bundle paging with a configurable default page size (10 by default).
- **Subscriptions** — a **REST-hook** subscription channel for `Patient`, `Organization` and `Invoice` resources, enabling push notifications to external systems.
- **Auth** — the same identity backbone as the rest of openIMIS. A `TechnicalUser` service account with the right integer rights, authenticating via JWT (or OIDC), is the clean way for a machine integrator to call FHIR.

Common mistake

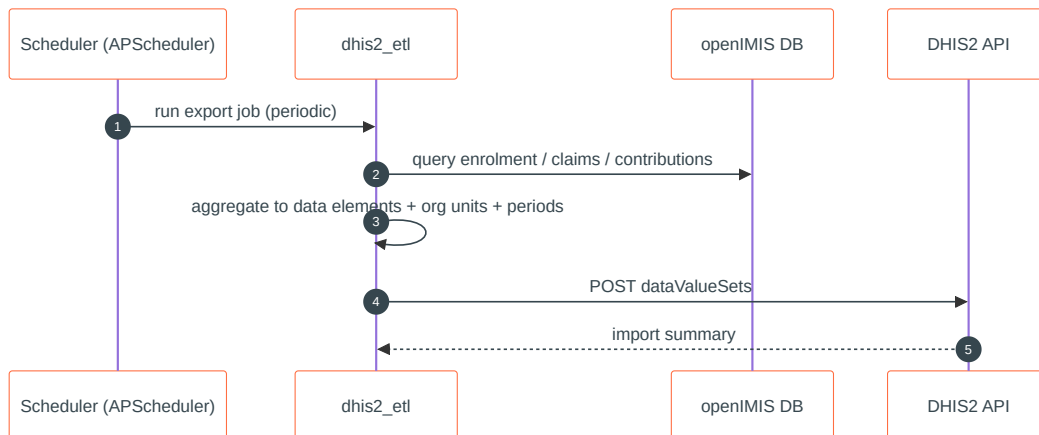
Do not confuse the **FHIR claim submission** path with the internal GraphQL `createClaim` mutation. Both create a claim, but FHIR is synchronous REST while GraphQL is the async `MutationLog`-polled pattern. Also: submitting a FHIR `Claim` still runs the same **adjudication pipeline** — FHIR is a **doorway**, not a bypass of the business rules.

Deep dive: why Location and Organization for a facility?

FHIR separates the **physical place** (`Location`) from the **legal entity** that operates it (`Organization`). A single openIMIS `HealthFacility` carries both concepts (its geography via **location**, its identity/legal form as an org), so the mapping layer can present it as either resource depending on the query. This is a good illustration of impedance mismatch between a pragmatic operational model and a normalised standard.

DHIS2 — aggregate reporting

DHIS2 is the world's most widely deployed health management information platform; ministries use it for aggregate national indicators. openIMIS integrates via `openimis-be-dhis2_etl.py`, which **extracts** operational data (enrolment, claims, contributions), **transforms** it into DHIS2 data-value/aggregate formats, and **loads** it to a DHIS2 instance through DHIS2's API.



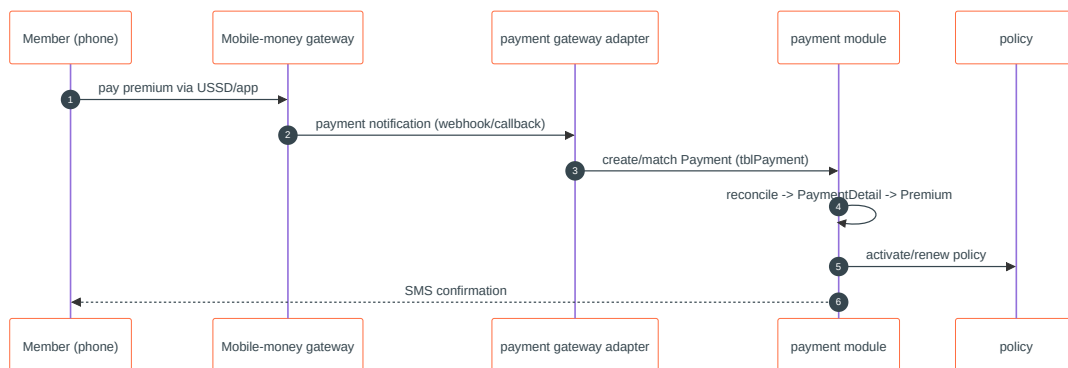
- It is an **aggregate**, one-directional push (openIMIS → DHIS2) — individual records are summarised into DHIS2 **data elements**, **org units** and **periods**, not sent row-by-row.
- Runs on the **scheduler** (APScheduler) that core provides, typically on a periodic cadence.
- The openIMIS **location hierarchy** maps to DHIS2 **org units**; getting that mapping right is the crux of a DHIS2 integration.

i Did you know?

Aggregate export exists because DHIS2 answers a different question than openIMIS: not "what is this one claim's status?" but "how many claims, of what value, in this district this quarter?" Sending individual records would both violate DHIS2's aggregate model and leak personal data.

Payment providers & mobile money

Because members often enrol and pay premiums by **mobile money**, openIMIS integrates with payment gateways. The pattern (see [Contribution & Payment](#)):



- Incoming notifications land as `Payment` records (status begins `NOTYETCONFIRMED`) and are **reconciled** to a policy/product, ultimately creating a `Premium` that activates the policy.
- The `Payment` model's `phone_number`, `transaction_no`, `receipt_no` and `date_last_sms` fields exist for exactly this loop.
- New providers are added as **adapters** that bind to the payment module's signals — no change to core payment logic.

⚡ Common mistake

Treat gateway callbacks as **untrusted**. Verify the provider's signature and make reconciliation **idempotent** (a duplicated callback must not create two `Premium` rows). The `transaction_no` is your idempotency key.

SMS & email notifications

Notifications ride on Django's standard facilities plus scheme-specific hooks:

- **Email** via Django's `EMAIL_*` settings and `send_mail` /templated backends.
- **SMS** via a configured gateway; enrolment/renewal/payment events trigger messages (the `date_last_sms` field on `Payment` tracks the last SMS sent).
- Delivery is typically wired through **service signals** — a notification module binds *after* a policy activation or claim event and dispatches the message, keeping messaging decoupled from business logic.

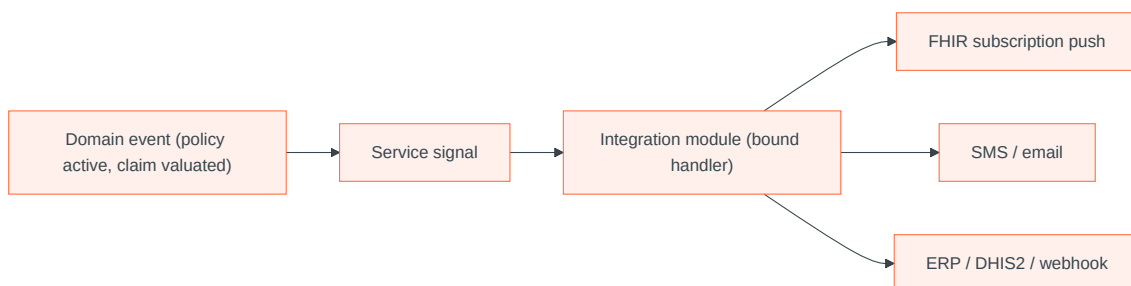
ERP & identity systems

Integration	Mechanism
Identity / SSO	openIMIS supports OpenID Connect / OAuth2 alongside its JWT auth (see Security), so an external identity provider can issue tokens. Users map to <code>core InteractiveUser / TechnicalUser</code> .
National ID	Beneficiary identity can be reconciled against a national registry via the external-API pattern or FHIR <code>Patient</code> identifiers.
ERP / finance	Financial artifacts from the invoice module can be exported/synced to an ERP; price lists can be sourced from an ERP via signals.

External APIs, webhooks & eventing

openIMIS's extension seams double as integration seams:

- **Inbound** — FHIR REST and GraphQL let external systems read/write with a `TechnicalUser` service account and integer rights.
- **Outbound push** — FHIR **REST-hook subscriptions** (Patient/Organization/ Invoice) and gateway callbacks provide event push.
- **Internal eventing** — core's **service signals** (`register_service_signal / bind_service_signal`) are the in-process event bus; an integration module binds to lifecycle events (policy activated, claim valuated) and fans them out to external systems. See [Claim signals](#) and [Calculation](#).



Prefer signals over patching

When you must react to an openIMIS event for an integration, **bind to a service signal** rather than editing the emitting module. Your integration then lives in its own module, survives upgrades, and can be toggled by adding or removing it from the assembly `openimis.json`.

Hands-on lab

Lab: read the same insuree two ways

1. Create an insuree with an active [policy](#).
2. Query them over **GraphQL** (`insurees { ... }`) and note the shape.
3. `GET /api_fhir_r4/Patient/` with a valid token and find the **same** person as a FHIR `Patient`. Note how the CHF number appears as an identifier.
4. `GET /api_fhir_r4/Coverage/` and locate the `Coverage` corresponding to the policy. Confirm both APIs surface the same underlying rows.
5. Write one sentence explaining when you would hand an external partner the FHIR endpoint vs. the GraphQL endpoint.

Knowledge check

Q1: Why does openIMIS expose FHIR to external systems but use GraphQL internally? (click for answer)

GraphQL is tailored to openIMIS's own frontend (async mutations, connections, cookie JWT). External **clinical** systems already speak the **FHIR** standard, so exposing FHIR lets partners integrate without learning openIMIS's bespoke schema. It is the internal-API vs. public-integration-API split.

Q2: Which FHIR resources do Insuree, Policy, Claim, and a claim's adjudication map to? (click for answer)

Insuree → **Patient**, Policy → **Coverage**, Claim → **Claim**, and the adjudication result → **ClaimResponse** (with Diagnosis → Condition, HealthFacility → Location/Organization, staff → Practitioner/PractitionerRole).

? Q3: Why is DHIS2 export aggregate and one-directional? (click for answer) ✓

DHIS2 answers aggregate questions (counts/values per org unit per period), not per-record ones. Sending individual claims would break DHIS2's aggregate data model and expose personal data, so `dhis2_etl` summarises into data elements/org units/periods and pushes them out.

? Q4: A mobile-money gateway sends the same payment callback twice. How do you avoid double-activating a policy? (click for answer) ✓

Make reconciliation **idempotent** using `transaction_no` as the key: if a `Payment` with that transaction already exists, ignore the duplicate. Also verify the callback signature — gateway callbacks are untrusted input.

? Q5: You need an external system notified whenever a policy activates. What is the cleanest mechanism? (click for answer) ✓

Bind to a core service signal for policy activation from your own integration module and push outward (FHIR subscription, webhook, SMS). This keeps the integration decoupled, upgrade-safe, and toggleable via the assembly manifest — no patching of the policy module.

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be-api_fhir_r4_py</code>	<code>api_fhir_r4/converters/</code>	The model ↔ FHIR resource mapping (Insuree → Patient, etc.).
<code>openimis-be-api_fhir_r4_py</code>	<code>api_fhir_r4/views.py, urls.py</code>	REST endpoints under <code>/api_fhir_r4/</code> .
<code>openimis-be-dhis2_etl_py</code>	<code>dhis2_etl/</code>	Aggregate extract/transform/load to DHIS2.
<code>openimis-be-payment_py</code>	<code>payment/</code>	Gateway reconciliation → <code>Premium</code> .
<code>openimis-be-core_py</code>	<code>core/service_signals.py</code>	The internal event bus integrations bind to.

Further reading

- Source: [openimis-be-api_fhir_r4_py](#), [openimis-be-dhis2_etl_py](#)
- HL7 FHIR R4 spec: hl7.org/fhir/R4
- DHIS2 developer docs: docs.dhis2.org
- [Claim](#), [Policy](#), [Insuree](#) — the mapped domain models.
- [Contribution & Payment](#) — mobile-money enrolment.
- Official docs: [openIMIS wiki](#).

Extension Guide

Part 15 — Extending openIMIS: Build a New Module

This is the chapter where everything you have learned becomes muscle memory. You will scaffold a **new backend module** (`openimis-be-<name>.py`) and a matching **new frontend module** (`openimis-fe-<name>.js`), wire them into the two assemblies, add Docker support, and package/publish them. We will build a small, illustrative module called `vacation` (imagine a benefit that tracks approved wellness leave for insurees) — pick any name for your own; the anatomy is identical for every real module.

Learning objectives

By the end of this chapter you will be able to:

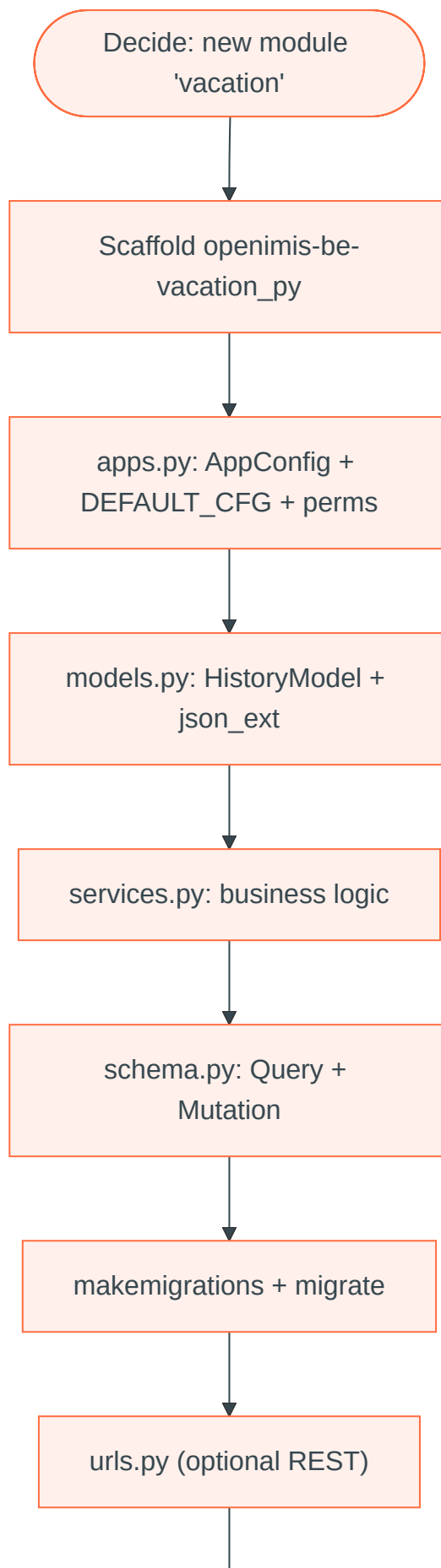
- Scaffold a backend module: `apps.py` (an `AppConfig` with `DEFAULT_CFG` + permission codes), `models.py` (core base models + `json_ext`), `services.py`, `schema.py` (`Query` / `Mutation`), `migrations/`, and `urls.py`.
- Register the module in the backend `openimis.json` manifest and install it editable.
- Scaffold a frontend module: the **config object** contributing `menus`, `routes`, `reducers`, and using `ModulesManager` + published components; register it in the frontend `openimis.json`.
- Add Docker support, package both halves (`setup.py` / `npm package.json`), and publish.

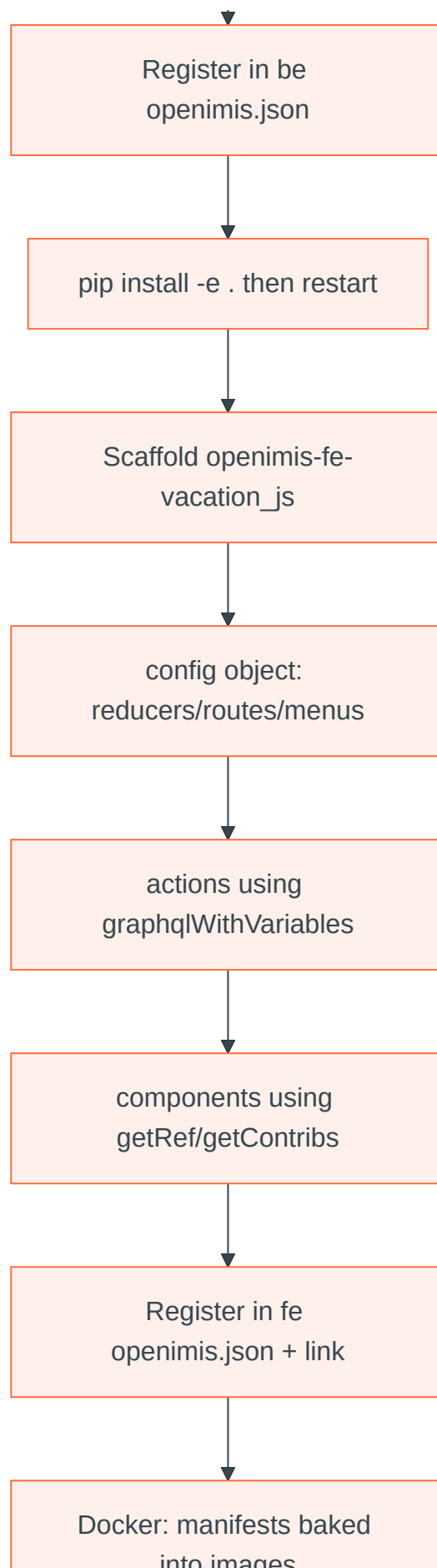
Prerequisites

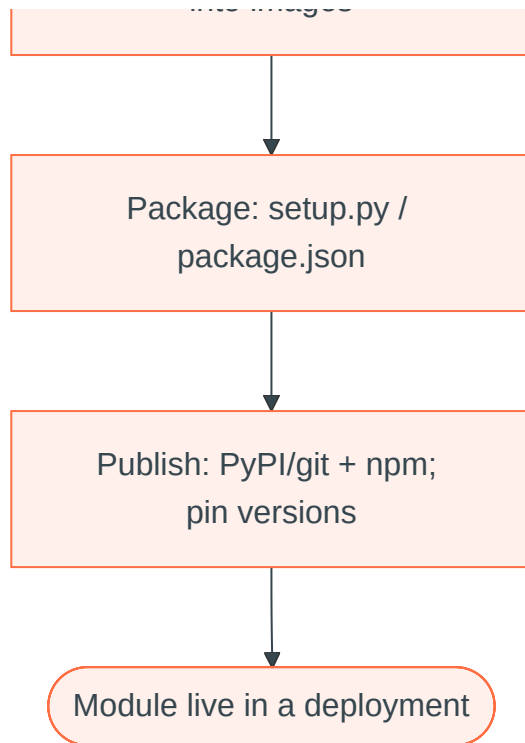
- [Backend Deep Dive](#) and [Plugin / Module System](#) — the module pattern.
 - [Frontend Architecture](#) — the config object, `ModulesManager`, the Redux GraphQL layer.
 - [The GraphQL Layer](#) — you should know queries, mutations, and the `OpenIMISMutation` async pattern.
 - [Core module](#) — base models (`HistoryModel`, `json_ext`), `OpenIMISMutation`, service signals.
 - [Set Up a Dev Environment](#) — you have `openimis-be_py` and `openimis-fe_js` running.
-

0. The big picture

A module is a **vertical slice** shipped as two repositories (backend + frontend), each independently versioned, each pinned by an assembly manifest. Nothing in core changes when you add a module — that is the whole point.







Work the backend first — the frontend calls it, so it helps to have a live GraphQL schema to build against.

1. Backend module

1.1 Repository layout

Create a repository named `openimis-be-vacation_py` with the standard module layout. The installed Python package is just `vacation`.


```

openimis-be-vacation_py/
├── setup.py
├── README.md
├── vacation/
│   ├── __init__.py
│   ├── apps.py
│   ├── models.py
│   ├── services.py
│   ├── schema.py
│   ├── gql_queries.py
│   ├── gql_mutations/
│   │   ├── __init__.py
│   │   ├── create_vacation.py
│   │   └── update_vacation.py
│   ├── signals.py
│   ├── urls.py
│   ├── apps_settings? (config lives in apps.py)
│   ├── migrations/
│   │   └── 0001_initial.py
│   └── tests/
│       └── test_services.py

```

1.2 apps.py — AppConfig, DEFAULT_CFG, permissions

Every module is a Django app. openIMIS convention: subclass `AppConfig`, define a `DEFAULT_CFG` dict of tunables, define integer **permission codes**, and load runtime config from the DB in `ready()` (the DB config, stored in core's `ModuleConfiguration`, *overlays* `DEFAULT_CFG` so operators can retune without code changes).

```
# vacation/apps.py (illustrative / simplified)
from django.apps import AppConfig

MODULE_NAME = "vacation"

# Permission codes are INTEGERS, unique across openIMIS. Pick an unused range.
DEFAULT_CFG = {
    "gql_query_vacations_perms": [160001],
    "gql_mutation_create_vacation_perms": [160002],
    "gql_mutation_update_vacation_perms": [160003],
    "gql_mutation_delete_vacation_perms": [160004],
    "default_validity_days": 30,
}

class VacationConfig(AppConfig):
    name = MODULE_NAME

    gql_query_vacations_perms = []
    gql_mutation_create_vacation_perms = []
    gql_mutation_update_vacation_perms = []
    gql_mutation_delete_vacation_perms = []
    default_validity_days = 30

    def _configure_permissions(self, cfg):
        VacationConfig.gql_query_vacations_perms =
cfg["gql_query_vacations_perms"]
        VacationConfig.gql_mutation_create_vacation_perms =
cfg["gql_mutation_create_vacation_perms"]
        VacationConfig.gql_mutation_update_vacation_perms =
cfg["gql_mutation_update_vacation_perms"]
        VacationConfig.gql_mutation_delete_vacation_perms =
cfg["gql_mutation_delete_vacation_perms"]

    def ready(self):
        from core.models import ModuleConfiguration
        cfg = ModuleConfiguration.get_or_default(MODULE_NAME, DEFAULT_CFG)
        self._configure_permissions(cfg)
        VacationConfig.default_validity_days =
cfg.get("default_validity_days", 30)
```



Common mistake

Permission codes must be unique integers across the whole platform. Two modules that reuse the same code silently share access. Before you pick a range, grep the other modules' `apps.py` (`gql_..._perms`) and choose a block nobody else uses. Document your range in the README.

Did you know?

`DEFAULT_CFG` is not just constants — it is the *contract* an operator can override per deployment. The value lives in `DEFAULT_CFG`, gets overlaid by the JSON stored in `ModuleConfiguration` (core), and is read back in `ready()`. That is how a country can re-tune your module without forking it. See [Configuration](#).

1.3 `models.py` — core base models + `json_ext`

Reuse core's base models rather than plain `django.db.models.Model`. `HistoryModel` gives you temporal validity (`validity_from` / `validity_to`), UUID identity, `legacy_id`, and a `json_ext` JSONField for schemaless extension fields — the openIMIS idiom for "let deployments add fields without a migration."

```
# vacation/models.py (illustrative / simplified)
from django.db import models
from core.models import HistoryModel
from insuree.models import Insuree

class Vacation(HistoryModel):
    insuree = models.ForeignKey(
        Insuree, on_delete=models.DO_NOTHING,
        related_name="vacations", db_column="InsureeID",
    )
    start_date = models.DateField(db_column="StartDate")
    end_date = models.DateField(db_column="EndDate")
    status = models.CharField(max_length=16, default="DRAFT",
db_column="Status")
    # json_ext is provided by HistoryModel: deployment-specific extra fields
    # e.g. instance.json_ext = {"reason": "wellness", "approvedBy": "..."}

    class Meta:
        db_table = "tblVacation"  # legacy-style table naming is the house
style
```

Did you know?

The `tbl` prefix and `db_column` CamelCase are not accidents — they are the **legacy heritage** from the original .NET/MSSQL IMIS. New tables follow the house style so the schema reads consistently. See [Database](#).

1.4 `services.py` — business logic

Keep business logic in services, not in resolvers. A service is a plain class/functions that validates and persists. Mutations call services; so can other modules (and tests). This is also

where you emit **service signals** so other modules can hook you without importing you.

```
# vacation/services.py (illustrative / simplified)
from datetime import date
from core.signals import register_service_signal
from .models import Vacation
from .apps import VacationConfig

class VacationService:
    def __init__(self, user):
        self.user = user

    @register_service_signal("vacation_service.create")
    def create(self, data: dict) -> Vacation:
        self._validate(data)
        vacation = Vacation(**data)
        vacation.save(username=self.user.username)  # HistoryModel audits the
writer
        return vacation

    def _validate(self, data: dict):
        if data["end_date"] < data["start_date"]:
            raise ValueError("end_date must not precede start_date")
```

Did you know?

`@register_service_signal("vacation_service.create")` lets another module `bind_service_signal("vacation_service.create", handler, after=True)` to run logic after every vacation is created — no import, no dependency. That decoupled seam is the backend twin of the frontend's `contributions`. See [Plugin / Module System](#).

1.5 `schema.py` and `gql_mutations/` — Query and Mutation

Your GraphQL types live in `gql_queries.py`; your `Query` and `Mutation` classes in `schema.py`. Mutations subclass core's `OpenIMISMutation` so they inherit the audited, asynchronous pattern (create a `MutationLog`, run the service, return a `clientMutationId`).

```
# vacation/gql_queries.py (illustrative / simplified)
import graphene
from graphene_django import DjangoObjectType
from core import ExtendedConnection
from .models import Vacation

class VacationGQLType(DjangoObjectType):
    class Meta:
        model = Vacation
        interfaces = (graphene.relay.Node,)
        connection_class = ExtendedConnection # adds totalCount / edgeCount
        filter_fields = {
            "id": ["exact"],
            "status": ["exact", "icontains"],
            "start_date": ["exact", "gte", "lte"],
        }
```

```
# vacation/gql_mutations/create_vacation.py (illustrative / simplified)
import graphene
from core.schema import OpenIMISMutation
from ..apps import VacationConfig
from ..services import VacationService

class CreateVacationMutation(OpenIMISMutation):
    _mutation_module = "vacation"
    _mutation_class = "CreateVacationMutation"

    class Input(OpenIMISMutation.Input):
        insuree_id = graphene.Int(required=True)
        start_date = graphene.Date(required=True)
        end_date = graphene.Date(required=True)

    @classmethod
    def async_mutate(cls, user, **data):
        if not user.has_perms(VacationConfig.gql_mutation_create_vacation_perms):
            raise PermissionError("unauthorized")
        try:
            VacationService(user).create(data)
            return None # None == success; MutationLog marked done
        except Exception as exc: # returned errors become MutationLog error
            detail
            return [{"message": str(exc)}]
```

```
# vacation/schema.py (illustrative / simplified)
import graphene
from core.schema import OrderedDjangoFilterConnectionField
from .apps import VacationConfig
from .gql_queries import VacationGQLType
from .gql_mutations.create_vacation import CreateVacationMutation
from .gql_mutations.update_vacation import UpdateVacationMutation

class Query(graphene.ObjectType):
    vacations = OrderedDjangoFilterConnectionField(
        VacationGQLType, orderBy=graphene.List(graphene.String),
    )

    def resolve_vacations(self, info, **kwargs):
        if not
info.context.user.has_perms(VacationConfig.gql_query_vacations_perms):
            raise PermissionError("unauthorized")
        return Vacation.objects.filter(*filter_validity())

class Mutation(graphene.ObjectType):
    create_vacation = CreateVacationMutation.Field()
    update_vacation = UpdateVacationMutation.Field()
```

The assembly repo (`openimis-be_py` , `openimis/schema.py`) combines every module's `Query` and `Mutation` into one root via multiple inheritance — you write only your slice.

Common mistake

Do not put permission checks *only* on the frontend. Every resolver and every mutation must call `user.has_perms([...])` with the **same integer codes** from `apps.py` . The frontend guard is UX; the backend check is security. Enforce in both.

1.6 Migrations

Standard Django. Because you used `db_table = "tblVacation"` and explicit `db_columns`, the migration is deterministic and legacy-consistent.

```
# from openimis-be_py, with the module installed editable
python manage.py makemigrations vacation
python manage.py migrate vacation
```

1.7 `urls.py` — optional REST endpoints

Most modules are pure GraphQL, but you can expose REST (reports, callbacks, FHIR). The assembly's `openimis/urls.py` collects each module's `urls.py` .

```
# vacation/urls.py (illustrative)
from django.urls import path
from . import views

urlpatterns = [
    path("vacation/export/", views.export_csv, name="vacation_export"),
]
```

1.8 Register in the backend `openimis.json` and install editable

Add your module to the backend assembly manifest so `INSTALLED_APPS` and the schema pick it up.

```
// openimis-be_py/openimis.json (add an entry)
{
  "name": "vacation",
  "pip": "openimis-be-vacation @ git+https://github.com/your-org/openimis-be-
vacation_py.git@v1.0.0"
}
```

For development, install the local checkout **editable** so code changes are live:

```
pip install -e ../openimis-be-vacation_py/
python manage.py migrate
python manage.py runserver # or restart the container
```

Confirm it worked: open GraphiQL at `/graphql` and check that `vacations` appears in the schema and a `createVacation` mutation exists.

2. Frontend module

2.1 Repository layout

Create `openimis-fe-vacation_js`; the npm package is `@openimis/fe-vacation`.

```
openimis-fe-vacation_js/
├─ package.json
├─ src/
│   ├─ index.js          # the config object (public API)
│   ├─ constants.js
│   ├─ reducer.js
│   ├─ actions.js        # graphqlWithVariables-based
│   └─ pages/
│       ├─ VacationsPage.js
│       └─ VacationPage.js
│   └─ components/
│       ├─ VacationSearcher.js
│       └─ VacationForm.js
│   └─ menus/
│       └─ VacationMainMenu.js
└─ translations/
    ├─ en.json
    └─ fr.json
```

2.2 The config object

This object *is* the module's API. It contributes a reducer, routes, and menu entries, and publishes reusable components.


```
// openimis-fe-vacation_js/src/index.js (illustrative / simplified)
import messages_en from "../translations/en.json";
import reducer from "../reducer";
import VacationsPage from "../pages/VacationsPage";
import VacationPage from "../pages/VacationPage";
import VacationMainMenu from "../menus/VacationMainMenu";
import VacationStatusPicker from "../components/VacationStatusPicker";
import { RIGHT_VACATION } from "../constants";

const ROUTE_VACATIONS = "vacation/vacations";
const ROUTE_VACATION = "vacation/vacation";

const DEFAULT_CONFIG = {
  "translations": [{ key: "en", messages: messages_en }],
  "reducers": [{ key: "vacation", reducer }],
  "refs": [
    { key: "vacation.VacationStatusPicker", ref: VacationStatusPicker },
    { key: "vacation.route.vacations", ref: ROUTE_VACATIONS },
  ],
  // Contribution: mount routes into core's router
  "core.Router": [
    { path: ROUTE_VACATIONS, component: VacationsPage },
    { path: ROUTE_VACATION + "/*:vacation_uuid?", component: VacationPage },
  ],
  // Contribution: add a top-level menu entry, right-gated
  "core.MainMenu": [
    {
      text: <FormattedMessage module="vacation" id="menu.vacations" />,
      icon: <BeachAccessIcon />,
      route: "/" + ROUTE_VACATIONS,
      filter: (rights) => rights.includes(RIGHT_VACATION),
    },
  ],
};

export function VacationModule(cfg) {
  return { ...DEFAULT_CONFIG, ...cfg };
}
```

2.3 Actions using `graphqlWithVariables` (not Apollo)

Data flows through the core Redux GraphQL layer — no Apollo. See [Frontend Architecture](#) for the full model.

```
// openimis-fe-vacation_js/src/actions.js (illustrative / simplified)
import { graphqlWithVariables, formatPageQuery } from "@openimis/fe-core";

export function fetchVacations(modulesManager, filters) {
  const projection = ["id", "status", "startDate", "endDate",
    "insuree{uuid, chfId, lastName}"];
  const payload = formatPageQuery("vacations", filters, projection);
  return graphqlWithVariables(payload, {}, "VACATION_VACATIONS");
}

export function createVacation(vacation, clientMutationLabel) {
  const mutation = `mutation ($input: CreateVacationMutationInput!) {
    createVacation(input: $input) { clientMutationId internalId }
  }`;
  return graphqlWithVariables(
    mutation, { input: vacation }, "VACATION_MUTATION",
    { clientMutationLabel }, // journalize tracks + polls this mutation
  );
}
```

The reducer handles `VACATION_VACATIONS_REQ/RESP/ERR` and writes results into `state.vacation`.

2.4 Components using `ModulesManager` and published components

Reuse core/other-module components **by key**, never by direct import.

```
// openimis-fe-vacation_js/src/components/VacationForm.js (illustrative)
import React from "react";
export default function VacationForm({ modulesManager, edited, onChange }) {
  const InsureePicker = modulesManager.getRef("insuree.InsureePicker");
  const readOnly = modulesManager.getConf("fe-vacation", "form.readOnly",
    false);
  return (
    <>
      <InsureePicker
        value={edited?.insuree}
        readOnly={readOnly}
        onChange={({insuree}) => onChange({ ...edited, insuree })}
      />
      { /* date inputs, status picker via
getRef("vacation.VacationStatusPicker") */ }
    </>
  );
}
```

2.5 Register in the frontend `openimis.json` and link

```
// openimis-fe_js/openimis.json (add an entry)
{ "name": "@openimis/fe-vacation", "npm": "@openimis/fe-vacation@^1.0.0" }
```

For development, link your local checkout so edits are live, then let the generator rebuild

`src/modules.js`:

```
# in openimis-fe-vacation_js
npm link
# in openimis-fe_js
npm link @openimis/fe-vacation
npm start # runs the Vite dev server; generator regenerates src/modules.js
```

Common mistake

After adding the manifest entry, if the menu does not appear, check three things in order: (1) is the package linked/installed? (2) did the generator include it in `src/modules.js`? (3) does the logged-in user have the integer right your `filter` checks? Nine times out of ten it is one of these, not your component.

1. `openimis-be-vacation_py` with the standard layout.
2. `apps.py`: `AppConfig`, `DEFAULT_CFG`, unique integer perms, `ready()` overlay.
3. `models.py`: subclass `HistoryModel`, use `json_ext`, `tbl`-style table.
4. `services.py`: business logic + `register_service_signal`.
5. `schema.py` / `gql_mutations/`: `Query` + `OpenIMISMutation` subclasses, `has_perms` checks.
6. `makemigrations` / `migrate`.
7. Add to backend `openimis.json`; `pip install -e`; verify in GraphiQL.
1. `openimis-fe-vacation_js` with the standard layout.
2. `src/index.js`: the config object (reducers, routes, menus, refs, translations).
3. `actions.js`: `graphqlWithVariables` — never Apollo.
4. Components: `getRef` / `getConf` / `getContribs` — never direct imports.
5. Add to frontend `openimis.json`; `npm link`; verify the menu/route render.

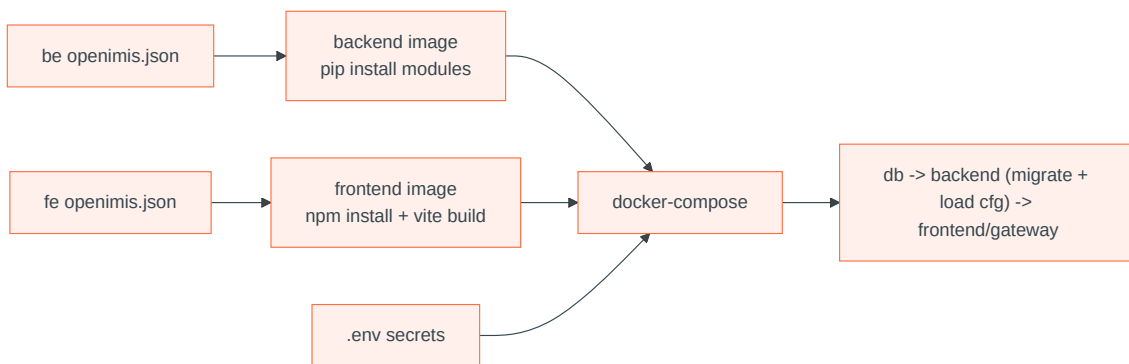
Backend recap

Frontend recap

3. Docker support

You rarely change images to add a module — the images build **from the manifests**. The distribution repo `openimis-dist_dkr` builds a backend image that `pip install`s from the backend `openimis.json`, and a frontend image that `npm-install`s from the frontend `openimis.json` and runs the Vite build. So "Docker support" for your module means:

1. Your module is listed in both `openimis.json` manifests with a resolvable source (git/PyPI, npm).
2. Any new **env vars / secrets** your module needs are added to `.env` and referenced in `docker-compose`.
3. Any startup data load (fixtures, default `ModuleConfiguration`) is wired into the backend startup `script/`.



See [Docker & Deployment](#) for the full compose topology.

4. Packaging and publishing

```
# openimis-be-vacation_py/setup.py (illustrative)
from setuptools import setup, find_packages

setup(
    name="openimis-be-vacation",
    version="1.0.0",
    packages=find_packages(),
    install_requires=["openimis-be-core", "openimis-be-insuree"],
    include_package_data=True,
)
```

Publish by tagging a semantic version and either pushing to PyPI or referencing the git tag directly from the backend `openimis.json` (`git+https://...@v1.0.0`). **Pin the version** in the manifest — the assembly is a lockfile of module versions.

```
// openimis-fe-vacation_js/package.json (illustrative)
{
  "name": "@openimis/fe-vacation",
  "version": "1.0.0",
  "main": "src/index.js",
  "peerDependencies": {
    "@openimis/fe-core": "^1.9.0",
    "react": "^18", "redux": "^4", "@mui/material": "^5"
  }
}
```

Declare shared libraries (React, MUI, Redux, `@openimis/fe-core`) as **peerDependencies** so the assembly provides one shared copy — never bundle your own React (that causes the "invalid hook call" crash). Publish to npm, then pin the version in the frontend `openimis.json`.

Backend (setup.py → PyPI/git)

Frontend (package.json → npm)

Did you know?

Both manifests are effectively **lockfiles**: they pin every module to an exact/compatible version. A reproducible openIMIS deployment is fully described by the two `openimis.json` files plus `.env`. See [Developer Workflow](#) for the release/versioning discipline.

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be- <name>_py</code>	<code><name>/apps.py</code>	<code>AppConfig</code> , <code>DEFAULT_CFG</code> , integer permission codes, <code>ready()</code> config overlay.
<code>openimis-be- <name>_py</code>	<code><name>/models.py</code>	Models on <code>HistoryModel</code> / <code>json_ext</code> ; legacy-style table names.
<code>openimis-be- <name>_py</code>	<code><name>/services.py</code>	Business logic; <code>register_service_signal</code> seams.
<code>openimis-be- <name>_py</code>	<code><name>/schema.py</code> , <code>gql_mutations/</code>	<code>Query</code> + <code>OpenIMISMutation</code> subclasses; <code>has_perms</code> .
<code>openimis-be_py</code>	<code>openimis.json</code>	Register the backend module + pin its version.
<code>openimis-be_py</code>	<code>openimis/schema.py</code>	Combines every module's <code>Query</code> / <code>Mutation</code> into the root schema.
<code>openimis-fe- <name>_js</code>	<code>src/index.js</code>	The config object — the module's entire public API.
<code>openimis-fe- <name>_js</code>	<code>src/actions.js</code>	<code>graphqlWithVariables</code> data flow (not Apollo).
<code>openimis-fe_js</code>	<code>openimis.json</code> , <code>openimis-config- vite.js</code>	Register the frontend module; the generator that builds <code>src/modules.js</code> .
<code>openimis- dist_dkr</code>	<code>docker-compose.yml</code> , <code>.env</code>	Images build from the manifests; add secrets/config here.

Hands-on lab

Lab 15.1 — Ship a backend module end to end

1. Scaffold `openimis-be-vacation_py` with the layout above.
2. Write `apps.py`, `models.py`, `services.py`, `schema.py`, and `gql_mutations/create_vacation.py`.
3. Add the module to the backend `openimis.json`, `pip install -e`, and `makemigrations / migrate`.
4. In GraphQL (`/graphql`), run the `vacations` query and the `createVacation` mutation. Confirm a `MutationLog` row appears and resolves.

Lab 15.2 — Add the frontend and see it in the app

1. Scaffold `openimis-fe-vacation_js`; write the config object, reducer, one action, one page, one menu.
2. Add it to the frontend `openimis.json`, `npm link`, and start the dev server.
3. Grant your dev user the integer right your menu `filter` checks.
4. Navigate to the new menu, open the searcher, and create a vacation — watch the mutation register in the journal (Redux DevTools) and the new row appear via the backend.

Exercises

1. You need a per-deployment tunable "max vacation days." Where does its default live, and how does an operator override it without code changes?
2. A reporting module wants to run logic after every vacation is created, without importing your module. What do you both do?
3. Explain why React must be a `peerDependency` of your frontend module.
4. Give the exact two files that must both list your module for a Docker build to include it.

Knowledge check

? Q1: Where do you put business logic — resolver, mutation, or service — and why? (click for answer) ✓

In the **service**. Resolvers/mutations are thin: they check `has_perms` and delegate. Services are reusable (other modules, tests, signals can call them), keep GraphQL concerns out of domain logic, and are where you `register_service_signal` for extensibility.

? Q2: What makes a mutation asynchronous and audited in openIMIS? (click for answer) ✓

Subclassing core's `OpenIMISMutation`. It creates a `MutationLog`, runs your `async_mutate`, and returns a `clientMutationId`; success is `return None`, errors are returned as a list. The frontend polls the `MutationLog` status via `journalize`.

? Q3: Your new frontend menu does not appear. Name the three things to check, in order. (click for answer) ✓

(1) Is the package installed/linked and listed in the frontend `openimis.json` so the generator put it in `src/modules.js`? (2) Did the module load (check `ModulesManager` / no console errors)? (3) Does the logged-in user hold the integer **right** the menu `filter` requires?

? Q4: Why must permission codes be globally unique integers, and where are they defined? (click for answer) ✓

They are defined in each module's `apps.py` (`gql_..._perms`). If two modules reuse a code, they unintentionally share access — a security bug. Pick an unused integer block and document it.

? Q5: What two files fully describe a reproducible openIMIS deployment's module set? (click for answer) ✓

The backend `openimis.json` (in `openimis-be_py`) and the frontend `openimis.json` (in `openimis-fe_js`) — each pins module versions like a lockfile. Add `.env` for secrets and you can rebuild the exact deployment.

Further reading

- [End-to-End Code Walkthrough](#) — trace a real feature through both halves you just learned to build.
- [Developer Workflow](#) — the setup, test, and release loop for module authors.
- [Plugin / Module System](#) and [Frontend Architecture](#) — the seams your module plugs into.
- [openimis-be-core_py](#) and [openimis-fe-core_js](#) — read the base classes you subclass.
- [Best Practices](#) — conventions reviewers will hold you to.

End-to-End Code Walkthrough

Part 17 — The Capstone: One Feature, Every Layer

This chapter is the payoff for everything else in the handbook. We pick **one concrete user action — a health facility clerk submitting a claim** — and follow it from the pixel they click all the way to a PostgreSQL row and back to the "Claim submitted" toast. At every hop we name the **real repository, directory, and file**. If you have read [Frontend Architecture](#), [The GraphQL Layer](#), and [Core module](#), this is where those threads tie together.

Learning objectives

By the end of this chapter you will be able to:

- Trace a claim submission through **every layer**: React form → Redux GraphQL action → HTTP `/graphql` → Graphene view → assembled schema → `OpenIMISMutation` → service → ORM → PostgreSQL → `MutationLog` /signals → response → frontend polling → UI update.
- Point to the **specific files** at each hop across `openimis-fe-claim_js`, `openimis-fe-core_js`, `openimis-be_py`, `openimis-be-claim_py`, and `openimis-be-core_py`.
- Explain *why* the response is asynchronous and how the frontend knows when the work is really done.

Prerequisites

- [Frontend Architecture](#) — the config object, the Redux GraphQL layer, `journalize` /polling.
- [The GraphQL Layer](#) — queries, mutations, the schema assembly.
- [Core module](#) — `OpenIMISMutation`, `MutationLog`, service signals, base models.
- [Claim module](#) — the domain we are tracing.
- [Request Lifecycle](#) — the backend request path in general.

0. The scenario

A clerk at a health facility opens a claim, adds diagnoses and service/item lines for an insuree, and clicks **Submit**. The claim must be created (or updated) and moved into a submitted state

so it can later be reviewed and valued. Everything below is that one click.

The hops, at a glance:

#	Layer	Repository	Key file(s)
1	Claim form / component	openimis-fe- claim_js	src/components/ClaimForm.js , pages/ClaimEditPage.js
2	Redux action creator	openimis-fe- claim_js	src/actions.js
3	GraphQL Redux layer	openimis-fe- core_js	src/helpers/api (graphqlWithVariables)
4	HTTP transport	(browser) →	POST /graphql with JWT cookie
5	GraphQL endpoint	openimis-be_py	openimis/urls.py , openimis/schema.py
6	Assembled schema	openimis-be_py	openimis/schema.py (combines module Mutation s)
7	Mutation class	openimis-be- claim_py	claim/schema.py , claim/gql_mutations.py
8	Async base + audit	openimis-be- core_py	core/schema.py (OpenIMISMutation), core/models.py (MutationLog)
9	Service (business logic)	openimis-be- claim_py	claim/services.py
10	ORM / models	openimis-be- claim_py	claim/models.py (Claim , ClaimItem , ClaimService)
11	Signals	openimis-be- core_py / openimis- be-claim_py	core/signals.py , claim/signals.py

#	Layer	Repository	Key file(s)
12	Database	PostgreSQL	<code>tblClaim</code> , <code>tblClaimItems</code> , <code>tblClaimServices</code> , <code>core_Mutation*</code>
13	Poll + UI update	<code>openimis-fe-</code> <code>core_js</code> + <code>openimis-fe-</code> <code>claim_js</code>	journalize helpers, <code>src/reducer.js</code>

1. The frontend form (`openimis-fe-claim_js`)

The clerk is looking at a claim edit page. The route was contributed by the claim module's config object (`src/index.js`) into core's router (see [Frontend Architecture](#)); the page renders a `ClaimForm` component in `openimis-fe-claim_js/src/components/`. The form is a controlled React component: it holds the in-progress claim (`edited`) in state, reuses **published components** from other modules via `ModulesManager` (e.g. `insuree.InsureePicker`, `medical.DiagnosisPicker`, item/service pickers), and localizes every label with `react-intl`.

```
// openimis-fe-claim_js/src/components/ClaimForm.js (illustrative /
simplified)
function ClaimForm({ modulesManager, edited, onChange }) {
  const InsureePicker = modulesManager.getRef("insuree.InsureePicker");
  const DiagnosisPicker = modulesManager.getRef("medical.DiagnosisPicker");
  // ... item lines, service lines, dates, health facility ...
  return (/* MUI inputs bound to `edited`, calling onChange */);
}
```

When the clerk clicks **Submit**, the page dispatches a Redux action — it does **not** call `fetch` or Apollo directly.

```
// openimis-fe-claim_js/src/pages/ClaimEditPage.js (illustrative)
onSubmit = () => {
  this.props.submit(this.state.claim, this.clientMutationLabel());
};
// mapDispatchToProps binds `submit` to the action creator below.
```

2. The Redux action creator (`openimis-fe-claim_js/src/actions.js`)

The claim module's `actions.js` builds the GraphQL mutation document and hands it to the core helper `graphqlWithVariables`. Note the payload contains only a **client mutation id / ticket request** — not a promise of the finished claim.

```
// openimis-fe-claim_js/src/actions.js (illustrative / simplified)
import { graphqlWithVariables } from "@openimis/fe-core";

export function submit(claim, clientMutationLabel) {
  const mutation = `
    mutation ($input: SubmitClaimsMutationInput!) {
      submitClaims(input: $input) { clientMutationId internalId }
    }`;
  const variables = { input: { uuids: [claim.uuid] } };
  return graphqlWithVariables(
    mutation, variables, "CLAIM_MUTATION",
    { clientMutationLabel, mutationType: "SUBMIT" }, // for the journal
  );
}
```

3. The custom GraphQL Redux layer (`openimis-fe-core_js`)

`graphqlWithVariables` lives in the frontend **core** module (`openimis-fe-core_js/src/helpers/api`). It:

1. dispatches a `CLAIM_MUTATION_REQ` action (so the UI can show "submitting"),
2. issues `POST /graphql` with the query + variables — the **JWT is sent automatically** because it is an `HttpOnly` cookie,
3. dispatches `CLAIM_MUTATION_RESP` (or `_ERR`) with the immediate response,
4. **registers the mutation in the journal** (using `clientMutationId / internalId`) so the poller will chase its `MutationLog` status.

There is no Apollo cache here — the response is Redux state, and the *real* completion arrives later, via polling (hop 13).

4–5. HTTP `/graphql` and the endpoint (`openimis-be_py`)

The browser sends one `POST /graphql`. On the server, `openimis-be_py` is the **assembly project**. `openimis/urls.py` routes `/graphql` to Graphene's `GraphQLView` (wrapped with

CSRF/JWT middleware). `django-graphql-jwt` reads the JWT from the cookie and attaches the authenticated `user` to `info.context`.

```
# openimis-be_py/openimis/urls.py (illustrative / simplified)
from graphene_django.views import GraphQLView
urlpatterns = [
    path("graphql", csrf_exempt(GraphQLView.as_view(graphiql=True)),
        name="graphql"),
    # ... each module's urls.py collected here ...
]
```

6. The assembled schema (`openimis-be_py/openimis/schema.py`)

Graphene needs one root `Mutation`. `openIMIS` builds it by importing every module's `schema.Mutation` and combining them with **Python multiple inheritance** — so `submitClaims` (defined in the claim module) becomes a field on the single root mutation.

```
# openimis-be_py/openimis/schema.py (illustrative / simplified)
import claim.schema, insuree.schema, policy.schema, core.schema

class Mutation(
    claim.schema.Mutation,
    insuree.schema.Mutation,
    policy.schema.Mutation,
    core.schema.Mutation,
    graphene.ObjectType,
):
    pass

schema = graphene.Schema(query=Query, mutation=Mutation)
```

Graphene resolves `submitClaims` to the `Field` the claim module registered.

7. The mutation class (`openimis-be-claim_py`)

In `openimis-be-claim_py/claim/schema.py`, the `Mutation` class exposes `submit_claims = SubmitClaimsMutation.Field()`. The mutation itself (in `claim/schema.py` or `claim/gql_mutations.py`) subclasses core's **`OpenIMISMutation`**, so it inherits the audited, asynchronous machinery.

```
# openimis-be-claim_py/claim/schema.py (illustrative / simplified)
from core.schema import OpenIMISMutation
from .apps import ClaimConfig
from .services import submit_claim
from .models import Claim

class SubmitClaimsMutation(OpenIMISMutation):
    _mutation_module = "claim"
    _mutation_class = "SubmitClaimsMutation"

    class Input(OpenIMISMutation.Input):
        uuids = graphene.List(graphene.String)

    @classmethod
    def async_mutate(cls, user, **data):
        errors = []
        if not user.has_perms(ClaimConfig.gql_mutation_submit_claims_perms):
            raise PermissionDenied("unauthorized")
        for uuid in data["uuids"]:
            claim = Claim.objects.get(uuid=uuid)
            errors += submit_claim(user, claim) # delegate to the service
        return errors # [] == success; non-empty == validation errors
```

8. The async base and audit (openimis-be-core_py)

OpenIMISMutation (in openimis-be-core_py/core/schema.py) is the linchpin. Its mutate (inherited by every mutation) does, roughly:

1. create a **MutationLog** row (core/models.py) with status *received/ongoing*, capturing the user, module, mutation class, and input,
2. call the subclass's `async_mutate(...)`,
3. if it returns an empty list, mark the MutationLog **success**; if it returns errors (or raises), mark it **failed** and store the error detail,
4. return { `clientMutationId`, `internalId` } to the caller **immediately** — the client got a *ticket*, not the finished claim.


```
# openimis-be-core_py/core/schema.py (illustrative / simplified)
class OpenIMISMutation(graphene.relay.ClientIDMutation):
    internal_id = graphene.Field(graphene.String)

    @classmethod
    def mutate_and_get_payload(cls, root, info, **data):
        mutation_log = MutationLog.objects.create(
            client_mutation_id=data.get("client_mutation_id"),
            json_content=data, user=info.context.user,
            module=cls._mutation_module, mutation_class=cls._mutation_class,
        )
        try:
            errors = cls.async_mutate(info.context.user, **data)
            mutation_log.mark_error(errors) if errors else
        mutation_log.mark_success()
        except Exception as exc:
            mutation_log.mark_error([{"message": str(exc)}])
        return cls(internal_id=mutation_log.id,
                    client_mutation_id=data.get("client_mutation_id"))
```

Did you know?

Whether the work runs inline or on a background worker, the **contract to the client is the same**: you get a `clientMutationId`, and the truth of what happened lives in `MutationLog`. That indirection is what makes every state-changing operation in openIMIS auditable — a hard requirement for a system moving public health-insurance money.

9. The service (`openimis-be-claim_py/claim/services.py`)

The mutation delegates to `submit_claim` in `claim/services.py`. **This is where the real domain logic lives**: validating the claim, checking the insuree's policy coverage window, transitioning the claim `status` to `submitted`, and persisting. The service emits a **service signal** so other modules (valuation via `calculation`, reporting, etc.) can react.

```
# openimis-be-claim_py/claim/services.py (illustrative / simplified)
from core.signals import register_service_signal
from .models import Claim

@register_service_signal("claim_service.submit")
def submit_claim(user, claim):
    errors = validate_claim(claim) # dates, lines, coverage, duplicates
    if errors:
        return errors
    claim.status = Claim.STATUS_SUBMITTED
    claim.submit_stamp = now()
    claim.save(username=user.username) # HistoryModel records who + when
    return []
```

10. The ORM / models (`openimis-be-claim_py/claim/models.py`)

The claim aggregate spans several tables. `Claim` (with its `ClaimItem` and `ClaimService` lines) subclasses core base models, inheriting temporal validity, UUIDs, `legacy_id`, and `json_ext`.

```
# openimis-be-claim_py/claim/models.py (illustrative / simplified)
from core.models import VersionedModel
from insuree.models import Insuree
from location.models import HealthFacility

class Claim(VersionedModel):
    STATUS_ENTERED, STATUS_SUBMITTED = 2, 4
    insuree = models.ForeignKey(Insuree, db_column="InsureeID", ...)
    health_facility = models.ForeignKey(HealthFacility, db_column="HFID", ...)
    status = models.SmallIntegerField(db_column="ClaimStatus")
    # ... claimed/approved amounts, dates ...
    class Meta:
        db_table = "tblClaim"
```

`claim.save(...)` issues `UPDATE tblClaim ...` (and inserts/updates the item/service line tables). Because `Claim` is versioned, the write respects the `validity_from/validity_to` temporal pattern rather than destructively overwriting history.

11–12. Signals and the database

Two kinds of signals fire:

- The **service signal** `claim_service.submit` — any module that called `bind_service_signal("claim_service.submit", handler, after=True)` now runs (e.g. queue the claim for valuation, emit an audit event, notify).
- Standard **Django ORM signals** (`post_save` on `Claim`) that modules or core may listen to.

PostgreSQL now holds: the updated `tblClaim` row (status = submitted), any `tblClaimItems` / `tblClaimServices` rows, and a `MutationLog` row (in the `core` mutation tables) marked success or error. The audit trail is complete.

13. The response, polling, and the UI update

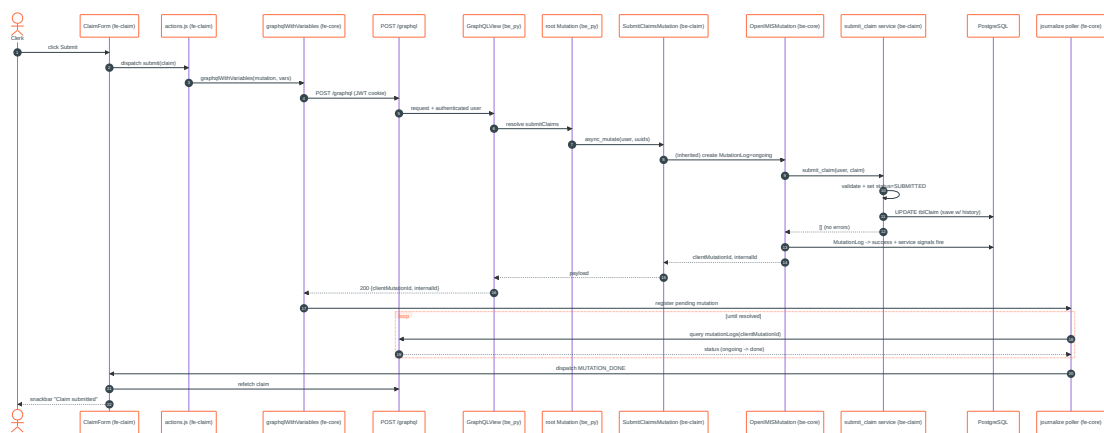
The immediate HTTP response carried only `{ clientMutationId, internalId }`. So how does the clerk's screen learn the claim was accepted?

The frontend **journal + poller** (in `openimis-fe-core_js`) periodically queries the backend for the `MutationLog` status of the registered `clientMutationId`:

```
query ($clientMutationId: String!) {  
  mutationLogs(clientMutationId: $clientMutationId) {  
    edges { node { status clientMutationId error } }  
  }  
}
```

When `status` flips to **done** (or **error**), the poller dispatches a completion action. The claim module reacts: it refetches the affected claim(s), and shows a success snackbar (or surfaces the validation errors from `MutationLog.error`). Only now does the clerk see "Claim submitted."

The whole journey, as a sequence diagram



Common mistake

Reading the immediate response as "the claim is submitted." It is **not** — it only means the mutation was *accepted*. Validation (dates, coverage, duplicate lines) happens in the service and its outcome lands in `MutationLog`. A UI that skips the poll will show false success and hide real validation errors. This is the single most common misunderstanding when moving from Apollo-style mutations to openIMIS.



Deep dive: where valuation happens



Submission does not *price* the claim. In openIMIS, valuation (what the scheme will pay) is driven by the **calculation rule** framework (`openimis-be-calculation_py` + `calcrule_*` modules), typically triggered downstream of submission via service signals and later review states. Our walkthrough stops at *submitted*; follow the `claim_service.submit` signal bindings and the [Calculation Rules](#) chapter to trace pricing.

Repository references

Repository	Directory / File	Why it matters
openimis-fe-claim_js	src/components/ClaimForm.js , pages/ClaimEditPage.js	The form the clerk uses; reuses published pickers via ModulesManager .
openimis-fe-claim_js	src/actions.js , src/reducer.js	Builds the mutation; handles the response/completion in Redux state.
openimis-fe-core_js	src/helpers/api (graphqlWithVariables), journalize helpers	The custom GraphQL layer + mutation polling.
openimis-be_py	openimis/urls.py	Routes /graphql to Graphene with JWT middleware.
openimis-be_py	openimis/schema.py	Combines each module's Query / Mutation into the root schema.
openimis-be-claim_py	claim/schema.py , claim/gql_mutations.py	SubmitClaimsMutation (an OpenIMISMutation); has_perms check.
openimis-be-claim_py	claim/services.py	submit_claim — validation + status transition; service signal.
openimis-be-claim_py	claim/models.py	Claim, ClaimItem, ClaimService on core base models.
openimis-be-core_py	core/schema.py (OpenIMISMutation), core/models.py (MutationLog)	The async, audited mutation base and its log.
openimis-be-core_py	core/signals.py	register_service_signal / bind_service_signal — the extension seam.

Hands-on lab

Lab 17.1 — Instrument the whole path yourself

1. Open Redux DevTools and submit a claim. Capture the `_REQ` and `_RESP` actions and the `internalId` in the response.
2. In GraphQL (`/graphql`), query `mutationLogs(clientMutationId: "...")` and watch the status move to done.
3. In a Django shell (`python manage.py shell`), fetch the `Claim` by uuid and confirm `status` is submitted and the history reflects the save.
4. Add a `bind_service_signal("claim_service.submit", ...)` handler in a scratch app that prints the claim id, resubmit, and confirm it fires — you have just extended the pipeline without touching the claim module.

Exercises

1. The clerk clicked Submit but nothing updates for a few seconds. Which component is responsible for eventually updating the UI, and how does it know the work finished?
2. A claim fails validation. Trace exactly where the error message is produced and where the frontend reads it.
3. Explain why `submitClaims` can be defined in `openimis-be-claim_py` yet still be a field on `openimis-be_py`'s root mutation.
4. Which single class turns an ordinary mutation into an audited, asynchronous one, and which repo defines it?

Knowledge check

 **Q1: The immediate GraphQL response to a submit contains only `clientMutationId/internalId`. Why, and where is the real outcome? (click for answer)** 

Because openIMIS mutations are **asynchronous and audited** via `OpenIMISMutation`: they create a `MutationLog`, run the service, and return a ticket. The real outcome (success or validation errors) is stored on the `MutationLog` row and read by the frontend poller.

? **Q2: How does `submitClaims`, defined in the claim module, become callable on the assembled schema? (click for answer)** ✓

`openimis-be_py/openimis/schema.py` combines every module's `schema.Mutation` via Python **multiple inheritance** into one root `Mutation`, so the claim module's `Field` is exposed on the single schema Graphene serves at `/graphql`.

? **Q3: Which file holds the claim submission business logic, and why not the mutation class? (click for answer)** ✓

`openimis-be-claim_py/claim/services.py` (`submit_claim`). Logic lives in services so it is reusable (tests, other modules, signals), keeps GraphQL out of the domain, and can advertise a service signal for extension.

? **Q4: How is the JWT transmitted on the submit request, and why is that secure? (click for answer)** ✓

It rides an `HttpOnly` cookie the browser attaches automatically to `POST /graphql`. JavaScript cannot read it, so an XSS bug cannot exfiltrate the token; the backend `django-graphql-jwt` middleware validates it and sets `info.context.user`.

? **Q5: Name two things that must be true in PostgreSQL after a successful submission. (click for answer)** ✓

The `tblClaim` row has `status = submitted` (with history preserved), and a `MutationLog` row for that `clientMutationId` is marked success (error empty). Item/service line tables reflect the claim's lines.

Further reading

- [Claim module](#) — the domain model behind this walkthrough.
- [Core module](#) — `OpenIMISMutation`, `MutationLog`, service signals in depth.
- [The GraphQL Layer](#) and [Frontend Architecture](#) — the two sides of the wire.
- [Extension Guide](#) — build your own module using the exact patterns traced here.
- [openimis-be-claim_py](#) and [openimis-fe-claim_js](#) — read the real source for each hop.

Developer Workflow

Part 16 — The Daily Loop: Setup, Debug, Test, Release

Building openIMIS features is not just writing code — it is operating a **multi-repository, modular system** with a disciplined edit → run → debug → test → release loop. This chapter is the practical playbook: how to wire up the assemblies with editable module installs, run the stack in Docker or natively, debug across the frontend/backend seam, test each module, keep the code clean, and cut a release that pins module versions. It is the connective tissue between [Set Up a Dev Environment](#) and [Best Practices](#).

Learning objectives

By the end of this chapter you will be able to:

- Clone and link the three assembly repos (`openimis-be_py` , `openimis-fe_js` , `openimis-dist_dkr`) with **editable module installs**.
- Run the inner development loop natively and via Docker.
- Debug across layers: backend logs, the Django shell, GraphQL at `/graphql` , and Redux DevTools.
- Run and write tests (pytest/Django on the backend, Jest on the frontend) and apply the linters/formatters (black, flake8, isort; eslint, prettier).
- Understand CI/CD (GitHub Actions), semantic **per-module versioning**, the assembly's manifest as a version lockfile, and the PR/contribution workflow in the openIMIS org.

Prerequisites

- [Set Up a Dev Environment](#) — you have the stack running once.
 - [Plugin / Module System](#) and [Frontend Architecture](#) — you know what a module is on each side.
 - [Extension Guide](#) — helpful if you are authoring a module while you read this.
 - [Best Practices](#) — the conventions your PRs will be held to.
-

1. Repository setup: three assemblies + editable modules

openIMIS is not one repo. You work across three **assemblies** and however many **module** repos your task touches.

Repo	Role
<code>openimis-be_py</code>	Backend Django project. Assembles modules via <code>openimis.json</code> .
<code>openimis-fe_js</code>	Frontend React SPA. Assembles modules via <code>openimis.json</code> + <code>openimis-config-vite.js</code> .
<code>openimis-dist_dkr</code>	Docker distribution. <code>docker-compose</code> topology, <code>.env</code> .
<code>openimis-be- <name>_py</code>	Backend modules (core, claim, insuree, ...).
<code>openimis-fe- <name>_js</code>	Frontend modules.

The trick that makes day-to-day work bearable is **editable installs / links**: instead of the manifest's pinned package, you point the assembly at a local checkout so your edits are live without reinstalling.

```
# sibling checkouts
git clone https://github.com/openimis/openimis-be_py.git
git clone https://github.com/openimis/openimis-be-core_py.git
git clone https://github.com/openimis/openimis-be-claim_py.git

cd openimis-be_py
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# override manifest-pinned packages with local, editable checkouts
pip install -e ../openimis-be-core_py/
pip install -e ../openimis-be-claim_py/

python manage.py migrate
python manage.py runserver
```

```
git clone https://github.com/openimis/openimis-fe_js.git
git clone https://github.com/openimis/openimis-fe-claim_js.git

cd openimis-fe-claim_js && npm install && npm link
cd ../openimis-fe_js && npm install
npm link @openimis/fe-claim # override the published package

npm start # Vite dev server; regenerates src/modules.js from openimis.json
```

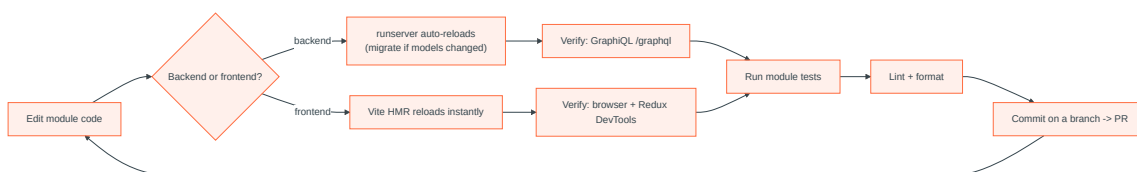
Backend (editable pip install)

Frontend (npm link)

⚡ Common mistake

Forgetting that the manifest still lists the **published** version. `pip install -e / npm link` override it for *your* machine, but the source of truth for a build remains `openimis.json`. If you add a brand-new module, you must add it to `openimis.json` too — the editable install alone will not put it into a Docker build or a teammate's checkout.

2. The inner development loop



- Change **backend models** → `makemigrations <module> + migrate` before the change takes effect.
- Change **backend logic** → `runserver` auto-reloads.
- Change **frontend** → Vite hot-reloads; watch Redux DevTools to confirm the action/state you expect.

3. Running via Docker

For an integration-faithful environment (matching production topology), use `openimis-dist_dkr`.

```
cd openimis-dist_dkr
cp .env.example .env      # fill in secrets/config
docker compose up
```

Startup order is meaningful: **db (PostgreSQL)** → **backend (runs migrations + loads module configurations)** → **frontend / gateway**. A reverse-proxy/gateway (Nginx) ties the frontend, `/api`, and `/graphql` together on one origin. See [Docker & Deployment](#) for the full topology (including optional OpenSearch for `opensearch_reports`).

Did you know?

The Docker images build **from the manifests**: the backend image `pip install s` from the backend `openimis.json`; the frontend image `npm-installs` and runs the Vite build from the frontend `openimis.json`. So a reproducible deployment is fully described by the two manifests plus `.env`.

Native for speed, Docker for fidelity

Iterate on a single module natively (fast reload, easy debugger). Switch to Docker to validate cross-service behavior, the gateway routing, and the startup/migration/config-load sequence before you open a PR.

4. Debugging across the stack

Symptom	Where to look	How
Mutation "succeeded" but nothing changed	Backend <code>MutationLog</code>	Query <code>mutationLogs(clientMutationId : ...)</code> in GraphQL; check status + error.
Query returns wrong/empty data	GraphQL at <code>/graphql</code>	Run the query directly; toggle fields to isolate.
Business logic bug	Django shell	<code>python manage.py shell</code> ; call the service directly with a test user.
403 / permission denied	<code>apps.py</code> perms vs. user rights	Compare the integer codes in <code>has_perms</code> to the user's rights.
Frontend not updating	Redux DevTools	Inspect dispatched actions and the module's state slice.
Wrong/undefined config value	Backend <code>ModuleConfiguration</code> / FE <code>getConfig</code>	Confirm the DB config overlay and the FE build-time config.

Backend

- **Logs:** Django + Gunicorn logs (stdout in Docker: `docker compose logs -f backend`).
- **Django shell:** `python manage.py shell` — instantiate a service with a user and call it; the fastest way to reproduce a bug without the UI.
- **GraphQL / GraphQL playground:** served at `/graphql` — an interactive schema explorer and query runner. Your best friend for reproducing exactly what the frontend sends.

Frontend

- **Browser devtools** for the network `POST /graphql` payloads and responses.
- **Redux DevTools** to watch `_REQ` / `_RESP` / completion actions and to time-travel through state — because openIMIS uses Redux (not Apollo), *all* server data is visible as plain Redux state.



Common mistake

Debugging a "failed" mutation only in the browser. The immediate response is just a ticket (`clientMutationId`); the real error is on the `MutationLog`. Always confirm the mutation's status via GraphQL when a submission behaves oddly. (See the [End-to-End Walkthrough](#).)

5. Testing

Each backend module ships tests under `<name>/tests/`. Run with Django's test runner or `pytest`.

```
# all tests for one module
python manage.py test claim
# or with pytest (if configured)
pytest claim/tests/ -v
```

Test the **services** directly (they hold the logic), plus schema-level tests that exercise resolvers/mutations with a permissioned user. Because modules subclass core base models, use the core test helpers/fixtures for users and validity windows.

Frontend modules use **Jest** (with React Testing Library) for component and reducer tests.

```
npm test          # watch mode
npm test -- --ci  # single run for CI
```

Test reducers as pure functions (dispatch an action shape, assert the new slice) and components in isolation, mocking `modulesManager.getRef / getContribs`.

Backend

Frontend

6. Linting and formatting

Tool	Purpose
black	Opinionated code formatting.
isort	Import ordering.
flake8	Lint / style errors.

```
black . && isort . && flake8
```



Tool	Purpose
eslint	Lint (rules incl. React hooks).
prettier	Formatting.

```
npm run lint && npm run format
```



Backend

Frontend

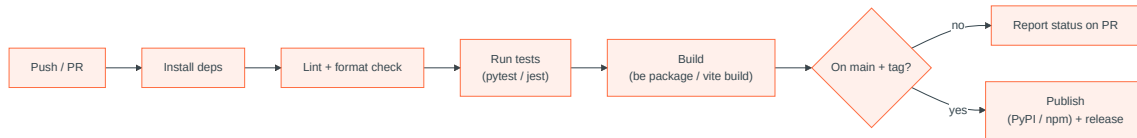


Automate it

Wire these into a pre-commit hook (backend) and lint-staged (frontend) so CI never fails on formatting. Reviewers should be reading logic, not whitespace.

7. CI/CD

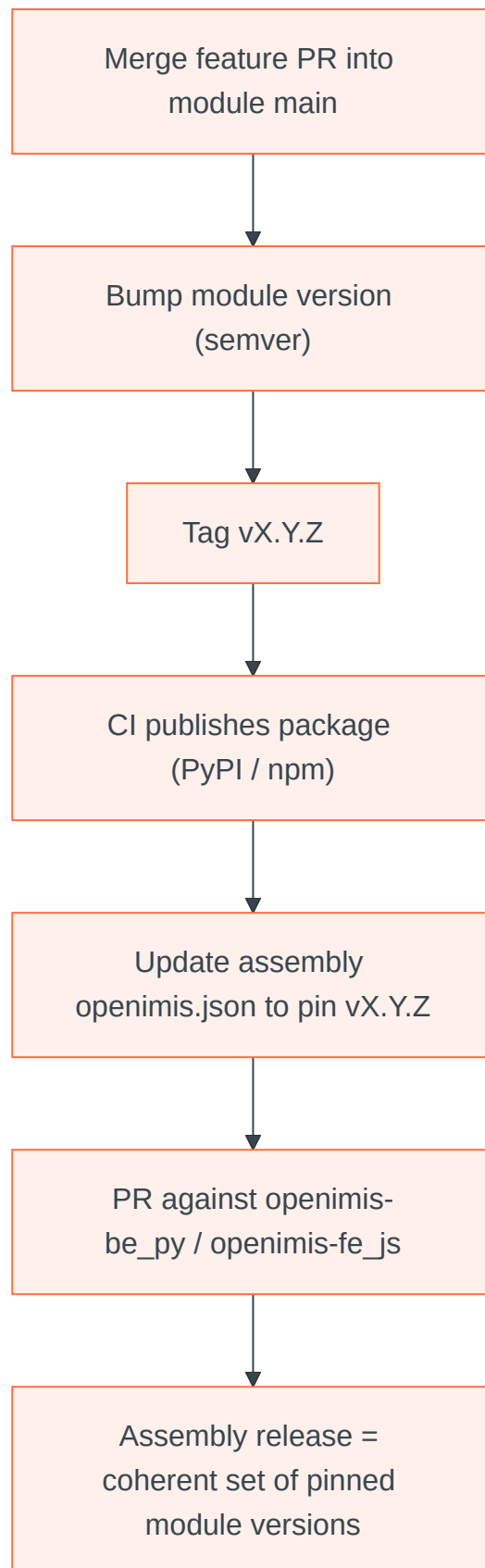
Each module repo and each assembly has **GitHub Actions** workflows (under `.github/workflows/`) that run on push/PR. The typical pipeline:



- **Backend** workflows spin up PostgreSQL (a service container), run migrations, and execute the module's tests.
 - **Frontend** workflows run eslint + Jest and a production Vite build.
 - **Publish** stages (on a tagged release) push the package to PyPI/npm so an assembly can pin the new version.
-

8. Release and versioning

openIMIS versions **per module** with semantic versioning (`MAJOR.MINOR.PATCH`). There is no single global version bump; instead each module evolves independently and the **assembly manifest pins the set**.



- **Backend:** `openimis-be_py/openimis.json` pins each module (git tag or PyPI version). Changing a pin is a reviewable PR against the assembly.

- **Frontend:** `openimis-fe_js/openimis.json` pins each npm module version.

Did you know?

The two `openimis.json` manifests are effectively **lockfiles**. An openIMIS release is not a single tarball — it is a *coherent set of module versions* declared in the assemblies, plus `.env`. That is what lets a country deployment upgrade one module without touching the rest.

Common mistake

Merging a breaking change into a module without bumping the **MAJOR** version. Downstream assemblies pin ranges; a silent breaking change inside a "compatible" range breaks every deployment that resolves it. Respect semver: breaking → MAJOR, feature → MINOR, fix → PATCH.

9. Contribution workflow

openIMIS is governed by the **openIMIS Initiative** and developed as an org on GitHub (github.com/openimis) with a Technical Advisory Group. Contribution is per-repository and per-module.

1. **Find the right repo.** A change to claims lives in `openimis-be-claim_py` and/or `openimis-fe-claim_js` — not in the assembly, unless you are pinning a version.
2. **Branch, don't push to main.** Feature branch → PR.
3. **Follow the module's ownership.** Modules have maintainers; large changes should be discussed (issue/TAG) before a big PR.
4. **CI must be green.** Lint, tests, build.
5. **Semantic version + changelog** on merge, then pin in the assembly if the deployment needs it.
6. **Cross-repo changes** (a backend schema change consumed by a frontend module) require coordinated PRs and version bumps in both.

Small, module-scoped PRs win

Because the platform is modular, the best PRs are too: scoped to one module, with tests, a version bump, and — if it changes a public seam (a service signal, a published component, a permission code) — a note in the module README. Reviewers can reason about one slice at a time.

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be_py</code>	<code>openimis.json</code> , <code>requirements.txt</code>	Backend module manifest (version lockfile); editable-install target.
<code>openimis-be_py</code>	<code>manage.py</code> , <code>openimis/settings.py</code>	Run migrations, tests, the dev server; dynamic <code>INSTALLED_APPS</code> .
<code>openimis-fe_js</code>	<code>openimis.json</code> , <code>openimis-config-vite.js</code> , <code>vite.config.js</code>	Frontend manifest, generator, Vite build/dev server.
<code>openimis-dist_dkr</code>	<code>docker-compose.yml</code> , <code>.env</code>	The Docker topology and secrets for a faithful local/prod run.
<code>openimis-be-<name>_py</code>	<code><name>/tests/</code> , <code>.github/workflows/</code>	Per-module tests and CI.
<code>openimis-fe-<name>_js</code>	<code>package.json</code> , <code>.github/workflows/</code>	Jest/eslint scripts and CI; npm publish.
Any module	<code>setup.py</code> / <code>package.json</code>	Semantic version bumped on release; pinned by the assembly.

Hands-on lab



Lab 16.1 — Wire an editable loop and change a module

1. Clone `openimis-be_py` + `openimis-be-claim_py`; `pip install -e` the claim module.
2. Add a harmless log line in a claim **service**, hit the code path via GraphQL, and confirm your edit runs without reinstalling.
3. Run `python manage.py test claim`. Make it green.
4. `black . && isort . && flake8`. Commit on a branch.



Lab 16.2 — Reproduce a bug three ways

1. Pick a mutation. Reproduce it in **GraphiQL** (`/graphql`).
2. Reproduce the same logic in the **Django shell** by calling the service directly.
3. Reproduce it from the **UI** while watching **Redux DevTools**. Note which layer gives you the clearest signal — that instinct is the skill.

Exercises

1. Your teammate cannot see a module you added, even after `pip install -e`. What did you forget?
2. Explain why openIMIS has no single global version number.
3. A submission behaves oddly in the UI. List the exact order of debugging tools you would reach for.
4. What must accompany a breaking change to a module before it is safe for downstream assemblies?

Knowledge check



Q1: What is the difference between an editable install and the manifest pin, and why do both exist? (click for answer)



The **manifest** (`openimis.json`) pins the published module version and is the source of truth for builds and teammates. An **editable install** (`pip install -e` / `npm link`) overrides that pin locally so your edits are live without reinstalling. Both exist because you develop against local checkouts but ship reproducible builds from the pinned manifest.



Q2: Where is the authoritative status of a mutation, and which tool shows it? (click for answer)



On the backend **MutationLog** row. Query it in **GraphiQL** at `/graphql` via `mutationLogs(clientMutationId: ...)`. The immediate HTTP response is only a ticket.

? Q3: How does openIMIS version and release, given ~47 modules? (click for answer) ✓

Per-module semantic versioning. Each module is tagged and published independently; the assembly's `openimis.json` pins the coherent set of versions (a lockfile). An assembly release is that set plus `.env` — not one global version.

? Q4: Which linters/formatters run on each side? (click for answer) ✓

Backend: **black** (format), **isort** (imports), **flake8** (lint). Frontend: **eslint** (lint) and **prettier** (format). CI enforces them.

? Q5: You changed a backend GraphQL field that a frontend module consumes. What does the workflow require? (click for answer) ✓

Coordinated PRs in **both** repos, appropriate semantic version bumps (breaking → MAJOR), CI green on each, and updated pins in the assemblies so the two versions are deployed together.

Further reading

- [Set Up a Dev Environment](#) — the first-time setup this chapter builds on.
- [Best Practices](#) — the conventions reviewers enforce.
- [Extension Guide](#) and [End-to-End Code Walkthrough](#) — apply the loop to real code.
- [Docker & Deployment](#) — the compose topology behind `docker compose up`.
- github.com/openimis — the org, the repos, per-repo GitHub Actions, and issues.

The Developer Academy

A **seven-level guided track** that turns a competent Django engineer into a confident openIMIS contributor — from a first mental model, through building a backend module and a frontend page, all the way to designing a new business module, hardening it, and shipping it to production.

The rest of this handbook is a *reference course*: read a chapter, learn a concept. The **Academy** is the opposite shape — it is a *practice course*. Each level gives you a small amount of teaching, then makes you **do things**: bring up the stack, write a migration, wire a resolver, publish a component, profile a slow query, review a design. The deep chapters remain your textbook; the Academy is the lab.

” The one-sentence version

Ten pages of theory teach you *what* openIMIS is; the Academy's seven levels teach you to *build inside it* — and every level ends with a graded challenge and a knowledge check.

How the Academy works

Each level is a self-contained page with the same rhythm, so you always know where you are:

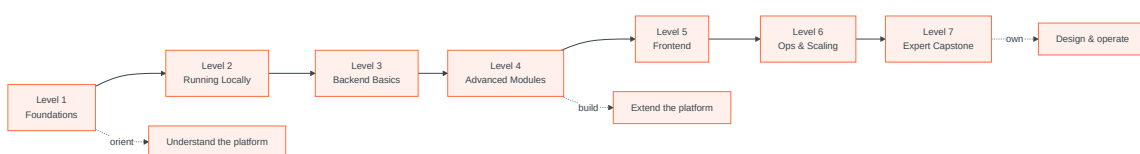
Section	What it is
Level badge	The difficulty tier — foundational, advanced, or expert.
Learning objectives	The concrete capabilities you will have when you finish.
Prerequisites	The prior level plus the deep chapters to read first.
Teaching content	A concise briefing — it <i>links</i> to the deep chapters rather than repeating them.
Hands-on labs	Step-by-step mini-labs. Type them; do not just read them.
Exercises	Smaller tasks to cement the ideas.
Challenge project	One larger, open-ended build that proves you learned the level.
Knowledge check	A short quiz; answers hidden in collapsibles so you self-test honestly.

How to actually pass a level

A level is "done" when you have (1) completed every hands-on lab on a real local stack, (2) finished the challenge project, and (3) can answer the knowledge-check questions **before** opening the collapsible. Reading alone does not count — openIMIS is learned in the terminal and the editor.

The learning arc

The seven levels are deliberately shaped as a curve: you start by *orienting*, spend the middle levels *building*, and finish by *owning* an entire module and its operations.



Read left to right, the phases are:

- **Orient (Levels 1–2).** Build the mental model, then get a real stack running on your machine and learn to poke it through GraphQL.
- **Build (Levels 3–5).** Create a backend module — model, migration, service, query, mutation — then extend it with signals, permissions, and configuration, then give it a React frontend.
- **Own (Levels 6–7).** Make it fast, observable, and production-ready; then, as a capstone, design a brand-new business module end to end and defend every architectural decision.

i Did you know?

The Academy's build phase mirrors how real openIMIS modules were born. Modules such as `individual`, `social_protection`, and the `calcrule_*` family were added *without forking core* — exactly the plugin discipline you will practice from Level 3 onward. If you can build a module the Academy way, you can contribute one upstream.

The seven levels

Level 1 — Foundations

Level 1

The platform, the plugin mental model, and the domain vocabulary. No code yet — just the map you will navigate for the next six levels.

Level 2 — Running Locally

Level 2

Bring up the Docker stack, log in, run your first GraphQL query in GraphQL, and change a module's configuration.

Level 3 — Backend Basics

Level 3

Build a minimal module from scratch: a model, a migration, a service,

and a working GraphQL query and mutation.

Level 4 — Advanced Modules

Level 4

Service signals, rights-based permissions, DB-backed configuration, plugin registration, and calculation-rule-style extension.

Level 5 — Frontend

Level 5

Pages, components, forms, GraphQL-in-Redux, and contributing menus and routes via `ModulesManager` and published components.

Level 6 — Ops & Scaling

Advanced

Performance, scaling, caching, observability, and a real production deployment topology.

Level 7 — Expert Capstone

Expert

Design a complete new business module end to end, then review its architecture, performance, security, and deployment.

Before you start

The Academy leans on the deep chapters constantly. You do **not** need to have read them all — each level tells you exactly which ones to open — but keep these three windows ready to

switch to:

- The [Architecture overview](#) — the shape of the whole system.
- The [Repository Map](#) — where every module actually lives.
- The [Reference glossary](#) — for any term that is new.



This is a community handbook

The Academy is an independent educational resource, not official openIMIS documentation. Module counts, file layouts, and configuration keys drift between releases. Where a step is version-dependent, the level says so and points you at the authoritative repository under github.com/openimis. Always treat the source code and the [official openIMIS wiki](#) as the final authority.

Ready? Open [Level 1 — Foundations](#) and build your mental model.

Further reading

- The full course home page: [Developer Academy handbook](#)
- All source repositories: github.com/openimis
- Official product and functional wiki: [openIMIS on Atlassian](#)
- GraphQL, from the source: graphql.org/learn

Level 1 — Foundations

Level 1

The first level builds no code. Its job is to give you the **mental model** you will carry through every later level: what openIMIS is, why it is shaped the way it is, and the vocabulary you must speak to read the source without getting lost.

Learning objectives

By the end of this level you will be able to:

- Explain in two sentences what openIMIS *does* and who deploys it.
- Draw the high-level architecture from memory: browser → gateway → GraphQL/REST → core → business modules → PostgreSQL.
- Explain the **plugin (module) architecture** and why "there is no single repository".
- Distinguish the two **assembly repos** (`openimis-be_py` , `openimis-fe_js`) from the ~47 **module repos**.
- Use the core domain terms — *insuree*, *policy*, *product*, *contribution*, *claim* — correctly.

Prerequisites

This is the entry level, so the only prerequisites are the reading chapters:

- [Getting Started — index](#) and [Platform Overview](#)
- [Terminology & Glossary Primer](#)
- [Architecture Overview](#)

You should already know Python, Django, the ORM, and PostgreSQL. You do **not** need to know GraphQL yet — that starts at Level 2.

Briefing: what openIMIS is

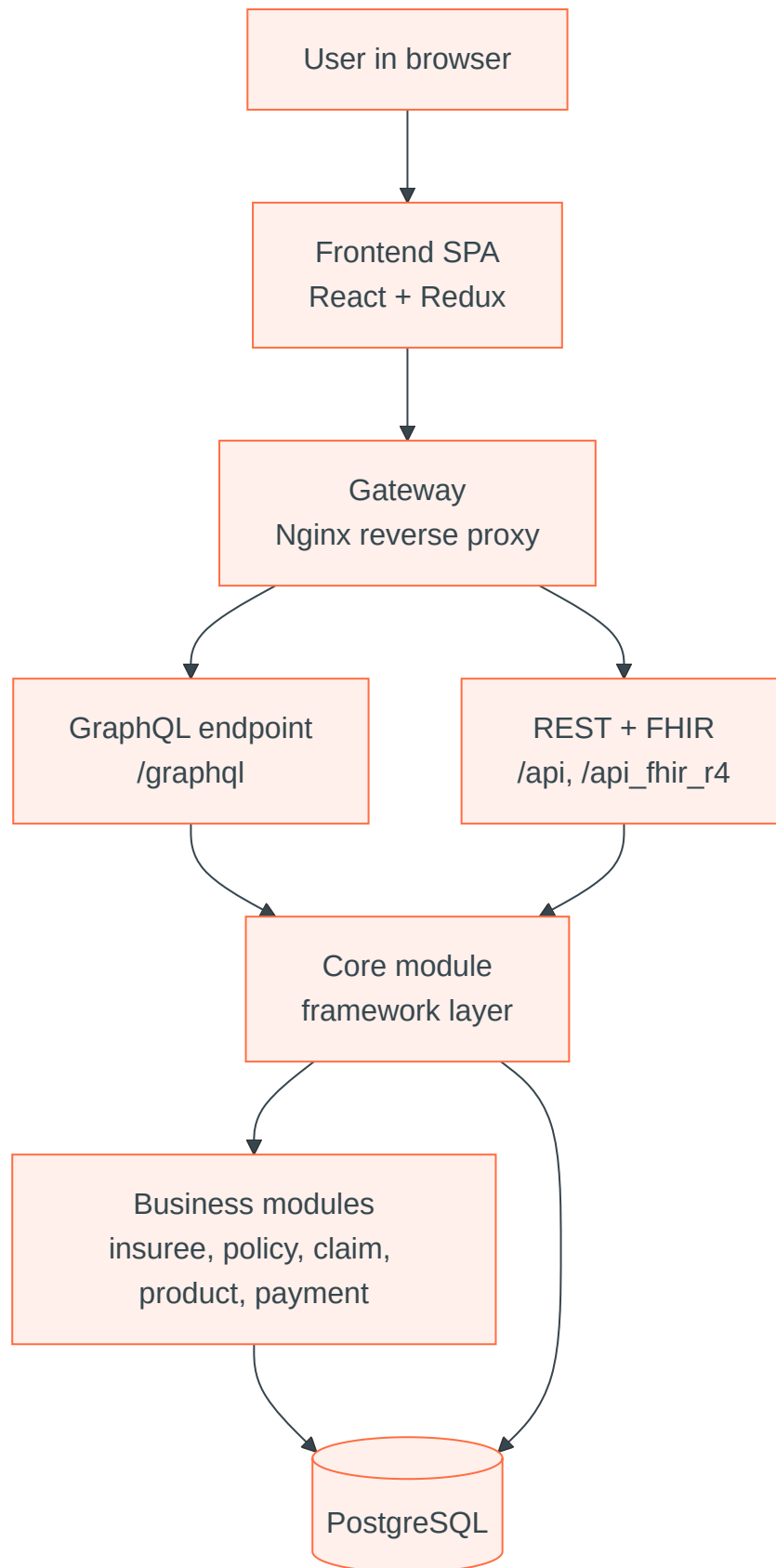
openIMIS — **open Insurance Management Information System** — is an open-source platform for running **health insurance and social health protection** schemes: registering beneficiaries, linking them to insurance products, recording contributions, and adjudicating and

paying claims from health facilities. It is a recognized **Digital Public Good** and runs national schemes across Africa and Asia.

The "why" behind its shape is historical. openIMIS began as **IMIS**, a monolithic **.NET + Microsoft SQL Server** application. Around 2018–2019 it was re-architected into a **Django + React** system decomposed into independently versioned **modules**. That modularization is the platform's central design idea: countries customize by adding or replacing modules, **never by forking core**.

For the full story, read the [Platform Overview](#). Here we only need the shape.

The architecture in one diagram



Commit that top-to-bottom flow to memory. Every later level zooms into one band of it. The [Architecture Overview](#) explains each band in depth.

The idea you must internalize: there is no single repository

A normal Django project lives in one repo you can clone and read. openIMIS does not. The running backend is **assembled** at startup from dozens of separate repositories:

Kind of repo	Example	Role
Backend assembly	<code>openimis-be_py</code>	A Django <i>project</i> that reads a manifest and wires modules together. Almost no business logic of its own.
Frontend assembly	<code>openimis-fe_js</code>	A React SPA that reads a manifest and imports frontend modules.
Backend module	<code>openimis-be-claim_py</code>	One domain area. Package name <code>claim</code> . Has its own <code>models.py</code> , <code>schema.py</code> , <code>services.py</code> .
Frontend module	<code>openimis-fe-claim_js</code>	The React side of the same domain area.

The backend assembly's `openimis.json` is the **module manifest** — roughly 47 modules, each with a git/pip source. At startup `openimis/settings.py` builds `INSTALLED_APPS` dynamically from that list, and `openimis/schema.py` stitches every module's GraphQL schema into one. The details are in the [Plugin / Module System](#) chapter and the [Repository Map](#); for now, just hold the idea.

Common mistake

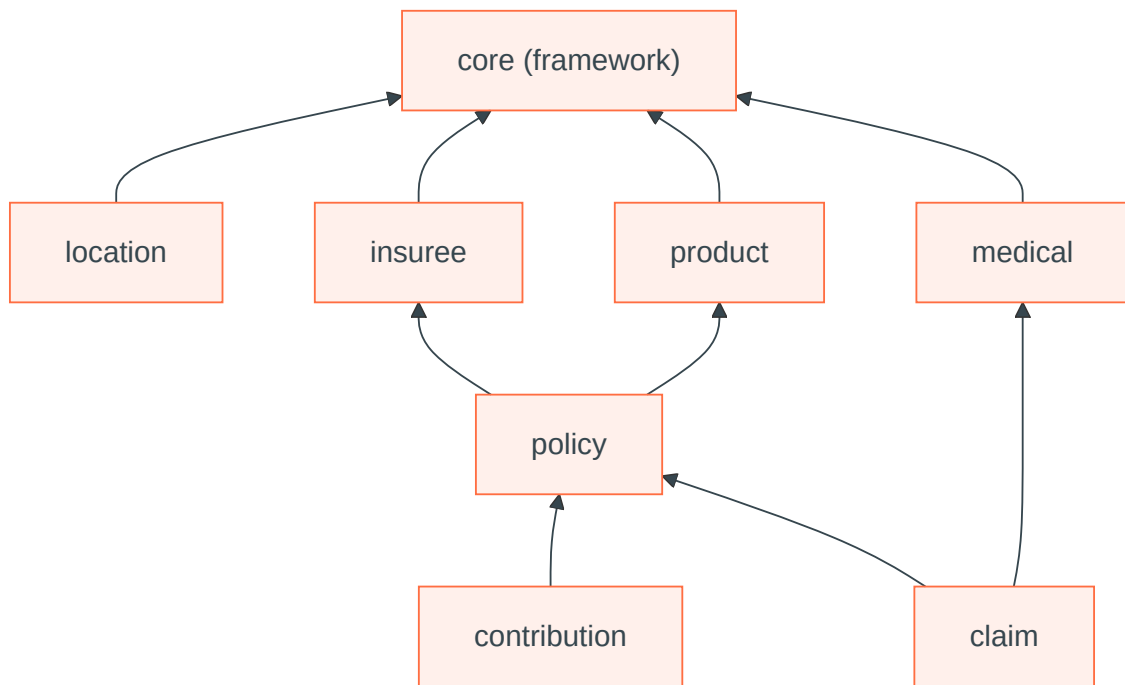
Do not try to understand openIMIS by opening one repository and reading it like a normal Django app. If a class seems to come from nowhere, it lives in **another module's repo**. The application only exists once the assembly repo has read its manifest and stitched the pieces together.

The domain vocabulary

You cannot read openIMIS code without the health-financing vocabulary. The minimum set:

Term	Meaning	Lives in module
Insuree	A person covered by the scheme (a beneficiary).	insuree
Individual	A newer, more general "person" abstraction.	individual
Product	An insurance product: what is covered, ceilings, prices.	product
Policy	A contract linking an insuree (family) to a product — i.e. coverage.	policy
Contribution	A premium paid against a policy.	contribution
Claim	A record of services delivered by a health facility, valued and paid.	claim
Location	Region / district / municipality / village hierarchy.	location
Health Facility	A hospital or clinic that delivers services and files claims.	location
Officer	A staff user (e.g. enrolment officer).	core

The full list is in the [Terminology Primer](#) and the [Reference glossary](#). The dependency intuition — core underneath everything, reference data (location , medical , product) next, then insuree / policy / contribution / claim on top — is captured here:



i Did you know?

The database still carries the scars of its .NET ancestry: many tables keep `tbl` prefixes and camelCase columns, and rows carry both a legacy integer key and a newer UUID. When you meet this in the [Database chapter](#), it will make sense — it is heritage, not chaos.

Hands-on labs

No stack yet — these labs build orientation, not code. Do them with the [Repository Map](#) and github.com/openimis open.

Lab 1.1 — Map the repositories

1. Open github.com/openimis and list every repository whose name matches `openimis-be-*_py`.
2. In a scratch file, write the module name (drop the `openimis-be-` prefix and `_py` suffix) for ten of them — e.g. `openimis-be-claim_py` → `claim`.
3. Next to each, guess its domain role in one line. Check yourself against the table in the [Modules index](#).

Done when: you can name at least ten backend modules and their jobs without looking.

Lab 1.2 — Find the two assemblies

1. Open `openimis-be.py` on GitHub and locate `openimis.json`. Skim it — notice each entry names a module and a source.
2. Open `openimis/settings.py` in the same repo and find where `INSTALLED_APPS` is *built from the module list* rather than hard-coded.
3. Open `openimis/schema.py` and find where module `Query` / `Mutation` classes are combined by multiple inheritance.

Done when: you can point to the manifest, the dynamic `INSTALLED_APPS`, and the schema stitching. (Details: [Plugin / Module System](#).)

Lab 1.3 — Trace a business flow on paper

Using only the vocabulary table, write the chain of records that must exist for a claim to be paid:

A **health facility** files a **claim** for an **insuree** whose **policy** (bought against a **product**) is valid because a **contribution** was paid.

Draw it as a small diagram. Compare with the dependency graph above.

Done when: your drawing matches the module dependency intuition.

Exercises

1. In two sentences, define openIMIS for a non-technical colleague. No jargon.
2. Explain to a fellow Django dev why they *cannot* just `git clone` one repo and run openIMIS.
3. List the differences between an **assembly repo** and a **module repo**.
4. For each term — *insuree*, *policy*, *product*, *contribution*, *claim* — give a one-line real-world example (e.g. "a claim is the clinic billing for a malaria consultation").
5. Explain why the database has both integer keys and UUIDs.

Challenge project

Produce a one-page "openIMIS orientation brief" you could hand to a new teammate on day one. It must contain:

- A hand-drawn (or diagrammed) architecture sketch: browser → gateway → GraphQL/REST → core → business modules → PostgreSQL.

- A table of at least eight modules with one-line roles.
- The domain vocabulary table in your own words.
- A short paragraph explaining the plugin/assembly idea and why forking core is discouraged.

Keep it to one page. If you cannot fit it, you do not yet understand it well enough — that is the point of the exercise. You will reuse this brief as the cover sheet for your Level 7 capstone.

Knowledge check

? Q1: In one sentence, what problem does openIMIS solve? (click for answer) ✓

It manages **health insurance / social health protection** schemes — registering beneficiaries, linking them to insurance products via policies, recording contributions, and adjudicating and paying claims — as an open-source Digital Public Good deployed by countries in Africa and Asia.

? Q2: Why is there 'no single repository' for the backend? (click for answer) ✓

The deployable backend is the **assembly repo** `openimis-be_py`, which contains almost no business logic. At startup it reads its **module manifest** (`openimis.json`), builds `INSTALLED_APPS` dynamically, and stitches each module's GraphQL schema together. The real code lives in ~47 separate `openimis-be-<name>_py` module repositories.

? Q3: What is the difference between an assembly repo and a module repo? (click for answer) ✓

An **assembly repo** (`openimis-be_py`, `openimis-fe_js`) is wiring: a manifest plus glue that composes many modules into one running app. A **module repo** (`openimis-be-claim_py`, `openimis-fe-claim_js`) owns one domain area's models, schema, services, and UI. Assemblies rarely change; modules are where features live.

? Q4: Put these in dependency order (which depends on which): claim, policy, product, core, contribution. (click for answer) ✓

`core` is the base. `product` and `policy` build on `core`; `policy` links an insuree to a `product`. `contribution` is paid against a `policy`. `claim` is filed against a `policy` (and references medical items). So bottom-up: **core** → **product** → **policy** → **contribution/claim**.

 **Q5: Why does the database carry both integer keys and UUIDs, and camelCase `tbl`-prefixed table names? (click for answer)** 

Heritage. openIMIS descends from the .NET/MSSQL application **IMIS**. The legacy schema used integer keys and `tbl`-prefixed camelCase names; the modular rewrite added UUIDs alongside the legacy integer keys and migrated MSSQL → PostgreSQL without discarding the old shape.

Further reading

- [Getting Started](#) and [Platform Overview](#)
- [Architecture Overview](#) and [Repository Map](#)
- [Terminology Primer](#) and [Reference glossary](#)
- All source repositories: github.com/openimis

When your orientation brief is done, continue to **Level 2 — Running Locally**.

Level 2 — Running Locally

Level 2

Theory is over. This level puts a **real openIMIS stack on your machine**, teaches you to log in, run your first GraphQL query in GraphiQL, and change a module's configuration — the loop you will repeat for the rest of the Academy.

Learning objectives

By the end of this level you will be able to:

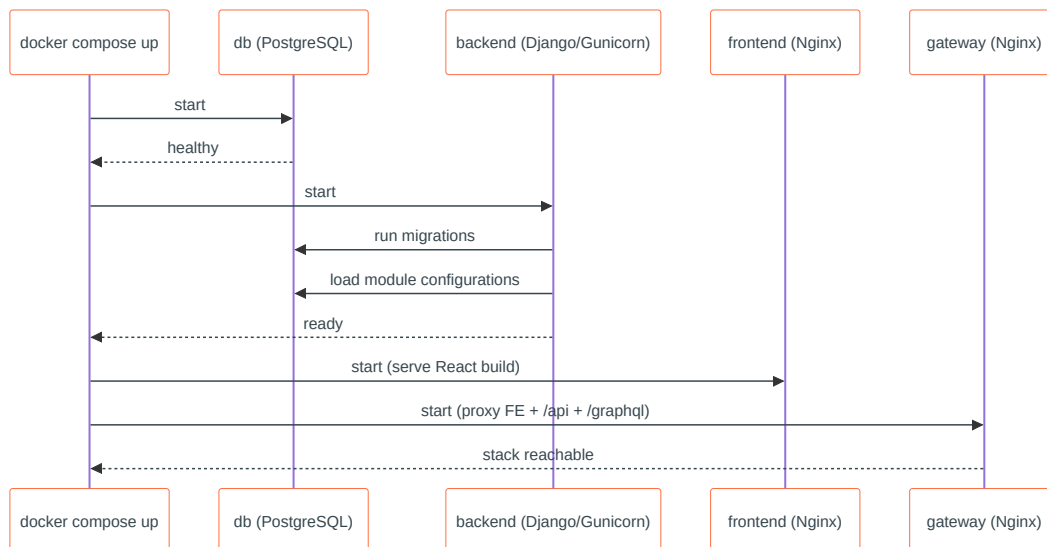
- Bring up the full Docker stack (`db` , `backend` , `frontend` , `gateway`) and reach the UI.
- Explain the startup order and why `backend` runs migrations and loads module configs before it is ready.
- Log in and reach the **GraphiQL** explorer.
- Read GraphQL from first principles and run a query and an introspection request.
- Change a **module configuration** value and observe the effect.

Prerequisites

- Completed **Level 1 — Foundations** — you have the mental model.
- Read [Set Up a Dev Environment](#) and the [Docker & Deployment chapter](#).
- Skim the [GraphQL chapter](#) — this level teaches just enough GraphQL to be dangerous; that chapter is the full treatment.
- You know Docker and `docker compose` already.

Briefing: how the stack comes up

The distribution repo `openimis-dist_dkr` is `docker-compose` based. The typical services and their **startup order** matter:



The key insight: **backend is not ready the instant the container starts**. Its entrypoint (`script/` in `openimis-be.py`) runs Django migrations and loads each module's configuration into the DB *before* Gunicorn serves traffic. The gateway ties the frontend, `/api`, and `/graphql` together behind one origin. Full detail: [Docker & Deployment](#).

Did you know?

A module's `DEFAULT_CFG` dict (in its `apps.py`) is *overlaid at startup* by a per-module JSON row stored in the database (`ModuleConfiguration` in core). That is why the backend touches the DB before it is ready — and why you can reconfigure a deployment without changing code. You will use this in Lab 2.4.

GraphQL in ninety seconds

You know REST; you may not know GraphQL. The differences that matter today:

REST	GraphQL
Many endpoints (<code>/insurees</code> , <code>/claims</code>).	One endpoint: <code>/graphql</code> .
Server decides the response shape.	Client asks for exactly the fields it wants.
Read = <code>GET</code> , write = <code>POST / PUT</code> .	Read = query , write = mutation .
Discover via docs.	Discover via introspection (the schema describes itself).

A query looks like the response you want, with the values removed:

```
query {  
  insurees(first: 3) {  
    edges {  
      node { uuid chfId lastName otherNames }  
    }  
  }  
}
```

`edges / node` are the **connection** (pagination) pattern — openIMIS uses it everywhere via core's `ExtendedConnection`, which also exposes `totalCount`. The full model — queries, mutations, connections, and openIMIS's asynchronous mutation pattern — is in the [GraphQL chapter](#).

Hands-on labs

You need Docker and `docker compose`. Exact service names, ports, and env keys are version-dependent — confirm against the `openimis-dist_dkr` repo and the [setup chapter](#).

Lab 2.1 — Bring up the stack

1. Clone the distribution repo:

```
git clone https://github.com/openimis/openimis-dist_dkr.git  
cd openimis-dist_dkr
```

2. Copy the sample environment file and review it. It supplies the DB password, Django secret, and admin credentials:

```
cp .env.example .env # exact name may differ; check the repo
```

3. Bring the stack up and watch the logs:

```
docker compose up -d  
docker compose logs -f backend
```

4. Wait until the backend log shows migrations complete and Gunicorn is serving. **Do not** open the UI before this — you will see a gateway error.

Done when: `docker compose ps` shows `db`, `backend`, `frontend`, and `gateway` all up, and the backend has finished migrating.



Common mistake

Opening the frontend before the backend has finished migrating produces confusing 502/blank-page errors. That is not a bug — it is the startup order in the sequence diagram above. Tail the backend log first.

Lab 2.2 — Log in

1. Open the gateway URL in your browser (commonly `http://localhost` — check your compose port mapping).
2. Log in with the admin credentials from your `.env`.
3. Click around: find the Insurees, Policies, and Claims areas in the menu. Notice which menu items you can and cannot see — that is **rights-based permissions** at work (Level 4).

Done when: you reach the authenticated home screen.



Did you know?

Your login does not return a token in the response body. openIMIS uses `django-graphql-jwt` and stores the JWT in an **HttpOnly cookie**, so JavaScript can never read it. That is a deliberate security choice — see the [Security chapter](#).

Lab 2.3 — Your first GraphQL query

1. Navigate to the GraphQL explorer, typically at `/graphql` behind the gateway (e.g. `http://localhost/graphql`). Because you logged in via the UI, the auth cookie travels with your request.
2. Run a simple query:

```
query {  
  insurees(first: 3) {  
    totalCount  
    edges { node { uuid chfId lastName otherNames } }  
  }  
}
```

3. Change the fields — remove `otherNames`, add `dob` — and re-run. Notice the response shape follows *your* request.
4. Run an **introspection** query to prove the schema is self-describing:

```
query {  
  __schema { queryType { name } }  
}
```

5. Open GraphQL's "Docs"/"Schema" panel and browse the `Query` type — every module's queries were stitched into it.

Done when: you have run a data query, edited its field set, and browsed the schema in the docs panel.

Lab 2.4 — Change a module configuration

You will change a configuration value and watch it take effect. Because config lives in the DB (`ModuleConfiguration`), overlaying `DEFAULT_CFG`, you do not edit code.

1. Pick a benign, observable config on a module — for example a claim-module toggle or a display default. Find candidate keys in the module's `apps.py` `DEFAULT_CFG` (e.g. `openimis-be-claim_py/claim/apps.py`).
2. Update the stored configuration. The supported path is version-dependent — the two common routes:
3. Via the admin/config UI if your build exposes one, or
4. By updating the module's `ModuleConfiguration` JSON row (see [Configuration chapter](#)).
5. Restart the backend so it reloads config at startup:

```
docker compose restart backend
```

6. Reload the UI / re-run a query and confirm the new behavior.

Done when: you changed a value in the DB-backed config (not in code) and saw the effect after a backend restart.

Exercises

1. From memory, list the four core services and the exact order they become ready. Why does `backend` touch the DB before serving?
2. Write a GraphQL query that returns the **first 5 policies** with only their `uuid` and an expiry/validity field. Run it.
3. Explain, to someone who only knows REST, what `edges { node { ... } }` is doing and why `totalCount` is useful.
4. Find where the JWT lives after login and explain why it is in an `HttpOnly` cookie rather than `localStorage`.

5. Name three things that would break if you deleted a module's `ModuleConfiguration` row and restarted.
-

Challenge project

Write a reproducible "local stack runbook" for a new teammate. It must let someone go from a clean machine to a working, logged-in stack, and prove it works. Include:

- The exact `git clone / .env / docker compose up` steps for your environment.
- A "how do I know it's ready?" check based on the backend log and `docker compose ps`.
- Three example GraphQL queries (an insuree list, a policy list, and one introspection query) with expected-shape output.
- A "change one config value" walkthrough (from Lab 2.4) with before/after evidence.
- A short troubleshooting section: the top three failures you hit and how you fixed them.

Test it by tearing everything down (`docker compose down -v`) and following your own runbook from scratch. If it does not work verbatim, fix the runbook, not your memory.

Knowledge check

 **Q1: Why is the backend container not ready the moment it starts? (click for answer)** 

Its entrypoint runs Django **migrations** and **loads every module's configuration** into the database before Gunicorn begins serving. Until that finishes, the gateway cannot proxy a working backend, which is why the startup order is `db → backend → frontend/gateway`.

 **Q2: What is the single GraphQL endpoint, and how do you discover what it supports? (click for answer)** 

Everything goes through **one** endpoint, `/graphql`. You discover its capabilities through **introspection** — the schema describes itself — which is exactly what GraphQL's Docs/Schema panel uses. There is no per-resource URL like REST.

? Q3: What do `edges` and `node` mean in an openIMIS query? (click for answer) ✓

They are the **connection** (Relay-style pagination) pattern. A connection has `edges`, each `edge` has a `node` (the actual object) and a cursor. openIMIS's `ExtendedConnection` (in core) adds `totalCount` and `edgeCount` so you can paginate and count in one request.

? Q4: Where does openIMIS store the auth token after login, and why there? (click for answer) ✓

In an **HttpOnly cookie**, set by `django-graphql-jwt`. HttpOnly means client JavaScript cannot read it, which mitigates token theft via XSS. This is why your GraphQL queries are authenticated automatically once you have logged in through the UI.

? Q5: You changed a config value in the DB but nothing happened. What did you forget? (click for answer) ✓

Module configuration is read at **startup**, when `DEFAULT_CFG` is overlaid by the stored `ModuleConfiguration` JSON. You must **restart the backend** (`docker compose restart backend`) for the new value to be loaded.

Further reading

- [Set Up a Dev Environment](#) and [Docker & Deployment](#)
- [GraphQL chapter](#) — the full model behind Lab 2.3
- [Configuration chapter](#) — the DB-backed config used in Lab 2.4
- [Security chapter](#) — JWT-in-cookie and OAuth2/OIDC
- Distribution repo: [openimis-dist_dkr](#)

With a running stack you can now build. Continue to [Level 3 — Backend Basics](#).

Level 3 — Backend Basics

Level 3

Now you build. This level walks you through creating a **minimal openIMIS backend module from scratch** — a model, a migration, a service, and a working GraphQL query and mutation — following openIMIS conventions so the result is a real plugin, not a toy.

Learning objectives

By the end of this level you will be able to:

- Scaffold a new backend module with the standard layout (`apps.py` , `models.py` , `schema.py` , `services.py` , `migrations/`).
- Register the module in the assembly's manifest so it loads into `INSTALLED_APPS` .
- Model data on openIMIS base classes (`HistoryModel` , `json_ext`) and generate a migration.
- Write a **service** and a GraphQL **query** exposing your model.
- Write a GraphQL **mutation** and understand the asynchronous `OpenIMISMutation` pattern.

Prerequisites

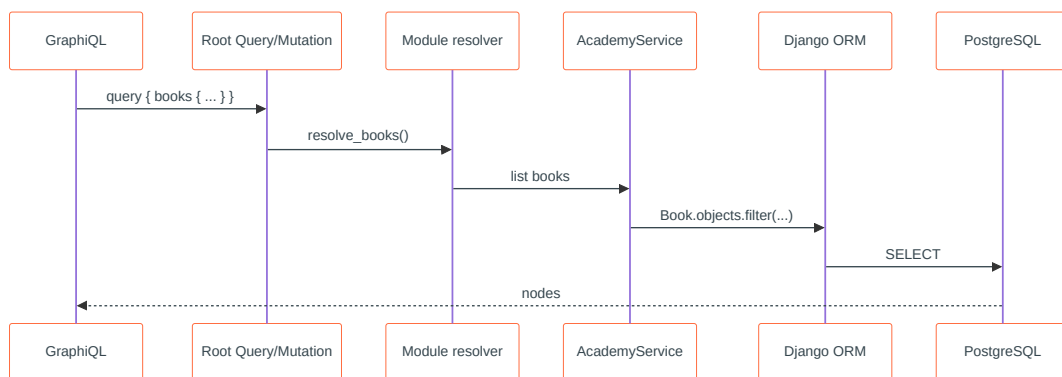
- Completed [Level 2 — Running Locally](#) — you have a working stack and can run GraphQL.
- Read the [Backend Deep Dive](#), the [Core module chapter](#), and the [GraphQL chapter](#).
- Helpful: the [End-to-End Code Walkthrough](#) and [Extending openIMIS](#), which this level mirrors in miniature.

Briefing: the anatomy of a module

Every backend module follows the same layout. You are about to create each file.

File	Responsibility
<code>apps.py</code>	An <code>AppConfig</code> subclass (e.g. <code>class AcademyConfig(AppConfig)</code>) with a <code>DEFAULT_CFG</code> dict; the module's identity.
<code>models.py</code>	Django models, usually on openIMIS base classes from core.
<code>migrations/</code>	Standard Django migrations.
<code>services.py</code>	Business logic. Resolvers and mutations call services, not the ORM directly.
<code>schema.py</code>	The GraphQL <code>Query</code> and <code>Mutation</code> classes that the assembly stitches in.
<code>gql_queries.py</code> / <code>gql_mutations/</code>	The GraphQL types and mutation classes.
<code>urls.py</code>	Any REST url patterns (optional).
<code>tests/</code>	Tests.

The request path you are building:



Base models you will use

Core provides base classes so your model gets openIMIS's temporal-versioning and extensibility for free (see [Core module](#)):

- **HistoryModel** — history-tracked; carries `validity_from` / `validity_to` and a UUID.

- `json_ext` — a `JSONField` present on many models for schema-free extensibility.

You will model a tiny "reading room" domain — a `Book` — precisely because it is *not* a real openIMIS entity, so you can practice the mechanics without domain distraction.

Did you know?

Because you build on `HistoryModel`, "deleting" a record does not physically remove it — it closes the `validity_to` window. openIMIS keeps history for audit. This temporal model is explained in the [Database chapter](#).

Hands-on labs

All code below is **illustrative** — it faithfully reflects openIMIS conventions but is simplified for teaching. Confirm exact base-class names and imports against `openimis-be-core_py`.

Lab 3.1 — Scaffold the module

1. Create a module repo folder next to your other openIMIS repos, named the openIMIS way:

```
mkdir -p openimis-be-academy_py/academy/migrations
cd openimis-be-academy_py
```

2. Make the package importable: add `academy/__init__.py` and `academy/migrations/__init__.py` (empty files).
3. Create `academy/apps.py`:

```
# illustrative
from django.apps import AppConfig

DEFAULT_CFG = {
    "gql_query_books_perms": [], # tighten in Level 4
    "gql_mutation_books_perms": [],
}

class AcademyConfig(AppConfig):
    name = "academy"
    default_auto_field = "django.db.models.BigAutoField"

    gql_query_books_perms = []
    gql_mutation_books_perms = []

    def _configure_permissions(self, cfg):
        AcademyConfig.gql_query_books_perms = cfg["gql_query_books_perms"]
        AcademyConfig.gql_mutation_books_perms =
        cfg["gql_mutation_books_perms"]

    def ready(self):
        from core.models import ModuleConfiguration
        cfg = ModuleConfiguration.get_or_default("academy", DEFAULT_CFG)
        self._configure_permissions(cfg)
```

Done when: the `academy` package exists with `apps.py` and empty migration package.

Lab 3.2 — Register the module in the manifest

1. In the backend assembly `openimis-be_py`, add an entry for `academy` to `openimis.json` (the module manifest), pointing at your local path.
2. For development, install it editable so Django can import it:

```
pip install -e ../openimis-be-academy_py/
```

3. Confirm `openimis/settings.py` builds `INSTALLED_APPS` from the module list and that `academy` now appears (add a temporary `print` or run `python manage.py shell` and check `django.conf.settings.INSTALLED_APPS`).

Done when: `academy` is in `INSTALLED_APPS`. (Mechanism: [Plugin / Module System](#).)



Common mistake

Forgetting `pip install -e` (or the manifest entry) means Django silently never loads your module — no error, just nothing. If your model/query "doesn't exist", check `INSTALLED_APPS` first.

Lab 3.3 — Model and migration

1. Create `academy/models.py`:

```
# illustrative
from django.db import models
from core.models import HistoryModel

class Book(HistoryModel):
    title = models.CharField(max_length=255)
    author = models.CharField(max_length=255, null=True, blank=True)
    copies = models.IntegerField(default=1)

    class Meta:
        managed = True
        db_table = "academy_Book"
```

2. Generate and apply the migration:

```
python manage.py makemigrations academy
python manage.py migrate academy
```

3. Verify in `psql` that the `academy_Book` table exists with the temporal columns inherited from `HistoryModel`.

Done when: the table exists and `makemigrations` produced a file in `academy/migrations/`.

Lab 3.4 — Service and GraphQL query

1. Create `academy/services.py`:

```
# illustrative
from .models import Book

class BookService:
    def __init__(self, user):
        self.user = user

    def list_books(self):
        return Book.objects.filter(validity_to__isnull=True)

    def create_book(self, data):
        return Book.objects.create(**data)
```

2. Create `academy/gql_queries.py`:

```
# illustrative
import graphene
from graphene_django import DjangoObjectType
from core import ExtendedConnection
from .models import Book

class BookGQLType(DjangoObjectType):
    class Meta:
        model = Book
        interfaces = (graphene.relay.Node,)
        connection_class = ExtendedConnection
        filter_fields = {"title": ["exact", "icontains"]}
```

3. Create `academy/schema.py`:

```
# illustrative
import graphene
from core.schema import OrderedDjangoFilterConnectionField
from .gql_queries import BookGQLType
from .models import Book

class Query(graphene.ObjectType):
    books = OrderedDjangoFilterConnectionField(
        BookGQLType, orderBy=graphene.List(graphene.String),
    )

    def resolve_books(self, info, **kwargs):
        return Book.objects.filter(visibility_to__isnull=True)
```

4. Restart the backend and run in GraphQL:

```
query { books { totalCount edges { node { id title author copies } } } }
```

It returns an empty connection — you have no data yet. That is success.

Done when: the `books` query appears in the schema and returns an empty connection without error.

Lab 3.5 — A mutation

openIMIS mutations follow the asynchronous `OpenIMISMutation` pattern: the mutation records a `MutationLog`, does its work (often in a service), returns a `clientMutationId`, and the client polls for status. Every mutation is audited. Full treatment: [GraphQL chapter](#).

1. Create `academy/gql_mutations/__init__.py`:

```
# illustrative
import graphene
from core.schema import OpenIMISMutation
from ..services import BookService

class CreateBookMutation(OpenIMISMutation):
    _mutation_module = "academy"
    _mutation_class = "CreateBookMutation"

    class Input(OpenIMISMutation.Input):
        title = graphene.String(required=True)
        author = graphene.String(required=False)
        copies = graphene.Int(required=False)

    @classmethod
    def async_mutate(cls, user, **data):
        data.pop("client_mutation_id", None)
        data.pop("client_mutation_label", None)
        BookService(user).create_book(data)
        return None # None => success; a list of errors => failure
```

2. Add a `Mutation` class to `academy/schema.py`:

```
# illustrative
import graphene
from .gql_mutations import CreateBookMutation

class Mutation(graphene.ObjectType):
    create_book = CreateBookMutation.Field()
```

3. Ensure the assembly's `openimis/schema.py` picks up `academy.schema.Query` and `academy.schema.Mutation` (it stitches every module's `Query` / `Mutation` by multiple inheritance).

4. Restart and run:

```
mutation {
  createBook(input: { title: "Django for APIs", author: "W. Vincent",
    copies: 2 }) {
    clientMutationId
    internalId
  }
}
```

5. Re-run the Lab 3.4 `books` query — your book is there.

Done when: the mutation succeeds and the created book appears in the query.

Did you know?

Returning `None` from `async_mutate` signals success; returning a list of error dicts signals failure and is recorded on the `MutationLog`. The client does not block on the mutation — it polls the log's status. This is why the openIMIS frontend has a "journalize"/polling helper (you will meet it in Level 5).

Exercises

1. Add an `isbn` field to `Book`, migrate, and expose it in the query. What migration did `makemigrations` produce?
2. Add a `resolve_books` filter so a `title_Contains` argument works from GraphQL.
3. Write an `UpdateBookMutation` that changes `copies`. Route it through `BookService`.
4. Explain what physically happens to a row when you "delete" a `HistoryModel` book (hint: `validity_to`). Write a `DeleteBookMutation` that closes the validity window instead of hard-deleting.
5. Break your module on purpose: remove it from `openimis.json`, restart, and confirm the `books` query vanishes. Explain why.

Challenge project

Build a complete `academy` mini-module with full CRUD over `Book`:

- `Book` model on `HistoryModel` with `title`, `author`, `copies`, and one `json_ext`-stored field of your choice.
- A `BookService` that is the *only* place touching the ORM.
- A filterable, paginated `books` query using `ExtendedConnection`.
- `CreateBookMutation`, `UpdateBookMutation`, and `DeleteBookMutation` (soft delete via validity window), all on `OpenIMISMutation`.
- A short `README` documenting how to install the module into the assembly and the GraphQL calls to exercise each operation.

Prove it end to end: from an empty table, create three books, update one, soft-delete one, and show the final `books` query result. Keep this module — Levels 4 and 5 extend it.

Knowledge check

? Q1: What is the job of a module's `apps.py` ? (click for answer) ✓

It defines the module's `AppConfig` subclass (its Django identity), holds the `DEFAULT_CFG` dict of configuration defaults, and typically loads the DB-stored `ModuleConfiguration` overlay in `ready()` — including wiring up permission codes via a `_configure_permissions` method. It is how the module announces itself to the assembly.

? Q2: You added a model but the GraphQL query says it doesn't exist. First thing to check? (click for answer) ✓

Whether the module is actually loaded: is it in `openimis.json`, was it `pip install -e'd`, and does it appear in `settings.INSTALLED_APPS` ? A module that is not in `INSTALLED_APPS` loads nothing, silently. Check that before debugging the schema.

? Q3: Why do resolvers and mutations call a service instead of the ORM directly? (click for answer) ✓

To keep business logic in one testable place, reusable by GraphQL, REST/FHIR, signals, and background jobs alike. The `schema.py` layer should stay thin — validation, permissions, and orchestration; the `services.py` layer owns the actual work.

? Q4: Describe the asynchronous `OpenIMISMutation` pattern. (click for answer) ✓

A mutation creates a `MutationLog`, runs its work (usually in a service) inside `async_mutate`, and returns a `clientMutationId/internalId` immediately. Returning `None` marks success; returning error dicts marks failure. The client **polls** the mutation log's status rather than blocking. Every mutation is thereby audited.

? Q5: What does `ExtendedConnection` add over a plain Relay connection? (click for answer) ✓

It adds `totalCount` and `edgeCount` to the connection, so a single paginated query can also report how many records exist — used pervasively across openIMIS queries.

Further reading

- [Backend Deep Dive](#) and [Core module](#)
- [GraphQL chapter](#) — connections and the async mutation pattern
- [Plugin / Module System](#) — how the manifest becomes `INSTALLED_APPS`
- [End-to-End Code Walkthrough](#) and [Extending openIMIS](#)
- [Database chapter](#) — `HistoryModel` and temporal validity

With a real module built, add the advanced seams in [Level 4 — Advanced Modules](#).

Level 4 — Advanced Module Development

Level 4

Your Level 3 module works, but it is naive: no permissions, no configuration discipline, and no way for *other* modules to hook into it. This level adds the real openIMIS extension seams — **service signals**, **rights-based permissions**, **DB-backed configuration**, **plugin registration**, and **calculation-rule-style** extension.

Learning objectives

By the end of this level you will be able to:

- Enforce **rights-based permissions** (integer permission codes) in resolvers and mutations.
- Define and load module configuration from the DB, overlaying `DEFAULT_CFG`.
- Emit a **service signal** and **bind** a handler in another module without importing it.
- Register a pluggable rule the **calculation-rule** way, discovered via signals.
- Explain when to use a *service signal* versus a plain Django ORM signal.

Prerequisites

- Completed **Level 3 — Backend Basics** — you have the `academy / Book` module.
 - Read the [Core module chapter](#) (signals, permissions, config), the [Calculation Rules chapter](#), and [Extending openIMIS](#).
 - Read the [Security chapter](#) for the rights model.
-

Briefing: the four extension seams

openIMIS lets a module be *both* extensible and permissioned without central coordination. Four mechanisms do the heavy lifting.



1. Rights-based permissions

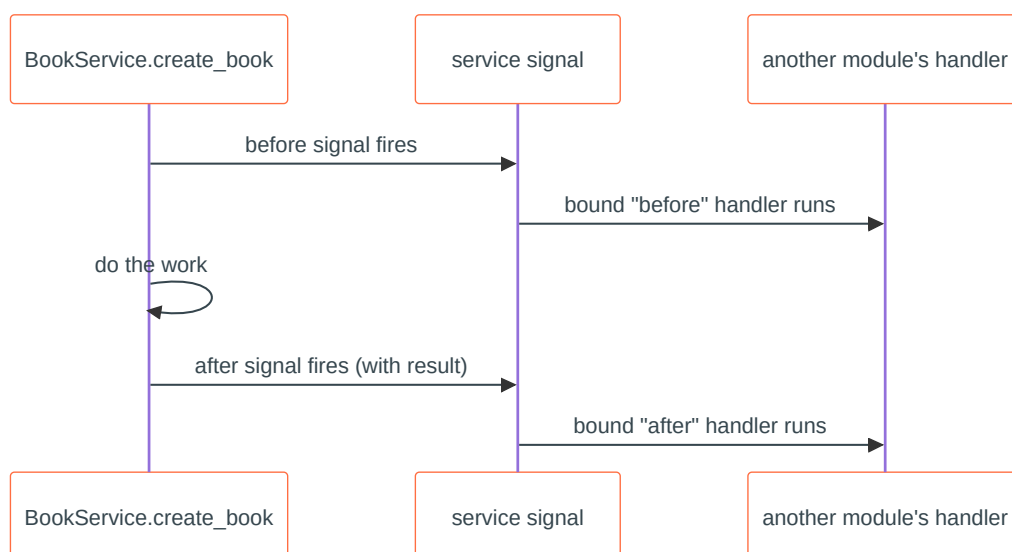
openIMIS permissions are **integers** defined in each module's `apps.py` (e.g. `gql_query_claims_perms = [111001]`). A logged-in `User` carries integer **rights** through their roles; a resolver or mutation checks `user.has_perms([<codes>])`. Details and the number ranges: [Security chapter](#) and [Core module](#).

2. DB-backed configuration

A module's `DEFAULT_CFG` dict is overlaid at startup by a per-module JSON stored in `ModuleConfiguration` (in core). Operators reconfigure deployments without code changes. You wired the skeleton of this in Level 3's `apps.py`; now you will *use* the loaded values.

3. Service signals — the core extension seam

`register_service_signal("module.service.method")` plus `bind_service_signal(...)` let a module hook **before or after** another module's service call *without importing it*. This is how modules extend each other safely.



Did you know?

Service signals are the backend twin of the frontend's `contributions` mechanism (Level 5). Both let one module inject behavior into another by **name**, never by import — that is what keeps ~47 modules from turning into a dependency knot.

4. Calculation rules

The `calculation` framework plus `calcrule_*` modules provide **pluggable pricing/valuation rules** registered via signals — used for contributions, capitation, third-party payment, and more. A rule advertises which entities it applies to and gets invoked when core asks "who can calculate this?". You will register a rule the same way. Full treatment: [Calculation Rules chapter](#).

Hands-on labs

All code is **illustrative** — confirm exact helper names/imports against `openimis-be-core_py` and `openimis-be-calculation_py`.

Lab 4.1 — Add permissions to the Book module

1. Assign real integer codes in `academy/apps.py`'s `DEFAULT_CFG` (pick an unused range for your teaching module):

```
# illustrative
DEFAULT_CFG = {
    "gql_query_books_perms": [990001],
    "gql_mutation_books_perms": [990002],
}
```

2. Guard the query resolver in `schema.py`:

```
# illustrative
from .apps import AcademyConfig

def resolve_books(self, info, **kwargs):
    user = info.context.user
    if not user.has_perms(AcademyConfig.gql_query_books_perms):
        raise PermissionError("Not allowed to view books")
    return Book.objects.filter(validity_to__isnull=True)
```

3. Guard `CreateBookMutation.async_mutate` similarly with `gql_mutation_books_perms`.
4. Log in as a user *without* the right and confirm the query is refused; grant the right to a role and confirm it works.

Done when: the same query succeeds or fails purely based on the user's rights.

Common mistake

Reading permission codes as *module-level constants captured at import time* can bite you: the authoritative values are set when config loads in `apps.ready()`. Reference them through the `AcademyConfig` class attributes (which `_configure_permissions` updates), not a stale local copy.

Lab 4.2 — Use DB-backed configuration

1. Add a functional config key to `DEFAULT_CFG`, e.g. `"max_copies_per_book": 100`.
2. In `BookService.create_book`, read it and enforce it:

```
# illustrative
from .apps import AcademyConfig
def create_book(self, data):
    if data.get("copies", 1) > AcademyConfig.max_copies_per_book:
        raise ValueError("Too many copies")
    return Book.objects.create(**data)
```

3. Load the key in `_configure_permissions` (or a sibling method) from `cfg`.
4. Change the stored `ModuleConfiguration` value to `5`, restart the backend, and confirm creating a 6-copy book now fails — **without editing code**.

Done when: a config change in the DB alters validation behavior after a restart.

Lab 4.3 — Emit and bind a service signal

1. Register a signal around your service. In `academy/signals.py`:

```
# illustrative
from core.signals import register_service_signal
```

and decorate/wrap `BookService.create_book` so a signal fires before and after it (follow the exact `register_service_signal` usage in `openimis-be-core.py`).

2. In a *second* module (or a `signals.py` in `academy` acting as the consumer), **bind** a handler:

```
# illustrative
from core.signals import bind_service_signal

def after_create_book(sender, result, **kwargs):
    # e.g. log, notify, or enrich – no import of BookService needed
    print("A book was created:", result)

bind_service_signal("academy_service.create_book", after_create_book, ...)
```

3. Create a book and confirm the bound handler runs, receiving the result — with no direct call between the two modules.

Done when: a handler in a different place reacts to `create_book` purely via the signal, not via an import.

Lab 4.4 — Register a calculation-rule-style extension

1. Read how a `calcrule_*` module advertises itself in [Calculation Rules](#) — a rule exposes metadata (which entity/context it applies to) and is discovered when core broadcasts a "who can calculate?" signal.
2. Create `calcrule_academy_late_fee` (illustrative) that computes a late fee for a book kept too long: it responds to the calculation broadcast, checks it applies to `Book`, and returns a value.
3. Trigger the calculation path and confirm your rule is selected and its value used.

Done when: your rule is discovered and invoked through the calculation framework's signal, not called directly.

Did you know?

A calculation rule and a service-signal handler are the same idea at different altitudes: both are *registered, then discovered by broadcast*. Calculation rules add a standard contract (metadata + `calculate`) so pricing logic is swappable per country without touching the modules that consume it.

Exercises

1. Give `academy` a *third* permission for deletion and guard `DeleteBookMutation`. Verify with two users.
2. Add a config key `default_author` and have `BookService` fill it when the client omits `author`.

3. Write a **before** service-signal handler that rejects creating a book whose title is blacklisted in config — without modifying `BookService`.
 4. Explain when you would use a plain Django `post_save` ORM signal versus a `register_service_signal`. Give one example of each.
 5. Sketch how the `calculation` framework decides *which* rule applies when several are registered.
-

Challenge project

Make `academy` a first-class, extensible module. Deliver:

- Three integer permissions (view / mutate / delete), all read from config and enforced in the schema layer.
- At least two functional config keys that change behavior at runtime (from the DB, no code edits).
- A `create_book` **service signal** with one bound consumer that reacts after creation (e.g. writes an audit note or fires a notification stub) — living outside `BookService`.
- One **before** handler that can *veto* a creation based on config.
- A `calcrule_academy_*` rule registered via the calculation framework and exercised end to end.
- A `README` "extension points" section documenting every seam you exposed, the way real module READMEs do.

Prove each seam independently: toggle a permission, change a config value, disable the signal consumer, and swap the calculation rule — showing behavior changes each time with the *rest of the module untouched*.

Knowledge check

 **Q1: How are openIMIS permissions represented and checked? (click for answer)** 

As **integer codes** defined in each module's `apps.py` (e.g. `gql_query_claims_perms = [111001]`). A `User` carries integer **rights** via their roles; resolvers and mutations enforce access with `user.has_perms([<codes>])`.

? Q2: Where does a module's runtime configuration ultimately come from? (click for answer) ✓

From the database. Each module ships a `DEFAULT_CFG` dict in `apps.py`, which is **overlaid at startup** by a per-module JSON row (`ModuleConfiguration` in core). Operators change deployment behavior by editing that row and restarting — no code changes.

? Q3: What problem do service signals solve that a direct function call does not? (click for answer) ✓

They let one module hook **before/after** another module's service *without importing it*. That decoupling is what lets ~47 independently versioned modules extend each other without creating hard dependencies or import cycles.

? Q4: When would you use a Django ORM signal (e.g. `post_save`) versus a service signal? (click for answer) ✓

Use an **ORM signal** to react to a low-level persistence event on a model regardless of who wrote it. Use a **service signal** to hook a specific business operation at the service boundary (with its validated inputs/result), which is the intended cross-module extension seam. Service signals express intent; ORM signals express persistence.

? Q5: How does a calculation rule get chosen at runtime? (click for answer) ✓

Rules are **registered via signals** and advertise metadata about which entity/context they apply to. When core broadcasts a "who can calculate this?" request, matching rules respond and the framework selects the applicable one and invokes its `calculate` — the consuming module never imports the rule directly.

Further reading

- [Core module chapter](#) — signals, permissions, config, `OpenIMISMutation`
- [Calculation Rules chapter](#) — the `calcrule_*` pattern
- [Security chapter](#) — rights, roles, permission ranges
- [Configuration chapter](#) — `ModuleConfiguration` in depth
- [Extending openIMIS](#)

Your backend is now a real extensible module. Give it a face in **Level 5 — Frontend**.

Level 5 — Frontend Customization

Level 5

Your `academy` module has a backend and a GraphQL API. Now give it a **React frontend** the openIMIS way — pages, components, forms, GraphQL-through-Redux (not Apollo), and menus and routes contributed through `ModulesManager` and published components.

Learning objectives

By the end of this level you will be able to:

- Scaffold a frontend module (`openimis-fe-academy_js`) and register it in the FE manifest.
- Explain the module **config object** keys: `translations`, `reducers`, `queries`, `mutations`, `menus`, `routes`, `contributions`, `refs`.
- Fetch data with the core `graphql` / `graphqlWithVariables` Redux actions — and understand why openIMIS is **not** Apollo.
- Handle the asynchronous mutation lifecycle with the "journalize"/polling helper.
- Contribute a menu entry and a route, and reuse another module's **published component** via `ModulesManager`.

Prerequisites

- Completed **Level 4 — Advanced Modules** — you have a permissioned `academy` backend.
- Read the [Frontend Architecture chapter](#).
- You know React and Redux. You do **not** need Apollo — openIMIS does not use it.

Briefing: the frontend is also assembled

The frontend mirrors the backend's plugin design. `openimis-fe_js` is the **assembly** React SPA (recently migrated to **Vite**); `openimis.json` is the FE **module manifest**, and `openimis-config-vite.js` generates the module import file (`src/modules.js`). Each module exports one **config object**.

Config key	What it contributes
<code>translations</code>	i18n messages (react-intl).
<code>reducers</code>	Redux reducers for the module's slice of state.
<code>queries / mutations</code>	GraphQL fragment/field lists used by the module's actions.
<code>refs / reference</code>	Published components/values other modules can look up by name.
<code>menus</code>	Menu entries injected into the navigation.
<code>routes</code>	React Router routes.
<code>contributions</code>	Named extension points — inject a component into e.g. <code>"insuree.MainMenu"</code> or <code>"core.AppBar"</code> .

ModulesManager — the cross-module registry

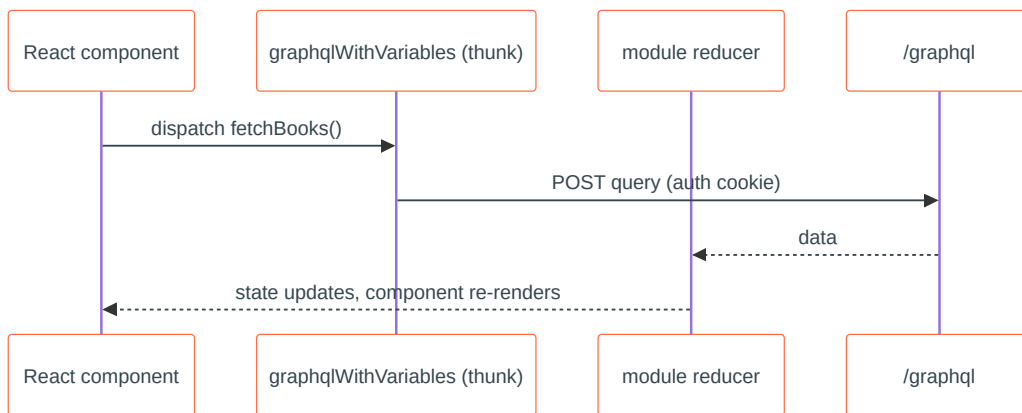
`ModulesManager` (from `core`) is how modules find each other's parts by name, never by import:

- `modulesManager.getRef("insuree.InsureePicker")` — fetch a published component.
- `modulesManager.getConf(...)` — read config.
- `modulesManager.getContribs("insuree.MainMenu")` — collect everything contributed to an extension point.

This is the frontend twin of backend **service signals** — extension by name, not by import. See [Frontend Architecture](#).

Why not Apollo?

openIMIS does GraphQL through a **custom Redux layer** in `core`, not Apollo:



For mutations, a **"journalize"/polling helper** watches the async `MutationLog` status (the backend pattern from Level 3) and updates Redux when the mutation resolves. So the frontend's async mutation UX is the mirror image of `OpenIMISMutation`.

i Did you know?

Because auth is a **HttpOnly cookie**, the frontend never handles a token. The Redux GraphQL actions just POST to `/graphql` and the cookie rides along — one reason openIMIS could skip Apollo's auth-link machinery.

Hands-on labs

All code is **illustrative** — confirm the exact action names, helper signatures, and config-object shape against `openimis-fe-core_js` and an existing module such as `openimis-fe-insuree_js`.

Lab 5.1 — Scaffold and register the FE module

1. Create the module folder the openIMIS way:

```
mkdir -p openimis-fe-academy_js/src
cd openimis-fe-academy_js
```

2. Create `src/index.js` exporting a config object:

```
// illustrative
import messages_en from "../translations/en.json";
import reducer from "../reducer";
import BooksPage from "../pages/BooksPage";

const ROUTE_BOOKS = "books";

const DEFAULT_CONFIG = {
  translations: [{ key: "en", messages: messages_en }],
  reducers: [{ key: "academy", reducer }],
  "core.Router": [{ path: ROUTE_BOOKS, component: BooksPage }],
  "core.MainMenu": [/* menu contribution, see Lab 5.4 */],
};
export const AcademyModule = (cfg) => ({ ...DEFAULT_CONFIG, ...cfg });
```

3. Register `academy` in the FE assembly `openimis-fe_js`'s `openimis.json`, then let `openimis-config-vite.js` regenerate `src/modules.js`.
4. Start the frontend dev server and confirm the module loads without console errors.

Done when: the module is imported into `src/modules.js` and the app boots.

Lab 5.2 — A Redux reducer and GraphQL fetch

1. Create `src/actions.js` using the core GraphQL Redux helpers:

```
// illustrative
import { graphqlWithVariables } from "@openimis/fe-core";

export function fetchBooks(params) {
  const payload = `
    query Books($search: String) {
      books(title_Icontains: $search) {
        totalCount
        edges { node { id title author copies } }
      }
    }`;
  return graphqlWithVariables(payload, { search: params.search },
    "ACADEMY_BOOKS");
}
```

2. Create `src/reducer.js` handling the `ACADEMY_BOOKS_REQ/RESP/ERR` actions the helper dispatches, storing `books`, `fetchingBooks`, and `errorBooks`.
3. Register the reducer in the config object (done in Lab 5.1).

Done when: dispatching `fetchBooks` populates your Redux slice with the backend data.

Lab 5.3 — A page, a list, and a form

1. Create `src/pages/BooksPage.js`: a container that dispatches `fetchBooks` on mount and reads `state.academy.books`.

2. Render the list with MUI components, wiring loading and error states from the reducer.
3. Add a `BookForm` component and a `createBook` action using the core mutation helper; on submit, dispatch the mutation and rely on the **journalize/polling** helper to reflect completion.

```
// illustrative
import { graphql } from "@openimis/fe-core";
export function createBook(book, clientMutationLabel) {
  const mutation = `
    mutation { createBook(input: {
      title: "${book.title}", author: "${book.author}", copies:
${book.copies},
      clientMutationLabel: "${clientMutationLabel}"
    }) { clientMutationId internalId } }`;
  return graphql(mutation, "ACADEMY_MUTATION_CREATE_BOOK",
    clientMutationLabel);
}
```

4. Create a book from the form and watch the list refresh once the async mutation resolves.

Done when: you can list and create books entirely from the UI.



Common mistake

Do not treat a mutation's immediate response as "the write is done". The response only gives you a `clientMutationId`; the write completes asynchronously on the backend. Use the `journalize/polling` helper (or re-fetch on completion) to know when to refresh — otherwise your list shows stale data.

Lab 5.4 — Contribute a menu and reuse a published component

1. Contribute a menu entry so `academy` appears in the navigation. Add to your config object a contribution to the main-menu extension point:

```
// illustrative
import { FormattedMessage } from "@openimis/fe-core";
"core.MainMenu": [{
  text: <FormattedMessage module="academy" id="menu.books" />,
  icon: "MenuBook",
  route: "/books",
}],
```

2. Guard the menu/route by the user's **rights** so only permitted users see it (mirror the Level 4 backend permissions).
3. Reuse a **published component** from another module via `ModulesManager` — for example, look up a picker or a searcher:

```
// illustrative
const Picker = props.modulesManager.getRef("core.SomePublishedComponent");
```

4. Confirm the menu appears (only for permitted users), the route loads `BooksPage`, and the reused component renders.

Done when: `academy` is reachable from the menu, route-guarded by rights, and reuses at least one published component.

Exercises

1. Add a search box to `BooksPage` that re-dispatches `fetchBooks` with `title_Contains`.
 2. Add French translations (`fr.json`) and switch the UI locale to verify react-intl picks them up.
 3. Publish one of your own components under a `refs` name and consume it via `modulesManager.getRef("academy.YourComponent")`.
 4. Explain, to a React dev used to Apollo, how openIMIS caches/stores query results instead (hint: Redux reducers).
 5. Trace the full path of "user clicks Create" → backend row created → list refreshed, naming every layer.
-

Challenge project

Ship the `academy` frontend module so the Book domain is fully usable in the UI:

- A registered FE module with `translations`, `reducers`, `routes`, and a `core.MainMenu` contribution.
- A `BooksPage` with a paginated, searchable list backed by `graphqlWithVariables` and a reducer.
- A `BookForm` for create and update, using the core mutation helper and the journalize/polling helper to refresh on completion.
- Route and menu **guarded by the user's rights**, consistent with the Level 4 backend permissions.
- At least one **published component** you expose via `refs`, plus one you **consume** from another module via `ModulesManager`.
- A `README` documenting the config-object keys you used and the extension points you exposed and consumed.

Prove it: log in as a permitted user and do a full create/search/update loop from the UI; log in as a user lacking the right and show the menu and route are hidden.

Knowledge check

? **Q1: openIMIS uses GraphQL but not Apollo. How does the frontend actually run queries? (click for answer)** ✓

Through a **custom Redux layer** in core: `graphql` / `graphqlWithVariables` action creators (thunks) POST to `/graphql`, and module **reducers** store the results in Redux state. There is no Apollo client or cache — state lives in Redux, and the `HttpOnly` auth cookie rides along automatically.

? **Q2: What does a frontend module's config object contribute, and name four keys. (click for answer)** ✓

It is how a module plugs into the assembly. Keys include `translations`, `reducers`, `queries`, `mutations`, `menus`, `routes`, `refs / reference`, and **contributions** (named extension points like `"insuree.MainMenu"` or `"core.AppBar"`).

? **Q3: What is `ModulesManager` and how is it like backend service signals? (click for answer)** ✓

It is core's cross-module registry: `getRef(name)` fetches a published component, `getContribs(point)` collects everything contributed to an extension point, `getConfig(...)` reads config. Like backend **service signals**, it lets modules find and extend each other **by name, never by import** — the same decoupling discipline on the frontend.

? **Q4: Why can't the UI assume a mutation is finished when the response arrives? (click for answer)** ✓

Because openIMIS mutations are **asynchronous** (`OpenIMISMutation`): the response only returns a `clientMutationId / internalId`; the actual work completes on the backend and is tracked in the `MutationLog`. The frontend uses a **journalize/polling** helper to detect completion before refreshing the view.



Q5: How do you make a menu item or route visible only to permitted users?
(click for answer)



Guard it by the logged-in user's integer **rights** — the same permission codes enforced on the backend (Level 4). Route guards and menu contributions check the user's rights, so the UI mirrors what the API would allow anyway.

Further reading

- [Frontend Architecture chapter](#) — Redux GraphQL, `ModulesManager`, contributions, Vite
- [GraphQL chapter](#) — the query/mutation model the FE consumes
- [Security chapter](#) — rights that guard routes and menus
- Reference module: [openimis-fe-insuree_js](#)

Your feature is now full-stack. Make it fast and production-ready in [Level 6 — Ops & Scaling](#).

Level 6 — Ops & Scaling

Advanced

A module that works on your laptop is not a module that survives a national deployment. This level is about **operating** openIMIS: finding and fixing performance problems, scaling the stack, caching, adding observability, and shipping a real production topology.

Learning objectives

By the end of this level you will be able to:

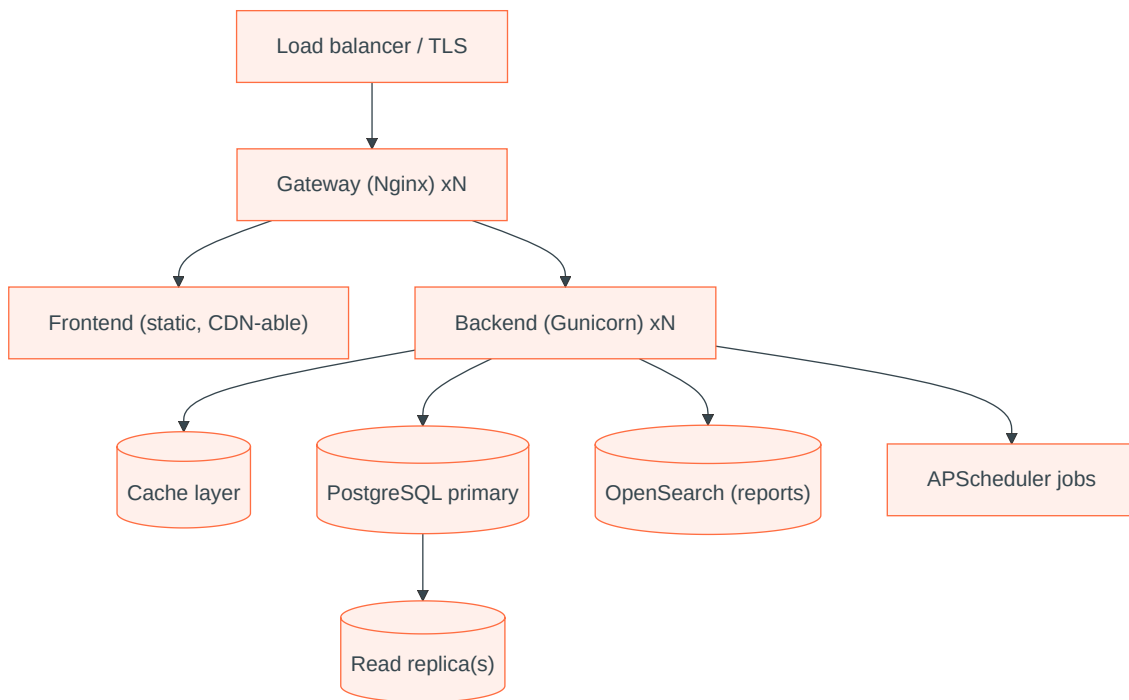
- Profile a slow GraphQL request and identify N+1 queries and missing indexes.
- Reason about **scaling** each service (`db` , `backend` , `frontend` , `gateway`) independently.
- Apply caching at the right layer and know what is *not* safe to cache.
- Add **observability**: structured logs, metrics, and request tracing across the gateway → GraphQL → service → DB path.
- Describe a production deployment topology and its failure modes.

Prerequisites

- Completed [Level 5 — Frontend](#) — you have a full-stack `academy` feature.
- Read the [Docker & Deployment chapter](#) and [Deployment & Operations](#).
- Read the [Database chapter](#) (indexes, temporal model) and the [Request Lifecycle](#).

Briefing: where openIMIS spends time and how it grows

A production deployment is the same four services from Level 2, but now each one is a scaling and failure boundary.



Key facts that shape ops decisions:

- The **frontend** is a static React build — cheap to scale, CDN-friendly, effectively stateless.
- The **backend** is Django/Gunicorn — scale horizontally by adding workers/replicas; it is stateless *except* for background jobs.
- **PostgreSQL** is the hard scaling boundary. The legacy-derived schema (temporal `validity_from/validity_to`, `tbl`-prefixed tables) means queries filter on validity constantly — index accordingly. See [Database chapter](#).
- **APScheduler** runs background tasks in-process; running multiple backend replicas naively can run a job *N times*. This is a classic ops trap.

The performance villain: N+1 queries

GraphQL makes N+1 easy: a `books` connection that resolves an author per node can fire one query per row. The fix is the same as any Django ORM N+1 —

`select_related/prefetch_related` in the **service or resolver** — but you have to see it first, which is what profiling is for. The [Request Lifecycle](#) shows where a resolver sits in the request.

Did you know?

openIMIS's temporal model means almost every query carries a `validity_to IS NULL` predicate to get "current" rows. Without an index that includes the validity columns, those filters table-scan large history tables. Composite indexes that match the common filter + validity pattern are often the single biggest win.



Common mistake

Running the APScheduler-backed jobs in every backend replica. If three replicas each start the scheduler, a nightly job runs three times — double-counting contributions, sending duplicate notifications, or corrupting aggregates. Run scheduled jobs in **one** designated instance (or an external scheduler), not in every worker.

Hands-on labs

Use your running stack from Level 2 and the `academy` feature from Levels 3–5. Seed enough data to make problems visible.

Lab 6.1 — Seed and profile

1. Seed a few thousand `Book` rows (a management command or a shell loop). Realistic volume makes real problems appear.
2. Enable SQL logging or attach Django Debug Toolbar / `django-silk` (dev only) and load `BooksPage` or run the `books` query.
3. Count the SQL queries for one page load. If resolving each node issues extra queries, you have found an **N+1**.

Done when: you can state, with a number, how many SQL queries one `books` request costs.

Lab 6.2 — Kill the N+1 and add an index

1. In `BookService.list_books`, add `select_related` / `prefetch_related` for any related fields the resolver touches; re-profile and confirm the query count drops.
2. Identify the predicate the `books` query filters on most (e.g. `title + validity_to IS NULL`). Add a matching migration index:

```
# illustrative
class Meta:
    indexes = [models.Index(fields=["title", "validity_to"])]
```

3. `makemigrations` / `migrate`, then `EXPLAIN ANALYZE` the underlying query before and after to prove the index is used.

Done when: query count is flat regardless of row count, and `EXPLAIN` shows an index scan instead of a sequential scan.

Lab 6.3 — Add a cache at the right layer

1. Choose a genuinely cacheable read — e.g. rarely-changing reference data (a `location` or `product` list), **not** per-user claim data.
2. Add caching (Django cache framework) around that read in its service, with a sane TTL and an invalidation hook on write.
3. Verify a cache hit avoids the DB, and that a write invalidates the entry.

Done when: a cacheable read is served from cache and correctly invalidated on change. Document *why* the data you chose is safe to cache.



Common mistake

Caching permissioned or temporal data by a key that ignores the user's rights or the validity window. You will serve one user another user's view, or stale "current" rows. Cache reference data, not authorization-sensitive results — and include validity/permission context in the key when in doubt.

Lab 6.4 — Add observability

1. Turn backend logs into **structured** logs (JSON) including a request/correlation id, the resolver name, and duration.
2. Propagate a correlation id from the gateway (Nginx) through to the backend so you can trace one request end to end across services.
3. Expose basic metrics — request rate, error rate, p95 latency, DB query counts. Wire them to whatever your environment uses (Prometheus/Grafana, cloud monitoring, etc.).
4. Load-test the `books` query (e.g. with `k6` / `hey`) and read your own dashboard to find the p95 and the bottleneck.

Done when: you can point at a single slow request in your logs and follow its correlation id from gateway to DB.

Lab 6.5 — Scale the backend safely

1. Run the backend with multiple Gunicorn workers/replicas.
2. Ensure scheduled jobs run in **exactly one** place (a dedicated worker/flag, or an external scheduler), not in every replica — verify a job fires once, not N times.
3. Confirm the frontend and gateway scale independently and that a backend restart is a rolling, non-breaking event behind the load balancer.

Done when: the backend runs multiple replicas, scheduled jobs run once, and a rolling backend restart does not drop the UI.

Exercises

1. For the `books` query, write the composite index you would add and justify the column order.
 2. Explain why the frontend is trivially scalable but PostgreSQL is not.
 3. List three reads in openIMIS that are safe to cache and three that are not — with reasons.
 4. Describe how you would prevent an APScheduler job from running in every replica.
 5. Define p50/p95/p99 latency and explain why p95 matters more than the average for user experience.
-

Challenge project

Produce an "operability report" for the `academy` feature proving it is production-ready:

- **Baseline:** query counts and p95 latency for `BooksPage` at 10, 1k, and 10k rows.
- **Fixes:** the N+1 removal and the index you added, each with before/after `EXPLAIN ANALYZE` and query-count evidence.
- **Caching:** one cacheable read with TTL, invalidation, and a note on why it is safe.
- **Observability:** a sample structured log line with a correlation id traced gateway → resolver → DB, plus a screenshot/description of your latency and error-rate metrics.
- **Scaling plan:** how many backend replicas, how the DB scales (replicas/pooling), how scheduled jobs are isolated, and the rolling-deploy story.
- **Failure modes:** what happens when the DB primary fails, when a backend replica dies mid-request, and when the cache is cold — and how the system degrades.

This report is a direct input to the Level 7 capstone's operations section.

Knowledge check

? Q1: Which service is the real scaling bottleneck, and why? (click for answer) ✓

PostgreSQL. The frontend is static and CDN-able; the backend (Django/Gunicorn) scales horizontally by adding stateless workers. The database is the shared source of truth — you scale it with indexing, connection pooling, read replicas, and query optimization, not by "adding a worker".

? Q2: Why are N+1 queries especially easy to introduce in GraphQL, and how do you fix them? (click for answer) ✓

Because a connection resolves fields per node, a related lookup can fire one query per row. You fix it the standard Django way — `select_related` / `prefetch_related` in the **service/resolver** — after *profiling* to see the extra queries (SQL logging, silk, debug toolbar).

? Q3: Why does openIMIS's temporal model matter for indexing? (click for answer) ✓

Almost every query filters `validity_to IS NULL` (plus `validity_from`) to select current rows from history tables. Without composite indexes that include the validity columns alongside the common filter columns, those predicates sequentially scan large tables. Matching indexes are often the biggest single performance win.

? Q4: What goes wrong if you run the APScheduler jobs in every backend replica? (click for answer) ✓

Each replica starts its own scheduler, so a job runs **once per replica** — duplicating notifications, double-counting aggregates, or corrupting data. Scheduled jobs must run in exactly one designated instance or via an external scheduler.

? Q5: Give an example of data that is safe to cache and one that is not. (click for answer) ✓

Safe: slowly-changing reference data like the `location` hierarchy or `product` definitions. **Not safe:** per-user, permissioned, or temporally-current data such as a specific insuree's claims — caching it risks serving stale or cross-user results unless the cache key includes user rights and validity context.

Further reading

- [Docker & Deployment](#) and [Deployment & Operations](#)
- [Database chapter](#) — indexing and the temporal model
- [Request Lifecycle](#) — where resolvers and services sit
- OpenSearch reporting: `openimis-be-opensearch_reports_py` on github.com/openimis

You can now operate openIMIS. Prove mastery in **Level 7 — Expert Capstone**.

Level 7 — Expert Capstone

Expert

This is the capstone. You will design a **complete new business module end to end** — not a teaching toy like `Book`, but a plausible openIMIS domain — and then defend it across four axes: **architecture, performance, security, and deployment**. Everything from Levels 1–6 converges here.

Learning objectives

By the end of this level you will be able to:

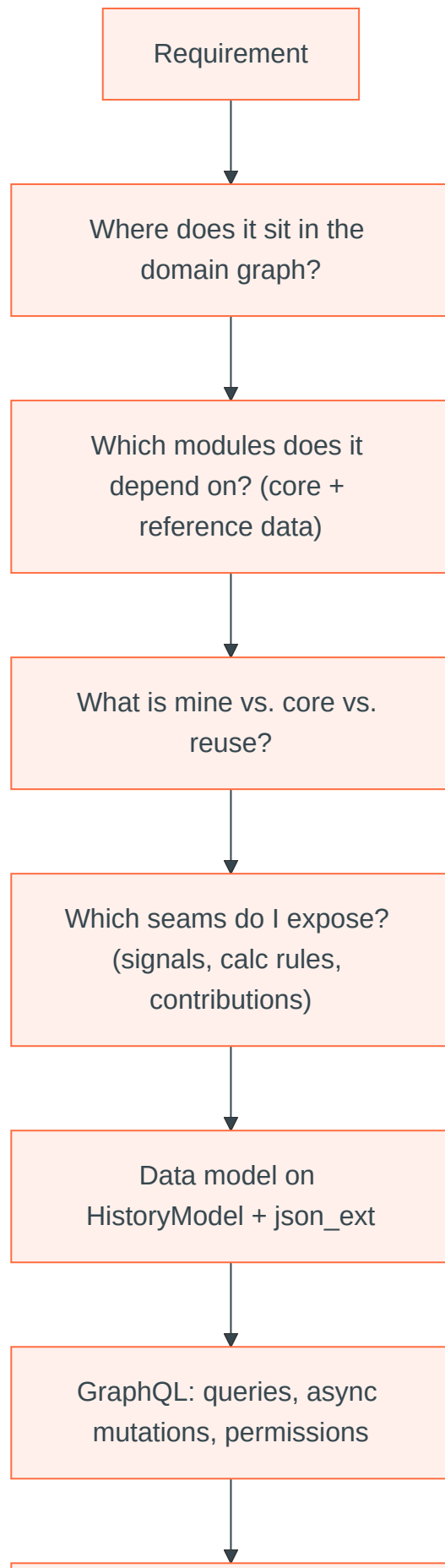
- Take a real health-financing requirement and decompose it into an openIMIS module with the right dependencies.
- Make and justify **architecture decisions**: what belongs in your module vs. core, which seams to expose, what to reuse.
- Build the module full-stack: models, services, GraphQL, permissions, signals, calculation rules, and a React frontend.
- Conduct a **performance review**, a **security review**, and design a **deployment strategy** for it.
- Present the whole thing as an Architecture Decision Record a maintainer could approve.

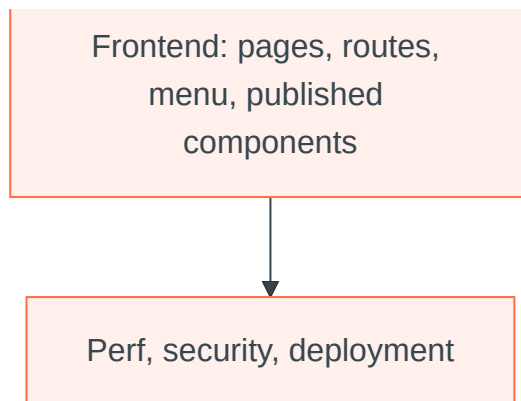
Prerequisites

- Completed [Level 6 — Ops & Scaling](#) — and, honestly, all prior levels.
- You should be comfortable across the whole handbook. The chapters you will lean on most: [Architecture Overview](#), [Plugin / Module System](#), [Core module](#), [GraphQL](#), [Security](#), [Database](#), [Docker & Deployment](#), and [Extending openIMIS](#).

Briefing: thinking like a module architect

Adding a module is easy; adding a *good* module is a design exercise. The maintainers' implicit checklist:





The design principles that separate a mergeable module from a fork:

Principle	In practice
Depend downward only	Build on <code>core</code> and reference modules; never create a cycle.
Reuse, don't reinvent	Use <code>HistoryModel</code> , <code>OpenIMISMutation</code> , <code>ExtendedConnection</code> , published FE components.
Extend by name	Expose service signals and FE contributions; consume others the same way.
Config over code	Deployment-varying behavior goes in <code>DEFAULT_CFG</code> / <code>ModuleConfiguration</code> .
Permission everything	Integer rights on every query and mutation, enforced on both tiers.
Audit everything	Async mutations + <code>MutationLog</code> ; temporal history via <code>validity_*</code> .

Did you know?

This is exactly how modules like `individual`, `social_protection`, and the `calcrule_*` family were added — new domains bolted onto `core` through its published seams, never by editing `core`. A capstone that respects these principles is, in principle, upstreamable.

The capstone challenge project

Pick **one** business domain (or propose your own of comparable scope). Each is a genuine gap a country deployment might fill:

Beneficiaries file grievances about rejected claims or enrolment; officers triage, respond, and resolve them within SLAs. Depends on `insuree / individual`, `claim`, `core`. Rich in workflow, permissions, and notifications.

Manage contracts between the scheme and health facilities: terms, validity periods, negotiated price lists, and contract-scoped claim rules. Depends on `location`, `product`, `claim`, `core`. Rich in temporal data and calculation rules.

Let insurees buy optional top-up coverage on top of a base policy, priced by a calculation rule and paid via contributions. Depends on `policy`, `product`, `contribution`, `calculation`, `core`. Rich in pricing and money flows.

Option A — Grievances & Appeals

Option B — Provider Contracting

Option C — Voluntary Top-up Products

Whichever you choose, deliver **all** of the following.

Part 1 — Architecture Decision Record

Write an ADR for the module. It must state:

- The requirement and the domain boundary (what is in scope, what is not).
- Its position in the [module dependency graph](#) and every module it depends on, with justification for each.
- What you build vs. what you reuse from `core` and other modules.
- The seams you expose (service signals, calculation rules, FE contributions, published components) and the seams you consume.
- At least three alternatives you rejected and why (e.g. "extend `claim` directly" vs. "new module").

Part 2 — Backend implementation

- Models on `HistoryModel` (with `json_ext` where extensibility helps), plus migrations and appropriate indexes (Level 6 discipline).

- Services as the sole ORM boundary.
- GraphQL: filterable/paginated queries with `ExtendedConnection`, and asynchronous `OpenIMISMutations` with a `MutationLog`.
- Integer **permissions** in `apps.py`, enforced in every resolver and mutation.
- At least one **service signal** you emit and one you bind, and — where the domain fits — a `calcrule_*`-style pluggable rule.
- DB-backed configuration for anything a deployment might vary.

Part 3 — Frontend implementation

- An `openimis-fe-<name>_js` module registered in the FE manifest, exporting a config object.
- A page with a paginated/searchable list and a create/update form, backed by the core `graphql / graphqlWithVariables` Redux actions and the journalize/polling helper.
- A menu entry and route, **guarded by rights** consistent with the backend.
- At least one component you publish via `refs` and one you consume via `ModulesManager`.

Part 4 — Reviews and strategy

- **Performance review:** the Level 6 treatment applied to your module — profiling, N+1 elimination, indexes, a caching decision, and a scaling note. Include before/after evidence.
 - **Security review:** see the checklist below.
 - **Deployment strategy:** how your module ships (manifest entry, migrations at startup, config seeding), a rollout/rollback plan, and its failure modes.
-

Hands-on labs

These labs *are* the capstone's spine — do them in order as you build.

Lab 7.1 — Domain decomposition

1. Write the requirement in three sentences. Circle every noun — those are candidate models. Circle every verb — those are candidate services/mutations.
2. Place your module on the dependency graph. Draw arrows only *downward* to existing modules. If you need an arrow upward or sideways into a peer, your boundary is wrong — redraw.

3. Decide, per noun, whether it is a new model, an extension of an existing one (`json_ext`), or a reference to another module's entity.

Done when: you have a one-page domain map with no dependency cycles.

Lab 7.2 — Build the vertical slice

1. Implement **one** end-to-end path first — one model, one query, one async mutation, one permission, one page — before breadth. Prove it works in GraphQL and the UI.
2. Only then widen to the full model set. A working thin slice de-risks everything after it.

Done when: a single feature works from React form → async mutation → `MutationLog` → DB → refreshed list.

Lab 7.3 — Security review

Run this checklist against your module (grounded in the [Security chapter](#)):

1. **AuthN:** every entry point requires an authenticated user; the JWT stays in the `HttpOnly` cookie; you never handle raw tokens.
2. **AuthZ:** every query and mutation checks `user.has_perms([...])` with the module's integer codes; the FE route/menu guards mirror them.
3. **Tenancy/scope:** results are scoped to what the user may see (e.g. by `location/officer`), not just "authenticated".
4. **Audit:** every write is an async mutation with a `MutationLog`; deletes are soft (validity window), preserving history.
5. **Input:** service-layer validation; no string-built SQL; GraphQL inputs typed and bounded.
6. **Secrets/config:** nothing sensitive in code or `DEFAULT_CFG`; secrets come from the environment (`.env`), not the repo.
7. **Data exposure:** the GraphQL types expose only intended fields; no accidental leak of internal or cross-user data through a connection.

Done when: you can answer every item with a file/line-of-reasoning reference, and you have fixed anything that failed.

Lab 7.4 — Deployment dry run

1. Register the module in the backend and frontend manifests; confirm it loads into `INSTALLED_APPS` and `src/modules.js`.
2. Confirm migrations run cleanly at container startup and that any required `ModuleConfiguration` is seeded.

3. Do a **rolling** backend restart behind the gateway and confirm no dropped requests; then practice a **rollback** (previous image + reverse migration story).

Done when: the module deploys, migrates, and rolls back without manual surgery.

Exercises

1. Justify, in two sentences, why your capstone is a *new module* rather than a patch to an existing one.
 2. Name the single most likely place your module introduces an N+1 and how you prevented it.
 3. Identify the one piece of behavior most likely to vary by country and show it living in config, not code.
 4. Which existing module's published component did you reuse, and what did reusing it save you?
 5. Describe the blast radius if your module's calculation rule threw an exception mid-calculation — and how you contained it.
-

Capstone acceptance criteria

Treat this as the maintainer's merge checklist. Your capstone passes when **all** are true:

- [] ADR present, with dependencies justified and rejected alternatives listed.
 - [] Backend: models on `HistoryModel`, services as ORM boundary, indexed migrations.
 - [] GraphQL: paginated queries (`ExtendedConnection`) and async `OpenIMISMutation`s, all permissioned.
 - [] At least one emitted and one bound **service signal**; a calc-rule where the domain fits.
 - [] Deployment-varying behavior in **DB-backed config**, not code.
 - [] Frontend: registered module, list + form, menu + route guarded by rights, one published and one consumed component.
 - [] Performance review with before/after evidence.
 - [] Security review checklist fully answered and remediated.
 - [] Deployment strategy with rollout, rollback, and failure modes.
 - [] No edits to `core` or peer modules — everything through published seams.
-

Knowledge check

? Q1: When should a new capability be a new module versus an extension of an existing one? (click for answer) ✓

New module when it is a distinct domain with its own models, permissions, and lifecycle, and it can depend *downward* on existing modules without editing them. Extend an existing one (often via `json_ext` or a service-signal handler) when the capability is an attribute or side-effect of an existing entity. The deciding test: can you build it without touching core or a peer? If yes, a clean new module is right.

? Q2: What does 'depend downward only' protect you from? (click for answer) ✓

Dependency cycles and un-mergeable coupling. If your module only depends on `core` and stable reference/domain modules beneath it — and extends peers via **signals/contributions** rather than imports — it can be versioned, deployed, and even upstreamed independently. Upward or sideways hard-dependencies create cycles and force forks.

? Q3: Name three things a security review must confirm for a new module. (click for answer) ✓

Any three of: every entry point is authenticated (JWT in HttpOnly cookie); every query/mutation enforces integer **rights** via `has_perms`; results are **scoped** to what the user may see; every write is audited via an async mutation + `MutationLog`; deletes are soft (validity window); inputs are validated with no string-built SQL; GraphQL types expose only intended fields; secrets come from the environment, not code/config.

? Q4: Where should country-specific pricing logic live, and why? (click for answer) ✓

In a **calculation rule** (`calcrule_*`) registered via signals, selected by metadata — plus any thresholds in **DB-backed configuration**. That keeps the consuming module ignorant of the specific formula, so a country can swap the rule or change config **without touching code** in the modules that use it.

 **Q5: Your capstone edits a file in `core` to make things work. What does that tell you? (click for answer)** 

That your design is wrong. Needing to edit core means you are missing (or ignoring) an extension seam — a service signal, a calculation-rule contract, a published component, or a config key. Re-derive the design so your module plugs into core's published seams; the whole point of openIMIS's architecture is that countries extend it **without forking core**.

Further reading

- [Architecture Overview](#) and [Plugin / Module System](#)
- [Extending openIMIS](#) and the [End-to-End Code Walkthrough](#)
- [Core module](#), [GraphQL](#), [Security](#), [Database](#)
- [Docker & Deployment](#) and [Deployment & Operations](#)
- [Architecture Critique](#) — where the maintainers themselves see room to improve
- All source repositories: github.com/openimis

You have completed the Academy. Return to the [Academy home](#) — or, better, take your capstone module and propose it upstream.

Architecture Critique

No architecture is free. Every design decision openIMIS made bought something and cost something, and a competent engineer evaluating the platform deserves an honest accounting of both sides of that ledger. This chapter is that accounting. It is deliberately balanced: openIMIS is a mature, thoughtfully engineered platform doing genuinely hard things, and it also carries real debt and real friction. We will name both, and — crucially — frame each weakness as a **tradeoff with a rationale**, because almost none of them are accidents.

We close with a **comparison** against seven other extensible platforms — Django Oscar, Saleor, ERPNext, OpenMRS, DHIS2, Odoo, and Medplum — so you can place openIMIS on the map of "big configurable domain platforms" and understand what it borrowed, what it invented, and where it sits.

Learning objectives

By the end of this chapter you will be able to:

- Articulate openIMIS's core architectural **strengths** and why they matter for its mission as a Digital Public Good.
- Name its principal **weaknesses** and explain the tradeoff each one represents, not merely that it is "bad".
- Distinguish **technical debt** (legacy artifacts to be paid down) from **deliberate design constraints** (things that look odd but are load-bearing).
- Reason about openIMIS's **scalability, maintainability, security, and developer experience** with specifics rather than vibes.
- **Compare** openIMIS to adjacent platforms and choose intelligently between them for a given problem.

Prerequisites

- [Architecture Overview](#) — you must already hold the "assembly repo + module manifest" mental model.
- [Plugin / Module System](#) — the plugin seams are the subject of most of the praise and most of the complaints below.
- [The Core Module](#) — `HistoryModel`, `OpenIMISMutation`, service signals, and the `User` model are all critiqued here.

- [GraphQL](#) — the async mutation pattern and schema assembly come up repeatedly.
- Helpful but optional: [Database](#), [Security](#), [Repository Map](#).

How to read a critique

A critique is not a verdict. The goal is not to decide whether openIMIS is "good" — it is demonstrably good enough that national health schemes run on it — but to make its tradeoffs *legible*, so you can extend it without fighting its grain, and evaluate it without either fanboyism or cynicism.

The shape of the assessment



Keep that shape in mind. The strengths and weaknesses are not independent lists — several weaknesses are the *shadow* cast by a strength. Modularity buys independence but costs onboarding. Temporal versioning buys auditability but costs query complexity. That coupling is the whole point of a tradeoff analysis.

Strengths

Modularity — the central bet, and it paid off

openIMIS's defining decision was to decompose a monolithic .NET application into roughly **47 independently versioned Django modules**, each in its own Git repository, assembled at startup by `openimis-be.py` (see [Plugin / Module System](#)). This is not cosmetic. It is what lets Tanzania, Nepal, Cameroon, and Chad each run different combinations of modules — and their own custom modules — **without forking core**.

Did you know?

The module manifest (`openimis.json`) is the load-bearing artifact of the whole platform. `openimis/settings.py` builds `INSTALLED_APPS` dynamically from it, and `openimis/schema.py` stitches the GraphQL schema from it. Change the manifest, and you change the application — no code edits required. That is the mechanism behind "config over fork."

Why it matters:

- **Independent release cadence.** A fix to `claim` does not force a release of `insuree`. Modules are pip-installable from pinned git refs.
- **Country customization is additive.** A deployment adds a module; it does not patch a shared file. Merge conflicts against upstream core effectively vanish.
- **The seams are explicit.** Service signals and the FE `contributions` registry give modules named, discoverable places to hook each other.

DPG governance — the architecture serves the mission

openIMIS is a recognized **Digital Public Good**, governed by the openIMIS Initiative with a community Technical Advisory Group and historically funded by SDC and GIZ. This is an architectural strength, not just a political one: the modular design *is* the governance model expressed in code. Because customization is additive, a national ministry can own its own modules while still consuming upstream security fixes to core. The commons stays coherent even as deployments diverge.

Config over fork — reconfigure a deployment without touching code

Each module ships a `DEFAULT_CFG` dict in its `apps.py`, which is overlaid at startup by a per-module JSON stored in the database (`ModuleConfiguration` in core). Operators retune permissions, feature flags, and business parameters against a running system. Combined with the modular manifest, this yields two independent axes of customization: **which modules run** (manifest) and **how each one behaves** (config). See [Configuration](#).

FHIR R4 — first-class health-domain interoperability

`openimis-be-api_fhir_r4.py` exposes REST FHIR endpoints under `/api_fhir_r4/`, mapping domain models to standard resources (Insuree → Patient, Policy → Coverage, Claim → Claim, HealthFacility → Location/Organization). For a health platform, speaking FHIR is table stakes for national interoperability, and openIMIS speaks it natively rather than through a bolt-on. See the [FHIR R4 module](#).



The strengths reinforce each other

Modularity makes FHIR a *module* you can include or exclude. DPG governance makes that module a shared asset. Config-over-fork lets each country map its own coding systems. The four strengths are one strength viewed from four angles: **customize without forking, forever.**

Weaknesses (as tradeoffs)

Each weakness below is presented as *tradeoff* → *rationale* → *cost you actually pay* → *how to live with it*. None of these should be read as "openIMIS is bad here." They are the bill for the strengths above.

Legacy database debt

The tradeoff. openIMIS inherited its schema from IMIS, a 15-year-old .NET + Microsoft SQL Server application. Many tables keep `tbl` prefixes and camelCase column names; the port went MSSQL → PostgreSQL.

Rationale. A greenfield schema would have meant abandoning years of production data and stored-procedure business logic that real schemes depended on. The migration prioritized **data continuity** over cleanliness — the correct call for a system holding live beneficiary records.

Cost you pay. ORM models carry `db_table / db_column` overrides; the schema reads inconsistently (new modules like `individual` are clean, old ones like the legacy `insuree / policy` tables are not); newcomers are surprised by column names. See [Database](#).

Dual keys — legacy integer IDs alongside UUIDs

The tradeoff. Legacy tables were keyed by integer IDs; openIMIS added UUIDs alongside them (the `legacy_id` field on versioned models points back at the old integer key).

Rationale. UUIDs are what you want for a distributed, multi-deployment DPG and for FHIR resource identity. But integer keys were embedded in existing data, reports, and foreign keys, so they could not simply be dropped.

Cost you pay. Two identity concepts coexist; you must know which one a given API or table expects. GraphQL exposes UUIDs; some legacy joins still lean on integers.



Common mistake

Do not assume the `id` you see in a GraphQL response is the same key used in a legacy join or a raw SQL report. openIMIS models frequently carry **both** a UUID and a legacy integer id (`legacy_id`). Confusing them silently returns the wrong row or an empty result. When in doubt, read the model in the module repo — do not guess.

Temporal versioning complexity

The tradeoff. Core's `VersionedModel / HistoryModel` give every business record `validity_from / validity_to` fields. A "record" is really a chain of time-sliced rows; the

"current" version is the one whose validity window is open.

Rationale. In social health protection, **history is not optional** — you must be able to answer "what was this insuree's coverage on the date of service?" for audit, claims adjudication, and dispute resolution. Temporal versioning bakes that into the data model rather than leaving it to application code.

Cost you pay. Every query must filter for validity, or it silently returns superseded rows. Updates create new versions rather than mutating in place. Joins across versioned tables multiply the where-clauses. This is genuinely the steepest conceptual hill for a Django engineer used to one-row-per-entity.



Deep dive: why 'just use django-simple-history' would not have worked



Off-the-shelf history libraries typically keep an audit *shadow table* while the main table stays single-row-per-entity. openIMIS instead makes validity a first-class part of the primary key space: the *live* table holds all versions, and business queries are expected to filter on the validity window. That is heavier, but it makes "as-of" queries a normal filter rather than a special audit-log lookup — essential when adjudicating a claim against the policy state *as it was on the service date*. The cost is that you cannot forget the filter, and the framework helpers (`filter_validity`) exist precisely because forgetting it is the number-one temporal bug. See [Database](#).

Async mutation polling ergonomics

The tradeoff. Every write goes through `OpenIMISMutation`: the mutation creates a `MutationLog`, kicks off work in a service, and returns a `clientMutationId`. The client then **polls** the mutation's status until it resolves.

Rationale. Writes in this domain can be slow (valuation, calculation rules, downstream effects) and **must be audited** — every mutation leaves a durable `MutationLog` row. The async pattern makes auditing universal and long-running work non-blocking. See [GraphQL](#).

Cost you pay. The developer experience of a write is heavier than a normal GraphQL mutation. You do not get the result inline; you get an id and must poll (the FE "journalize" helper does this for you). Error surfacing is indirect — failures land in the `MutationLog`, not in the immediate HTTP response. For simple synchronous writes this feels like ceremony.



This is the ergonomics complaint developers voice most

New contributors expect `mutation { createInsuree(...) { insuree { id } } }` to return the created insuree. It does not — it returns a `clientMutationId` you must poll on. This is intentional and audited, but it is the single biggest "why is this so indirect?" moment. Budget time for it.

Custom frontend GraphQL layer instead of Apollo

The tradeoff. The React frontend does **not** use Apollo Client. GraphQL runs through a custom Redux layer in FE core: `graphql` / `graphqlWithVariables` action creators plus a "journalize" polling helper that watches `MutationLog` status.

Rationale. The async, poll-based mutation model does not fit Apollo's optimistic-update / normalized-cache assumptions cleanly, and the platform predates the current Apollo ecosystem maturity. A custom layer gave precise control over the polling lifecycle and kept everything inside the existing Redux store the rest of the app already used. See [Frontend Architecture](#).

Cost you pay. You cannot lean on the enormous Apollo ecosystem — devtools, cache normalization, codegen, community answers. New frontend engineers must learn a bespoke data layer instead of a transferable industry-standard one. Caching and refetch logic are hand-rolled.

Heavy multiple-inheritance schema assembly

The tradeoff. `openimis/schema.py` builds the root `Query` and `Mutation` by importing each module's `schema.Query` / `schema.Mutation` and combining them via **Python multiple inheritance** into one class each.

Rationale. It is a strikingly simple mechanism: a module contributes to the API merely by defining `Query` / `Mutation` classes, and the assembly repo mixes them in. No registry boilerplate, no manual field wiring.

Cost you pay. The root schema is an MRO (method resolution order) of dozens of base classes. Name collisions between modules are resolved by inheritance order, which is implicit. Tracing "where does this GraphQL field come from?" means knowing the module list and the MRO, not grepping one file. It is elegant until it is confusing.



Deep dive: the MRO as an implicit API contract



Because the root `Query` inherits from every module's `Query`, two modules that define a resolver or field of the same name will silently shadow one another according to the order they appear in the assembly's base-class list. There is no compile-time collision error — Python just picks the first in the MRO. In practice modules namespace their fields to avoid this, but the *mechanism* offers no protection. This is the price of the "just define a class and it's wired" simplicity. Contrast with a registry pattern, which would surface a duplicate-registration error but require every module to call `register(...)`.

Steep onboarding

The tradeoff. There is no single repository you can clone and read like a normal Django project. The application only exists once the assembly repo has read the manifest and stitched ~47 modules together.

Rationale. This is the direct, unavoidable consequence of modularity. You cannot have both "one repo you can read top to bottom" and "47 independently versioned repos countries customize freely." openIMIS chose the latter because its mission requires it.

Cost you pay. Time-to-first-contribution is long. A newcomer must internalize the manifest, dynamic `INSTALLED_APPS`, schema assembly, service signals, the async mutation pattern, temporal versioning, dual keys, *and* the FE `ModulesManager` before they can confidently change anything. This handbook exists largely to shorten that curve.



The recurring pattern

Notice that six of the seven weaknesses are the **shadow of a strength**: legacy DB debt is the shadow of data continuity; dual keys of the UUID migration; temporal complexity of auditability; async ergonomics of universal auditing; the custom FE layer of the async model; onboarding of modularity. Only the multiple-inheritance assembly is a pure implementation choice you could imagine differently without giving anything up.

Technical debt

Debt is specifically the stuff that is *not* load-bearing — artifacts that could be paid down without changing the platform's identity.

Debt item	What it is	Why it persists	Interest you pay
<code>tbl</code> -prefixed / camelCase tables	Legacy MSSQL naming ported verbatim	Renaming means migrating live national data	Cognitive friction, inconsistent schema
Legacy stored-procedure heritage	Some business logic descended from SPs	Rewrites are risky against production schemes	Logic split between eras/conventions
Dual integer + UUID keys	Two identity systems coexist	Legacy FKs and reports embed integers	Ambiguity about which key an API wants
Mixed module maturity	New modules clean, old ones legacy-shaped	Modules evolve independently	Inconsistent conventions across the codebase
Hand-rolled FE GraphQL layer	Custom Redux data layer, no Apollo	Rewrite is large; current layer works	No ecosystem leverage; bespoke knowledge
Sparse per-module docs	Docs vary in depth per repo	Community capacity is finite	Source-diving required (this site helps)



Debt vs. design

Temporal versioning, service signals, and the async mutation pattern are **not debt** — they are deliberate, load-bearing design. Do not "refactor them away." The `tbl` prefixes and dual keys **are** debt — but debt that is rational to keep servicing rather than repay, because the principal (a live-data migration) is enormous.

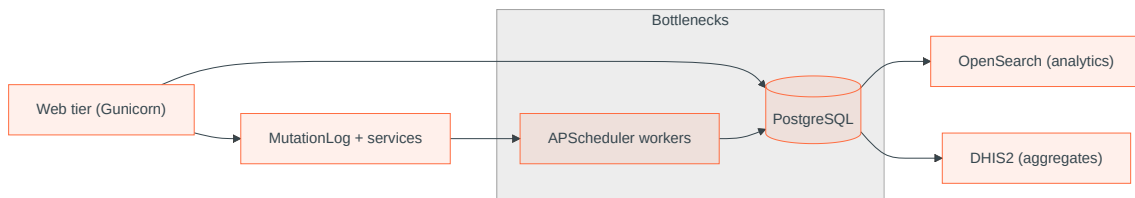
Scalability concerns

openIMIS is a fairly standard Django application at the request layer, so most Django scaling wisdom applies — but the domain adds specific pressure points.

- **Read amplification from temporal joins.** Validity-filtered queries across several versioned tables can generate wide, multi-condition SQL. Without the right indexes on

`validity_from` / `validity_to` and foreign keys, these degrade as history accumulates. See performance tips in [Best Practices](#).

- **Async mutation backlog.** The `MutationLog` + service model plus the scheduler (APScheduler) means write throughput is bounded by worker capacity, not just the web tier. Heavy claim batches (valuation via calculation rules) are the classic hotspot.
- **Single PostgreSQL by default.** The reference `openimis-dist_dkr` deployment runs one PostgreSQL. Read replicas, partitioning of the largest historical tables (claims), and connection pooling are deployment concerns you must add.
- **OpenSearch offloads analytics.** `opensearch_reports` exists precisely because heavy reporting on the transactional store does not scale — analytics are pushed to OpenSearch and aggregates to DHIS2.
- **Schema assembly is startup cost, not request cost.** The multiple-inheritance stitch happens once at boot; it does not tax individual requests.



⚠ The honest scaling summary

openIMIS scales like a well-built Django app with an audit-heavy write path and a history-heavy read path. It is not a horizontally-sharded, cloud-native system out of the box — nor does its target context (national schemes on modest infrastructure) usually demand that. Plan capacity around the **write/audit worker path** and **historical read indexing**, not the web tier.

Maintainability

In favor. Modularity localizes change: a bug lives in one module's repo, tested by that module's own test suite. Service signals decouple modules so you can alter one without importing it into another. `DEFAULT_CFG` centralizes tunables.

Against. The same modularity distributes knowledge across ~47 repos with uneven documentation and convention drift. Cross-cutting changes (touching core base models, say) ripple through everything that inherits them. The implicit MRO in schema assembly makes some "where does this come from?" questions genuinely hard.

The net is a platform that is **highly maintainable in the small** (fix one module) and **demanding in the large** (evolve a core contract). That is the correct optimization for a DPG:

most contributors touch one module; only the core team touches the framework, and they are the ones equipped to reason about the ripple.

Security considerations

See [Security](#) for the full treatment; the critique-level points:

- **Rights-based permissions are integer codes.** Every mutation/resolver enforces `user.has_perms([<int codes>])`, with codes defined per module in `apps.py` (e.g. `gql_query_claims_perms = [111001]`). This is uniform and auditable, but the codes are **opaque magic numbers** — a security reviewer cannot read intent from `[111001]` without a lookup. Trade: consistency and DB-driven role config vs. readability.
- **JWT in an HttpOnly cookie.** Auth uses `django-graphql-jwt` with the token in an HttpOnly cookie (mitigating XSS token theft), plus OIDC/OAuth2 for external IdPs. Cookie-based auth means **CSRF posture matters** — you must trust the gateway and CSRF config. See the cookie-issue troubleshooting in [Troubleshooting](#).
- **Universal audit.** The async `MutationLog` gives every write a durable audit trail essentially for free — a genuine security *strength* born from the ergonomically awkward async pattern.
- **Custom User composition.** The openIMIS `User` composes `InteractiveUser` (humans), `TechnicalUser` (service accounts), and `Officer / i_user`. Powerful and domain-accurate, but a non-standard auth surface reviewers must learn rather than assume Django defaults.



Security review pitfall

Because permissions are integer codes, it is easy to grant a role the wrong number and not notice — `111002` looks a lot like `111001`. Always resolve the code to its named permission in the owning module's `apps.py` during review. Never eyeball integer permission lists.

Developer experience

Dimension	Verdict	Detail
First-run setup	Good	<code>openimis-dist_dkr</code> boots the whole stack via docker-compose
Editable dev loop	Good	<code>pip install -e ../openimis-be-<mod>_py/</code> for hot module iteration
Discoverability	Mixed	Seams are explicit but spread across 47 repos; MRO is implicit
Writing a mutation	Heavy	Async <code>MutationLog</code> + polling is more ceremony than plain GraphQL
Frontend data layer	Bespoke	Custom Redux GraphQL, not Apollo — knowledge doesn't transfer out
Testing	Good	Per-module test suites + factories; run one module in isolation
Debugging	Mixed	<code>MutationLog</code> centralizes write errors, but they're indirect
Docs	Improving	Official wiki + per-repo READMEs of varying depth; this handbook

The honest DX summary: openIMIS rewards investment. The first month is steep — async mutations, temporal versioning, dual keys, MRO assembly, a bespoke FE layer. After that, the modularity that made onboarding hard makes *day-to-day work pleasant*: you work in one small, well-bounded module with its own tests, and the platform's seams do what you need. It is a platform optimized for the **committed contributor**, not the drive-by one — an appropriate optimization for infrastructure that countries run for a decade.

” One-line verdict

openIMIS is a principled, mission-fit architecture whose every rough edge is the price of a real capability — auditability, history, or customization — that its domain genuinely requires. Learn the grain and it works with you.

Comparison with adjacent platforms

openIMIS belongs to a family of **large, extensible, domain-specific platforms**. Comparing it to its neighbors clarifies what is distinctive. Two are e-commerce frameworks that pioneered "config over fork" (Oscar, Saleor); two are health platforms in its own domain (OpenMRS, Medplum); two are broad business/ERP platforms with mature plugin systems (ERPNext, Odoo); and one is the global health data platform openIMIS actually integrates with (DHIS2).

Platform	Domain	Stack	Extensibility model	GraphQL / API
openIMIS	Health insurance / social protection	Django + React	Modules assembled from manifest; service signals; config-over-fork	GraphQL (Graphene, async mutations) + REST/FHIR R4
Django Oscar	E-commerce	Django (no default FE)	Fork-the-app override: subclass and shadow core apps	REST (DRF) via add-ons
Saleor	E-commerce	Django + separate JS storefront	API-first headless; app/webhook extensions	GraphQL-first (native, sync)
ERPNext	ERP (accounting, HR, inventory)	Frappe (Python) + JS	Metadata-driven DocTypes; custom apps on Frappe	REST + RPC; some GraphQL community
OpenMRS	Clinical EMR	Java (Spring) + React (OWA/microfrontends)	OSGi-style modules; extension points	REST (FHIR module available)
DHIS2	Aggregate health data / HMIS	Java + React	App platform + metadata model	REST (rich), some GraphQL-ish query
Odoo	ERP / business apps	Python + OWL (JS)	Model inheritance (<code>_inherit</code>), addon modules	XML-RPC / JSON-RPC (no native GraphQL)

Platform	Domain	Stack	Extensibility model	GraphQL / API
Medplum	Healthcare backend / FHIR	TypeScript (Node) + React	FHIR-native data model; bots/subscriptions	FHIR REST + GraphQL native

Reading the table

Extensibility model — the core axis. openIMIS's "manifest of independently versioned modules + service signals + config-over-fork" is closest in *spirit* to **Odoo's** addon-and-inheritance model and **OpenMRS's** runtime module system, and closest in *governance* to **DHIS2** (both are DPGs serving national health systems). It differs sharply from **Django Oscar**, whose extensibility is fork-and-override: in Oscar you subclass and shadow core apps in your own project, whereas openIMIS keeps your customization in a *separate module* the manifest loads. openIMIS's approach scales to many independent customizers better; Oscar's is simpler for a single shop.

GraphQL / API. openIMIS and **Saleor** and **Medplum** are the GraphQL members of this family, but with a twist: Saleor and Medplum offer *synchronous, industry-standard* GraphQL (Medplum even exposes FHIR-native GraphQL), while openIMIS layers its **async, MutationLog -audited mutation pattern** on top of Graphene. That gives openIMIS universal write-auditing that the others lack by default, at the cost of the polling ergonomics discussed above. **OpenMRS**, **DHIS2**, **ERPNext**, and **Odoo** are REST/RPC-first, with FHIR as an add-on where health-relevant.

Domain fit. In its own lane, openIMIS's true peers are **OpenMRS**, **DHIS2**, and **Medplum**. But the fit is complementary, not competitive: OpenMRS/Medplum manage *clinical* records (the encounter, the patient chart); openIMIS manages *financing* (who is covered, what was claimed, who pays); DHIS2 aggregates *statistics*. A realistic national stack might run OpenMRS or Medplum for clinical data, openIMIS for insurance, and push aggregates to DHIS2 — which is exactly why openIMIS ships `dhis2_etl` and FHIR R4. See [Integrations](#).

Plugin mechanism, specifically. openIMIS's multiple-inheritance schema stitch is unusual. **Odoo** uses model inheritance (`_inherit`) pervasively, so it is the nearest analogue — Odoo developers will find openIMIS's "mix in every module's class" instinct familiar. **OpenMRS** (`.omod` modules) and **DHIS2** (installable apps) favor a more explicit runtime-registration model with clearer boundaries but more ceremony. **Saleor** and **Medplum** push extensions *out of process* (webhooks, bots) for isolation, trading in-process power for operational decoupling — the opposite of openIMIS's in-process, in-schema composition.

Maturity and target users. openIMIS is **mature within a narrow niche**: deep in health financing for LMICs, thin outside it. **Odoo**, **ERPNext**, and **DHIS2** are broader and larger.

Medplum is the newest and most modern-stack (TypeScript, FHIR-native) but health-tech-startup-oriented rather than ministry-oriented. If you want a modern greenfield healthcare backend, Medplum is the shiny option; if you must run a *national insurance scheme as a governed public good*, openIMIS's specific tradeoffs — auditability, temporal history, FHIR, config-over-fork, DPG governance — are exactly the ones you want, and no general platform matches them.

When would you pick openIMIS?

Pick openIMIS when the problem is **health financing / social protection at national scale**, you need **auditable history and FHIR interoperability**, and you want **many deployments to customize without forking a commons**. Pick a neighbor when your problem is clinical records (OpenMRS/Medplum), aggregate statistics (DHIS2), commerce (Oscar/Saleor), or general business/ERP (Odoo/ERPNext). The platforms are more often *complements* than substitutes.

Repository references

Repository	Directory / File	Why it matters
<code>openimis-be_py</code>	<code>openimis.json</code>	The module manifest — the artifact behind "config over fork" and modularity
<code>openimis-be_py</code>	<code>openimis/schema.py</code>	The multiple-inheritance schema assembly critiqued above
<code>openimis-be_py</code>	<code>openimis/settings.py</code>	Dynamic <code>INSTALLED_APPS</code> — modularity made concrete
<code>openimis-be-core_py</code>	<code>core/models.py</code>	<code>HistoryModel</code> / <code>VersionedModel</code> — temporal versioning, dual keys, <code>legacy_id</code>
<code>openimis-be-core_py</code>	<code>core/schema.py</code>	<code>OpenIMISMutation</code> — the async, audited mutation pattern
<code>openimis-be-core_py</code>	<code>core/apps.py</code>	<code>DEFAULT_CFG</code> , integer permission codes, <code>_configure_permissions()</code>
<code>openimis-fe_js</code>	FE core <code>graphql/graphqlWithVariables</code> + <code>journalize</code>	The custom, non-Apollo GraphQL layer
<code>openimis-be-api_fhir_r4_py</code>	<code>api_fhir_r4/</code>	FHIR R4 mapping — the interoperability strength
<code>openimis-dist_dkr</code>	<code>docker-compose.yml</code>	Reference deployment — the scalability baseline

Knowledge check

? Q1: Name two weaknesses that are the direct 'shadow' of a strength, and state the strength each casts. (click for answer) ✓

Examples: **onboarding difficulty** is the shadow of **modularity** (you can't have 47 independently customizable repos *and* one readable repo); **async mutation polling ergonomics** is the shadow of **universal write auditing** (the `MutationLog` that makes writes awkward is the same thing that audits every write); **temporal query complexity** is the shadow of **auditable history**; **dual keys** are the shadow of the **UUID migration** needed for a distributed DPG. Any two with correct pairing.

? Q2: Why does openIMIS not use Apollo Client on the frontend? (click for answer) ✓

Because its **async, poll-based mutation model** (return a `clientMutationId`, then poll `MutationLog` status) does not fit Apollo's optimistic-update / normalized-cache assumptions cleanly, and the platform predates current Apollo maturity. A custom Redux layer (`graphql / graphqlWithVariables` + a journalize polling helper) gave precise control over the polling lifecycle inside the existing store. The cost is losing the Apollo ecosystem.

? Q3: Which two platforms in the comparison are openIMIS's closest analogues for its *plugin mechanism*, and why? (click for answer) ✓

Odoo (pervasive model inheritance via `_inherit`, mirroring openIMIS's "mix in every module's class" multiple-inheritance stitch) and **OpenMRS** (a runtime module system for a health platform, though more explicit and ceremony-heavy). DHIS2 is the closest *governance* analogue (fellow health DPG) but uses installable apps rather than in-process class composition.

? Q4: A reviewer sees a role granted permission `[111002]`. What is the critique-level concern and the correct action? (click for answer) ✓

Permissions are **opaque integer codes**; `111002` is easy to confuse with a neighbor like `111001`, and intent is not readable from the number. The correct action is to **resolve the code to its named permission in the owning module's `apps.py`** rather than eyeballing it. This is the readability tradeoff of the otherwise-uniform, DB-configurable rights system.

 **Q5: You need a national stack that manages clinical encounters, insurance coverage, and statistical reporting. How do openIMIS and its neighbors fit together? (click for answer)** 

They are **complements, not substitutes**: use **OpenMRS** or **Medplum** for clinical records (patient chart, encounters), **openIMIS** for insurance financing (coverage, claims, provider payment), and push aggregates to **DHIS2** for statistics. openIMIS ships `dhis2_etl` and FHIR R4 precisely to sit in this ecosystem. See [Integrations](#).

Further reading

- [Architecture Overview](#) and [Plugin / Module System](#) — the strengths in depth.
- [Best Practices](#) — how to work *with* the grain the critique describes.
- [Database](#) — the temporal model and legacy schema in full.
- [Security](#) — rights, JWT cookies, and the audit trail.
- Official openIMIS wiki: openimis.atlassian.net/wiki
- Peer platforms: [Saleor](#), [Medplum](#), [OpenMRS](#), [DHIS2](#), [Odoo](#), [ERPNext](#), [Django Oscar](#).

Best Practices

This chapter is the field manual for working *with* openIMIS's grain rather than against it. The [Architecture Critique](#) explained why the platform is shaped the way it is; this chapter turns that understanding into concrete habits: how to format code, how to structure a change, how to test it, how to keep it fast, and which well-worn traps to sidestep. Treat it as a checklist you return to, not a one-time read.

Learning objectives

By the end of this chapter you will be able to:

- Apply openIMIS's **coding standards** on both backend (black/flake8/isort) and frontend (eslint/prettier).
- Follow the platform's core **architectural patterns**: the service layer, service signals over hard imports, config over fork, and base-model reuse.
- Structure a change through the recommended **workflow**, from branch to PR.
- Write **tests** the openIMIS way: per-module suites and factories.
- Diagnose and avoid the classic **performance** traps (N+1, missing validity filters, absent indexes).
- Recognize and fix the most **common pitfalls** from a lookup table.

Prerequisites

- [The Core Module](#) — services, signals, base models.
- [Architecture: Backend Deep Dive](#) — module layout.
- [GraphQL](#) — the async mutation pattern you will be testing.
- [Database](#) — validity filtering and indexes.
- Recommended: [Architecture Critique](#) — the *why* behind the *how* below.

The one principle under everything

Almost every best practice below reduces to one idea: **extend at the seams, never through the middle**. openIMIS gives you explicit extension points — modules, service signals, `contributions`, `DEFAULT_CFG`. Use them. The moment you reach past a seam (hard-importing another module, patching core, mutating a versioned row in place) you have signed up for pain.

Coding standards

Backend — black, flake8, isort

openIMIS backend modules follow standard Python tooling. Configure your editor to run these on save; configure CI to reject anything that fails.

Tool	Role	Typical invocation
black	Opinionated formatter — settles all whitespace/quote debates	<code>black</code> <code><package>/</code>
isort	Sorts and groups imports deterministically	<code>isort</code> <code><package>/</code>
flake8	Lints for unused names, undefined refs, style violations	<code>flake8</code> <code><package>/</code>

```
# Illustrative pre-commit sweep for a single module
cd ../openimis-be-claim_py
isort claim/
black claim/
flake8 claim/
python -m pytest claim/tests/
```



Order matters

Run **isort** → **black** → **flake8**. isort and black can disagree if run in the wrong order (isort reorders imports, black may reformat them); running isort first, black second, then linting last avoids the churn. A `pre-commit` config that pins this order is the cleanest setup.

Did you know?

black is *intentionally* unconfigurable about most things. That is a feature: across ~47 module repos maintained by different teams, an opinionated formatter means every file reads the same regardless of author. Do not fight it with `# fmt: off` except for genuinely tabular data.

Frontend — eslint, prettier

The React frontend (`openimis-fe_js` and each `openimis-fe-<name>_js` module) uses ESLint for correctness and Prettier for formatting.

Tool	Role
prettier	Formats JS/JSX/JSON/CSS — the black of the frontend
eslint	Catches bugs and enforces React/hooks rules

```
# Illustrative frontend checks
npx prettier --write "src/**/*.{js,jsx}"
npx eslint "src/**/*.{js,jsx}"
```



Common mistake

Do not disable ESLint's `react-hooks/exhaustive-deps` wholesale to silence warnings. In the openIMIS FE, effects that fetch GraphQL and dispatch Redux actions are exactly where a missing dependency causes stale data or infinite refetch loops. Fix the dependency array; don't mute the rule.

Architectural patterns

The service layer — business logic lives in services, not resolvers

Keep GraphQL mutations/resolvers thin. Their job is auth, input marshalling, and delegation. The actual work belongs in a module's `services.py`, in a service class that can be called from GraphQL, REST/FHIR, a management command, or a test — without any of them knowing about the others.

```
# Illustrative – the shape, not the exact code
class ClaimService:
    def __init__(self, user):
        self.user = user

    def create(self, data):
        self.validate(data)          # business rules here
        claim = Claim.objects.create(**data)
        # trigger valuation, signals, etc.
        return claim
```

```
# The mutation stays thin (illustrative)
class CreateClaimMutation(OpenIMISMutation):
    @classmethod
    def async_mutate(cls, user, **data):
        if not user.has_perms(ClaimConfig.gql_mutation_create_claims_perms):
            raise PermissionDenied("unauthorized")
        ClaimService(user).create(data)  # delegate
```

Why the service layer earns its keep

Because REST/FHIR (`api_fhir_r4`), GraphQL, and background jobs all need the same business logic, putting it in a service is what stops it from being copied three times and drifting. When you fix a validation rule, you fix it once.

Service signals over hard imports — the extension seam

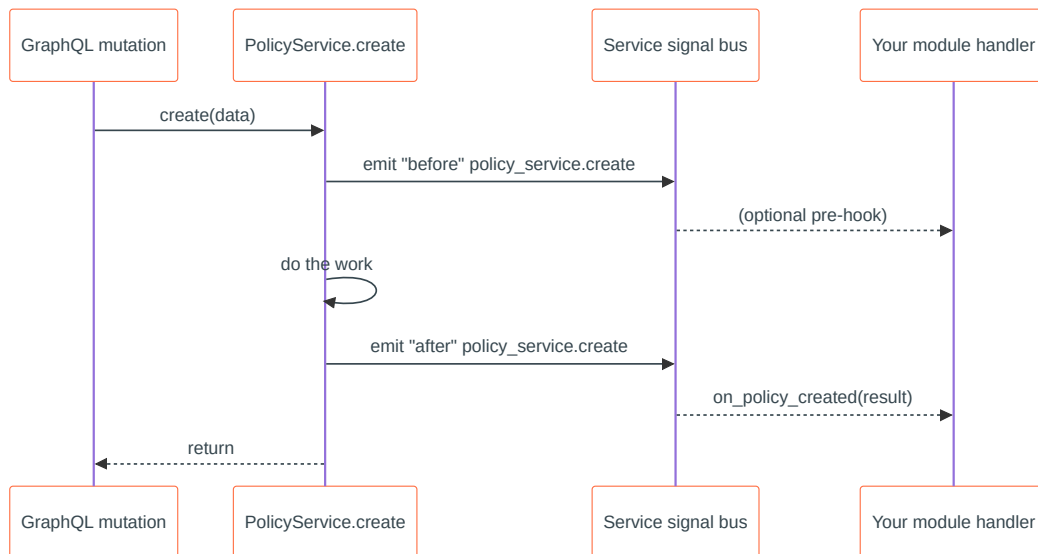
When your module needs to react to *another* module's operation, do **not** import that module's service and call it. Bind to its **service signal**. Core's

`register_service_signal("module.service.method") + bind_service_signal(...)` let you run logic *before/after* another service's call without a code dependency on it. See [The Core Module](#).

```
# Illustrative – react after a policy is created, from a different module
from core.signals import bind_service_signal

def on_policy_created(sender, result=None, **kwargs):
    # do your module's follow-up work
    ...

bind_service_signal(
    "policy_service.create",
    on_policy_created,
    bind_type="after",  # illustrative kwarg
)
```

⚡ Common mistake

`from policy.services import PolicyService` inside an unrelated module creates a hard dependency that (a) breaks if `policy` is not in the deployment's manifest, and (b) welds two modules together so neither can be released independently. That defeats the entire modular architecture. **Bind a signal instead.**

Config over fork — never patch core to change behavior

If you need different behavior, reach for the two customization axes first: change **which modules run** (the manifest) or **how a module behaves** (its `DEFAULT_CFG`, overridden by the `DB ModuleConfiguration`). Forking core or copy-editing another module's source is almost always wrong. See [Configuration](#).

You want to...	Wrong way	Right way
Change a business parameter	Edit the module's source	Override its config in <code>ModuleConfiguration</code>
Add a feature to the domain	Patch an existing module	Ship a new module in the manifest
React to another module	Hard-import its service	Bind a service signal
Change a permission	Hardcode a role check	Configure integer rights on the role
Inject FE UI	Fork the FE module	Register a contribution at a named point

Base models — inherit, don't reinvent

Core provides `HistoryModel`, `HistoryBusinessModel`, `VersionedModel`, `UUIDModel`, and the `json_ext` `JSONField`. New business entities should inherit the right base so they get temporal versioning, UUIDs, `legacy_id`, and extensibility for free — consistent with every other module.

Reach for `json_ext` before a schema migration

When a country needs an extra field that is not core to the domain, prefer the `json_ext` `JSONField` over adding a column. It keeps deployments interoperable (no divergent schemas) and is exactly what the field exists for. Add a real column only when the data is queried/indexed heavily. See [Database](#).

Recommended workflow



1. **Work in the owning module's repo**, not the assembly repo. A change to claims logic is a PR to `openimis-be-claim_py`, not to `openimis-be_py`.
2. **Use editable installs** for the dev loop: `pip install -e ../openimis-be-claim_py/` so edits are live in the running assembly.

3. **Branch, never commit to the default branch.** One change, one branch, one PR.
4. **Service and tests first, API last** — so the logic is validated independently of GraphQL's async ceremony.
5. **Update the manifest pin** (`openimis.json`) only when the deployable assembly must move to your new module version.

Editable installs are the single biggest DX win

Without `pip install -e`, every module edit means reinstalling from git — a miserable loop. With it, the assembly repo imports your working tree directly. Set this up on day one.

Testing practices

Per-module test suites

Each module owns its tests under `<name>/tests/`. Run a module's suite in isolation — you do not need the whole platform to test one module's logic.

```
# Illustrative – run just the insuree module's tests
cd ../openimis-be-insuree_py
python -m pytest insuree/tests/
```



Factories over fixtures

Prefer factory functions/classes that build valid domain objects (an insuree, a policy, a product) over brittle JSON fixtures. Factories keep tests readable and survive schema evolution far better than frozen fixture files.

```
# Illustrative factory usage
insuree = create_test_insuree()           # sensible valid defaults
policy = create_test_policy(insuree=insuree, product=create_test_product())
```



Test the service, then the mutation

Write most of your assertions against the **service** (synchronous, direct). Add a thinner set of tests for the **mutation** to confirm auth and the async `MutationLog` wiring. This mirrors the service-layer pattern and keeps you from fighting polling in every test.



Remember validity in test assertions

When asserting that an update happened, remember it created a **new version** rather than mutating the old row. Query with a validity filter (or the framework's `filter_validity` helper) or you may assert against a superseded version and get confusing failures. See the temporal model in [Database](#).

Performance tips

The [critique's scalability section](#) named the hotspots; here is how to avoid them in code.

Technique	When	Why
<code>select_related(...)</code>	Following a ForeignKey/OneToOne you will read	One join instead of one query per row
<code>prefetch_related(...)</code>	Following a reverse/many relation	Batches the related fetch, kills N+1
Validity filter (<code>filter_validity</code>)	Every query on a versioned model	Excludes superseded rows; also prunes the scan
Indexes on <code>validity_from/validity_to</code> + FKs	Large historical tables (claims)	Keeps temporal joins from degrading as history grows
<code>.only()</code> / <code>.values()</code>	Wide tables, list views	Fetch only the columns you render
Push analytics to OpenSearch	Heavy reporting	Keep the transactional DB out of report scans

The N+1 trap, concretely

```
# BAD (illustrative) – one query per insuree to get its family
for insuree in Insuree.objects.filter(*filter_validity()):
    print(insuree.family.location.name)  # N+1: family, then location, per
row

# GOOD
qs =
Insuree.objects.filter(*filter_validity()).select_related("family__location")
for insuree in qs:
    print(insuree.family.location.name)  # one query
```



Common mistake — the silent missing validity filter

A query on a versioned model *without* a validity filter returns **every historical version** of every record, not the current ones. It usually "works" in dev with little history and then returns duplicates and wrong counts in production. Make `filter_validity()` a reflex on every versioned query. This is the most common performance *and* correctness bug in openIMIS.

Debugging

- **Writes fail silently?** Look in the `MutationLog`, not the HTTP response. The async mutation pattern records errors there. Query the log by `clientMutationId`. See [GraphQL](#).
- **A GraphQL field seems to come from nowhere?** It is contributed by a module's `schema.Query / Mutation` and mixed into the root via multiple inheritance in `openimis/schema.py`. Check the module list and the MRO, not one file. See [Repository Map](#).
- **Permission denied with a valid user?** Resolve the integer permission code to its name in the owning module's `apps.py` and confirm the user's role carries that exact code. Off-by-one integer codes are a classic. See [Security](#).
- **Empty query results after an update?** Suspect a missing validity filter or an assertion against a superseded version.
- **Frontend data stale or looping?** Check the effect's dependency array and the journalize polling — not Apollo (there is none).
- **Enable SQL logging** to hunt N+1: turn on Django's `django.db.backends` logger in dev and watch the query count per request.

Common pitfalls

Pitfall	Symptom	Fix
Missing validity filter	Duplicate/old rows, wrong counts	Add <code>filter_validity()</code> to every versioned query
Hard-importing another module	Deployment breaks when that module is absent	Bind a service signal instead
Expecting a mutation to return the object	Only get a <code>clientMutationId</code>	Poll <code>MutationLog</code> / use the FE journalize helper
Confusing UUID and legacy integer id	Wrong row or empty result on a join	Read the model; know which key the API/join expects
N+1 on relations	Slow list views, query storms	<code>select_related</code> / <code>prefetch_related</code>
Patching core to change behavior	Merge hell on upstream updates	Config override or a new module (config over fork)
Eyeballing integer permission codes	Wrong role grants, silent auth holes	Resolve codes to names in <code>apps.py</code> before granting
Mutating a versioned row in place	Lost history, broken as-of queries	Create a new version; let validity windows manage state
Adding a column for a country-specific field	Divergent schemas, interop breakage	Prefer <code>json_ext</code> unless heavily queried
Editing the assembly repo for module logic	Change in the wrong repo, no module release	PR the owning module repo; bump the manifest pin
Running isort after black	Formatter churn / CI flip-flop	isort → black → flake8, in that order

Contribution guidelines



A well-formed openIMIS contribution

1. **Right repo.** The change lives in the module it belongs to (`openimis-be-<name>.py` / `openimis-fe-<name>.js`), not the assembly.
2. **Branch + PR.** Never commit to a default branch; one focused PR.
3. **Tests included.** New logic ships with per-module tests using factories; the module suite passes.
4. **Formatted + linted.** `isort/black/flake8` (backend) or `prettier/eslint` (frontend) all clean.
5. **Seams, not surgery.** Extends via service signals / `contributions` / config — no hard cross-module imports, no core patches.
6. **Config-driven.** New tunables are added to `DEFAULT_CFG` , not hardcoded.
7. **History-safe.** Versioned models are updated by new versions, and queries carry validity filters.
8. **Docs & config keys noted.** Any new `DEFAULT_CFG` key or permission code is documented in the PR.



Where to coordinate

Substantial changes — especially anything touching core base models or the schema assembly — should be discussed with the community first (the openIMIS Technical Advisory Group and the relevant repo maintainers). Core is load-bearing for every deployment; changes there ripple through everything that inherits from it. See the [Architecture Critique](#) on maintainability-in-the-large.

Hands-on lab



Lab: harden a module change end to end

Pick any backend module (`insuree` is a good choice) and:

1. `pip install -e` it into a running assembly.
2. Add a trivial method to its service (e.g. a derived read helper) plus a per-module test using a factory.
3. Deliberately write a query on a versioned model **without** a validity filter, observe the extra rows, then fix it.
4. Introduce an N+1 (loop over a relation), turn on SQL logging, count the queries, then collapse it with `select_related`.
5. Run `isort` → `black` → `flake8` and the module's test suite until green.
6. Write the PR description against the contribution checklist above.

You will have exercised the service layer, factories, the validity trap, the N+1 fix, the tooling order, and the contribution flow — the whole chapter in one change.

Exercises

1. Rewrite a hypothetical hard import (`from policy.services import PolicyService`) as a service-signal binding. Which `bind_type` do you need if you must run *before* the policy is saved?
2. Given a country that needs one extra optional field on `Insuree`, argue for `json_ext` vs. a real column. Under what condition does the column win?
3. Sketch the minimal set of tests for a new mutation: which assertions go against the service, which against the mutation, and why?

Knowledge check



Q1: Why bind a service signal instead of importing another module's service?
(click for answer)



A hard import creates a code dependency that breaks if that module is absent from the deployment's manifest and welds the two modules together so neither can release independently — defeating the modular architecture. A service signal lets you run before/after another service's call with no import and no dependency.

? **Q2: What is the correct tool order for backend formatting/linting, and why? (click for answer)** ✓

isort → **black** → **flake8**. isort reorders imports first, black then settles formatting (including on those imports), and flake8 lints last. Reversing isort and black causes formatter churn where the two disagree and CI flip-flops.

? **Q3: Your list view is slow and SQL logging shows one query per row for a related object. What is the fix, and which method for a forward FK vs. a reverse/many relation? (click for answer)** ✓

It's an N+1. Use `select_related` for forward ForeignKey/OneToOne relations (SQL join), and `prefetch_related` for reverse or many-to-many relations (a batched second query). Also confirm the base query still carries its validity filter.

? **Q4: Where do errors from a failed mutation actually surface, and how do you find them? (click for answer)** ✓

In the `MutationLog`, not the immediate HTTP/GraphQL response — the async mutation pattern records status and errors there. Find them by the `clientMutationId` the mutation returned (the FE journalize helper polls this automatically).

? **Q5: A country needs a business parameter changed. Rank the options from best to worst. (click for answer)** ✓

Best: **override the module's config** via `ModuleConfiguration` (config over fork) — no code change, no fork. Next: if it's genuinely new behavior, **ship a new module**. Worst: **patch the module's source or fork core**, which creates permanent merge pain against upstream and breaks the commons model.

Further reading

- [Architecture Critique](#) — the rationale behind these rules.
- [The Core Module](#) — services, signals, base models in depth.
- [GraphQL](#) — the async mutation pattern you test around.
- [Database](#) — validity filtering and indexing.
- [Configuration](#) — the config-over-fork machinery.
- [Extending openIMIS](#) — a full new-module walkthrough.

- [Troubleshooting](#) — when the pitfalls above bite.
- Tooling: [black](#), [flake8](#), [isort](#), [ESLint](#), [Prettier](#).

Glossary

A single alphabetized reference for openIMIS's domain and platform vocabulary. Keep it open in a second tab while you read the rest of the handbook — most terms here get a full chapter elsewhere, and each entry links to it where useful.



How to use this page

Definitions are one to three sentences: enough to unblock you, not a substitute for the chapter. Where a term has a home, follow the link. Terms are grouped by first letter; use your browser's find (`Ctrl/Cmd-F`) for a fast jump.



Two vocabularies in one platform

openIMIS mixes a **domain** vocabulary (insuree, policy, claim, contribution — the language of health financing) with a **platform** vocabulary (module, manifest, service signal, `MutationLog` — the language of the architecture). This glossary interleaves both; the [Terminology Primer](#) teaches the essential subset narratively.

A

AppConfig

A module's Django `AppConfig` subclass, defined in its `apps.py` (e.g. `class CoreConfig(AppConfig)`). In openIMIS it does more than standard Django: it holds the module's `DEFAULT_CFG`, often a `_configure_permissions()` method, and reads runtime configuration from the database at startup. See [Backend Deep Dive](#).

Assembly repo

A deployable repository that contains almost no business logic — only a manifest plus wiring — and assembles many module repositories into one running application. openIMIS has two: `openimis-be_py` (backend) and `openimis-fe_js` (frontend). See [Architecture Overview](#).

async_mutate

The method a subclass of `OpenIMISMutation` implements to do a mutation's actual work. It runs the business logic (via a service), and the framework wraps it in the `MutationLog` audit + `clientMutationId` polling machinery. See [GraphQL](#).

Audit

The property, pervasive in openIMIS, that every write leaves a durable trace. Realized primarily through the `MutationLog` (every mutation) and temporal versioning (every business record's history). See [Security](#).

B

Base models

The reusable model classes core provides so every module gets consistent behavior: `HistoryModel`, `HistoryBusinessModel`, `VersionedModel`, `UUIDModel`. New entities inherit the right base for temporal versioning, UUIDs, `legacy_id`, and `json_ext`. See [The Core Module](#).

bind_service_signal

The core function a module calls to hook logic *before* or *after* another module's service method, without importing it. The consumer half of the service-signal extension seam (see `register_service_signal`). See [Best Practices](#).

C

Calculation rule

A pluggable pricing/valuation rule, registered via signals through `openimis-be-calculation_py` and implemented in `calcrule_*` modules. Used to value contributions, capitation, third-party payment, and claims. See [Calculation Rules](#).

Capitation

A provider-payment method in which a health facility is paid a fixed amount per enrolled person per period, rather than per service delivered. In openIMIS it is computed by a calculation rule. See [Calculation Rules](#).

Claim

A record that a health facility delivered services/items to an insuree under a policy, submitted for adjudication and payment. Valued via calculation rules and paid through the payment/invoice modules. Table lives in the `claim` module. See [Claim](#).

ClaimAdmin (Claim Administrator)

The actor at a health facility responsible for submitting and managing claims. A domain role distinct from a system permission. See [Claim](#).

clientMutationId

The identifier an `OpenIMISMutation` returns immediately instead of the created object. The client polls the `MutationLog` by this id to learn when the async work finished and whether it succeeded. See [GraphQL](#).

Config over fork

openIMIS's guiding customization principle: change *which* modules run (the manifest) or *how* a module behaves (its config), never fork core. See [Configuration](#) and the [Architecture Critique](#).

Contribution

A premium payment recorded against a policy — the money an insuree (or a payer on their behalf) pays for coverage. Distinct from a *payment*, which is money openIMIS pays *out* to providers. Lives in the `contribution` module. See [Contribution & Payment](#).

Contributions (frontend)

A **different** meaning from the domain term above: on the frontend, a module's config object exposes a `contributions` map of **named extension points** (e.g. `"insuree.MainMenu"`, `"core.AppBar"`) into which other modules inject components. The FE analogue of the backend service-signal seam. See [Frontend Architecture](#).

Core module

`openimis-be-core_py` — the shared framework every other module depends on. It provides base models, the custom `User`, Graphene helpers, `OpenIMISMutation`, service signals, the scheduler, and audit. See [The Core Module](#).

D

DEFAULT_CFG

The Python dict in a module's `apps.py` holding its default configuration (feature flags, business parameters, permission code assignments). Overlaid at startup by a per-module JSON stored in the DB (`ModuleConfiguration`), enabling reconfiguration without code changes. See [Configuration](#).

DHIS2

District Health Information Software 2 — the widely used open-source aggregate health-data / HMIS platform. openIMIS pushes aggregate data to it via `openimis-be-dhis2_etl_py`. A complementary system, not a competitor. See [Integrations](#).

Digital Public Good (DPG)

An open-source solution that adheres to the DGPA standard (open licensing, privacy, standards, no harm) and advances the SDGs. openIMIS is a recognized DPG, governed by the openIMIS Initiative. See the [home page](#).

Distribution repo

`openimis-dist_dkr` — the docker-compose-based repository that ties the backend, frontend, database, and gateway into a runnable deployment. See [Docker & Deployment](#).

Dual keys

openIMIS's coexistence of legacy integer primary keys and added UUIDs on many models (the integer preserved as `legacy_id`). A consequence of the MSSQL → PostgreSQL migration. See [Database](#) and the [Architecture Critique](#).

E

Enrolment Officer

The field actor who enrolls families/insurees into policies and collects contributions. Modeled via `Officer` and linked to the openIMIS `User`. A core domain role in community-based schemes. See [Insuree](#).

ExtendedConnection

A core Graphene helper that extends the standard Relay connection with `totalCount` and `edgeCount`, so paginated GraphQL lists report their sizes. See [GraphQL](#).

Extension point

Any named seam where a module can attach behavior without modifying the target: backend **service signals** and the frontend `contributions` map are the two primary kinds. See [Extending openIMIS](#).

F

Family

A grouping of insurees (typically a household) that holds policies in community-based insurance. The head of family and members are insurees. Lives in the `insuree` module. See [Insuree](#).

FHIR R4

Fast Healthcare Interoperability Resources, Release 4 — the HL7 standard for exchanging health data. `openimis-be-api_fhir_r4_py` exposes REST FHIR endpoints under `/api_fhir_r4/`, mapping domain models to resources (Insuree → Patient, Policy → Coverage, Claim → Claim). See [FHIR R4 API](#).

filter_validity

A core helper that appends the temporal validity filter (open validity window) to a queryset so it returns *current* versions only. Forgetting it is the number-one temporal bug. See [Database](#).

G

Gateway

The Nginx reverse proxy in the reference deployment that fronts the frontend, `/api`, and `/graphql`, presenting one origin to the browser. See [Docker & Deployment](#).

Graphene / graphene-django

The Python library openIMIS uses to build its GraphQL schema. Each module defines Graphene `Query / Mutation` classes that the assembly stitches together. See [GraphQL](#).

GraphQL

The query language and runtime the frontend uses to talk to the backend. A single `/graphql` endpoint serves typed queries and mutations. Taught from first principles in [GraphQL](#).

H

HistoryModel

A core base model giving records full change history via temporal `validity_from` / `validity_to` fields (plus `legacy_id`, `json_ext`). Updating a record creates a new version rather than overwriting. See [The Core Module](#).

HistoryBusinessModel

A core base model extending the history/temporal behavior with business-object conveniences shared across domain entities. See [The Core Module](#).

I

Individual

The newer, generic person entity from `openimis-be-individual_py`, used by social protection and beneficiary-registry use cases — a cleaner-schema counterpart to the legacy `Insuree`. See [Individual & Social Protection](#).

Insuree

The insured person in the health-financing domain — a member of a family covered (or to be covered) by a policy. The legacy core beneficiary entity, in `openimis-be-insuree_py`. Contrast with the newer `Individual`. See [Insuree](#).

InsureePolicy

The link record associating an insuree with a policy, capturing that person's coverage under it (dates, status). See [Policy](#).

InteractiveUser

The part of the openIMIS `User` representing a human who logs into the UI (carries roles and integer rights). Contrast with `TechnicalUser`. See [Security](#).

INSTALLED_APPS (dynamic)

openIMIS builds Django's `INSTALLED_APPS` at startup from the loaded module list (the manifest) rather than hardcoding it — the mechanism that makes the platform modular. Built in `openimis/settings.py`. See [Plugin / Module System](#).

J

json_ext

A `JSONField` present on openIMIS base models for storing extra, deployment- or country-specific data without a schema migration. Prefer it over a new column for optional/rarely-queried fields. See [Database](#).

Journalize

The frontend helper that polls a mutation's `MutationLog` status after a write and updates the Redux store when it resolves — the FE side of the async mutation pattern. See [Frontend Architecture](#).

JWT (JSON Web Token)

The bearer token openIMIS uses for authentication, via `django-graphql-jwt`, stored in an **HttpOnly cookie** to reduce XSS token theft. OIDC/OAuth2 is also supported for external identity providers. See [Security](#).

L

legacy_id

A field on versioned models holding the original legacy integer primary key, preserving the link to pre-migration data and reports alongside the newer UUID. See [Database](#).

Location hierarchy

The tree of administrative and facility locations (e.g. region → district → municipality → village, plus health facilities), foundational reference data from `openimis-be-location.py`. Most domain records hang off a location. See [Location & Health Facility](#).

M

Manifest (module manifest)

The `openimis.json` file in each assembly repo listing the modules to load (with pip/git sources on the backend). The load-bearing artifact behind "config over fork." See [Plugin / Module System](#).

Medical (items & services)

Reference data for the medical items (drugs, consumables) and services a scheme covers, from `openimis-be-medical.py` — the building blocks of price lists and claims. See [Medical & Product](#).

Module

An independently versioned unit of functionality living in its own repository (`openimis-be-
<name>_py` / `openimis-fe-<name>_js`), loaded via the manifest. The central design idea of openIMIS. See [Plugin / Module System](#).

ModuleConfiguration

The core model that stores each module's runtime configuration JSON in the database, overlaying its `DEFAULT_CFG` at startup so operators can reconfigure a deployment without code changes. See [Configuration](#).

ModulesManager

The frontend cross-module registry (from FE core) used to look up published refs, config, and contributions — e.g. `modulesManager.getRef("insuree.InsureePicker")` , `.getContribs("insuree.MainMenu")` . The FE analogue of the backend plugin seam. See [Frontend Architecture](#).

MutationLog

The core model recording every mutation: its `clientMutationId` , status, and any error. It is both the audit trail and the object the client polls for async completion. See [GraphQL](#).

O

Officer

The domain model for field staff (e.g. enrolment officers), linked to the openIMIS `User` . See [Insuree](#).

OpenIMISMutation

The core base class implementing the **asynchronous mutation pattern**: create a `MutationLog` , do the work in a service, return a `clientMutationId` for the client to poll. Every mutation subclasses it and is audited. See [GraphQL](#).

OpenSearch

The search/analytics engine openIMIS uses to offload heavy reporting from the transactional database, driven by `opensearch_reports` . See [Integrations](#).

OrderedDjangoFilterConnectionField

A core Graphene helper providing filterable, orderable Relay connections for GraphQL list queries. See [GraphQL](#).

P

Payer

The entity that pays contributions on behalf of insurees (e.g. an employer, a government subsidy, a donor). See [Contribution & Payment](#).

Payment

Money openIMIS pays *out* — chiefly provider payment for adjudicated claims, handled by the payment/invoice modules. Contrast with *contribution* (money paid *in*). See [Contribution & Payment](#).

Policy

A coverage contract linking an insuree/family to a product for a validity period — the record that says "this person is covered under these benefits now." Lives in `openimis-be-policy_py`. See [Policy](#).

PrefixFilterset

A core helper for building Graphene/Django filtersets with a shared field prefix, reducing filter boilerplate across modules. See [GraphQL](#).

Price List

The set of prices a product/scheme applies to covered medical items and services — the reference used when valuing claims. See [Medical & Product](#).

Product

An insurance product: the benefit package, rules, price lists, and parameters an insuree is covered under via a policy. From `openimis-be-product_py`. See [Medical & Product](#).

R

register_service_signal

The core decorator/function that marks a service method as emitting before/after signals (named like `"module.service.method"`), so other modules can bind to it without importing it. The producer half of the service-signal seam. See [Best Practices](#).

Rights / Permissions

openIMIS authorizes actions by **integer rights codes** (e.g. `111001`) carried by a user's roles and checked with `user.has_perms([...])`. Codes are defined per module in `apps.py`. See [Security](#).

S

Schema assembly

The process in `openimis/schema.py` of importing each module's Graphene `Query / Mutation` and combining them into one root `Query / Mutation` via Python multiple inheritance. See [Plugin / Module System](#).

Scheduler (APScheduler)

The background-task scheduler core uses for periodic jobs (data loads, ETL, maintenance). See [The Core Module](#).

Service (service layer)

A module's `services.py` class holding business logic, callable from GraphQL, REST/FHIR, jobs, and tests alike — keeping resolvers/mutations thin. See [Best Practices](#).

Service signal

The core extension seam letting a module run logic before/after another module's service call *without importing it* (`register_service_signal` + `bind_service_signal`). Distinct from plain Django ORM signals, which openIMIS also uses. See [The Core Module](#).

Social Protection

The broader domain — beyond health insurance — of benefit and cash-transfer programs, supported by `openimis-be-social_protection_py` on top of the generic `Individual` model. See [Individual & Social Protection](#).

T

TechnicalUser

The part of the openIMIS `User` representing a service account (machine-to-machine access, integrations), as opposed to `InteractiveUser` (humans). See [Security](#).

Temporal versioning

The design in which business records carry `validity_from` / `validity_to` windows and updates create new versions, so you can query the state "as of" any date — essential for adjudicating claims against past coverage. See [Database](#).

U

User (openIMIS User)

Core's custom user model, which *composes* `InteractiveUser` (humans), `TechnicalUser` (service accounts), and `Officer / i_user`. Roles carry integer rights. Not the vanilla Django user. See [Security](#).

UUIDModel

A core base model providing a UUID primary key, part of the identity system added during the migration away from purely integer keys. See [Database](#).

V

ValidityFrom / ValidityTo (`validity_from` / `validity_to`)

The temporal fields on versioned models bounding when a version is the current truth. An open (null) `validity_to` marks the live version. See [Database](#).

VersionedModel

The core base model implementing temporal versioning (`validity_from` / `validity_to` , `legacy_id`) — the foundation for auditable, as-of-queryable business data. See [The Core Module](#).

W

Workflow / tasks_management

Modules (`openimis-be-workflow_py` , `openimis-be-tasks_management_py`) supporting configurable multi-step business processes and human task queues (e.g. claim review workflows). See [Developer Workflow](#).

Further reading

- [Terminology Primer](#) — the essential subset, taught narratively for newcomers.

- [Repository Appendix](#) — every repository, package, and responsibility in one master table.
- [Troubleshooting](#) — when a term above turns into a real bug.
- Official openIMIS wiki: openimis.atlassian.net/wiki.

Troubleshooting

A field guide to the problems openIMIS developers actually hit, organized by area. Each area opens with a **symptom** → **likely cause** → **fix** table for fast triage, followed by detailed callouts for the trickiest cases. Everything here is practical; when a fix depends on your version or deployment, we say so and point you at the authoritative file.



Triage method

Work outside-in: is it **setup/Docker** (the stack won't come up), **data/ migrations** (the DB is wrong), **API/auth** (requests are rejected), **module/ config** (behavior is wrong), or **frontend** (the UI is broken)? Jump to that section's table first, then the callouts.



The three openIMIS-specific reflexes

Before deep debugging, check the three things newcomers get wrong most: (1) a **missing validity filter** on a versioned query, (2) expecting a **mutation to return the object** instead of a `clientMutationId`, and (3) a change made in the **assembly repo** that belongs in a **module** repo. See [Best Practices](#).

Setup / install

Symptom	Likely cause	Fix
<code>ModuleNotFoundError</code> or for a module package	Module in <code>openimis.json</code> but not installed	Regenerate <code>modules-requirements.txt</code> and <code>pip install -r</code> it, or <code>pip install -e ../openimis-be-<name>_py/</code>
Edits to a module don't take effect	Module installed from git, not editable	Reinstall as editable: <code>pip install -e ../openimis-be-<name>_py/</code>
<code>INSTALLED_APPS</code> missing your app	Module absent/typo'd in the manifest	Check the module entry in <code>openimis.json</code> ; <code>settings.py</code> builds apps from it
Version conflicts on <code>pip install</code>	Pinned module refs disagree on a dep	Align the pins in <code>openimis.json</code> ; recreate the virtualenv clean
Import errors only for one module	Module's own deps not installed	Install that module's requirements; check its <code>setup.py</code> / <code>pyproject</code>



Common mistake: editing a module that was installed from git

If a module was installed from its pinned git ref (the default from `modules-requirements.txt`), your local edits to a sibling checkout have **no effect** — Python is importing the installed copy, not your working tree. Reinstall it editable (`pip install -e ../openimis-be-<name>_py/`) so the assembly imports your checkout directly. This is the single most common "why isn't my change doing anything?" during setup. See [Set Up a Dev Environment](#).

Docker

Symptom	Likely cause	Fix
<code>backend</code> exits immediately at boot	Migrations/config load failed	Read <code>backend</code> logs; the start script runs migrations + module config loads first
<code>backend</code> can't reach <code>db</code>	Started before Postgres was ready	Ensure startup order <code>db</code> → <code>backend</code> ; add a healthcheck/wait-for-db
Frontend loads but <code>/graphql</code> 502s	<code>backend</code> down or gateway misrouted	Check backend health; verify the gateway proxies <code>/graphql</code> and <code>/api</code>
Env/secrets not applied	<code>.env</code> not passed to compose	Confirm <code>.env</code> path and variable names match compose expectations
OpenSearch service unhealthy	Memory limits / <code>vm.max_map_count</code>	Raise Docker memory; set <code>vm.max_map_count</code> if using <code>opensearch_reports</code>
Ports already in use	Host port collision	Change host port mappings in the compose file

⚠ Startup order is load-bearing

The reference `openimis-dist_dkr` stack must come up **db** → **backend** → **frontend/gateway**. The `backend` container runs migrations and loads module configurations at start; if it races Postgres it dies. If your `backend` restarts in a loop, it is almost always (a) the database not ready, or (b) a migration/config failure — read the *first* error in its logs, not the last. See [Docker & Deployment](#).

🔗 Deep dive: reading a crash-looping backend

`docker compose logs backend` shows the story in order. Look for: a Postgres connection refusal (ordering/healthcheck issue) near the top; a Django migration traceback (schema/data issue — see the migrations section); or a module config load error (a bad `ModuleConfiguration` JSON or a module that fails to import). Fix the *earliest* failure; later ones are usually cascades. The container's entrypoint lives under `script/` in `openimis-be_py`.

Database / migrations

Symptom	Likely cause	Fix
<code>relation "tbl..."</code> <code>does not exist</code>	Migrations not applied / wrong DB	Run migrations; confirm you're pointed at the right database
Query returns duplicate/old rows	Missing validity filter	Add <code>filter_validity()</code> ; you're seeing historical versions
Update seems to lose the old value	Working as designed (new version)	Query current version with validity filter; history is intentional
Join returns wrong/empty rows	UUID vs. legacy integer key confusion	Confirm which key the join/API expects; check <code>legacy_id</code>
<code>column "camelCase"</code> <code>does not exist</code>	Legacy <code>db_column</code> mismatch in raw SQL	Use the ORM, or match the exact legacy column name/case
Migration conflict on merge	Two branches added migrations	Resolve/renumber migrations in the owning module; re-run

Common mistake: the silent missing validity filter

A versioned model queried without a validity filter returns **every historical version** of every record. It looks fine in a fresh dev DB with little history, then returns duplicates and wrong counts in production. Make `filter_validity()` a reflex on every query against a `VersionedModel` / `HistoryModel`. This is both the top correctness bug and a top performance bug. See [Database](#).

Why the schema looks inconsistent

Old tables carry `tbl` prefixes and camelCase columns inherited from the legacy MSSQL IMIS system; newer modules (e.g. `individual`) use clean names. If a raw query fails on a column name, you are almost certainly fighting the legacy casing — prefer the ORM, which maps it for you via `db_column`. See the [Architecture Critique](#) on legacy DB debt.

GraphQL / permissions errors

Symptom	Likely cause	Fix
<code>PermissionDenied</code> with a valid login	Role lacks the exact integer right	Grant the specific code; resolve it in the module's <code>apps.py</code>
Mutation "succeeds" but nothing changed	Read the wrong result — it's async	Poll the <code>MutationLog</code> by <code>clientMutationId</code> for real status/errors
Mutation returns no object	Working as designed (async pattern)	Query the created record afterward; use FE journalize
Unknown field / field from nowhere	Contributed by a module via inheritance	Trace the module list + MRO in <code>openimis/schema.py</code>
Empty list where data exists	Missing validity filter in resolver	Add the validity filter to the queryset
<code>totalCount</code> / <code>edgeCount</code> missing	Not using <code>ExtendedConnection</code>	Use core's <code>ExtendedConnection</code> for counts



Common mistake: trusting the mutation's immediate response

An `OpenIMISMutation` returns a `clientMutationId`, **not** the created/updated object, and its immediate response does **not** reflect whether the work succeeded. Errors land in the `MutationLog`. If a write "did nothing," poll the log by that id and read its status and error fields — do not conclude success from a 200 response. See [GraphQL](#).



Deep dive: debugging an integer permission failure



openIMIS authorizes with integer rights codes checked via `user.has_perms([<codes>])`, defined per module in `apps.py` (e.g. `gql_query_claims_perms = [111001]`). When a valid user is denied: (1) find the exact code the resolver/mutation requires in the owning module's `apps.py`; (2) confirm the user's role actually carries **that** integer, not a neighbor (`111002` vs `111001` is a classic off-by-one); (3) remember roles and their rights are DB-configured, so the running config may differ from source defaults. Never eyeball raw integer lists. See [Security](#).

Auth / JWT cookie issues

Symptom	Likely cause	Fix
Logged in but every request 401/403	JWT cookie not sent	Ensure same origin via gateway; cookie is HttpOnly, check domain/path
Cookie set but dropped by browser	<code>Secure</code> / <code>SameSite</code> / domain mismatch	Serve over the gateway origin; align cookie flags with your scheme (http/https)
Works in dev, fails behind proxy	Gateway not forwarding cookies/headers	Configure the proxy to pass <code>Cookie</code> /auth headers to <code>/graphql</code>
Token expires mid-session	Expiry/refresh not handled	Rely on the FE refresh flow; check <code>django-graphql-jwt</code> expiry settings
External IdP login fails	OIDC/OAuth2 misconfiguration	Verify client id/secret, redirect URIs, and provider metadata

The cross-origin cookie trap

openIMIS stores its JWT in an **HttpOnly cookie**. Cookies are origin-bound, so if the browser talks to the frontend on one origin and `/graphql` on another, the cookie may not be attached and every request looks unauthenticated. The reference deployment avoids this by putting frontend, `/api`, and `/graphql` behind **one gateway origin**. If you split them, you must get `SameSite`, `Secure`, `Domain`, and `Path` exactly right. When "login works but nothing is authorized," suspect the cookie before the credentials. See [Security](#).

Module loading / config

Symptom	Likely cause	Fix
Module's feature absent at runtime	Not in the manifest / not loaded	Add it to <code>openimis.json</code> ; confirm it's in <code>INSTALLED_APPS</code> at boot
Config change ignored	Editing <code>DEFAULT_CFG</code> at runtime	Change the DB <code>ModuleConfiguration</code> ; <code>DEFAULT_CFG</code> is only the default
Config change needs a restart	Config read at startup	Restart the backend so it reloads module configs
Permission code has no effect	Not wired into a role/config	Assign the code to a role; check <code>_configure_permissions()</code>
Another module's hook not firing	Bound to the wrong signal name	Match <code>register_service_signal</code> name exactly in <code>bind_service_signal</code>

DEFAULT_CFG is the default, not the truth

A module's `apps.py` `DEFAULT_CFG` is only the fallback. The **effective** configuration is the DB `ModuleConfiguration` overlaid on it at startup. If a config change "does nothing," you probably edited the code default while the DB holds an override — change the override, then restart so it reloads. See [Configuration](#).

Deep dive: a service-signal hook that never runs

If binding to another module's service does nothing: (1) confirm the exact signal name string matches the producer's `register_service_signal(...)` — these are plain strings and a typo fails silently; (2) confirm your binding module is actually loaded (in the manifest) so its `bind_service_signal` call executes at startup; (3) confirm you bound the right phase (before vs. after); (4) verify the producing service method is the one actually invoked on your code path. See [Best Practices](#).

Frontend build / Vite

Symptom	Likely cause	Fix
Module not appearing in the FE	Missing from FE <code>openimis.json</code>	Add it; <code>openimis-config-vite.js</code> regenerates <code>src/modules.js</code>
<code>src/modules.js</code> stale	Config script not re-run	Re-run the Vite config generation step; restart the dev server
<code>getRef / getContribs</code> returns nothing	Component not published / wrong name	Register it in the module config; match the published name exactly
Old CRA/CRACO instructions fail	Repo migrated to Vite	Use the Vite scripts, not the legacy CRA build
GraphQL call has no cache/devtools	There is no Apollo here	Use the custom <code>graphql / graphqlWithVariables</code> Redux layer + <code>journalize</code>
Stale data or infinite refetch	Bad effect dependency array	Fix <code>react-hooks/exhaustive-deps</code> ; don't disable the rule

⚠ There is no Apollo — stop looking for it

The frontend does **not** use Apollo Client. GraphQL runs through a custom Redux layer in FE core: `graphql / graphqlWithVariables` action creators plus a **journalize** polling helper for the async mutation status. If you are looking for an Apollo cache, devtools, or `useQuery`, you will not find them — learn the custom layer instead. See [Frontend Architecture](#).

🔗 Deep dive: a component other modules can't find

The FE `ModulesManager` resolves components by published name (`modulesManager.getRef("insuree.InsureePicker")`) and injects UI at `contributions` points (`getContribs("insuree.MainMenu")`). If a lookup returns nothing: (1) the owning module must export it under that exact key in its config object (`refs / reference`, `contributions`); (2) that module must be in the FE manifest so its config is registered; (3) after adding it, regenerate `src/modules.js` via the Vite config script and restart. Name mismatches are the usual culprit. See [Frontend Architecture](#).

Performance

Symptom	Likely cause	Fix
List views slow, query count huge	N+1 on relations	<code>select_related</code> (FK) / <code>prefetch_related</code> (reverse/M2M)
Queries slow as data grows	No index on validity/FK columns	Index <code>validity_from/validity_to</code> and join keys on big tables
Reports hammer the transactional DB	Analytics on the live store	Offload to OpenSearch via <code>opensearch_reports</code>
Write throughput stalls	Async worker/scheduler saturation	Scale workers; batch valuation; watch the <code>MutationLog</code> backlog
Whole-object fetches are heavy	Selecting all columns	<code>.only()</code> / <code>.values()</code> for list projections

Find N+1 fast

Turn on Django's `django.db.backends` logger in development and watch the query count for one request. If you see one query per row for a related object, that's an N+1 — collapse it with `select_related` / `prefetch_related`. Do this before reaching for caching. See the performance section of [Best Practices](#).

When you're truly stuck



A repeatable escalation path

1. **Isolate the layer** using the triage method at the top.
2. **Read the earliest error**, not the last — especially in Docker logs and migrations.
3. **Check the three reflexes**: validity filter, async mutation polling, right repo/right key.
4. **Find the source of truth**: the owning module's `apps.py` (config/perms), `models.py` (keys/validity), `services.py` (logic), or `openimis/schema.py` (where a GraphQL field comes from).
5. **Confirm the manifest**: is the module even loaded? `openimis.json` + `INSTALLED_APPS` at boot.
6. **Ask the community** with the isolated symptom, the earliest error, and what you already ruled out.

Further reading

- [Best Practices](#) — how to avoid most of these before they happen.
- [Architecture Critique](#) — why several of these traps exist (legacy DB, dual keys, async mutations, no Apollo).
- [Glossary](#) — quick definitions for any term above.
- [Set Up a Dev Environment](#) — the clean-install path.
- [Security](#) and [Configuration](#) — the auth and config subsystems in depth.
- Official openIMIS wiki: openimis.atlassian.net/wiki.

Repository Appendix

This is the **exhaustive reference** to openIMIS's repositories: what each one is, what package it installs, what it is responsible for, what it depends on, and how mature it is. Where the [Repository Map](#) tells the *story* of how the repos fit together, this page is the *lookup table* you return to when you need a fact fast.

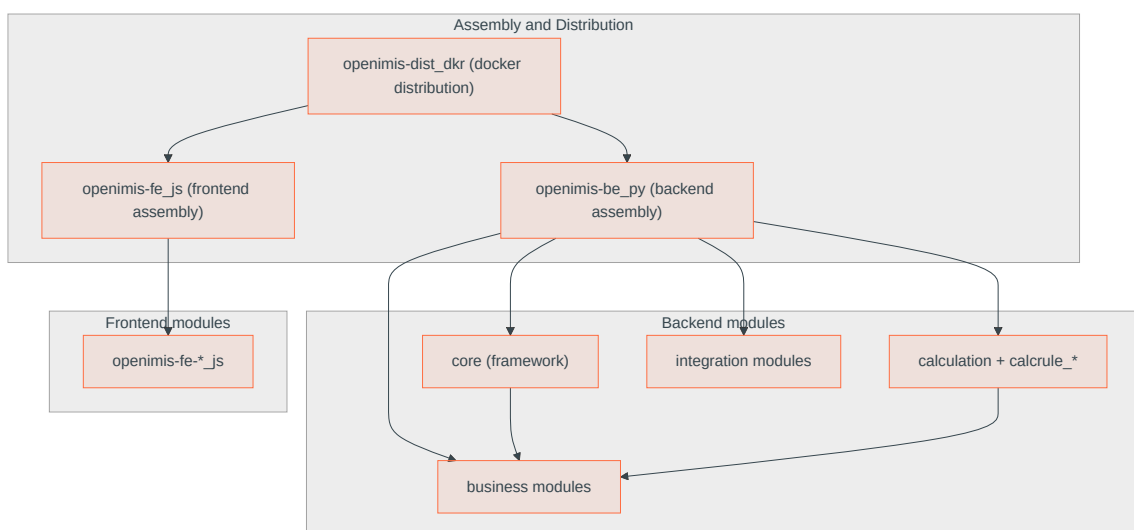
Two views of the same territory

Read the [Repository Map](#) first for the narrative — how modules depend on each other, how the assembly stitches them. Come **here** for the flat, categorized master list. This page is intentionally dense and skimmable.

Versions drift

openIMIS is fast-moving and modular: the exact set of modules (~47 on the backend), their names, and their maturity change between releases. Treat the tables below as an accurate map of the *categories and conventions*; confirm the current, exact set against each deployment's `openimis.json` manifest and github.com/openimis. "Maturity" is a rough guide, not an official rating.

How the repositories are organized



Naming conventions (memorize these; they are consistent):

Kind	Repository pattern	Installed name
Backend module	<code>openimis-be-<name>.py</code>	Python package <code><name></code>
Calculation rule	<code>openimis-be-calcrule-<name>.py</code>	Python package <code>calcrule-<name></code>
Frontend module	<code>openimis-fe-<name>.js</code>	JS module <code><name></code>
Backend assembly	<code>openimis-be.py</code>	the Django project
Frontend assembly	<code>openimis-fe.js</code>	the React app
Distribution	<code>openimis-dist_dkr</code>	docker-compose stack

Assembly & distribution repositories

Repository	Package / Artifact	Responsibility	Key dependencies	Maturity
<code>openimis-be_py</code>	Django project	Backend assembly: reads <code>openimis.json</code> , builds <code>INSTALLED_APPS</code> , assembles the GraphQL schema and URLs, runs migrations/config loads at start	All backend modules; Django, Graphene, graphql_jwt	Mature / core infra
<code>openimis-fe_js</code>	React SPA	Frontend assembly: reads FE <code>openimis.json</code> , generates <code>src/modules.js</code> via <code>openimis-config-vite.js</code> , hosts Redux store + routing	All FE modules; React, Redux, MUI, Vite	Mature / core infra
<code>openimis-dist_dkr</code>	docker-compose	Distribution: db + backend + frontend + gateway (+ optional OpenSearch); <code>.env</code> config; startup ordering	The two assemblies, PostgreSQL, Nginx	Mature

Neither assembly holds business logic

`openimis-be_py` and `openimis-fe_js` are *manifests plus wiring*. Almost all behavior lives in the module repos below. Understanding these two glue repos first is the highest-leverage early investment. See [Architecture Overview](#).

Backend — framework

Repository	Package	Responsibility	Key dependencies	Maturity
<code>openimis-be-core_py</code>	<code>core</code>	The shared framework: base models (<code>HistoryModel</code> , <code>VersionedModel</code> , <code>UUIDModel</code> , <code>json_ext</code>), custom User / Interactive User / TechnicalUser, Graphene helpers (<code>ExtendedConnection</code> , <code>OrderedDjangoFilterConnectionField</code> , <code>PrefixFilterset</code>), <code>OpenIMISMutation</code> , service signals, <code>ModuleConfiguration</code> , scheduler, audit	Django, Graphene, <code>graphql_jwt</code> , <code>APScheduler</code>	Mature / foundational

Everything else depends on `core`. See [The Core Module](#).

Backend — business & reference-data modules

Repository	Package	Responsibility	Key dependencies	Maturity
<code>openimis-be-location.py</code>	<code>location</code>	Location hierarchy (regions/districts/...) and health facilities — foundational reference data	core	Mature
<code>openimis-be-medical.py</code>	<code>medical</code>	Medical items and services catalog — building blocks of claims and price lists	core	Mature
<code>openimis-be-product.py</code>	<code>product</code>	Insurance products, benefit packages, price lists	core, medical, location	Mature
<code>openimis-be-insuree.py</code>	<code>insuree</code>	Insurees, families, officers — the (legacy) beneficiary registry	core, location	Mature
<code>openimis-be-individual.py</code>	<code>individual</code>	Generic person/individual entity (clean schema) for beneficiary registries	core	Growing
<code>openimis-be-social_protection.py</code>	<code>social_protection</code>	Social protection benefit programs on top of <code>individual</code>	core, individual	Growing
<code>openimis-be-policy.py</code>	<code>policy</code>	Policies linking insurees/families to products (coverage)	core, insuree, product	Mature

Repository	Package	Responsibility	Key dependencies	Maturity
openimis-be-contribution_py	contribution	Premiums recorded against policies (money in)	core, policy	Mature
openimis-be-claim_py	claim	Claims: service delivery by facilities against policies, adjudication	core, insuree, policy, medical, location	Mature
openimis-be-payment_py	payment	Payments (money out) and payment processing	core, contribution, policy	Mature
openimis-be-invoice_py	invoice	Invoicing / bill generation	core, payment	Growing
openimis-be-contract_py	contract	Contracts (e.g. group/formal-sector agreements)	core, policy, insuree	Growing
openimis-be-payroll_py	payroll	Payroll for benefit disbursement	core, payment, individual	Newer
openimis-be-tools_py	tools	Data import/export and administrative utilities	core, various domain modules	Mature

See the [Modules](#) section for chapter-level treatment of each.

Backend — calculation framework & rules

Repository	Package	Responsibility	Key dependencies	Maturity
<code>openimis-be-calculation.py</code>	<code>calculation</code>	The calculation-rule framework : registers/dispatches pluggable pricing/valuation rules via signals	core	Mature
<code>openimis-be-calcule*_py</code>	<code>calcrule_*</code>	Individual pluggable rules (e.g. contribution valuation, capitation, third-party payment) registered with the framework	core, calculation, relevant domain module	Varies by rule

How calcrule modules plug in

A `calcrule_*` module registers itself with the `calculation` framework via signals; the framework picks the applicable rule at runtime for contributions, capitation, claim valuation, and similar. This is the pricing analogue of the service-signal seam. See [Calculation Rules](#).

Backend — integration & reporting modules

Repository	Package	Responsibility	Key dependencies
<code>openimis-be-api_fhir_r4_py</code>	<code>api_fhir_r4</code>	REST FHIR R4 endpoints under <code>/api_fhir_r4/</code> , mapping domain models to FHIR resources (Insuree → Patient, Policy → Coverage, Claim → Claim, HealthFacility → Location/Organization)	core, insuree, policy, claim, location, medical
<code>openimis-be-dhis2_etl_py</code>	<code>dhis2_etl</code>	Pushes aggregate data to DHIS2	core, claim, policy, location
<code>openimis-be-report_py</code>	<code>report</code>	Reporting engine / report definitions	core, domain modules
<code>openimis-be-opensearch_h_reports_py</code>	<code>opensearch_h_reports</code>	Analytics/reporting offloaded to OpenSearch	core, OpenSearch
<code>openimis-be-tasks_management_py</code>	<code>tasks_management</code>	Human task queues (e.g. review/approval tasks)	core
<code>openimis-be-workflow_py</code>	<code>workflow</code>	Configurable multi-step business processes	core, tasks_management

See [Integrations](#) and [FHIR R4 API](#).

Frontend modules

Frontend modules mirror the backend domain and follow `openimis-fe-<name>.js`. Each exports a **config object** with keys such as `translations`, `reducers`, `refs / reference`, `queries`, `mutations`, `menus`, `routes`, and **contributions** (named extension points). The `ModulesManager` in FE core is the cross-module registry. See [Frontend Architecture](#).

Repository	JS module	Responsibility	Maturity
openimis-fe-core_js	core	FE framework: Redux store, ModulesManager , the custom graphql / graphqlWithVariables layer + journalize polling, MUI theme, i18n, route guards	Mature / foundational
openimis-fe-location_js	location	Location & health-facility UI and pickers	Mature
openimis-fe-medical_js	medical	Medical items/services UI	Mature
openimis-fe-product_js	product	Products & price-list UI	Mature
openimis-fe-insuree_js	insuree	Insuree/family registry UI, InsureePicker	Mature
openimis-fe-individual_js	individual	Individual registry UI	Growing
openimis-fe-social_protection_js	social_protection	Social protection UI	Growing
openimis-fe-policy_js	policy	Policy/coverage UI	Mature
openimis-fe-contribution_js	contribution	Contribution UI	Mature
openimis-fe-claim_js	claim	Claim capture and review UI	Mature
openimis-fe-payment_js	payment	Payment UI	Mature

Repository	JS module	Responsibility	Maturity
<code>openimis-fe-invoice_js</code>	<code>invoice</code>	Invoice UI	Growing
<code>openimis-fe-contract_js</code>	<code>contract</code>	Contract UI	Growing
<code>openimis-fe-tasks_management_js</code>	<code>tasks_management</code>	Task-queue UI	Growing

Backend/frontend module pairing

Most backend modules have a frontend counterpart of the same `<name>` (`openimis-be-claim_py` ↔ `openimis-fe-claim_js`). The two are versioned and released independently but share a domain vocabulary. Not every backend module has a UI (e.g. pure integration modules), and not every FE concern maps 1:1.

Where do I find X?

The fast lookup for "which file/repo owns this concept?"

I'm looking for...	It lives in...
The GraphQL schema assembly	<code>openimis-be_py</code> → <code>openimis/schema.py</code>
The backend module manifest	<code>openimis-be_py</code> → <code>openimis.json</code>
Dynamic <code>INSTALLED_APPS</code>	<code>openimis-be_py</code> → <code>openimis/settings.py</code>
URL wiring + <code>/graphql</code> endpoint	<code>openimis-be_py</code> → <code>openimis/urls.py</code>
Module config loading	<code>openimis-be_py</code> → <code>openimisconf/load_openimis_conf.py</code>
Container start scripts (migrate, load config)	<code>openimis-be_py</code> → <code>script/</code>
Base models (<code>HistoryModel</code> , <code>VersionedModel</code> , <code>json_ext</code>)	<code>openimis-be-core_py</code> → <code>core/models.py</code>
<code>OpenIMISMutation</code> (async mutation pattern)	<code>openimis-be-core_py</code> → <code>core/schema.py</code>
<code>DEFAULT_CFG</code> , integer permissions, <code>_configure_permissions()</code>	<code><module>/apps.py</code> (e.g. <code>core/apps.py</code>)
A module's business logic	<code><module>/services.py</code>
A module's GraphQL queries/mutations	<code><module>/schema.py</code> , <code>gql_queries.py</code> , <code>gql_mutations/</code>
Service-signal registration/binding	<code>openimis-be-core_py</code> → <code>core/signals.py</code> (+ per module)
<code>ModuleConfiguration</code> model	<code>openimis-be-core_py</code> → <code>core/models.py</code>
The calculation-rule framework	<code>openimis-be-calculation_py</code>
A specific pricing rule	the relevant <code>openimis-be-calcrule*_py</code>
FHIR resource mappings	<code>openimis-be-api_fhir_r4_py</code> → <code>api_fhir_r4/</code>

I'm looking for...	It lives in...
The frontend module manifest	<code>openimis-fe_js</code> → <code>openimis.json</code>
The generated FE module import file	<code>openimis-fe_js</code> → <code>src/modules.js</code> (from <code>openimis-config-vite.js</code>)
The FE <code>ModulesManager</code> + GraphQL layer	<code>openimis-fe-core_js</code> (FE <code>core</code>)
The docker-compose stack + <code>.env</code>	<code>openimis-dist_dkr</code>

The two files you'll open most

`openimis/schema.py` (to find where a GraphQL field comes from) and a module's `apps.py` (to find its config keys and integer permission codes) are the two most-consulted files during real debugging. Bookmark them. See [Troubleshooting](#).

Cross-references

- [Repository Map](#) — the narrative version of this appendix: how the repos depend on and assemble each other.
- [Architecture Overview](#) — the assembly model.
- [Plugin / Module System](#) — how the manifest becomes a running application.
- [Modules](#) — chapter-level coverage of each business module.
- [Glossary](#) — one-line definitions for any term here.
- Source of truth: github.com/openimis and each deployment's `openimis.json`.